

```
In [2]: import os
import sys

import matplotlib.colors as colors
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scanpy as sc
import gseapy as gp
import seaborn as sns
from celloracle import motif_analysis as ma
import celloracle as co
co.__version__
import os, sys, shutil, importlib, glob
from tqdm.notebook import tqdm
%matplotlib inline
```

Matplotlib is building the font cache; this may take a moment.

```
In [4]: plt
```

```
Out[4]: <module 'matplotlib.pyplot' from '/x/xxx/xxx/celloracle/lib/python3.10/site-packages/matplotlib/pyplot.py'>
```

```
In [2]: !module load bedtools
peaks = pd.read_csv("/xxx/analysis/PT_subset/cicero/caroline_multiome_PT1_2_inj_Clu
peaks = peaks.x.values

cicero_connections = pd.read_csv("/xxx/analysis/PT_subset/cicero/caroline_multiome_
cicero_connections.head()
ref_genome="mm10"
tss_annotated = ma.get_tss_info(peak_str_list=peaks, ref_genome="mm10")
tss_annotated.tail()

integrated = ma.integrate_tss_peak_with_cicero(tss_peak=tss_annotated,
                                              cicero_connections=cicero_connection
print(integrated.shape)
integrated.head()
```

```
que bed peaks: 169864
tss peaks in que: 19430
(424484, 3)
```

```
Out[2]:
```

	peak_id	gene_short_name	coaccess
0	chr10_100014172_100014455	Kitl	0.070975
1	chr10_100014172_100014455	Tmtc3	0.002443
2	chr10_100015333_100016700	Kitl	1.000000
3	chr10_100015333_100016700	Tmtc3	0.021577
4	chr10_100025599_100025875	Kitl	0.052610

```
In [3]: #Remove peaks with weak coaccessibility scores
peak = integrated[integrated.coaccess >= 0.8]
peak = peak[["peak_id", "gene_short_name"]].reset_index(drop=True)

peak.to_csv("/xxx/analysis/PT_subset/cicero/caroline_multiome_PT1_2_inj_Clusters_pr

import genomepy
genomepy.install_genome(name=ref_genome, provider="UCSC", genomes_dir=None)

peaks = ma.check_peak_format(peak, ref_genome, genomes_dir=None)

# Instantiate TFinfo object
tfi = ma.TFinfo(peak_data_frame=peaks,
                ref_genome=ref_genome,
                genomes_dir=None)

tfi.scan(fpr=0.02,
        motifs=None, # If you enter None, default motifs will be loaded.
        verbose=True)
tfi.to_hdf5(file_path="/xxx/analysis/PT_subset/cicero/caroline_multiome_PT1_2_inj_C
```

```
22:38:52 | INFO | Downloading assembly summaries from UCSC
Peaks before filtering: 18070
Peaks with invalid chr_name: 0
Peaks with invalid length: 0
Peaks after filtering: 18070
No motif data entered. Loading default motifs for your species ...
Default motif for vertebrate: gimme.vertebrate.v5.0.
For more information, please see https://gimmemotifs.readthedocs.io/en/master/overview.html
```

Initiating scanner...

```
2024-08-04 22:39:18,940 - DEBUG - using background: genome mm10 with size 200
Calculating FPR-based threshold. This step may take substantial time when you load a
new ref-genome. It will be done quicker on the second time.
```

Motif scan started .. It may take long time.

```
Scanning: 0%|          | 0/16458 [00:00<?, ? sequences/s]
```

```
In [4]: # Reset filtering
tfi.reset_filtering()

# Do filtering
tfi.filter_motifs_by_score(threshold=10)

# Format post-filtering results.
tfi.make_TFinfo_dataframe_and_dictionary(verbose=True)
df = tfi.to_dataframe()
df.head()

df = tfi.to_dataframe()
df.to_parquet("/xxx/analysis/PT_subset/cicero/caroline_multiome_PT1_2_inj_Clusters_
```

Filtering finished: 12531408 -> 2375998

1. Converting scanned results into one-hot encoded dataframe.

0%| | 0/16458 [00:00<?, ?it/s]

2. Converting results into dictionaries.

0%| | 0/16374 [00:00<?, ?it/s]

0%| | 0/1091 [00:00<?, ?it/s]

In [5]: `#https://morris-lab.github.io/CellOracle.documentation/notebooks/04_Network_analysi`

```
# visualization settings
plt.rcParams['figure.figsize'] = [6, 4.5]
plt.rcParams["savefig.dpi"] = 300

save_folder = "/xxx/analysis/PT_subset/scanpy/figures"
os.makedirs(save_folder, exist_ok=True)
sc.settings.figsir = save_folder

adata = sc.read_h5ad("/xxx/Chrm_457_multiome4_Dec2023_Pad4/Sobj_PT_Clusters_anndata

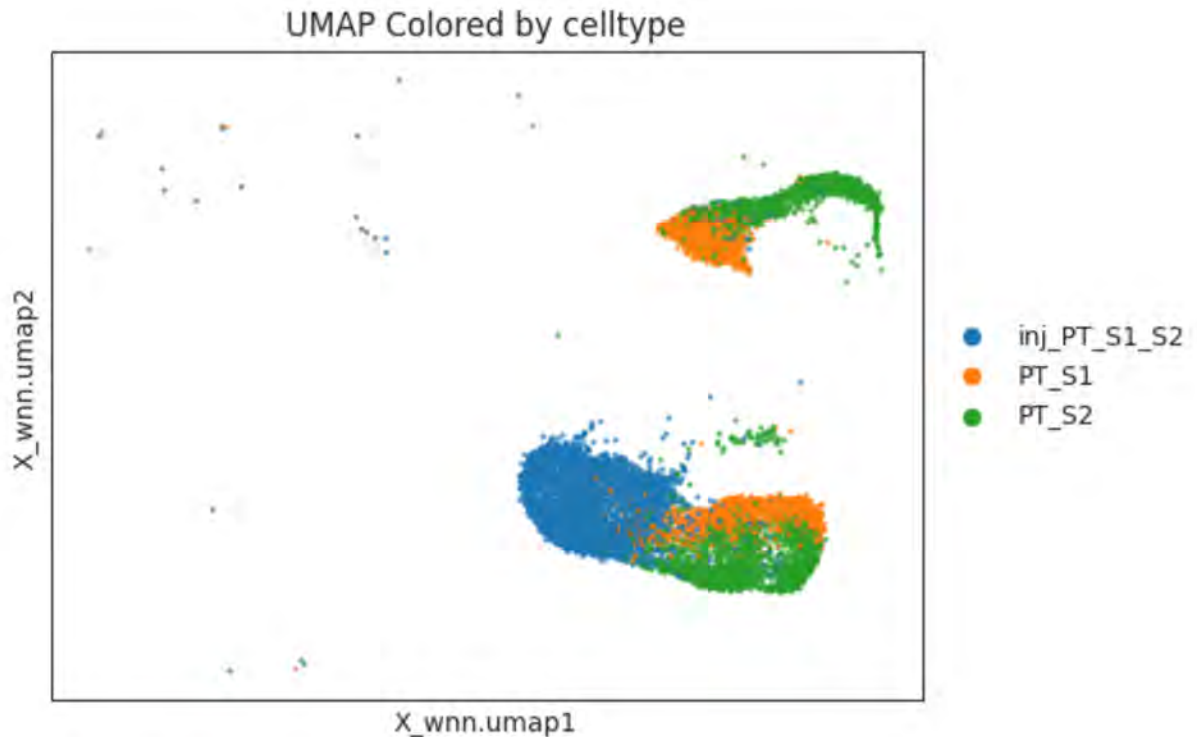
#####
#sc Preprocessing
#####

# Only consider genes with more than 1 count
sc.pp.filter_genes(adata, min_counts=1)
# Normalize gene expression matrix with total UMI count per cell
sc.pp.normalize_per_cell(adata, key_n_counts='n_counts_all')
# Select top 2000 highly-variable genes
#####WANT TO ADD GENES HERE OF INTEREST
###FIND MARKERS ON CL0-1 vs 4-5
#testing<-FindMarkers(obj=obj,ident.1 = colnames(obj[,obj$RNA_snn_res.0.9 %in% c(0,

filter_result = sc.pp.filter_genes_dispersion(adata.X,
                                                flavor='cell_ranger',
                                                n_top_genes=2000,
                                                log=False)

adata = adata[:, filter_result.gene_subset]
```

In [6]: `sc.pl.embedding(adata, basis='X_wnn.umap', color='celltype', size=20, alpha=0.7, ti
plt.savefig(f"{save_folder}/PT1_2_inj_UMAP.pdf")
plt.show()
#adata`



<Figure size 600x450 with 0 Axes>

```
In [7]: # Renormalize after filtering
sc.pp.normalize_per_cell(adata)
# keep raw count data before Log transformation
adata.raw = adata
adata.layers["raw_count"] = adata.raw.X.copy()

# Log transformation and scaling
sc.pp.log1p(adata)
sc.pp.scale(adata)
# PCA
sc.tl.pca(adata, svd_solver='arpack')

# Diffusion map
sc.pp.neighbors(adata, n_neighbors=4, n_pcs=20)

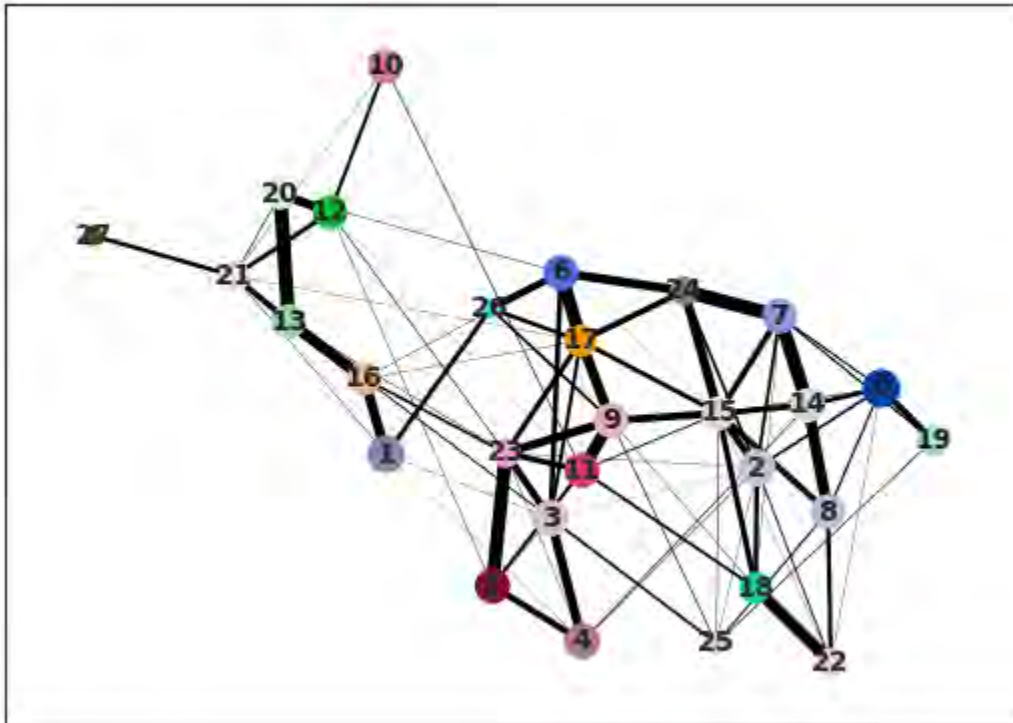
sc.tl.diffmap(adata)
# Calculate neighbors again based on diffusionmap
sc.pp.neighbors(adata, n_neighbors=10, use_rep='X_diffmap')
sc.tl.louvain(adata, resolution=0.8)
# PAGA graph construction

In [8]: #!pip install cython --upgrade
#!pip install fa2
sc.tl.paga(adata, groups='louvain')
plt.rcParams["figure.figsize"] = [6, 4.5]
sc.pl.paga(adata)
plt.savefig(f"{save_folder}/PT1_2_inj_PAGA.pdf")
plt.show()

sc.tl.draw_graph(adata, init_pos='paga', random_state=123)
```



```
plt.savefig(f"{save_folder}/PT1_2_inj_PAGA2.pdf")
plt.show()
sc.pl.draw_graph(adata, color='louvain', legend_loc='on data', save="LOUVAIN.pdf")
plt.savefig(f"{save_folder}/PT1_2_inj_louvain.pdf")
plt.show()
```

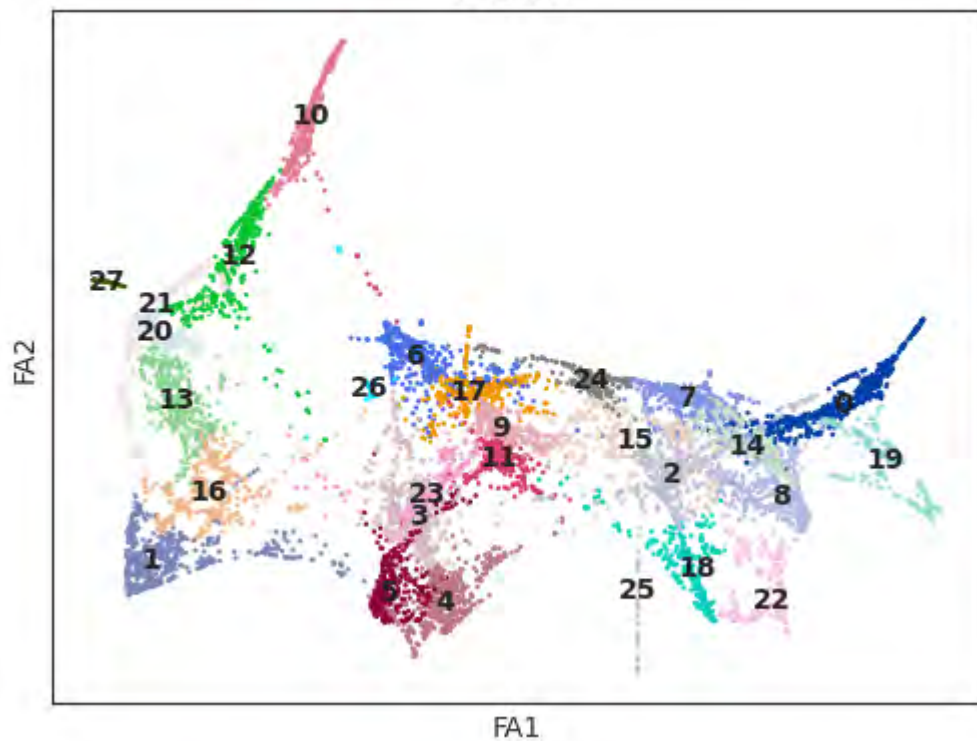


<Figure size 600x450 with 0 Axes>

<Figure size 600x450 with 0 Axes>

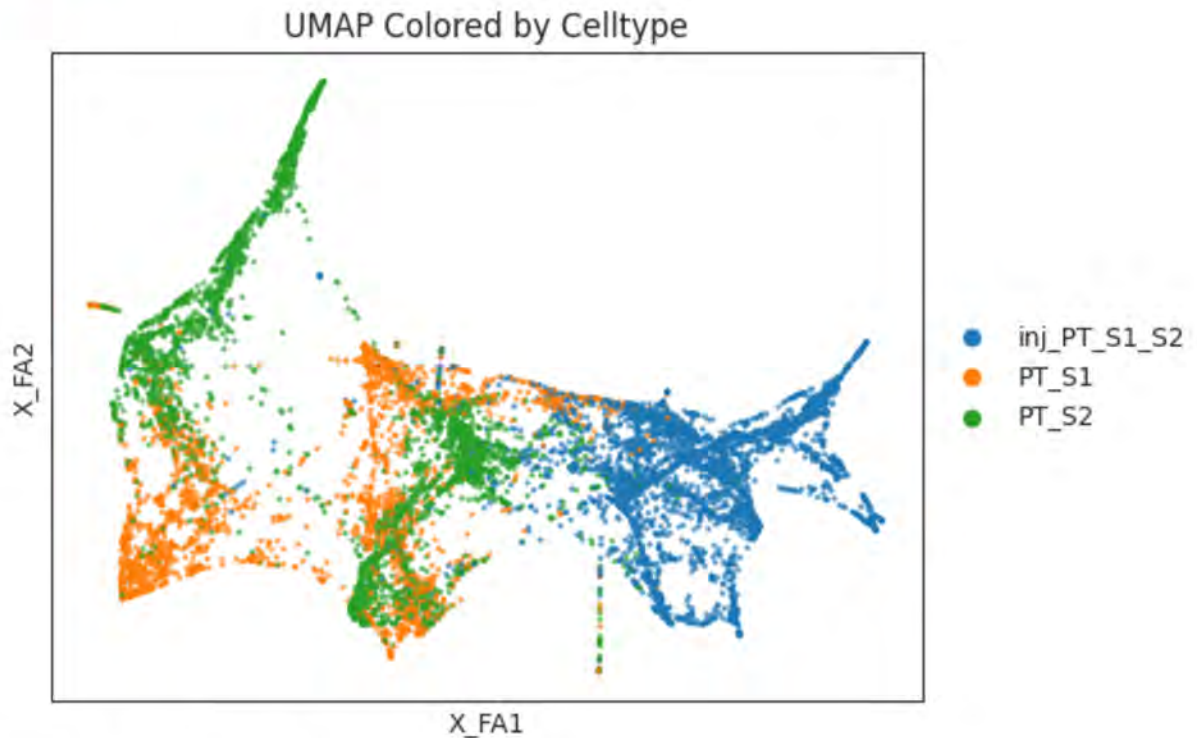
WARNING: saving figure to file figures/draw_graph_faLOUVAIN.pdf

louvain



<Figure size 600x450 with 0 Axes>

```
In [11]: sc.pl.embedding(adata, basis='X_draw_graph_fa', color='celltype', size=20, alpha=0.
plt.savefig(f"{save_folder}/PT1_2_inj_FA_celltype.pdf")
plt.show()
#adata
```



<Figure size 600x450 with 0 Axes>

```
In [13]: plt.rcParams["figure.figsize"] = [4.5, 4.5]
markers = {"S1":["Angpt1", "Slc5a12"], "S2":["Slc13a3", "Acaa1b"], "S1_2_inj":["Fgg"]
}

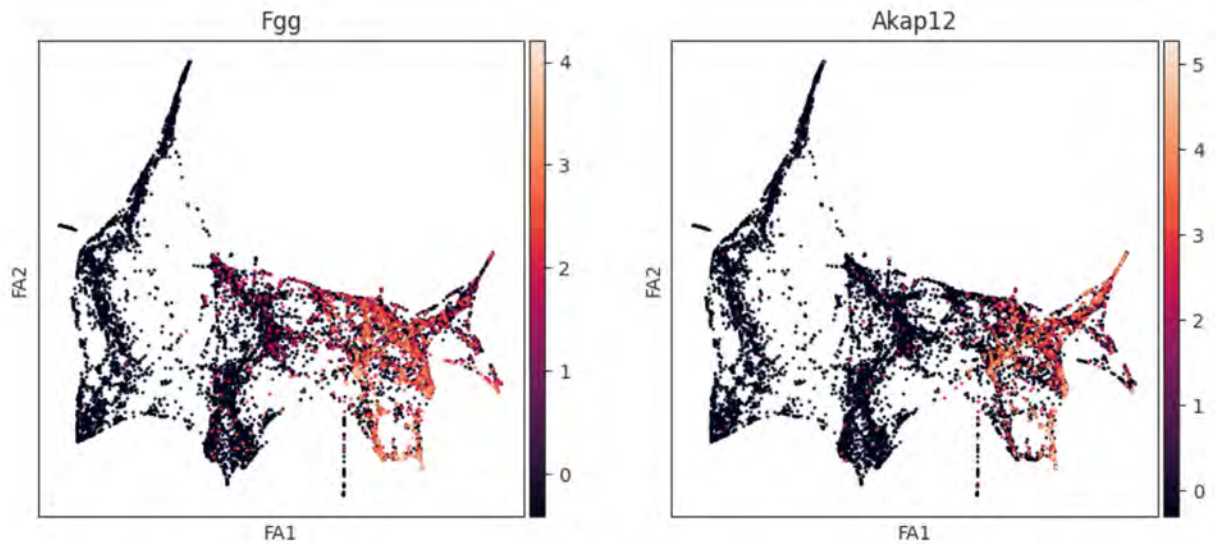
for cell_type, genes in markers.items():
    print(f"marker gene of {cell_type}")
sc.pl.draw_graph(adata, color=genes, use_raw=False, ncols=2)

plt.show()

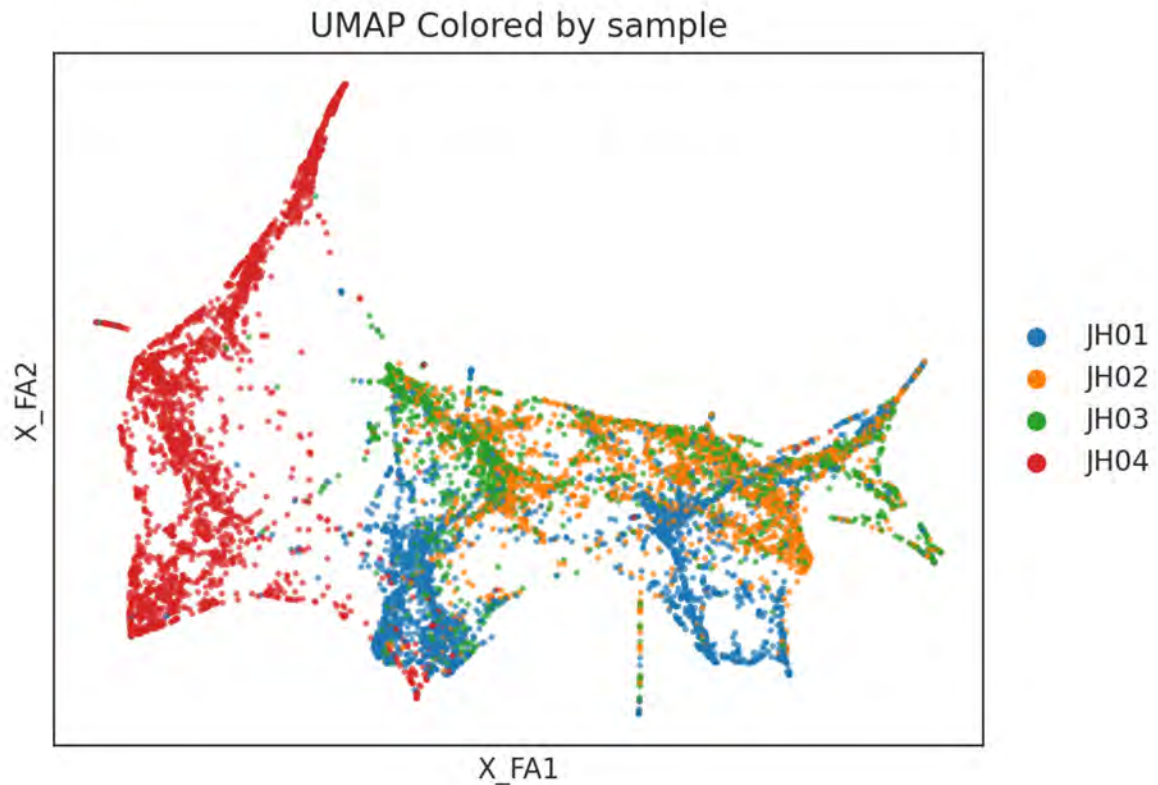
#https://github.com/morris-Lab/CellOracle/blob/master/docs/notebooks/03_scrna-seq_d

adata.write_h5ad("/xxx/analysis/PT_subset/Sobj_PT1_2_inj_anndata_process.h5ad")
```

marker gene of S1
 marker gene of S2
 marker gene of S1_2_inj



```
In [33]: sc.pl.embedding(adata, basis='X_draw_graph_fa', color='orig.ident', size=20, alpha=
plt.savefig(f"{save_folder}/PT1_2_inj_FA_sample.pdf")
plt.show()
```



<Figure size 600x450 with 0 Axes>

```
In [17]: adata = sc.read_h5ad("/xxx/analysis/PT_subset/Sobj_PT1_2_inj_anndata_process.h5ad")

#adata = sc.read_h5ad("/xxx/Chrm_457_multiome4_Dec2023_Pad4/Sobj_full_anndata_proce
#wnnUmaps = '/xxx/Chrm_457_multiome4_Dec2023_Pad4/wnnUMAP.csv'
#wnn_umap_df = pd.read_csv(wnnUmaps)
#wnn_umap_array = wnn_umap_df.to_numpy()
#adata.obsm['X_wnn.umap'] = wnn_umap_array
```



```

#df= pd.read_parquet('/xxx/analysis/cicero/caroline_multiome_full_base_GRN_datafram
#adata = sc.read_h5ad("/xxx/Chrm_457_multiome4_Dec2023_Pad4/Sobj_full_anndata_proce
#adata2 = sc.read_h5ad("/xxx/Chrm_457_multiome4_Dec2023_Pad4/Sobj_full_anndata.h5ad

print(f"Cell number is :{adata.shape[0]}")
print(f"Gene number is :{adata.shape[1]}")

adata.X = adata.layers["raw_count"].copy()
#adata.X = adata.layers["raw_count"].copy()

# Try to load variable gene list
try:
    variable_gene_list = adata.var.variable_gene.values
except:
    variable_gene_list = sc.pp.filter_genes_dispersion(adata.X,
                                                        flavor='seurat',
                                                        n_top_genes=1999,
                                                        log=True)

# Print the dtype of variable_gene_list to inspect its structure
print("dtype of variable_gene_list:", variable_gene_list.dtype)

# Extract the indices of the variable genes
variable_gene_indices = variable_gene_list['gene_subset']

# Extract the valid variable gene names using the indices
valid_variable_gene_list = adata.var_names[variable_gene_indices]

# Print the valid variable gene list
print("Valid variable_gene_list:", valid_variable_gene_list)

# Subset the AnnData object using the valid variable gene list
# Select genes
adata = adata[:, valid_variable_gene_list]

print("dimensional reduction: ", list(adata.obsm.keys()))
embedding = "X_wnn.umap"
clustering= "newIdents"
# Visualize
if embedding == "X_draw_graph_fa":
    sc.pl.draw_graph(adata, color=clustering)

elif embedding == "X_umap":
    sc.pl.umap(adata, color=clustering)

elif embedding == "X_tsne":
    sc.pl.tsne(adata, color=clustering)

sc.pl.embedding(adata, basis='X_draw_graph_fa', color='celltype', size=20, alpha=0.

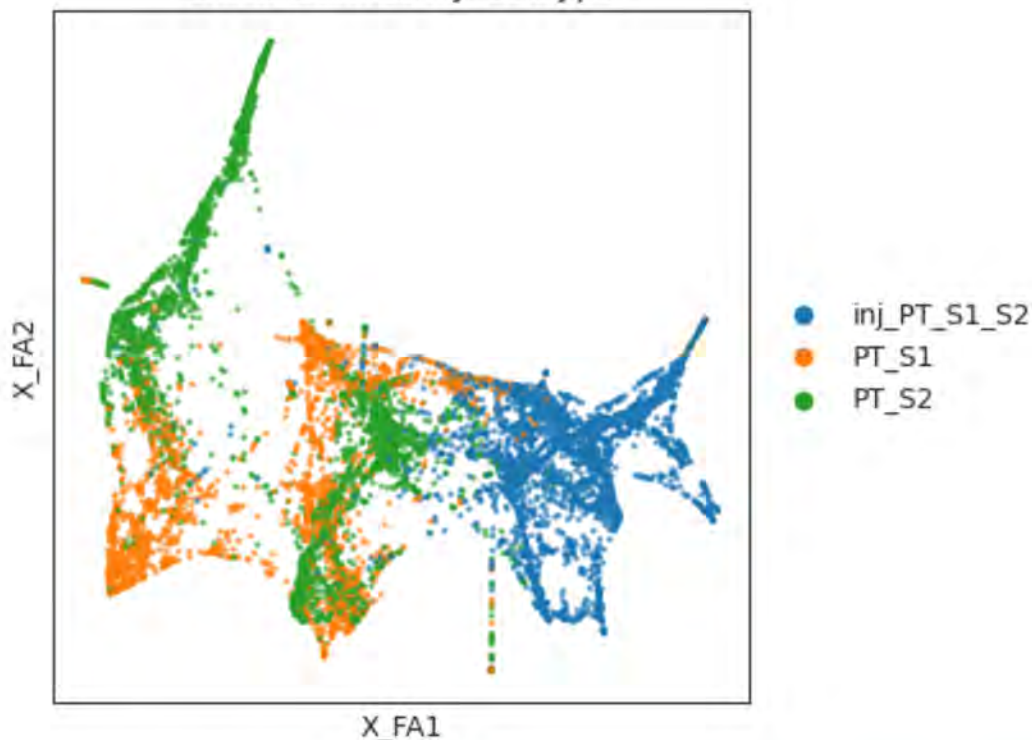
```

```

Cell number is :9706
Gene number is :1999
dtype of variable_gene_list: (numpy.record, [('gene_subset', '?'), ('means', '<f4'),
('dispersions', '<f4'), ('dispersions_norm', '<f4')])
Valid variable_gene_list: Index(['Atp6v1h', 'Prex2', 'Hs6st1', 'Cnnm4', 'Npas2', 'I1
1rl1', 'Stat1',
    'Ccnyl1', 'Unc80', 'Rpl37a',
    ...
    'Ch25h', 'Rbp4', 'Aldh18a1', 'Tm9sf3', 'Sfrp5', 'Pyroxd2', 'Hps1',
    'Slc25a28', 'Pdzd7', 'Sorcs1'],
    dtype='object', length=1998)
dimensional reduction: ['X_diffmap', 'X_draw_graph_fa', 'X_harmony.atac', 'X_harmon
y.peaks', 'X_harmony.rna', 'X_harmony.sct', 'X_lsi', 'X_pca', 'X_umap.atac', 'X_umap
.peaks', 'X_umap.rna', 'X_umap.sct', 'X_wnn.umap']

```

UMAP Colored by Celltype



```

In [19]: adata.write_h5ad("/xxx/analysis/PT_subset/Sobj_PT1_2_inj_anndata_process_2.h5ad")

In [20]: #Load base-GRN data
base_GRN= pd.read_parquet("/xxx/analysis/PT_subset/cicero/caroline_multiome_PT1_2_i

In [37]: #2. Make Oracle object
# Instantiate Oracle object
oracle = co.Oracle()

In [38]: oracle.import_anndata_as_raw_count(adata=adata,
                                             cluster_column_name="louvain",
                                             embedding_name='X_draw_graph_fa')

WARNING: adata.X seems to be already log-transformed.

In [39]: oracle.import_TF_data(TF_info_matrix=base_GRN)

```

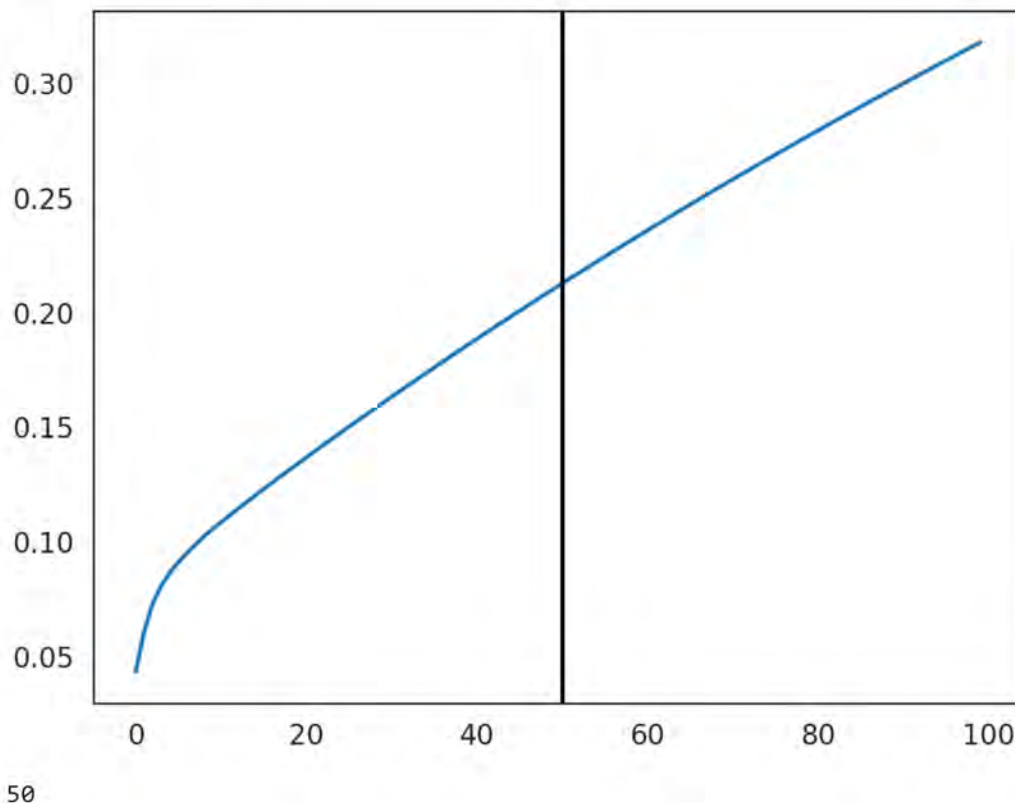
```
In [40]: oracle.to_hdf5("/xxx/analysis/PT_subset/Sobj_PT1_2_inj.celloracle.oracle")
```

```
In [41]: # visualization settings
%config InlineBackend.figure_format = 'retina'
%matplotlib inline
plt.rcParams['figure.figsize'] = [6, 4.5]
plt.rcParams["savefig.dpi"] = 300
```

```
In [42]: #3. KNN imputation

# Perform PCA
oracle.perform_PCA()

# Select important PCs
plt.plot(np.cumsum(oracle.pca.explained_variance_ratio_)[:100])
n_comps = np.where(np.diff(np.diff(np.cumsum(oracle.pca.explained_variance_ratio_))
# Make sure n_comps is between 30 and 50
# It likes to go to 0 for noise controls
n_comps = max(min(n_comps, 50), 30)
plt.axvline(n_comps, c="k")
plt.show()
print(n_comps)
```



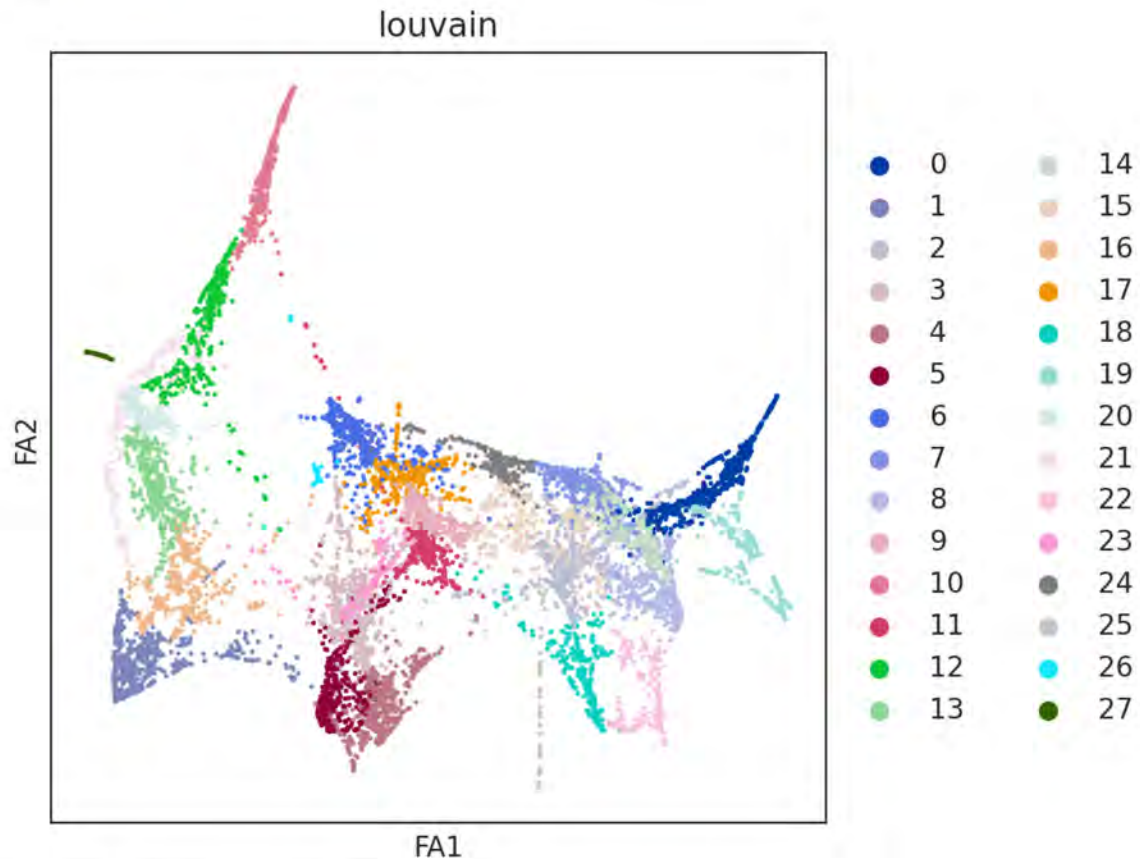
```
In [43]: n_cell = oracle.adata.shape[0]
print(f"cell number is :{n_cell}")
k = int(0.025*n_cell)
print(f"Auto-selected k is :{k}")
```

```
cell number is :9706
Auto-selected k is :242
```



```
In [44]: oracle.knn_imputation(n_pca_dims=n_comps, k=k, balanced=True, b_sight=k*8,
                                b_maxl=k*4, n_jobs=4)
oracle.to_hdf5("/xxx/analysis/PT_subset/Sobj_PT1_2_inj_step2.celloracle.oracle")
```

```
In [132... # GRN calculation
# Check clustering data
sc.pl.draw_graph(oracle.adata,color="louvain")
plt.savefig(f"{save_folder}/PT1_2_inj_FA_louvain.pdf")
plt.show()
```



<Figure size 500x500 with 0 Axes>

```
In [46]: #5.1. Get GRNs
# Calculate GRN for each population in "louvain_annot" clustering unit.
# This step may take some time.(~30 minutes)
links = oracle.get_links(cluster_name_for_GRN_unit="louvain", alpha=10,
                           verbose_level=10,n_jobs=20)

# Save Links object.
links.to_hdf5(file_path="/xxx/analysis/PT_subset/Sobj_PT1_2_inj_links.celloracle.li

0%|          | 0/28 [00:00<?, ?it/s]
Inferring GRN for 0...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 1...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 10...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 11...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 12...
```

```

0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 13...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 14...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 15...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 16...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 17...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 18...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 19...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 2...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 20...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 21...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 22...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 23...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 24...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 25...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 26...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 27...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 3...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 4...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 5...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 6...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 7...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 8...
0%|          | 0/1643 [00:00<?, ?it/s]
Inferring GRN for 9...
0%|          | 0/1643 [00:00<?, ?it/s]

```

```

In [47]: # Set cluster name
         # Save as csv
         for cluster in links.cluster:
             links.links_dict[cluster].to_csv(f"{save_folder}/raw_GRN_for_{cluster}.csv")

```

```

In [48]: # 6. Network preprocessing
         # 6.1. Filter network edges

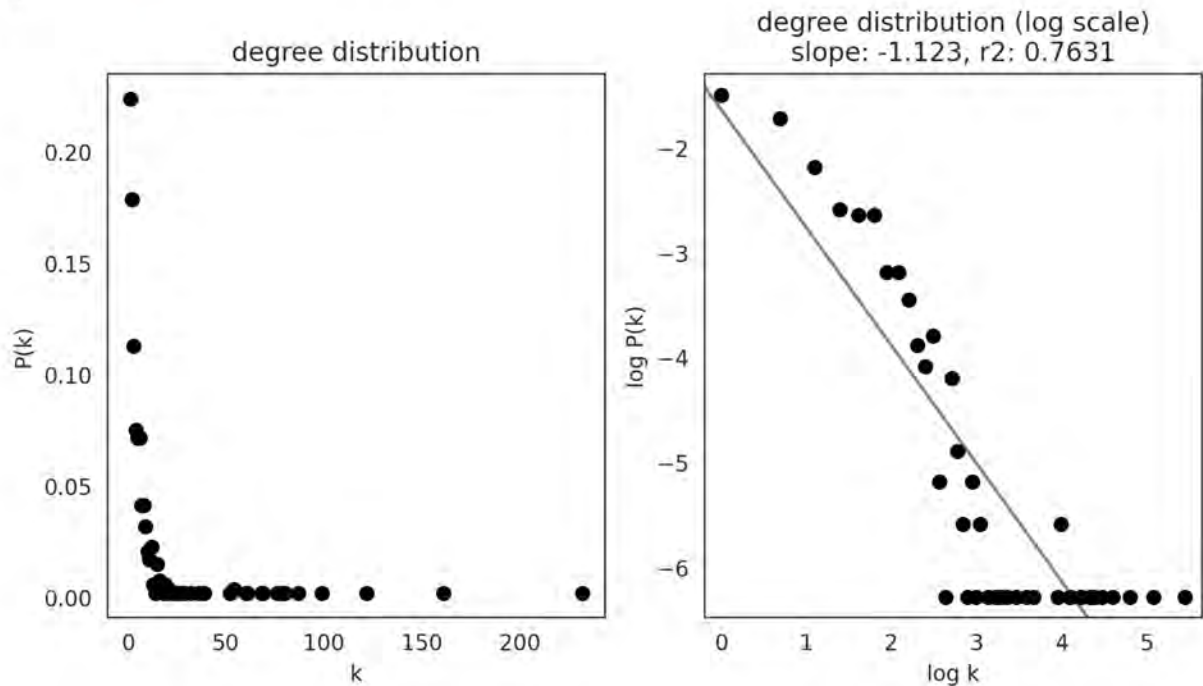
```

```
links.filter_links(p=0.001, weight="coef_abs", threshold_number=2000)
```

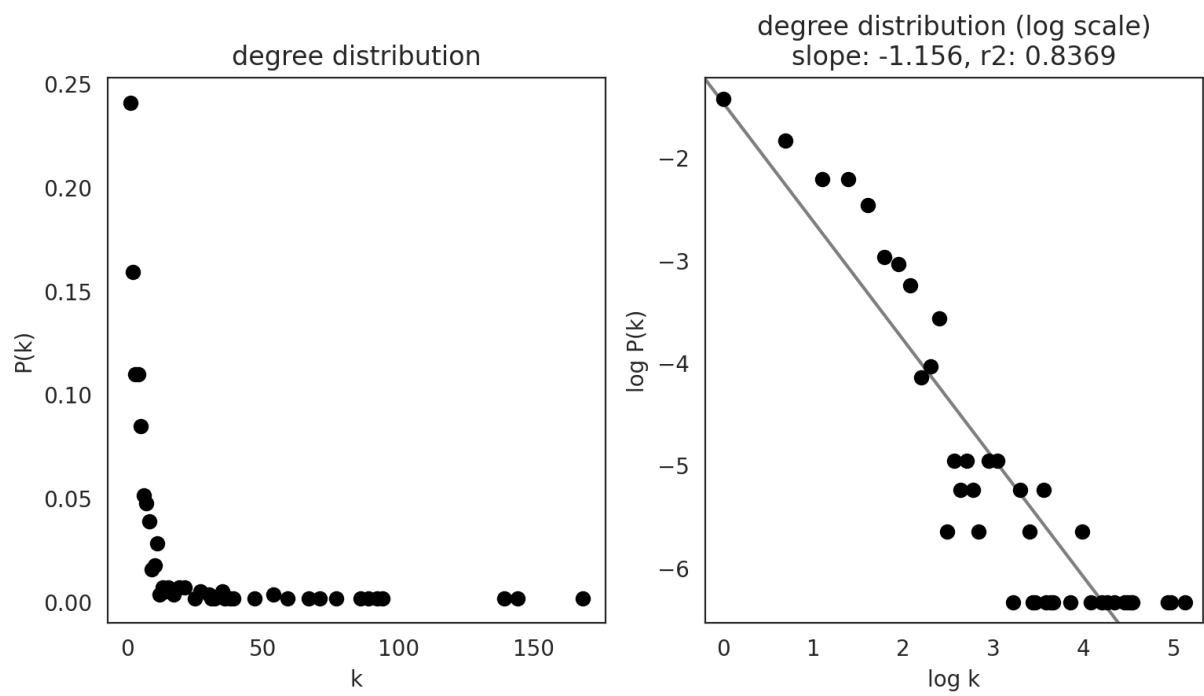
```
In [49]: #6.2. Degree distribution
plt.rcParams["figure.figsize"] = [9, 4.5]

links.plot_degree_distributions(plot_model=True, save=f"degree_distribution/")
# Save each plot separately
for i in plt.get_fignums():
    fig = plt.figure(i)
    fig.savefig(os.path.join(save_folder, f"degree_distribution_{i}.png"), bbox_inches=
    # Optionally show the plots
plt.show()
```

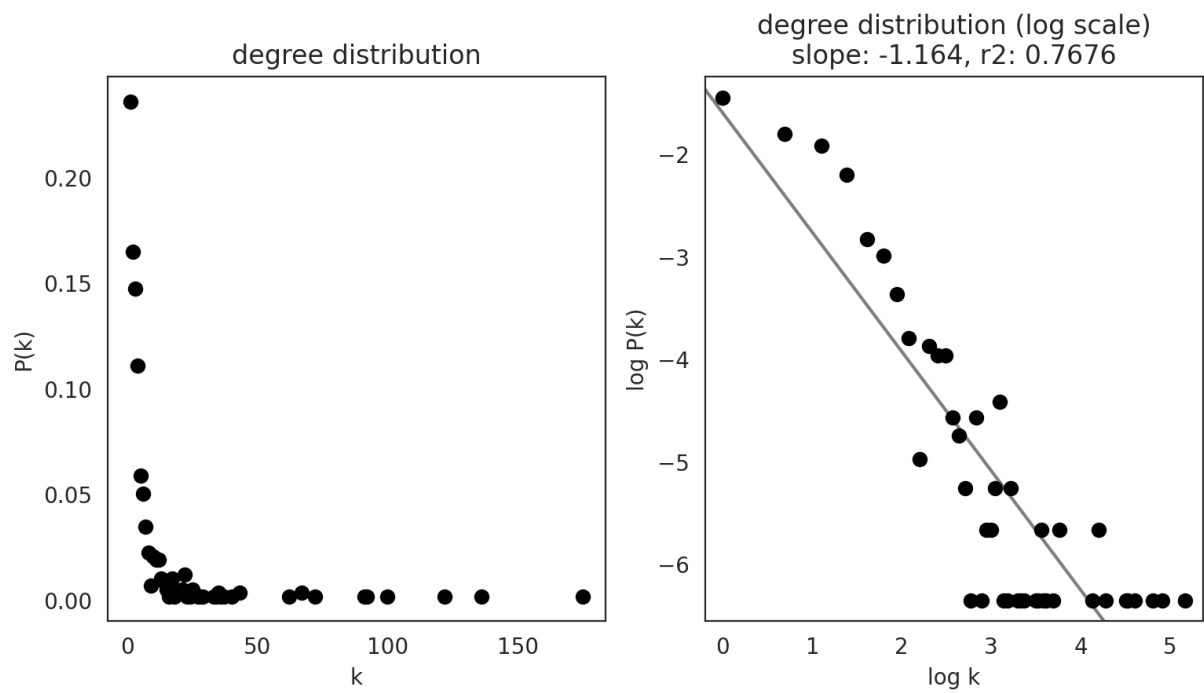
0



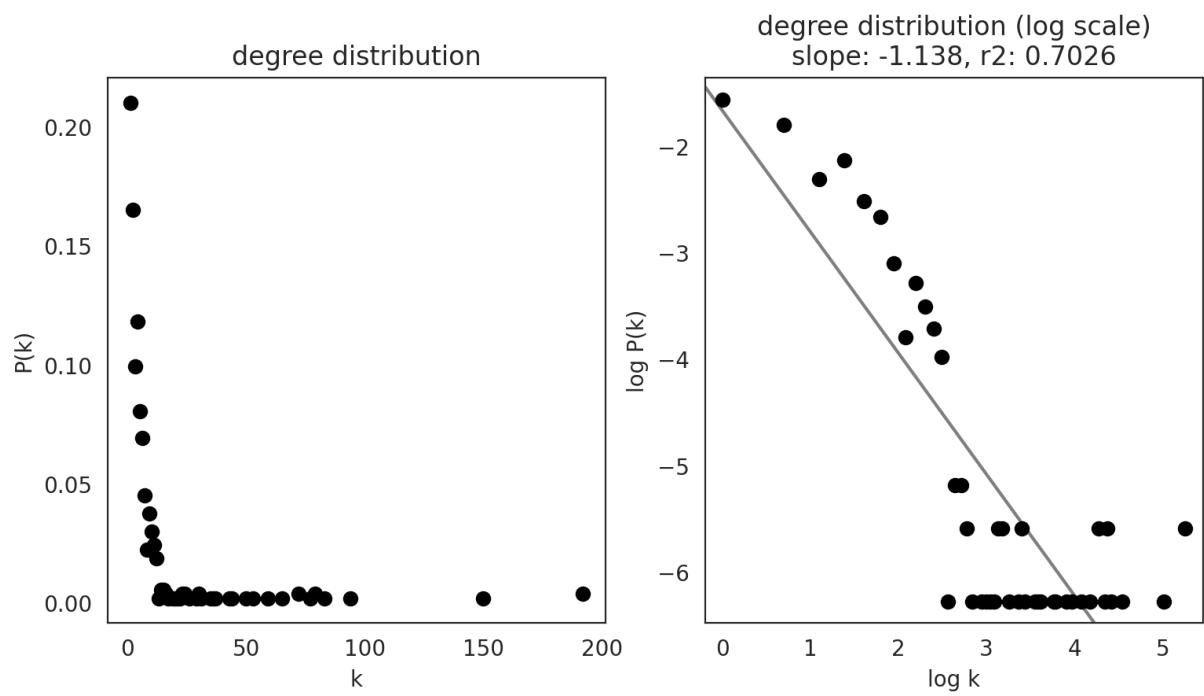
1



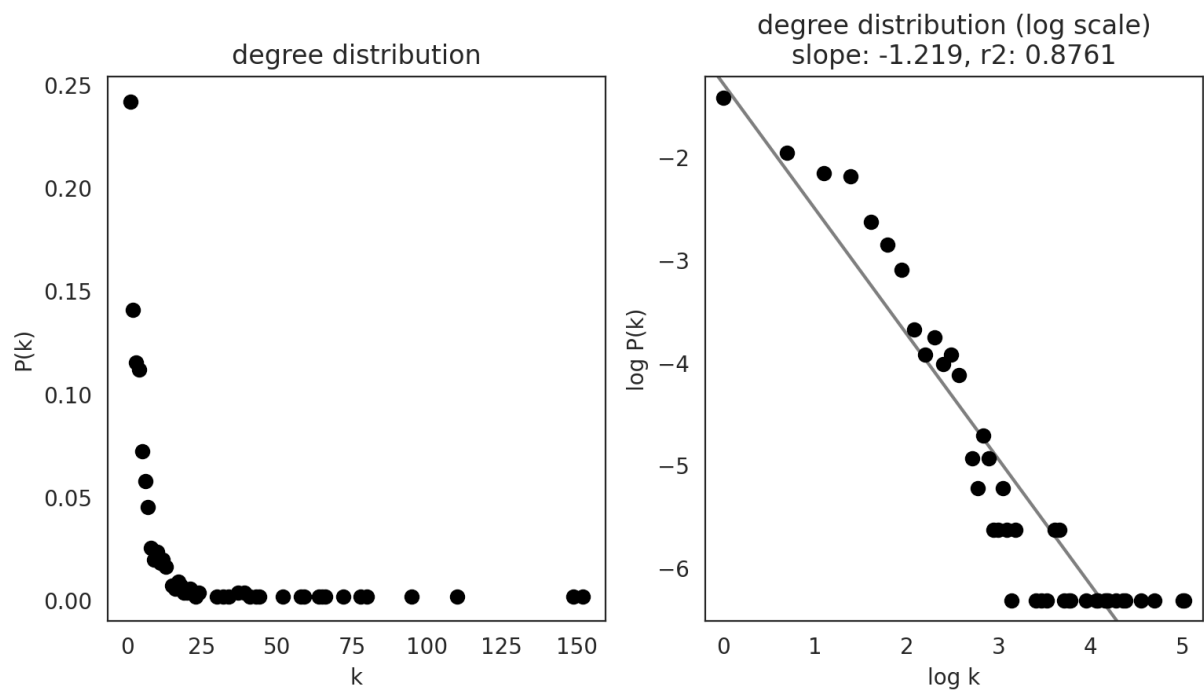
10



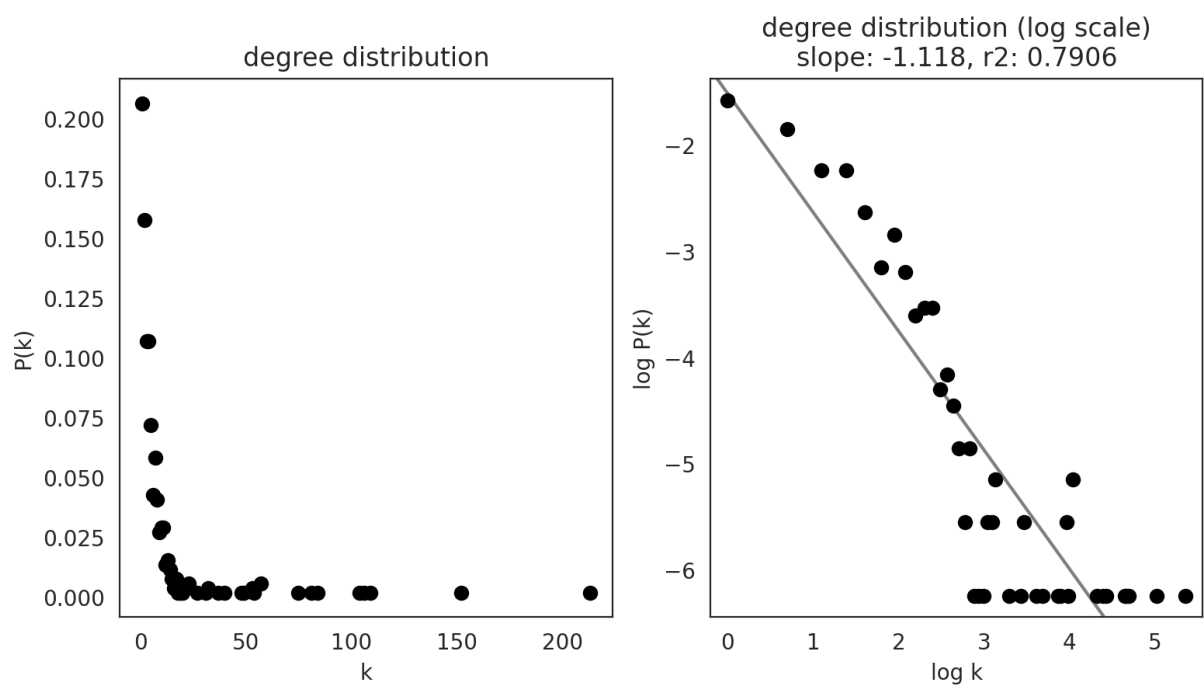
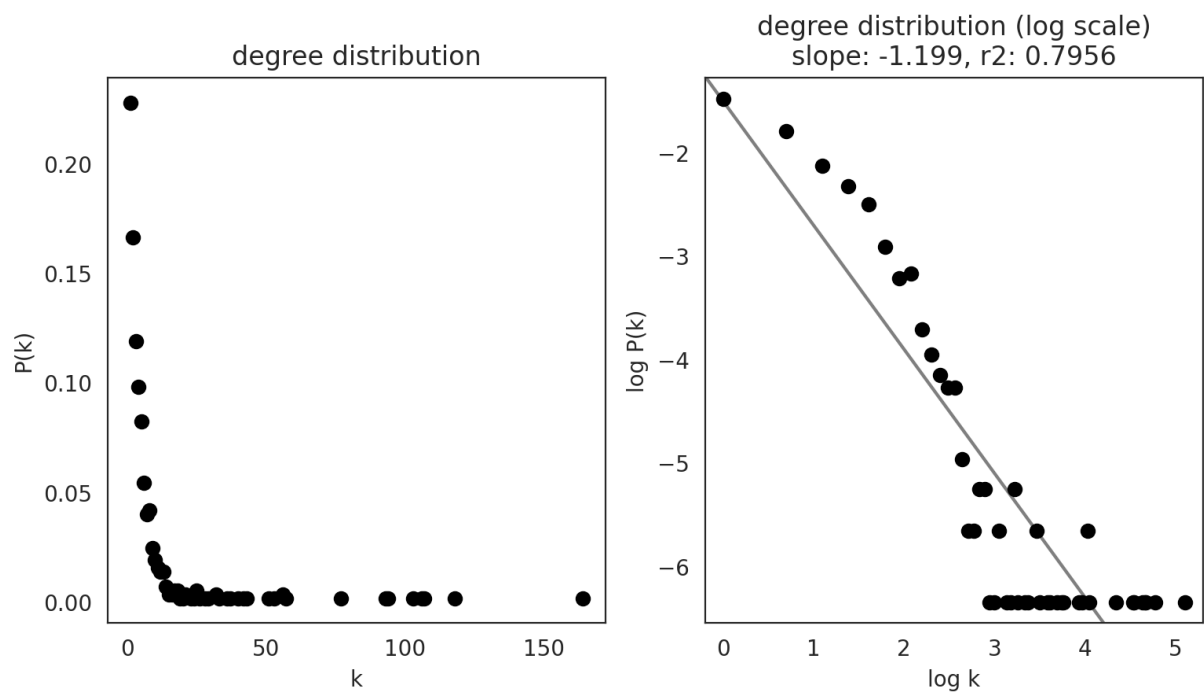
11

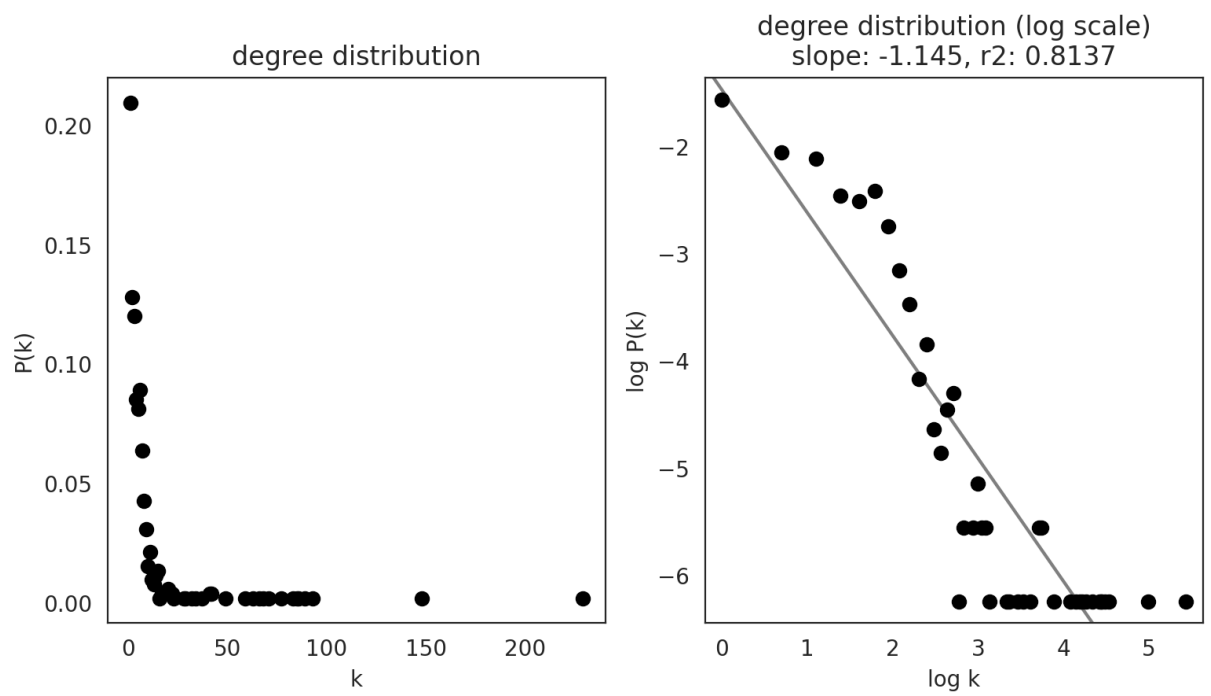


12

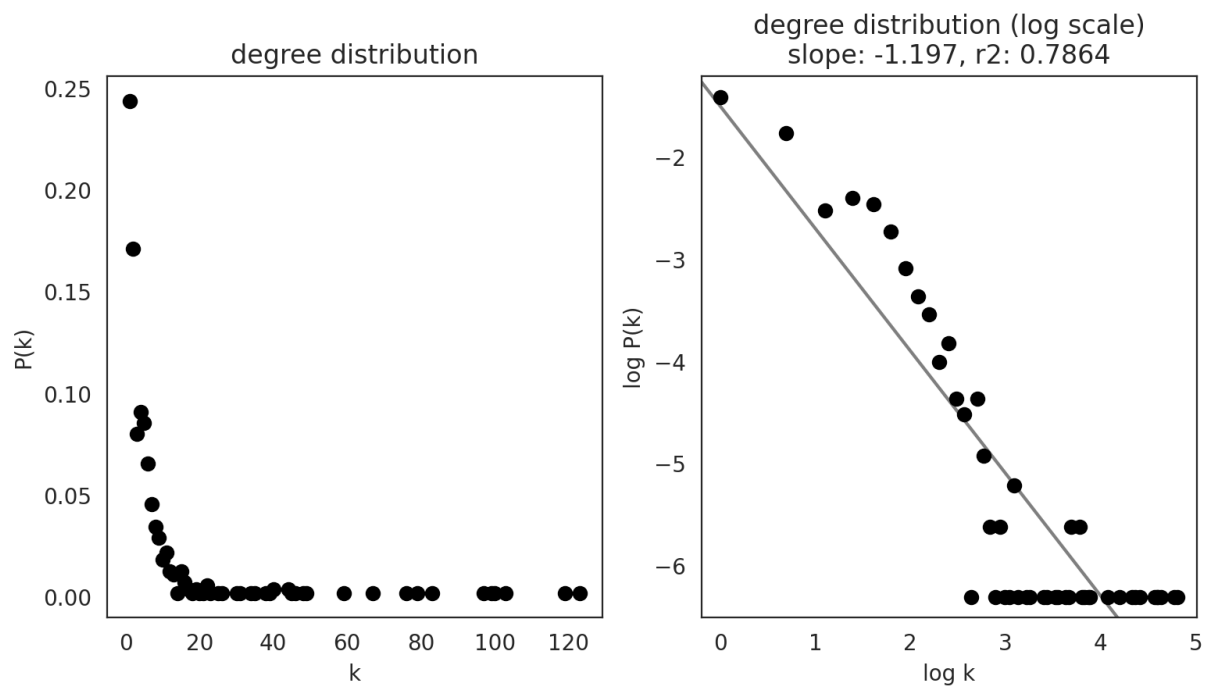


13

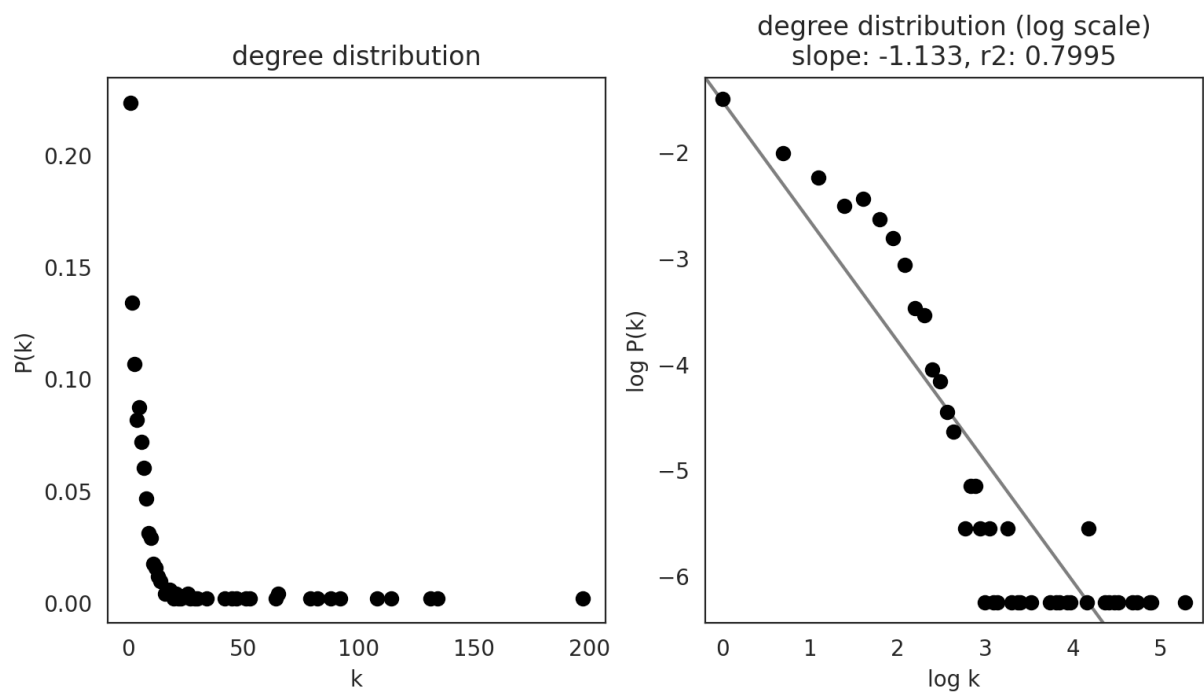




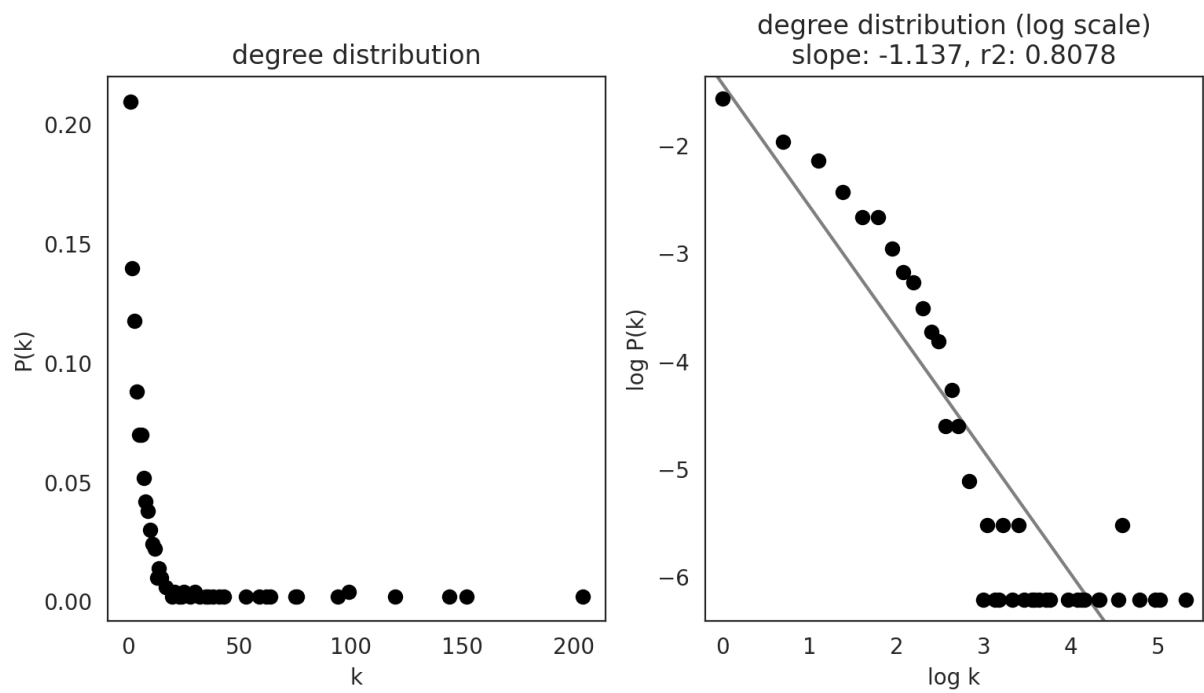
16



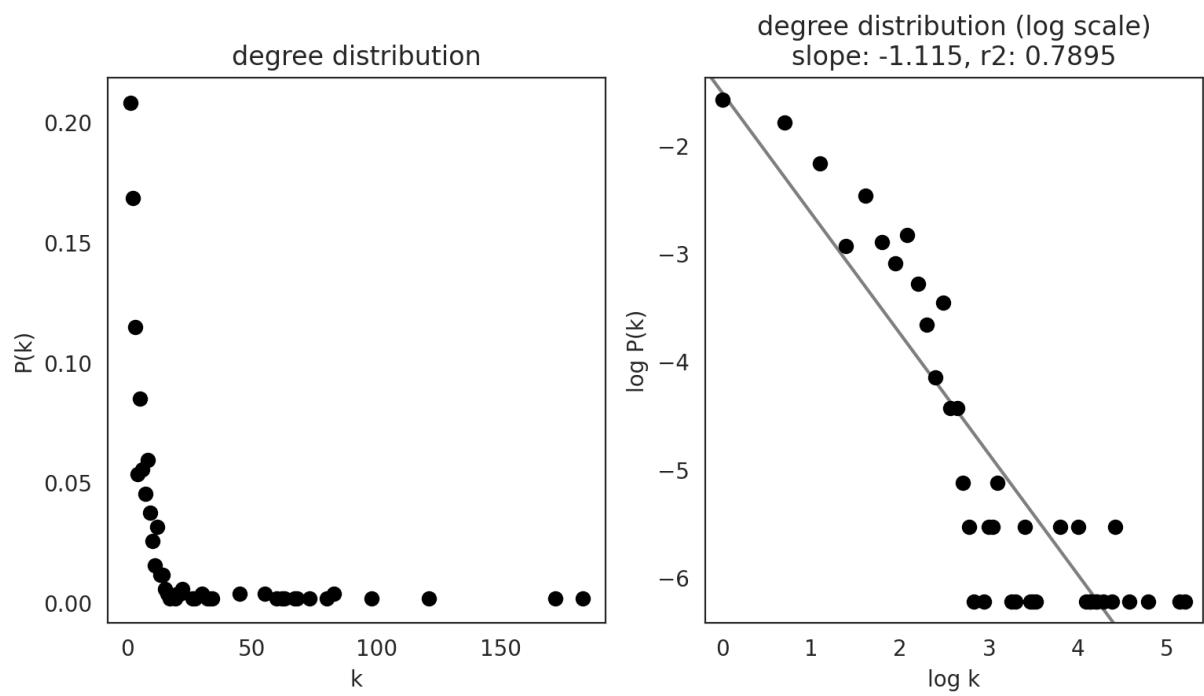
17



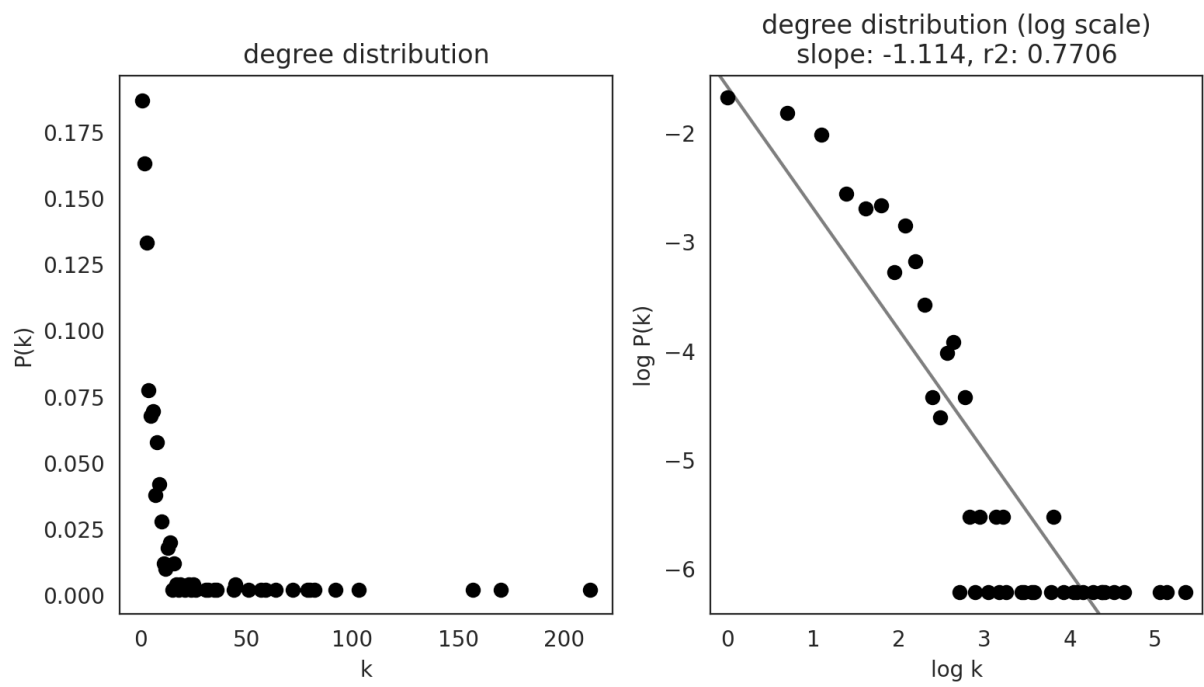
18



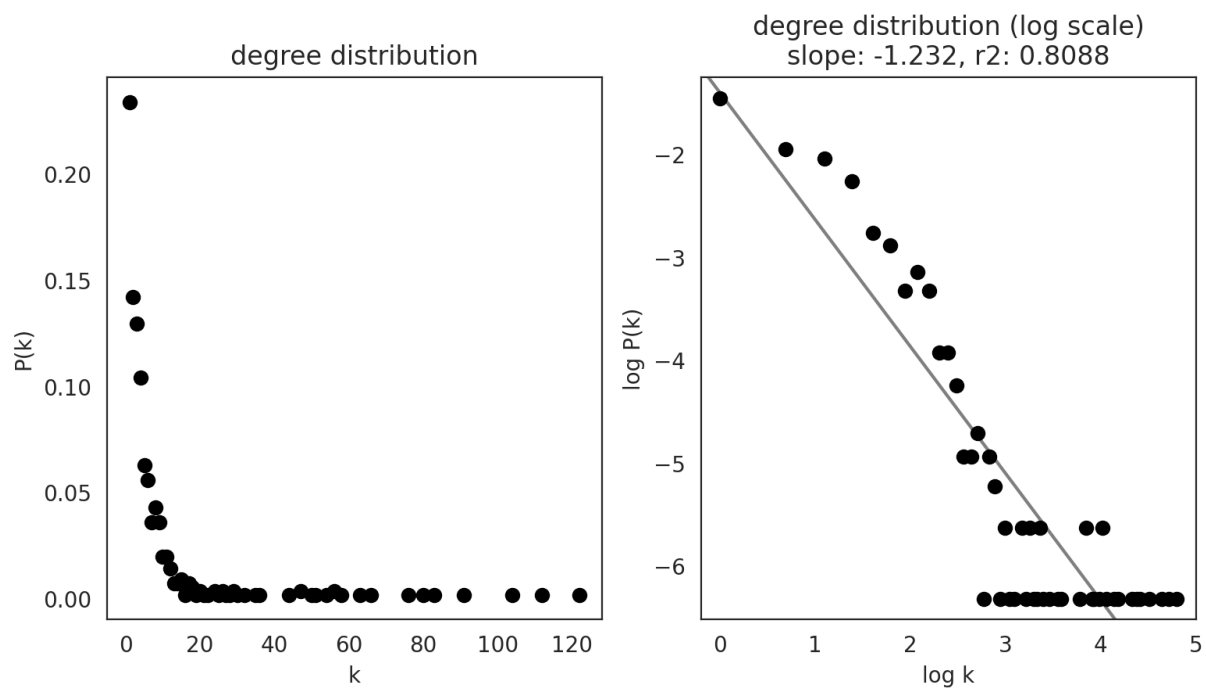
19



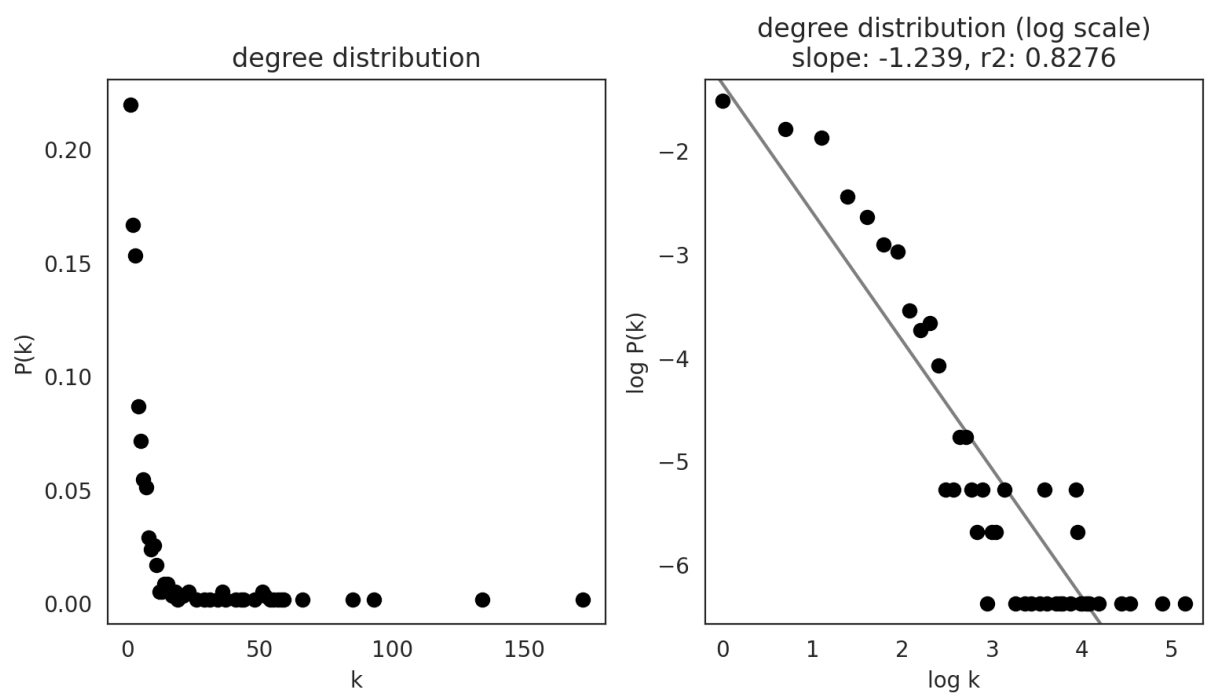
2



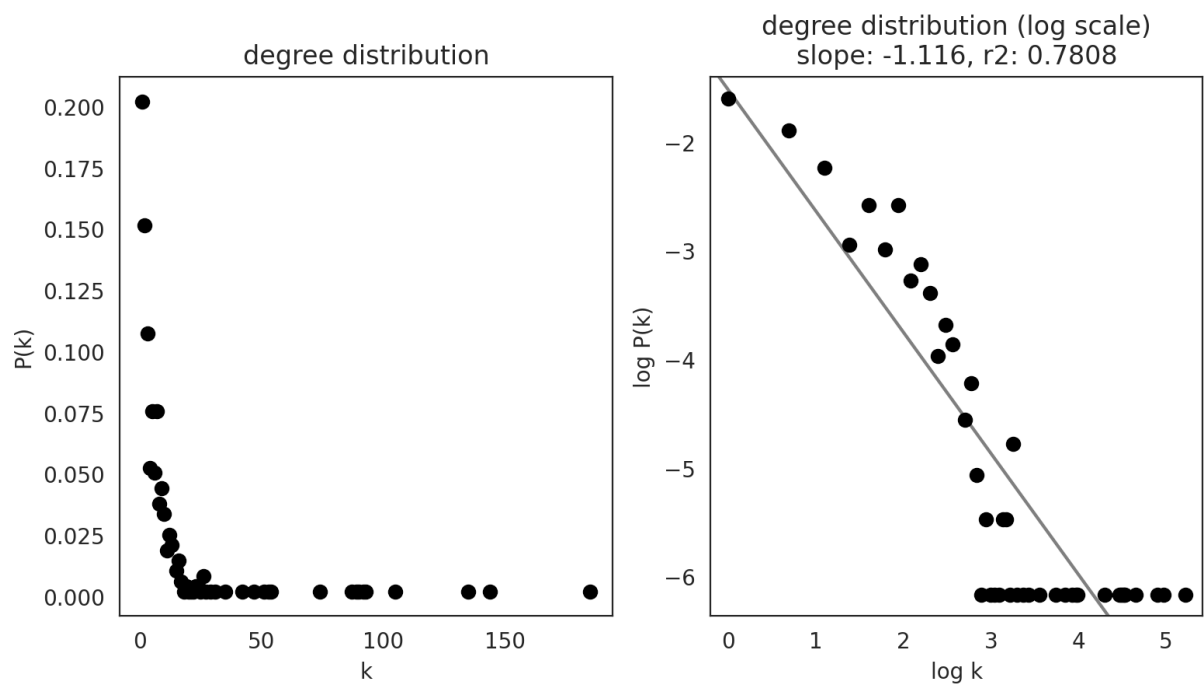
20



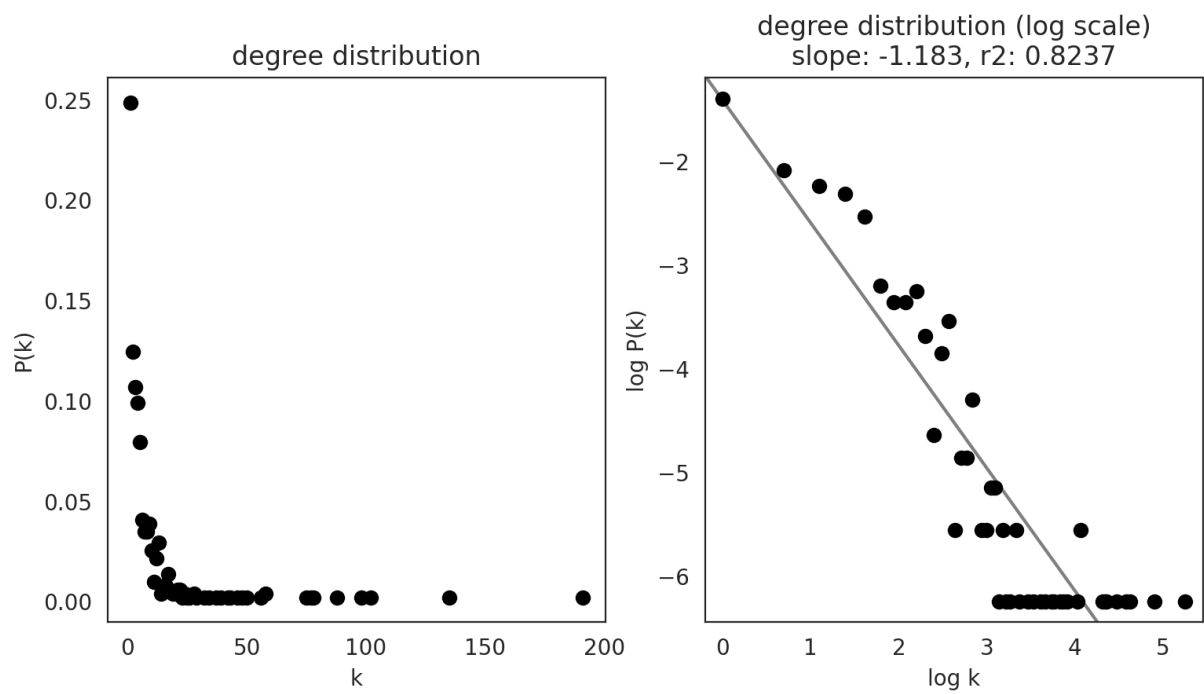
21



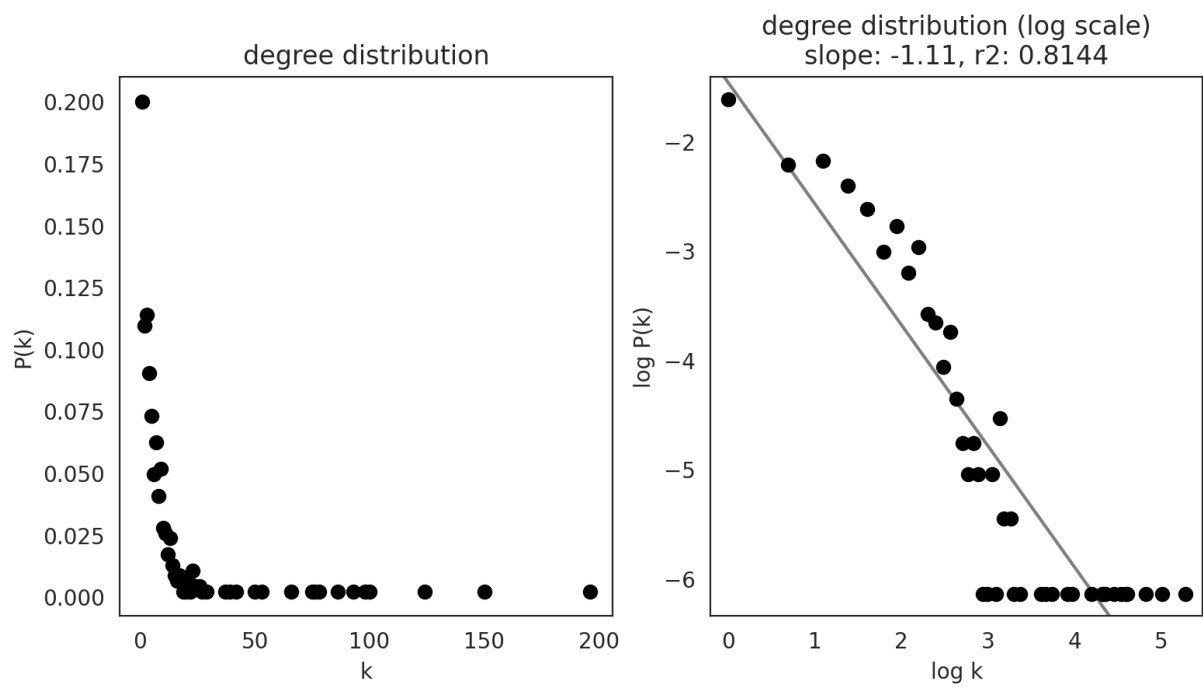
22



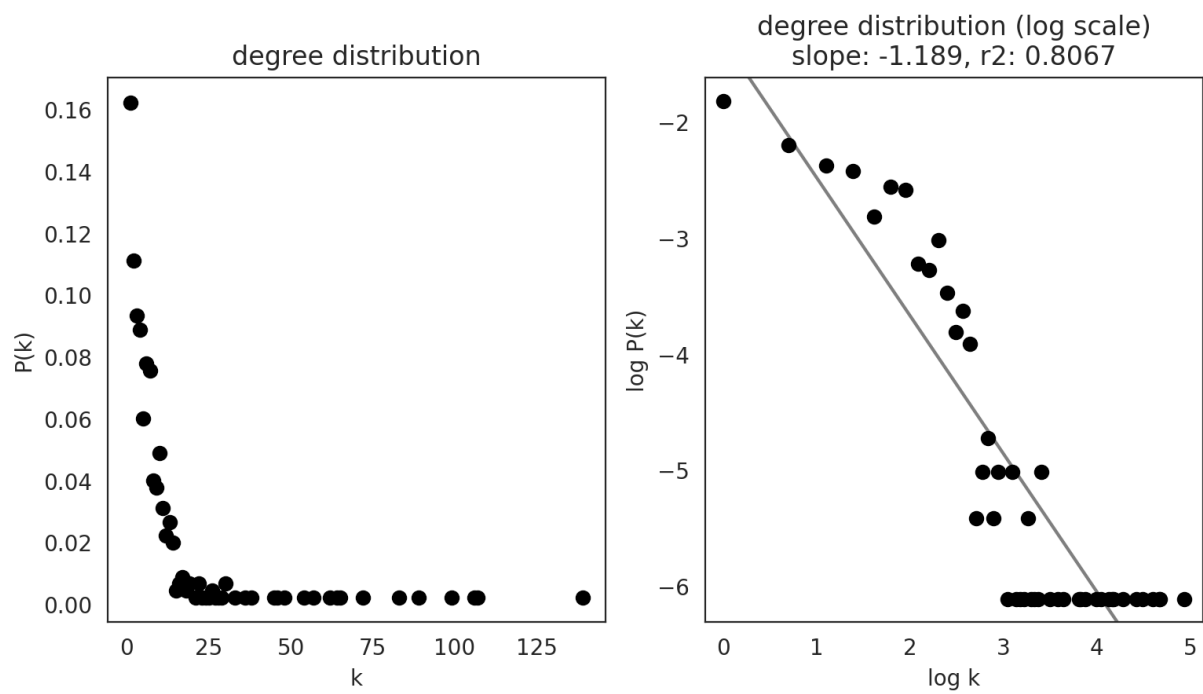
23



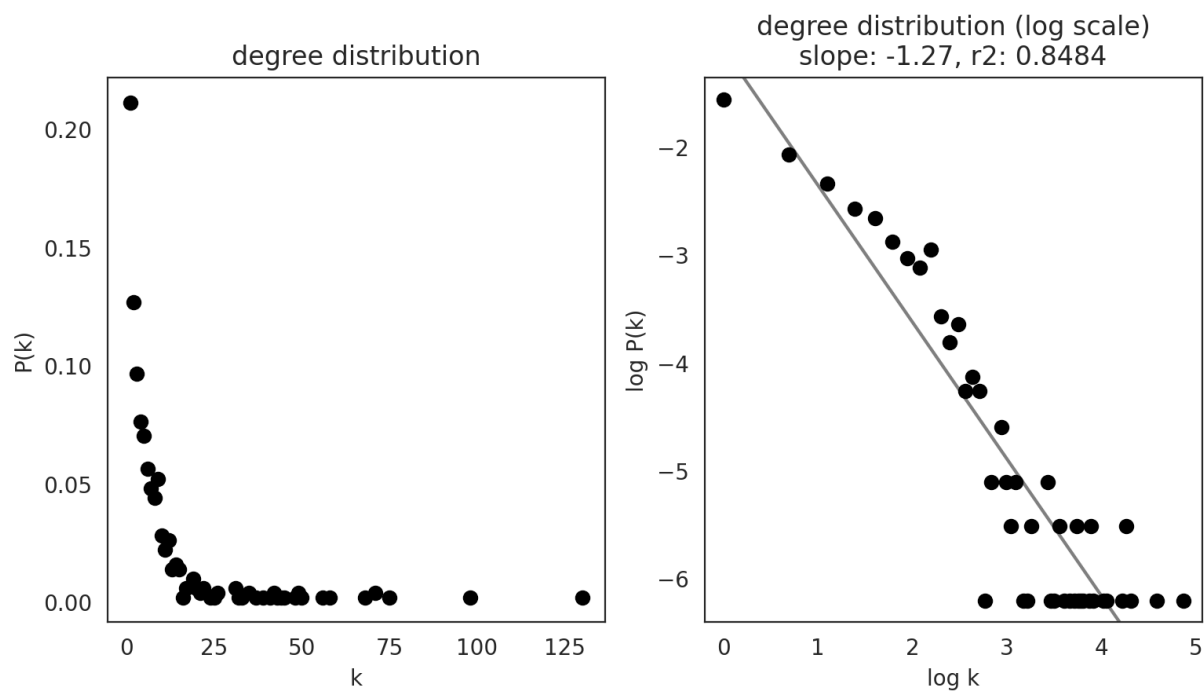
24



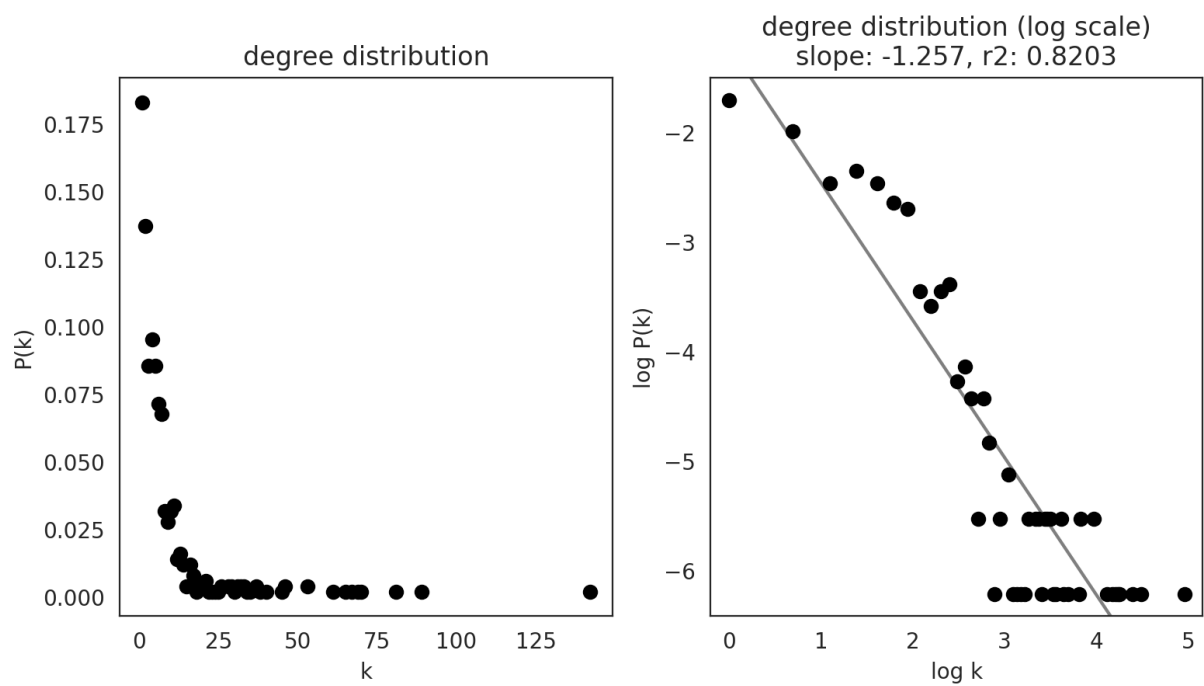
25



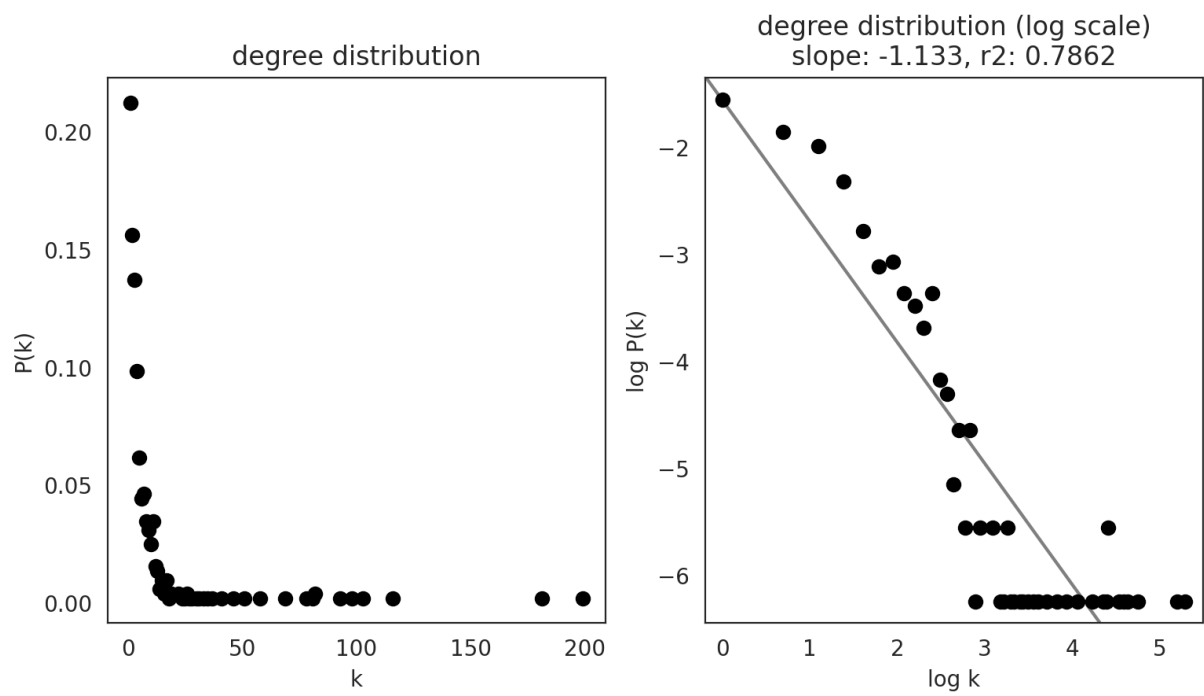
26



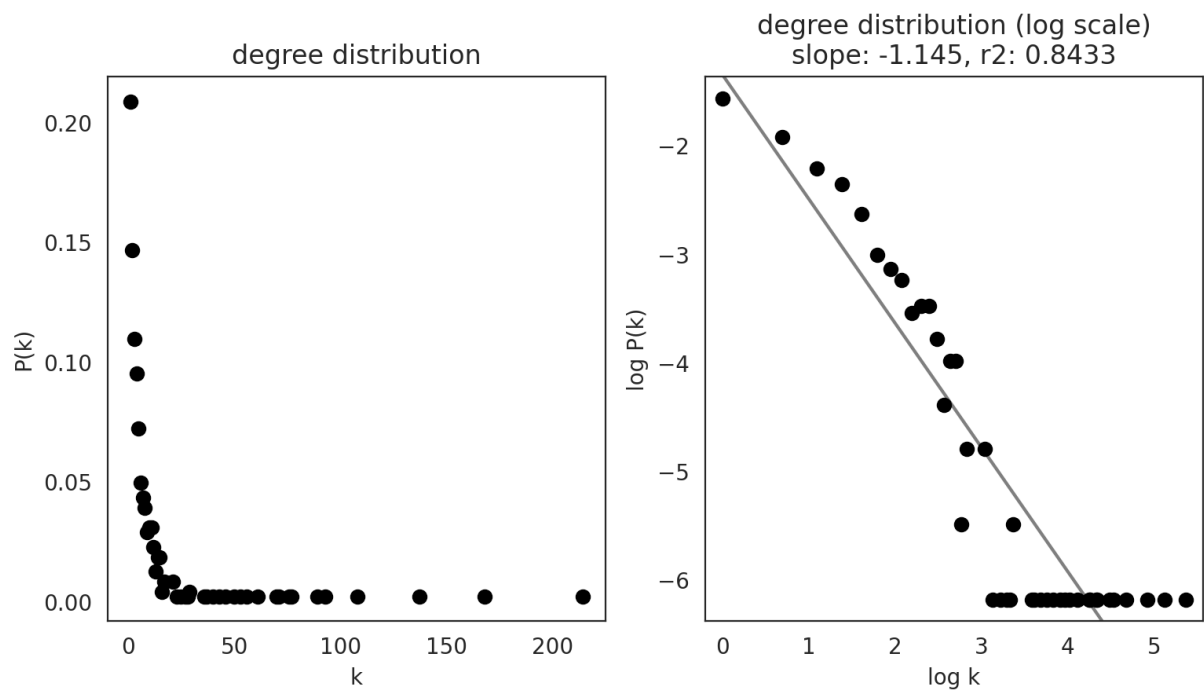
27



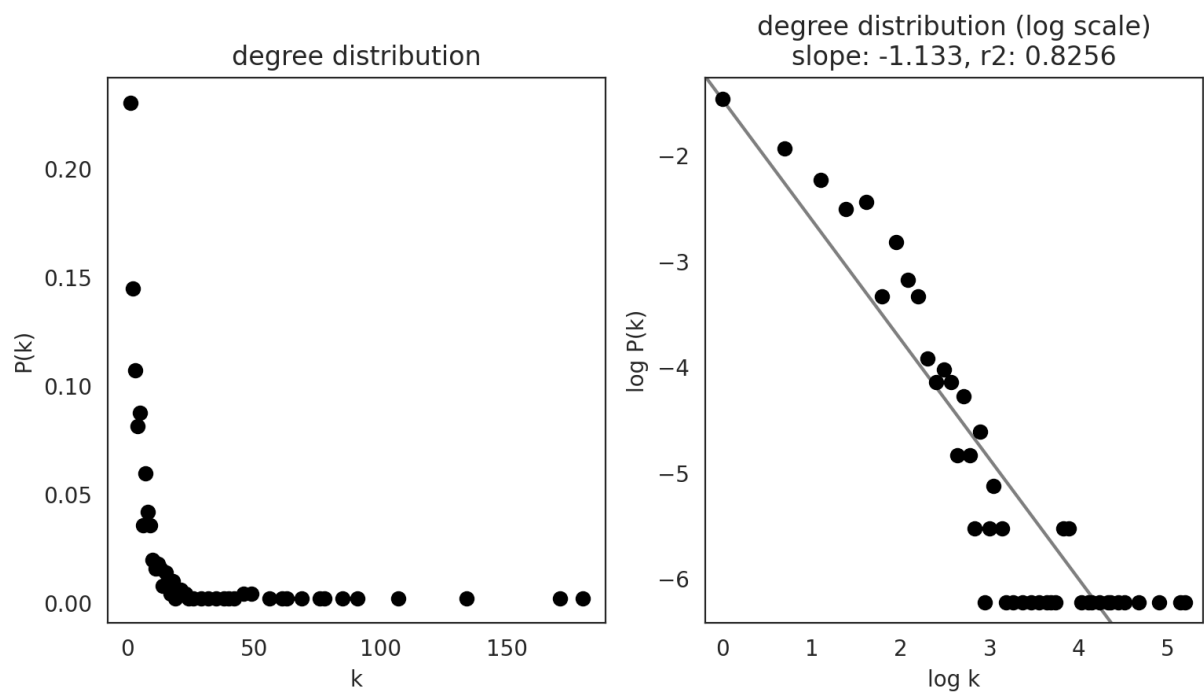
3



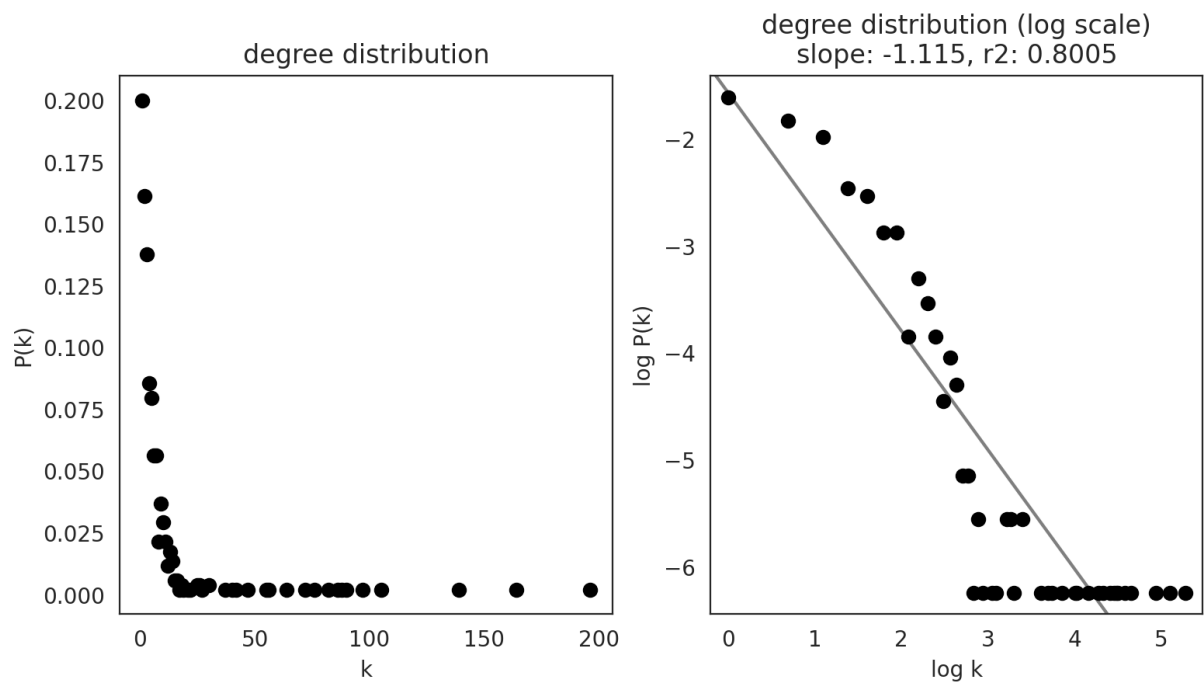
4



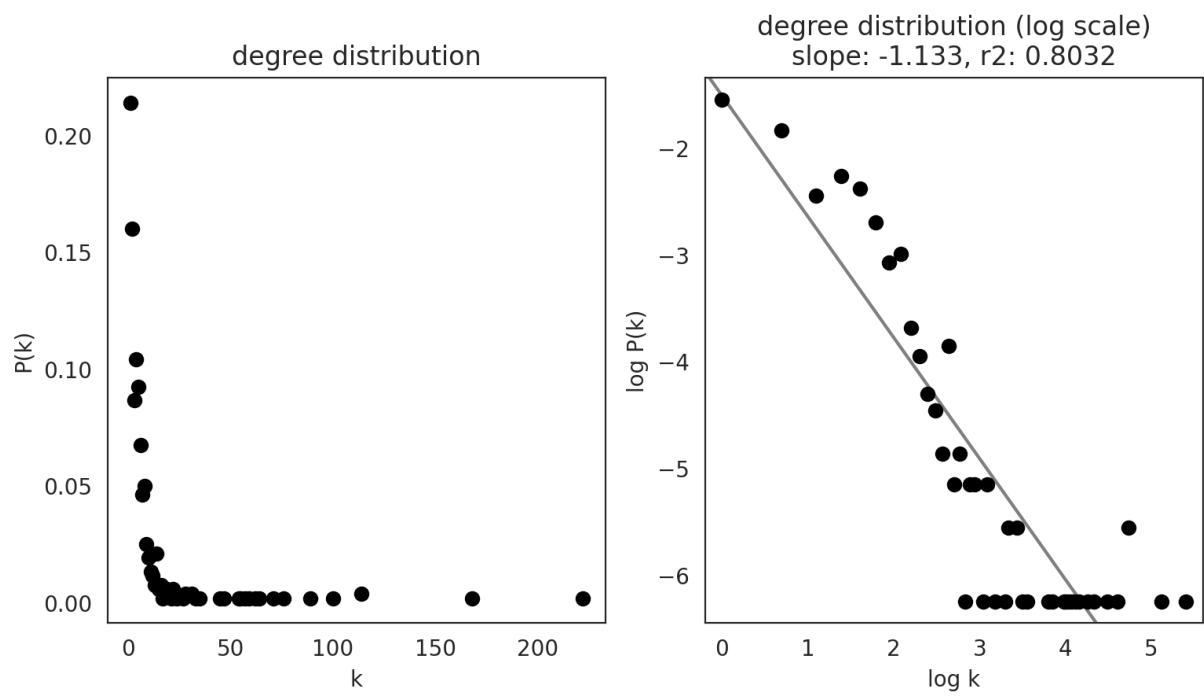
5



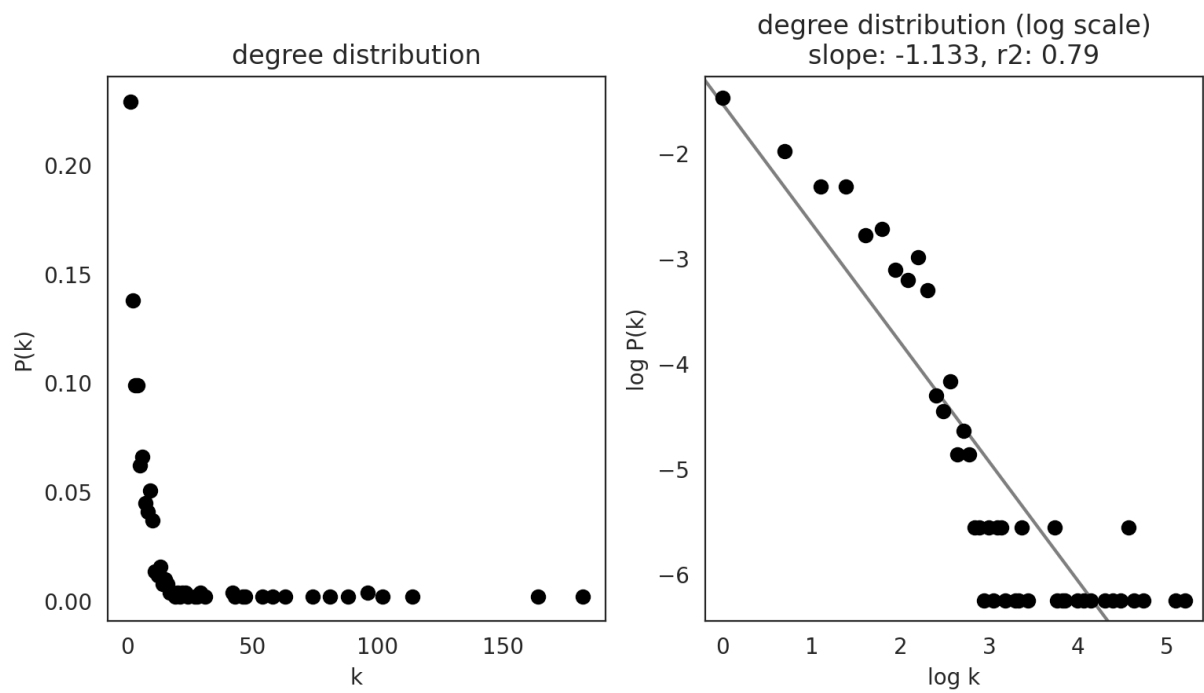
6



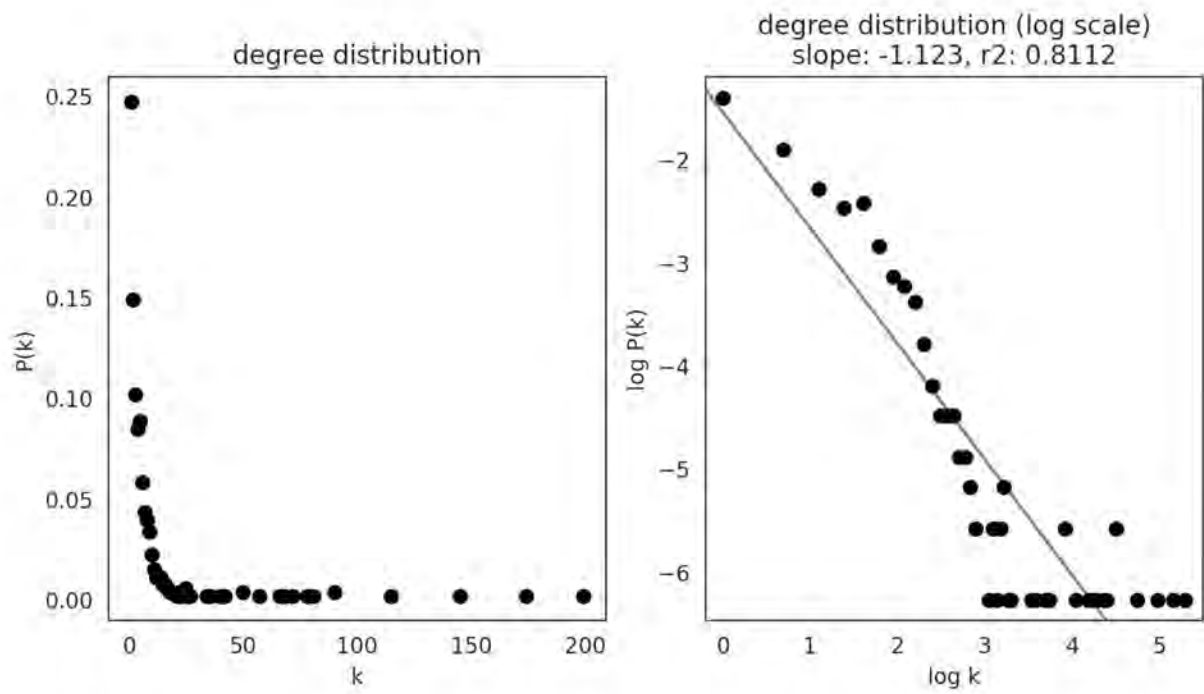
7



8



9



```
In [50]: save_folder
```

```
Out[50]: '/xxx/analysis/PT_subset/scanpy/figures'
```

```
In [51]: # Calculate network scores.
links.get_network_score()
links.merged_score.head(100)
```

```
Out[51]:
```

	degree_all	degree_centrality_all	degree_in	degree_centrality_in	degree_out	degr
Bcl6	55	0.103578	6	0.011299	49	
Slc17a1	8	0.015066	8	0.015066	0	
Esr1	79	0.148776	0	0.000000	79	
ErbB4	15	0.028249	15	0.028249	0	
Klf6	236	0.444444	7	0.013183	229	
...	...	...	...	...	...	...
Hspb1	2	0.003766	2	0.003766	0	
Cadps2	7	0.013183	7	0.013183	0	
Spp1	5	0.009416	5	0.009416	0	
Slc5a2	1	0.001883	1	0.001883	0	
Slc6a18	3	0.005650	3	0.005650	0	

100 rows × 9 columns

```
In [52]: # Save Links object.
```

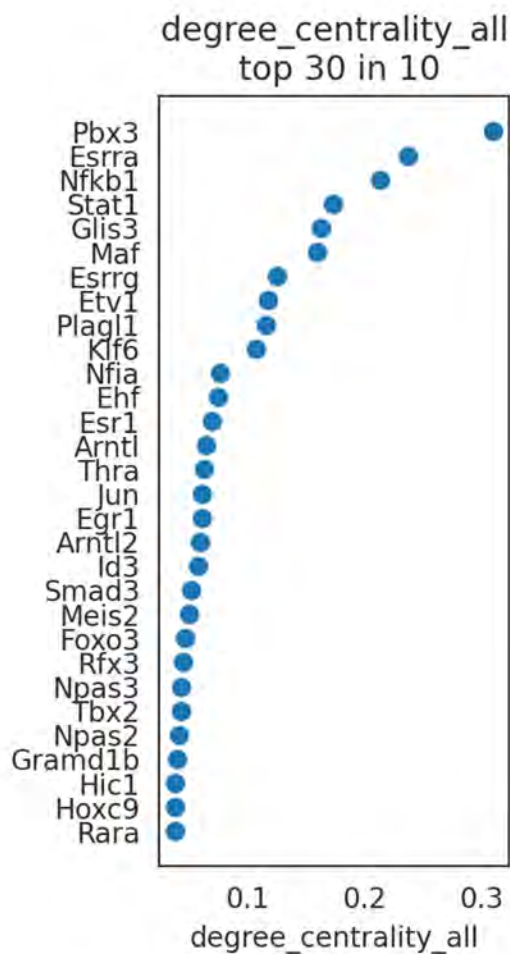
```
links.to_hdf5(file_path="/xxx/analysis/PT_subset/Sobj_PT1_2_inj_filtered_links.cell
```

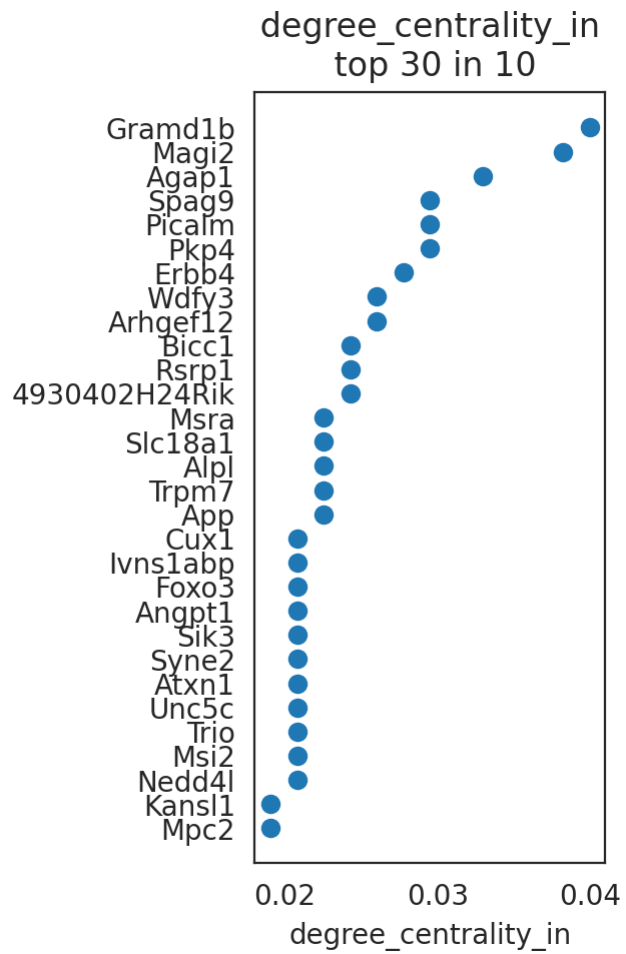
```
In [53]: #2. Make predictive models for simulation
links.filter_links()
oracle.get_cluster_specific_TFdict_from_Links(links_object=links)
oracle.fit_GRN_for_simulation(alpha=10,
                             use_cluster_specific_TFdict=True)
oracle.to_hdf5("/xxx/analysis/PT_subset/Sobj_PT1_2_inj_step3.celloracle.oracle")

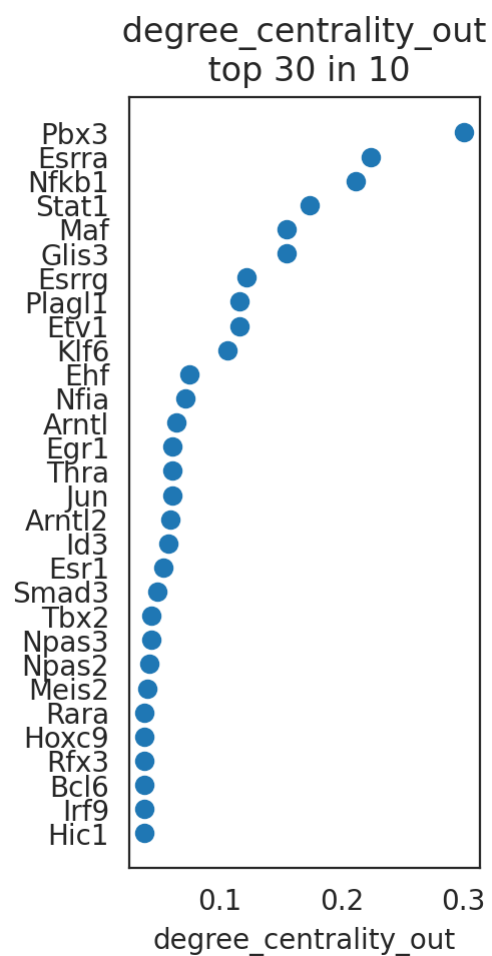
0%|          | 0/28 [00:00<?, ?it/s]
```

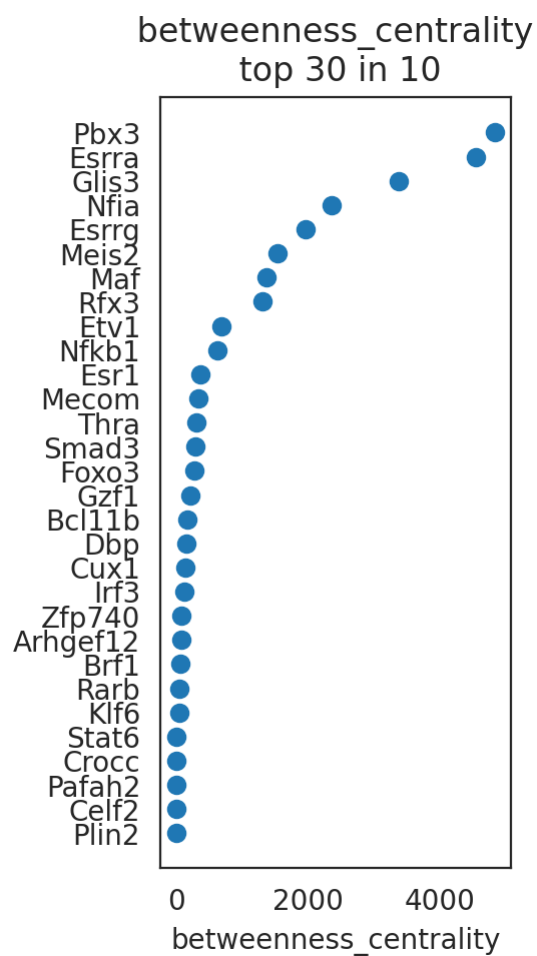
```
In [135... plt.rcParams['savefig.facecolor'] = 'white'

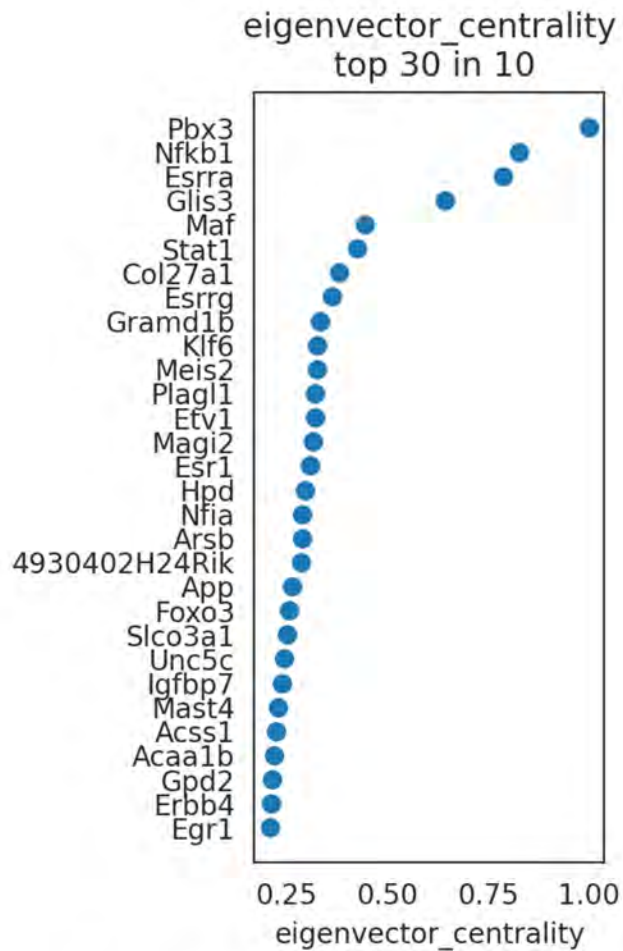
links.plot_scores_as_rank(cluster="10", n_gene=30)
plt.savefig(f"{save_folder}/PT1_2_inj_centrality_0.pdf")
plt.show()
```











<Figure size 500x500 with 0 Axes>

```
In [144... # Plot the score comparison

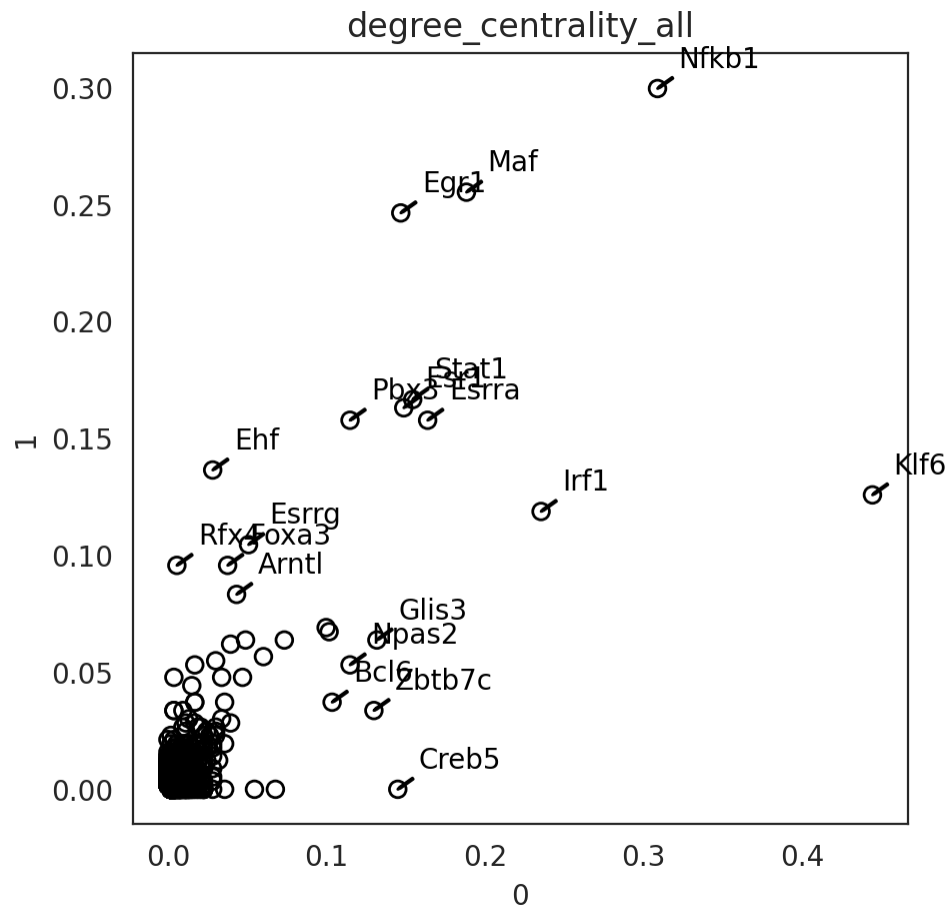
# Assume `links` is an instance of the object that has the `plot_score_comparison_2`
# and `save_folder` is the path to the folder where you want to save the plots

num_clusters = 27 # Total number of clusters
# Iterate through all unique pairs of clusters
for i in range(num_clusters):
    for j in range(i + 1, num_clusters):
        print(f"Processing clusters {i} and {j}")

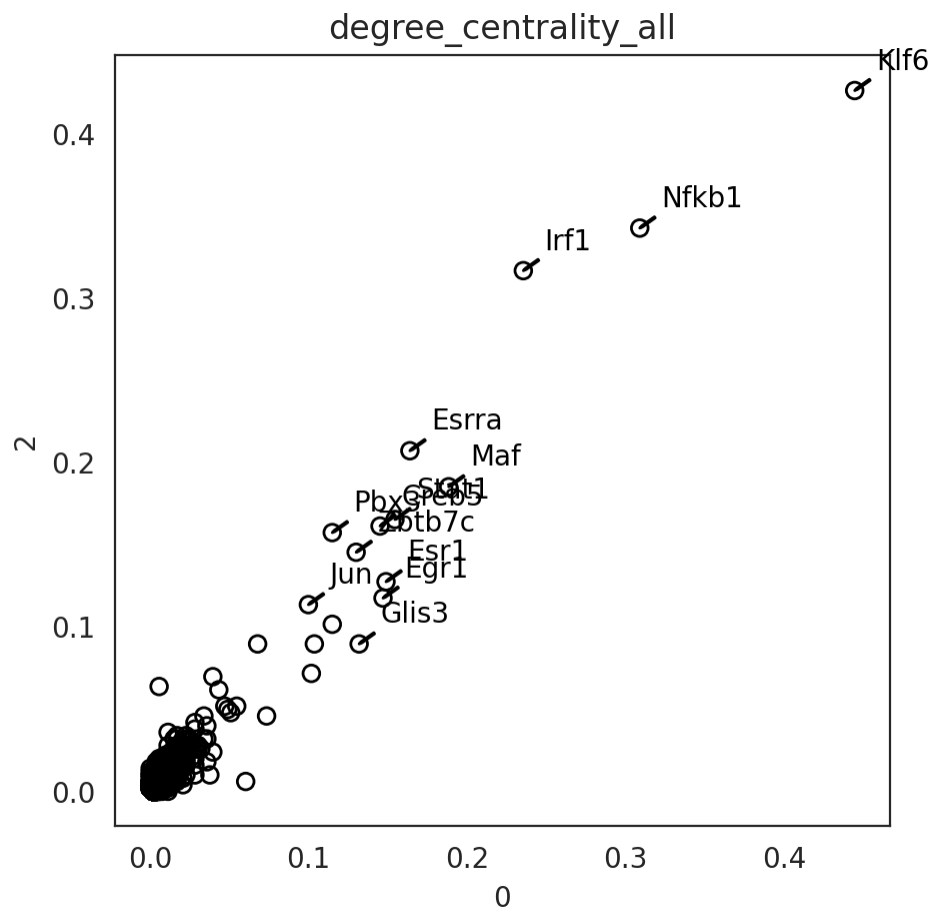
        # Create the plot for the current pair of clusters
        links.plot_score_comparison_2D(value="degree centrality_all",
                                       cluster1=str(i), cluster2=str(j),
                                       percentile=98,
                                       save=f"{save_folder}/inj_PT1_2_centrality")

        plt.show()
```

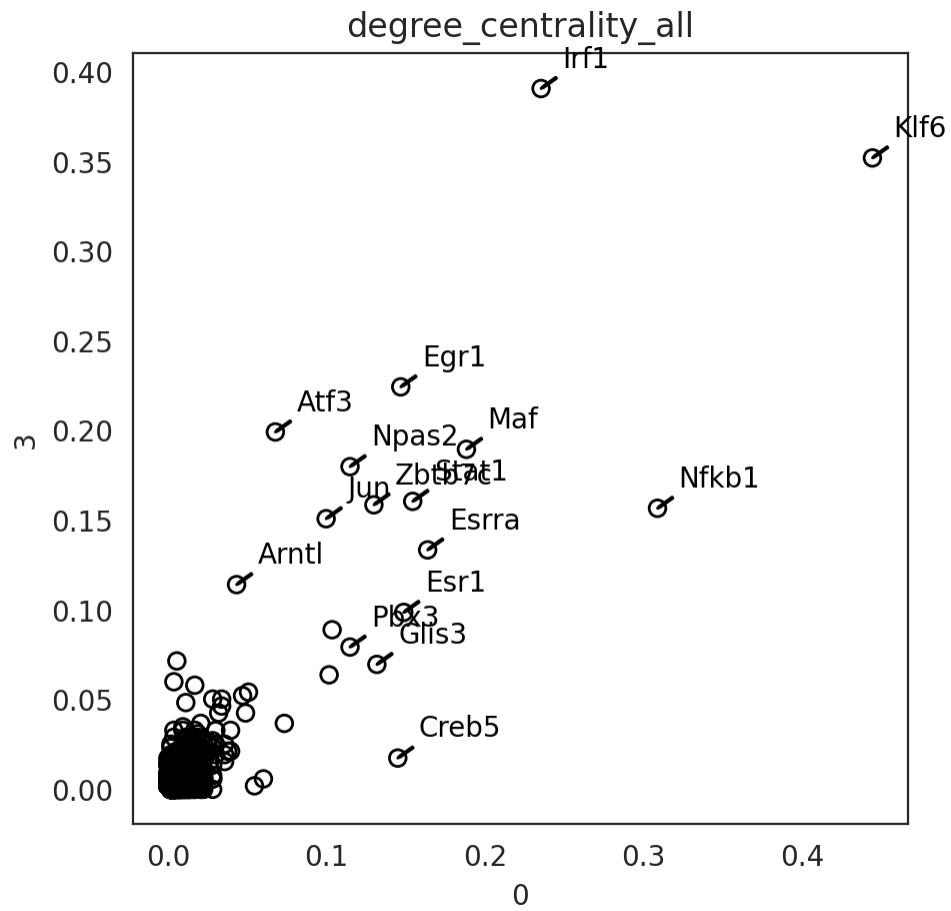
Processing clusters 0 and 1



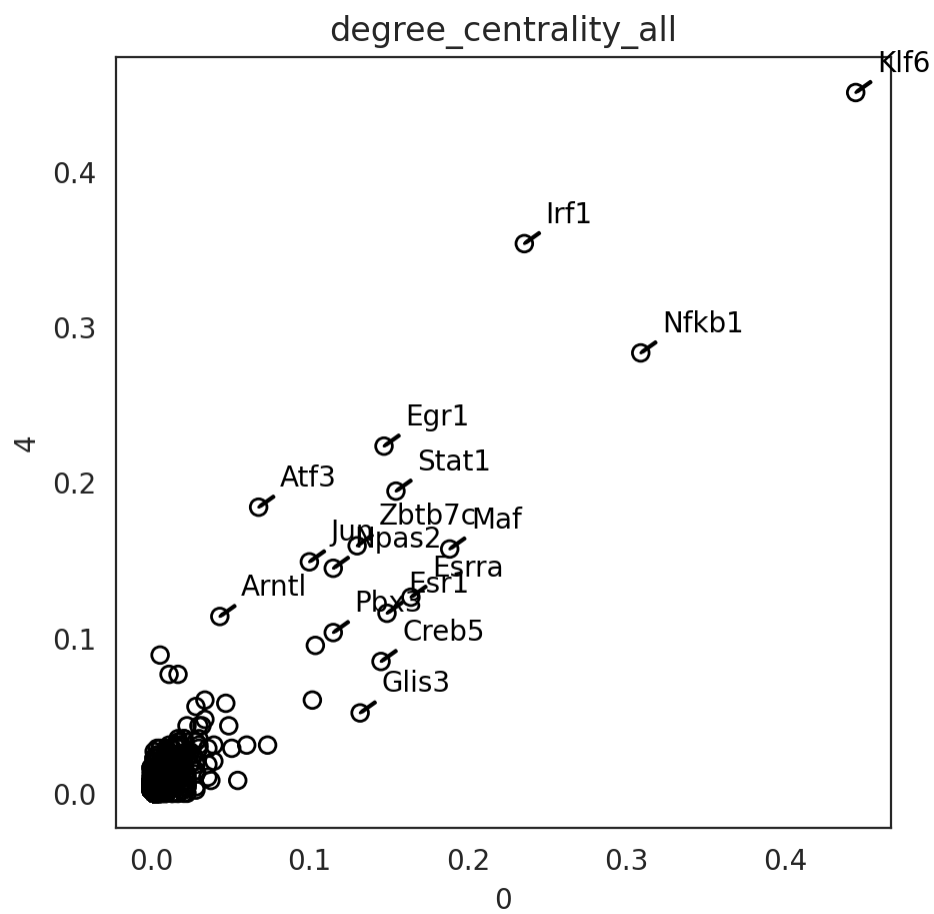
Processing clusters 0 and 2



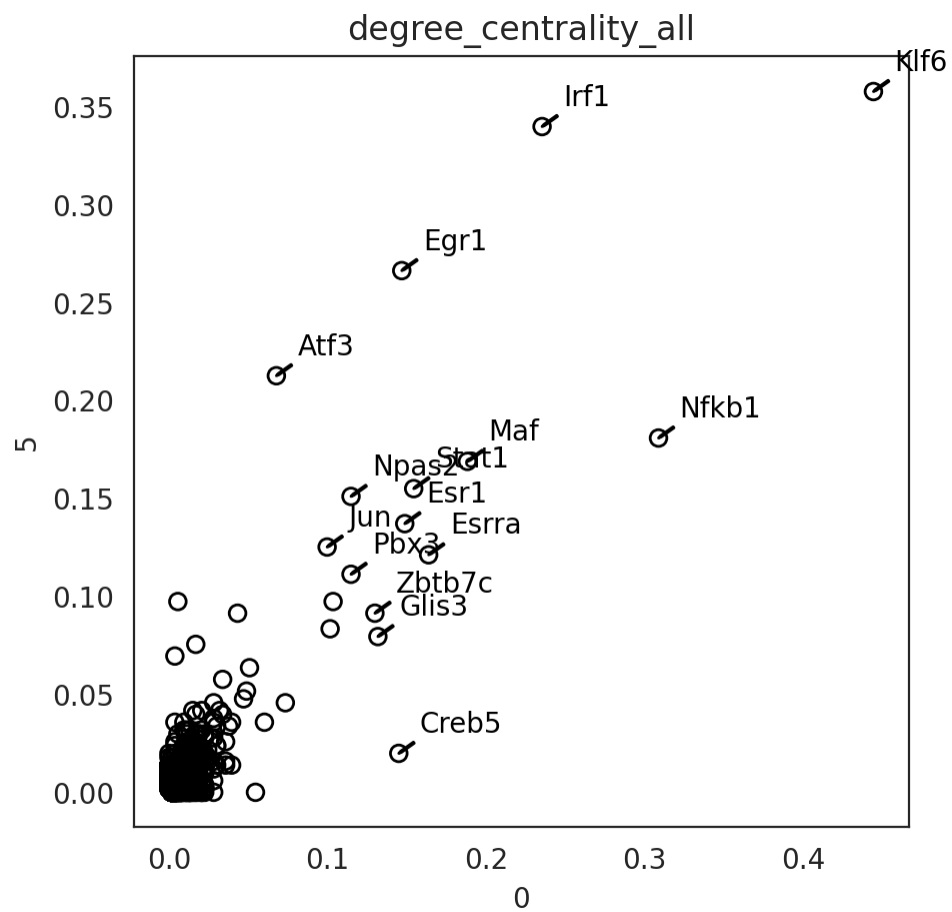
Processing clusters 0 and 3



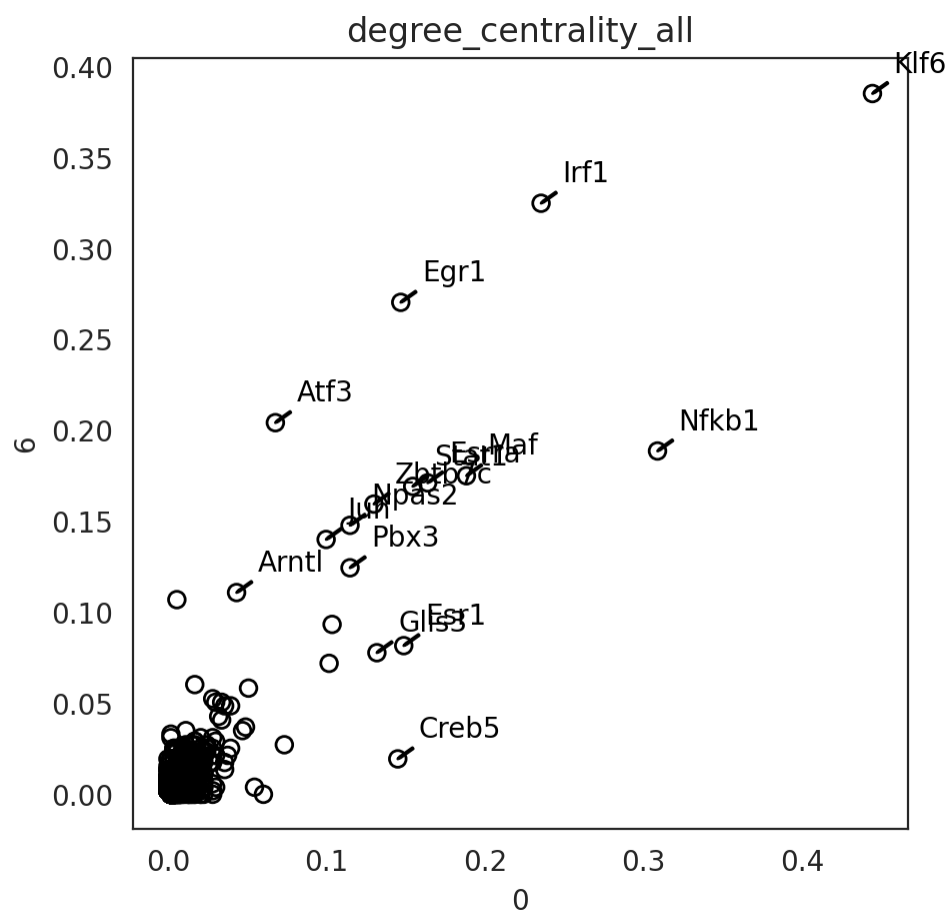
Processing clusters 0 and 4



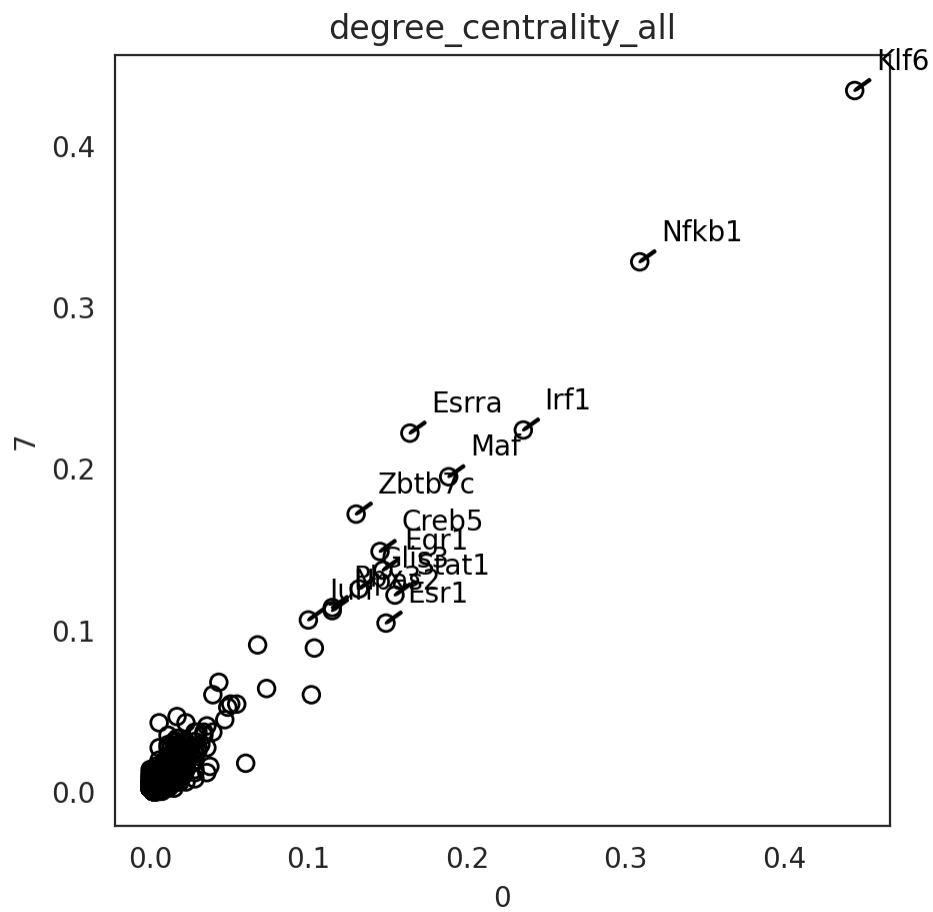
Processing clusters 0 and 5



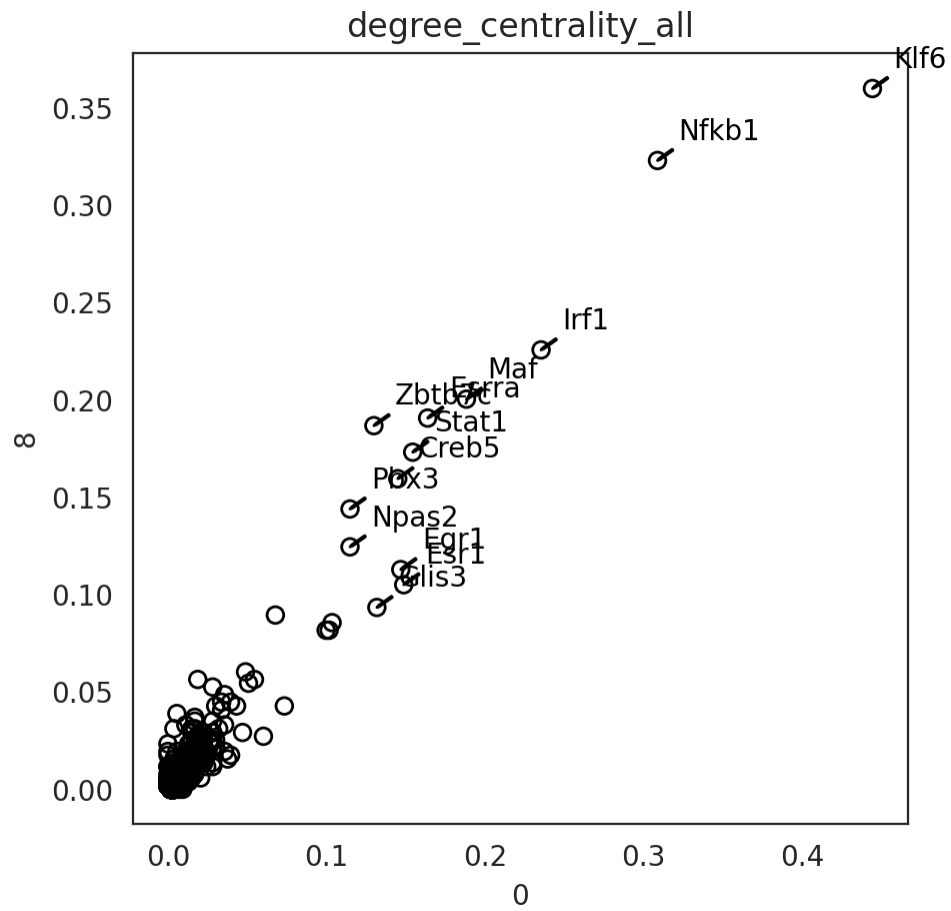
Processing clusters 0 and 6



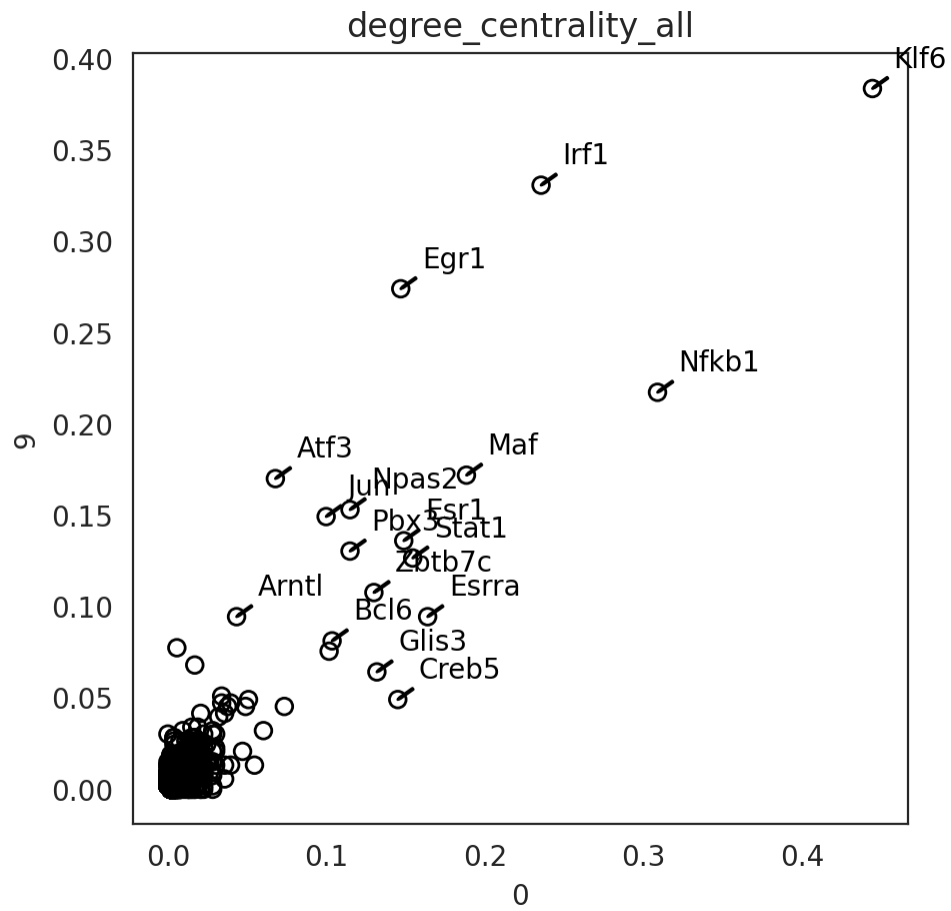
Processing clusters 0 and 7



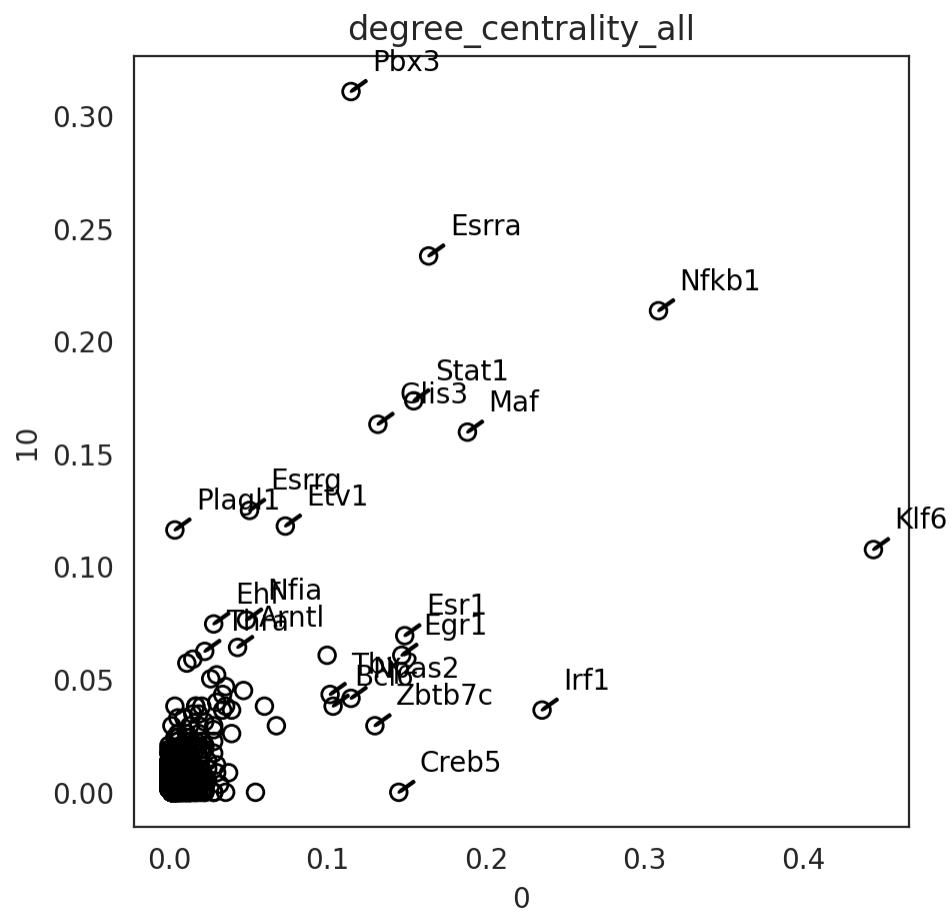
Processing clusters 0 and 8



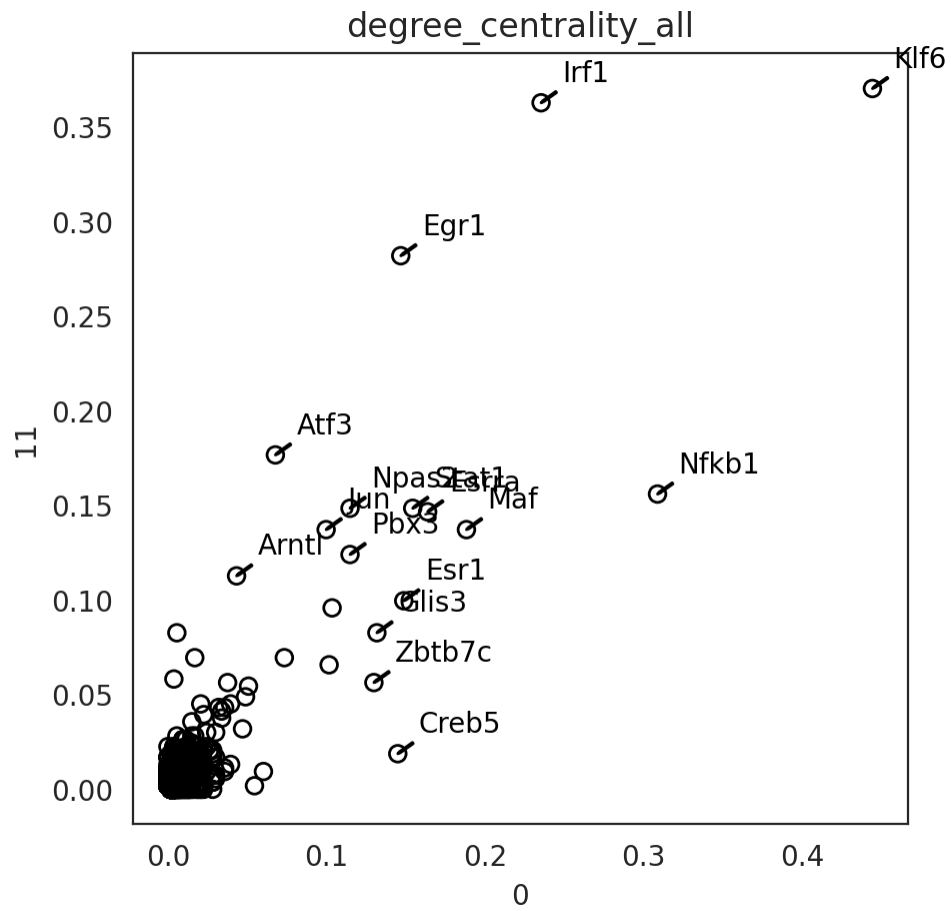
Processing clusters 0 and 9



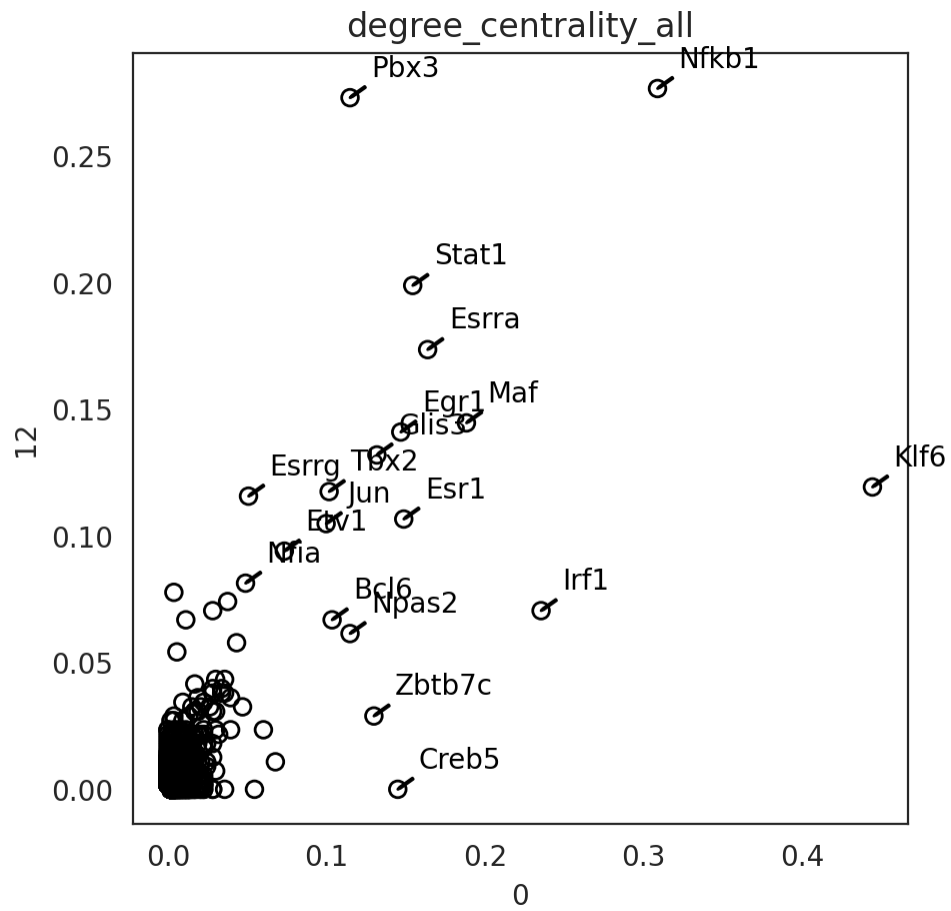
Processing clusters 0 and 10



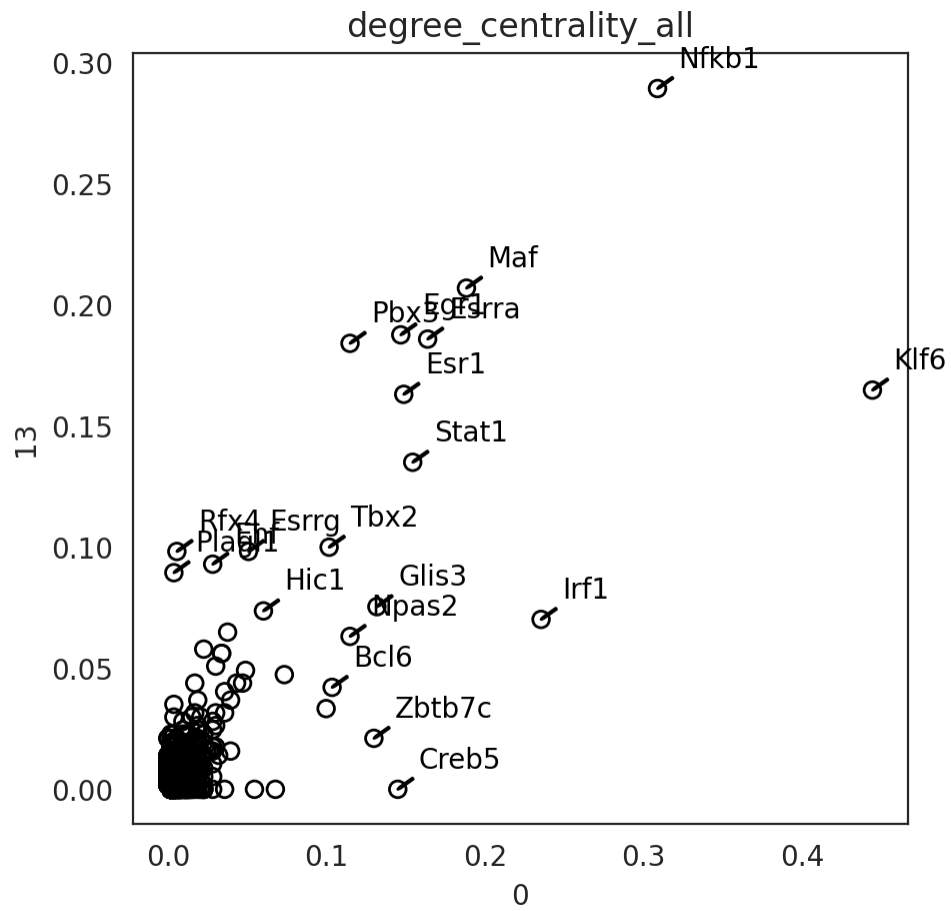
Processing clusters 0 and 11



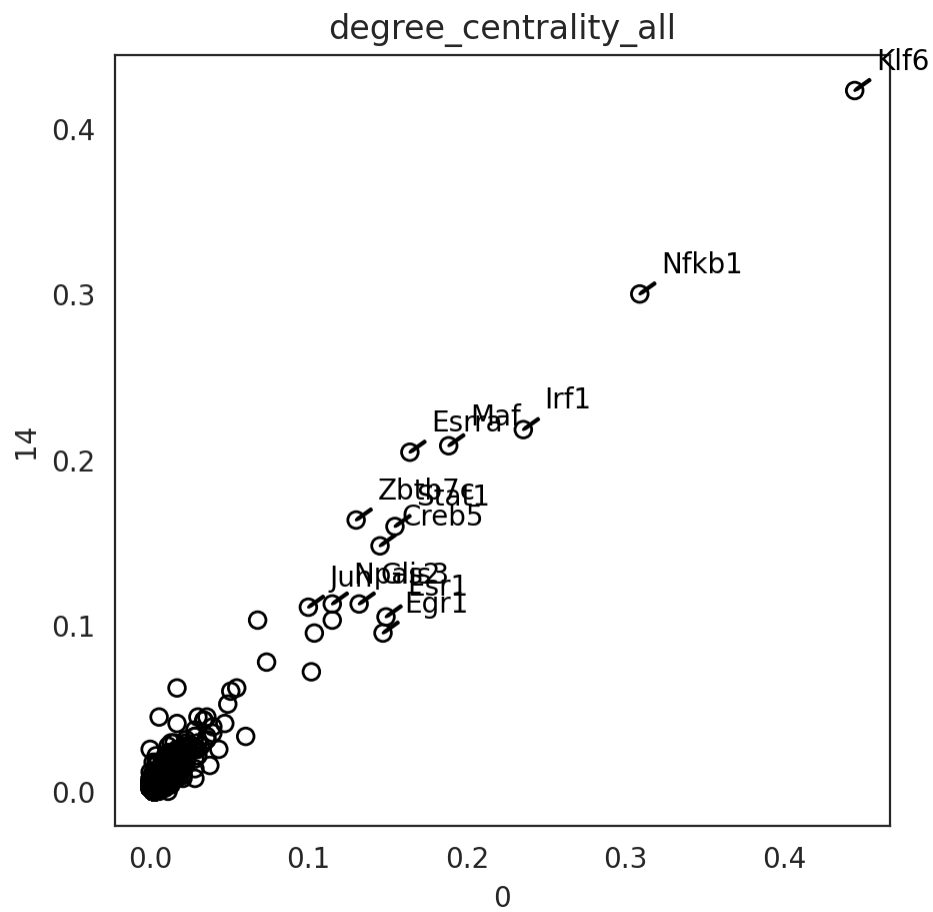
Processing clusters 0 and 12



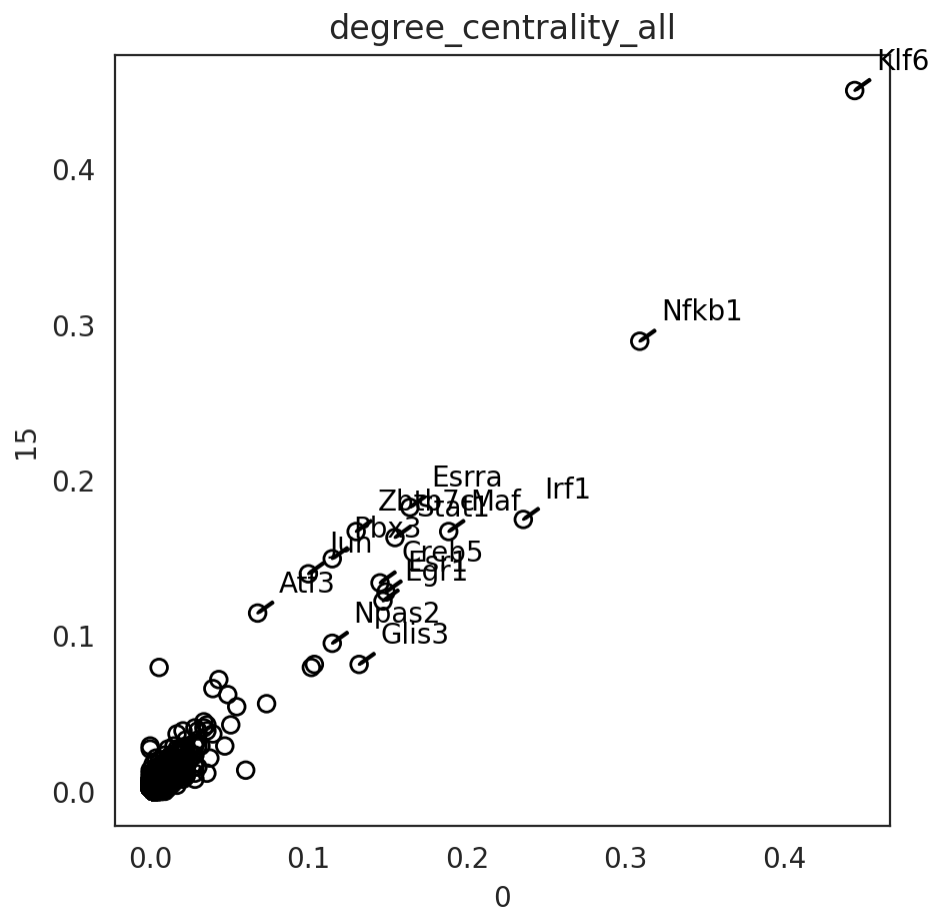
Processing clusters 0 and 13



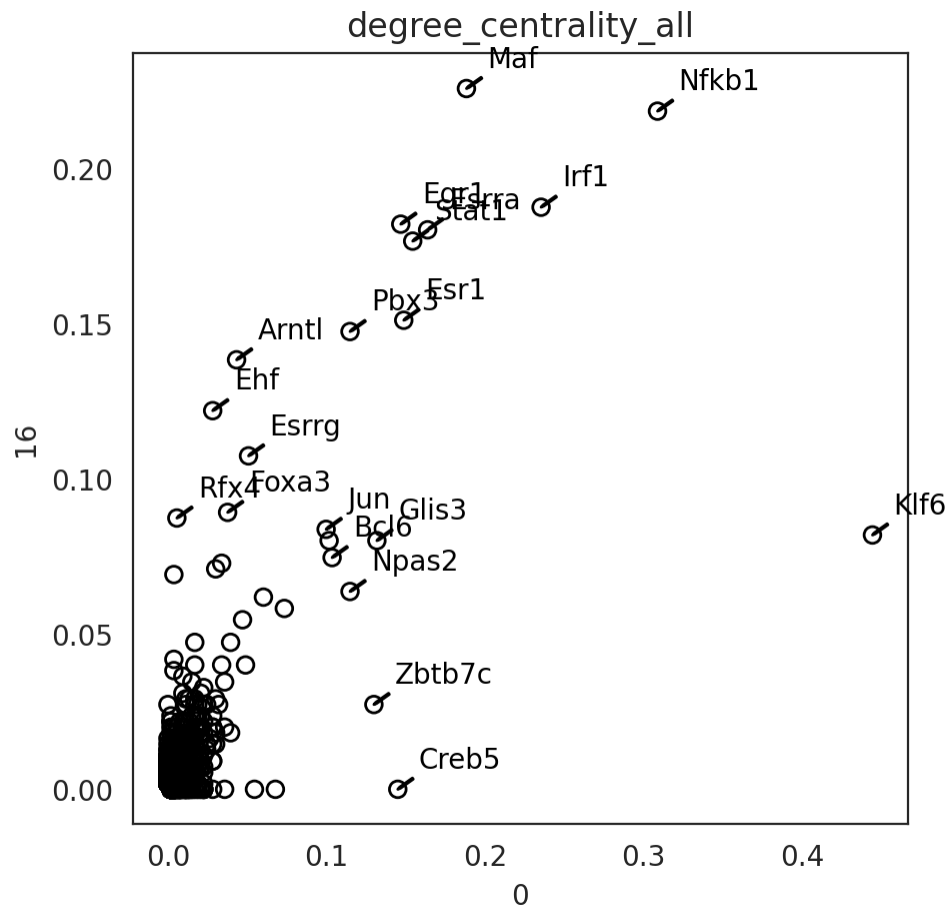
Processing clusters 0 and 14



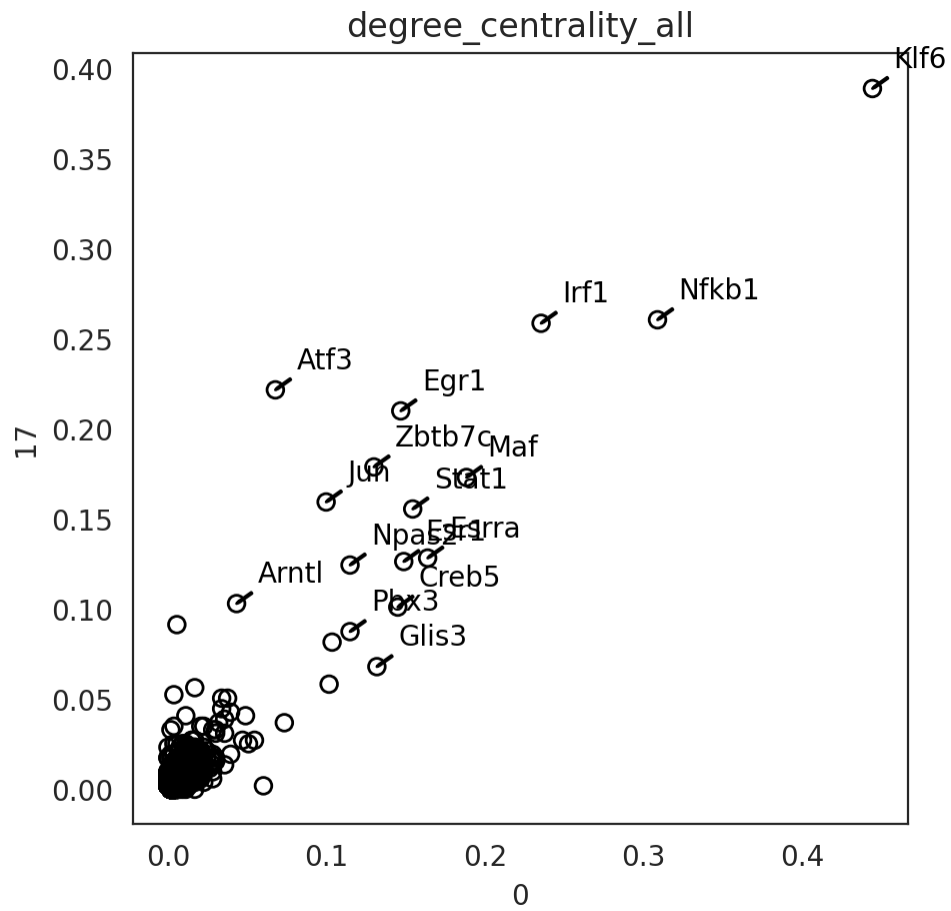
Processing clusters 0 and 15



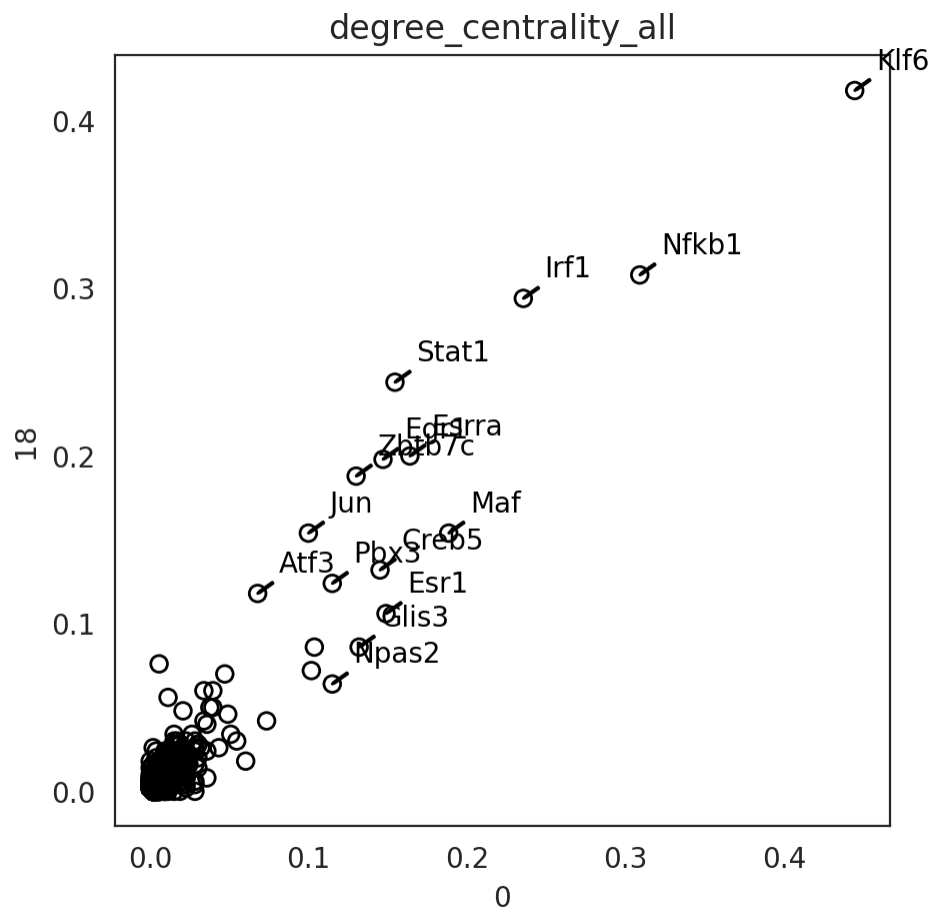
Processing clusters 0 and 16



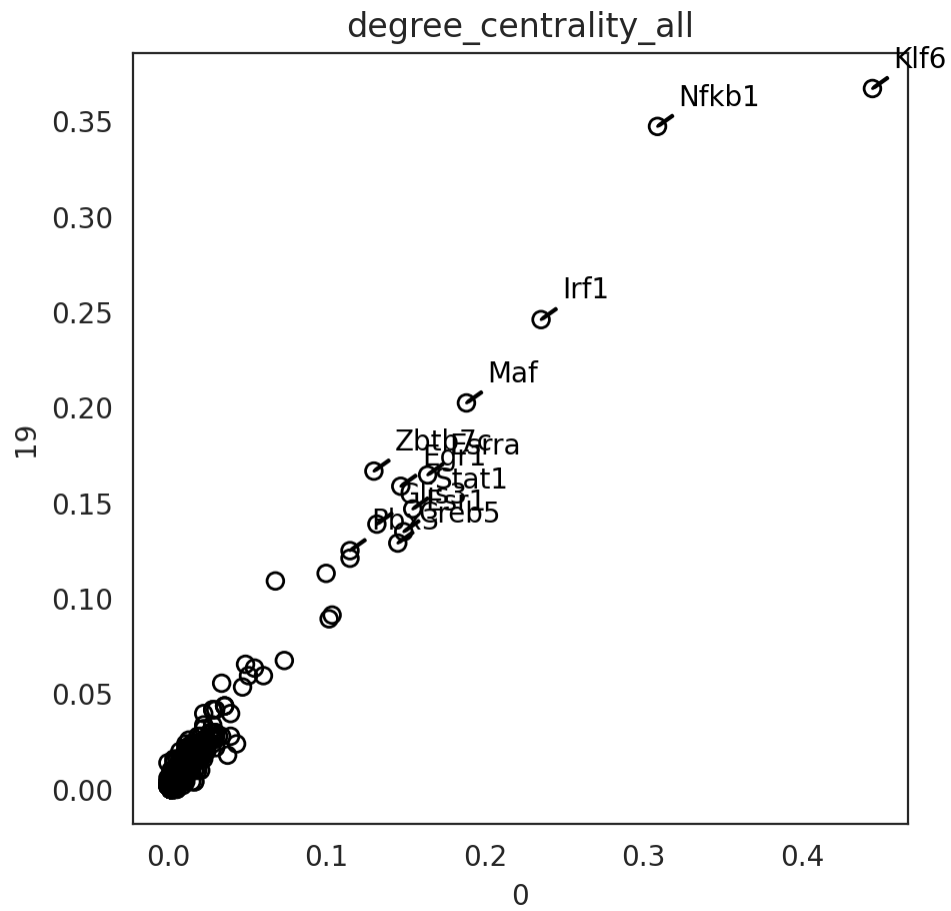
Processing clusters 0 and 17



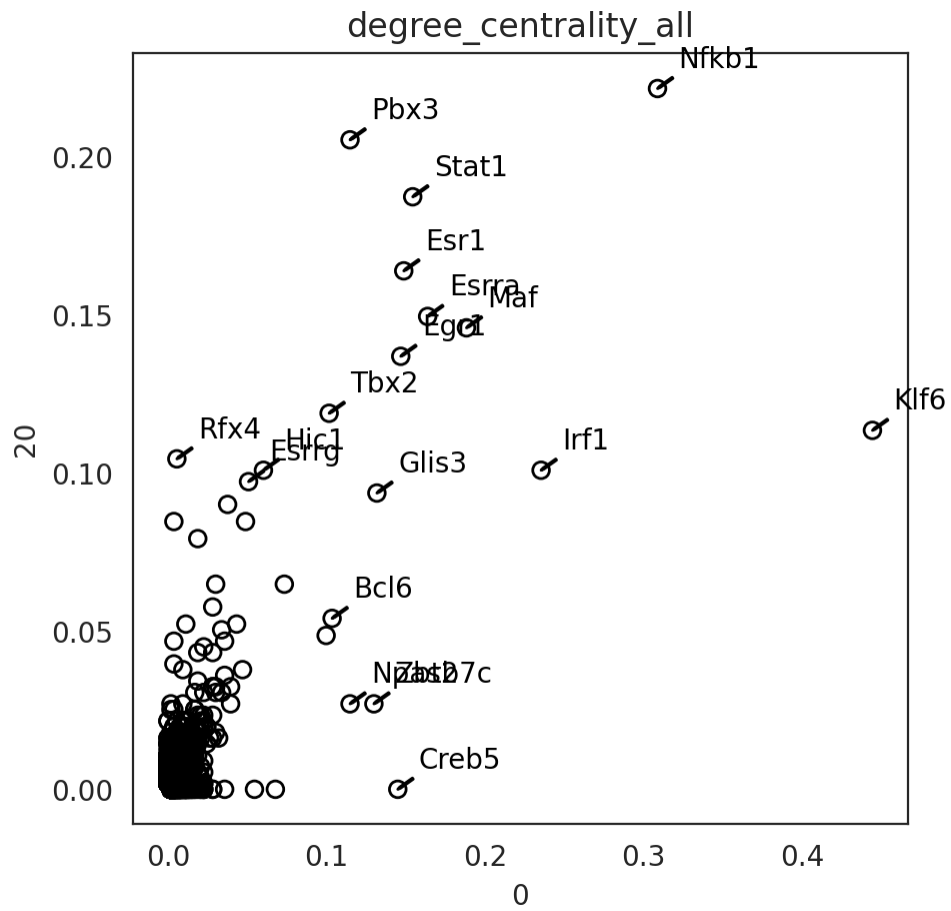
Processing clusters 0 and 18



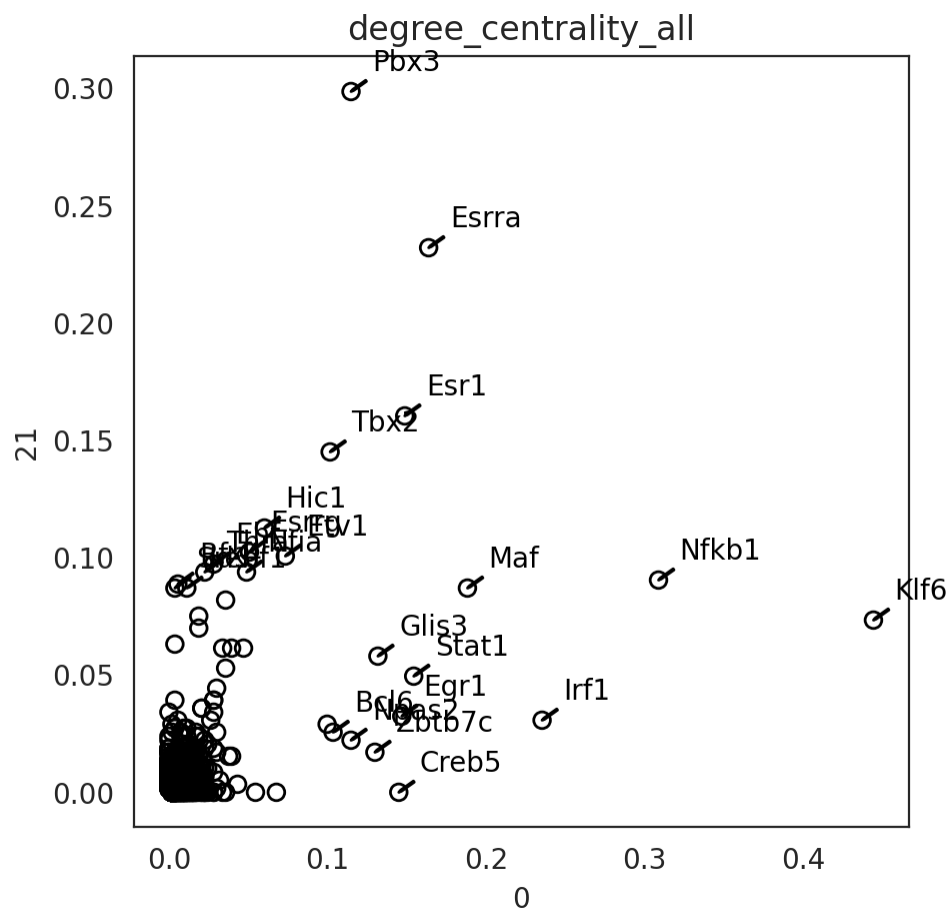
Processing clusters 0 and 19



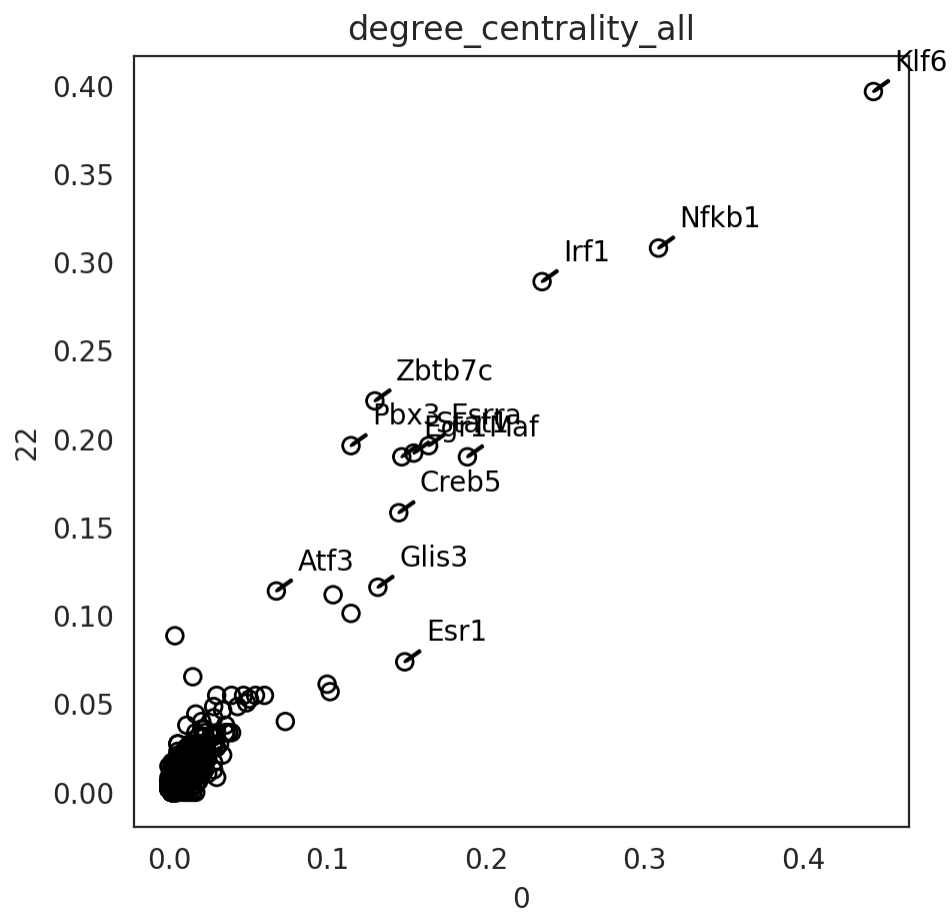
Processing clusters 0 and 20

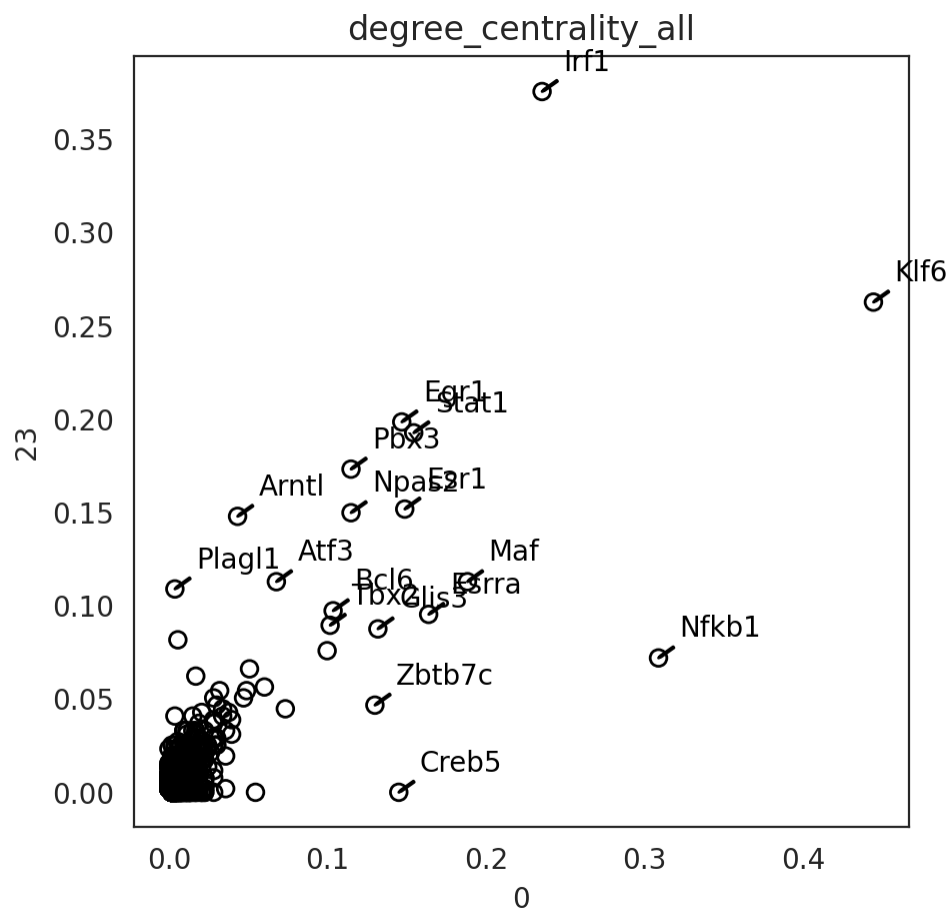


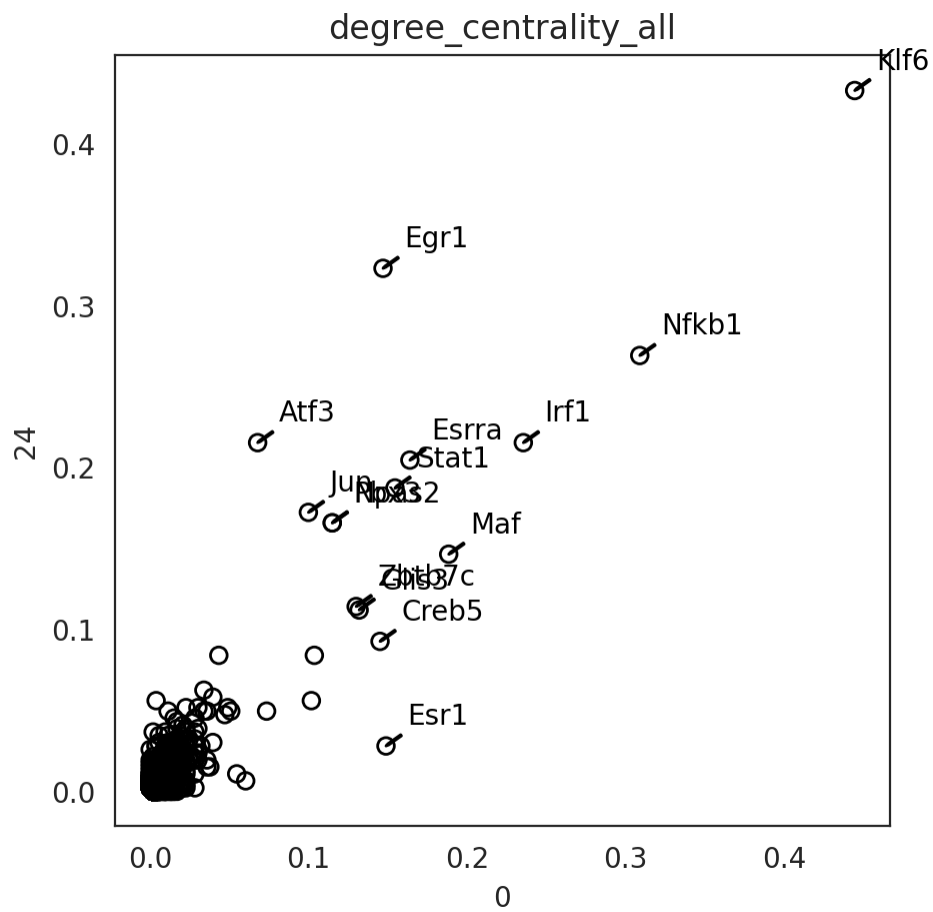
Processing clusters 0 and 21



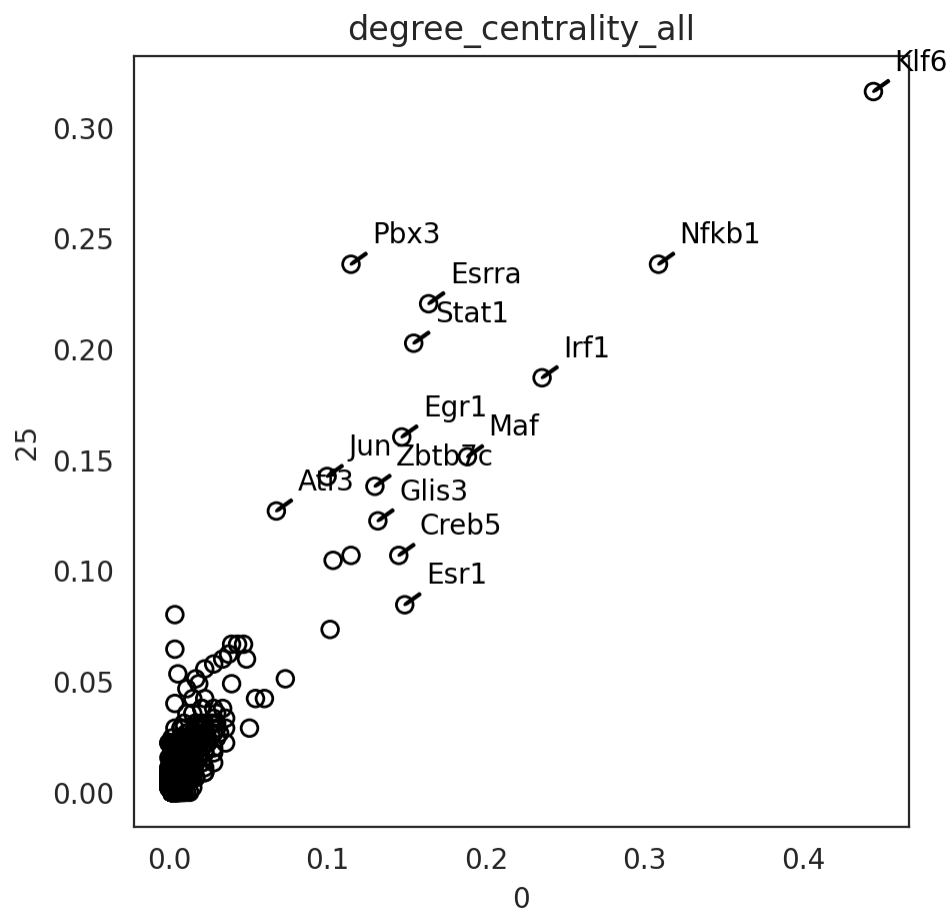
Processing clusters 0 and 22



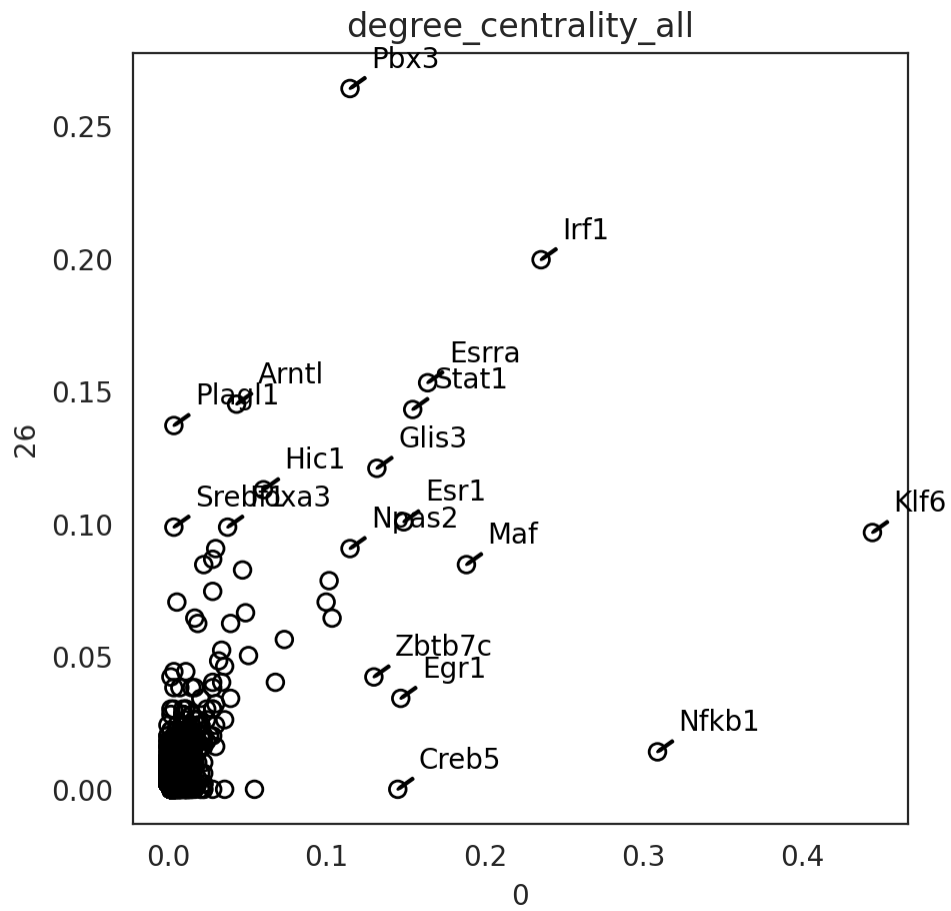




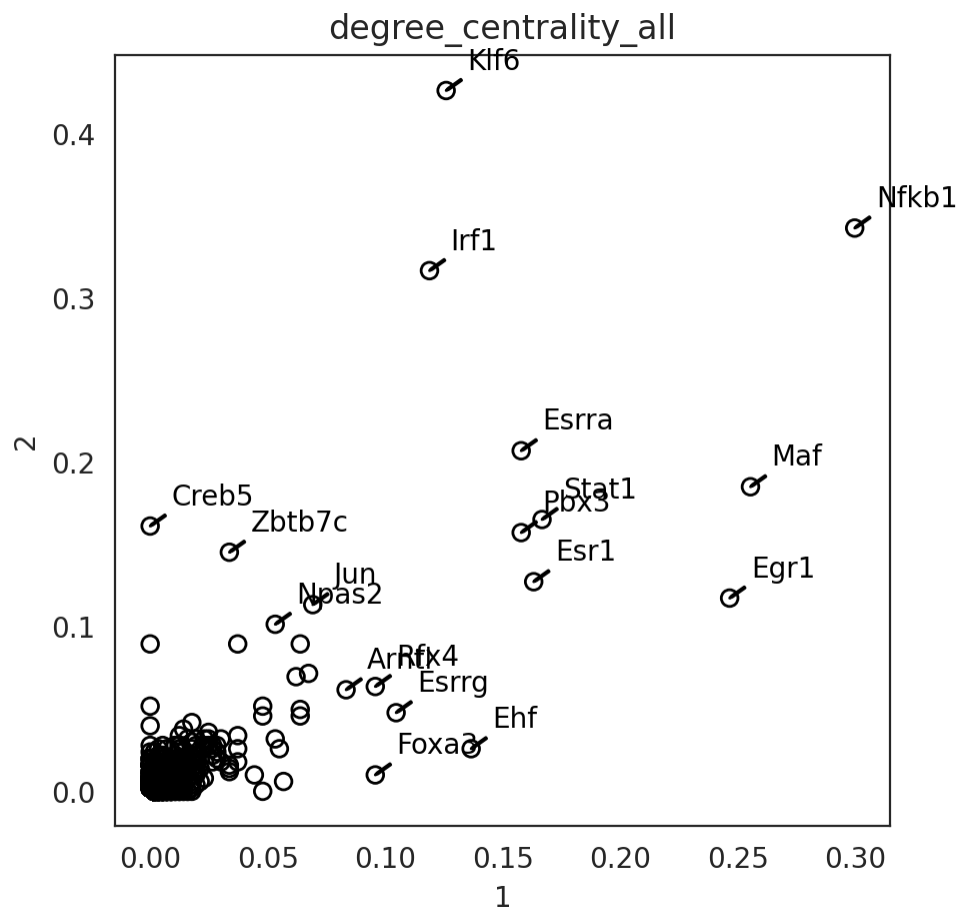
Processing clusters 0 and 25



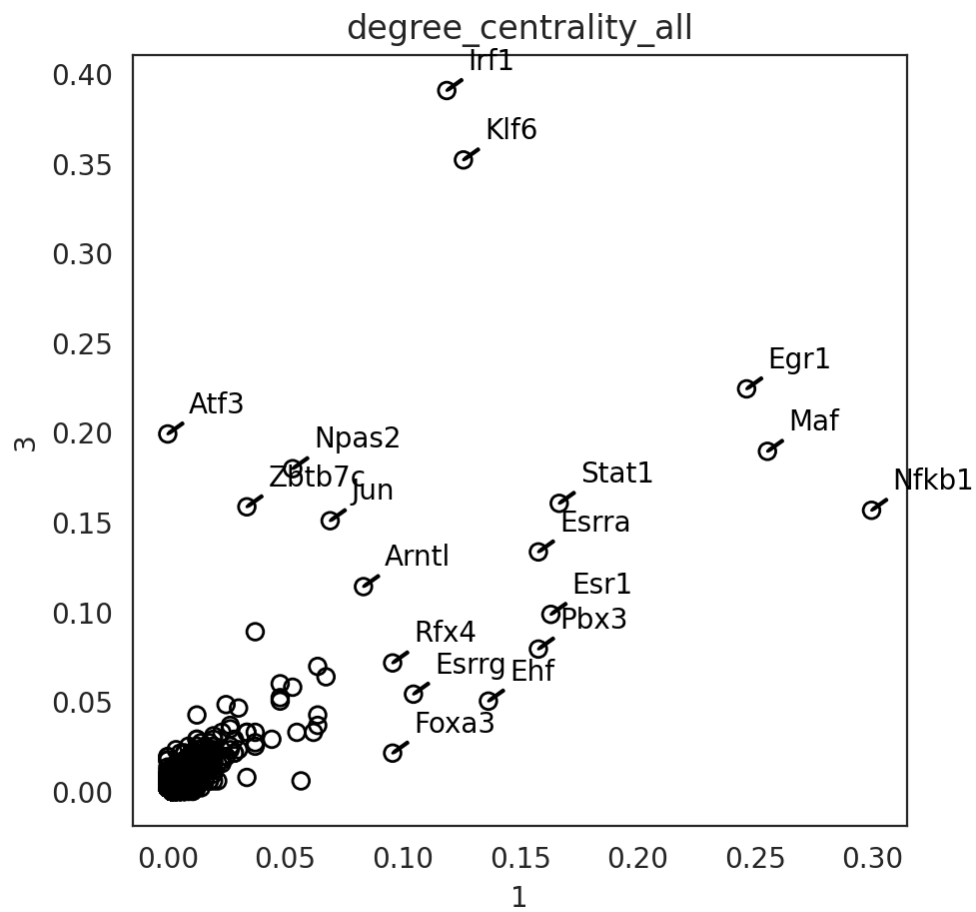
Processing clusters 0 and 26



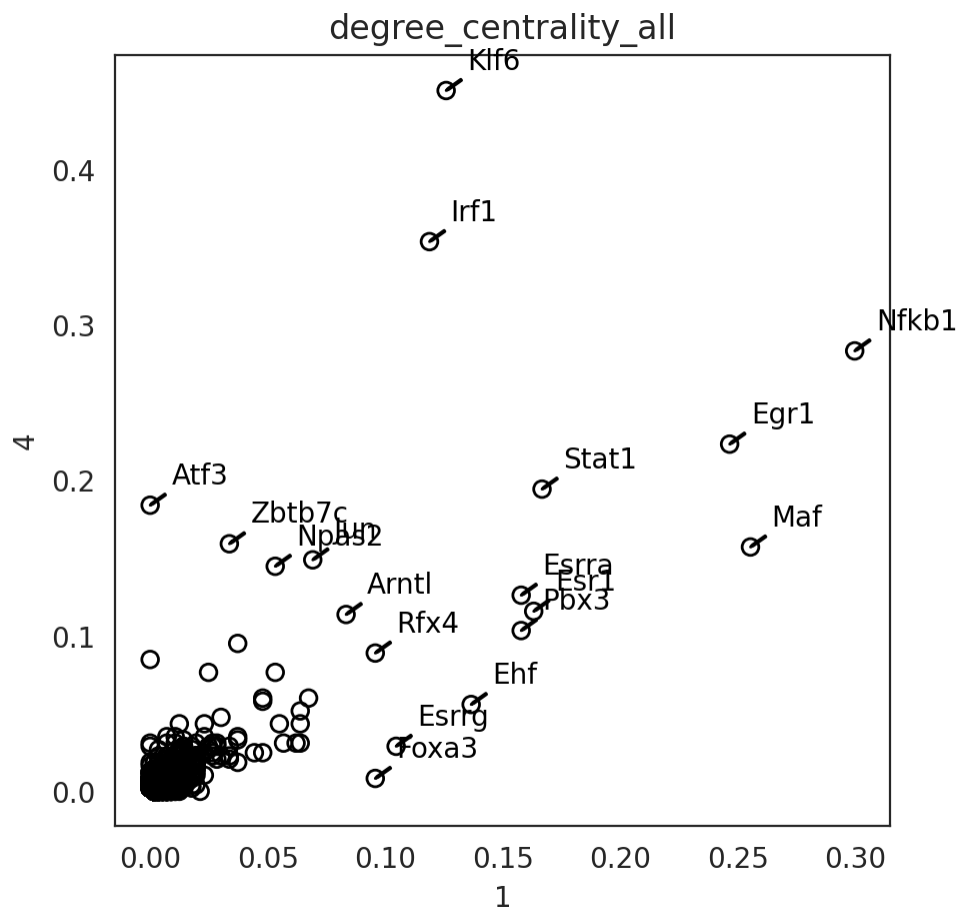
Processing clusters 1 and 2



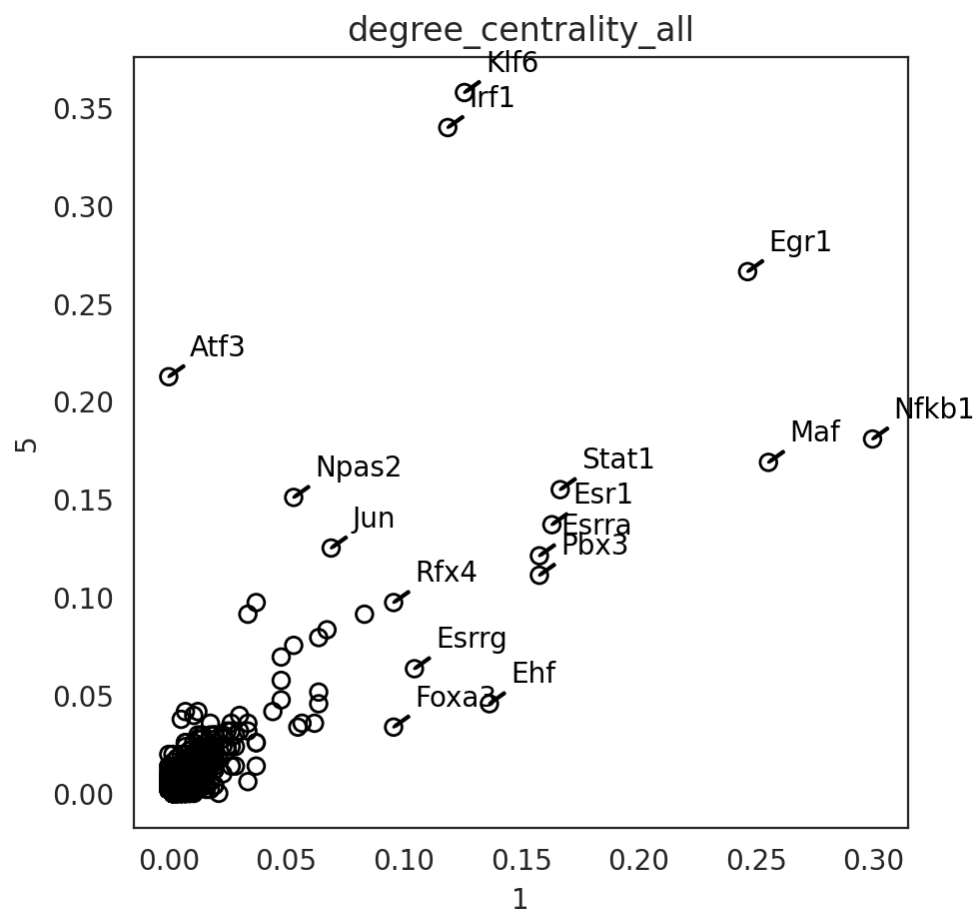
Processing clusters 1 and 3



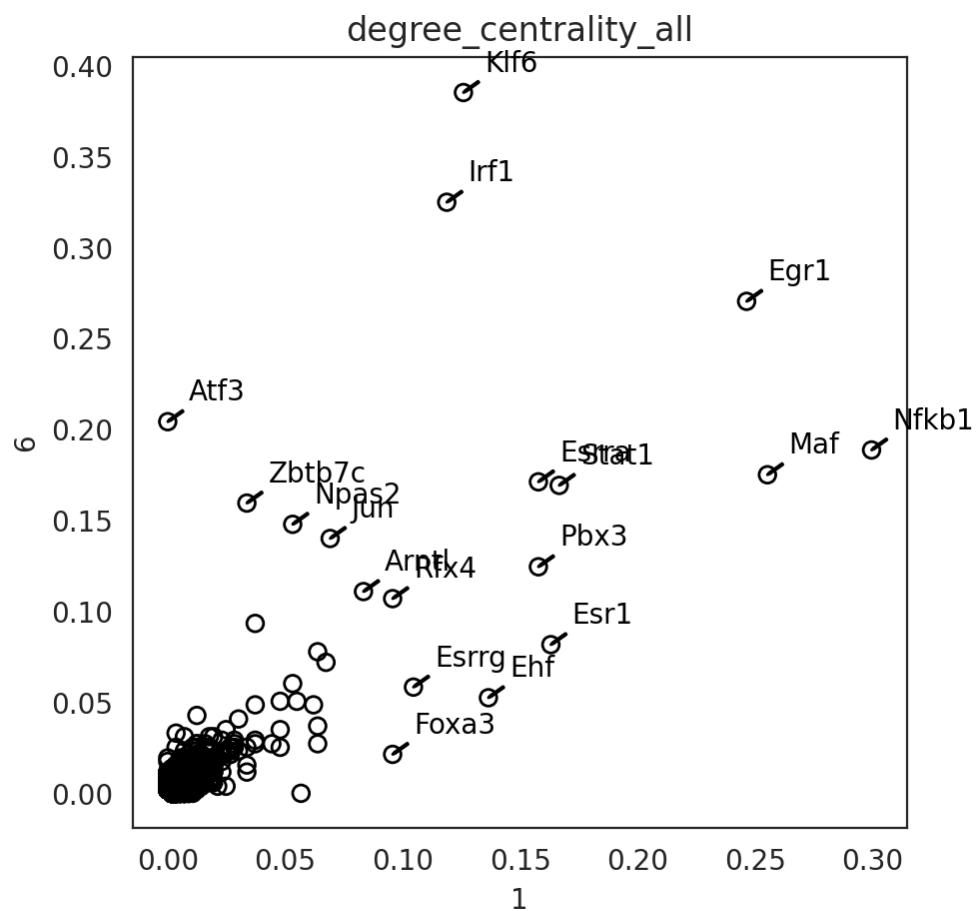
Processing clusters 1 and 4



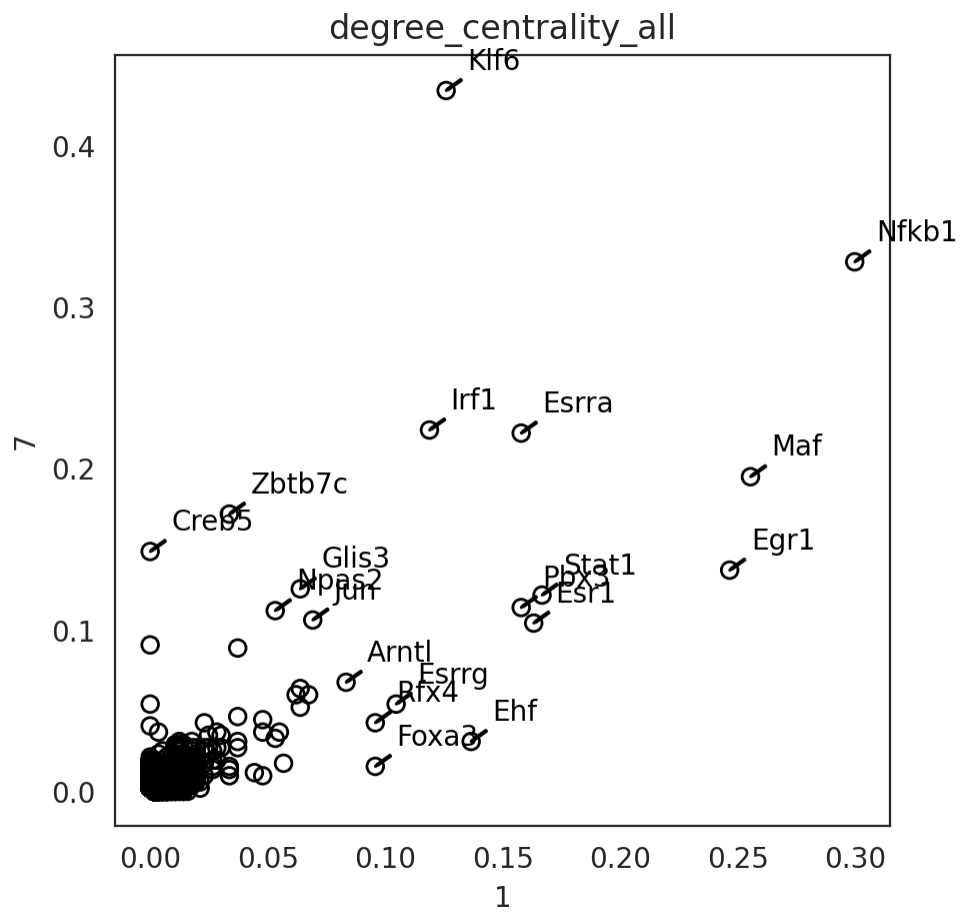
Processing clusters 1 and 5



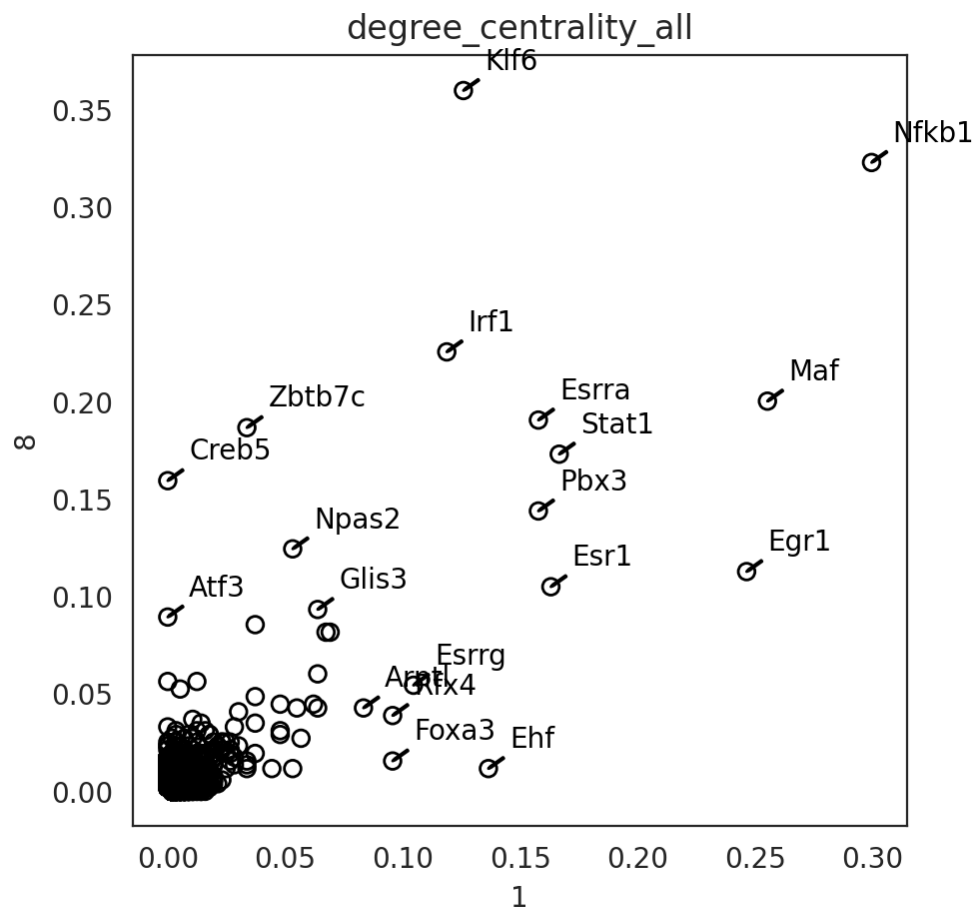
Processing clusters 1 and 6



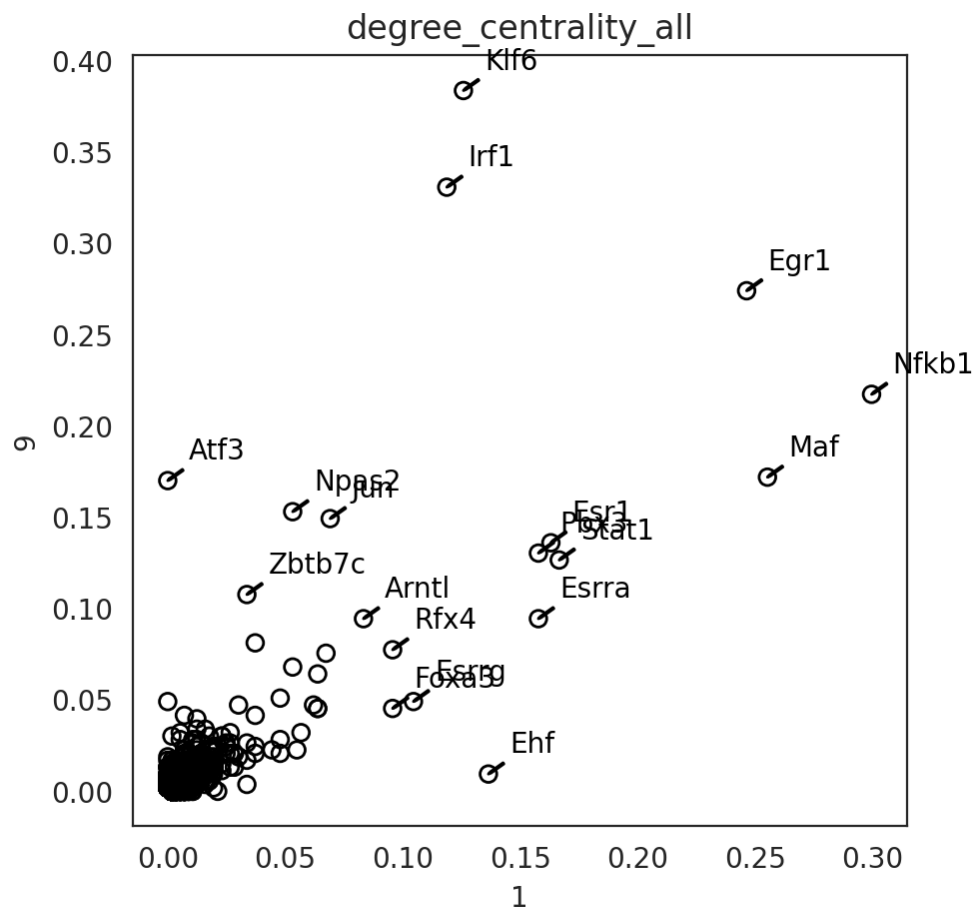
Processing clusters 1 and 7



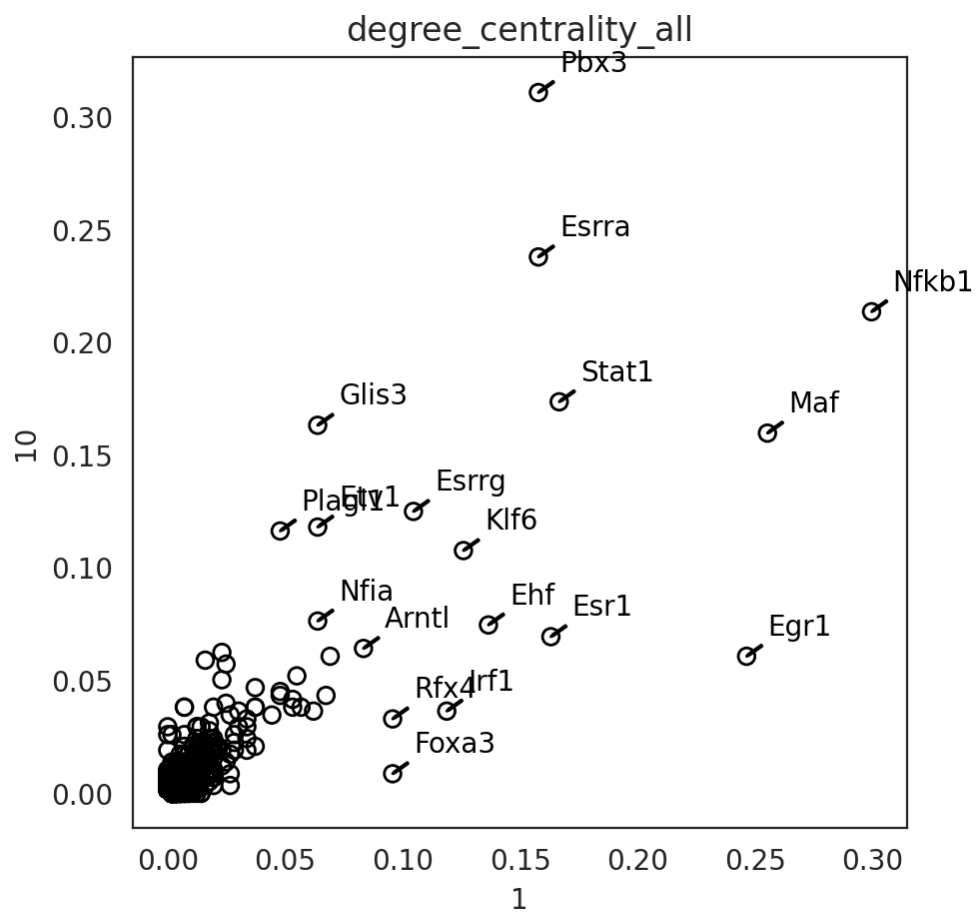
Processing clusters 1 and 8



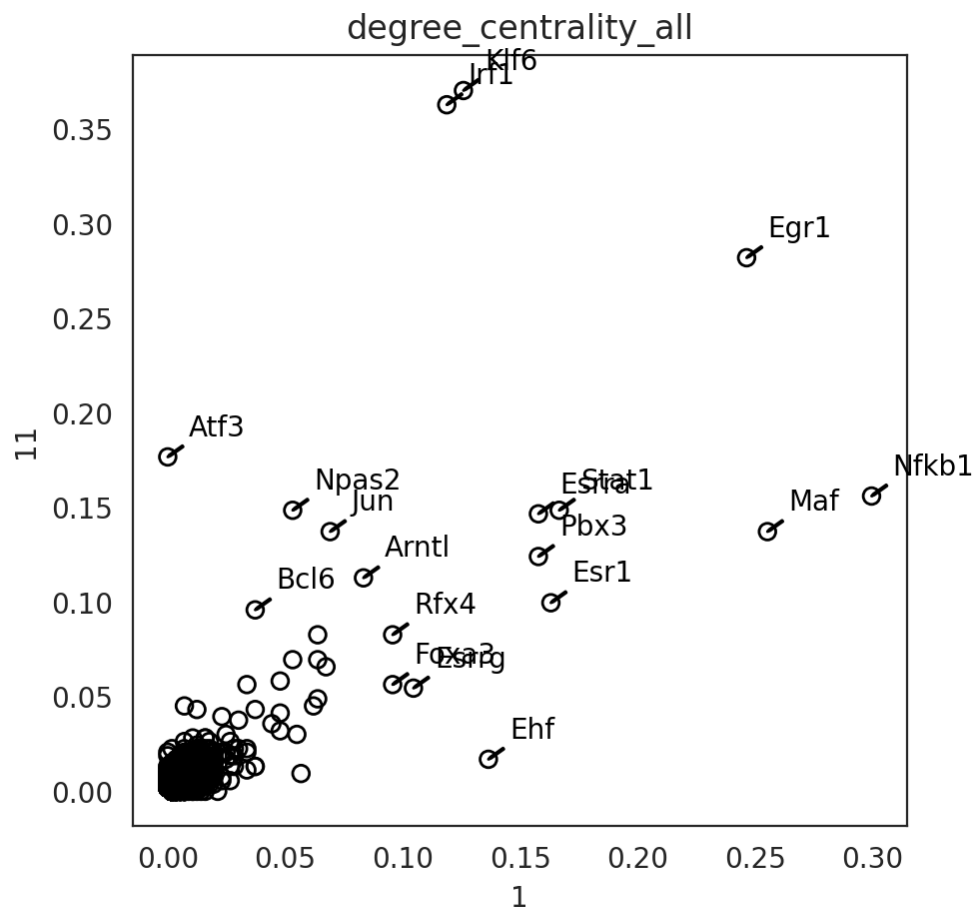
Processing clusters 1 and 9



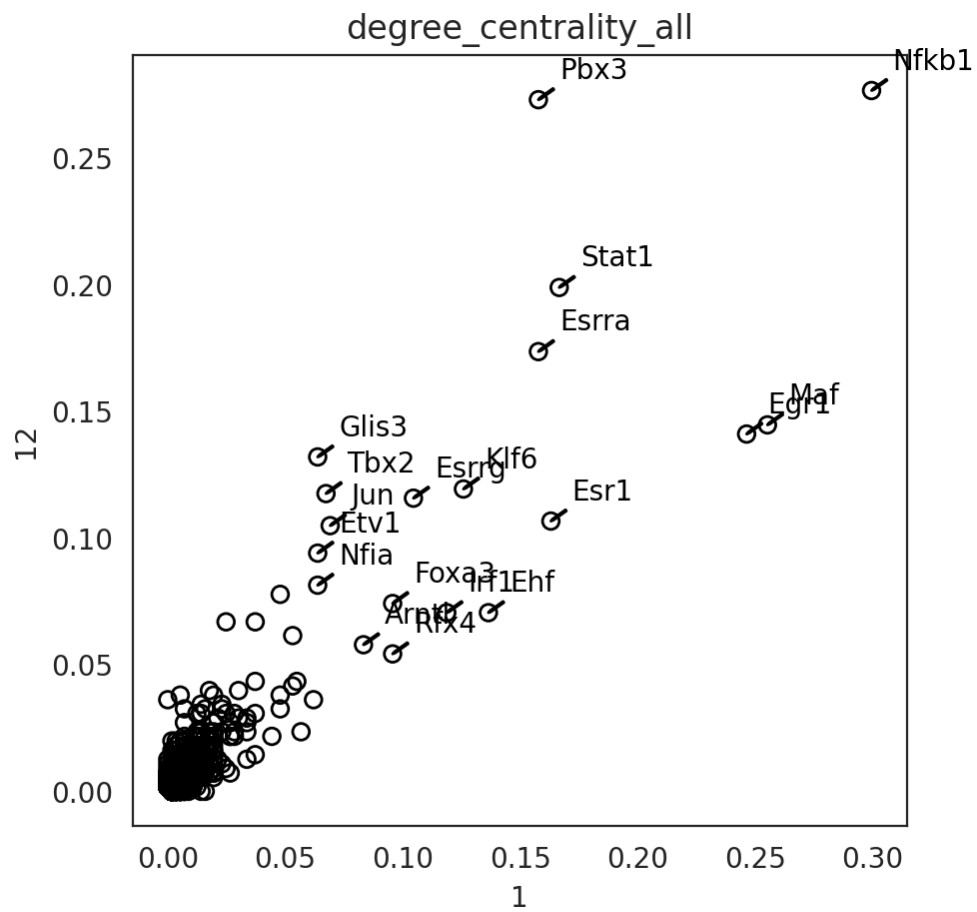
Processing clusters 1 and 10



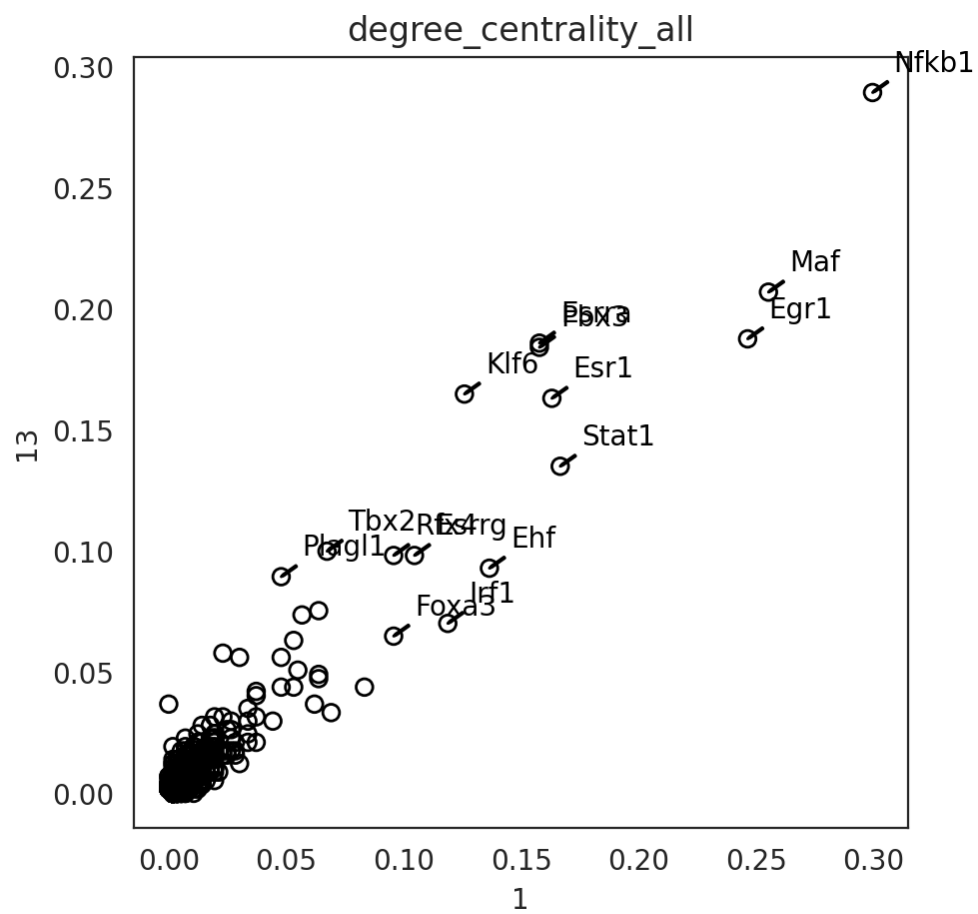
Processing clusters 1 and 11



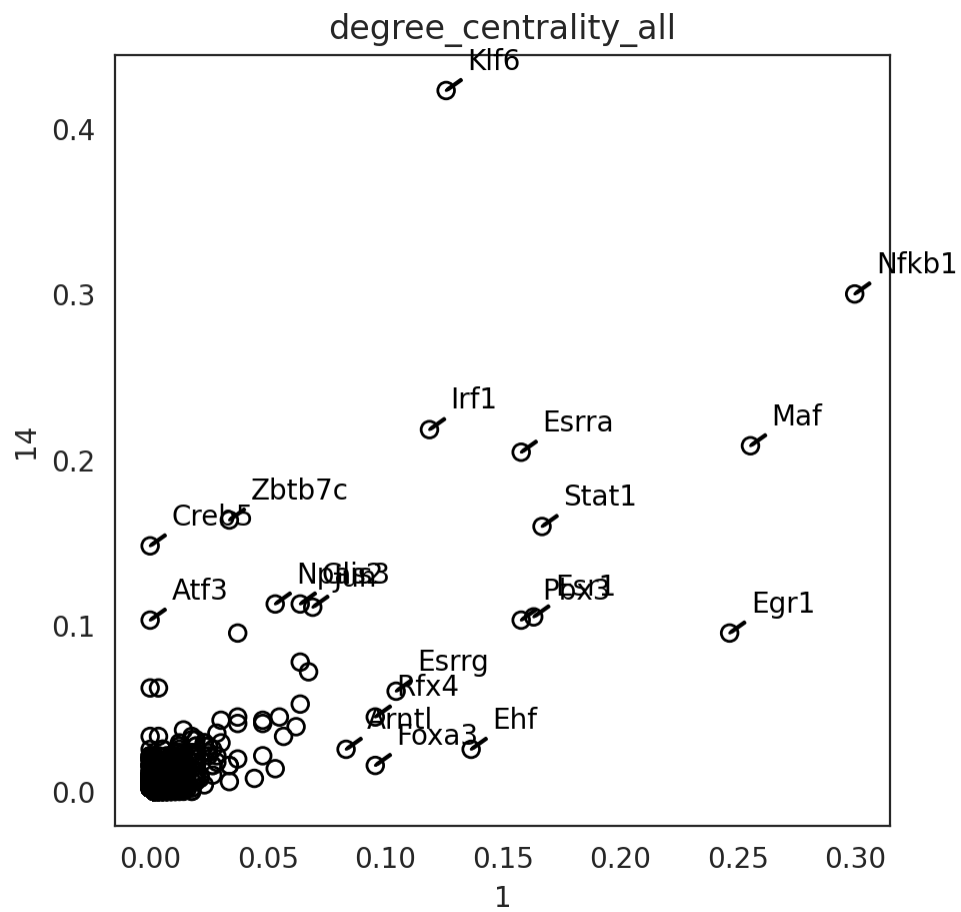
Processing clusters 1 and 12



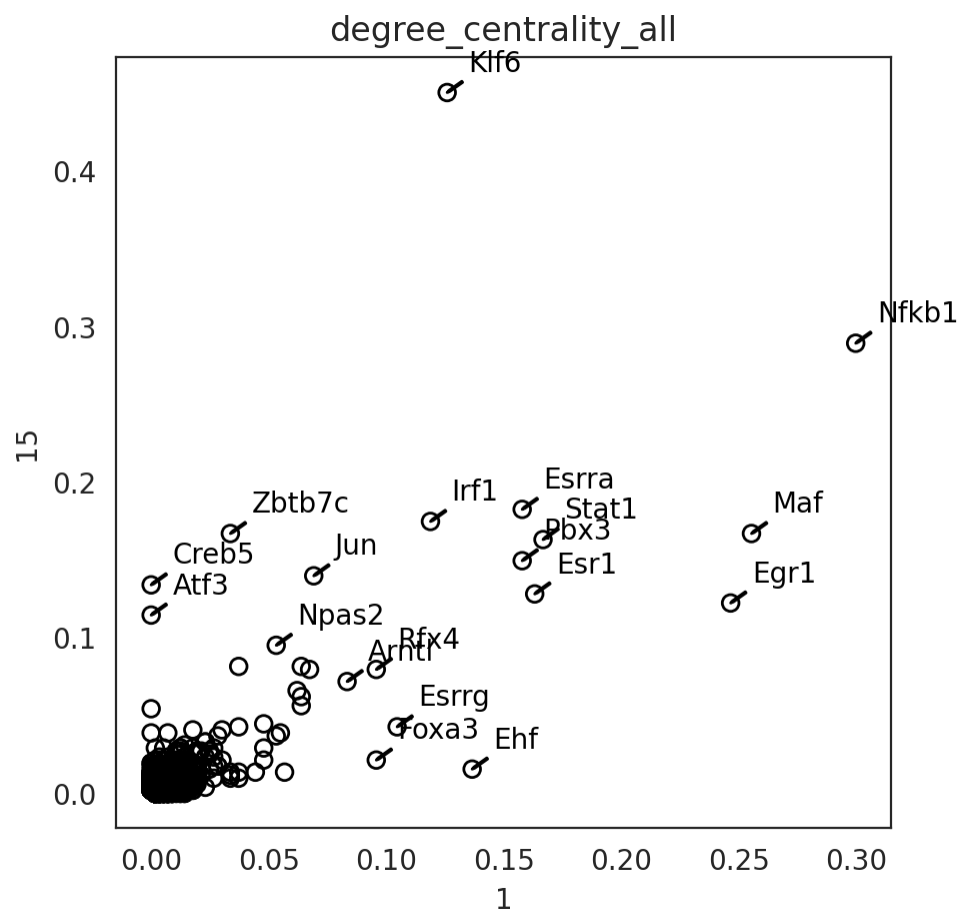
Processing clusters 1 and 13



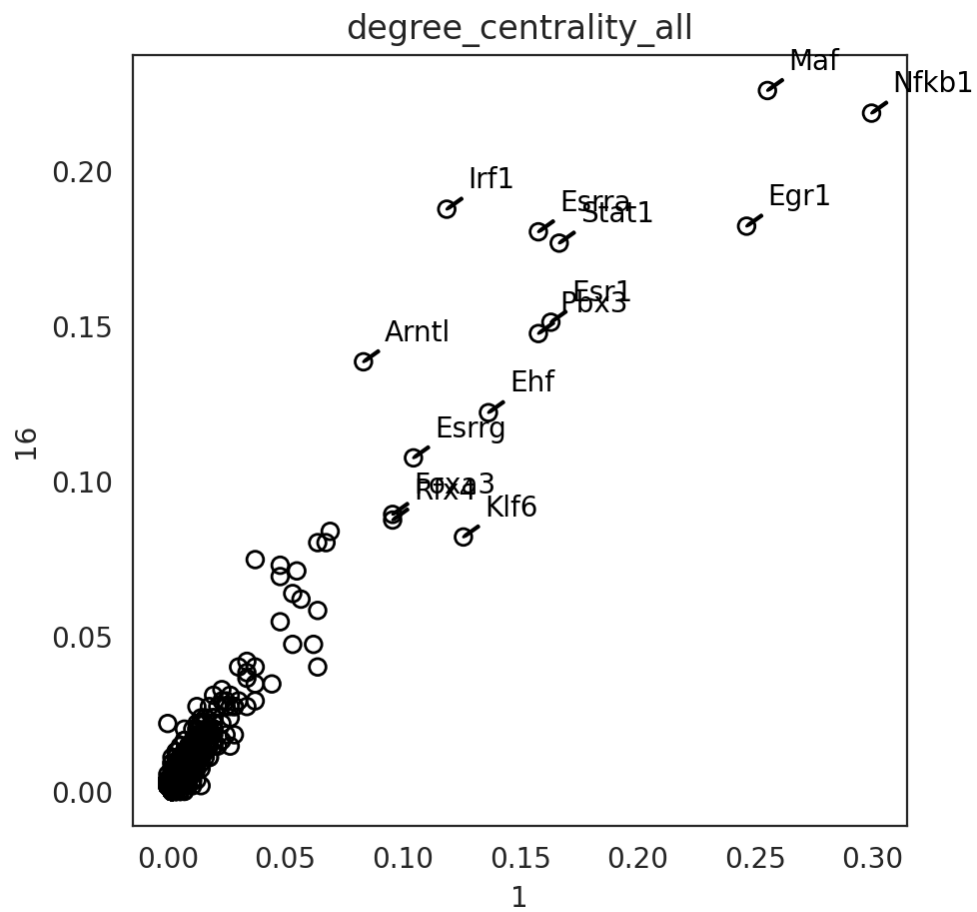
Processing clusters 1 and 14



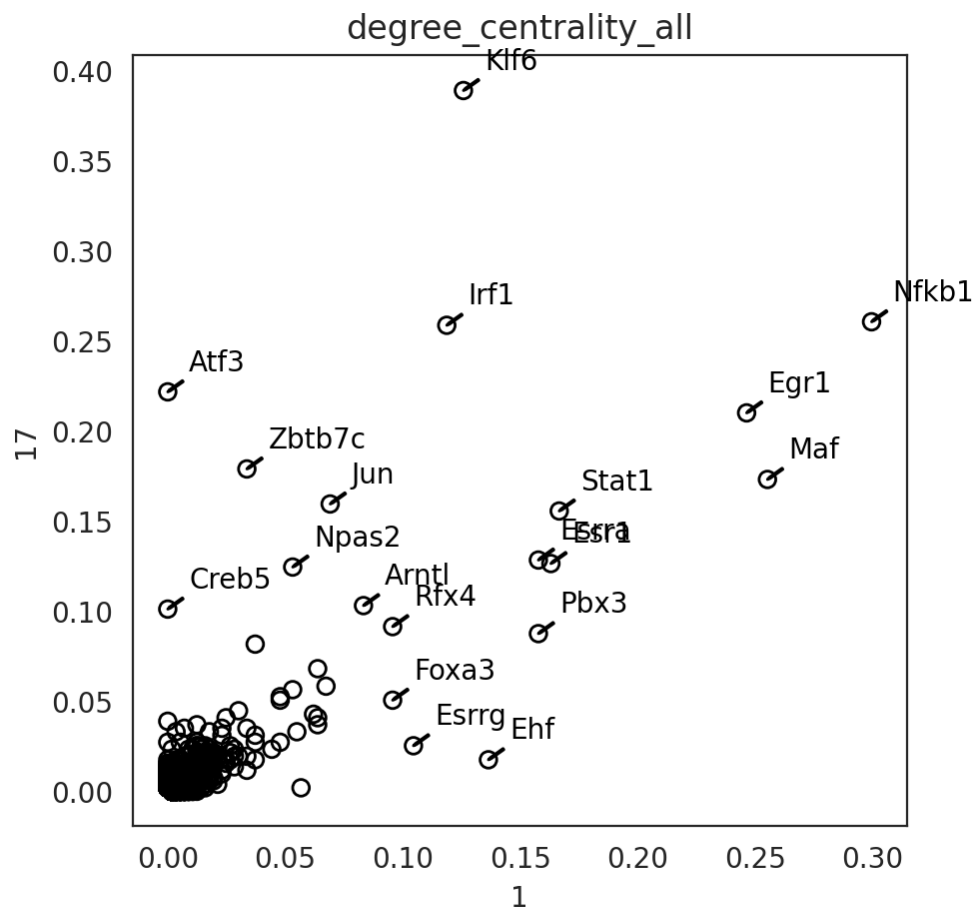
Processing clusters 1 and 15



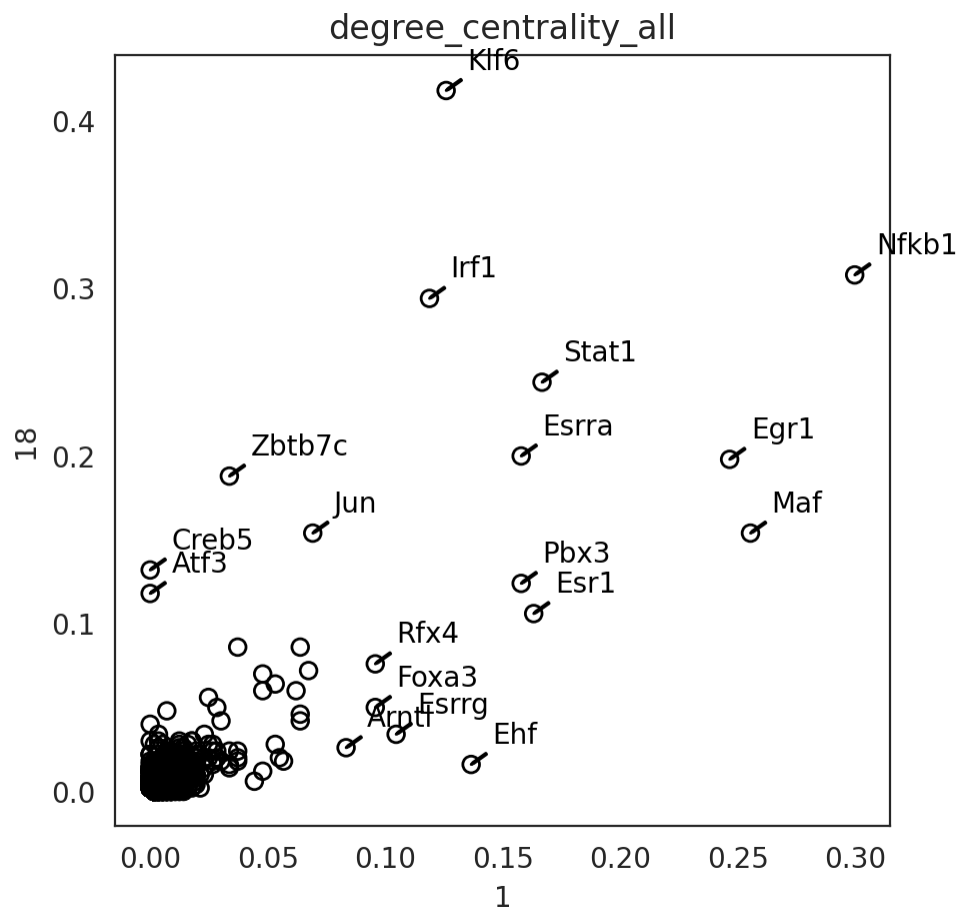
Processing clusters 1 and 16



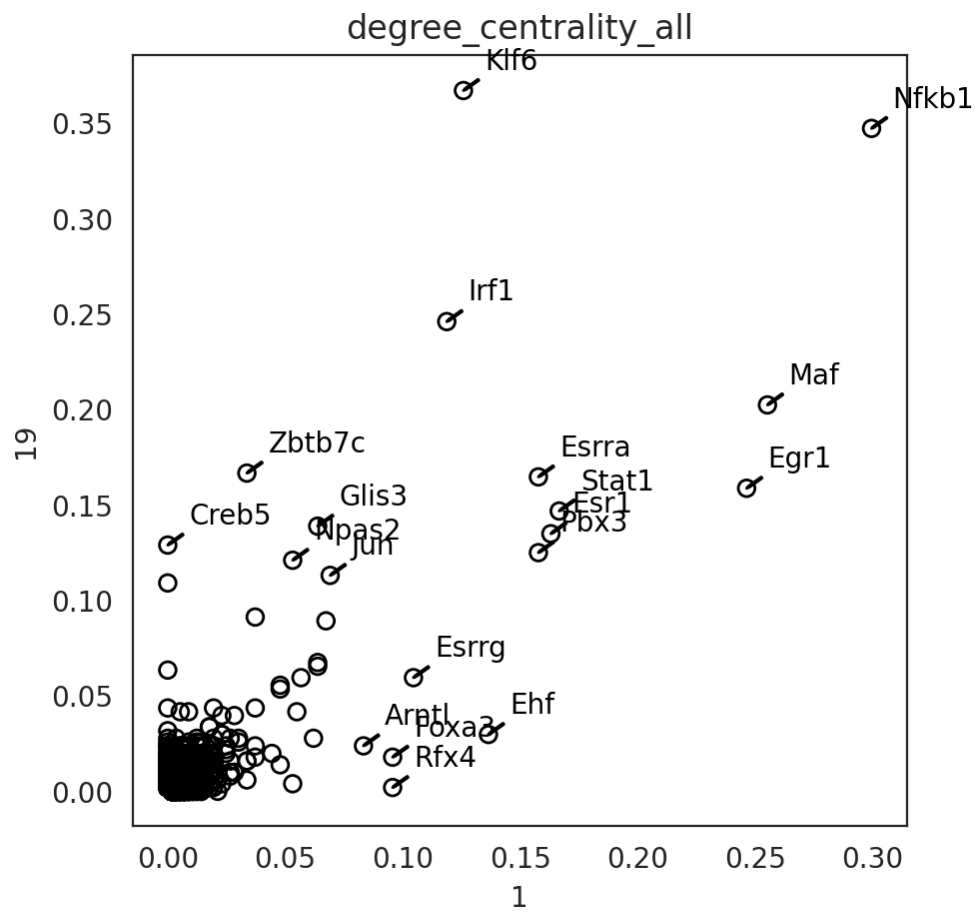
Processing clusters 1 and 17



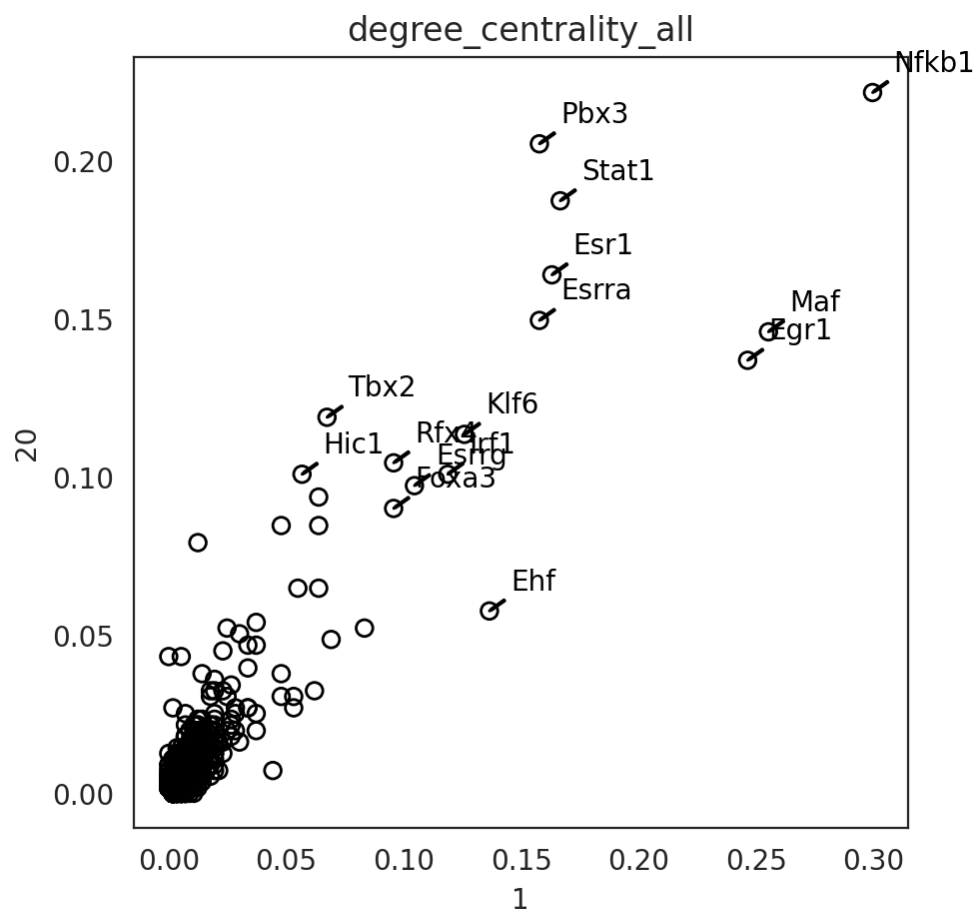
Processing clusters 1 and 18



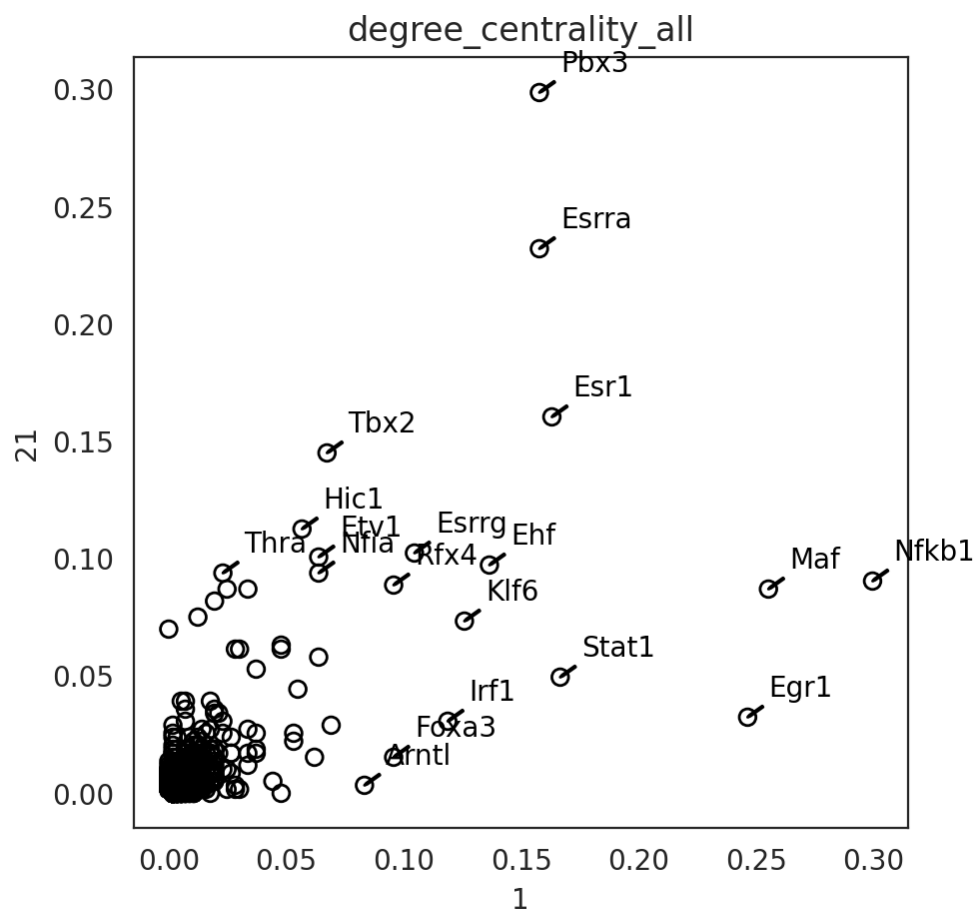
Processing clusters 1 and 19



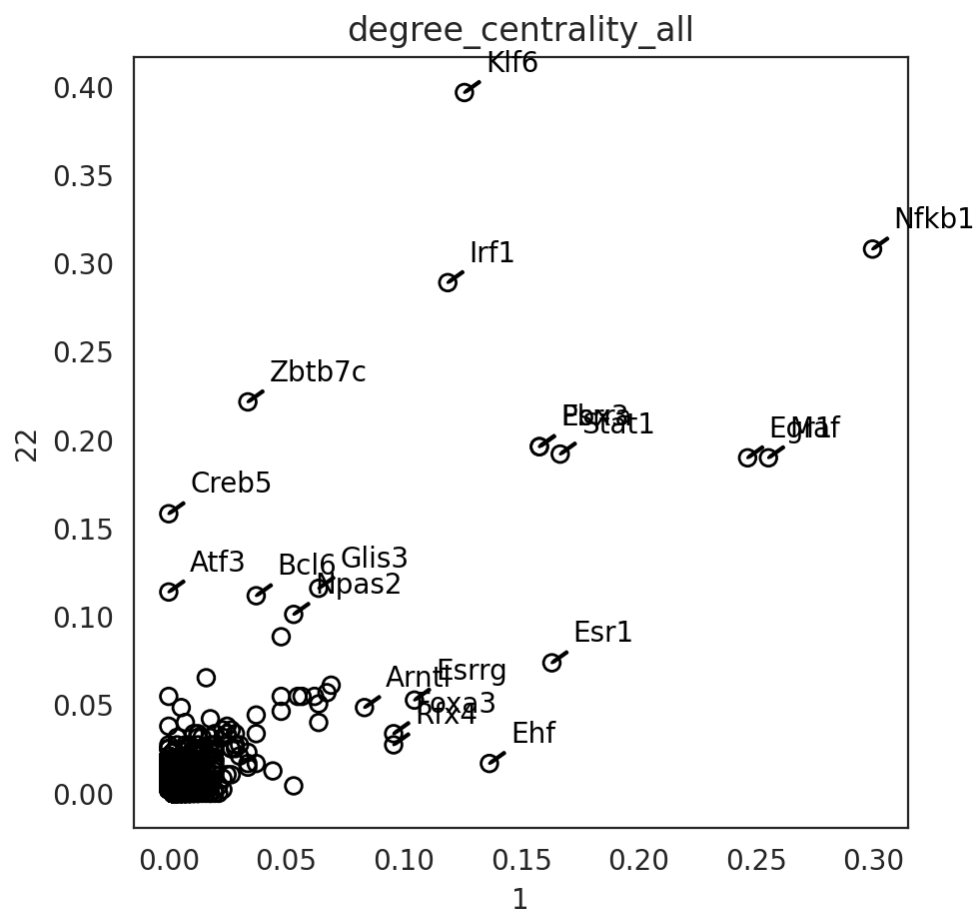
Processing clusters 1 and 20



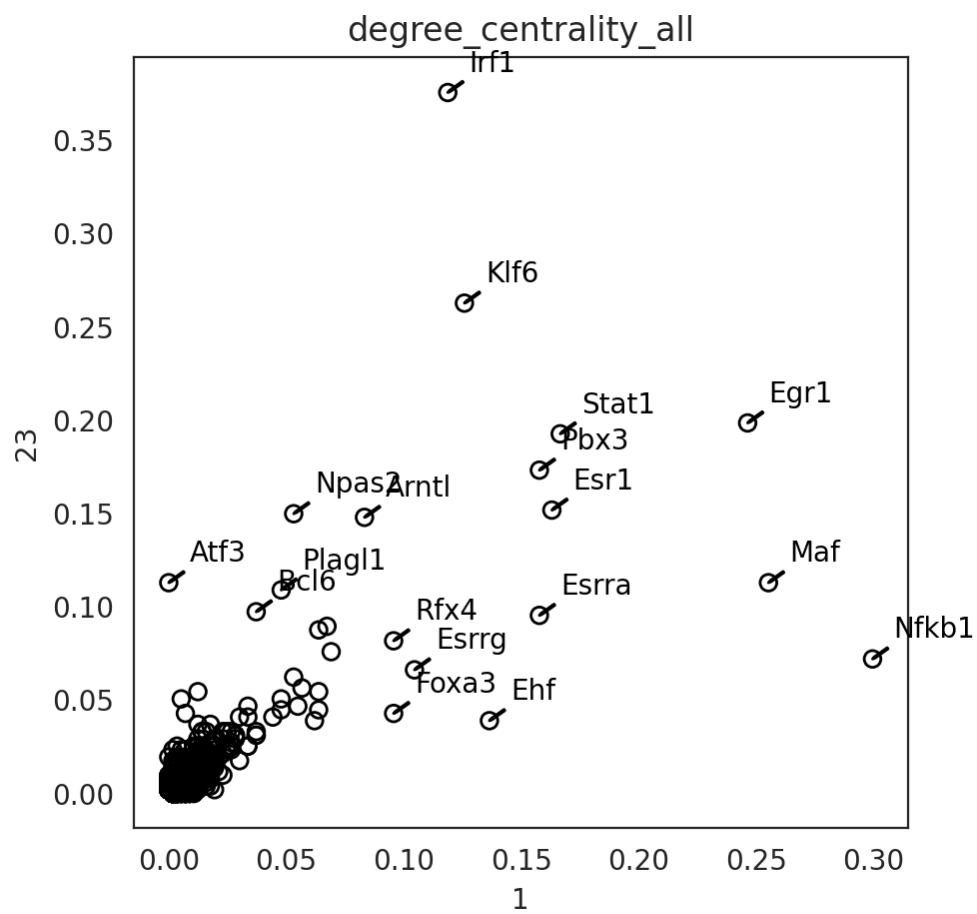
Processing clusters 1 and 21



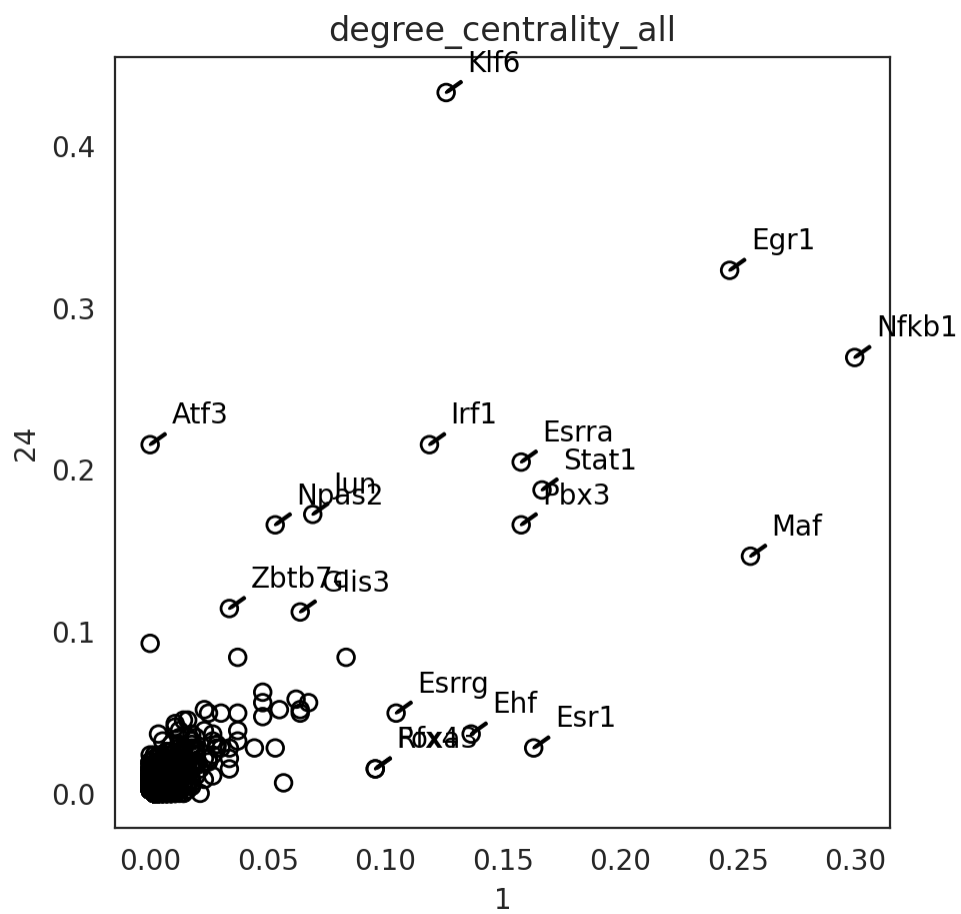
Processing clusters 1 and 22



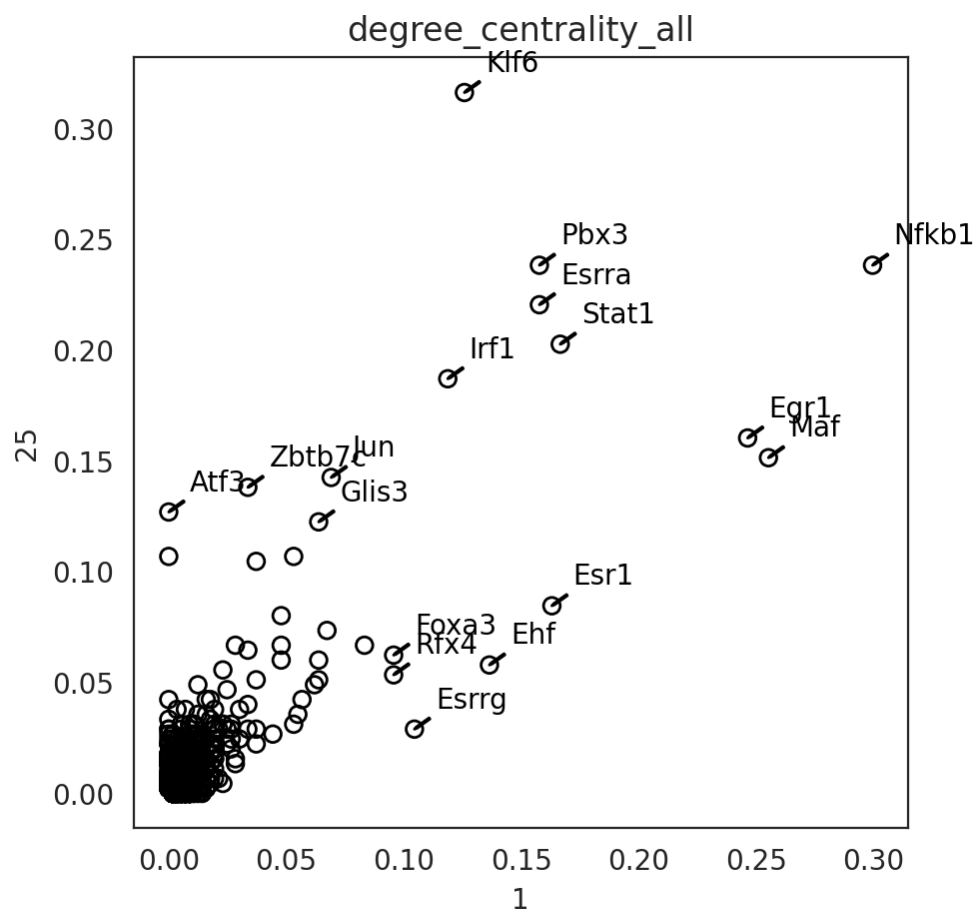
Processing clusters 1 and 23



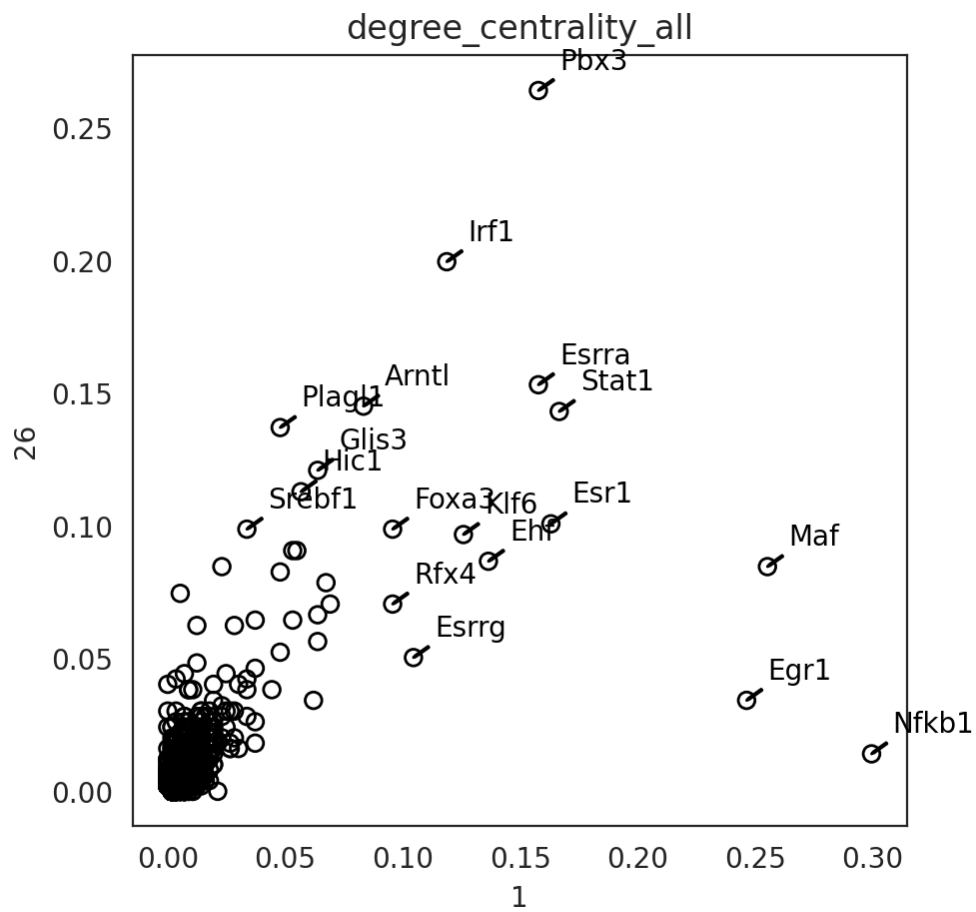
Processing clusters 1 and 24



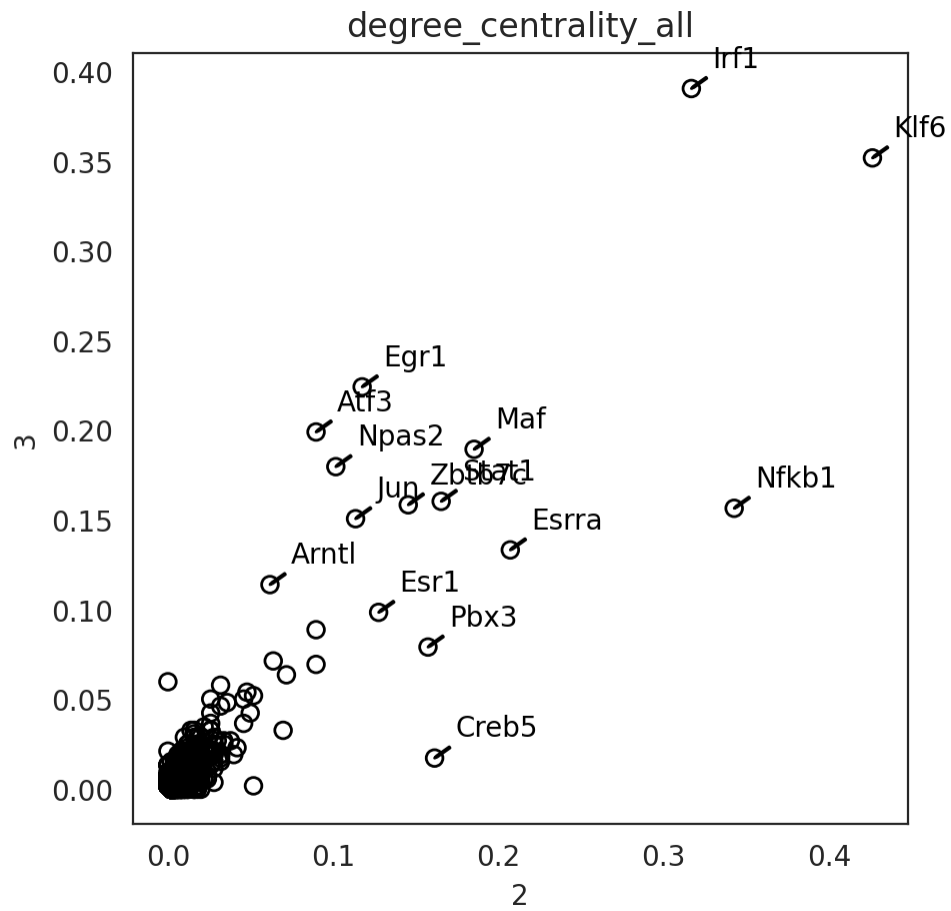
Processing clusters 1 and 25



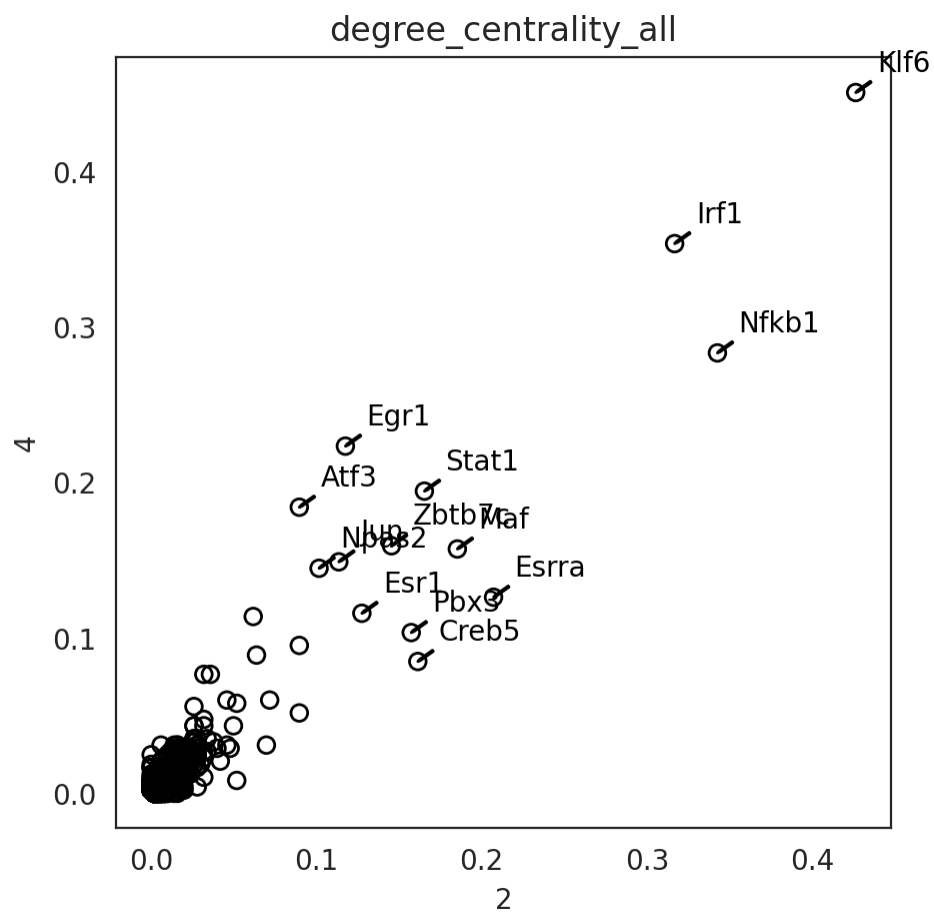
Processing clusters 1 and 26



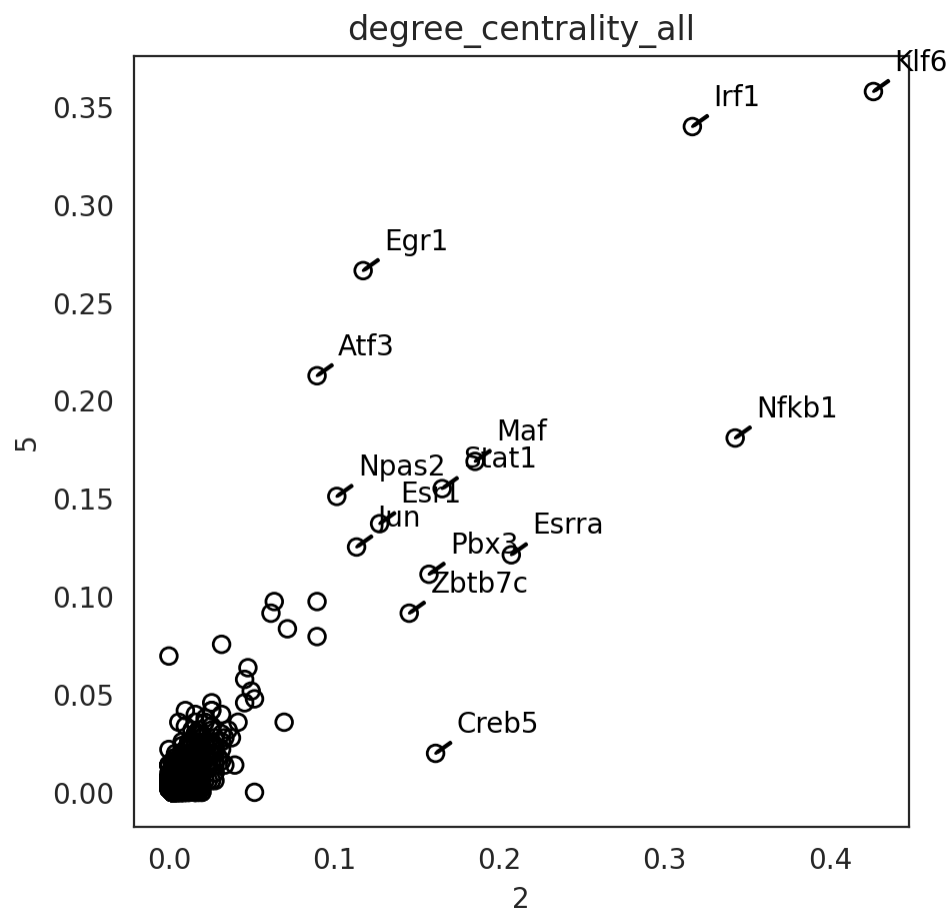
Processing clusters 2 and 3



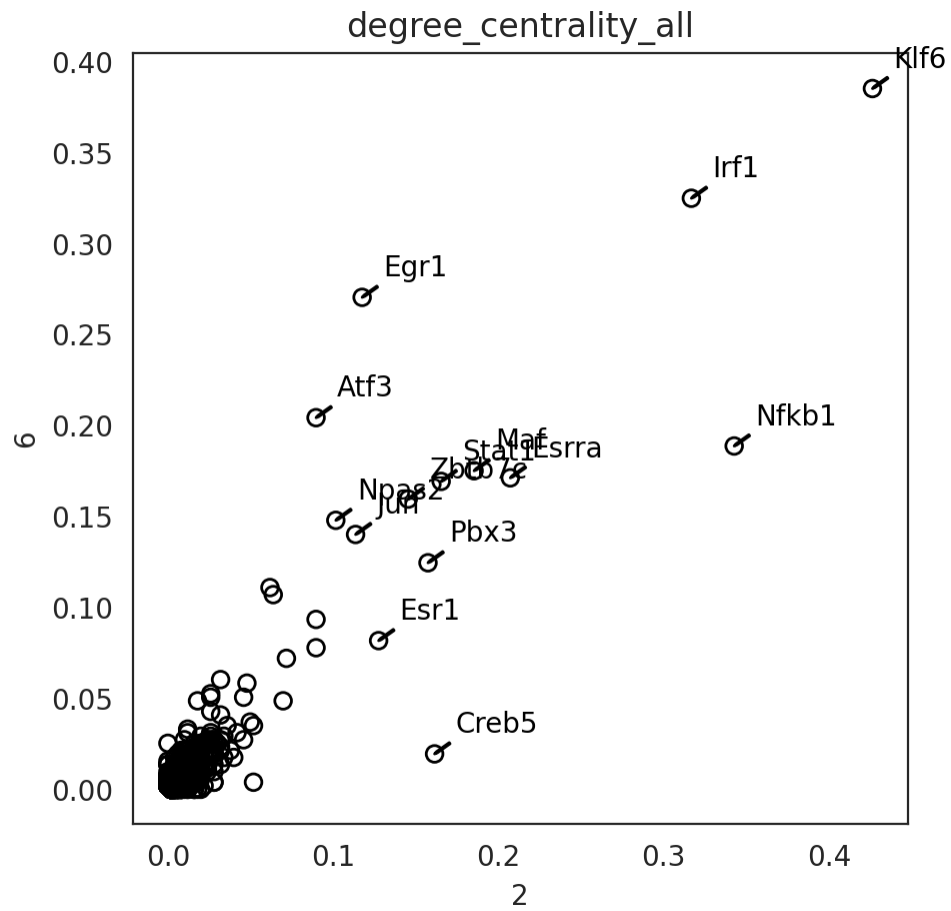
Processing clusters 2 and 4



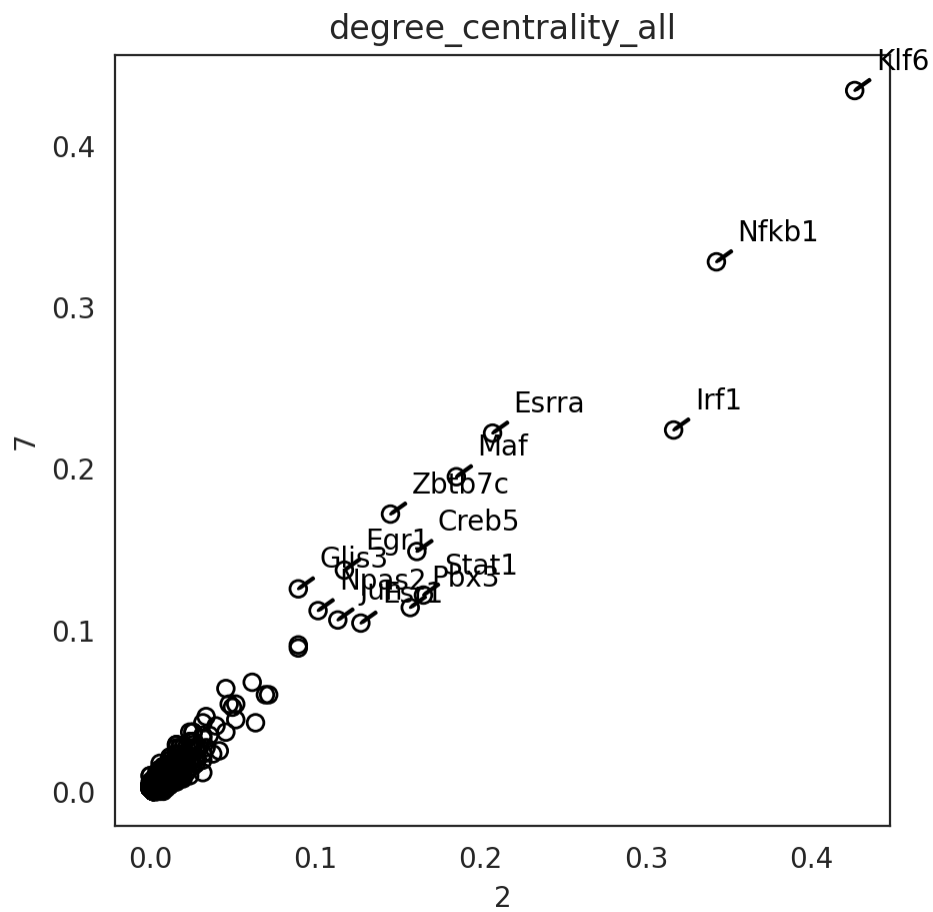
Processing clusters 2 and 5



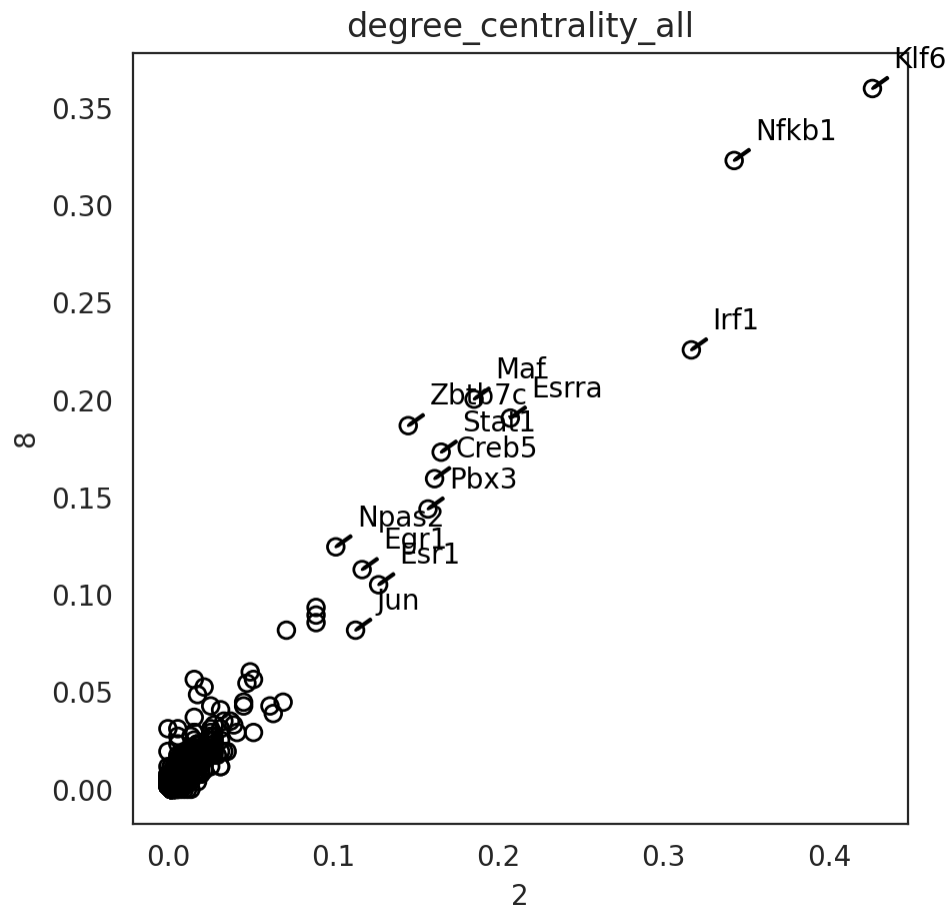
Processing clusters 2 and 6



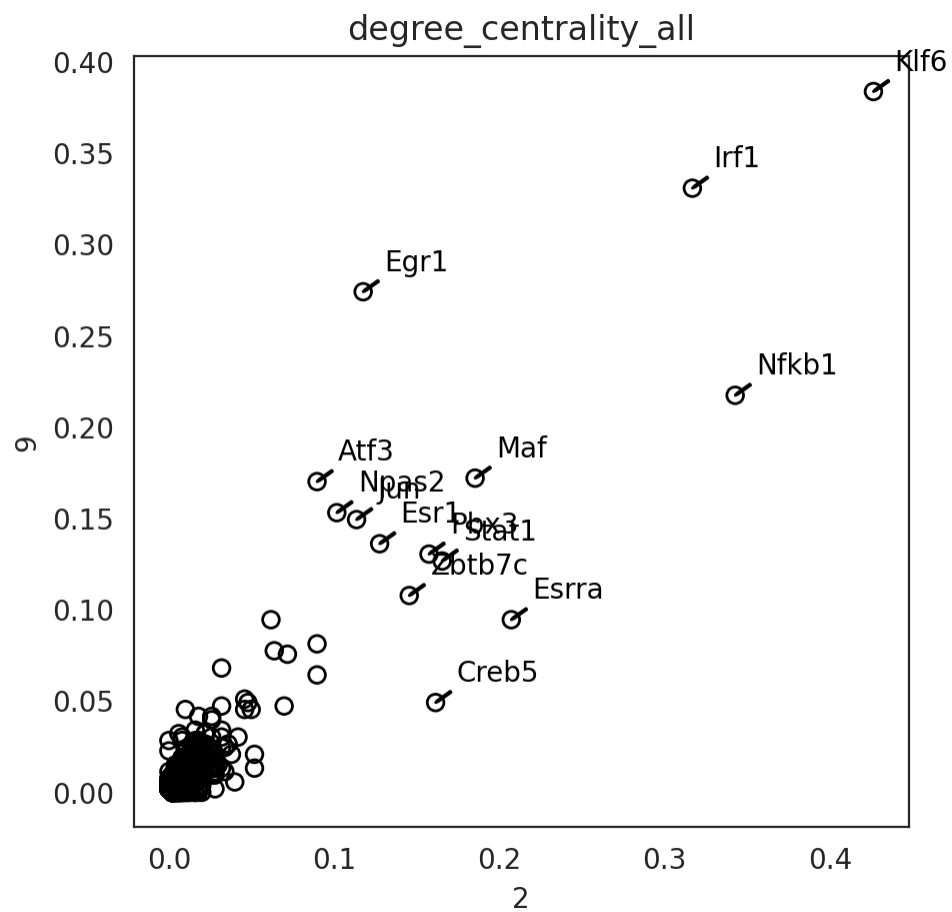
Processing clusters 2 and 7



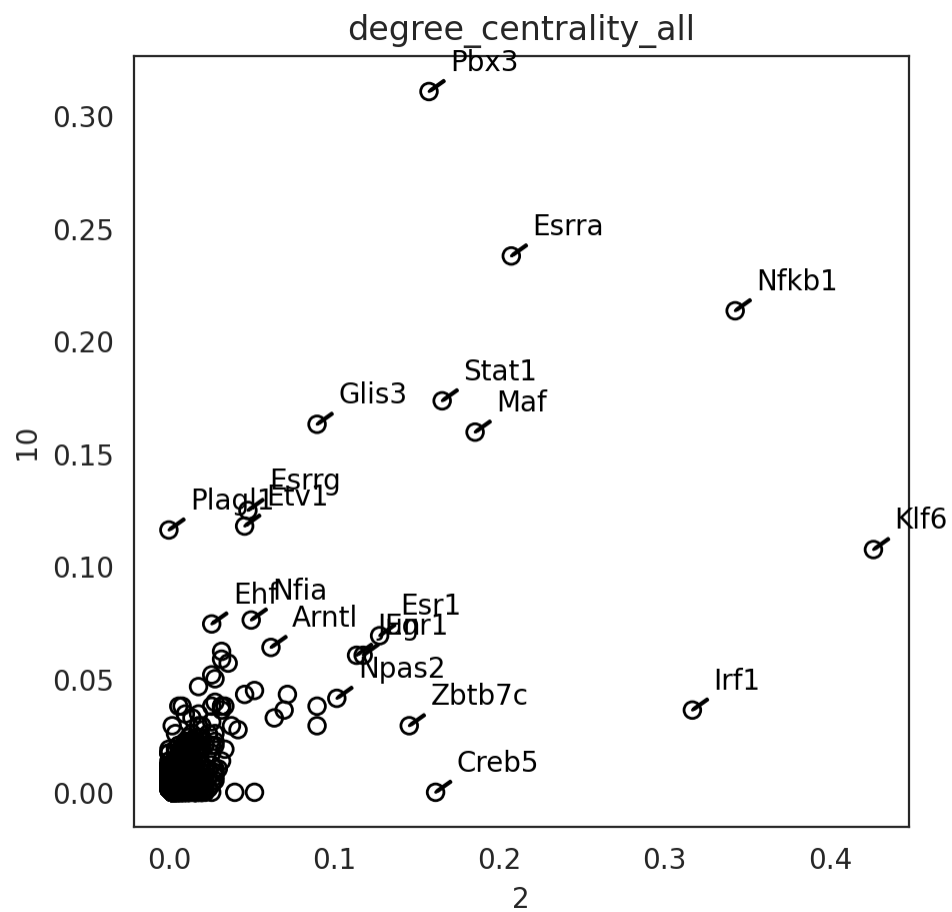
Processing clusters 2 and 8



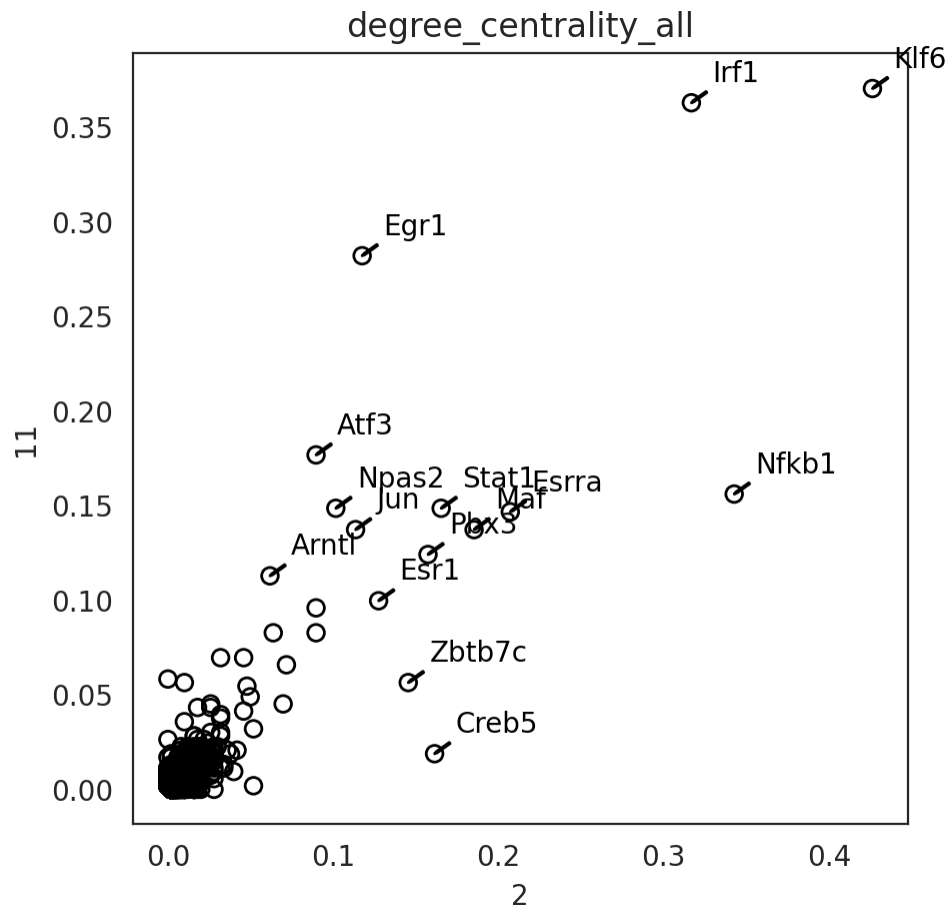
Processing clusters 2 and 9



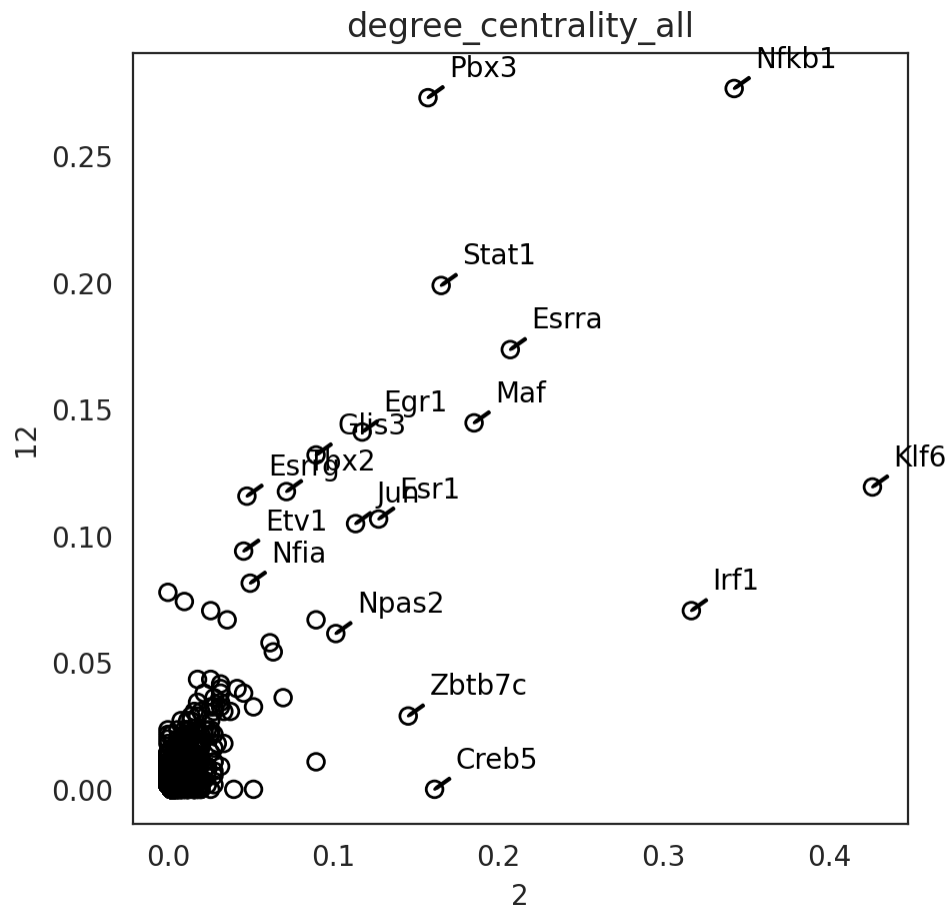
Processing clusters 2 and 10



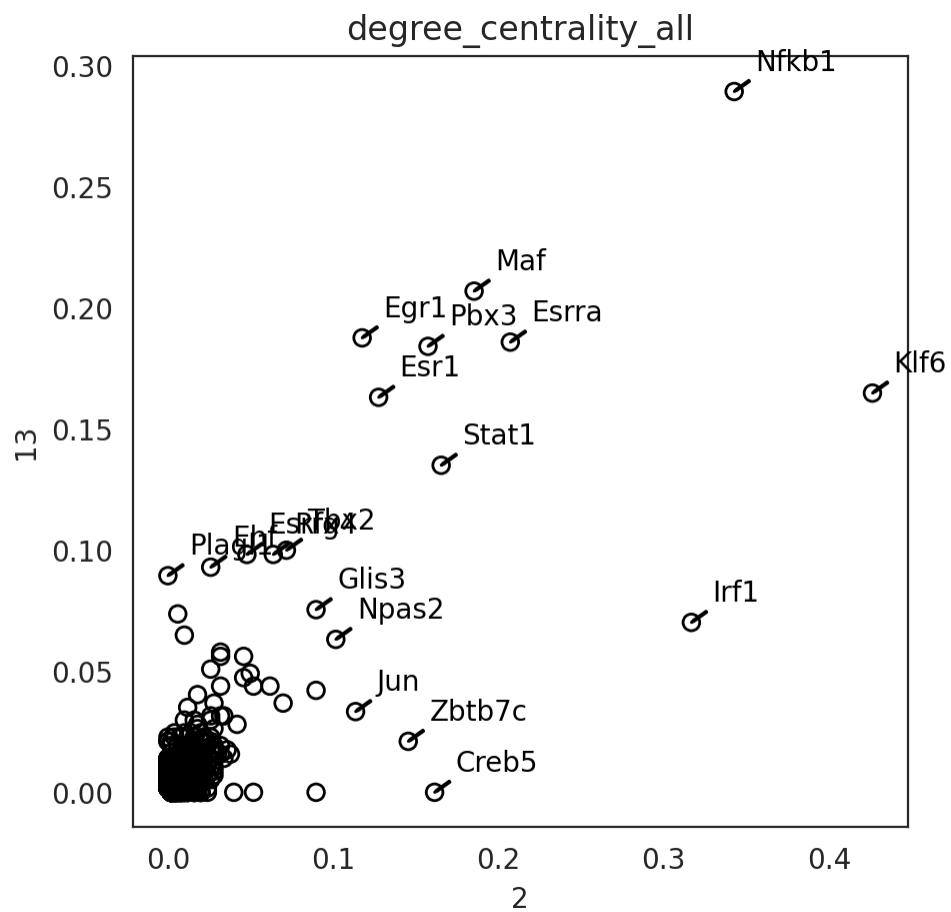
Processing clusters 2 and 11



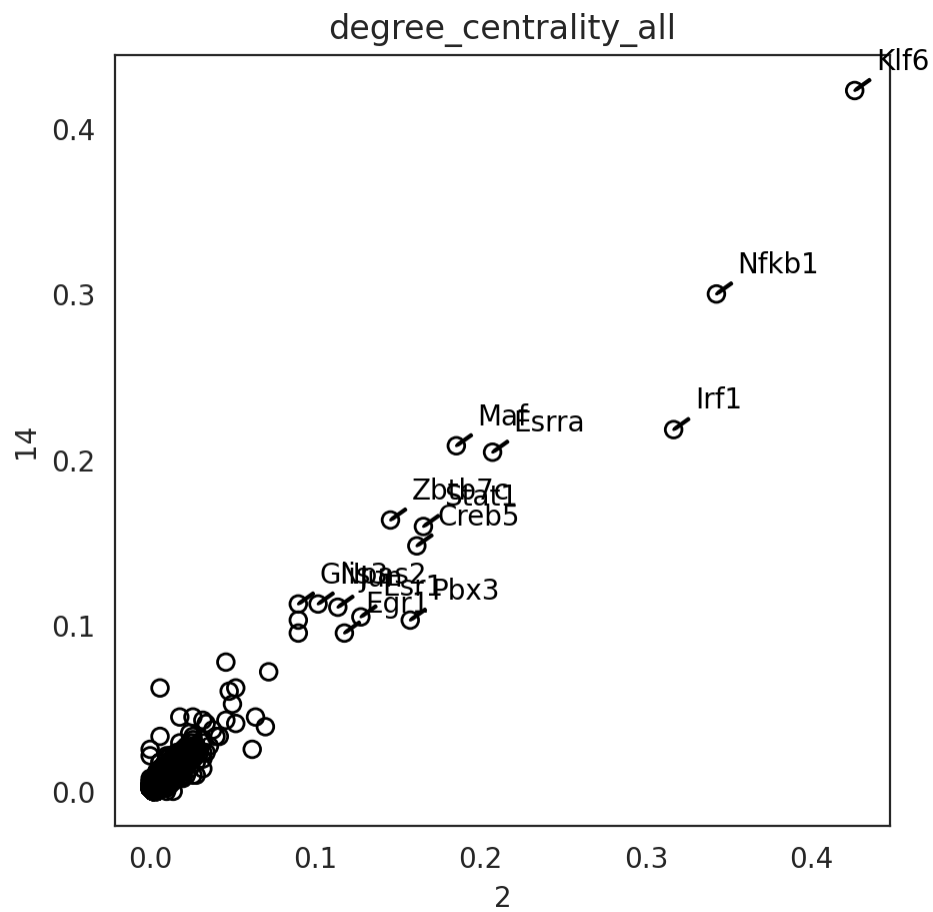
Processing clusters 2 and 12



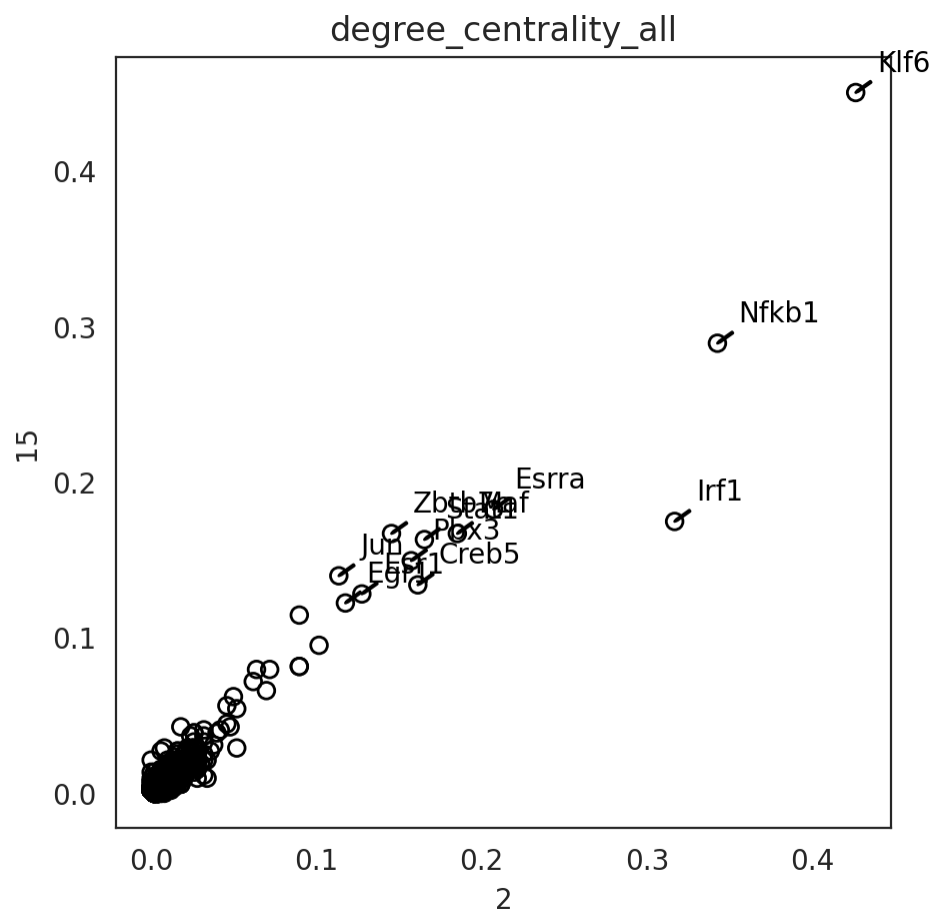
Processing clusters 2 and 13



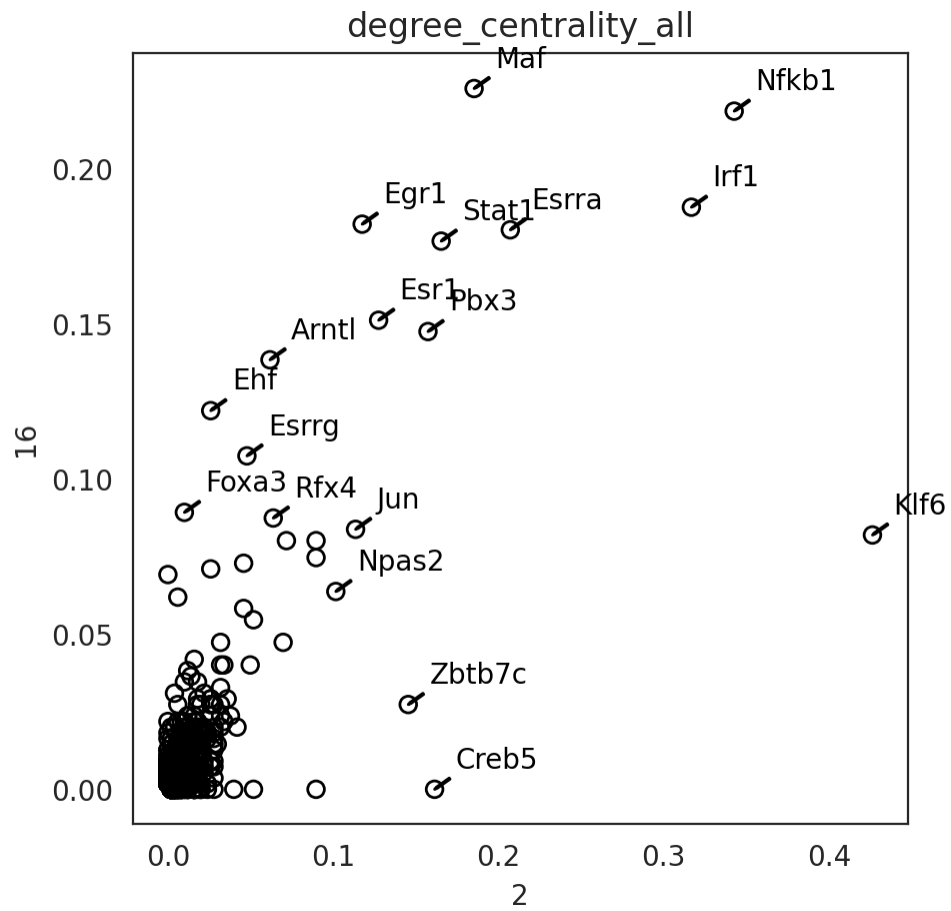
Processing clusters 2 and 14



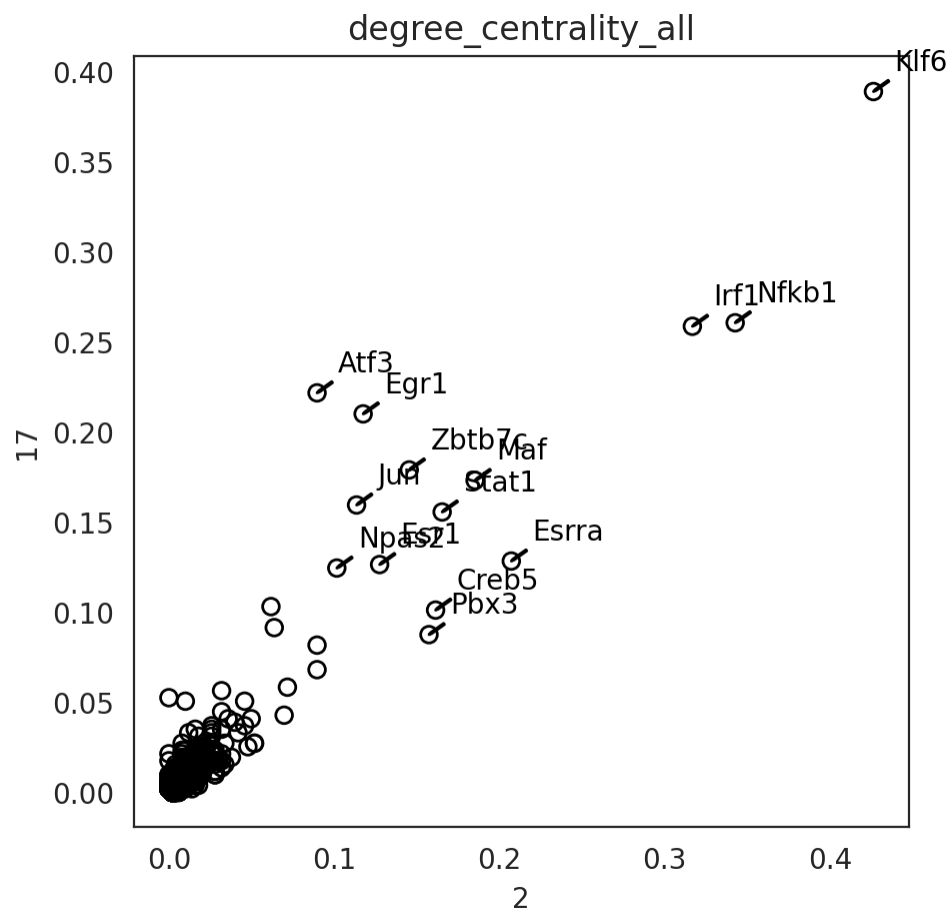
Processing clusters 2 and 15



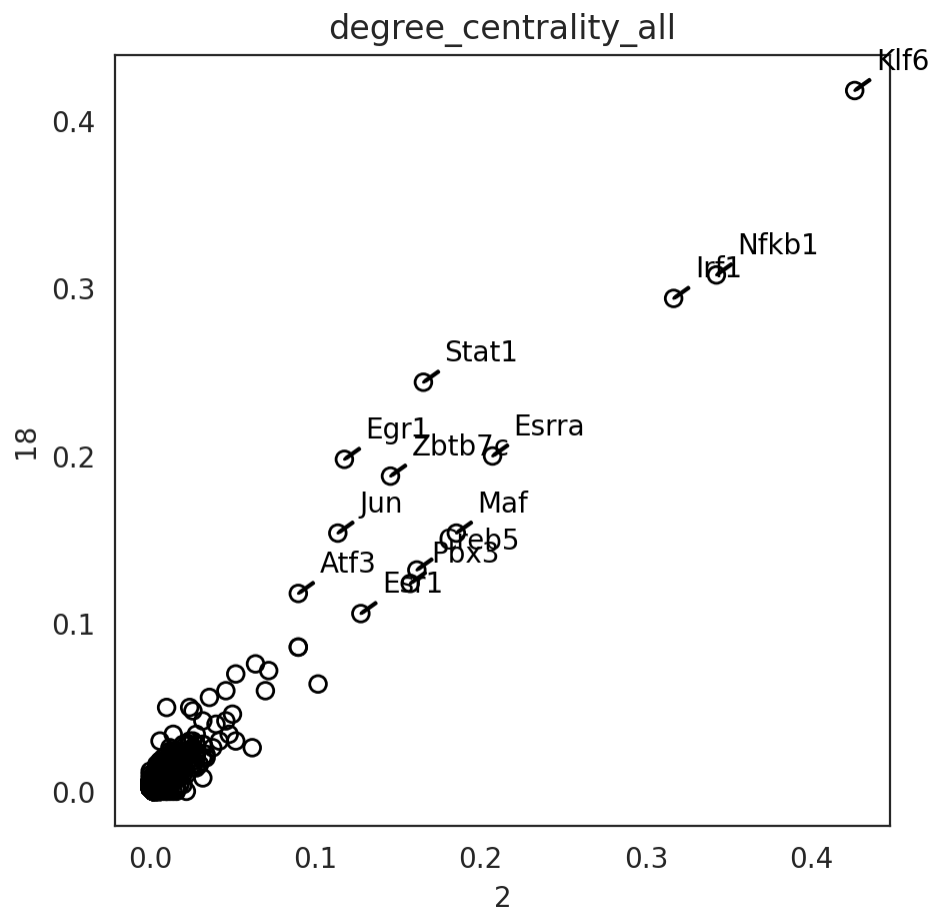
Processing clusters 2 and 16



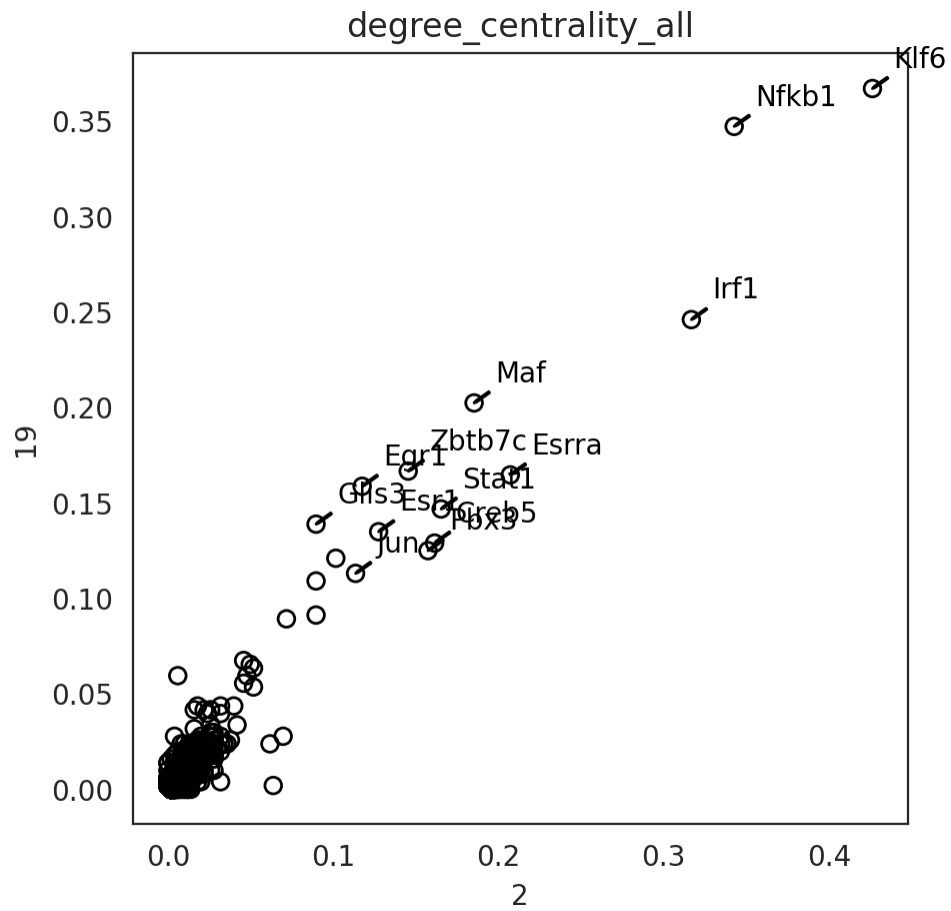
Processing clusters 2 and 17



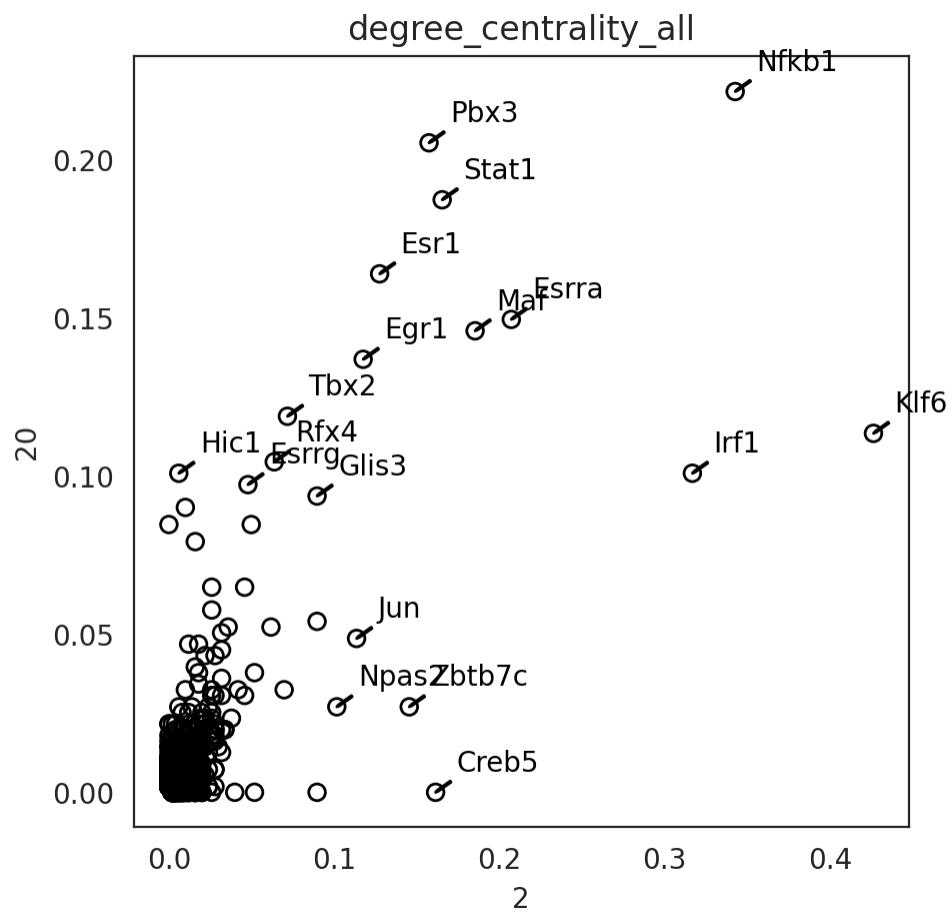
Processing clusters 2 and 18



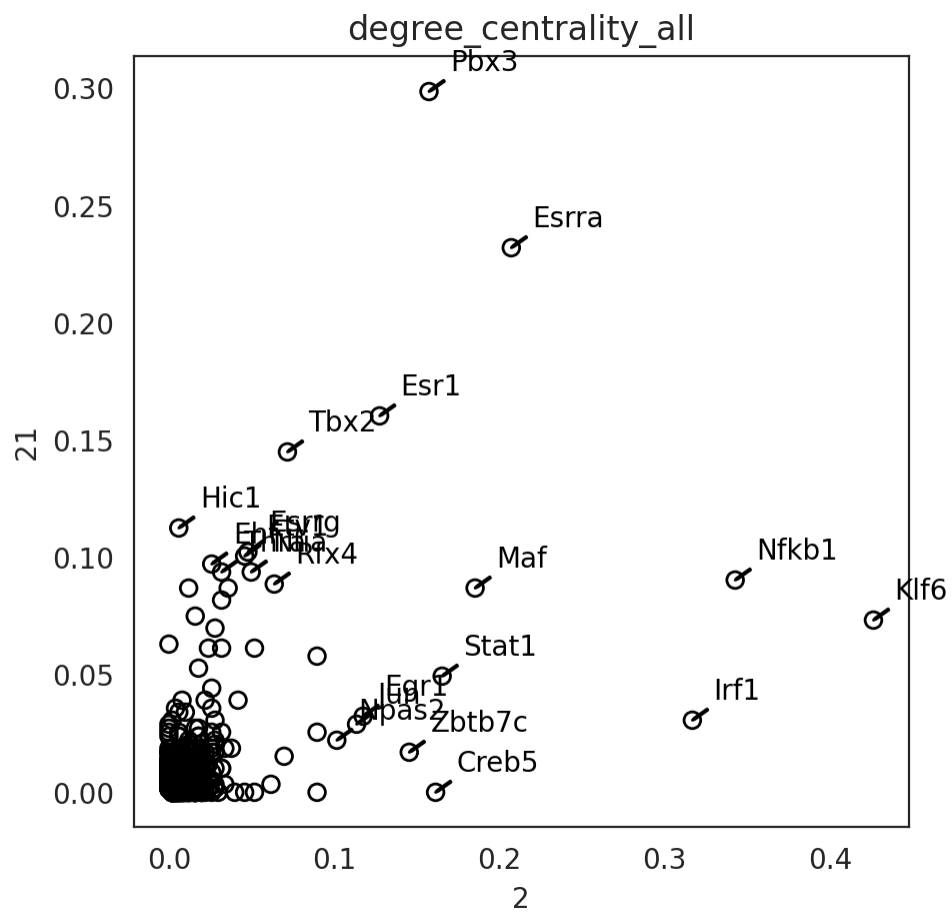
Processing clusters 2 and 19



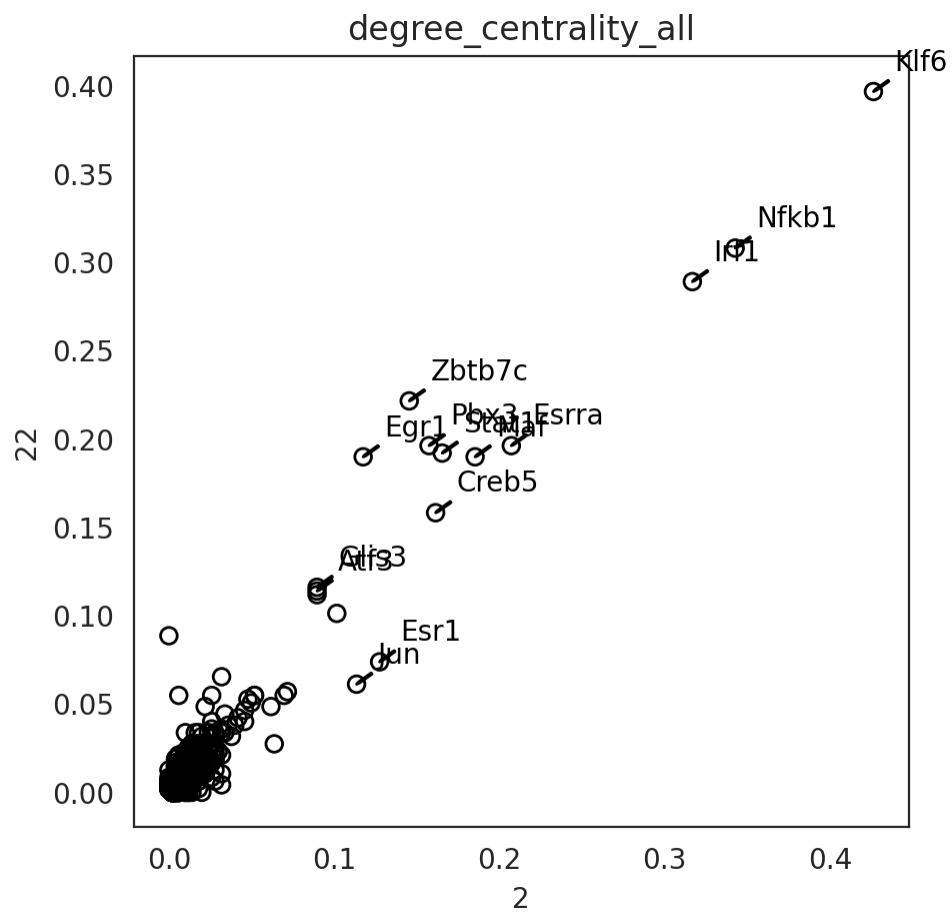
Processing clusters 2 and 20



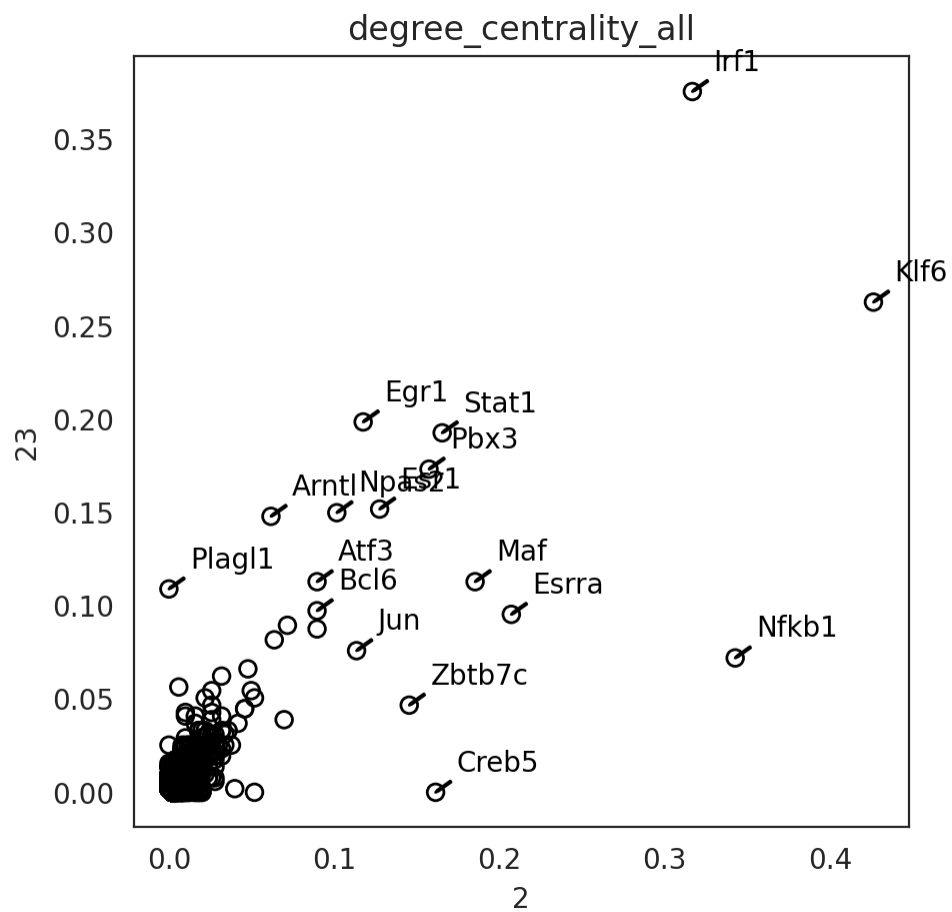
Processing clusters 2 and 21



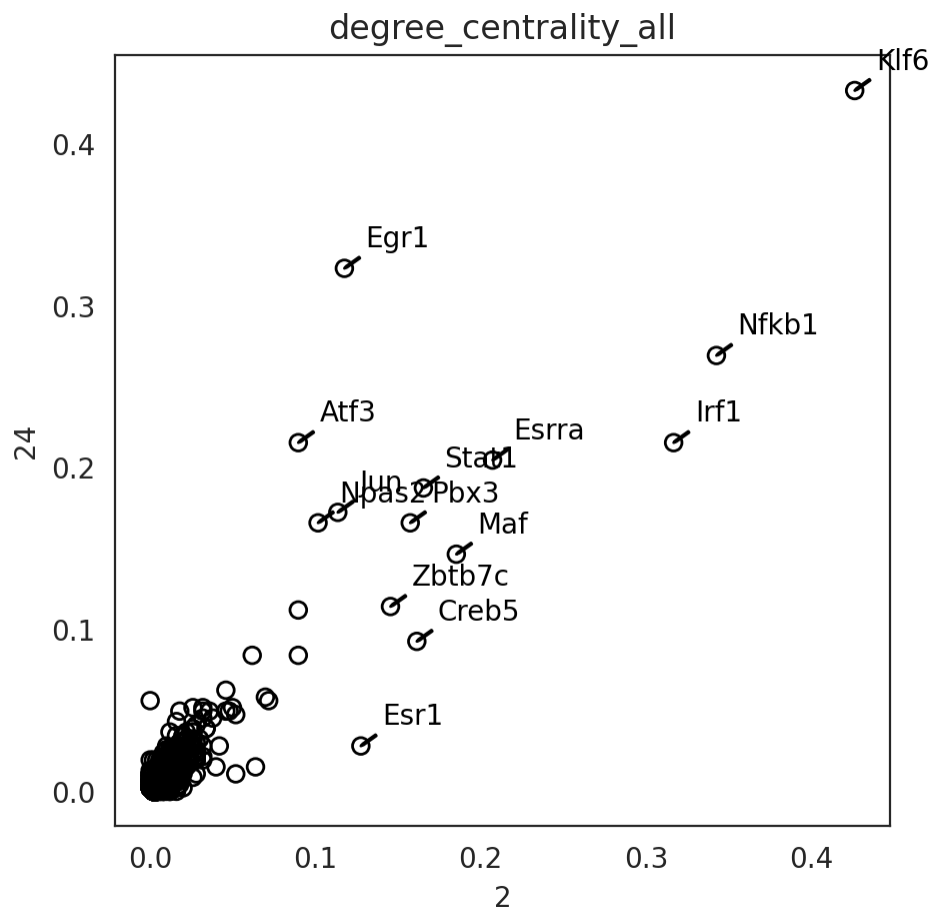
Processing clusters 2 and 22



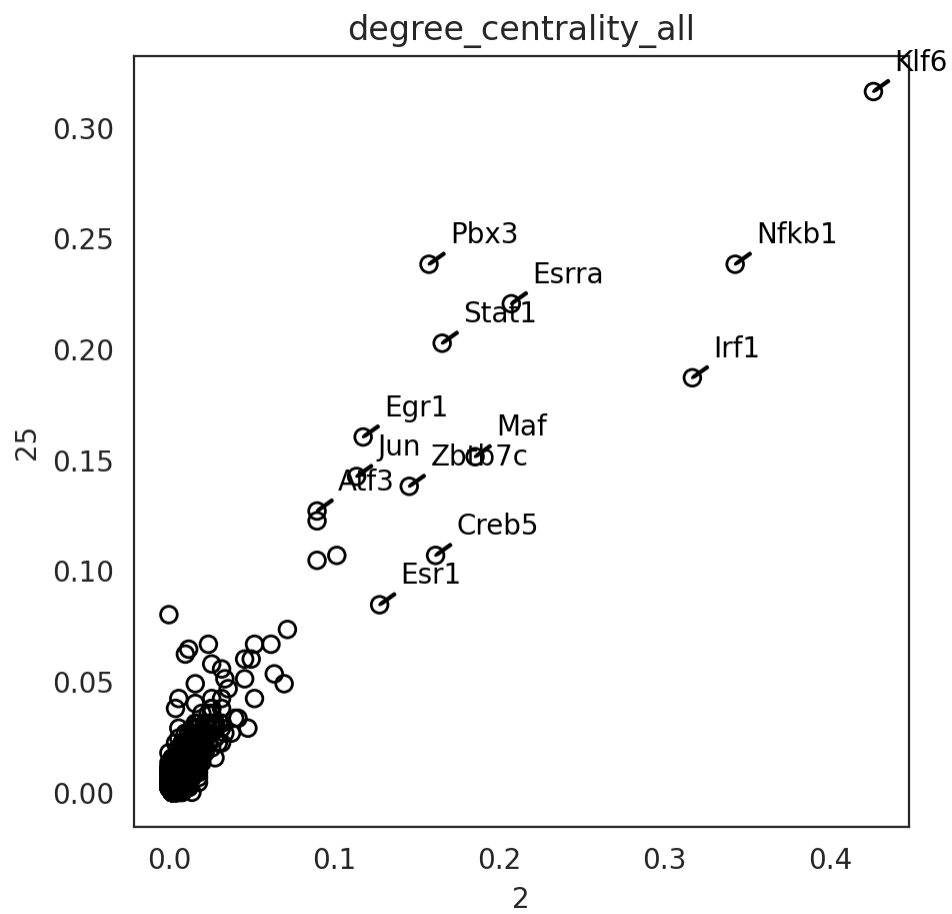
Processing clusters 2 and 23



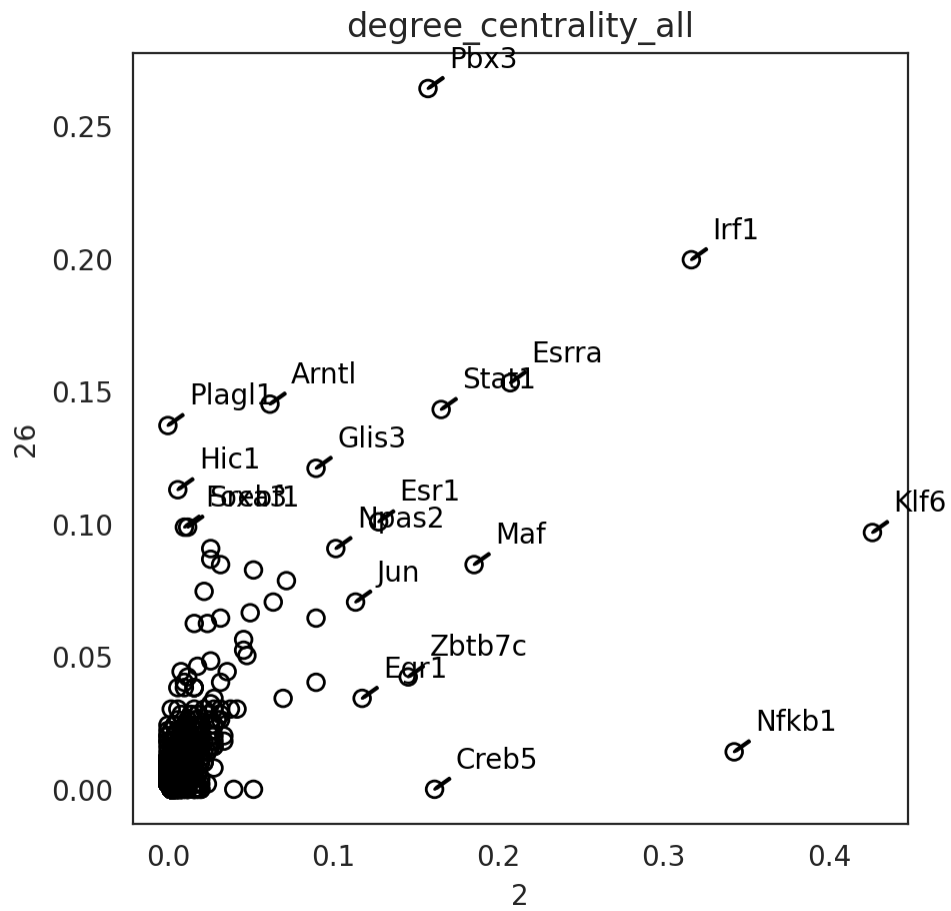
Processing clusters 2 and 24



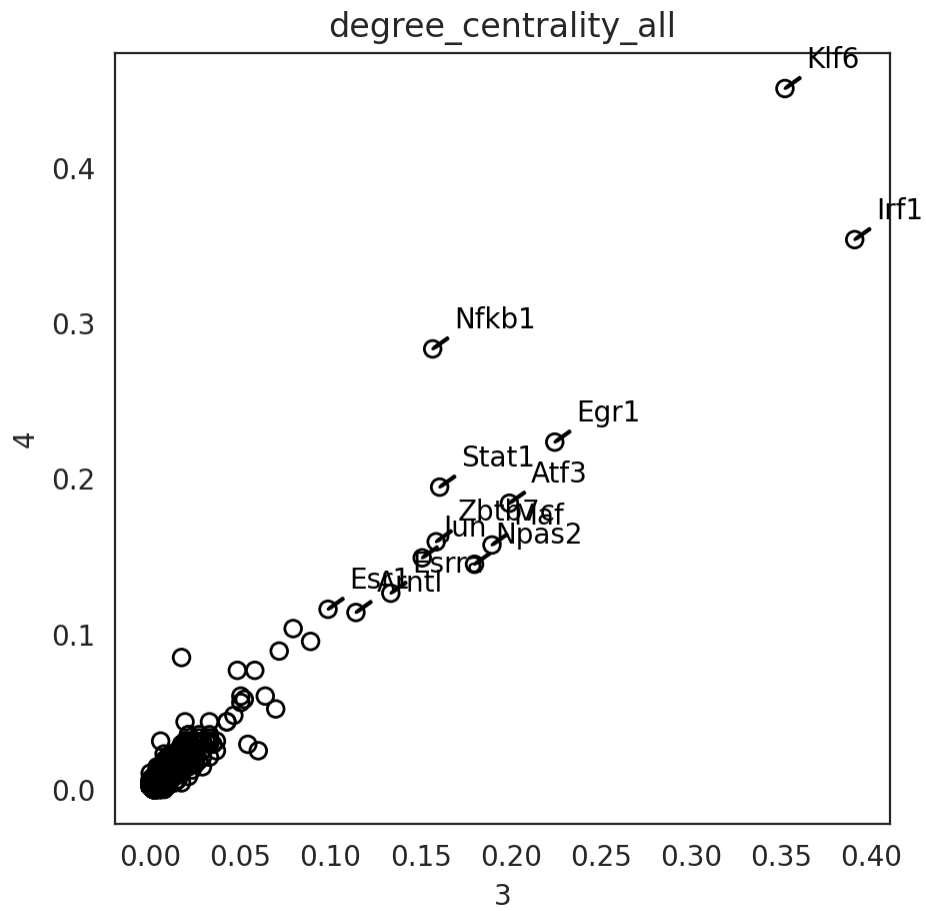
Processing clusters 2 and 25



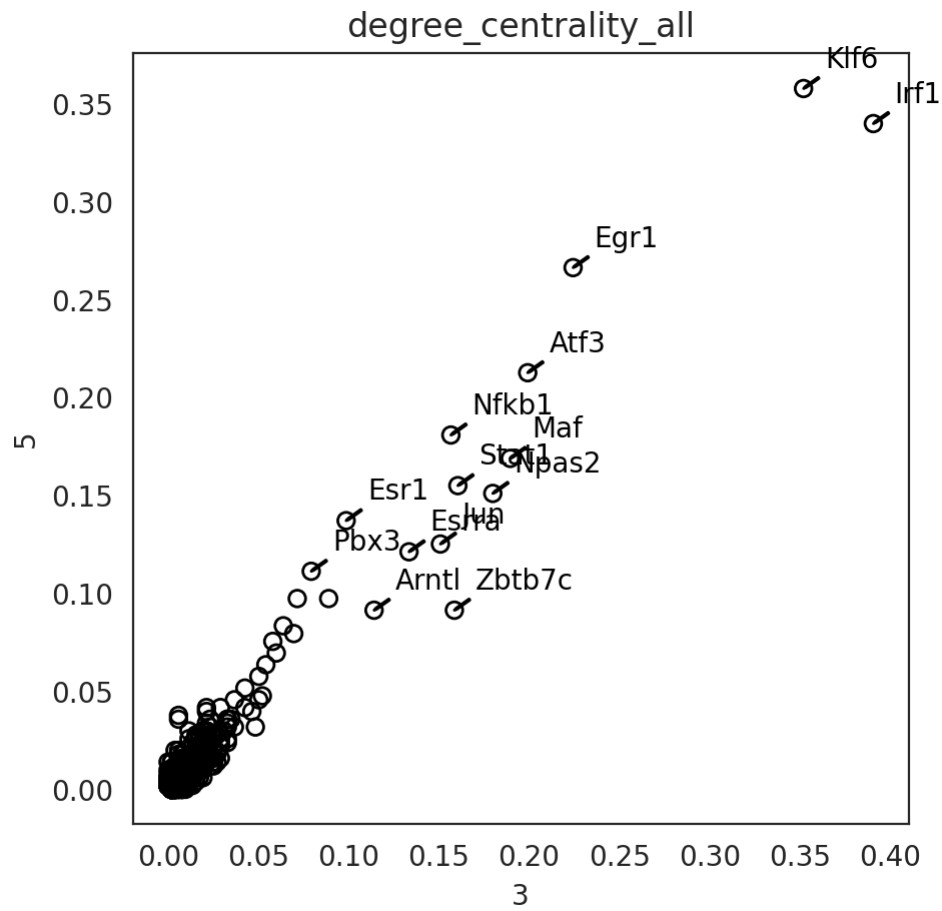
Processing clusters 2 and 26



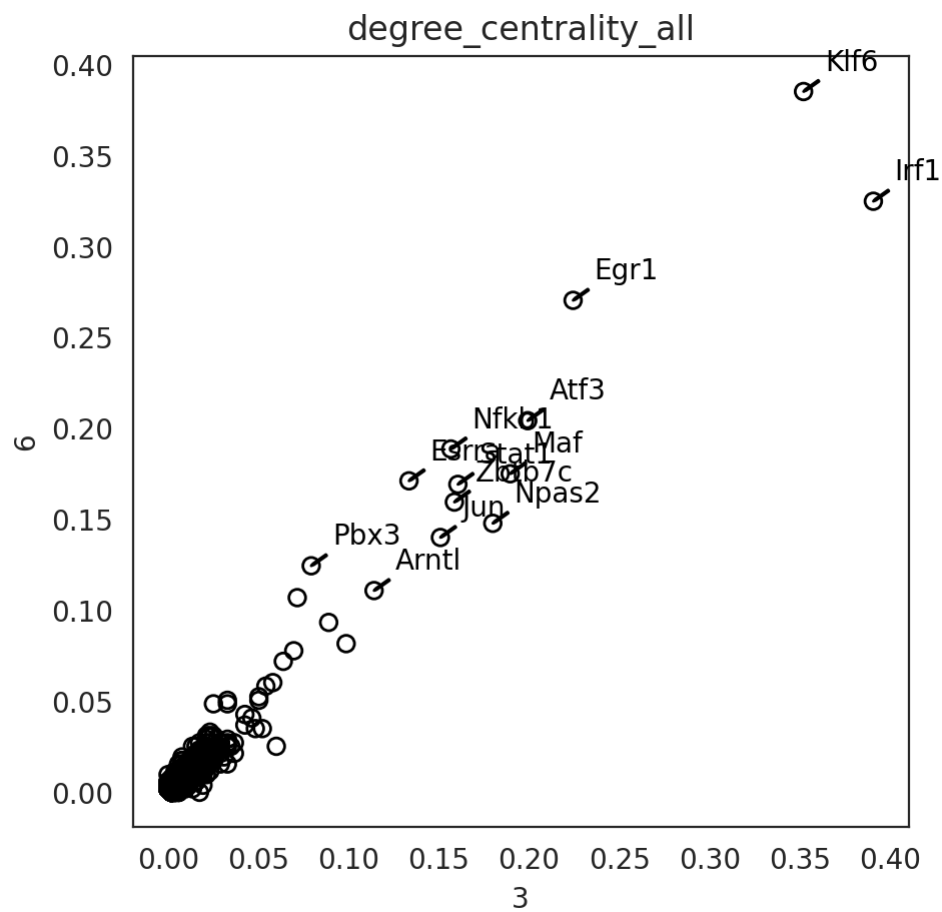
Processing clusters 3 and 4



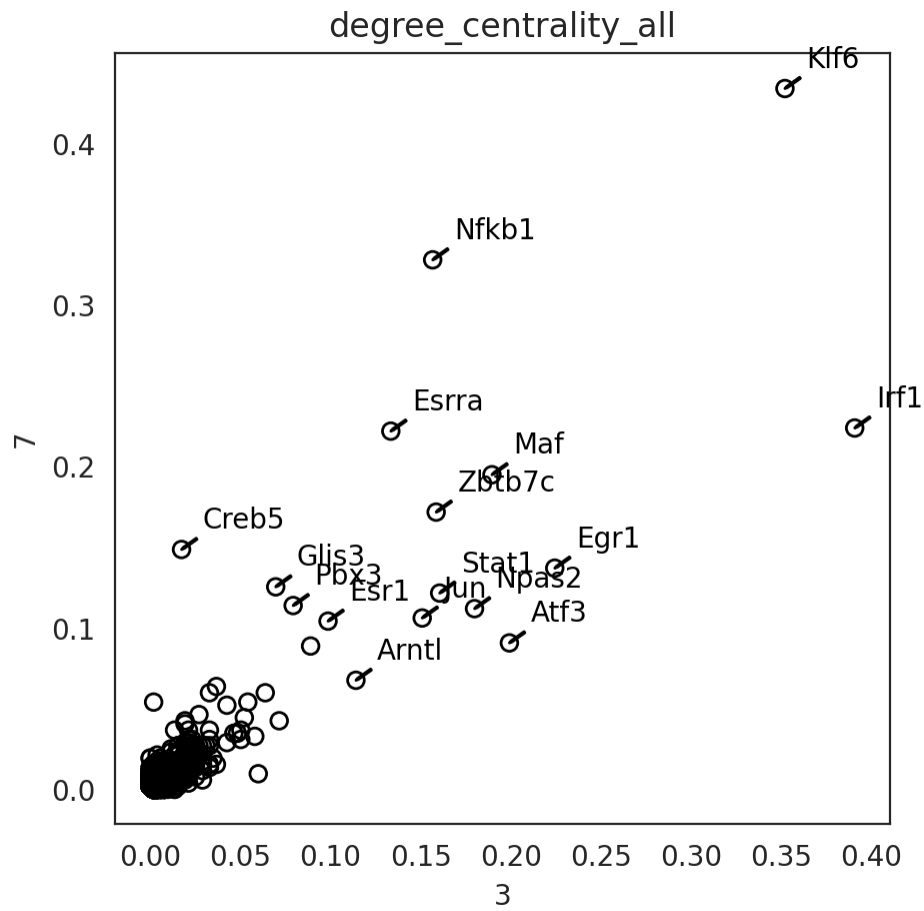
Processing clusters 3 and 5



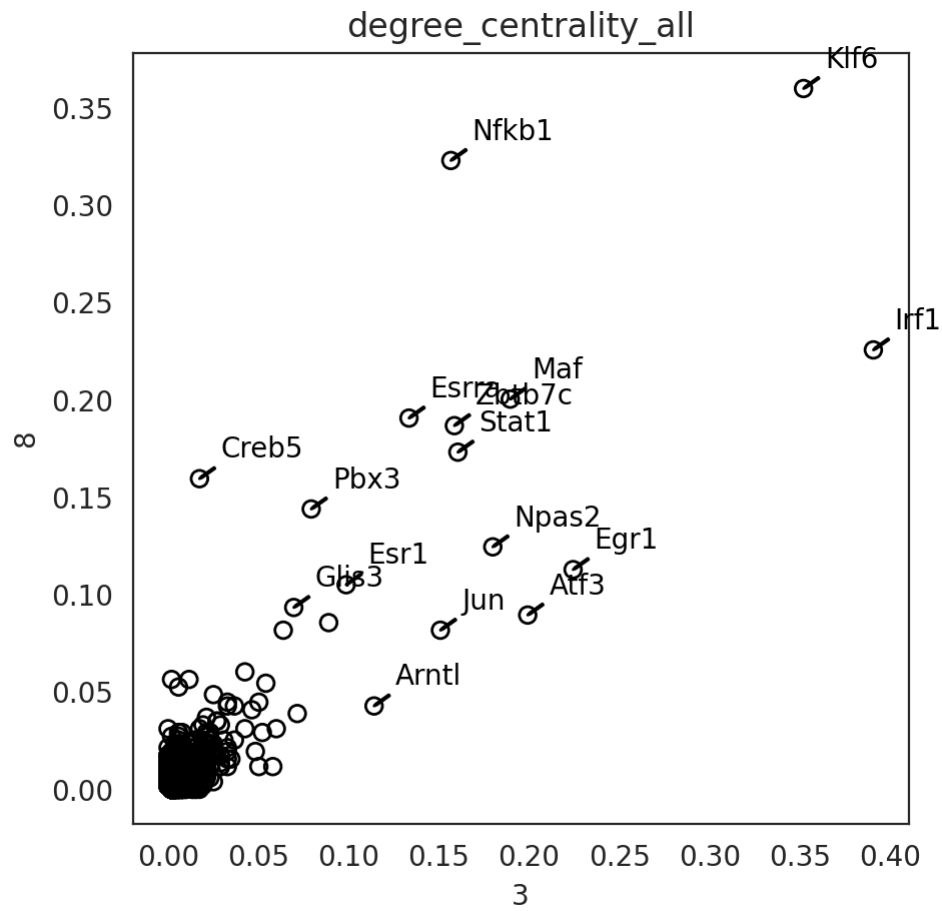
Processing clusters 3 and 6



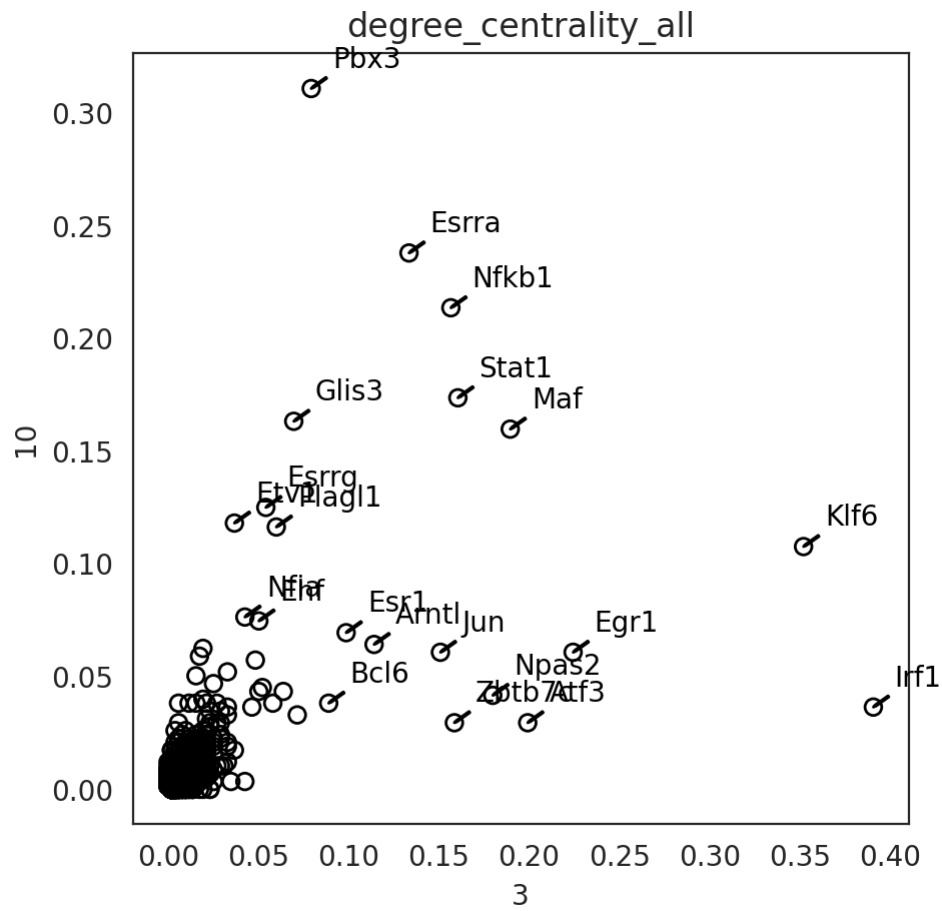
Processing clusters 3 and 7



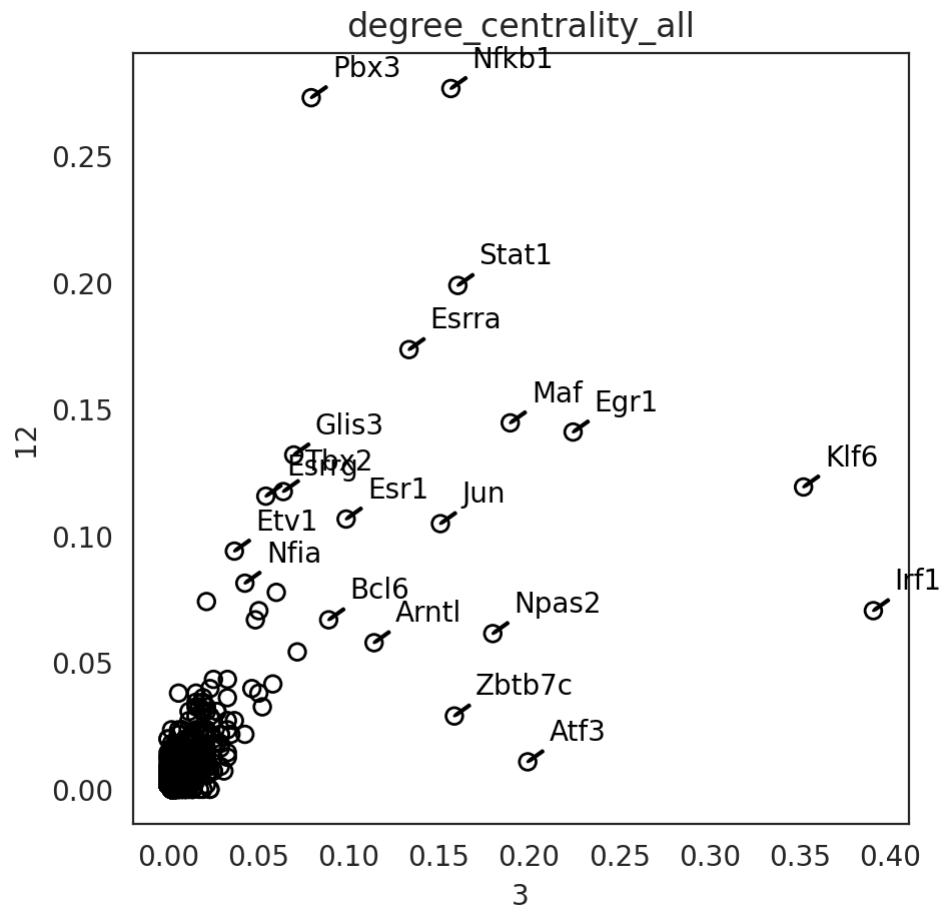
Processing clusters 3 and 8



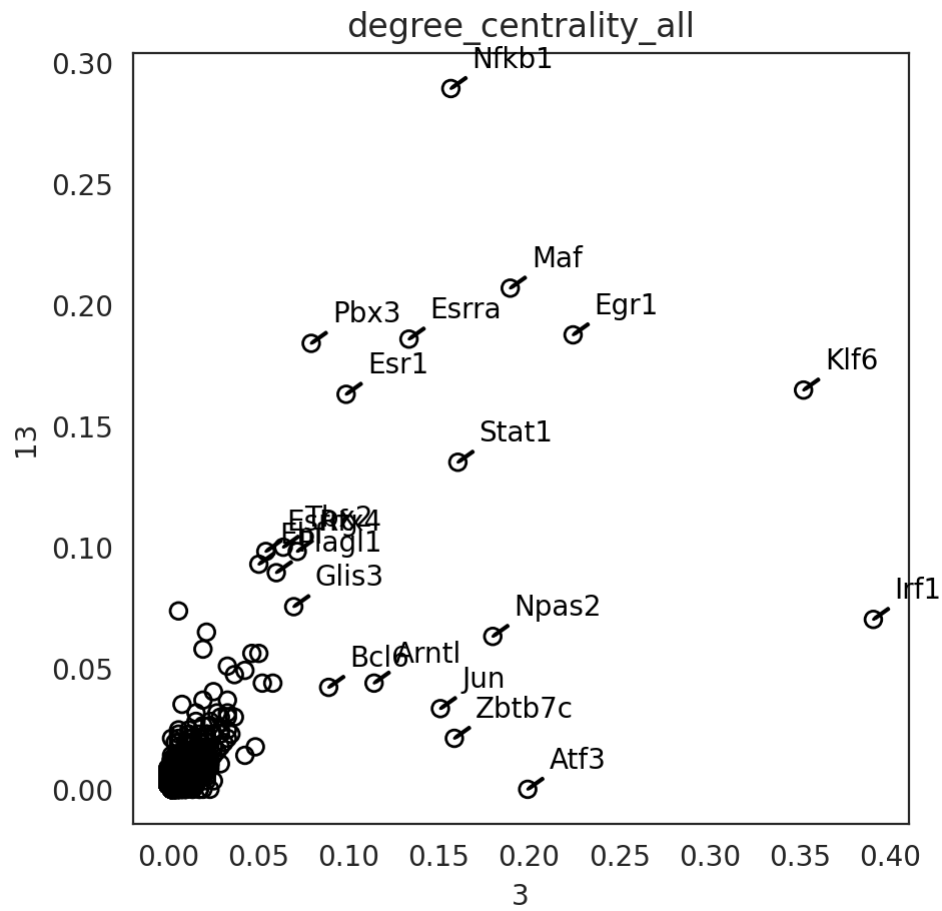
Processing clusters 3 and 9



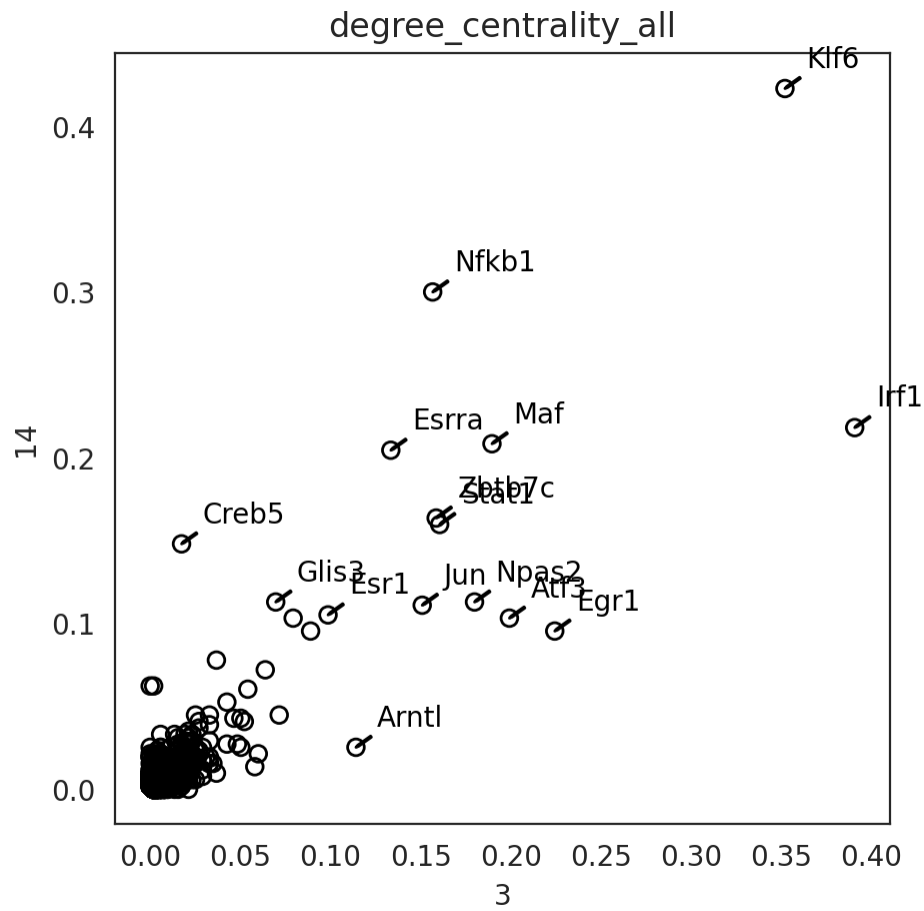
Processing clusters 3 and 11



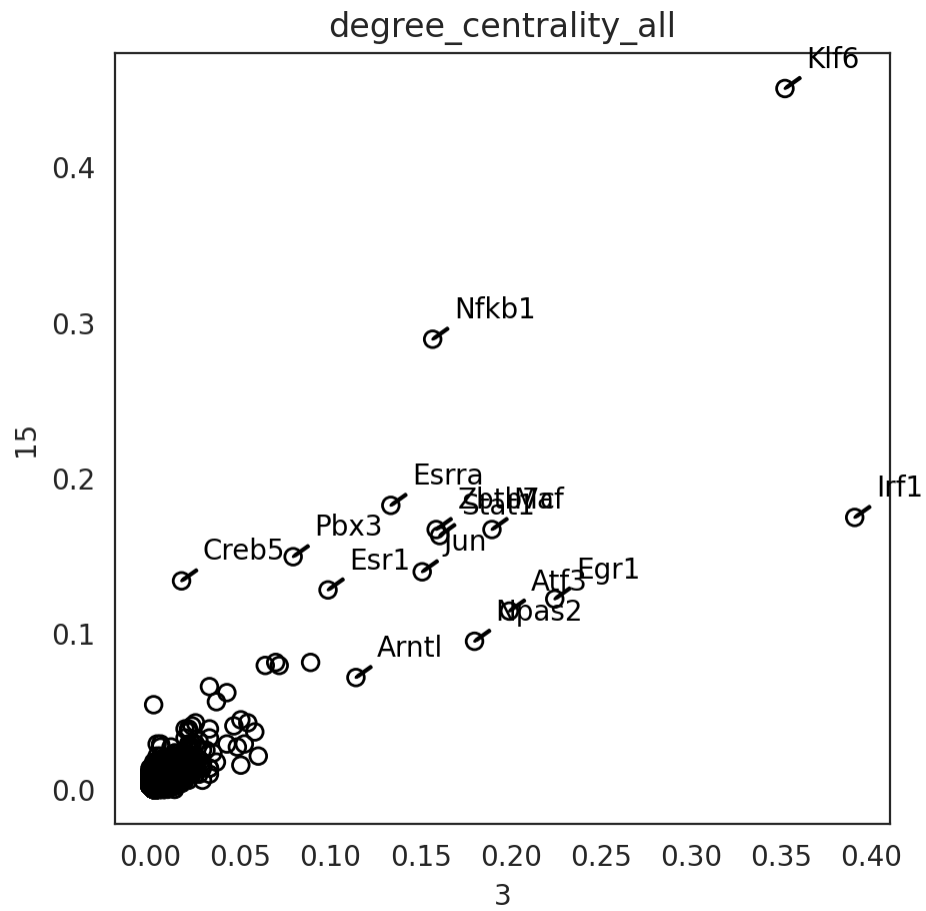
Processing clusters 3 and 13



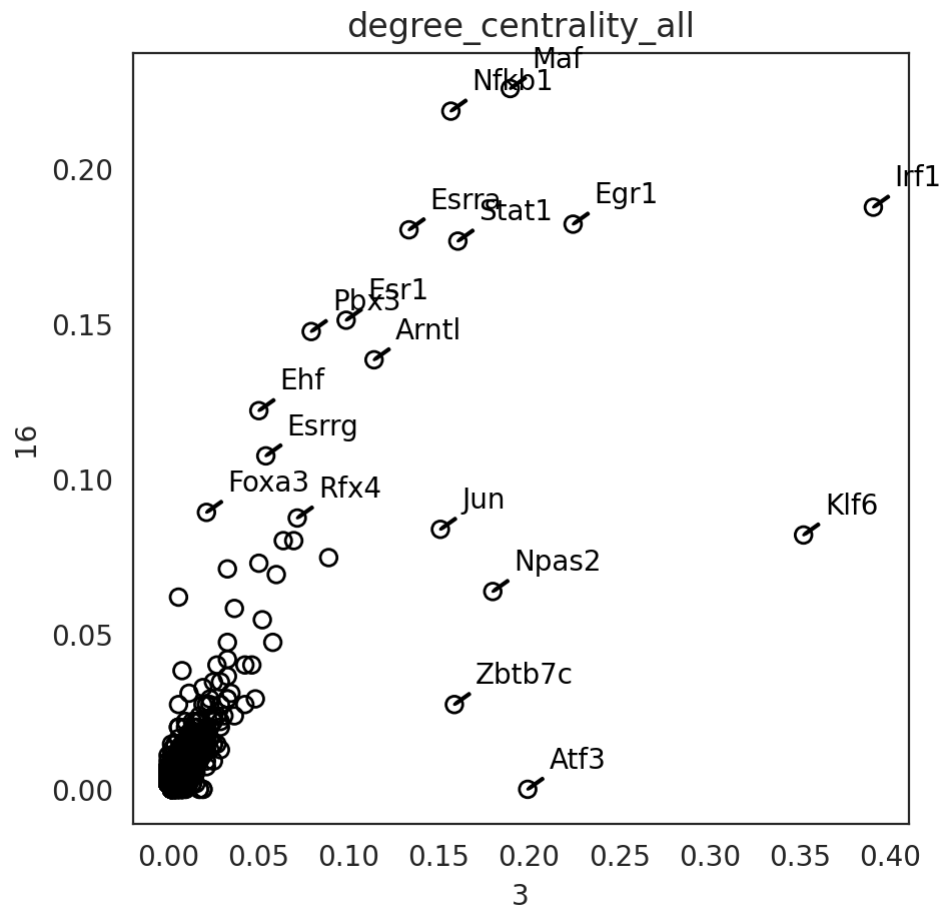
Processing clusters 3 and 14



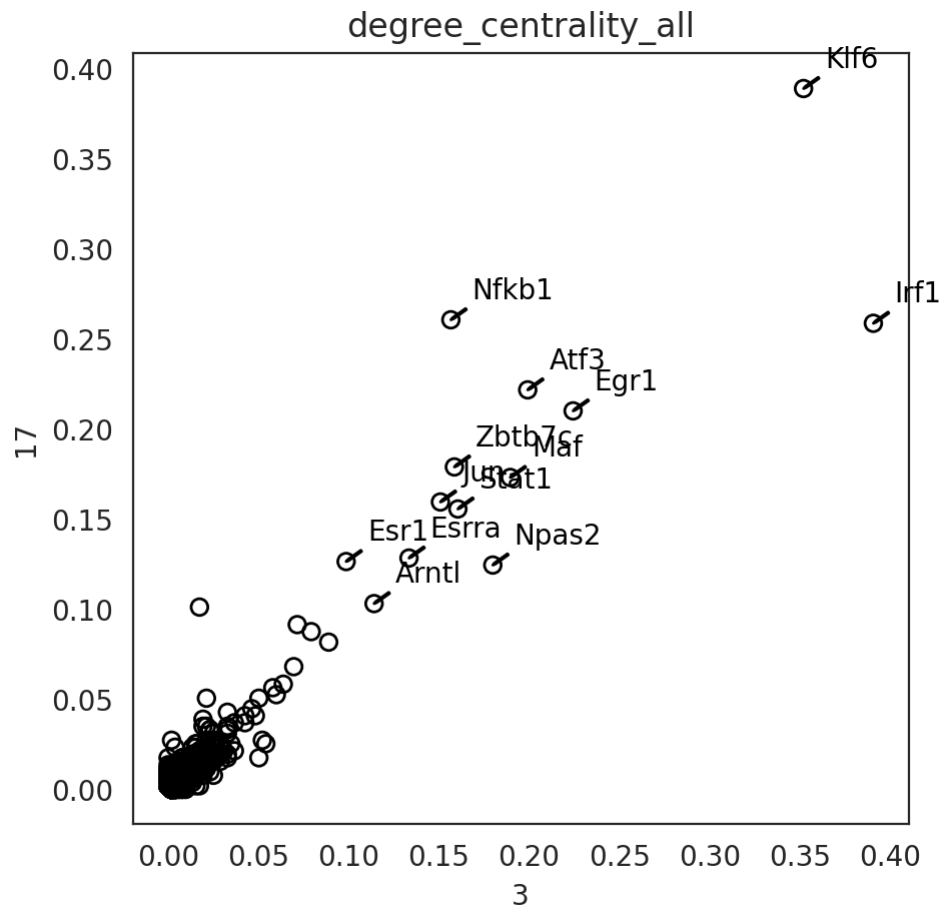
Processing clusters 3 and 15



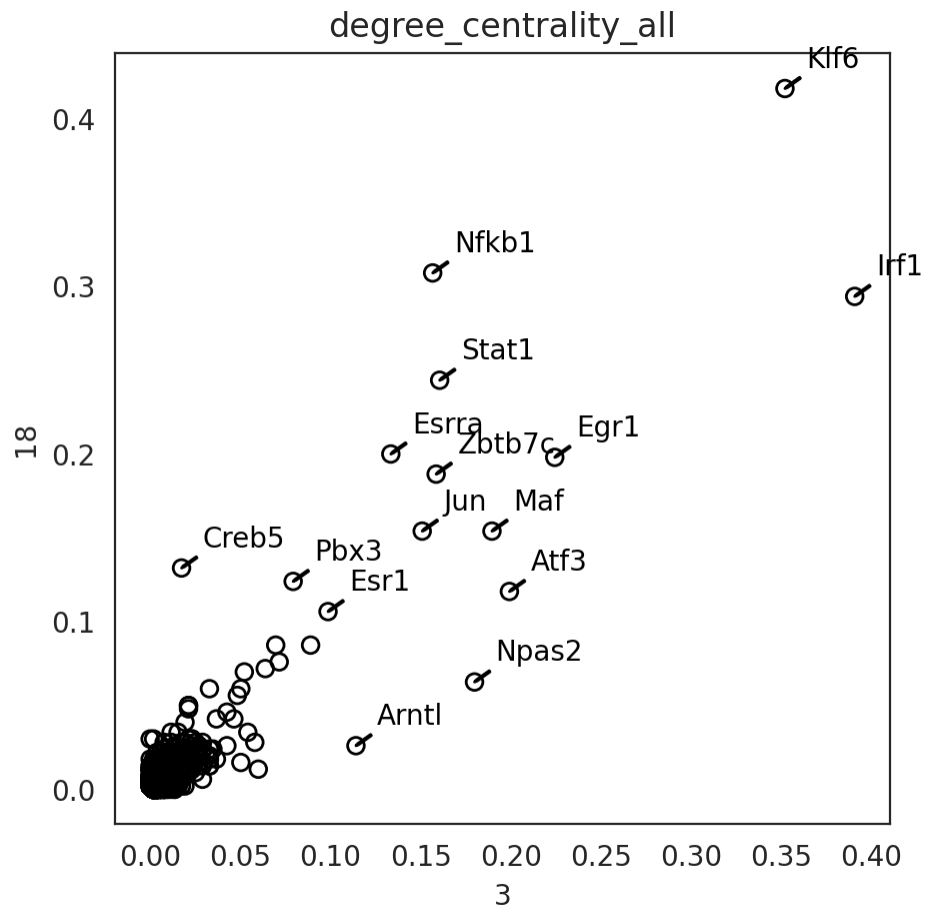
Processing clusters 3 and 16



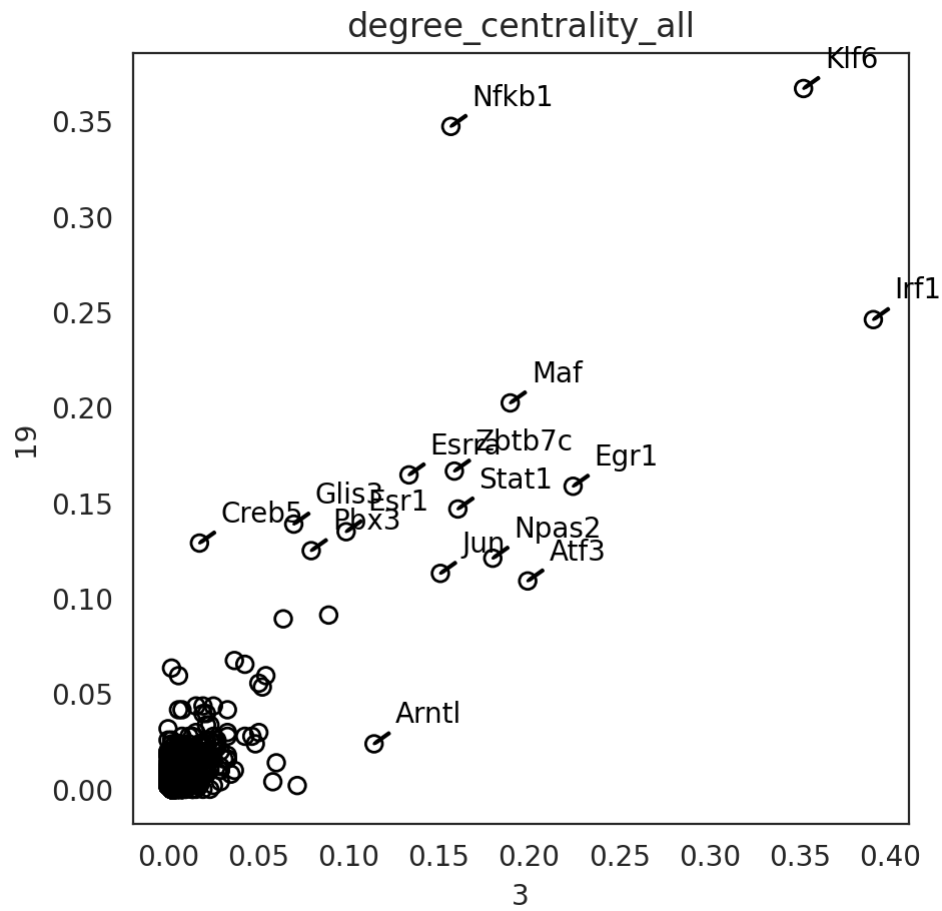
Processing clusters 3 and 17



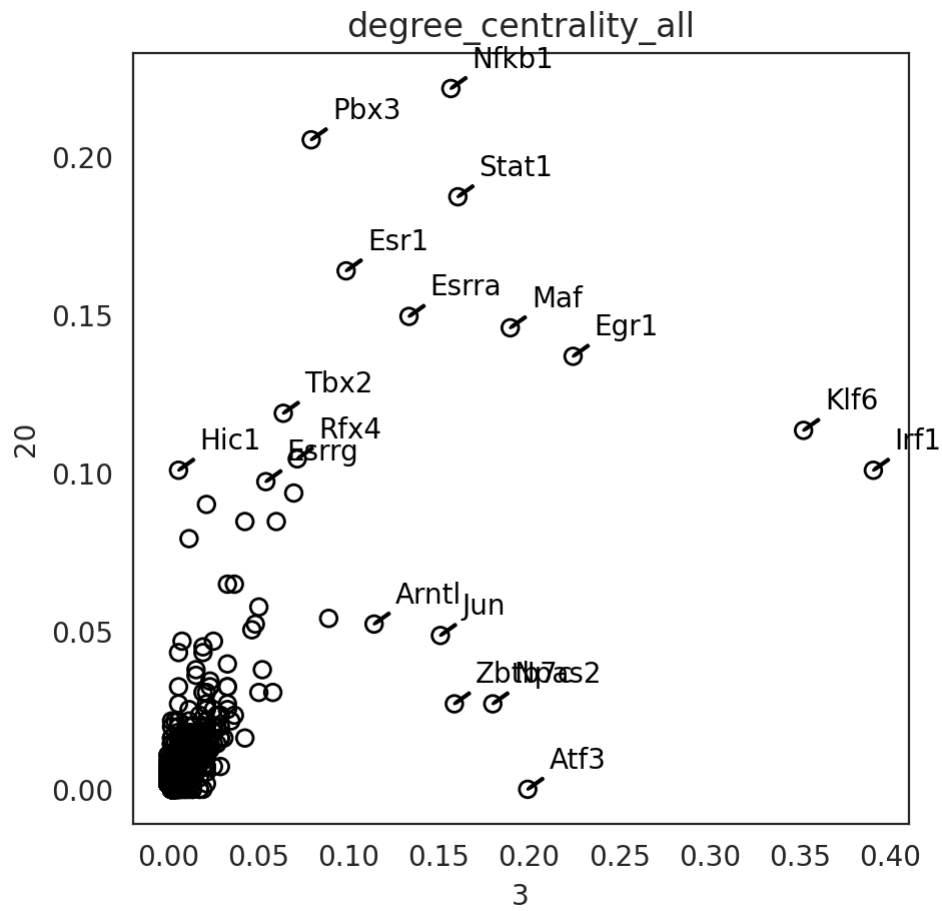
Processing clusters 3 and 18



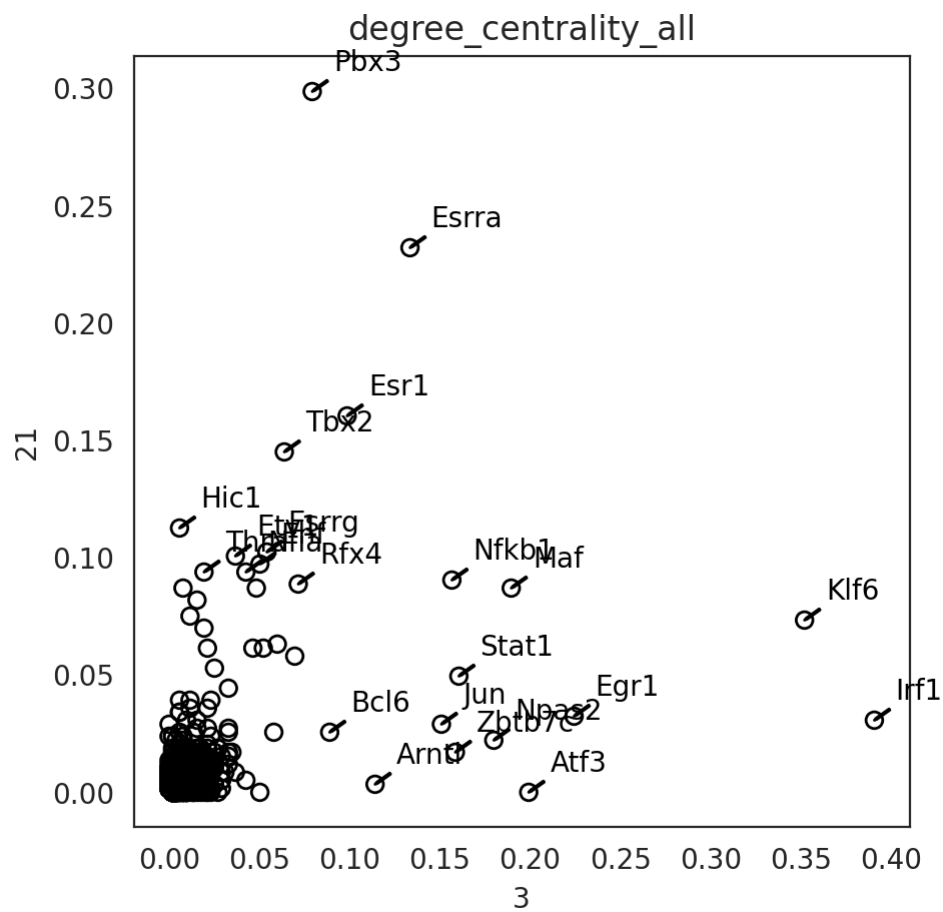
Processing clusters 3 and 19



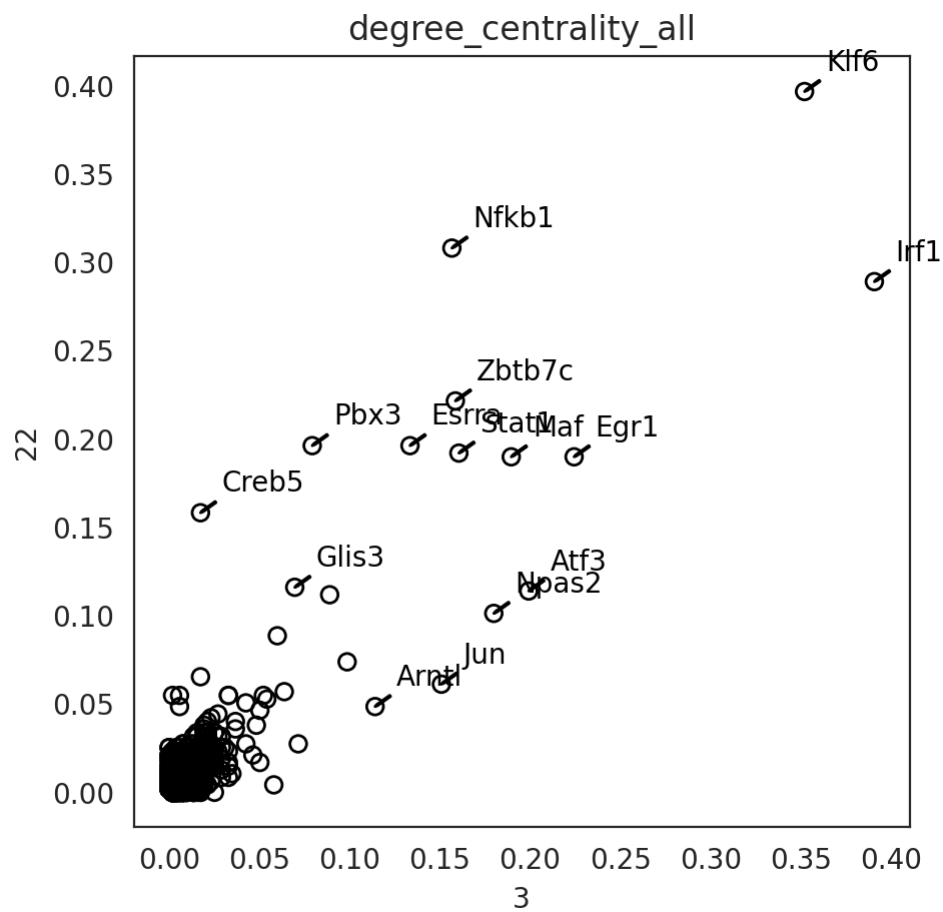
Processing clusters 3 and 20



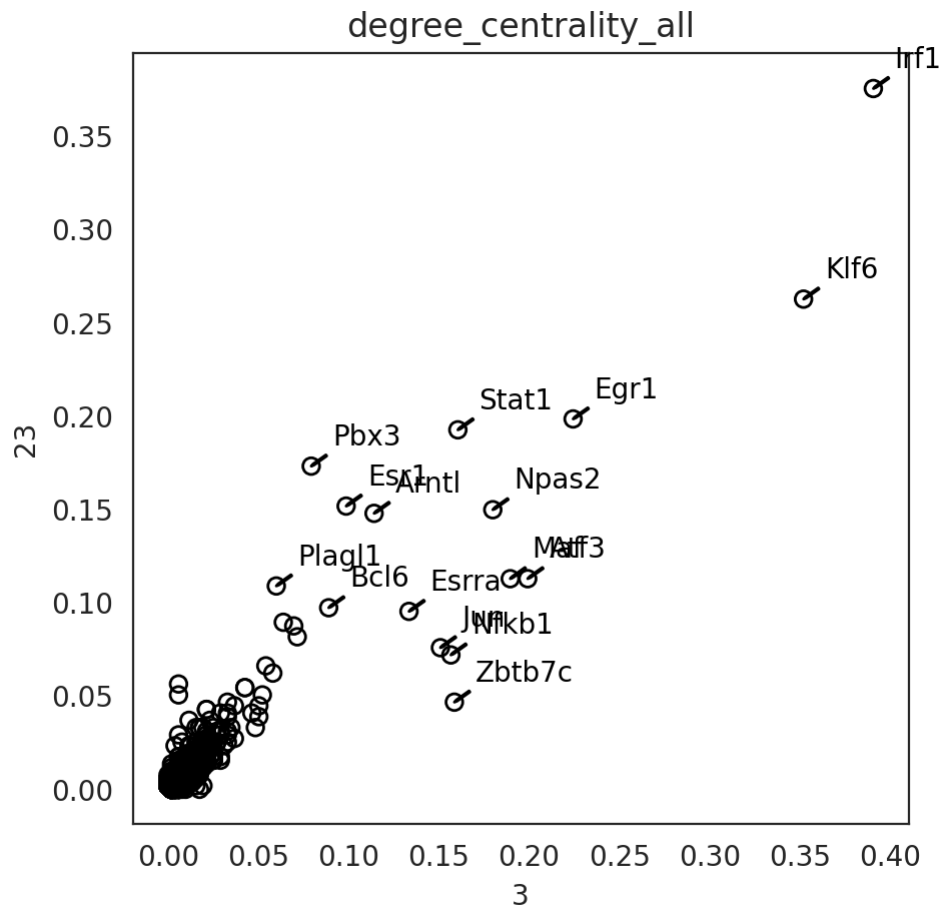
Processing clusters 3 and 21

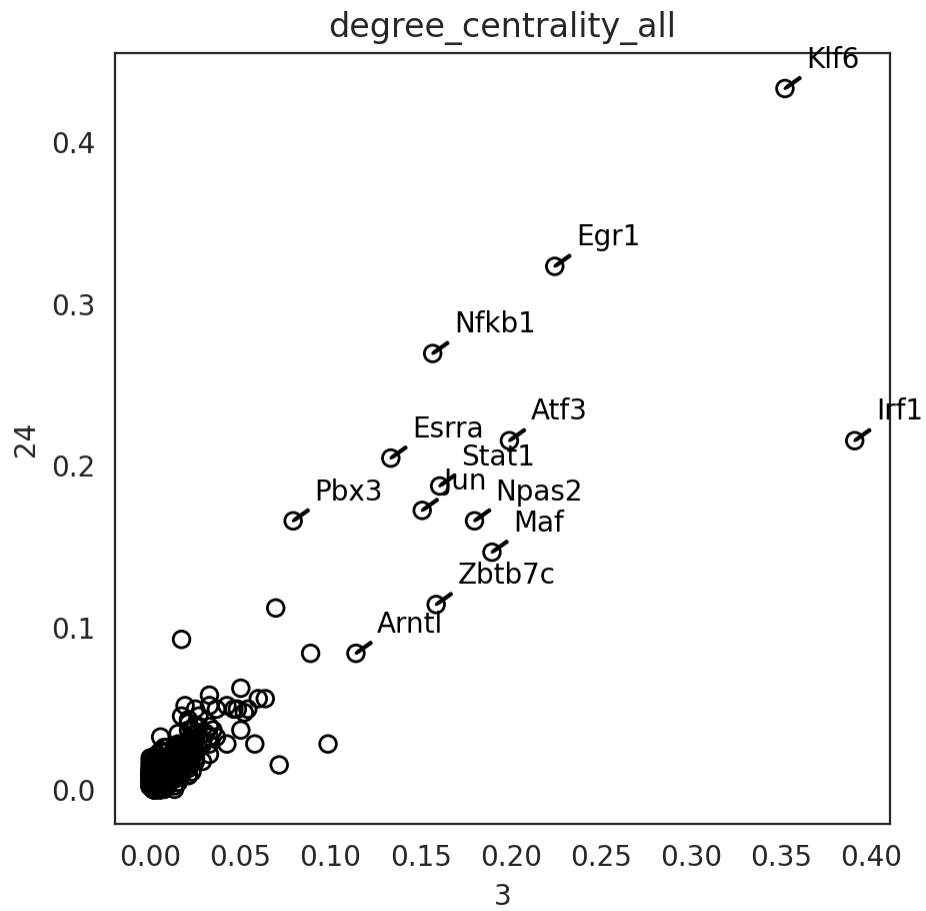


Processing clusters 3 and 22

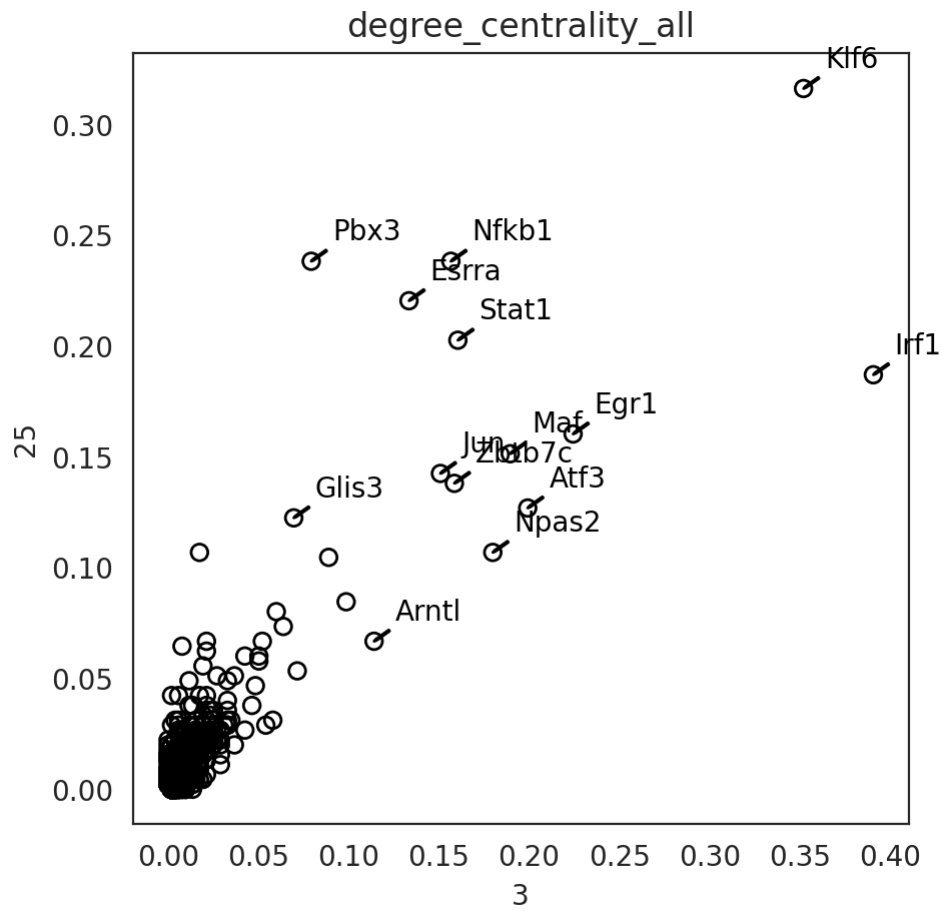


Processing clusters 3 and 23

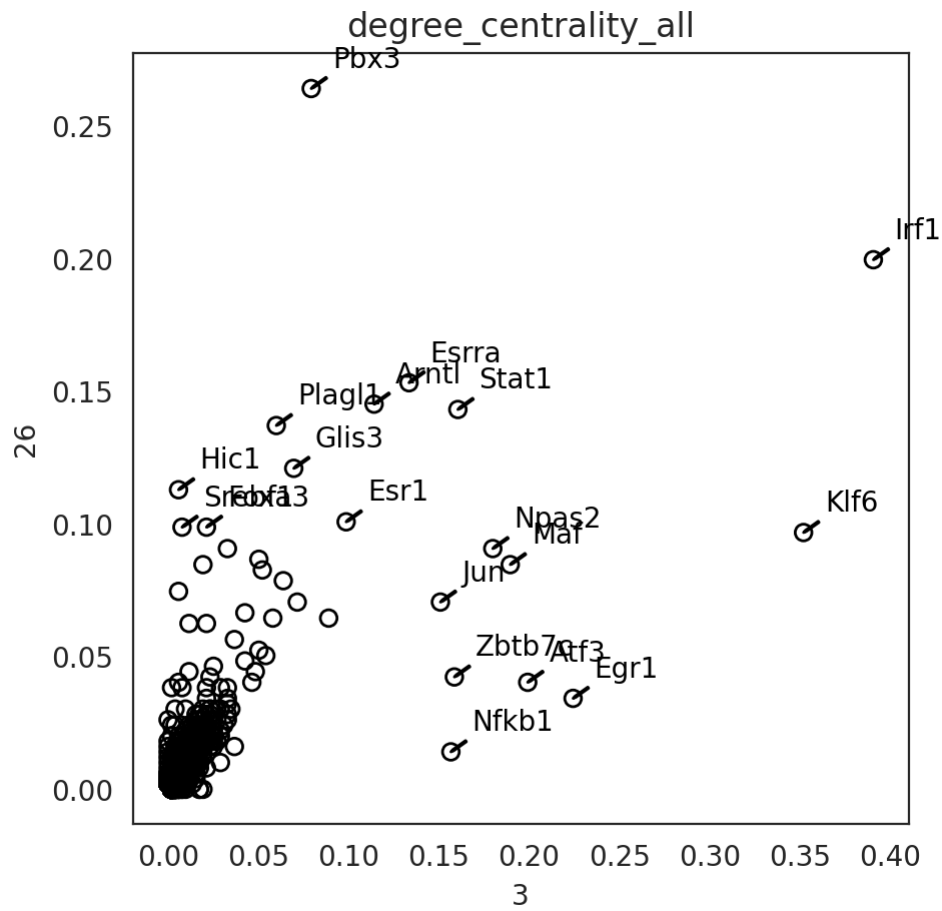




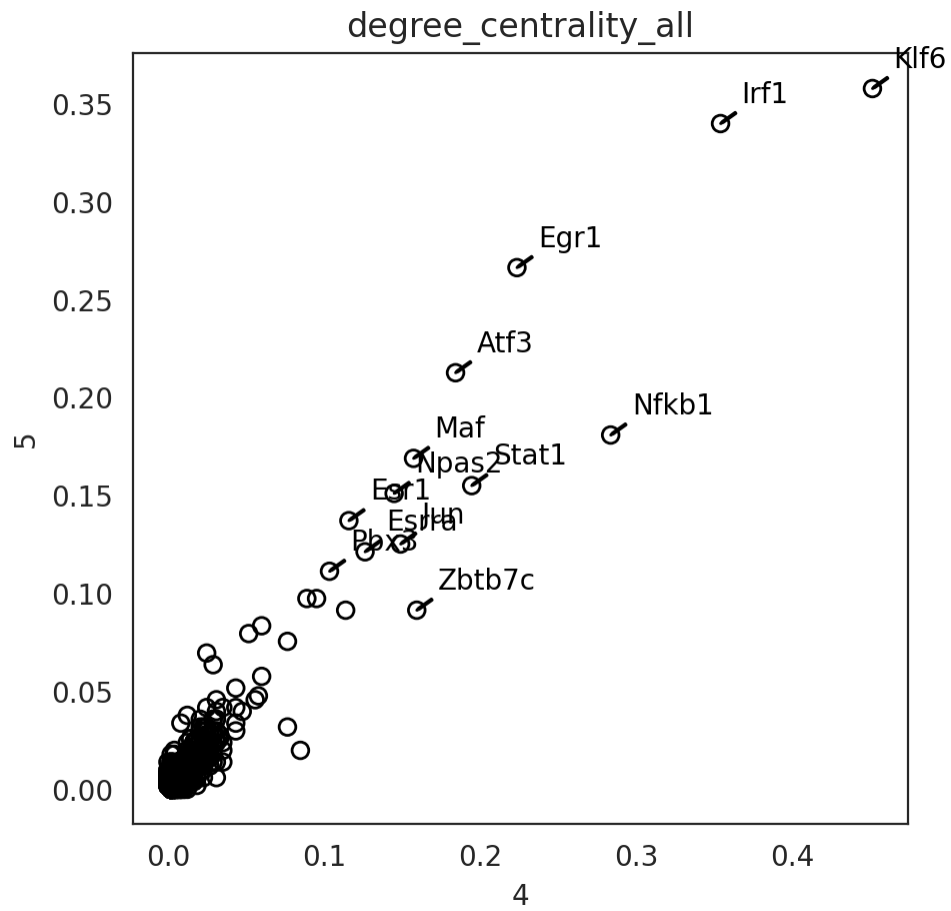
Processing clusters 3 and 25



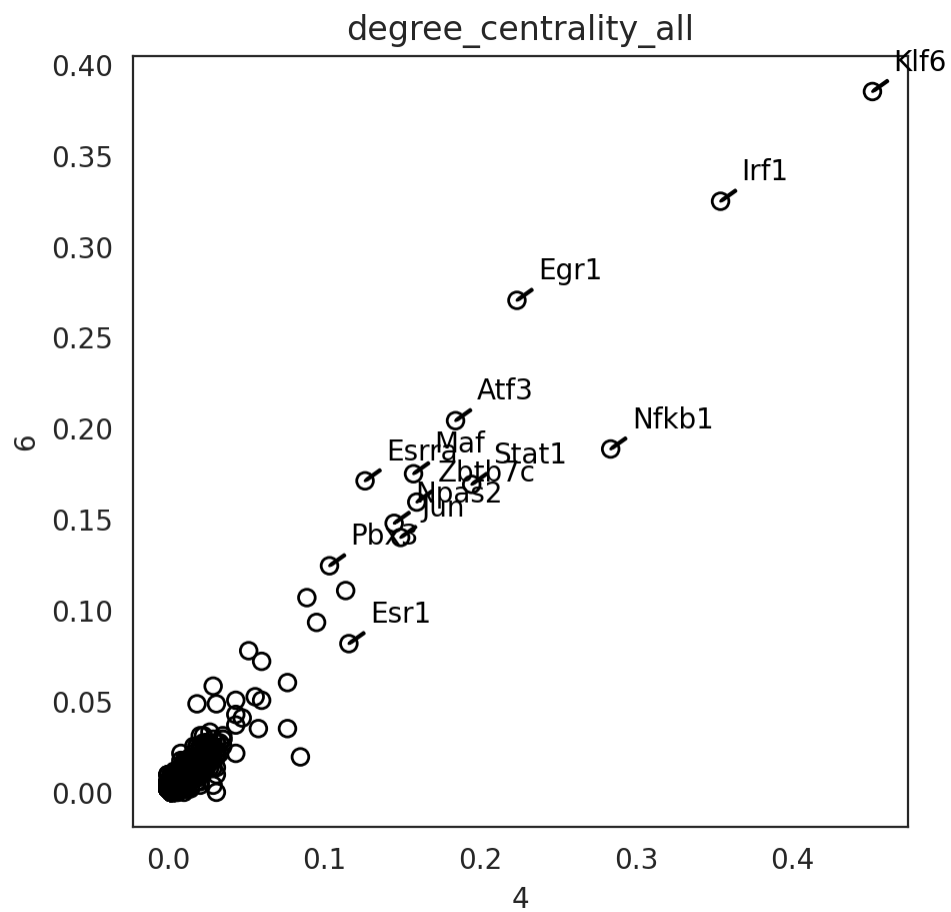
Processing clusters 3 and 26

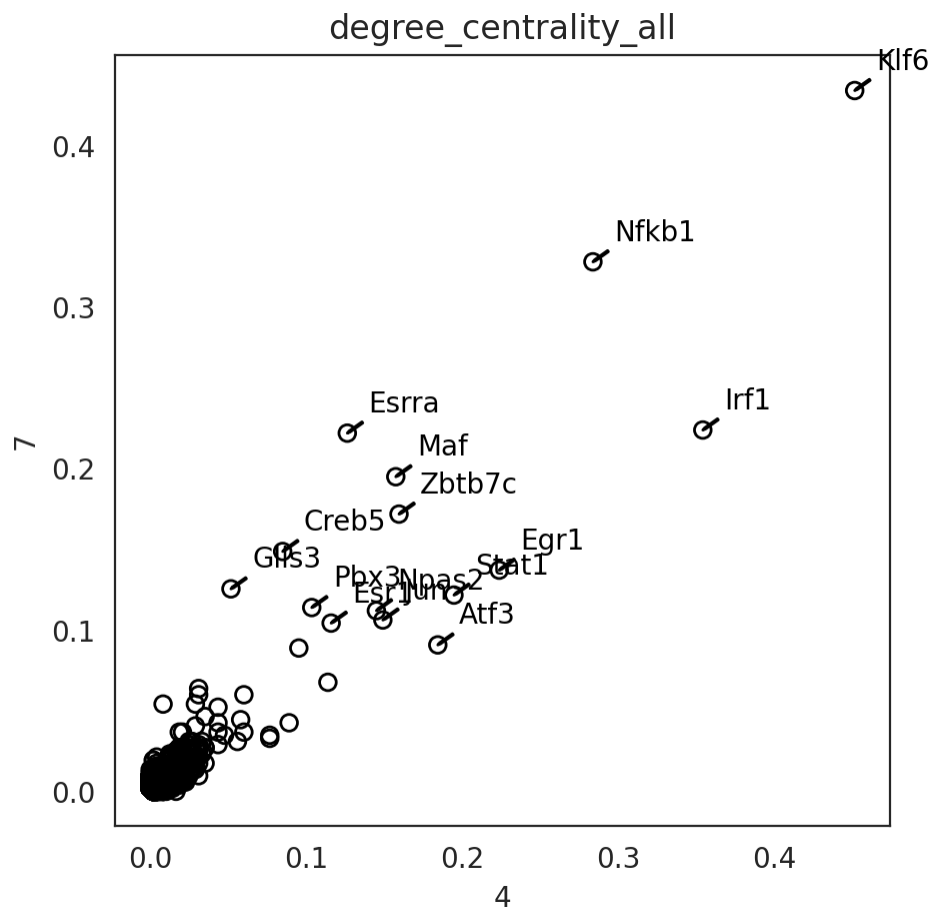


Processing clusters 4 and 5

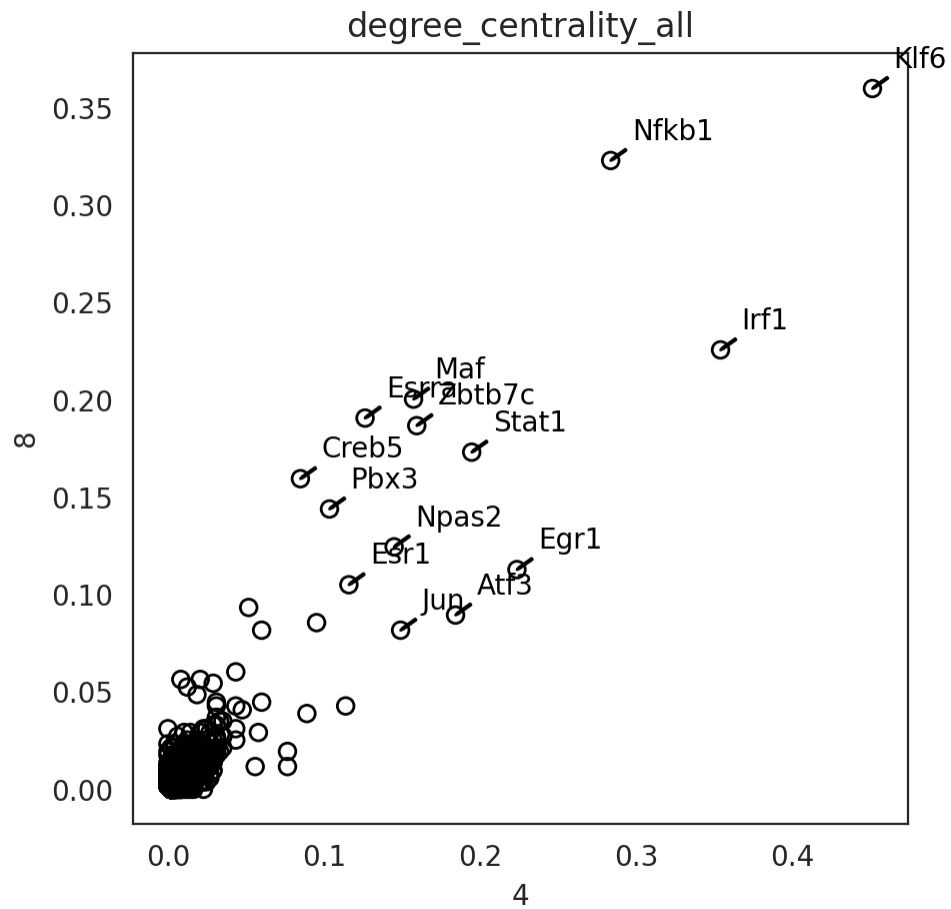


Processing clusters 4 and 6

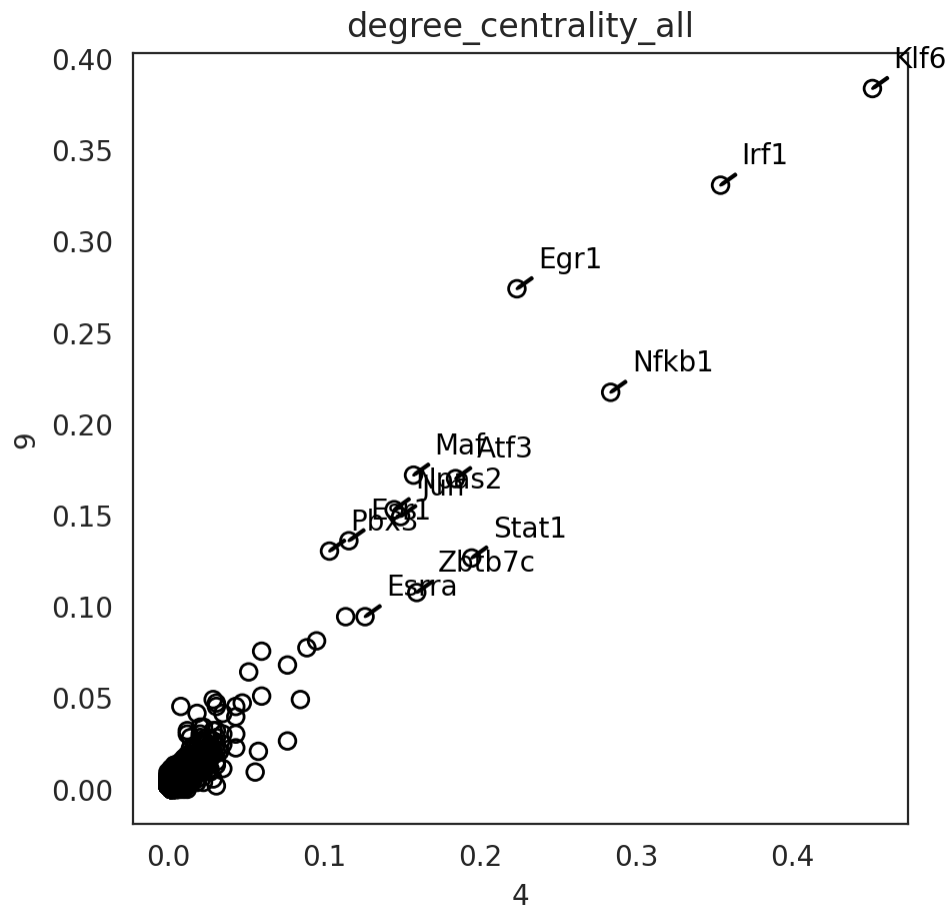


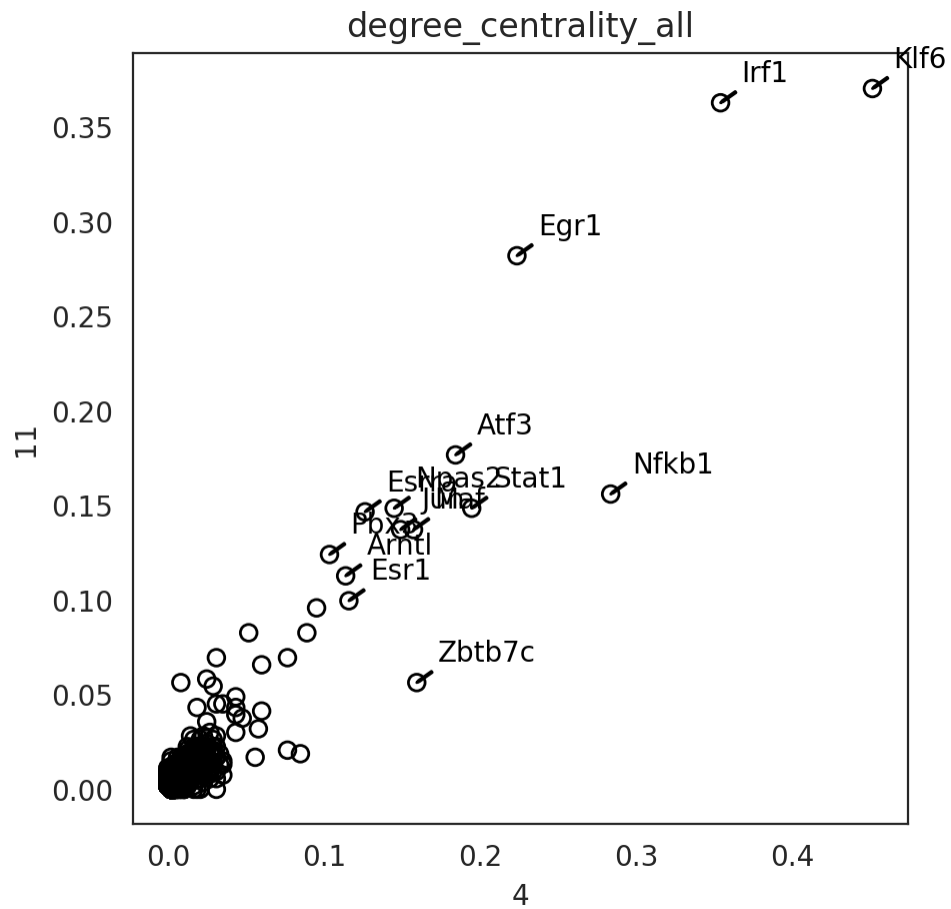


Processing clusters 4 and 8

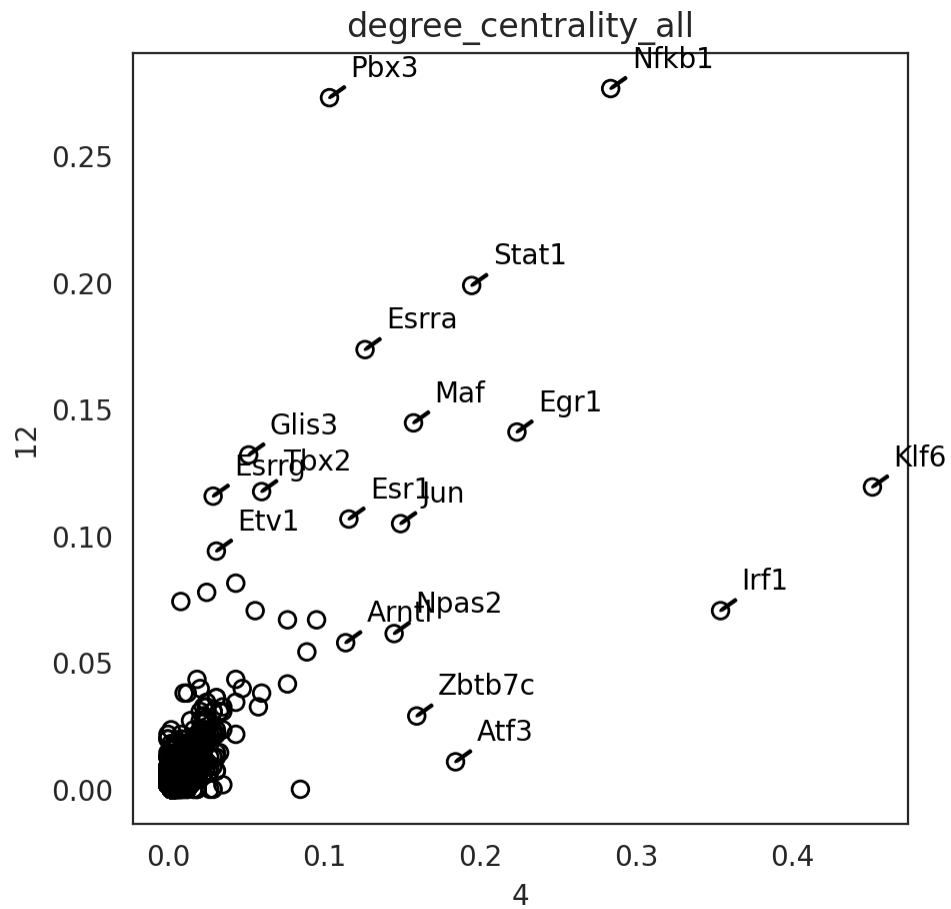


Processing clusters 4 and 9

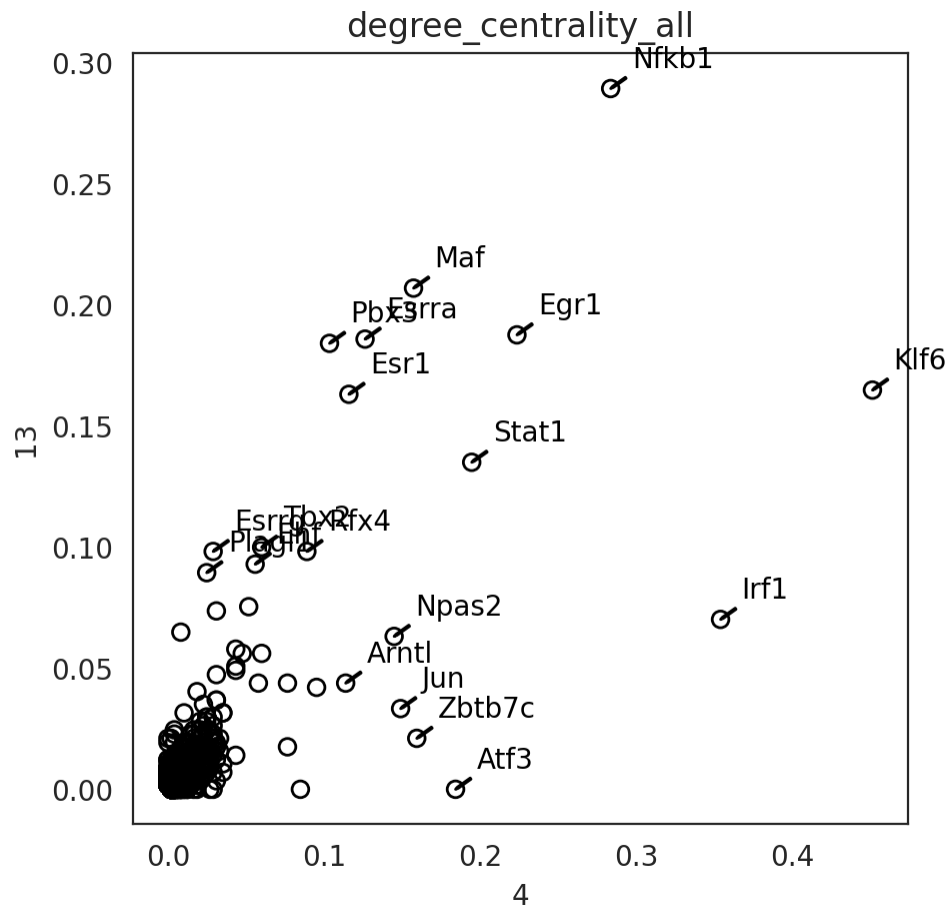




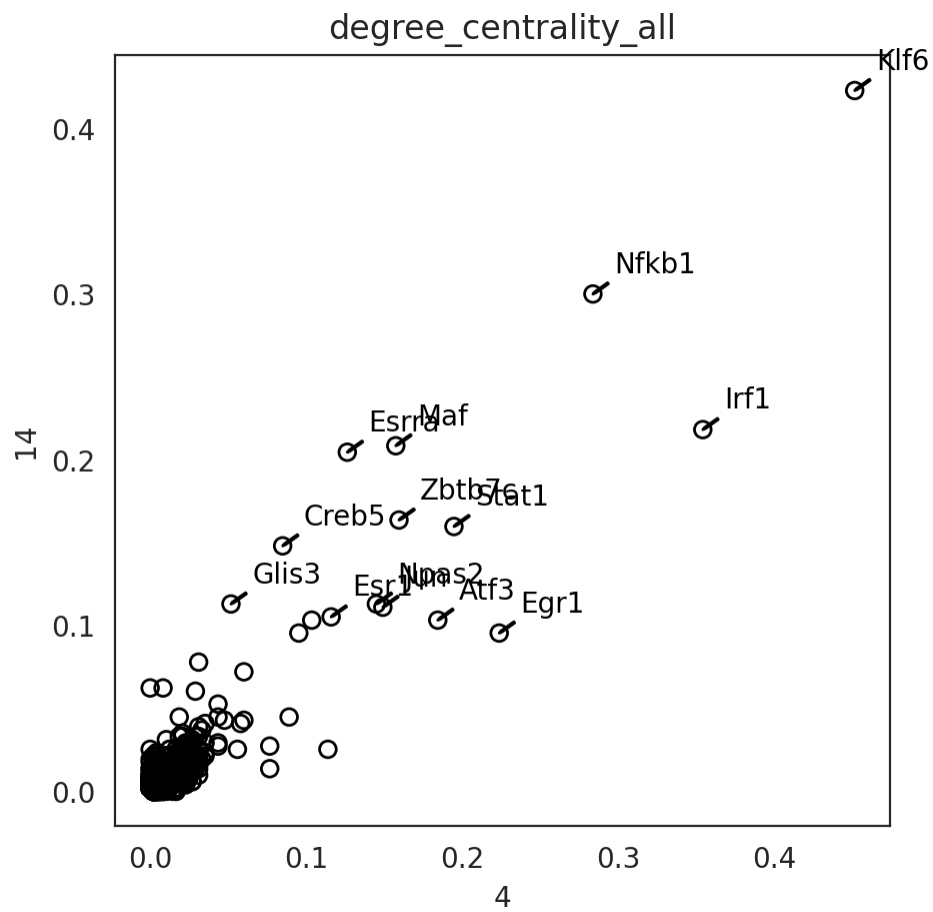
Processing clusters 4 and 12



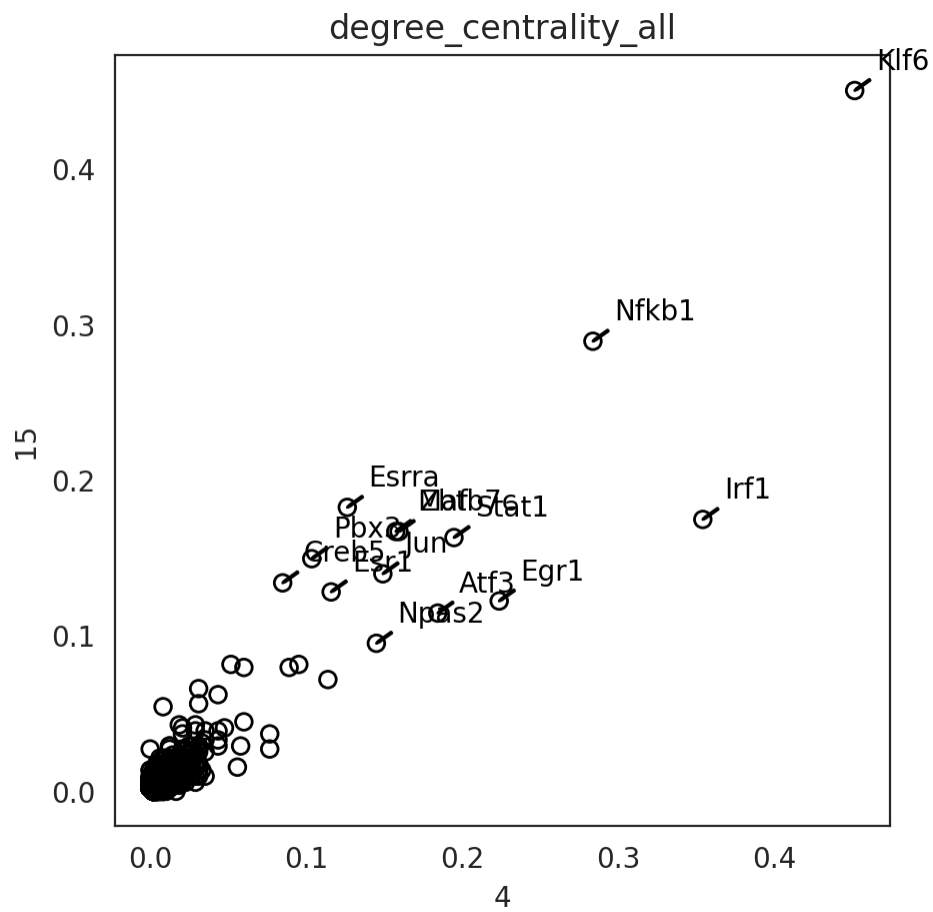
Processing clusters 4 and 13



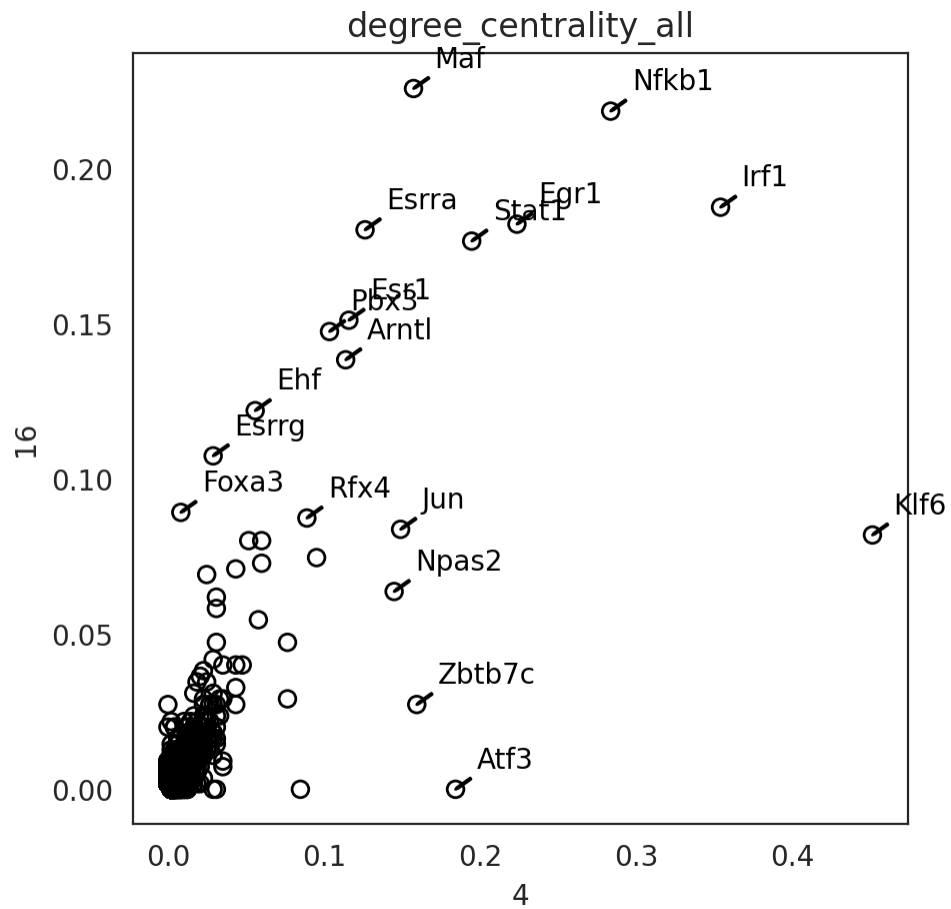
Processing clusters 4 and 14



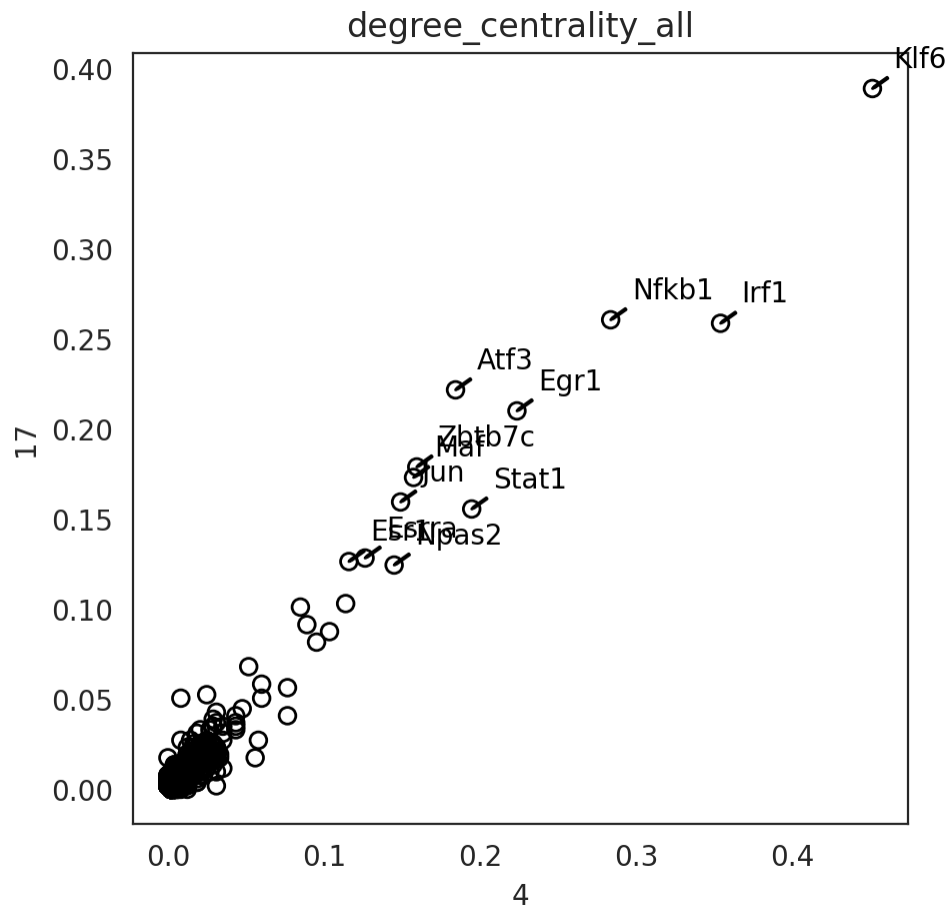
Processing clusters 4 and 15



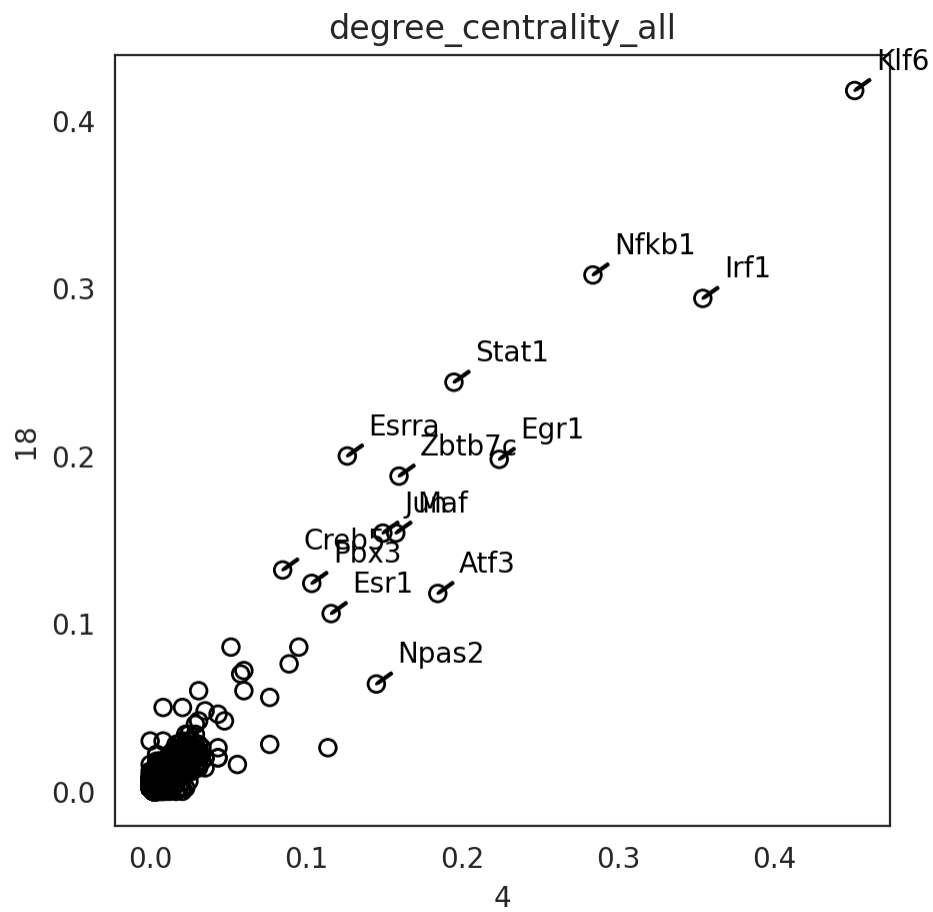
Processing clusters 4 and 16



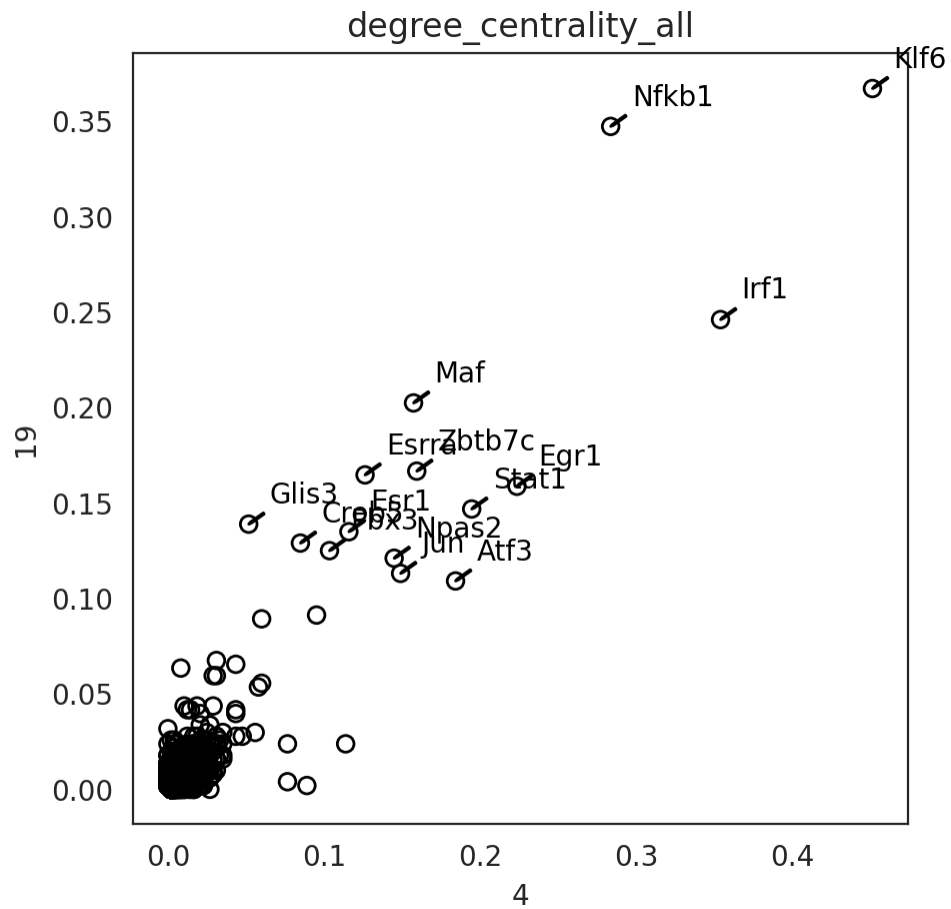
Processing clusters 4 and 17



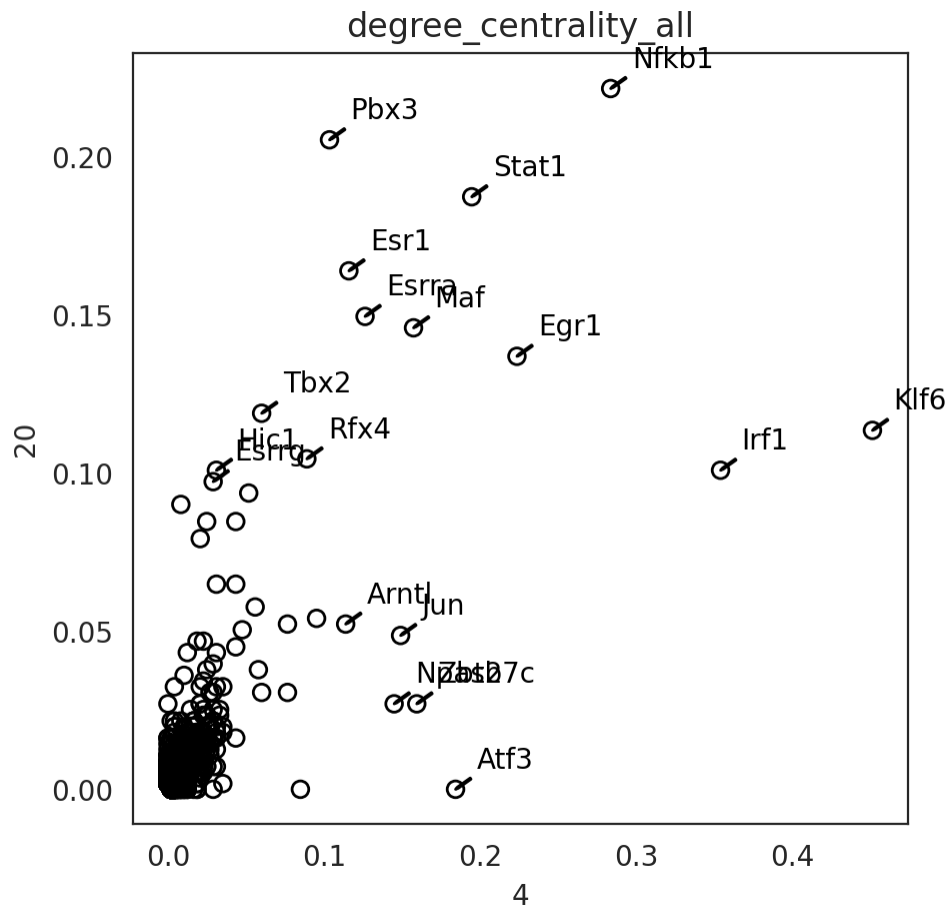
Processing clusters 4 and 18



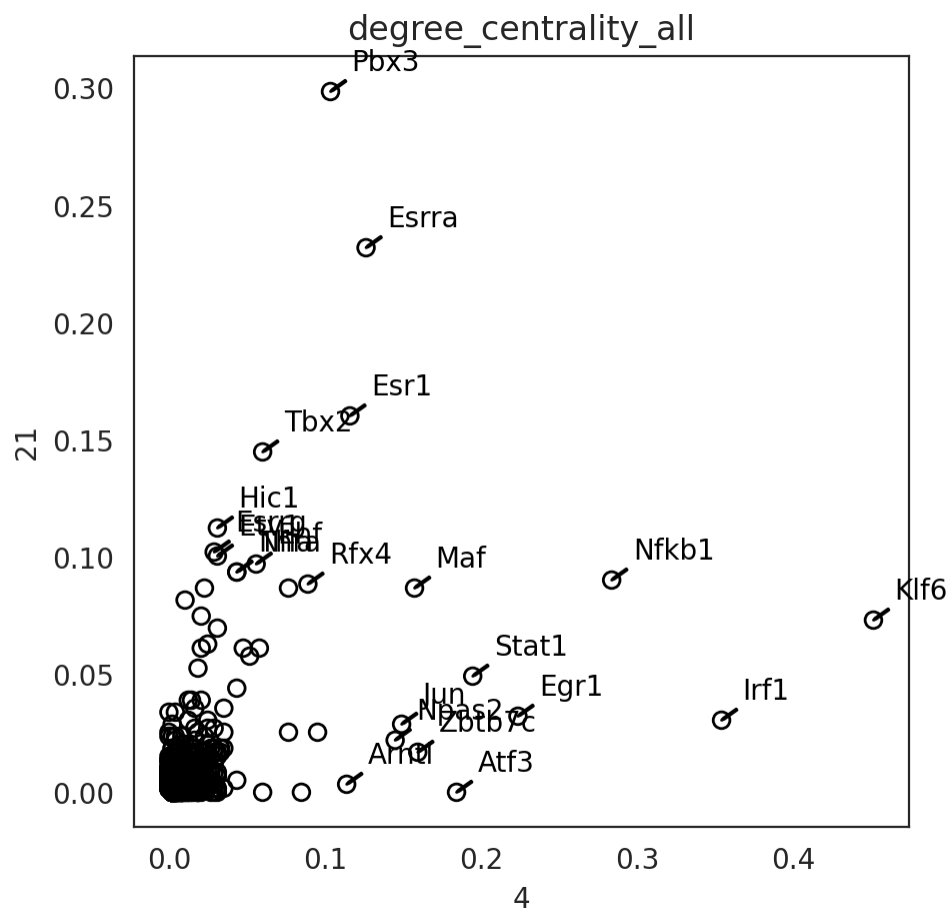
Processing clusters 4 and 19

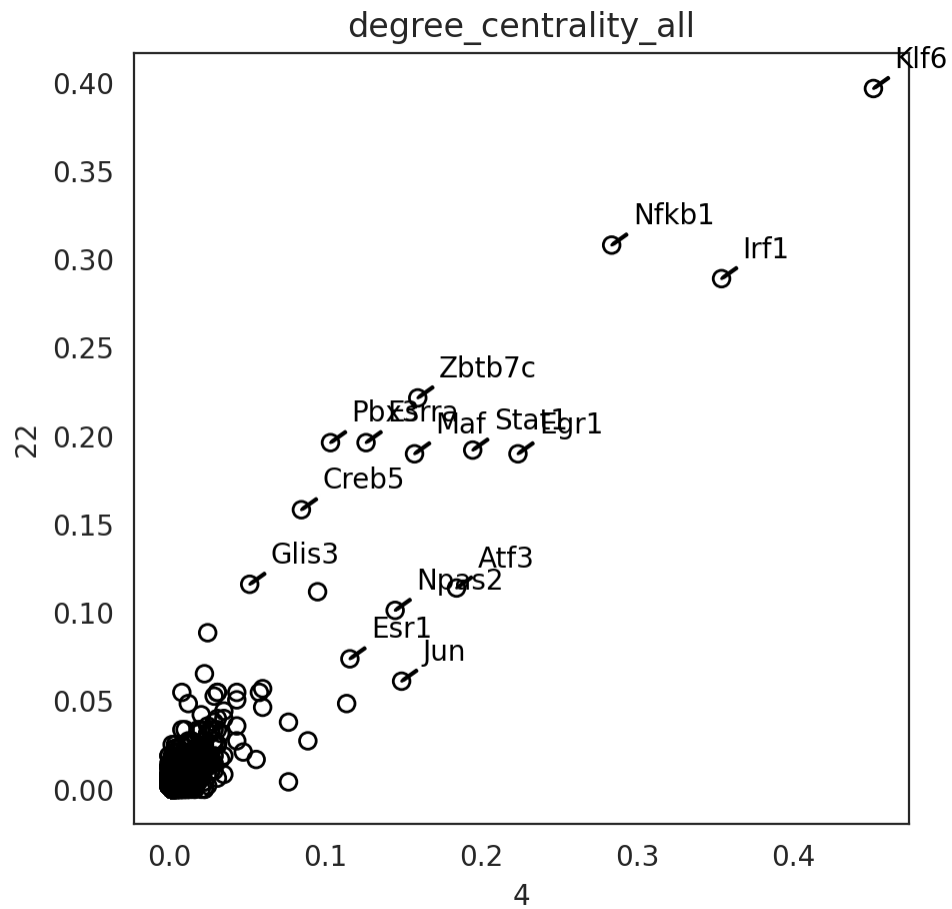


Processing clusters 4 and 20

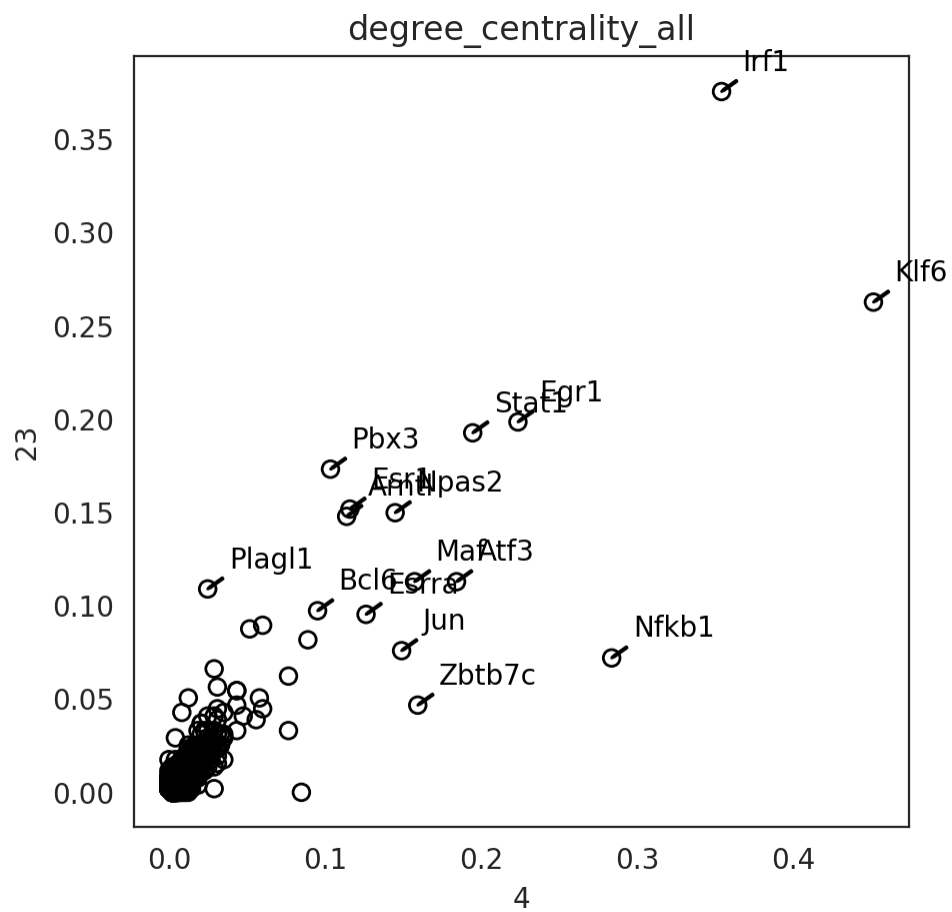


Processing clusters 4 and 21

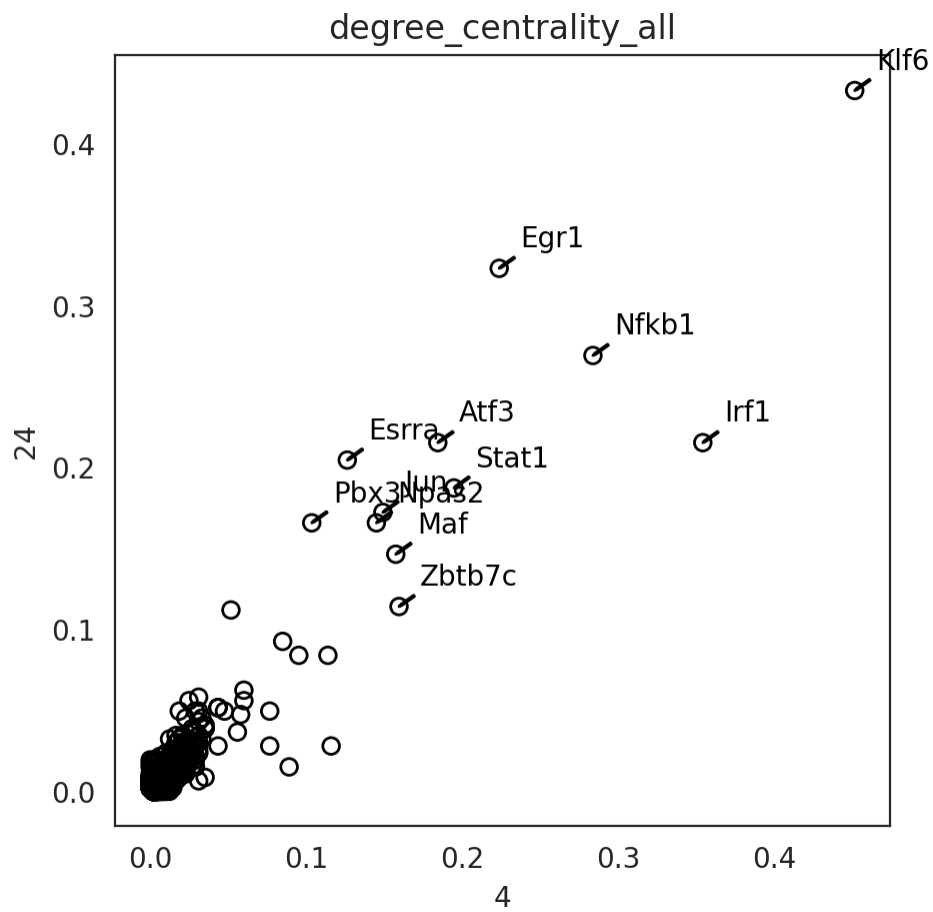




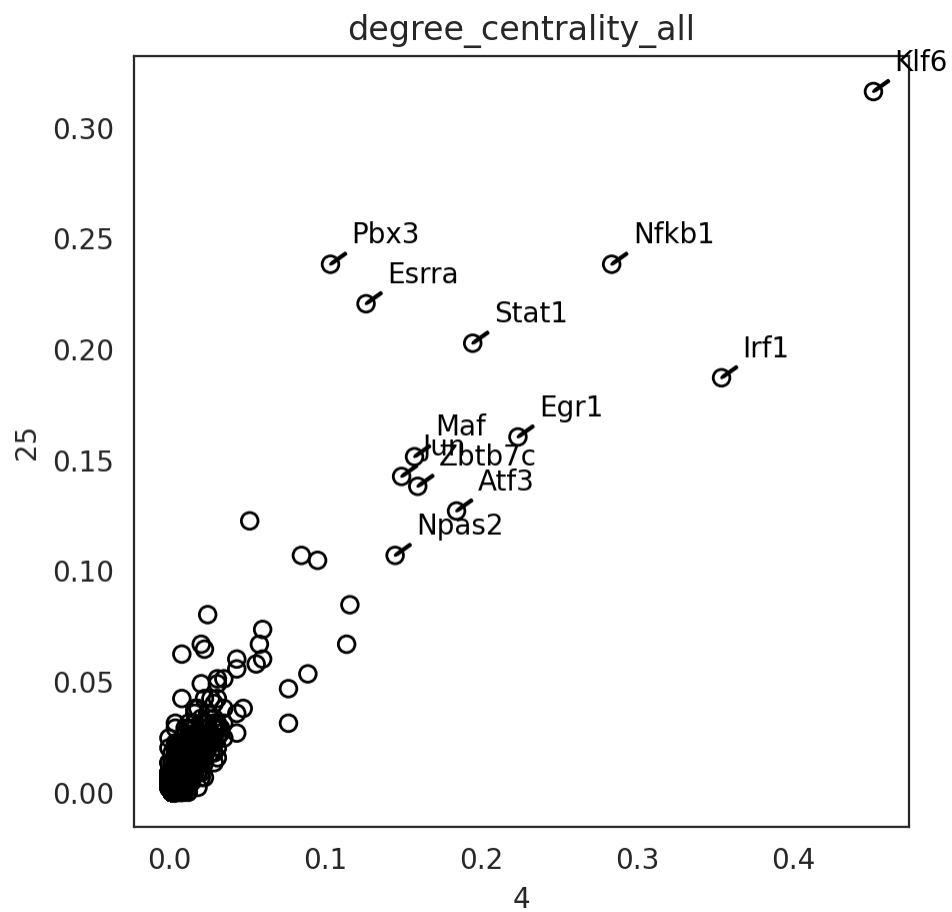
Processing clusters 4 and 23



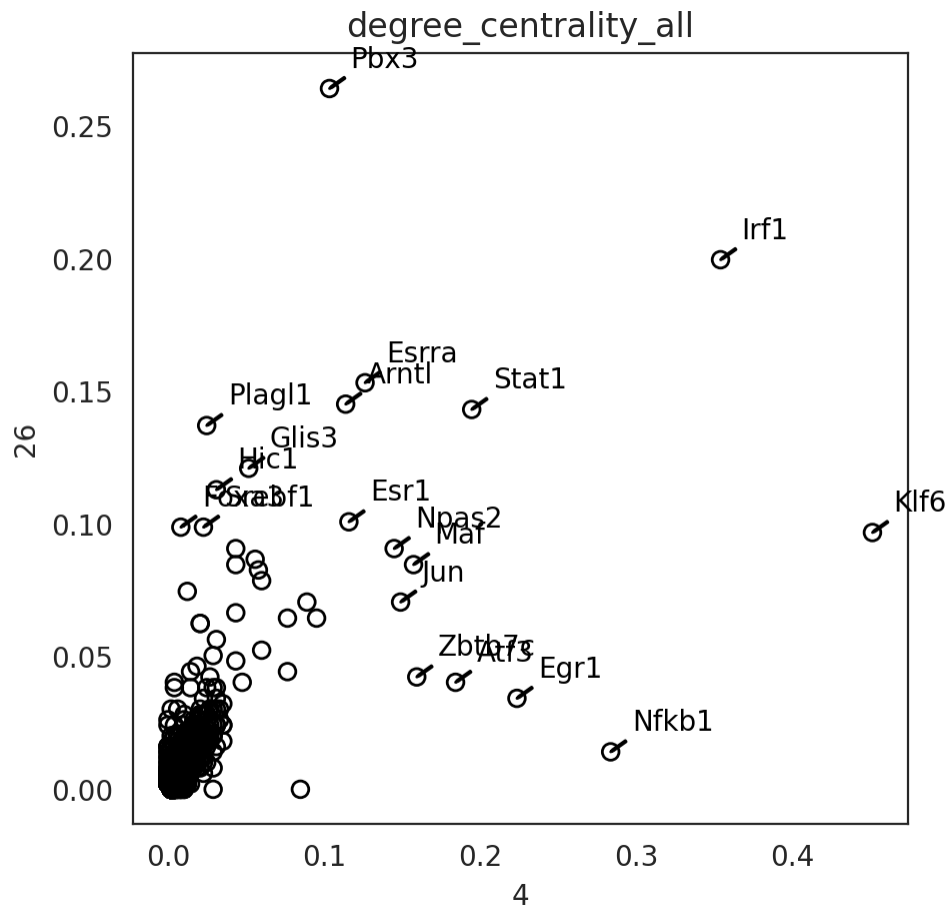
Processing clusters 4 and 24



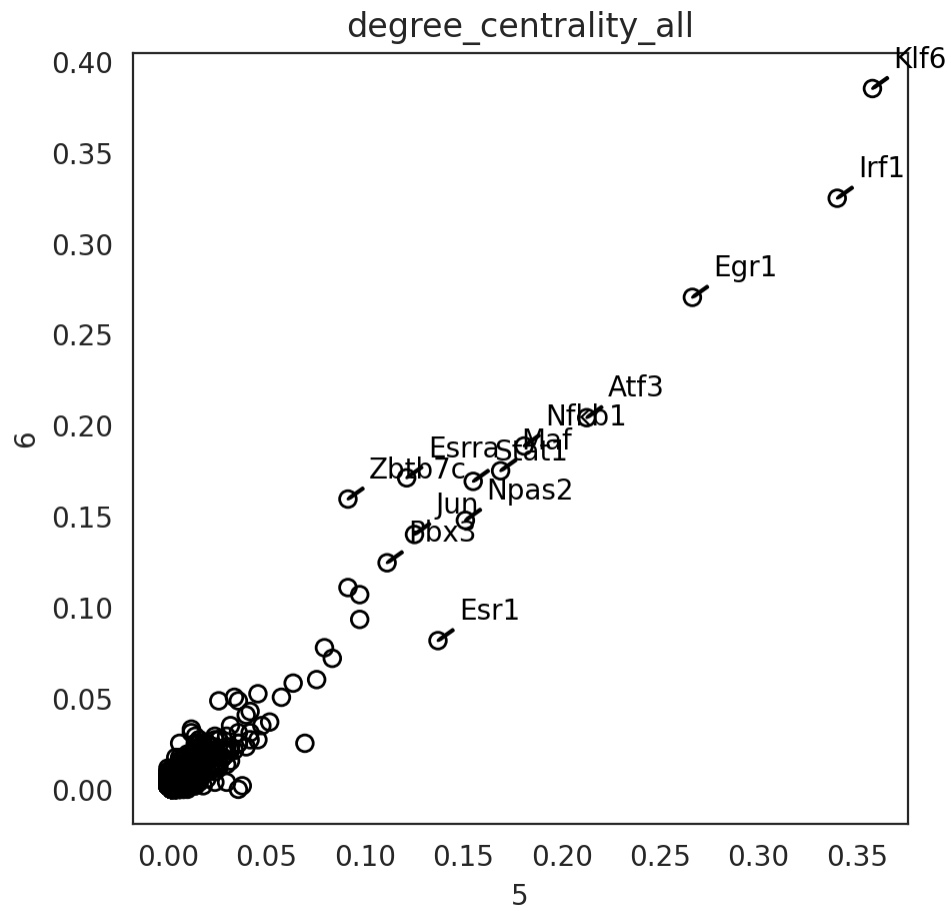
Processing clusters 4 and 25

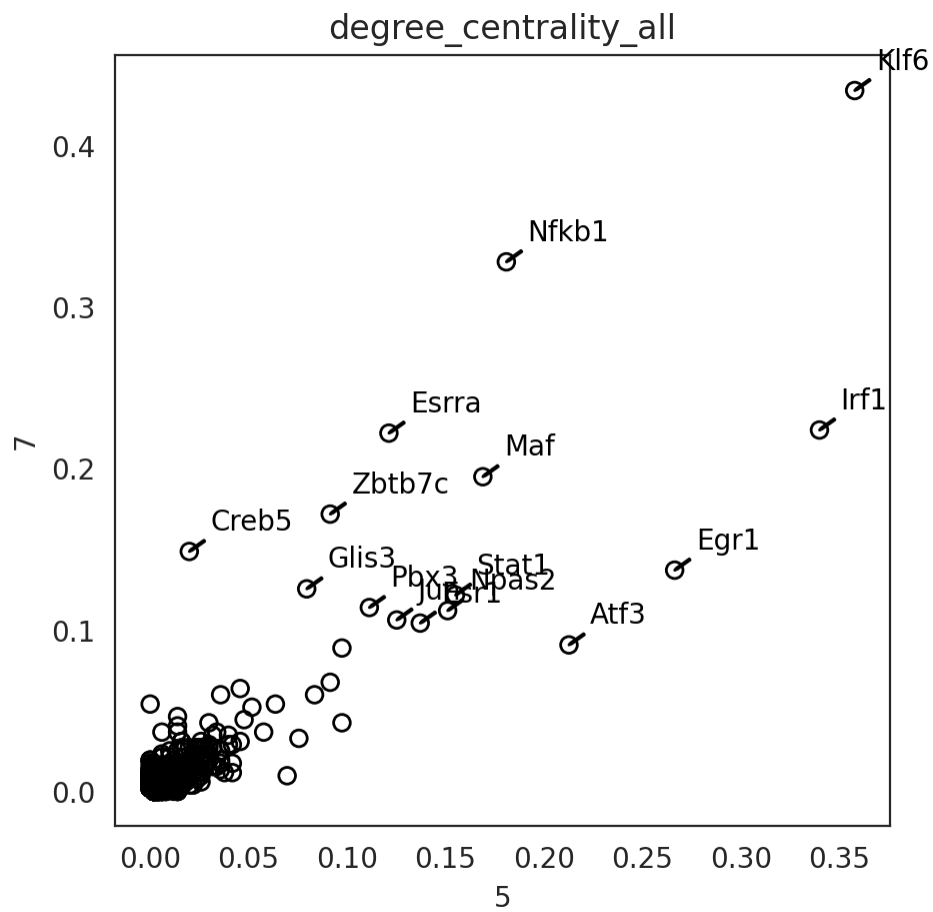


Processing clusters 4 and 26

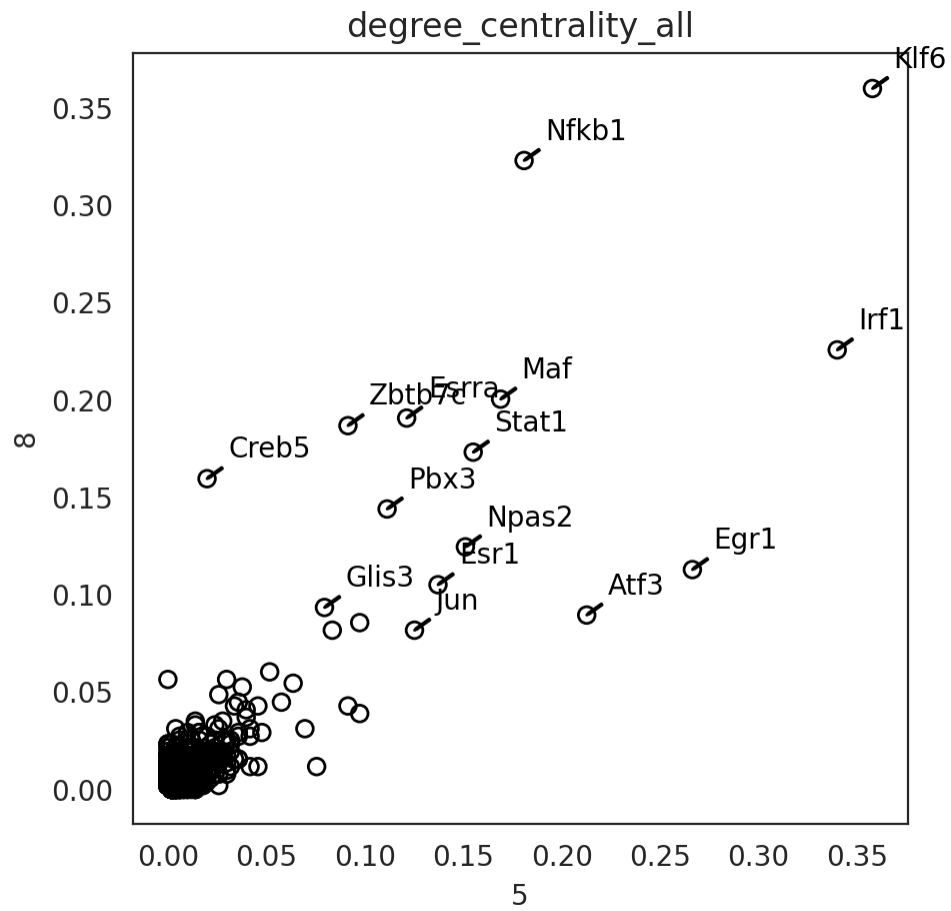


Processing clusters 5 and 6

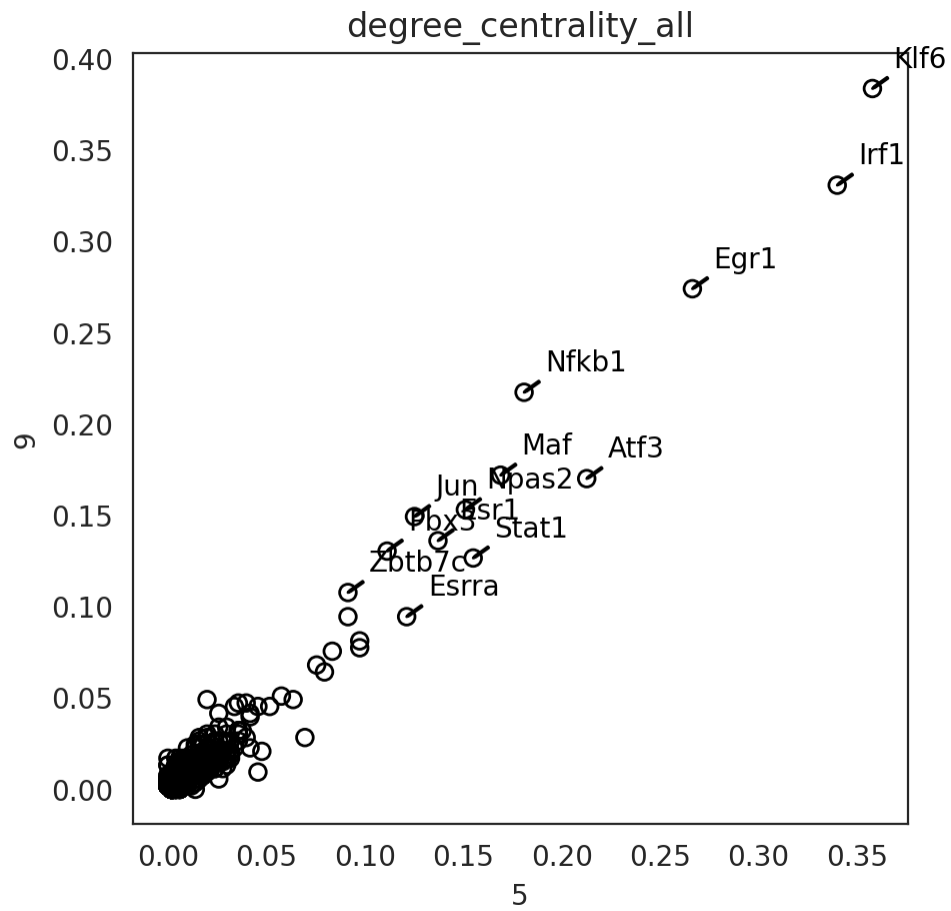




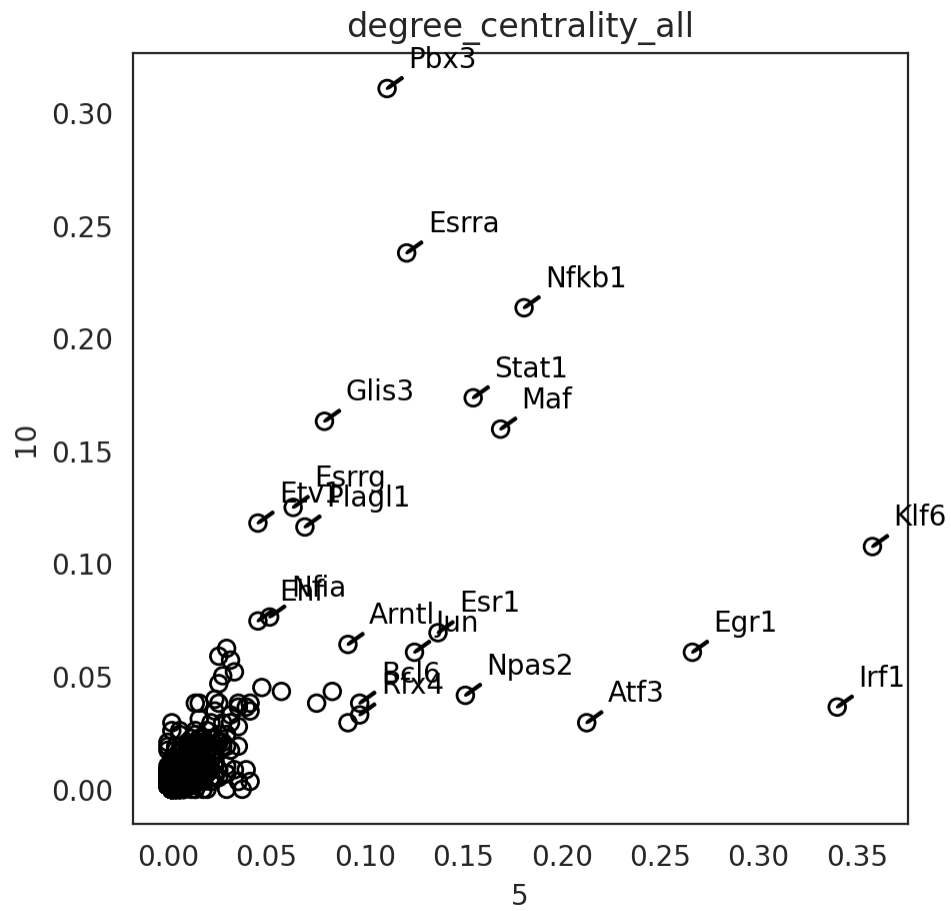
Processing clusters 5 and 8



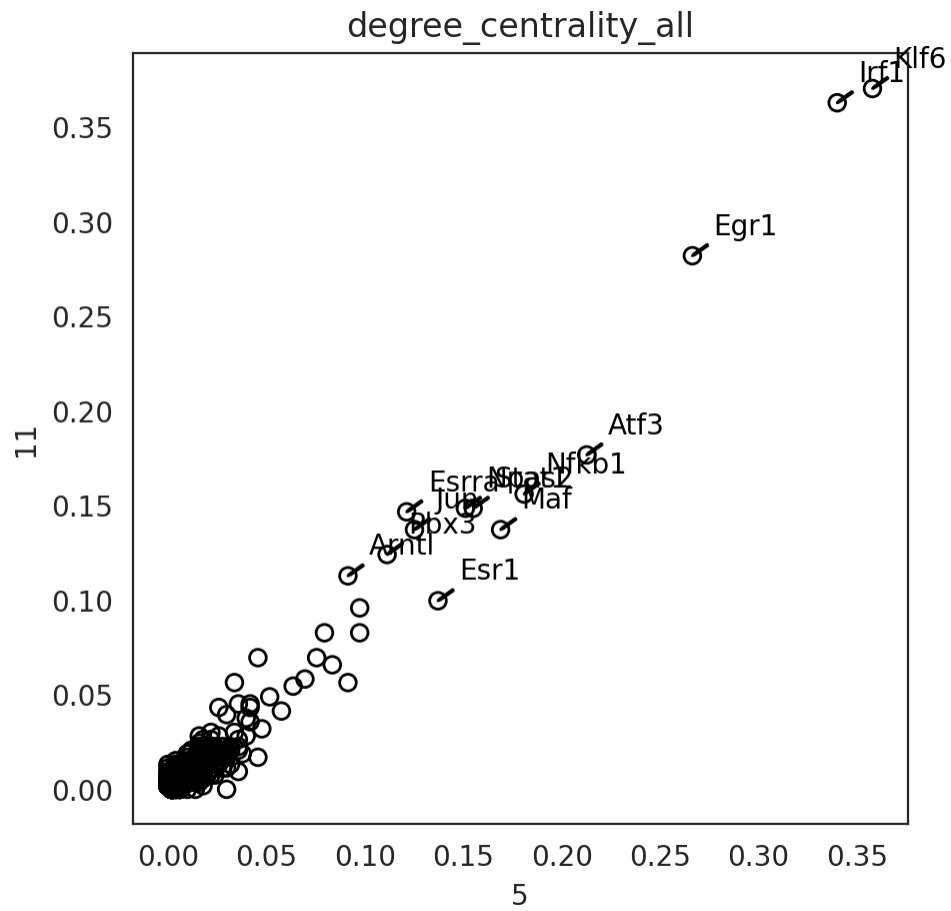
Processing clusters 5 and 9



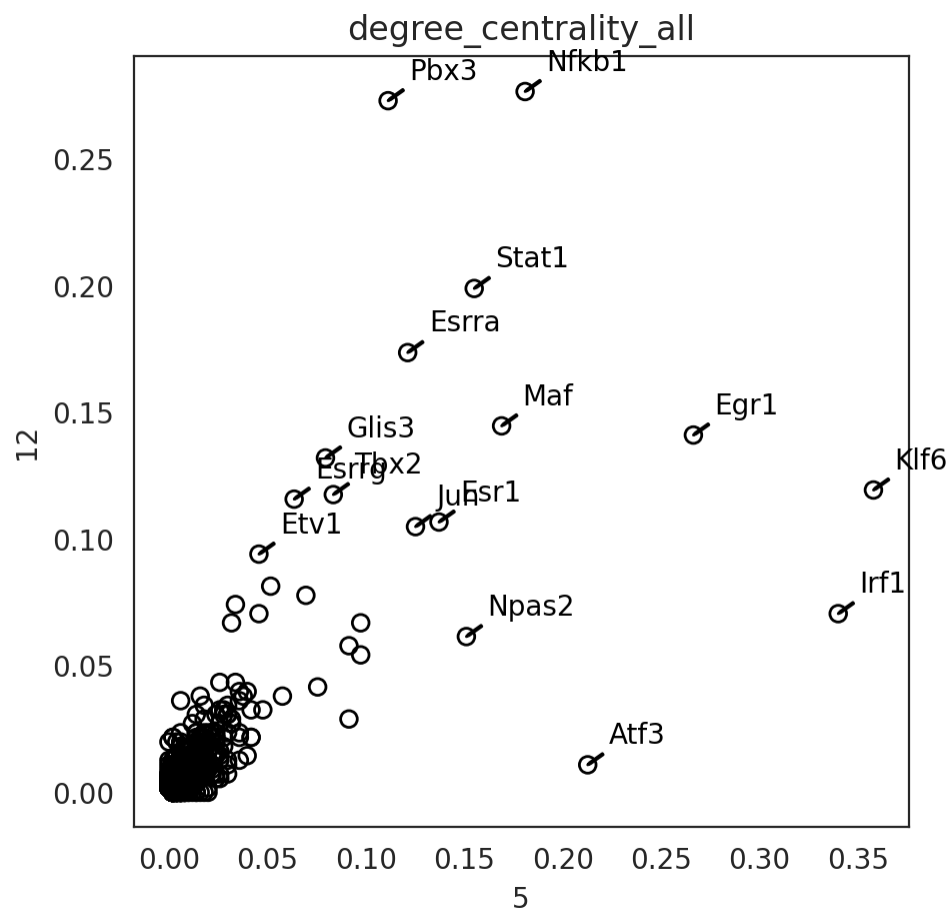
Processing clusters 5 and 10



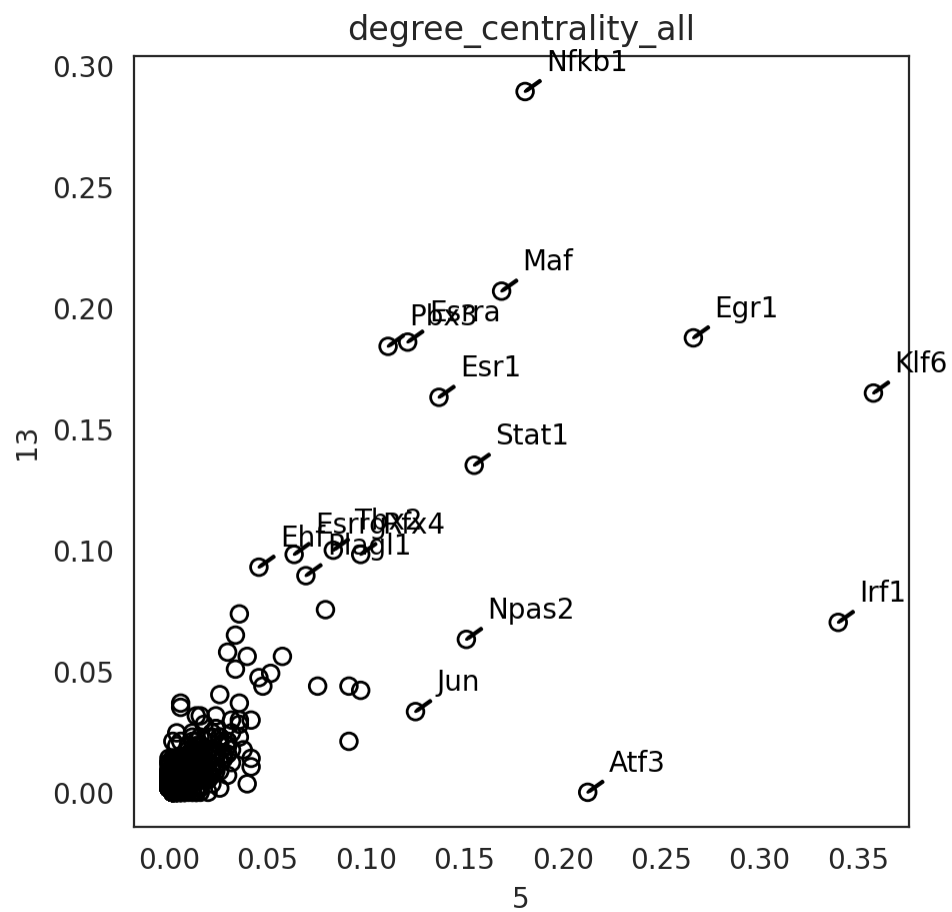
Processing clusters 5 and 11



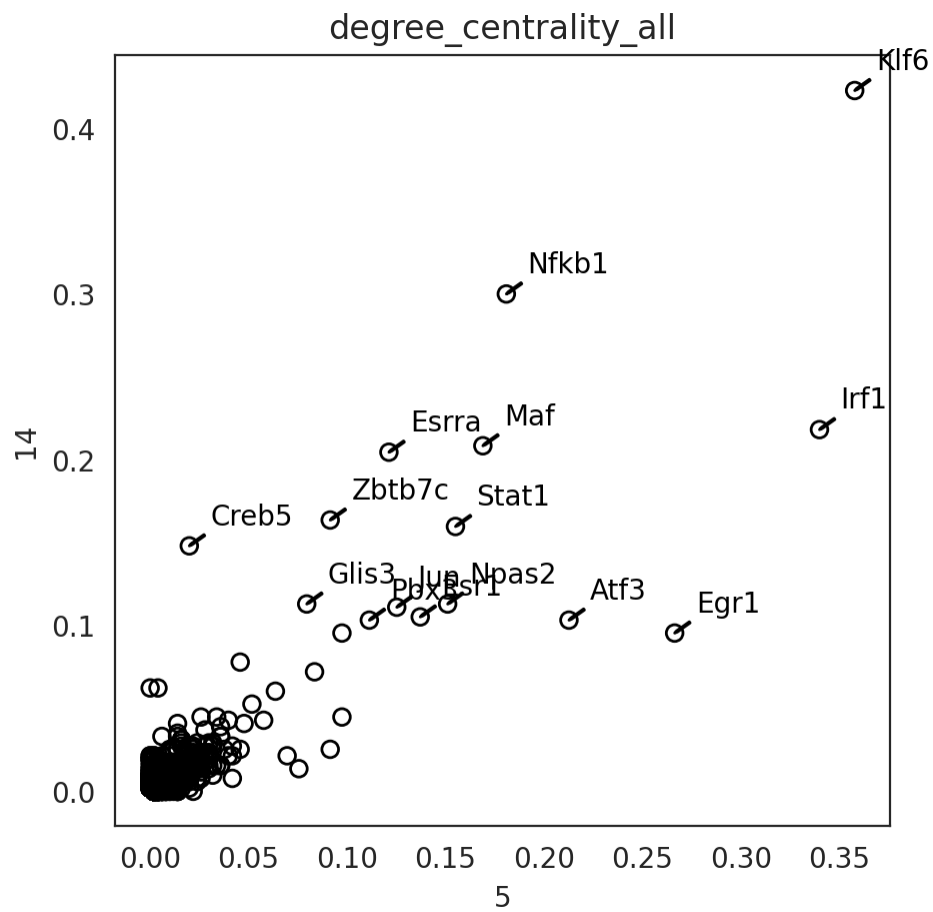
Processing clusters 5 and 12



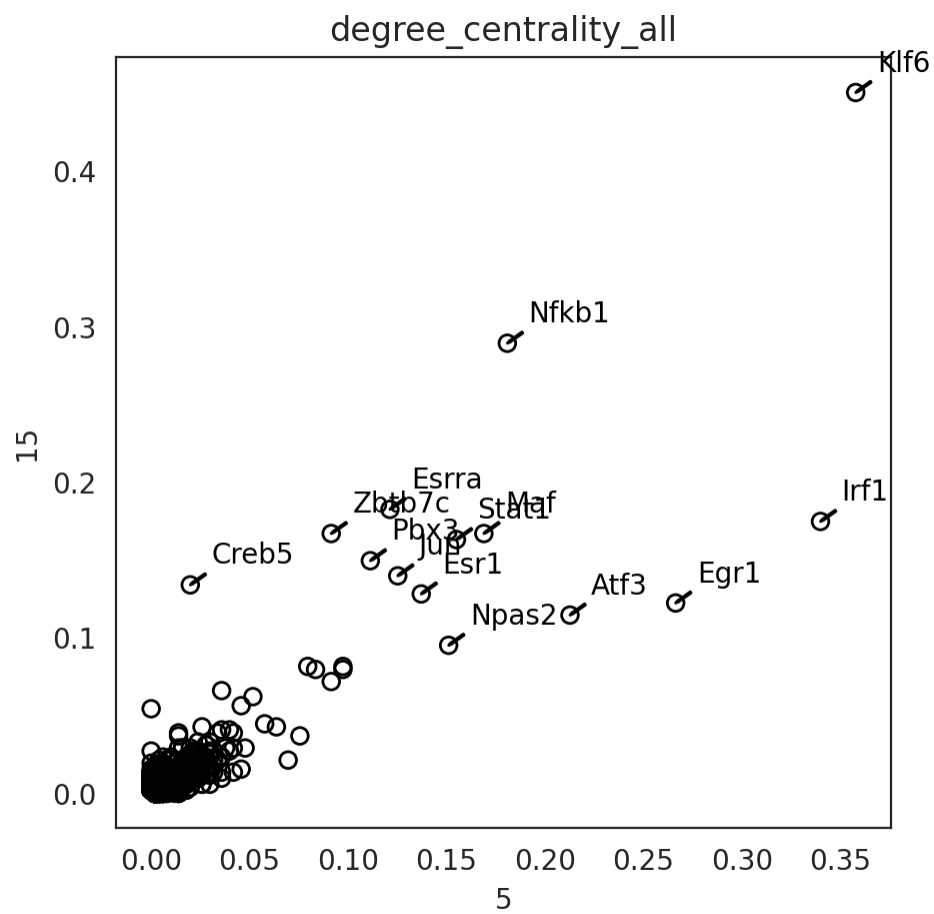
Processing clusters 5 and 13



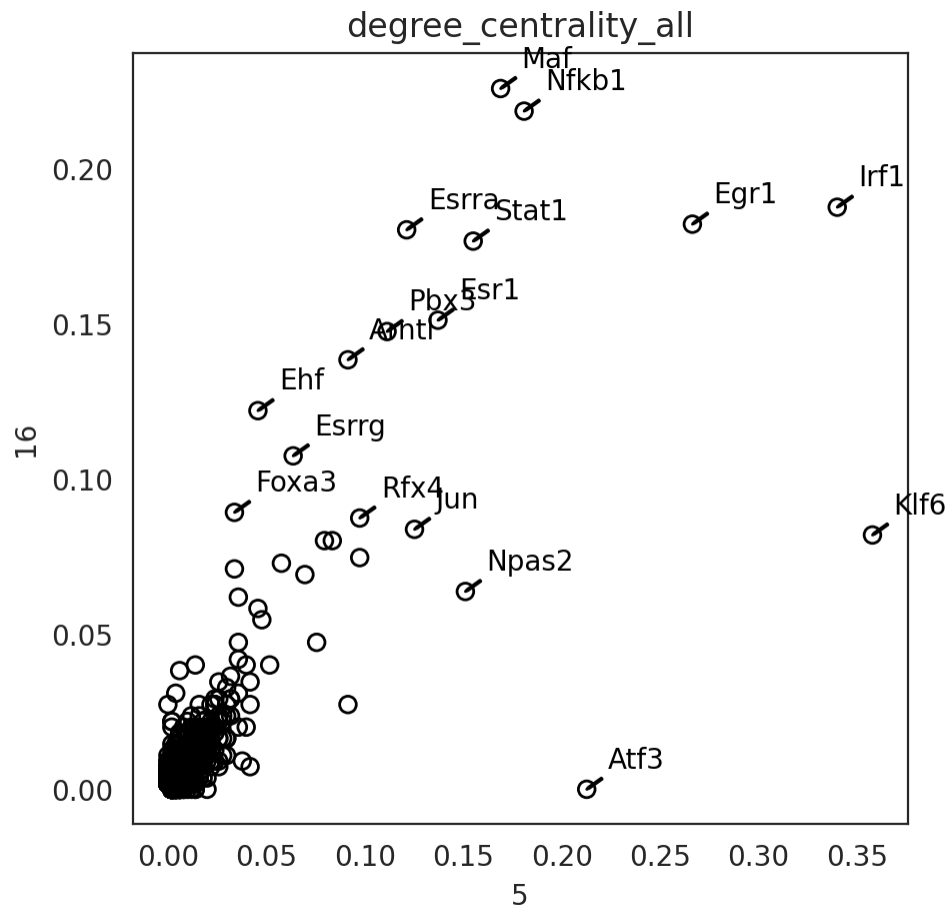
Processing clusters 5 and 14



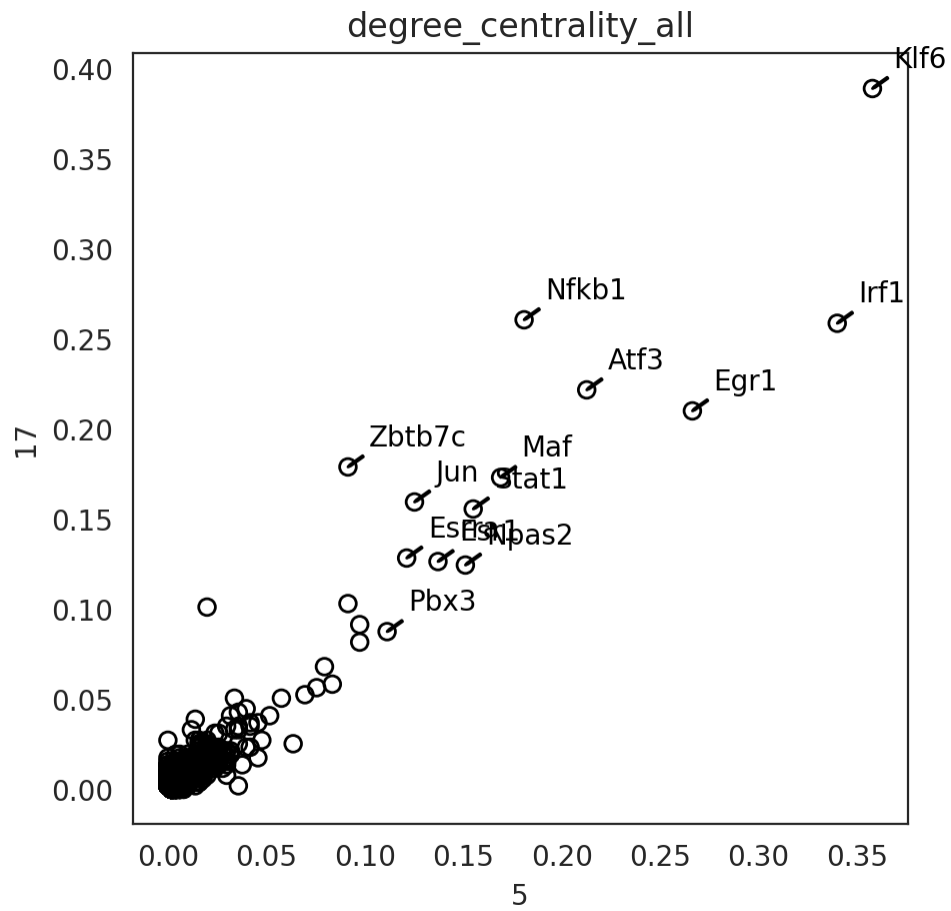
Processing clusters 5 and 15



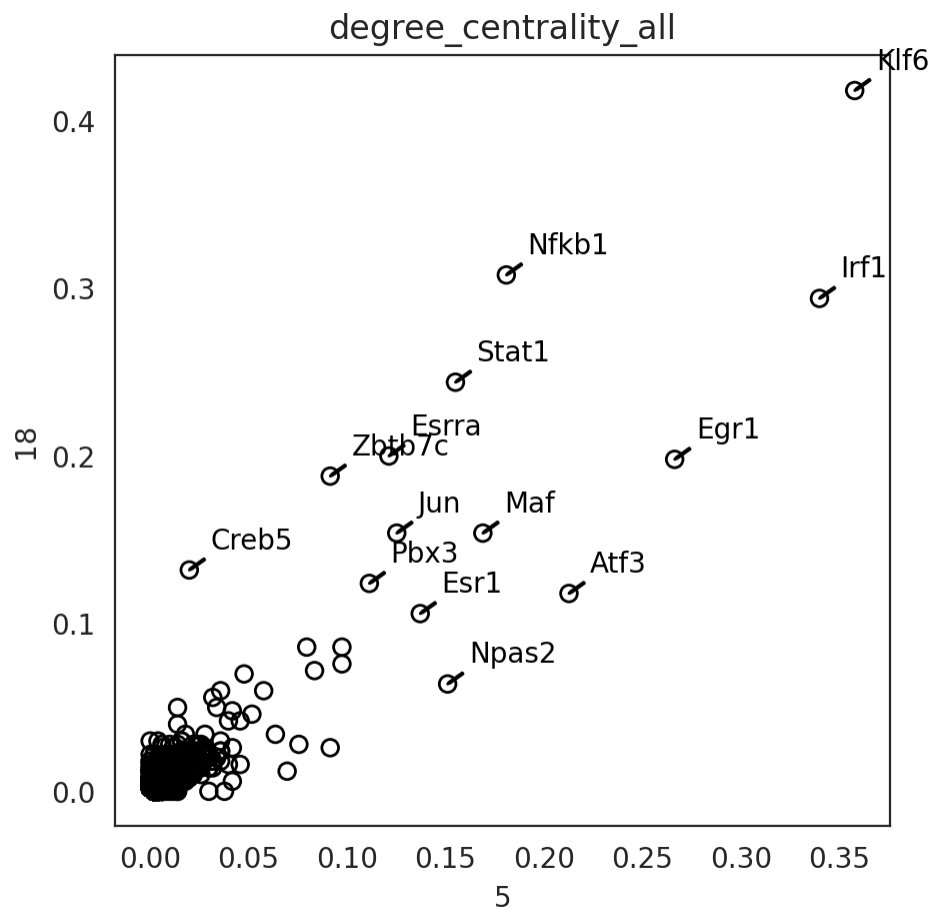
Processing clusters 5 and 16



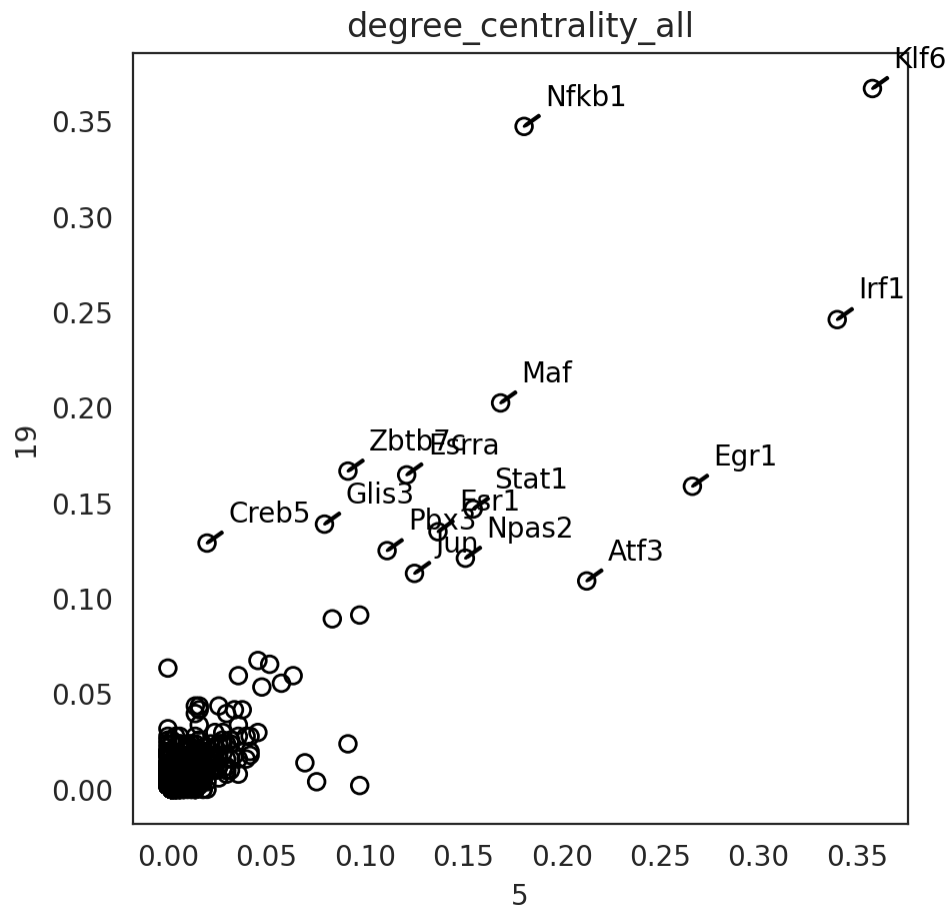
Processing clusters 5 and 17



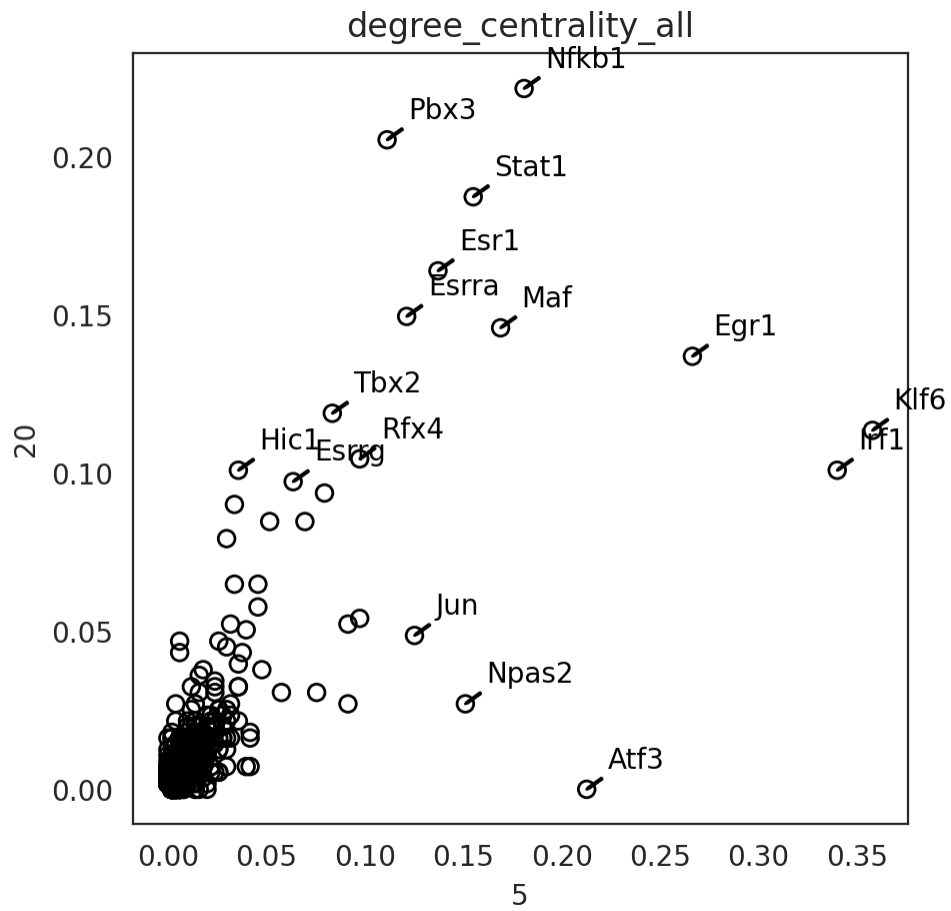
Processing clusters 5 and 18



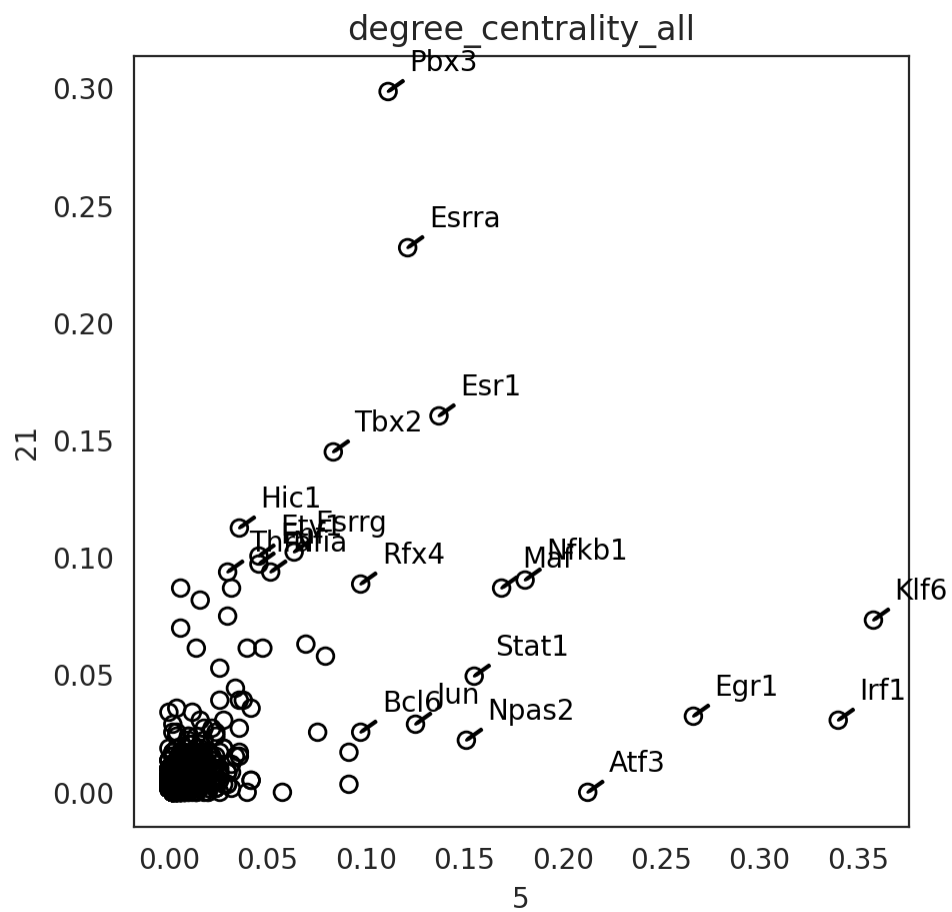
Processing clusters 5 and 19



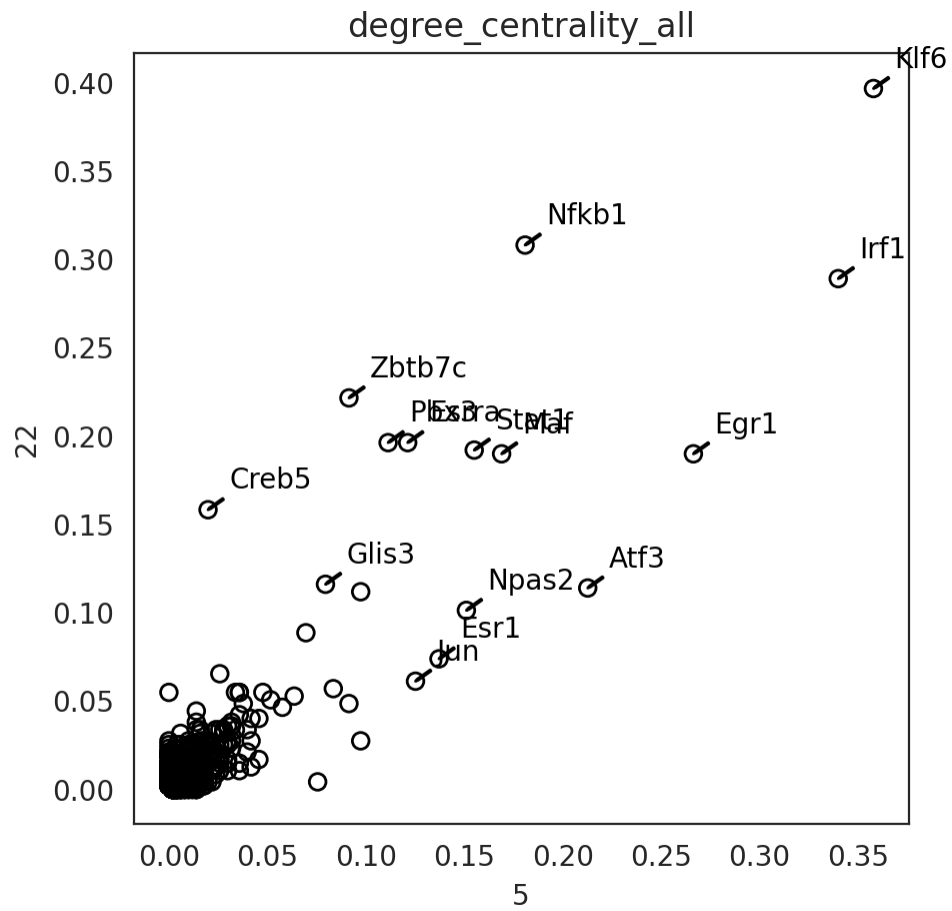
Processing clusters 5 and 20



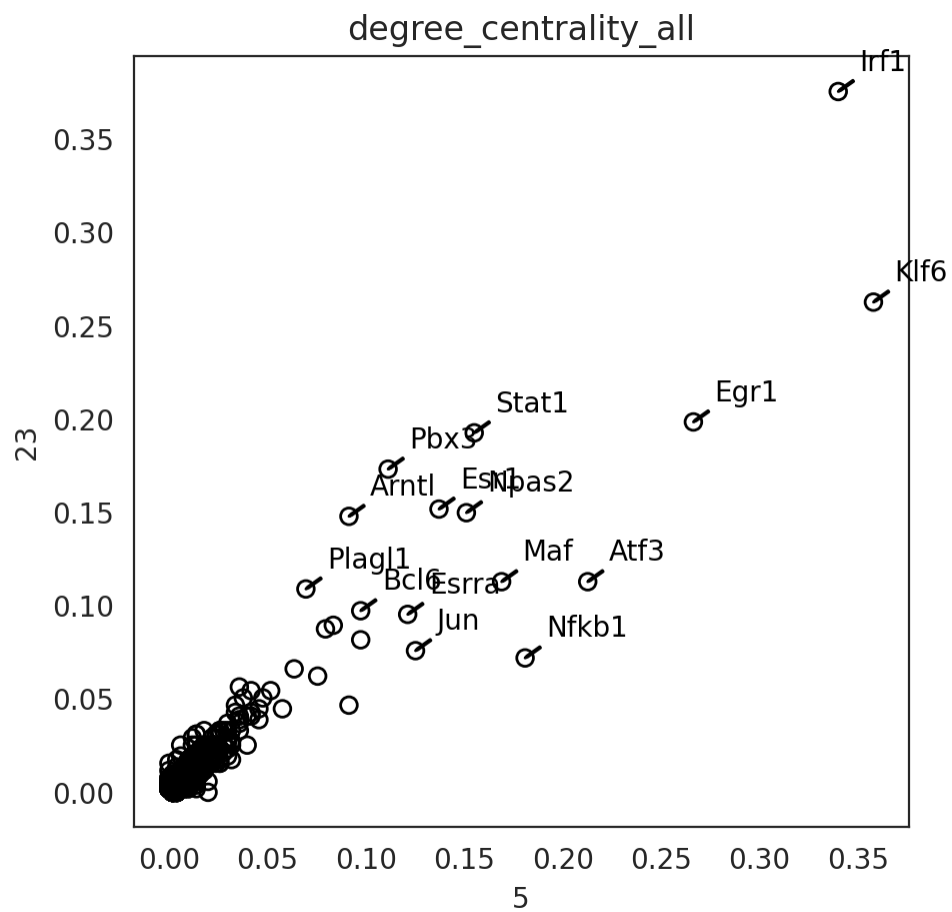
Processing clusters 5 and 21



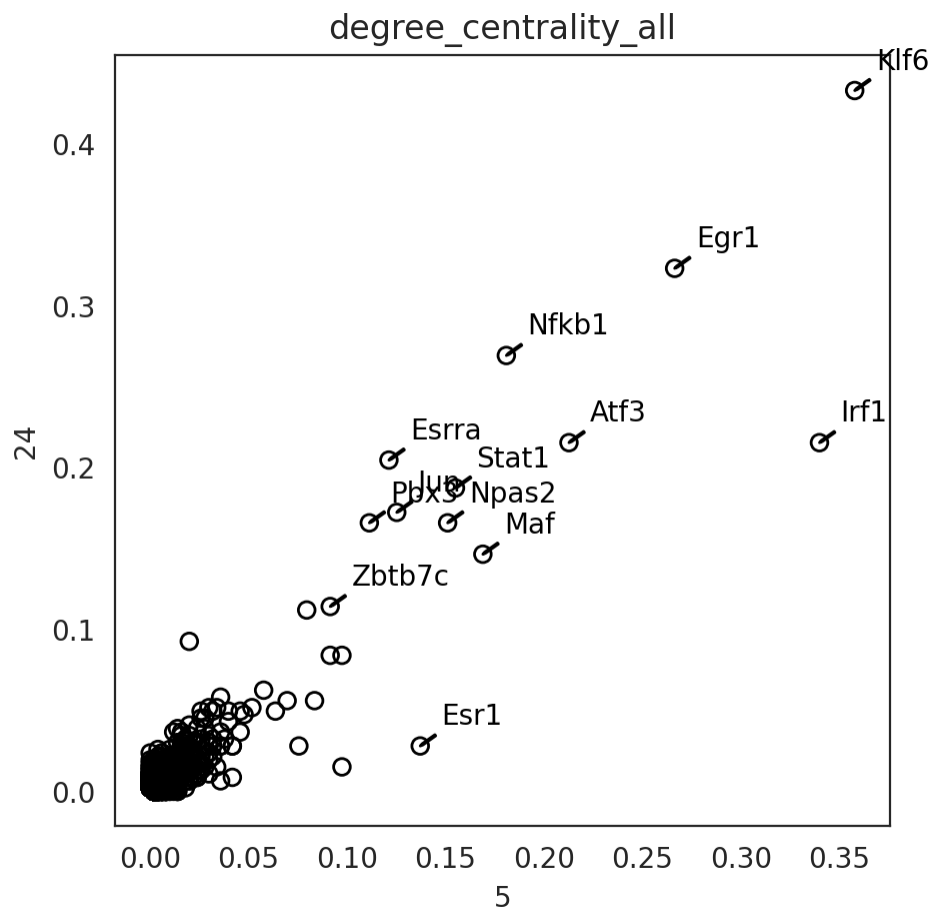
Processing clusters 5 and 22



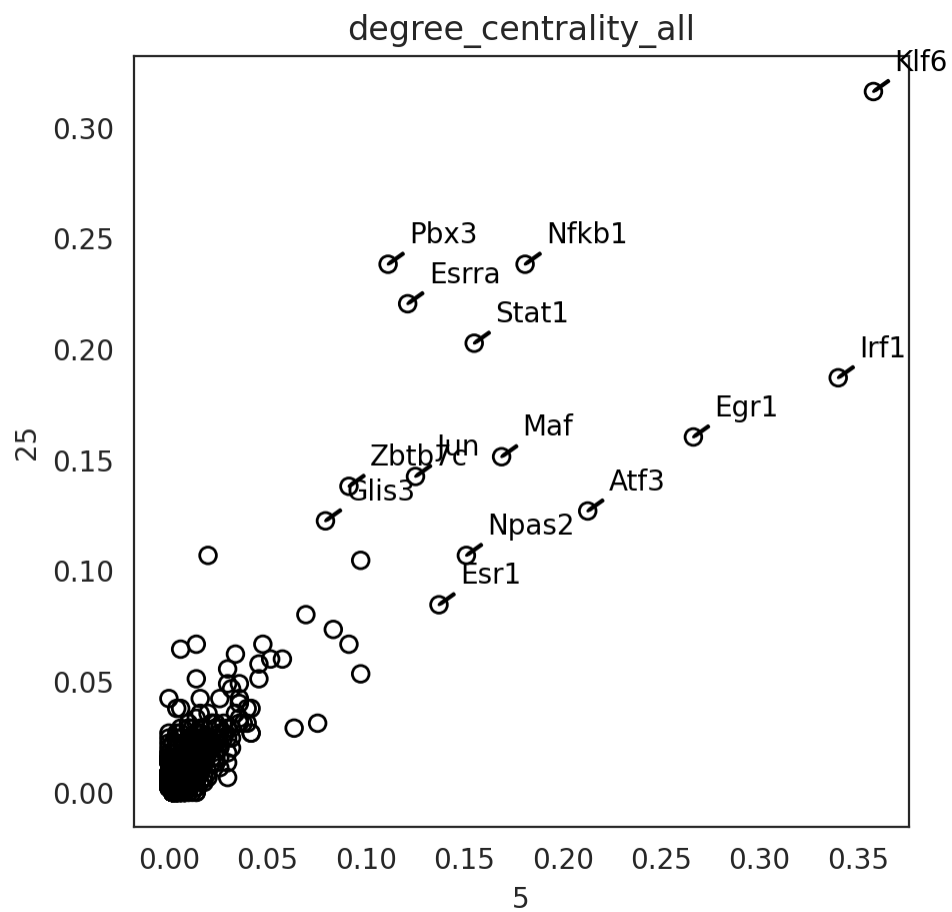
Processing clusters 5 and 23



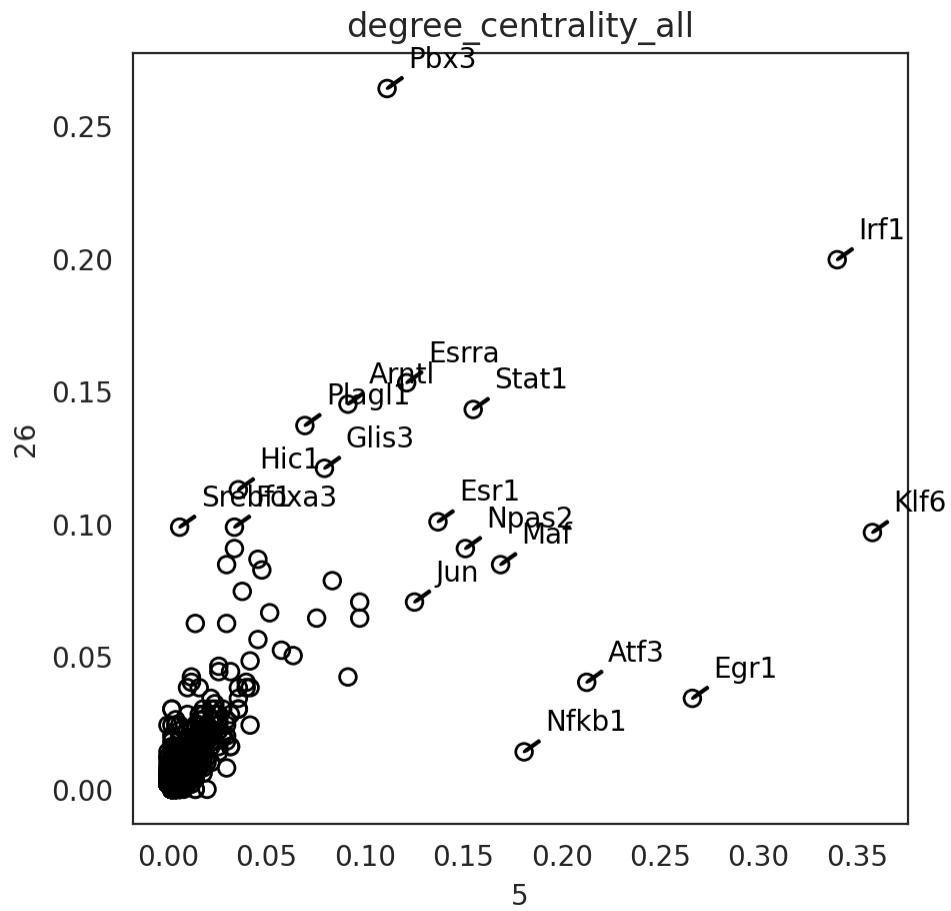
Processing clusters 5 and 24



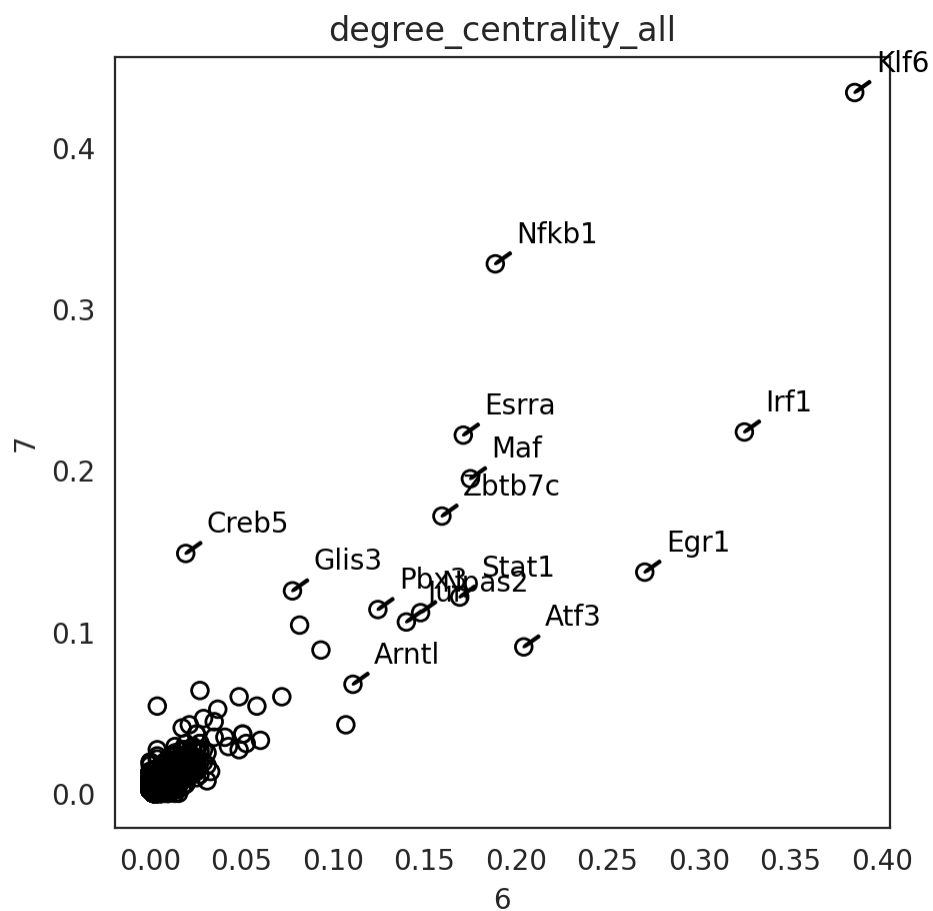
Processing clusters 5 and 25



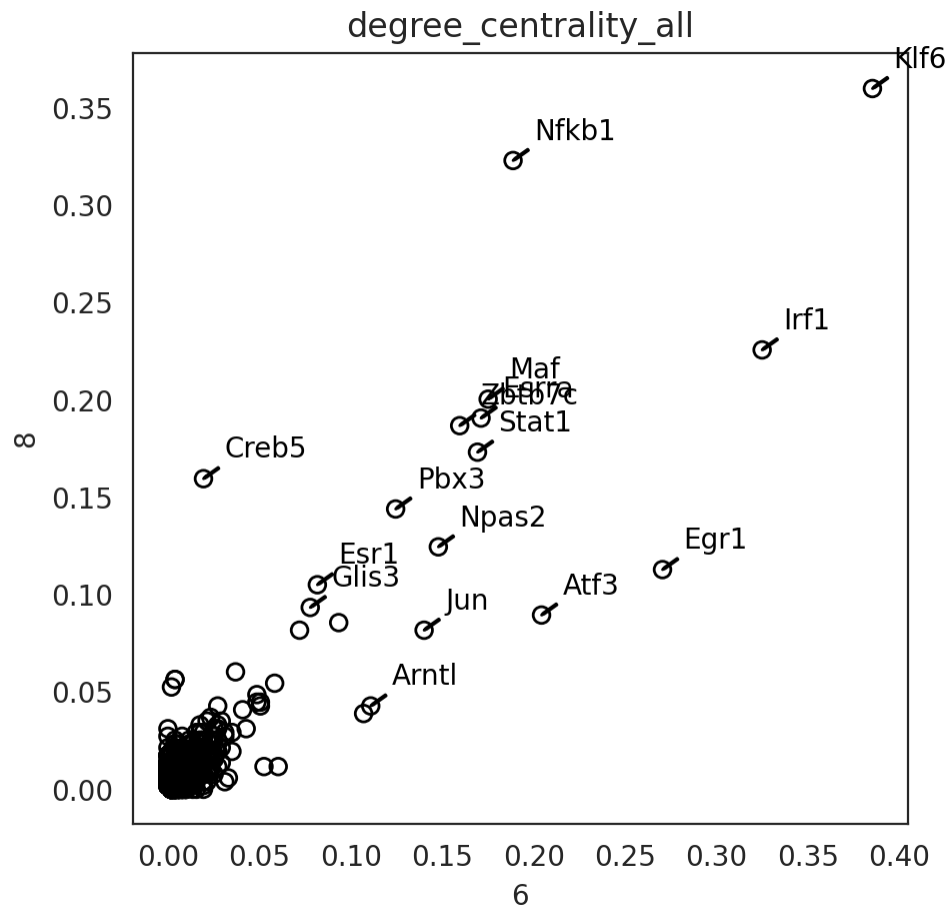
Processing clusters 5 and 26



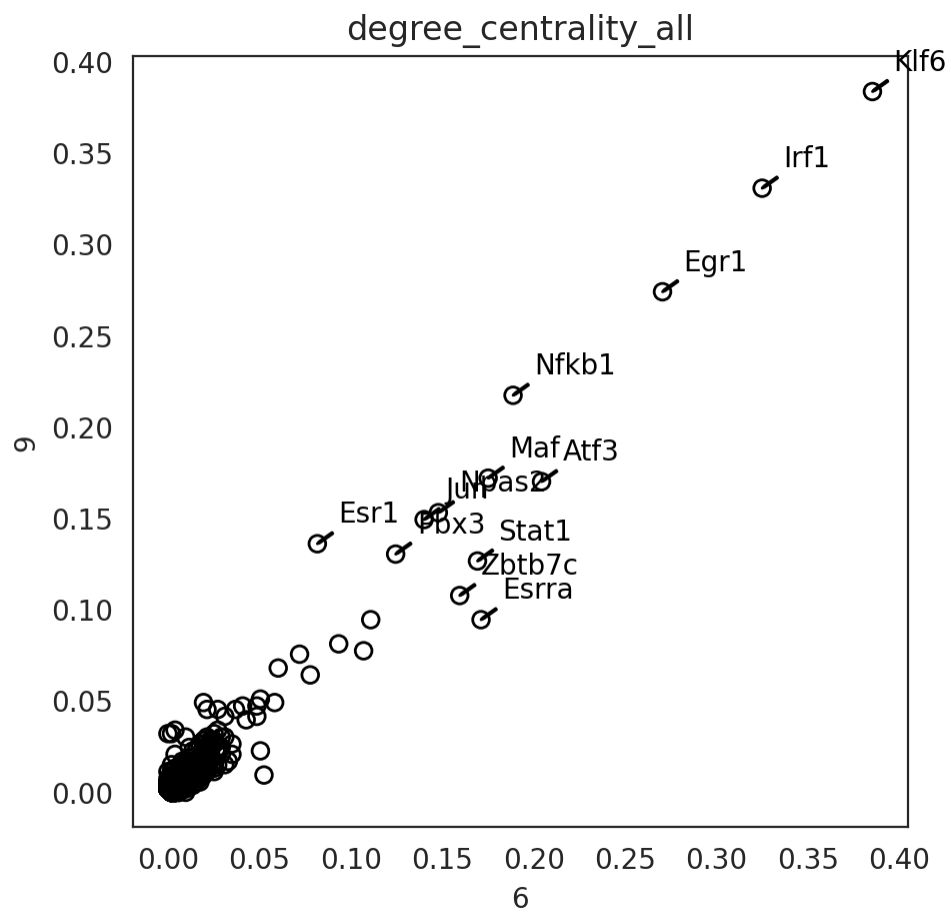
Processing clusters 6 and 7



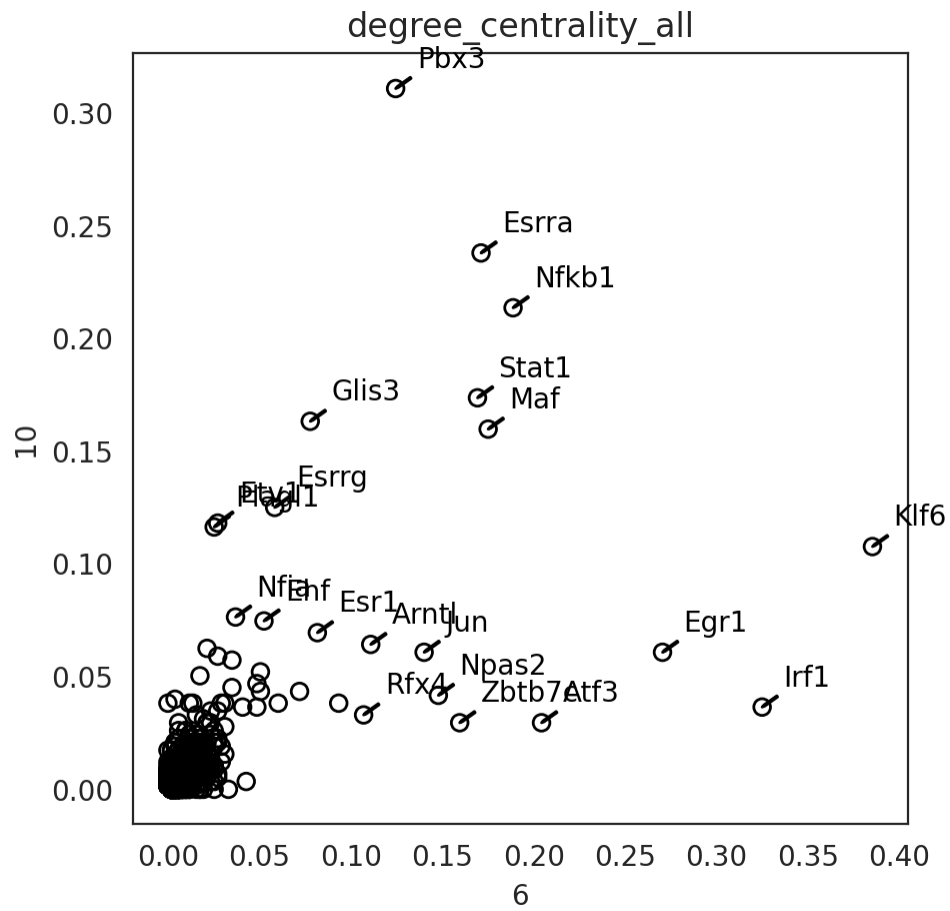
Processing clusters 6 and 8



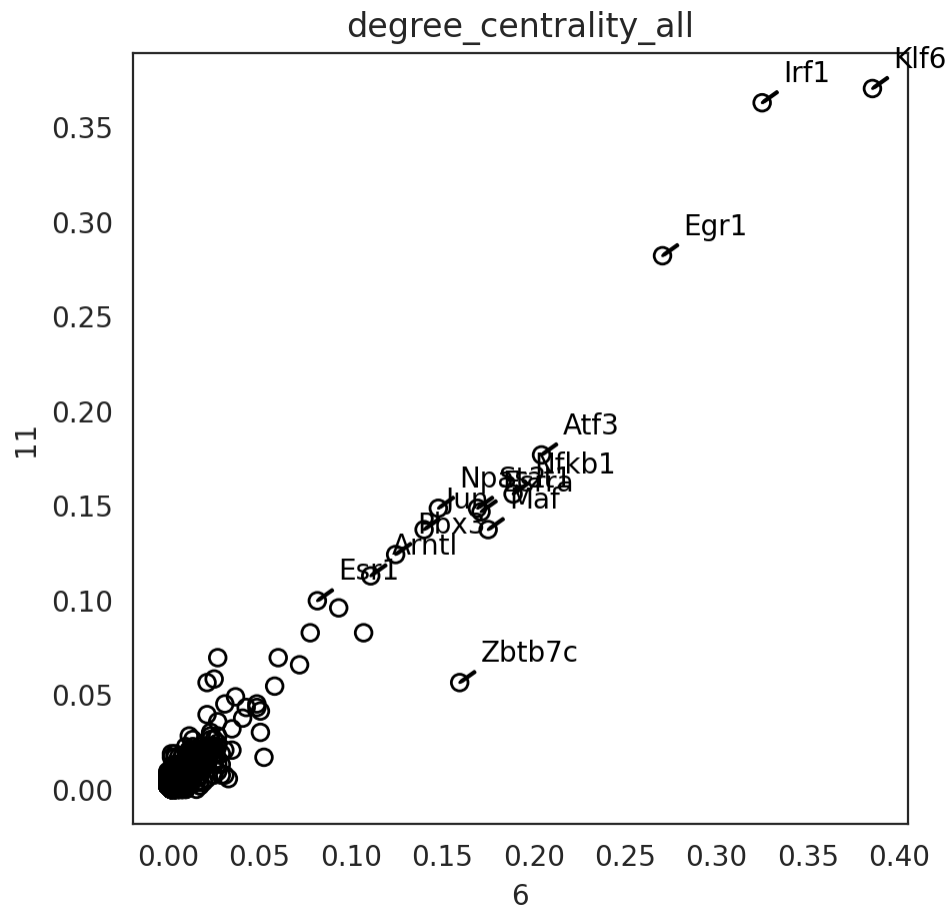
Processing clusters 6 and 9



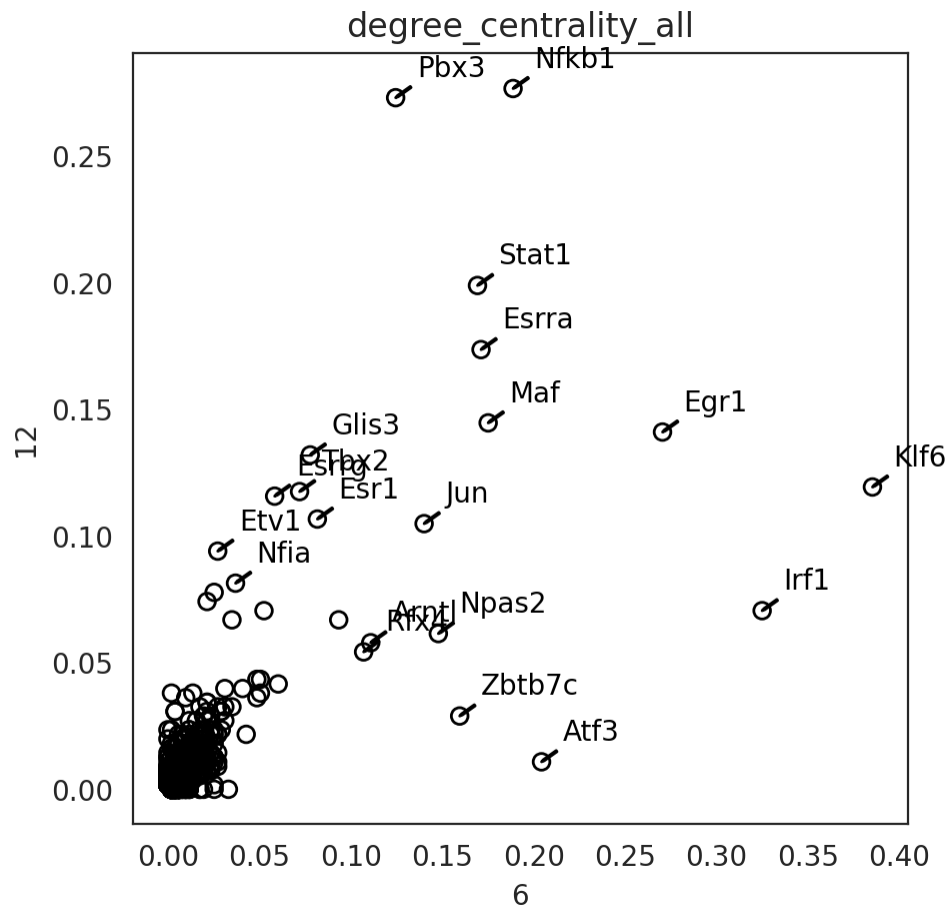
Processing clusters 6 and 10



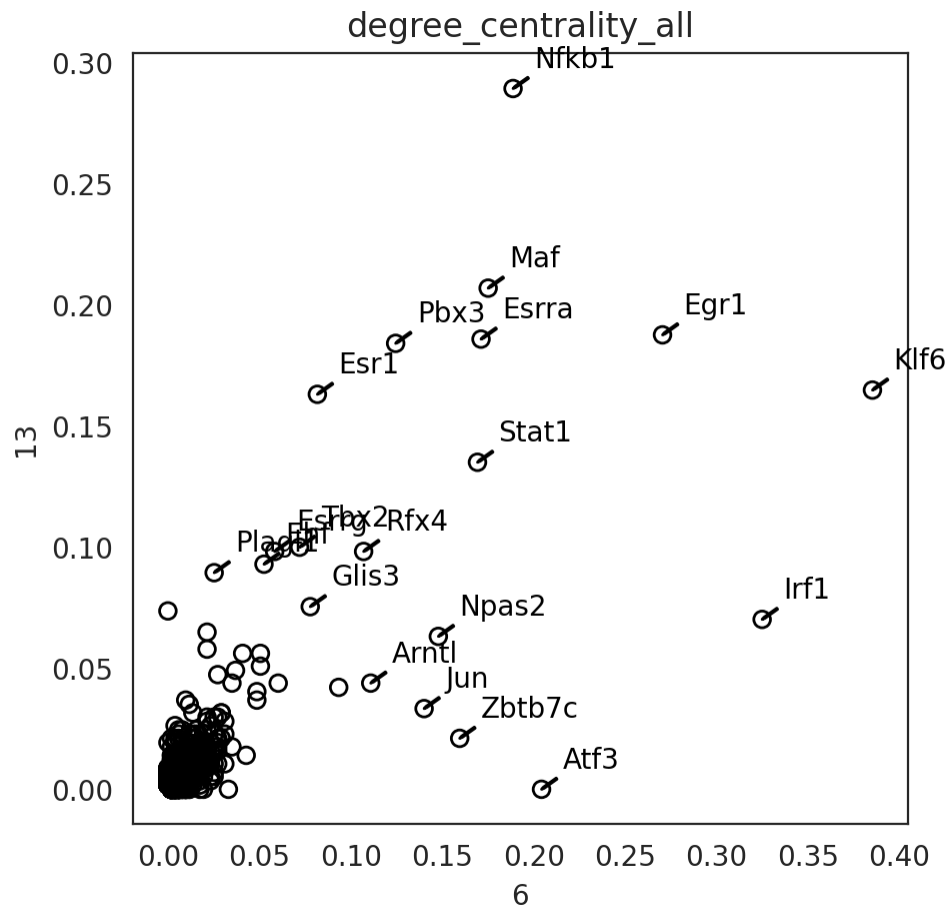
Processing clusters 6 and 11



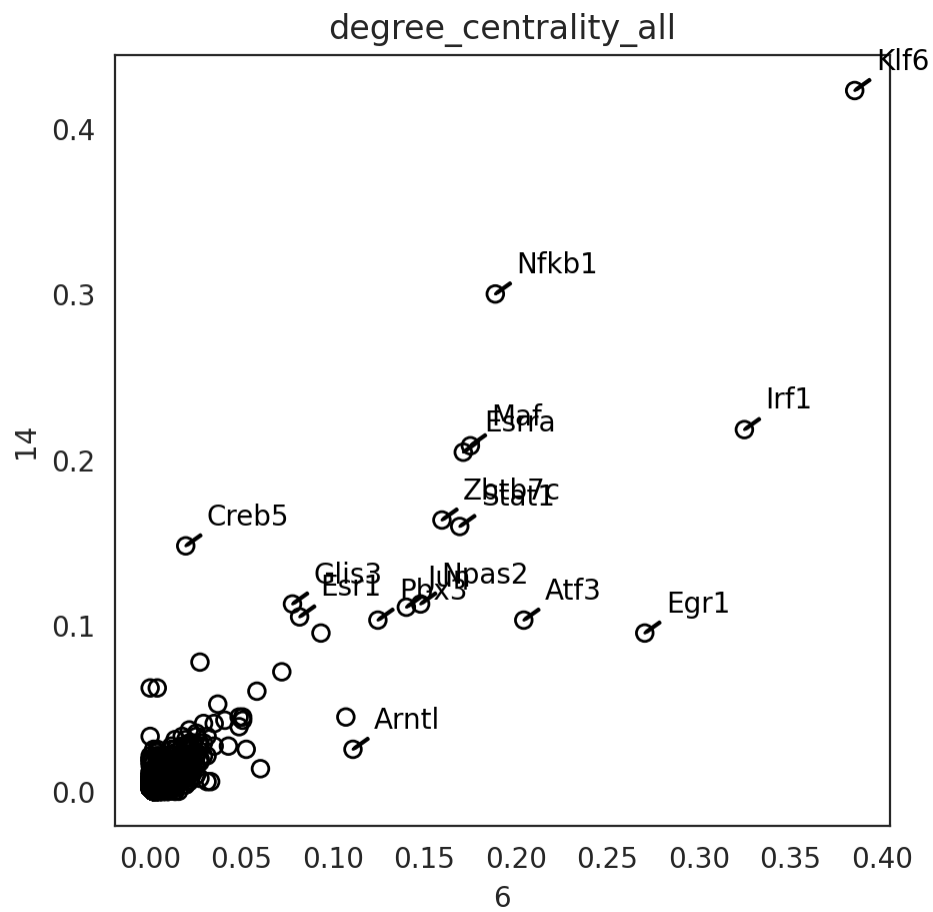
Processing clusters 6 and 12



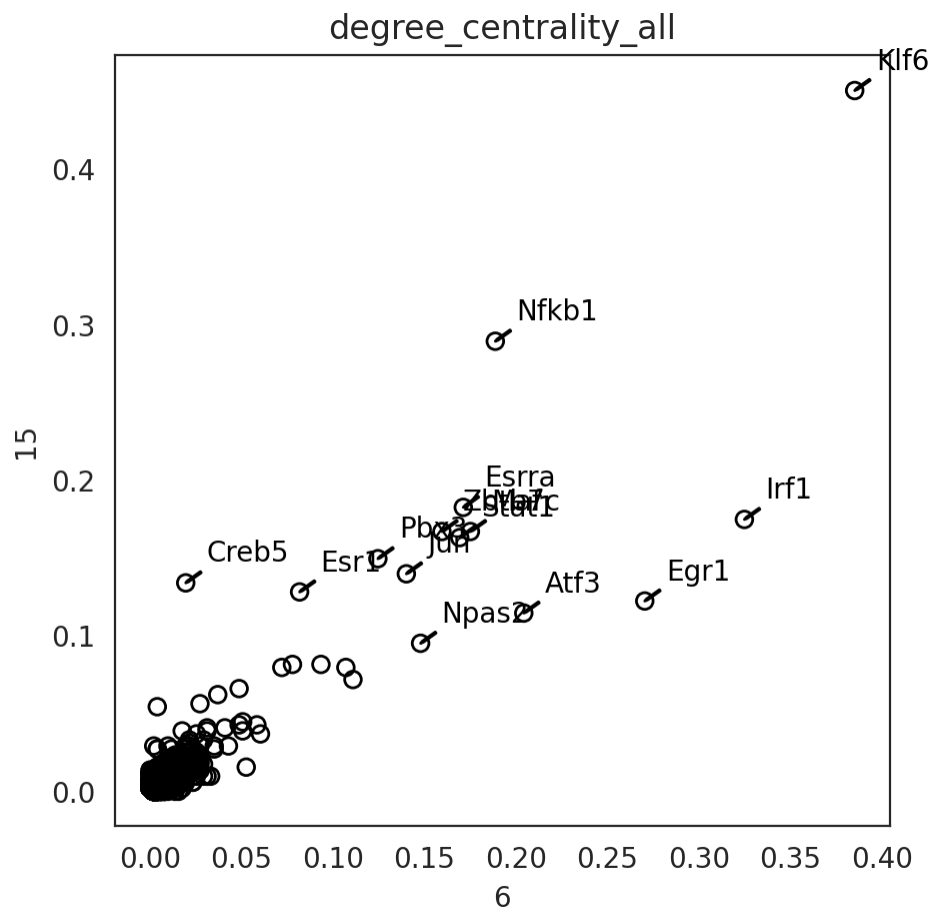
Processing clusters 6 and 13



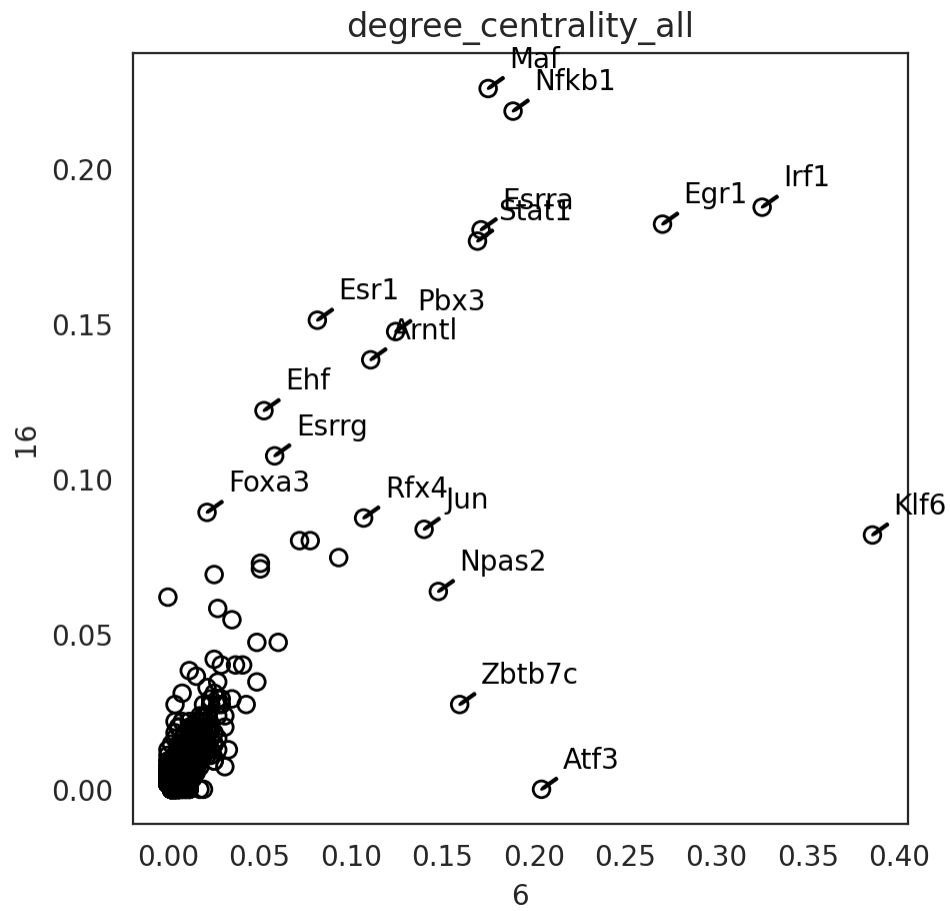
Processing clusters 6 and 14



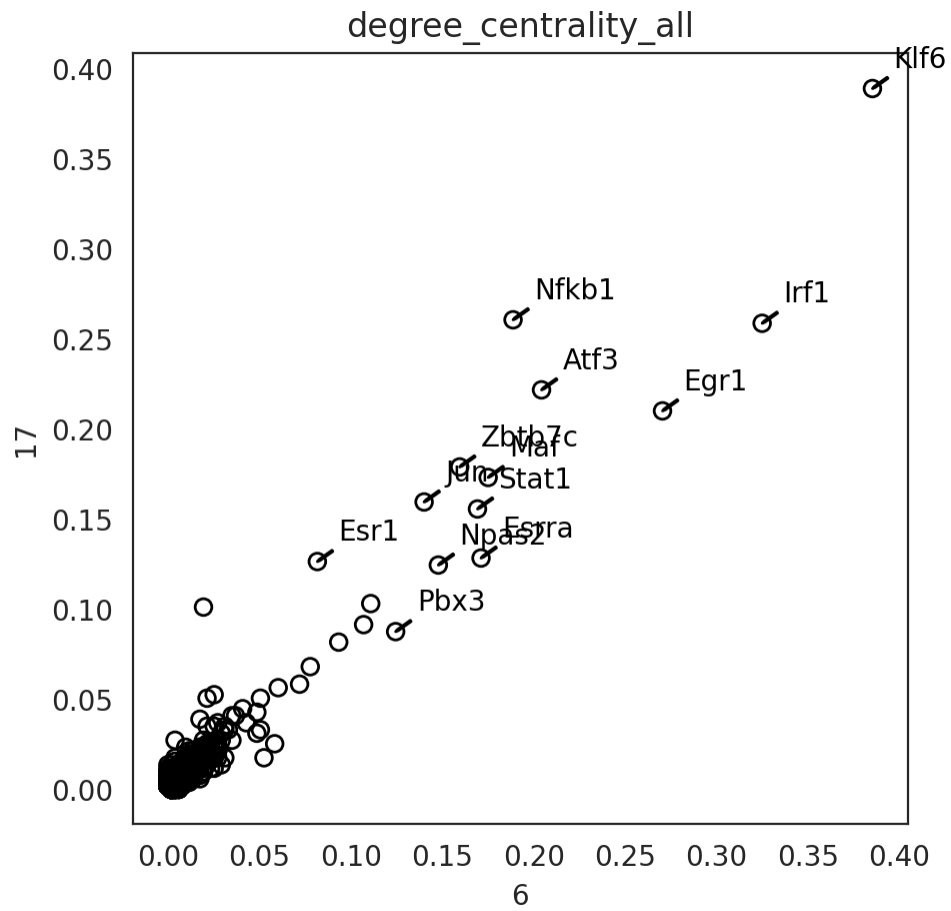
Processing clusters 6 and 15



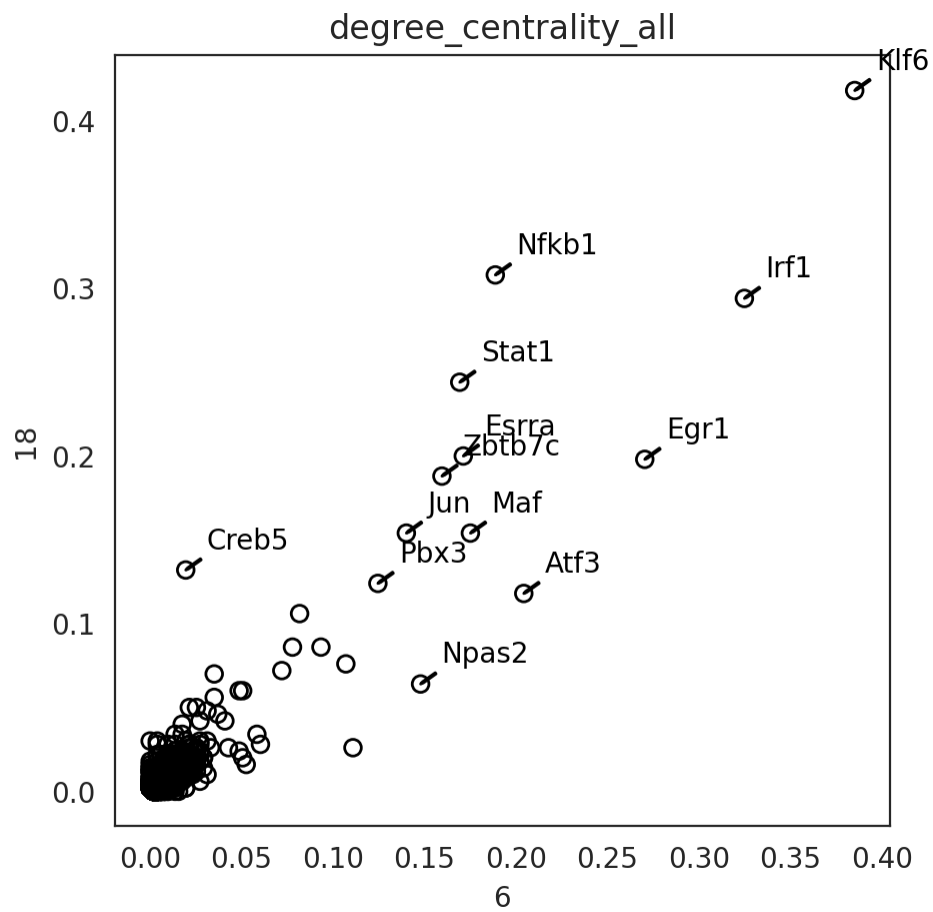
Processing clusters 6 and 16



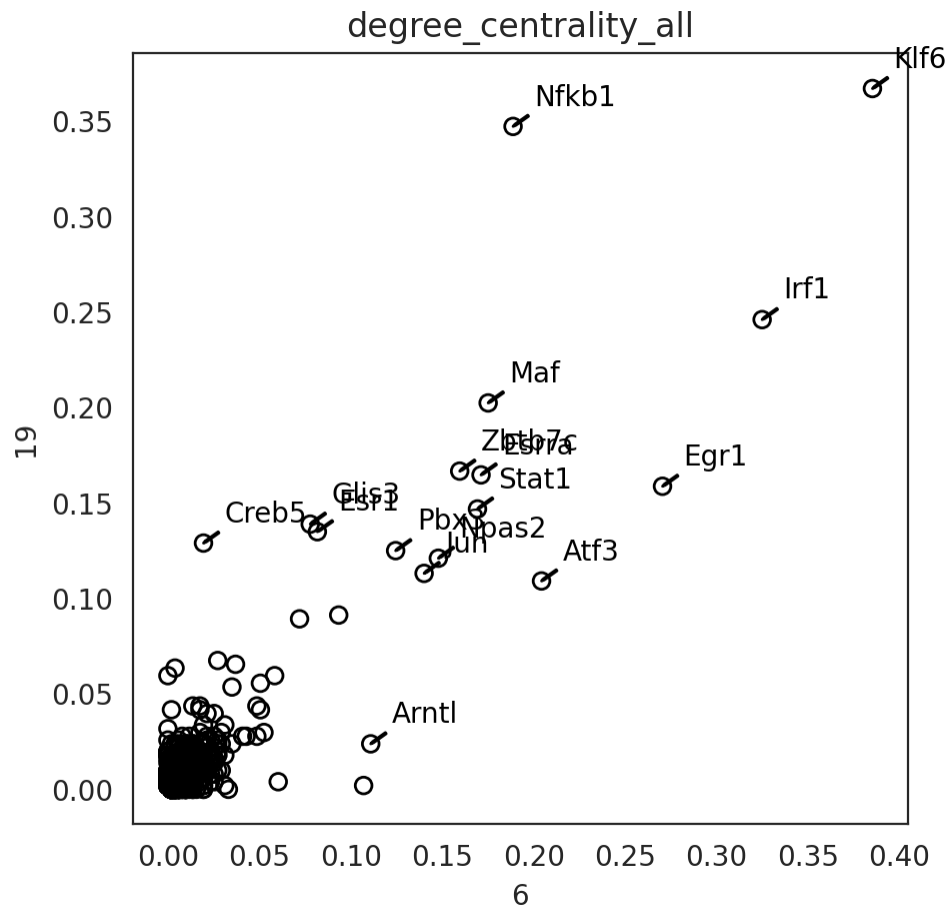
Processing clusters 6 and 17



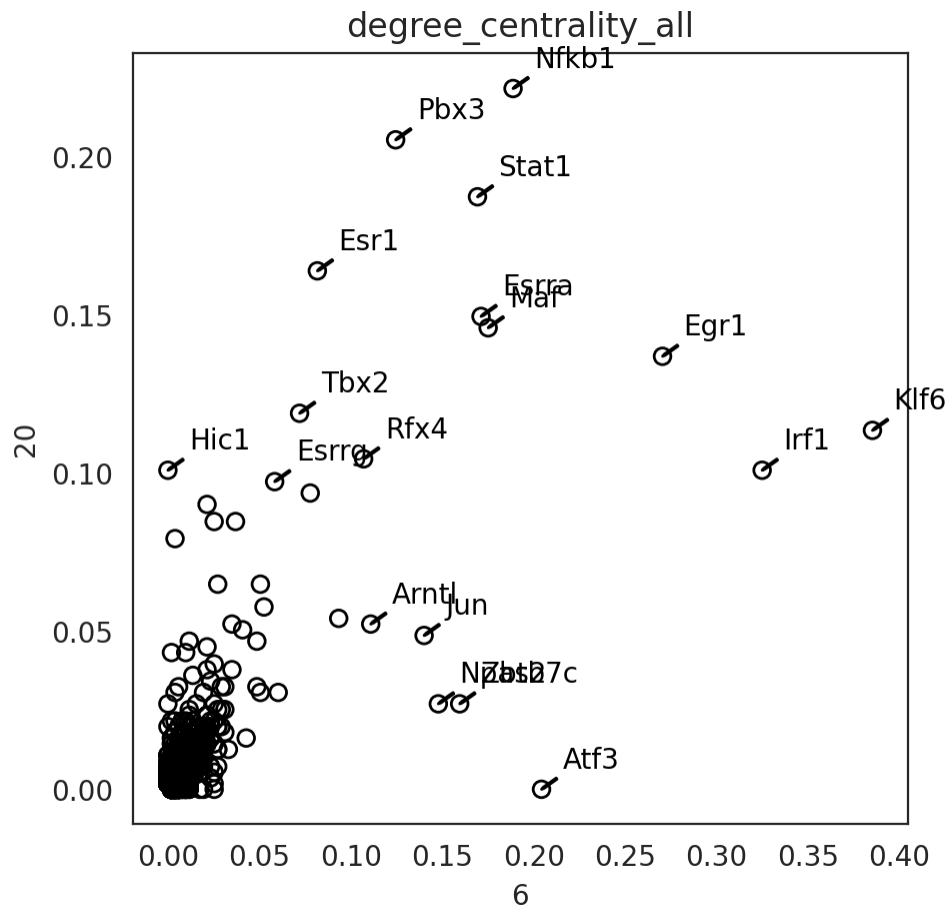
Processing clusters 6 and 18



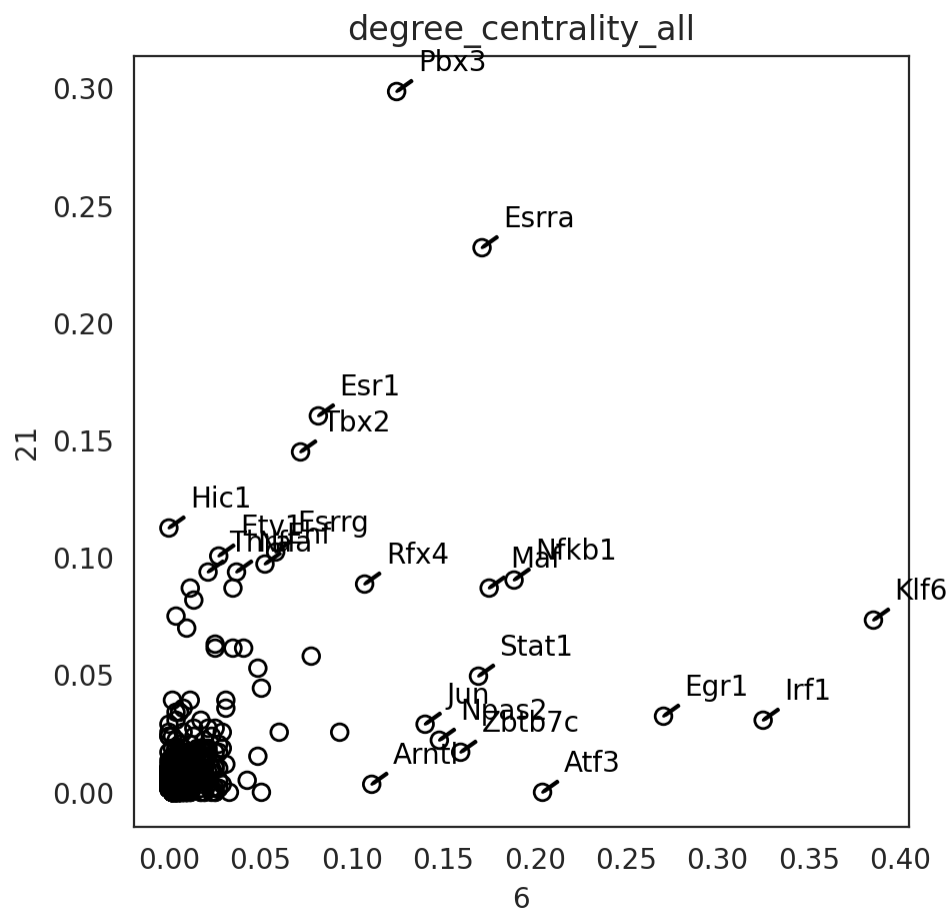
Processing clusters 6 and 19



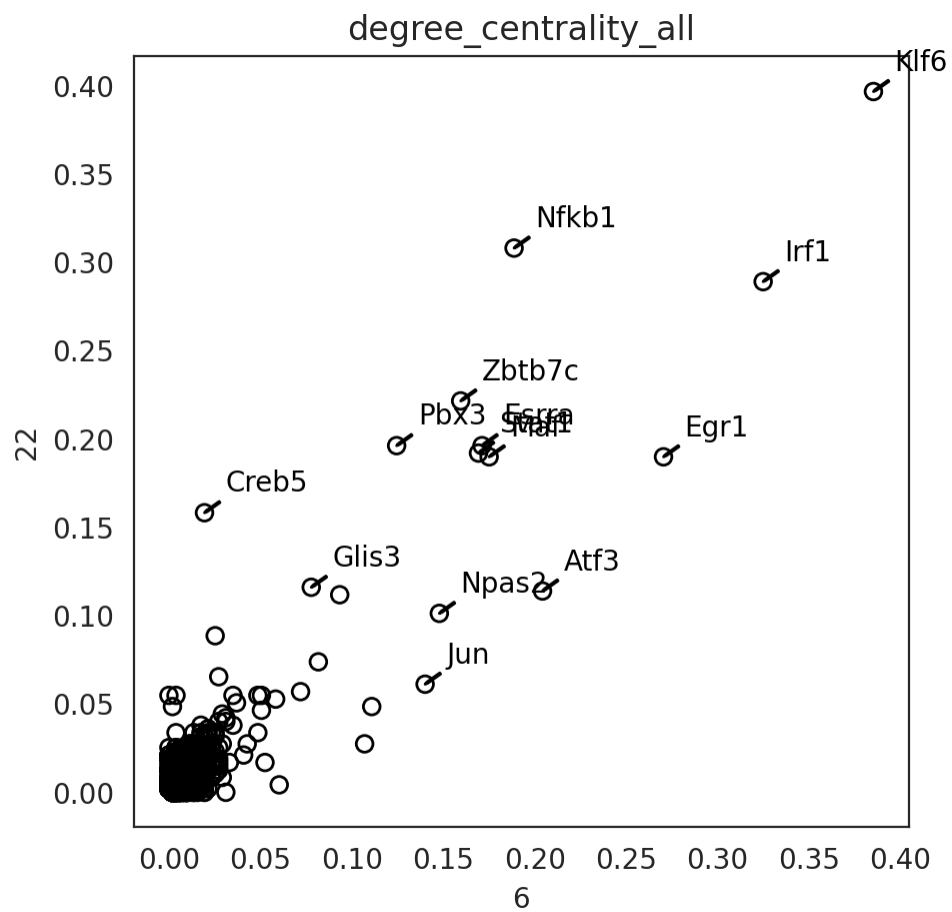
Processing clusters 6 and 20



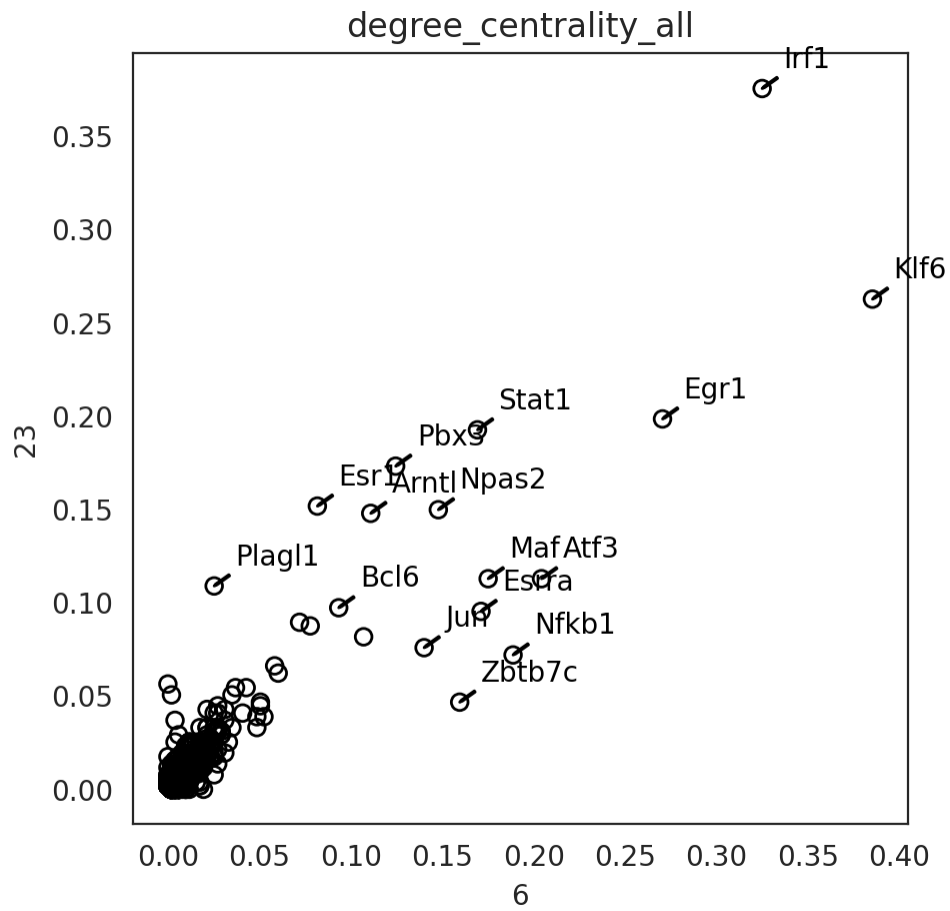
Processing clusters 6 and 21

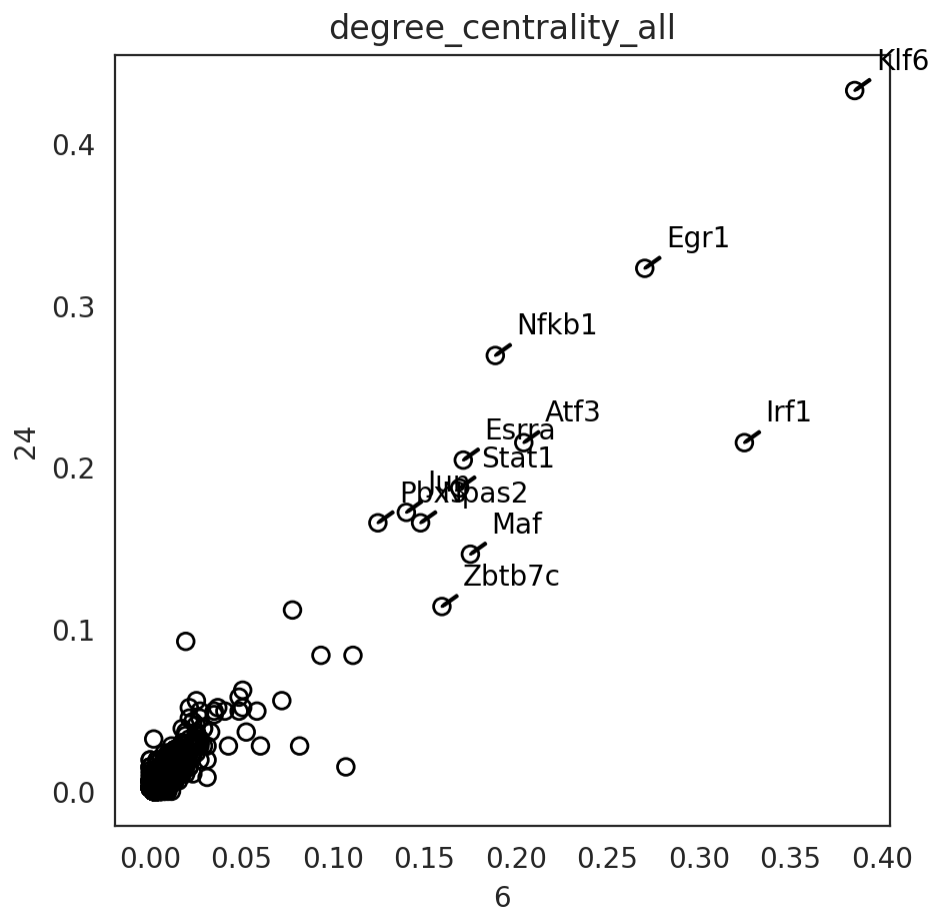


Processing clusters 6 and 22

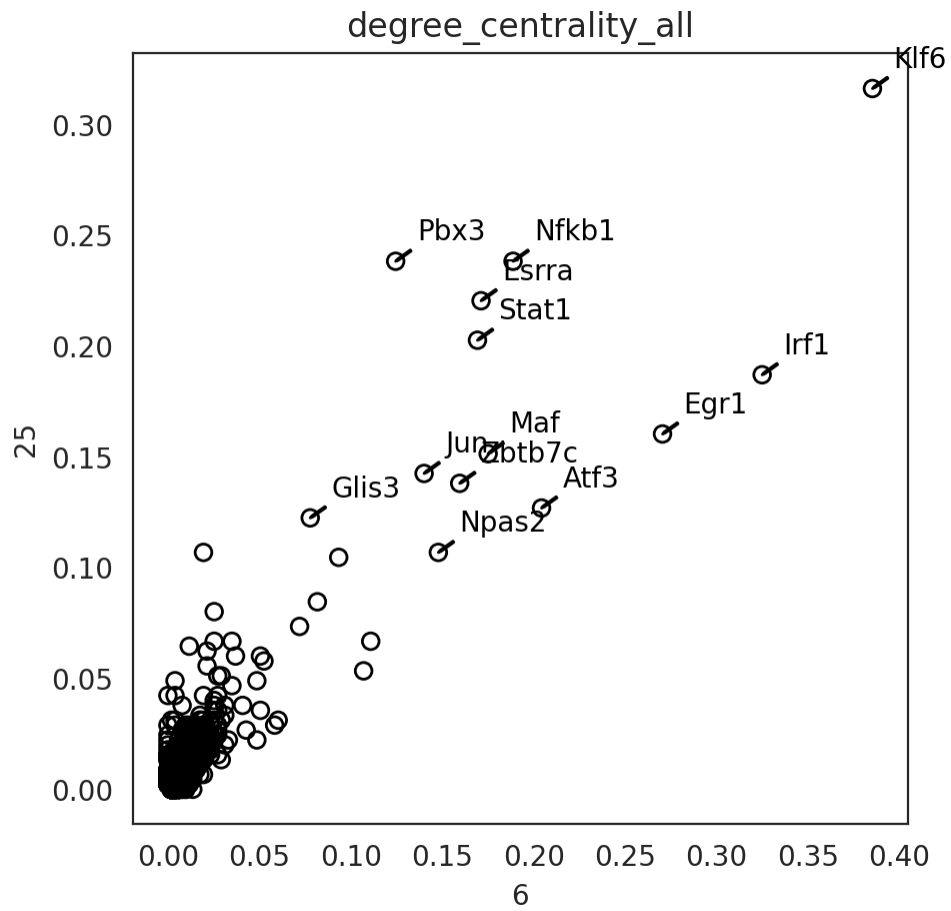


Processing clusters 6 and 23

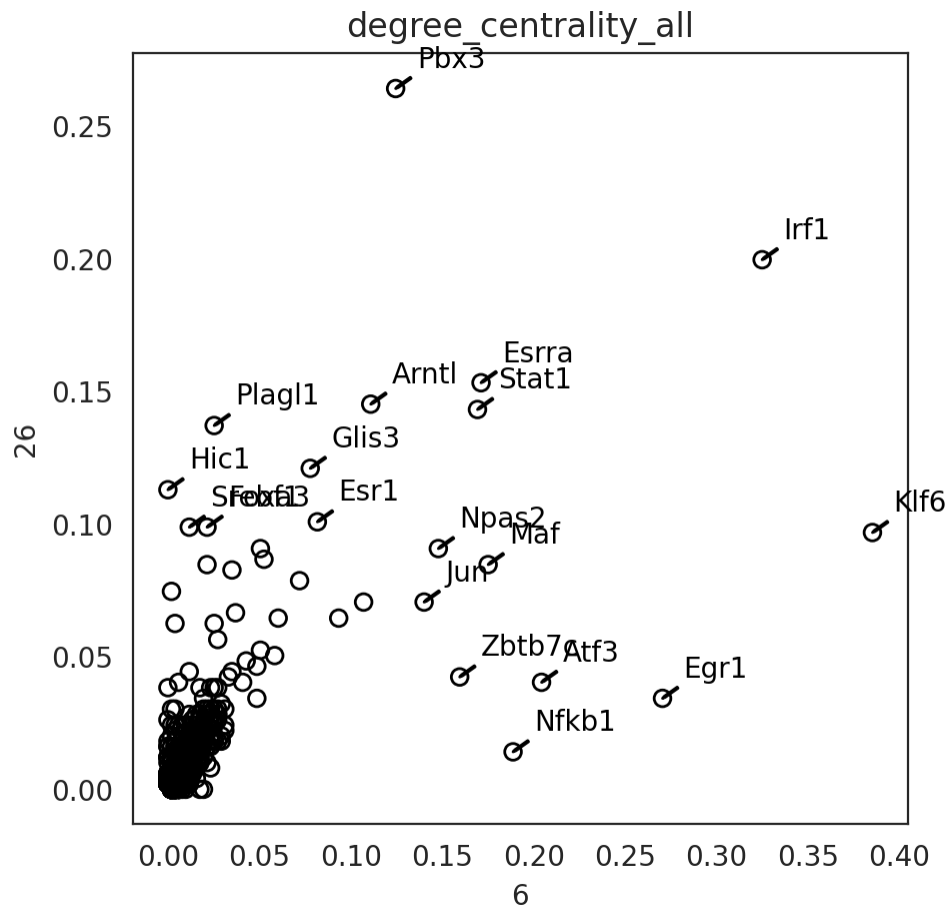




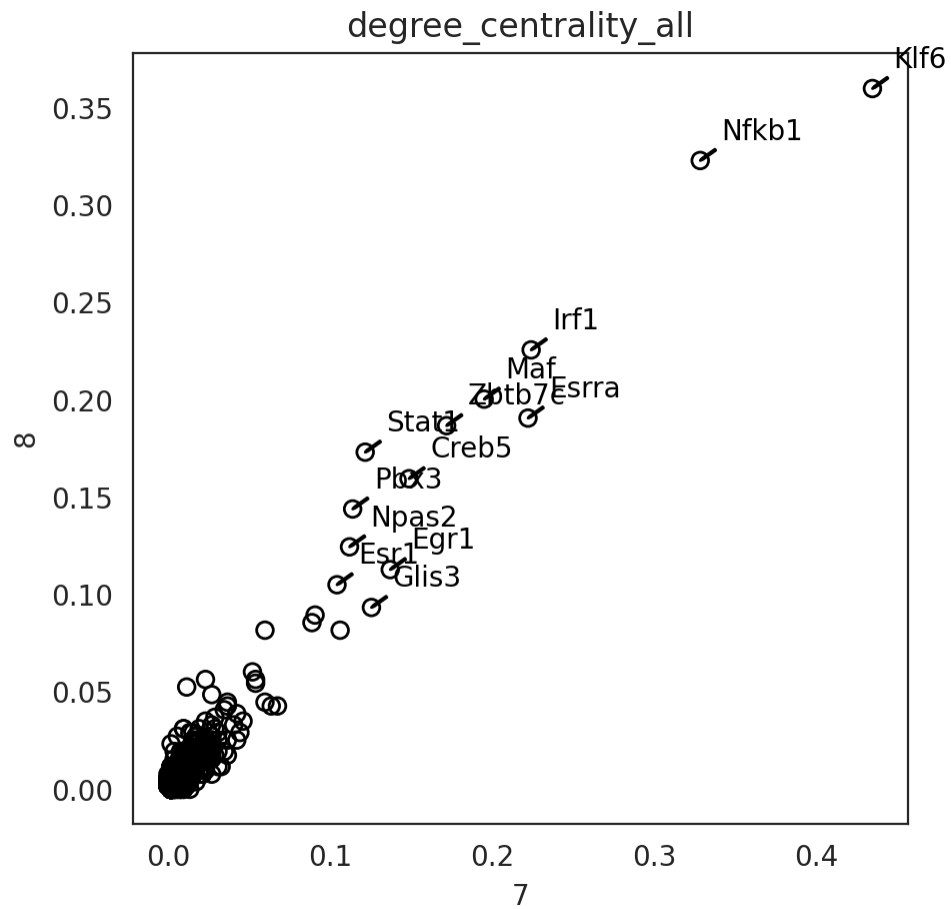
Processing clusters 6 and 25



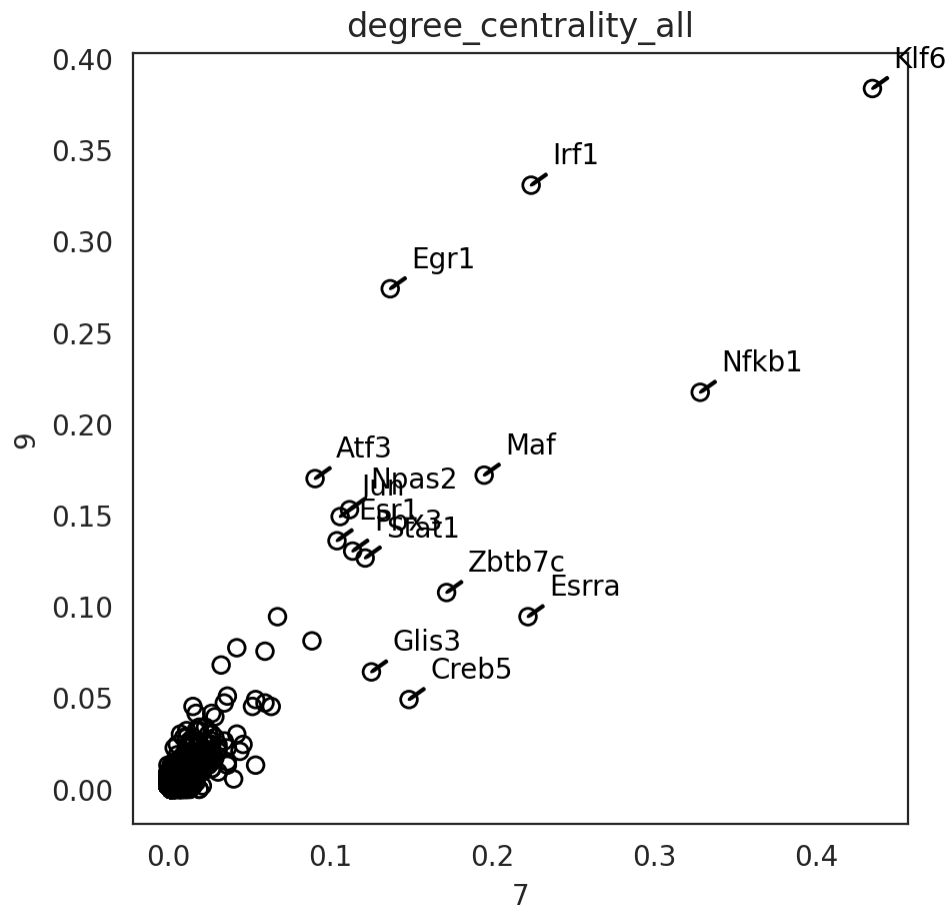
Processing clusters 6 and 26



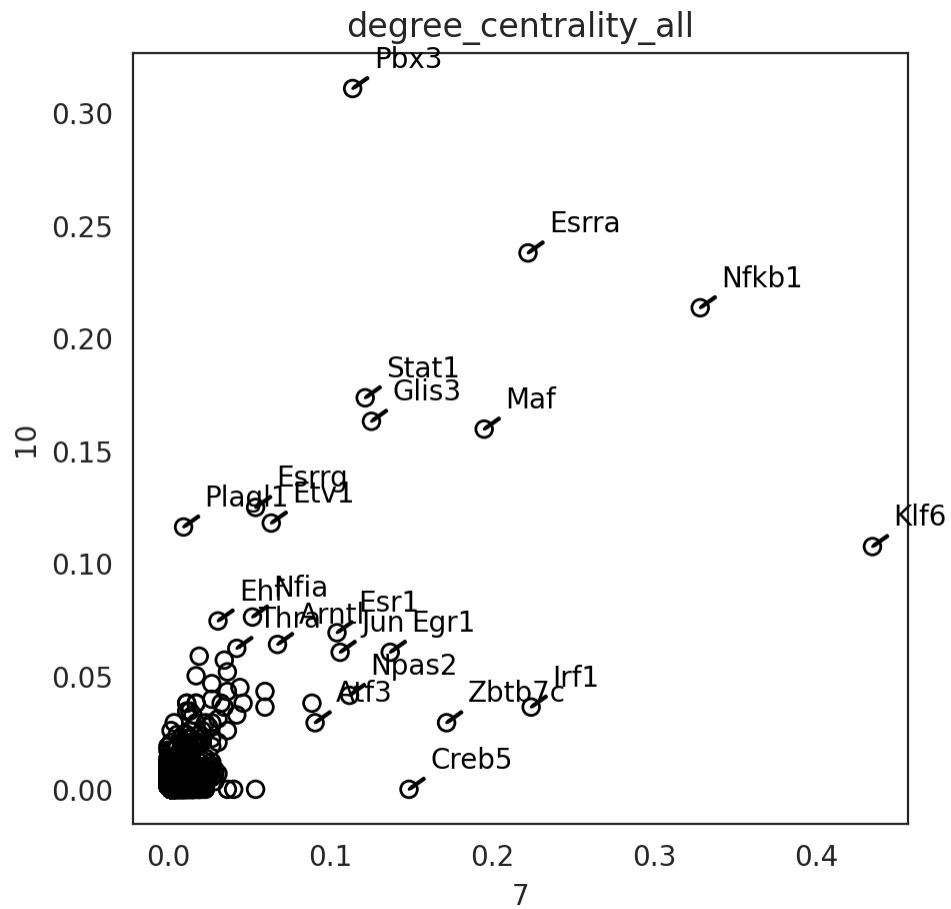
Processing clusters 7 and 8



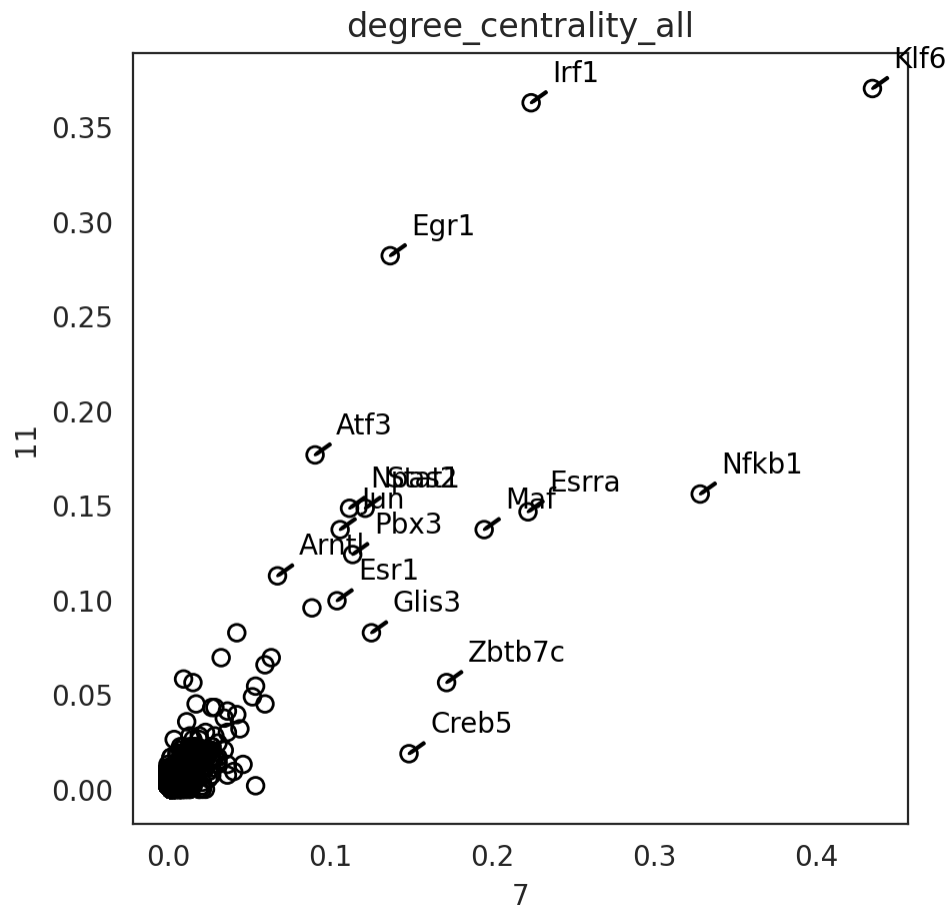
Processing clusters 7 and 9



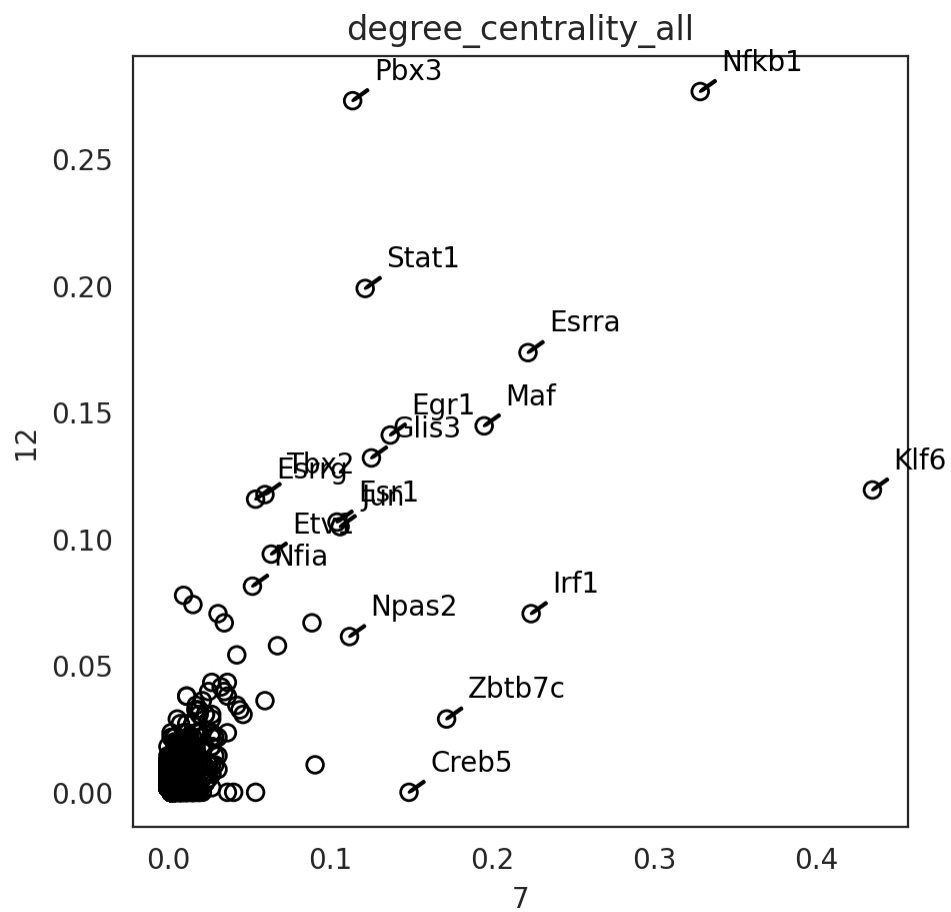
Processing clusters 7 and 10



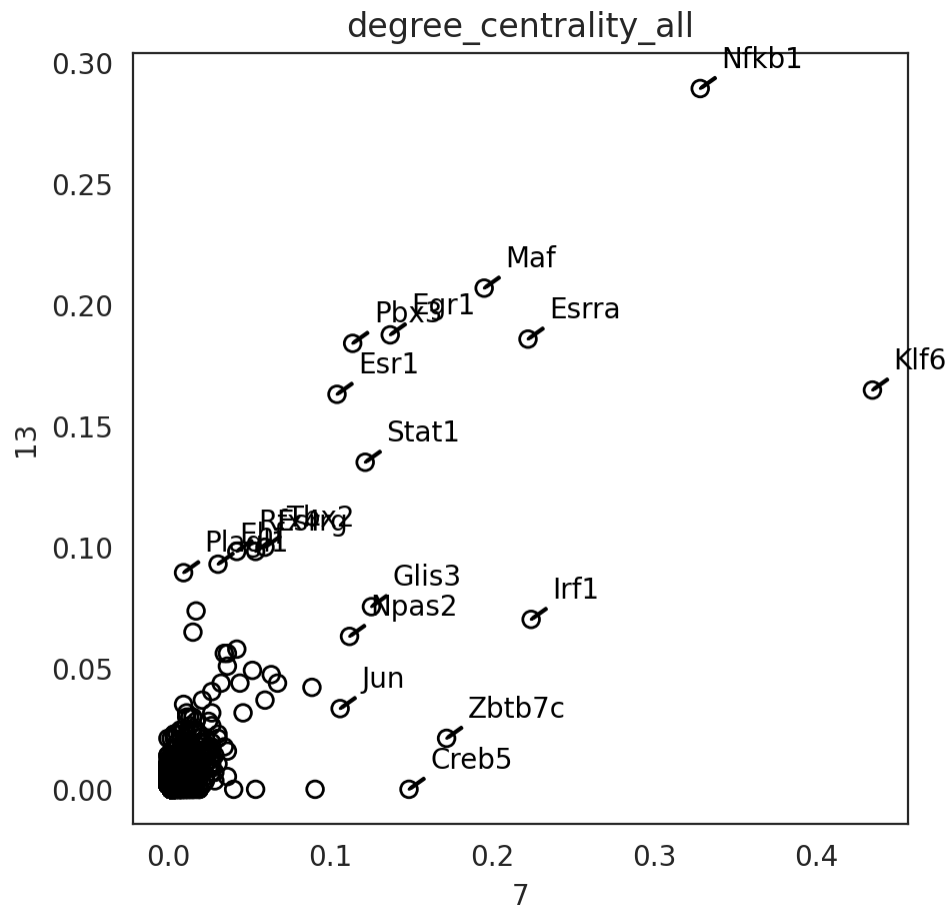
Processing clusters 7 and 11



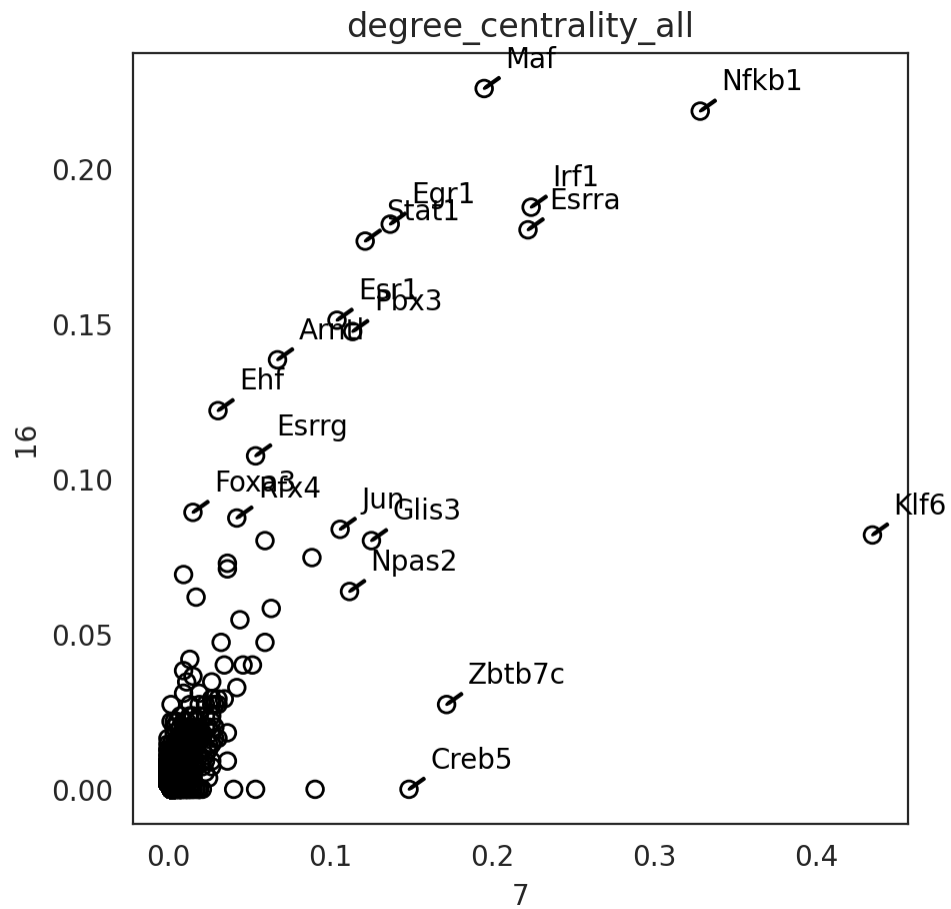
Processing clusters 7 and 12



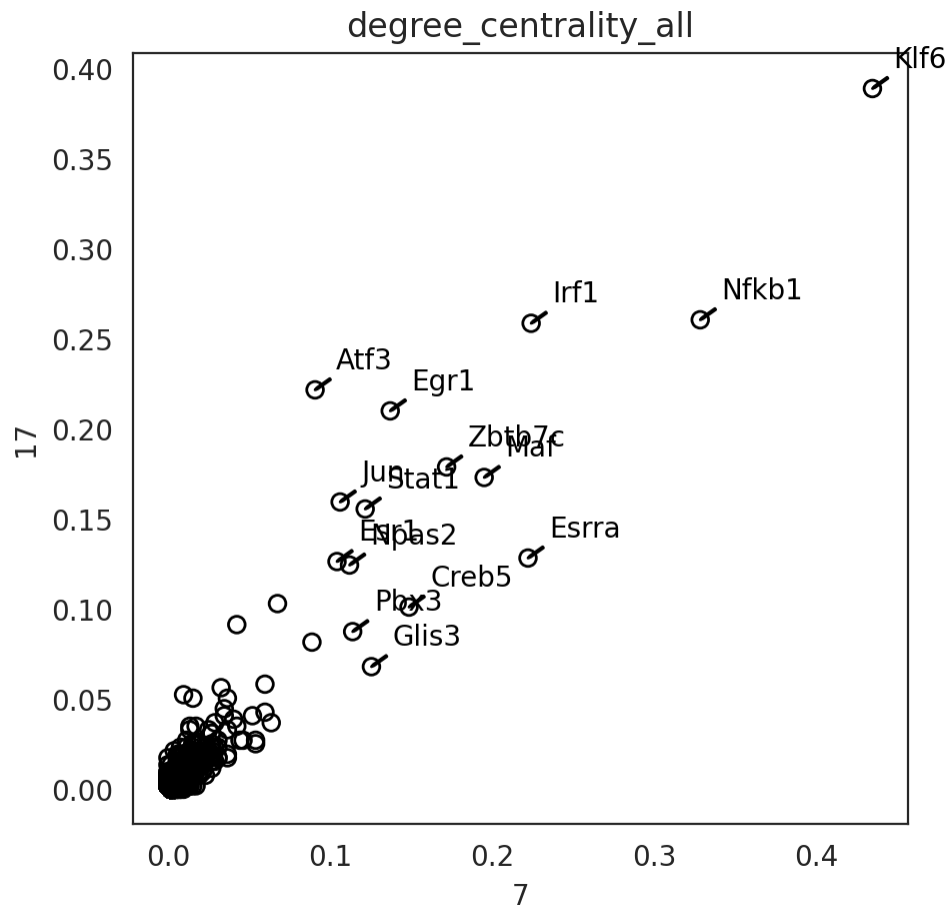
Processing clusters 7 and 13



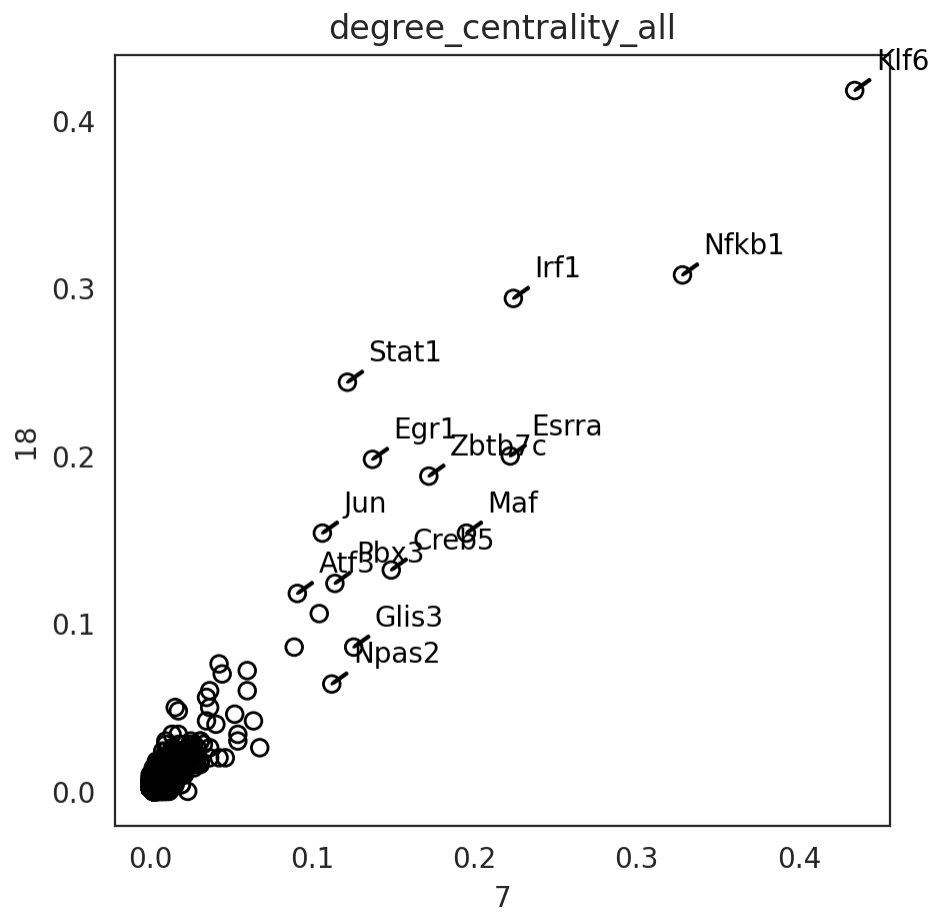
Processing clusters 7 and 14



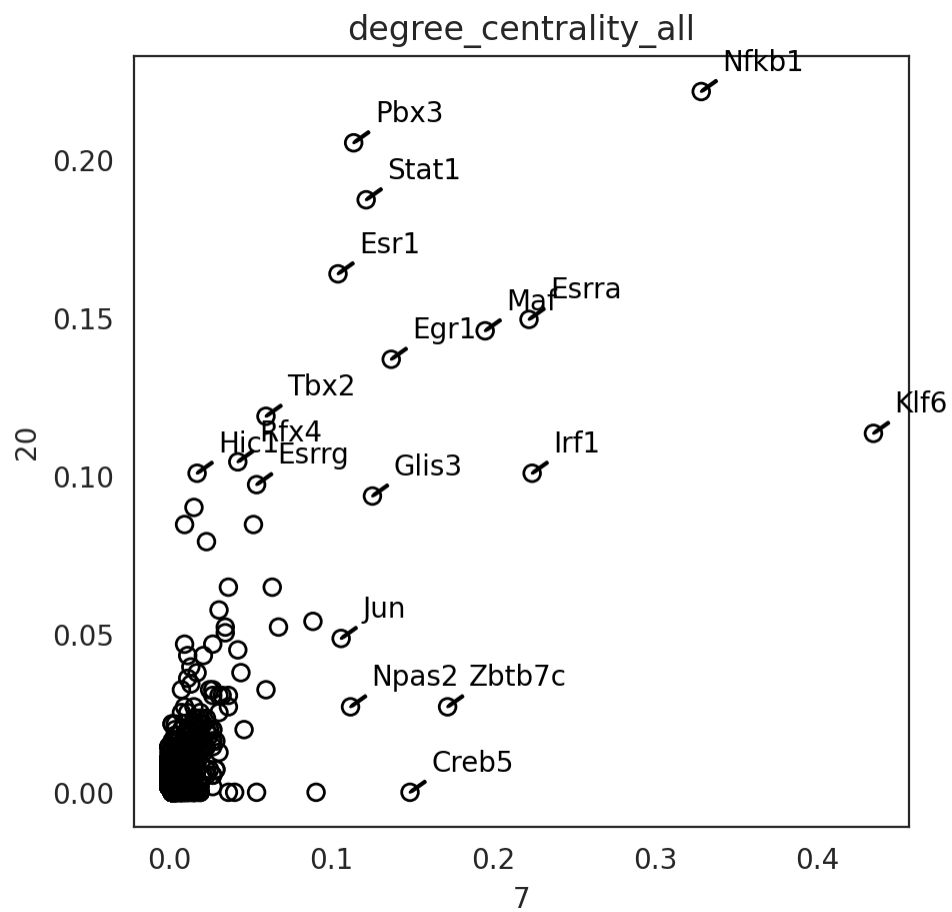
Processing clusters 7 and 17



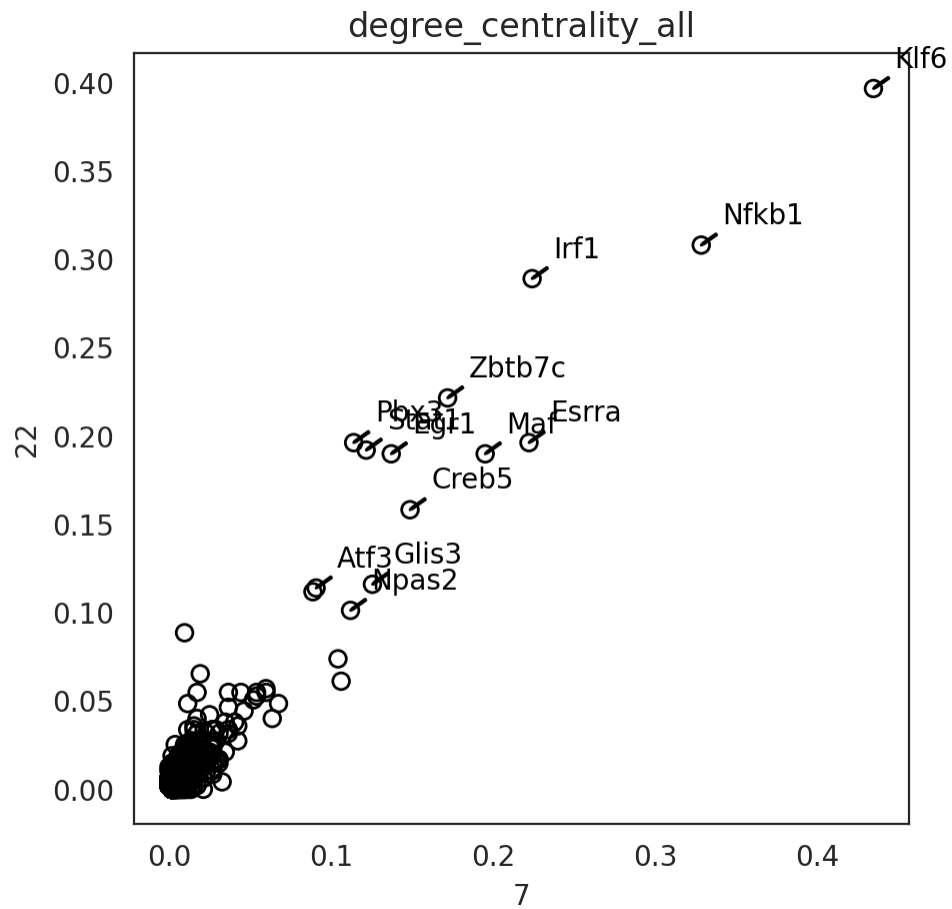
Processing clusters 7 and 18



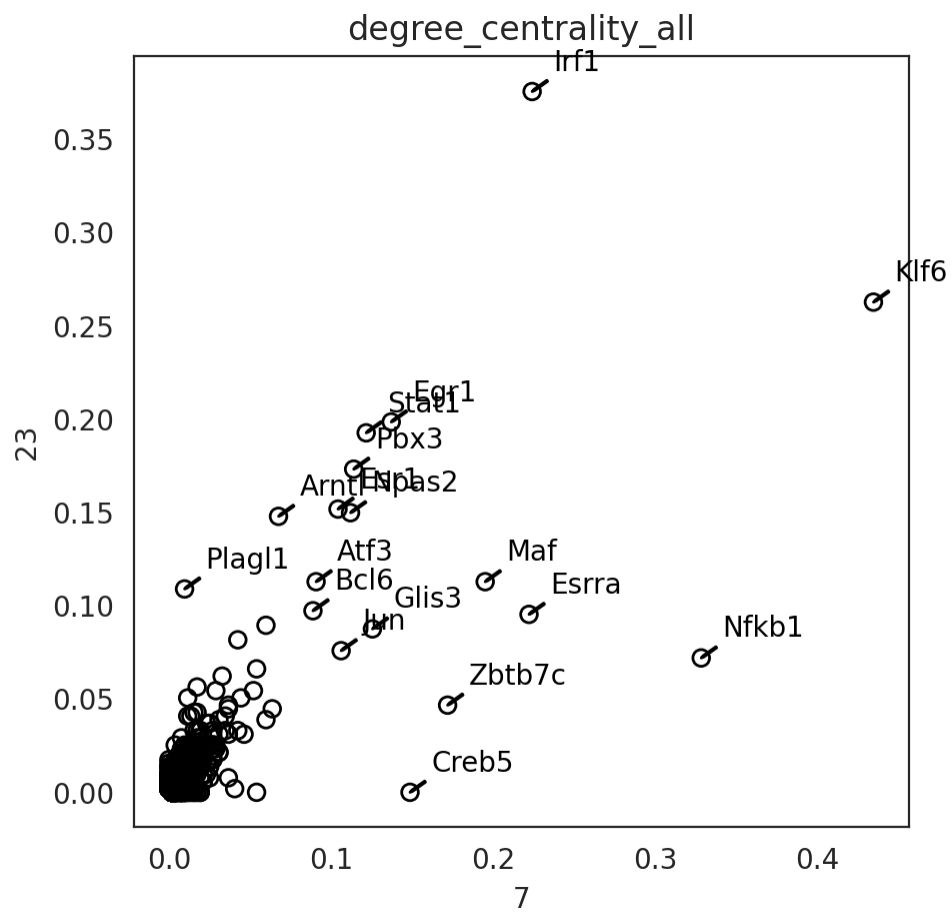
Processing clusters 7 and 19



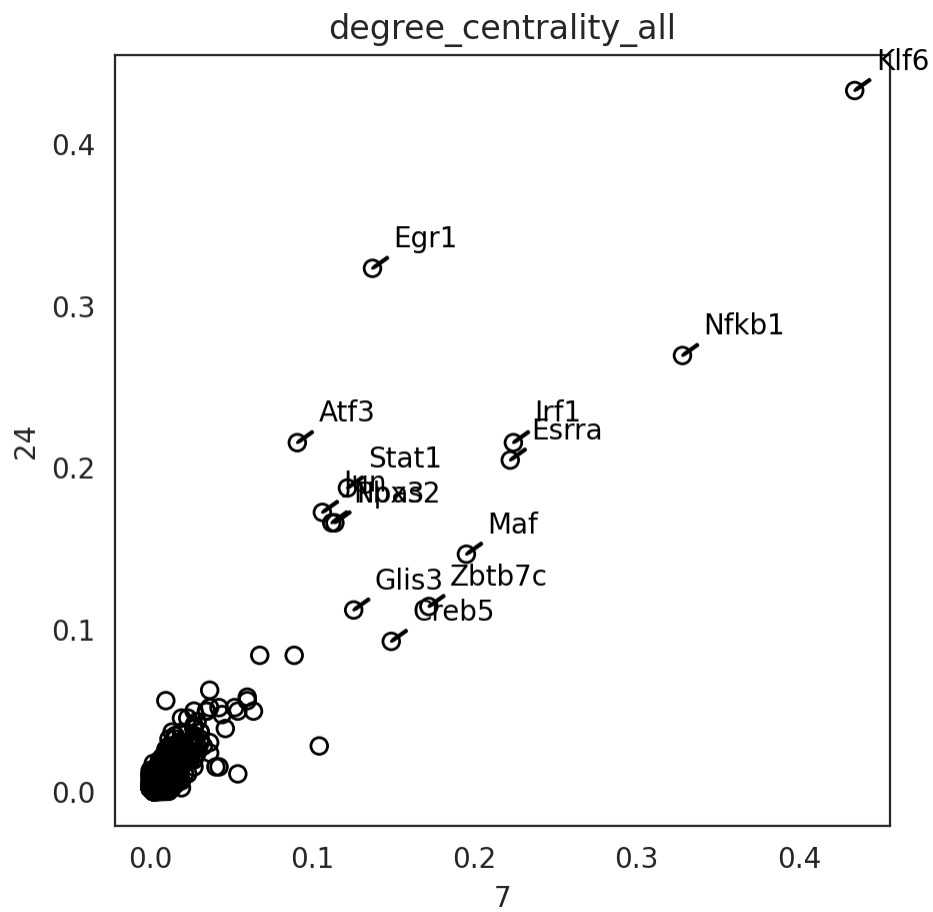
Processing clusters 7 and 21



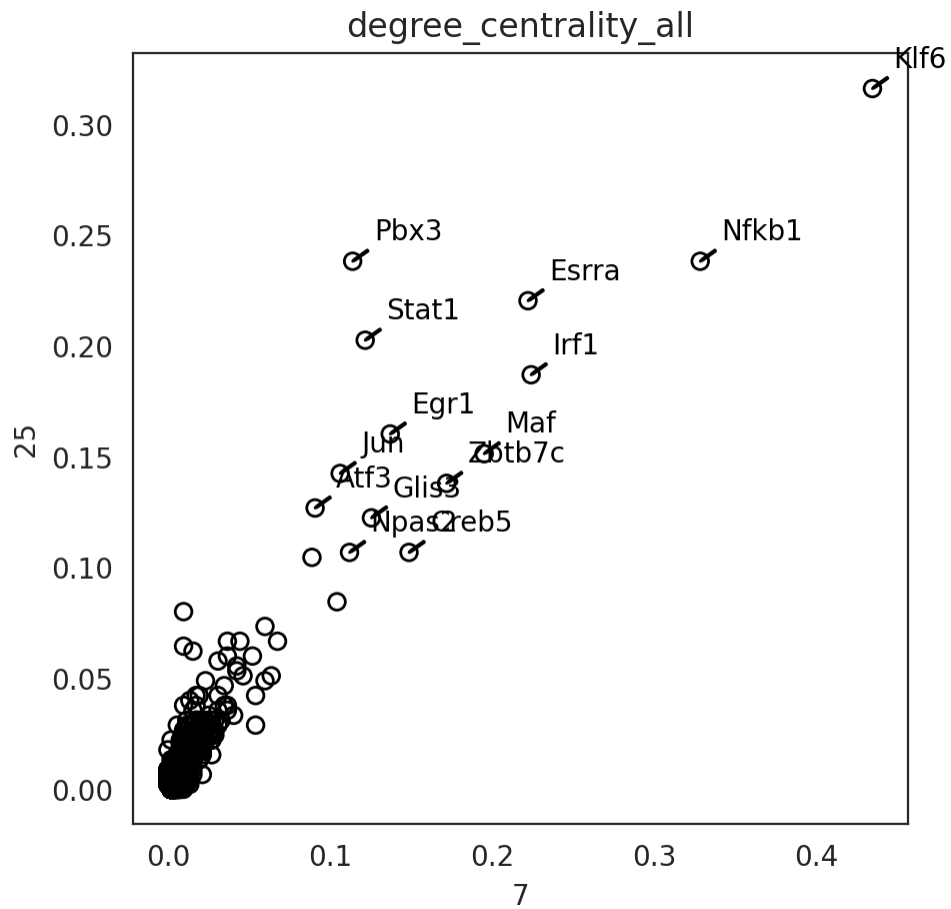
Processing clusters 7 and 23



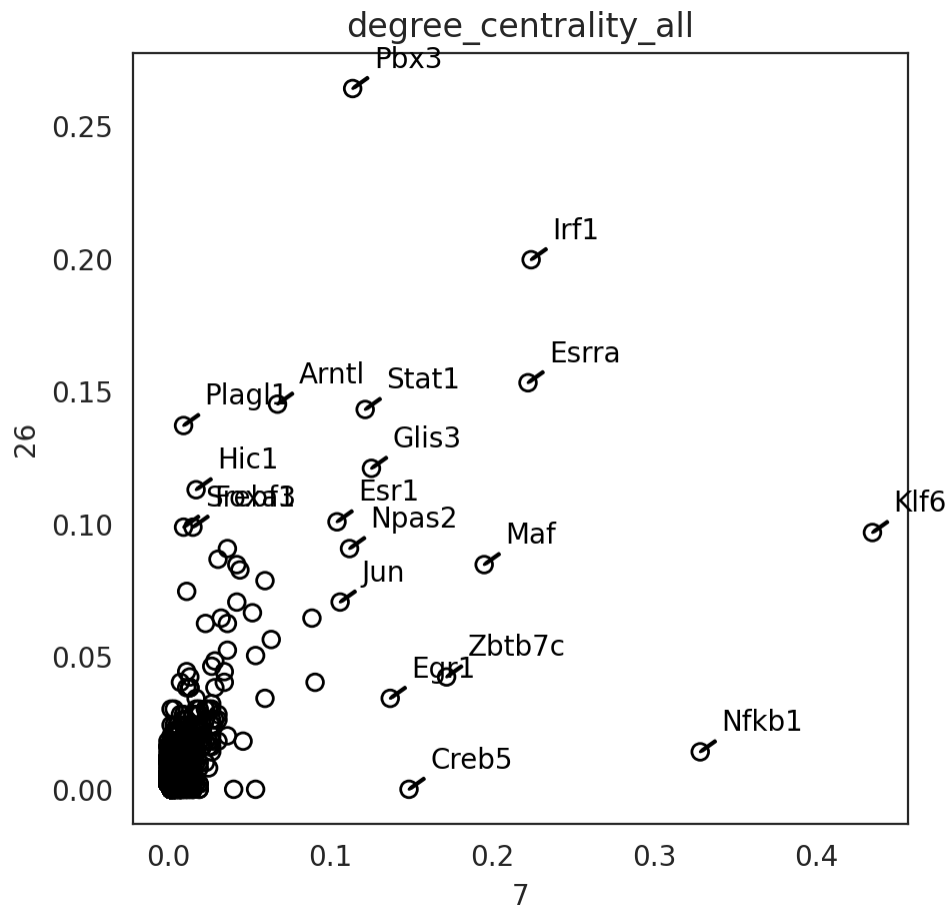
Processing clusters 7 and 24



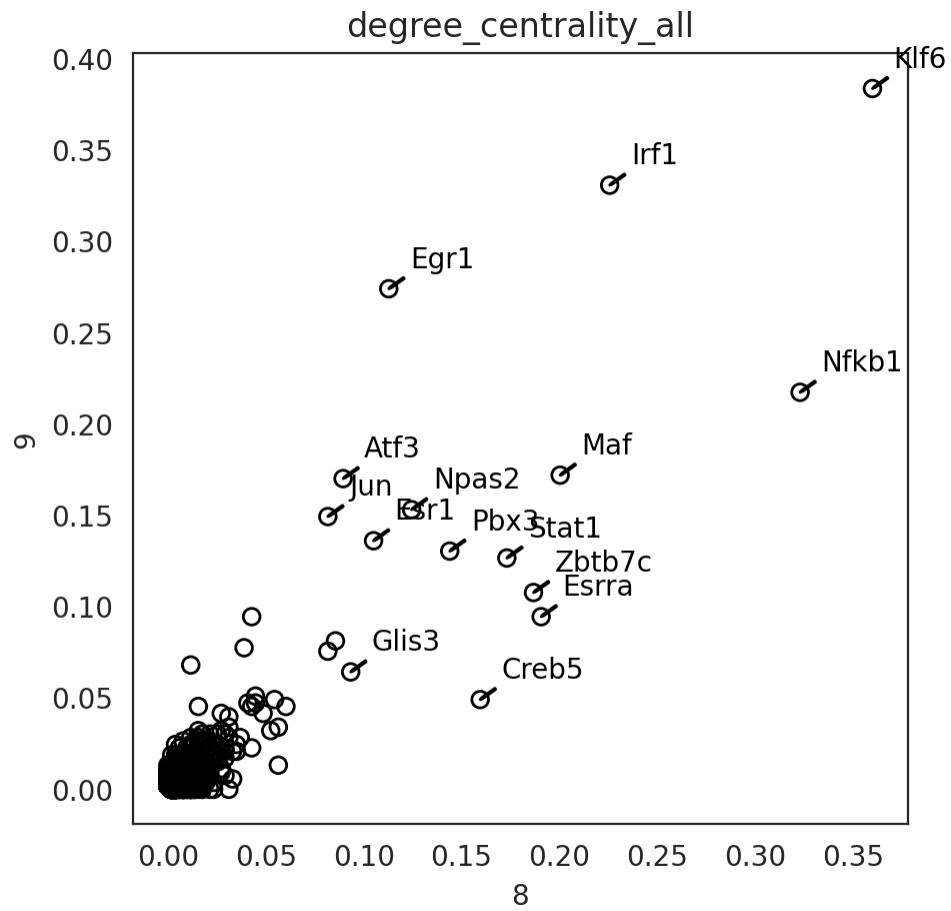
Processing clusters 7 and 25



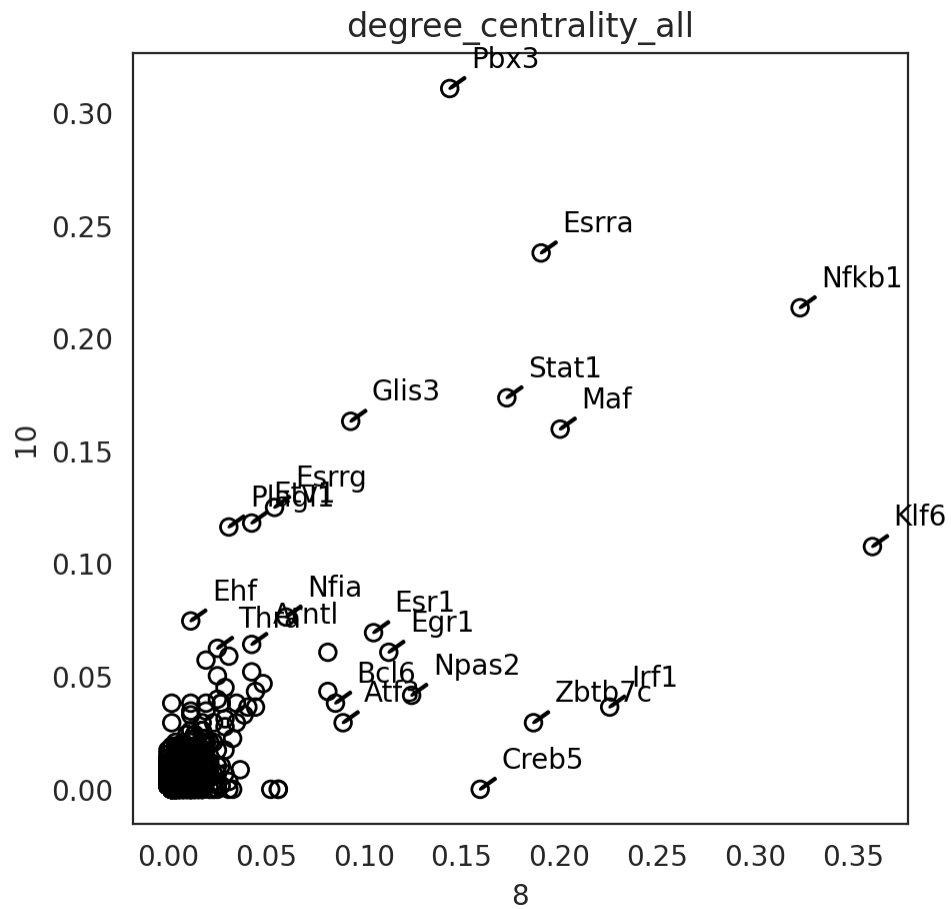
Processing clusters 7 and 26



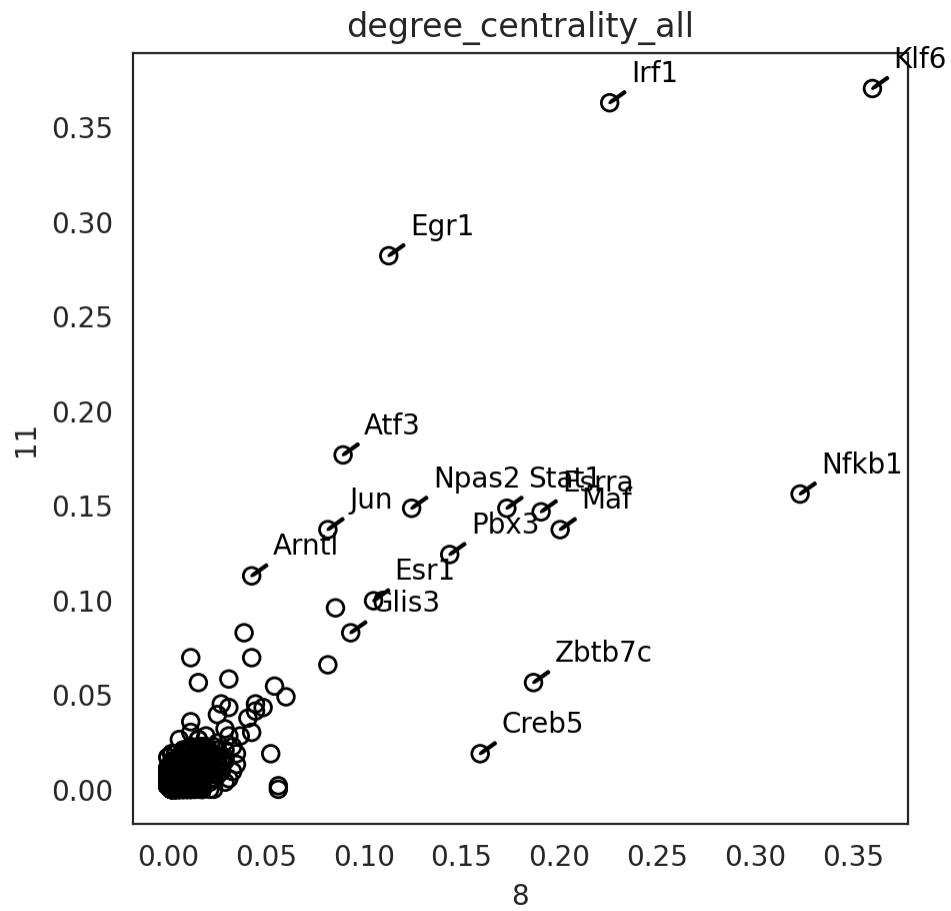
Processing clusters 8 and 9



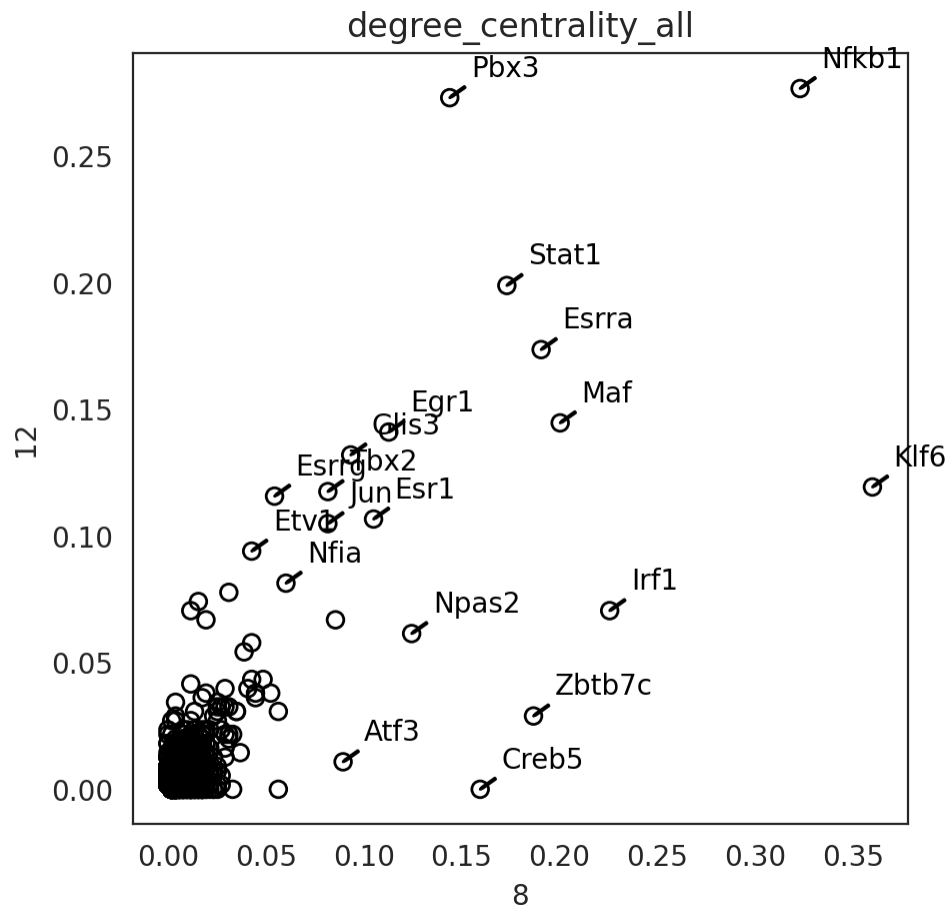
Processing clusters 8 and 10



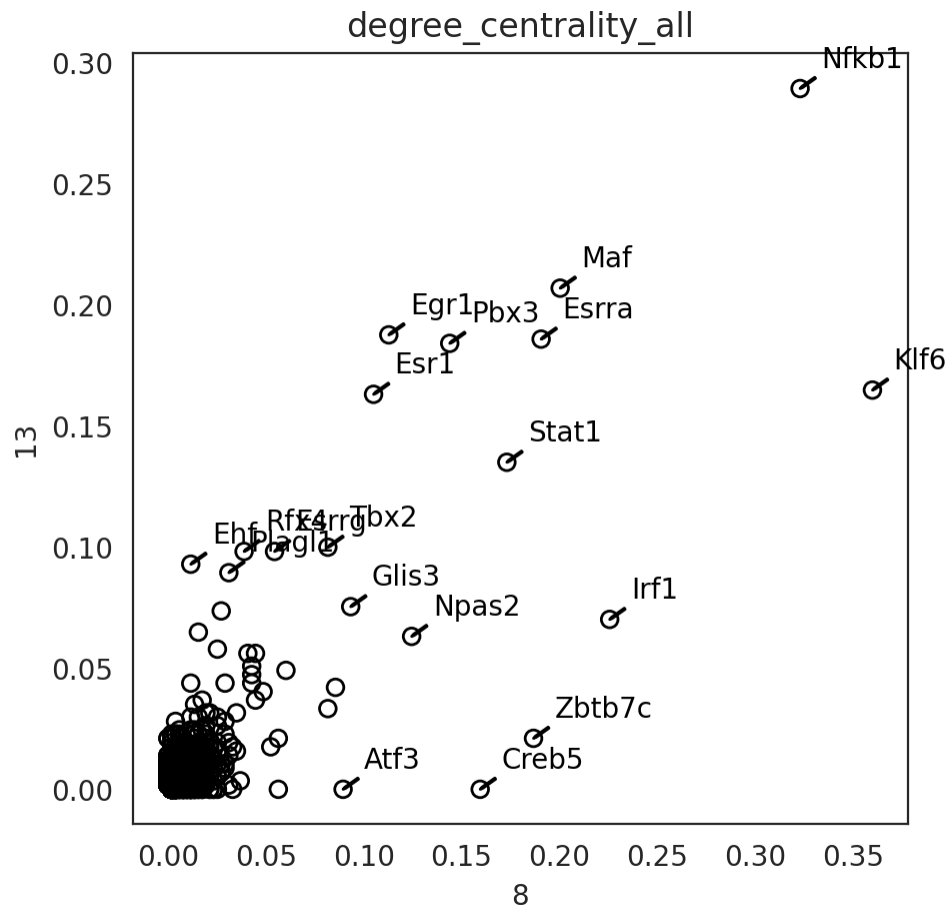
Processing clusters 8 and 11



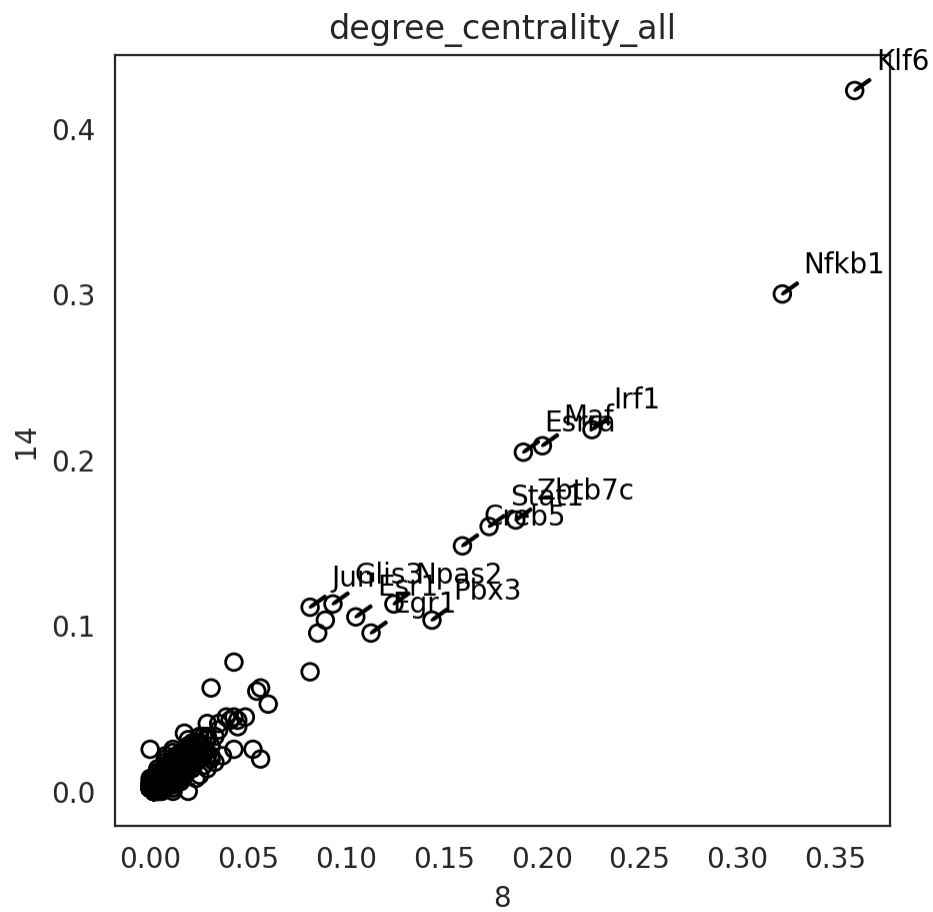
Processing clusters 8 and 12



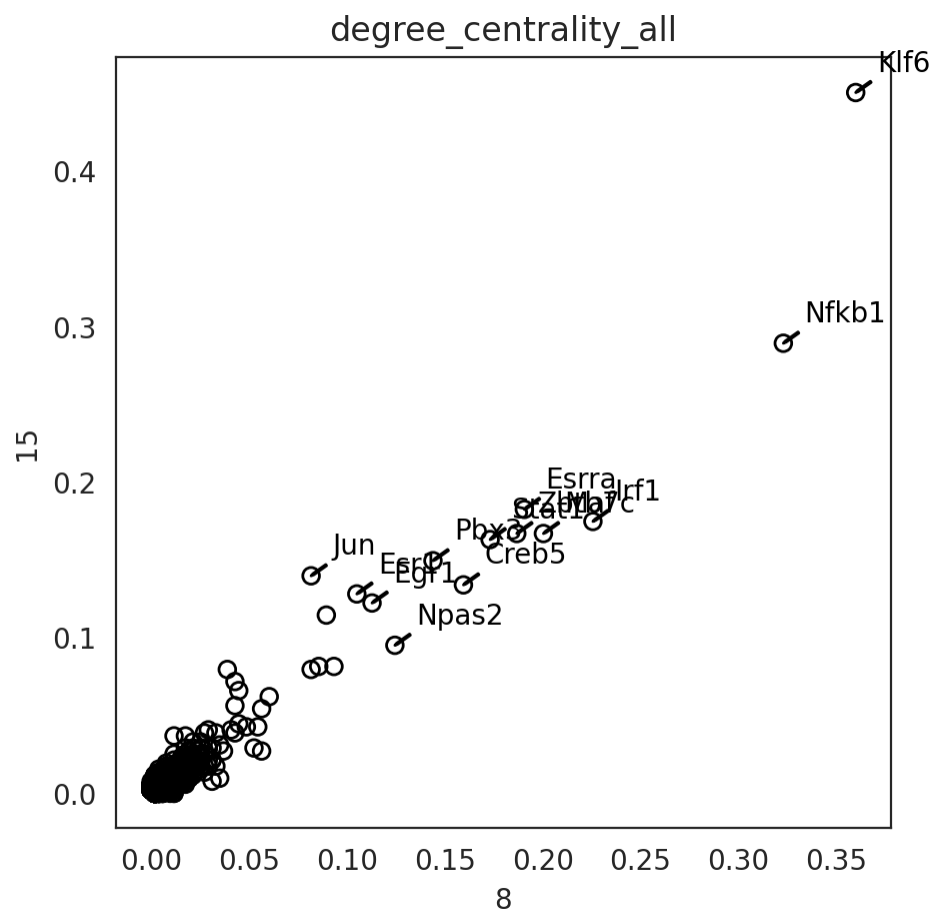
Processing clusters 8 and 13



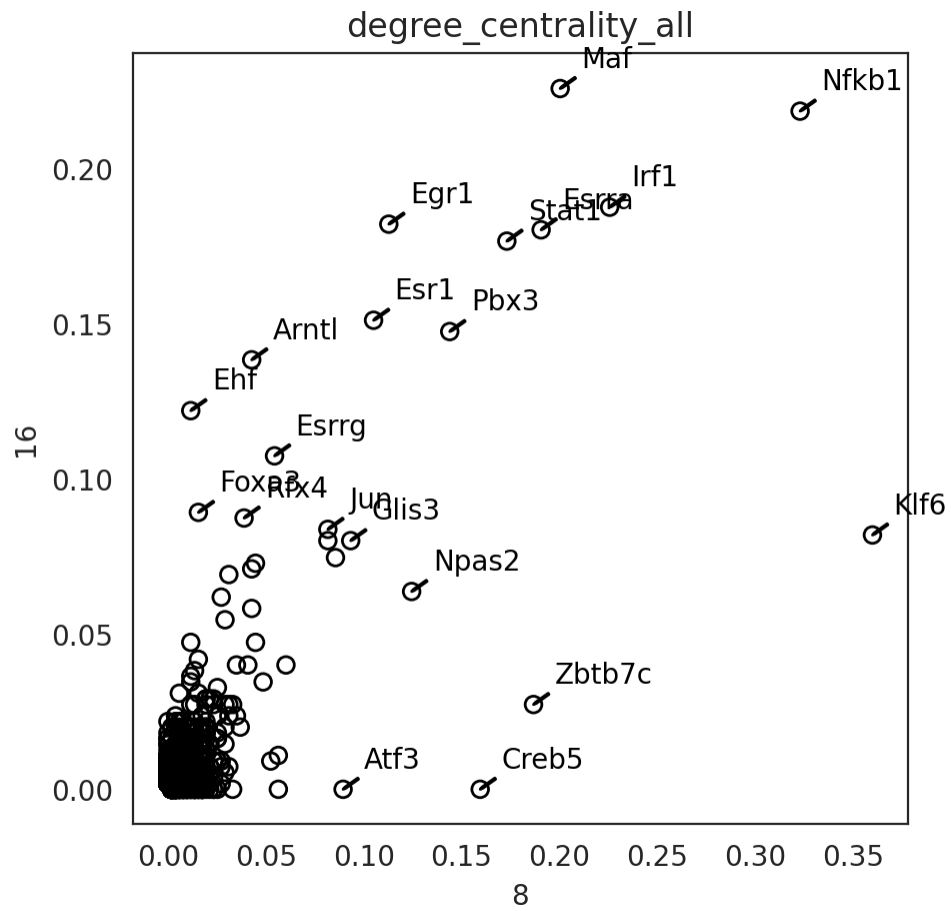
Processing clusters 8 and 14



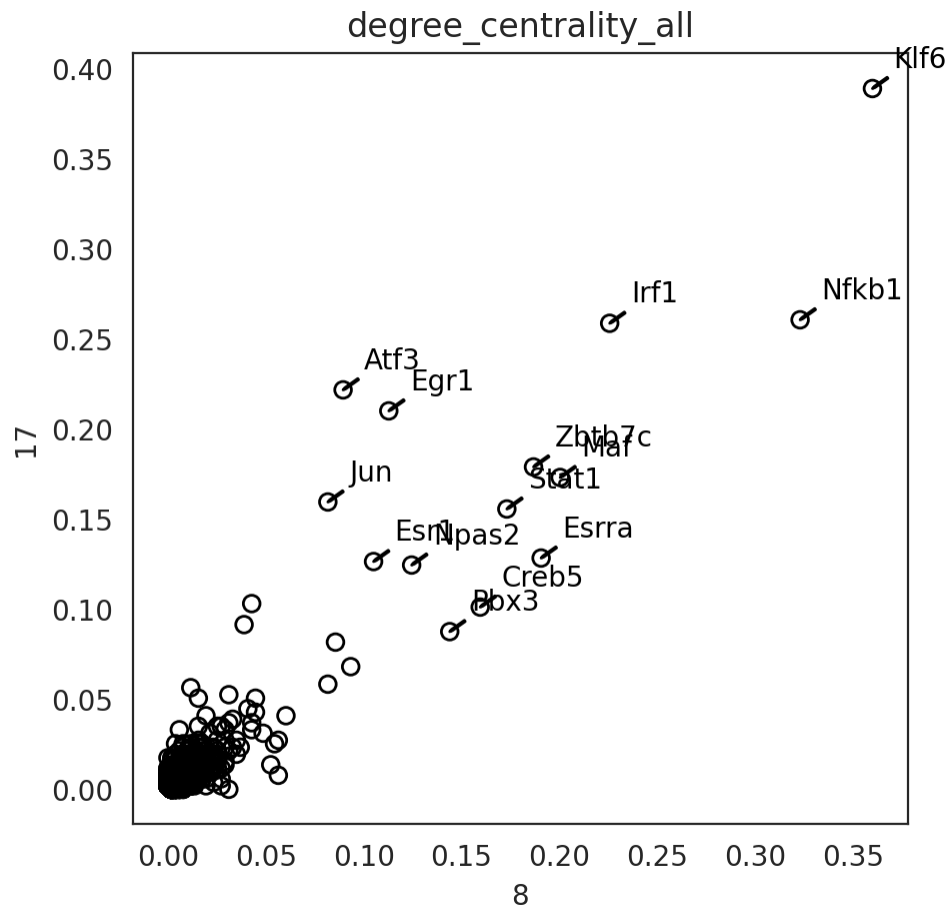
Processing clusters 8 and 15



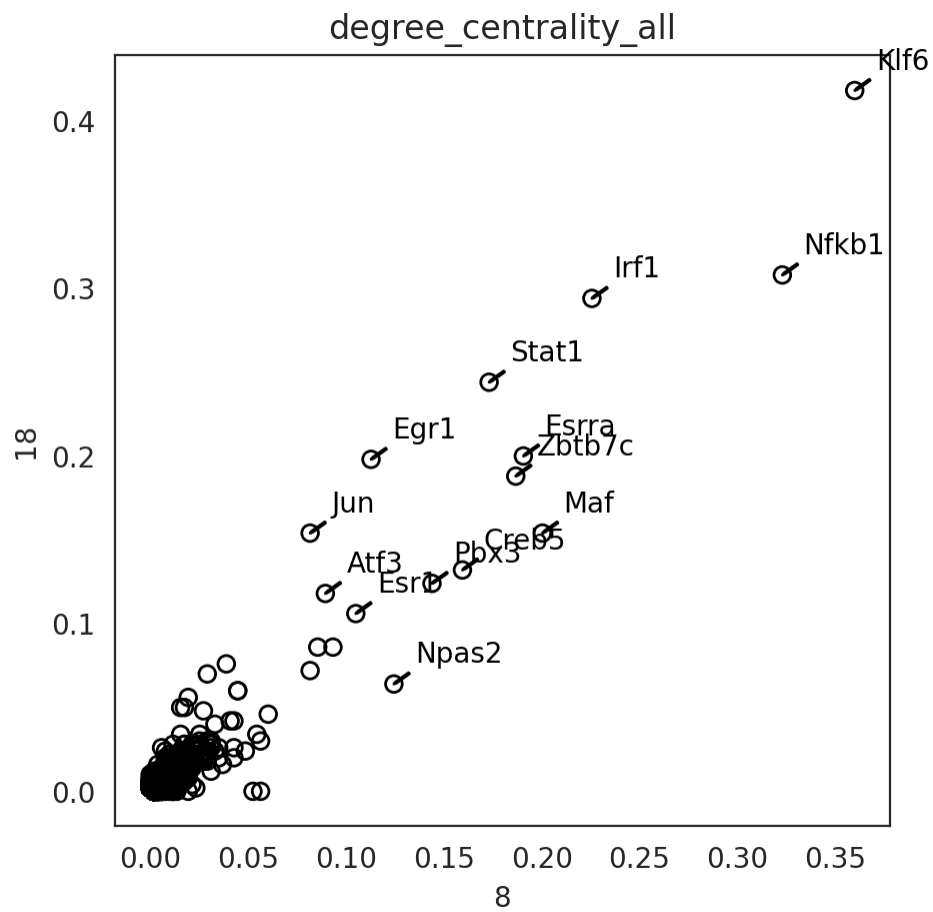
Processing clusters 8 and 16



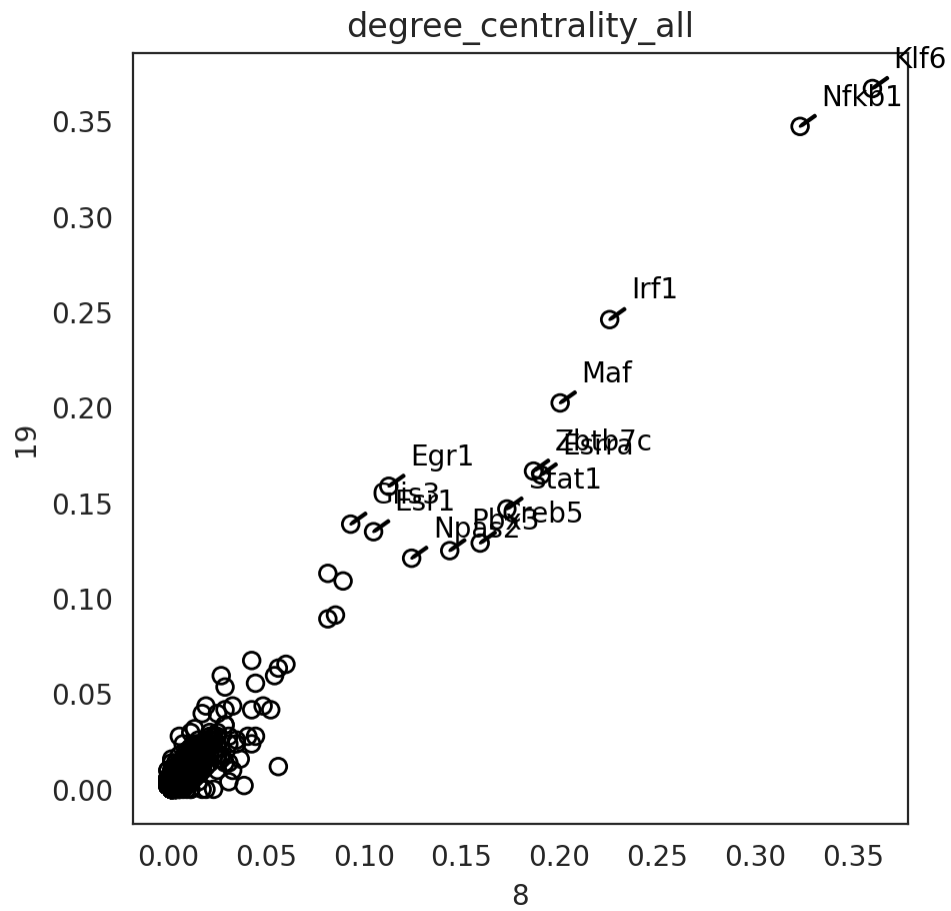
Processing clusters 8 and 17



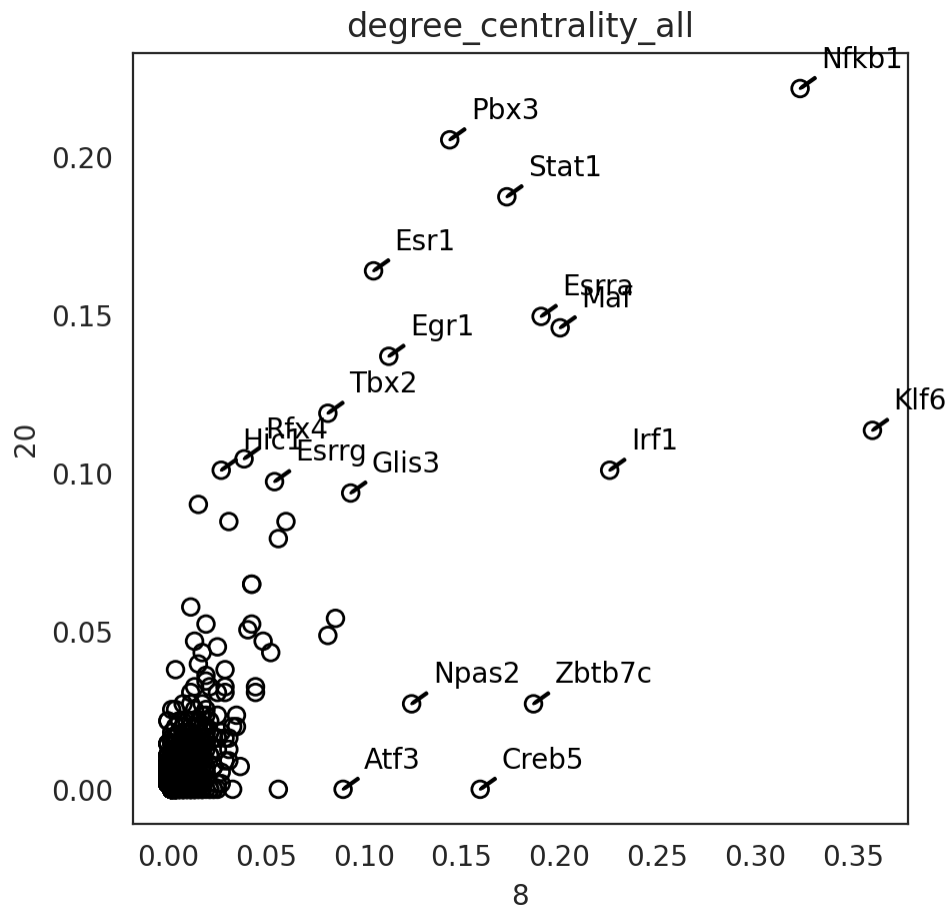
Processing clusters 8 and 18



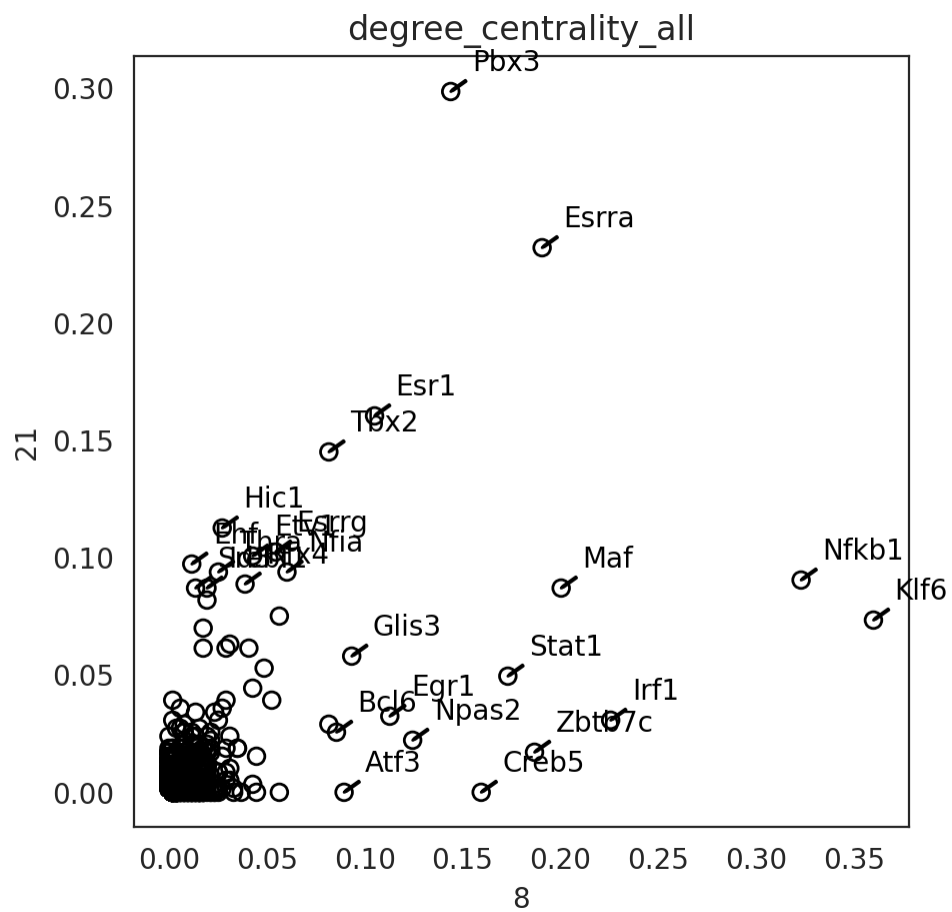
Processing clusters 8 and 19

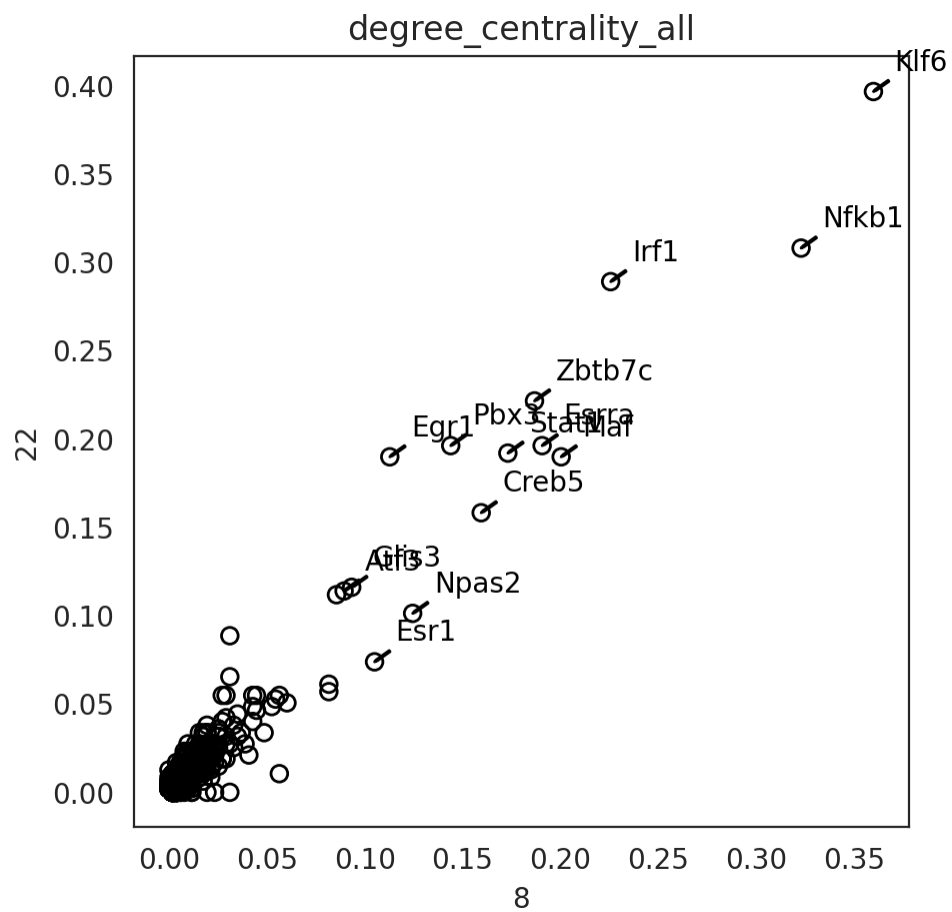


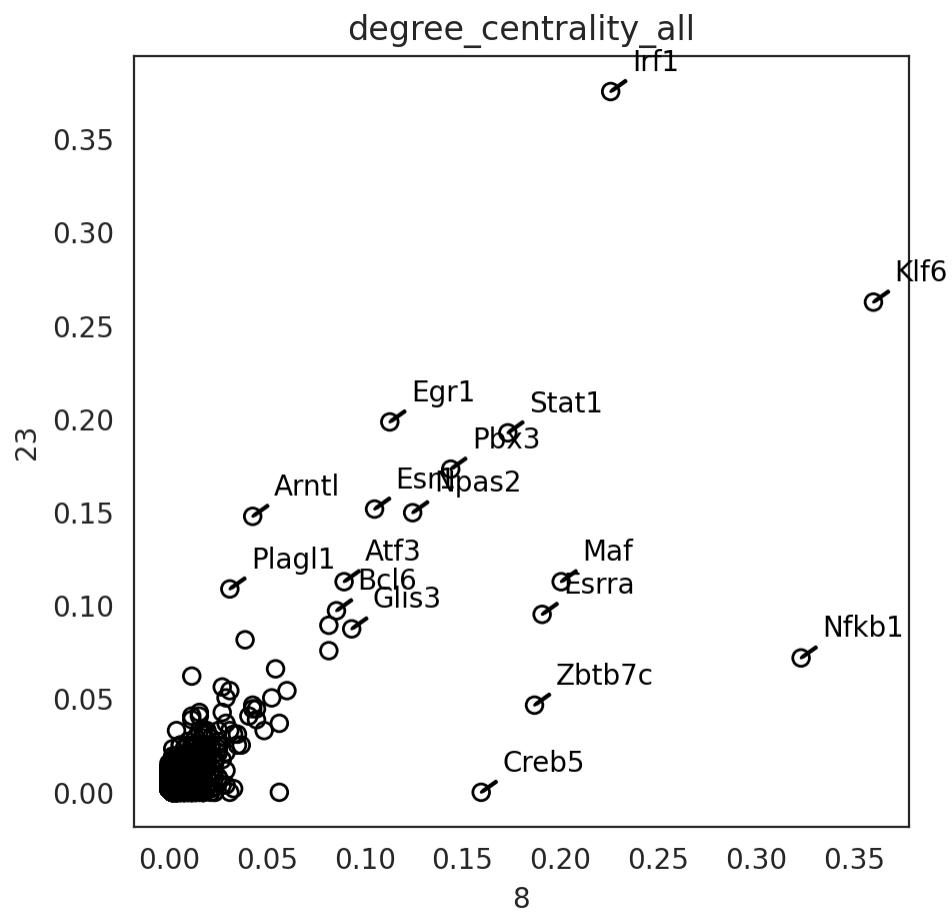
Processing clusters 8 and 20



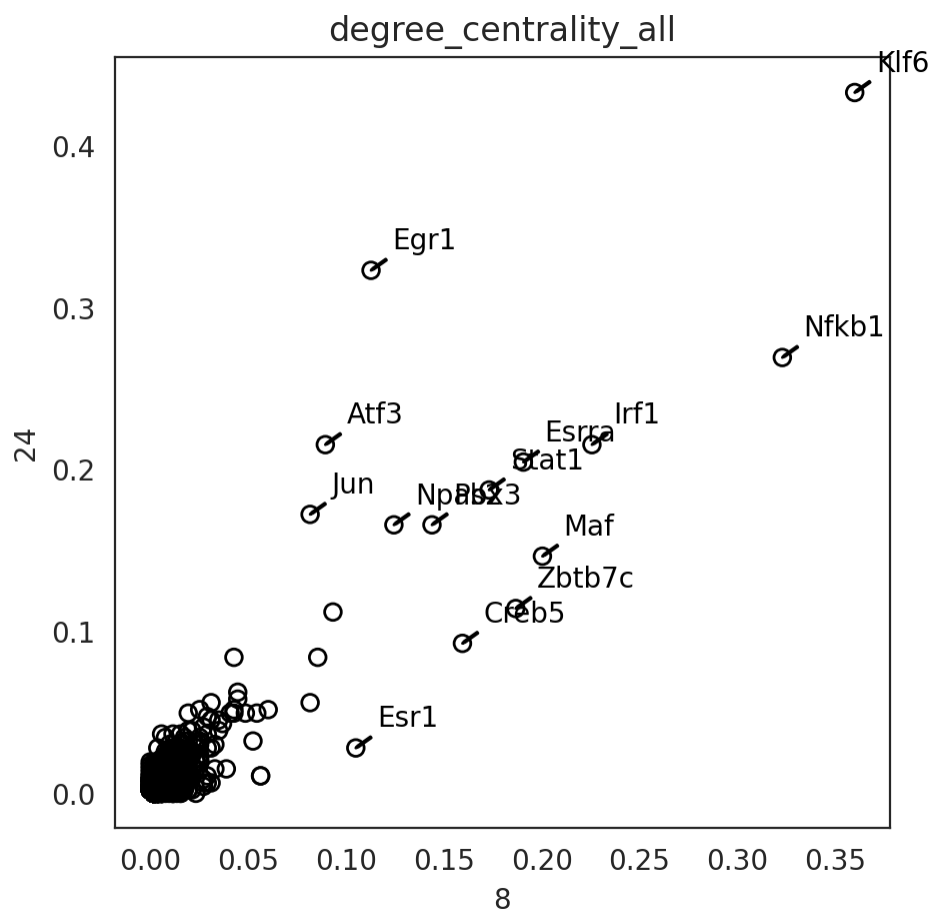
Processing clusters 8 and 21



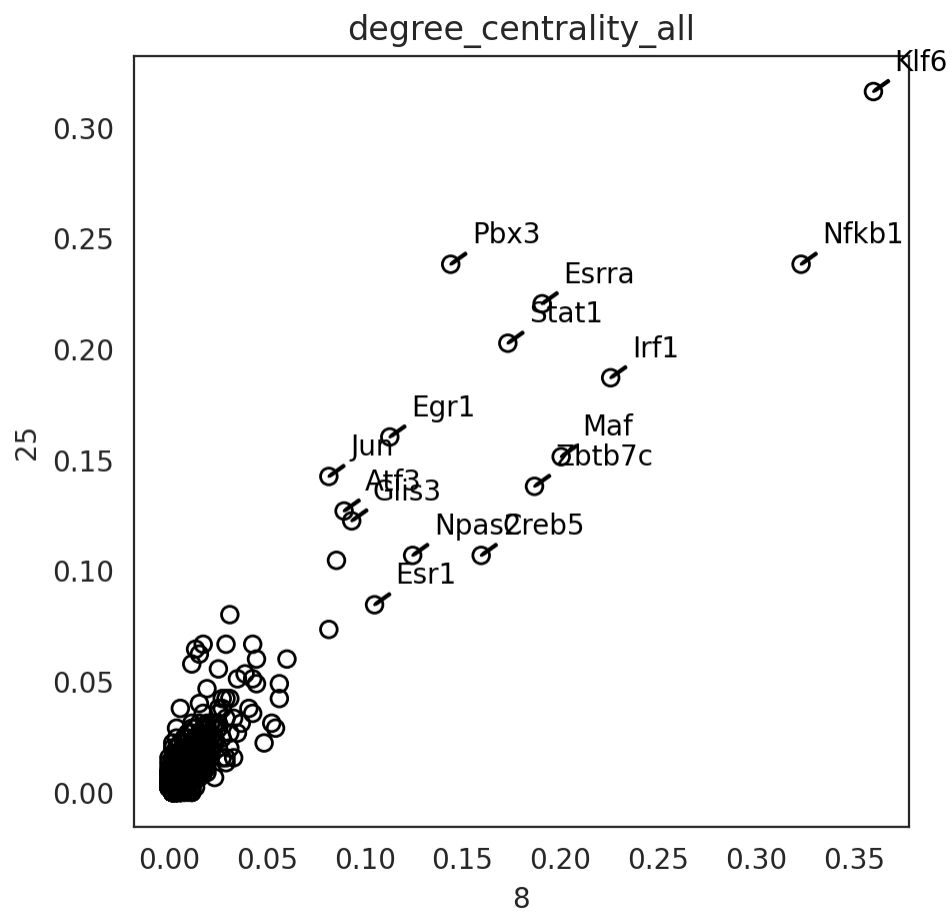




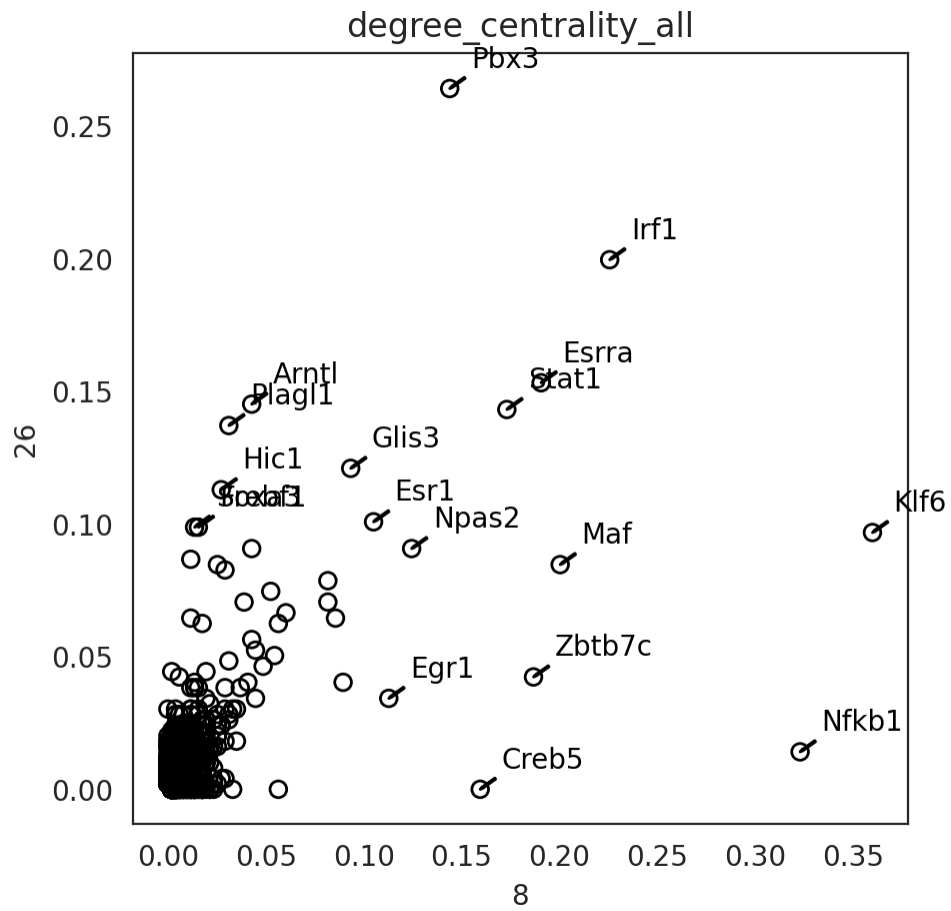
Processing clusters 8 and 24



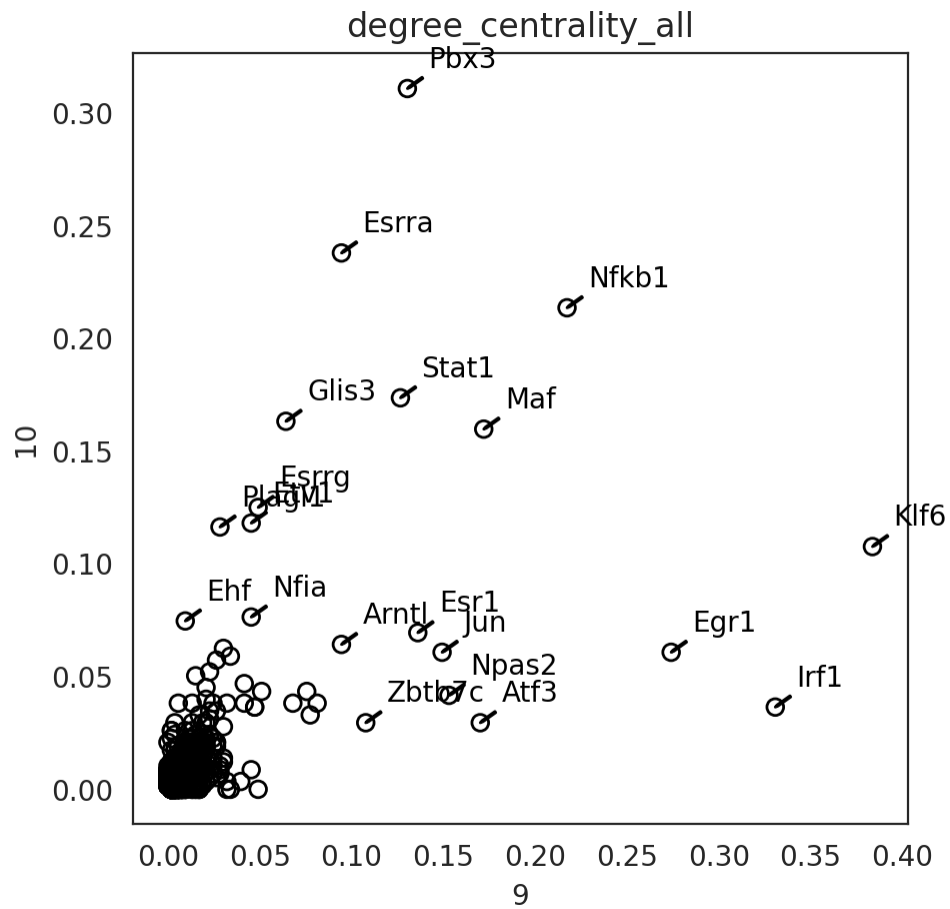
Processing clusters 8 and 25



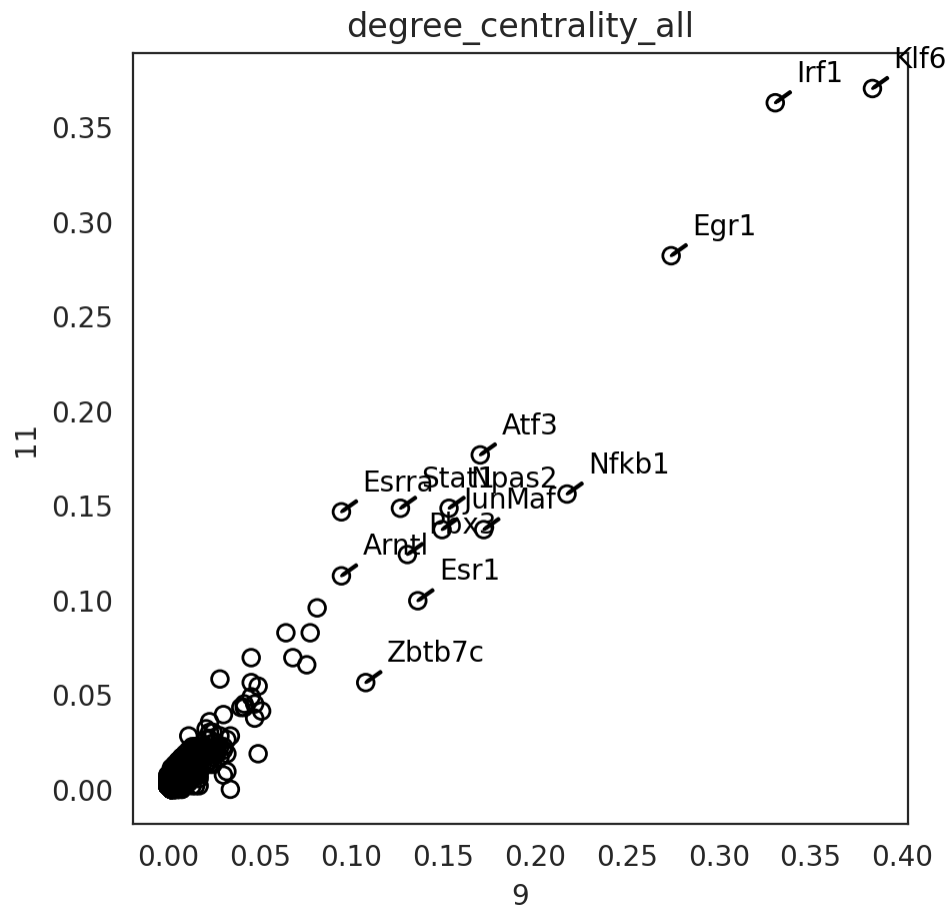
Processing clusters 8 and 26



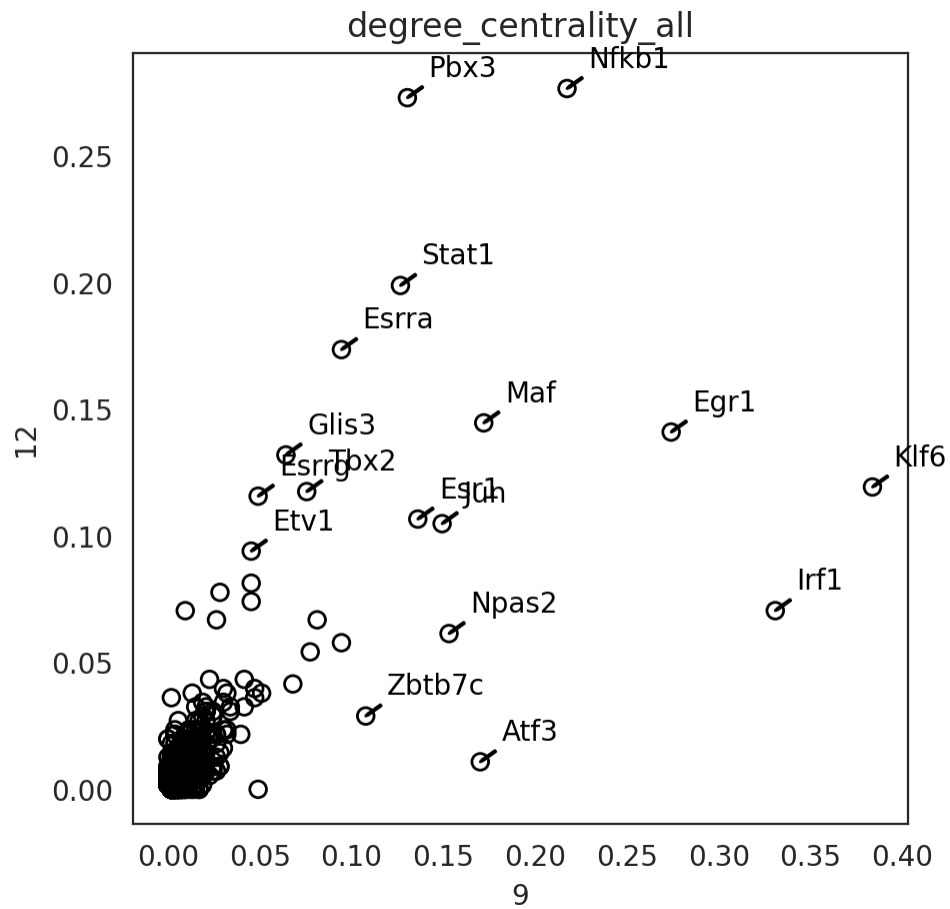
Processing clusters 9 and 10



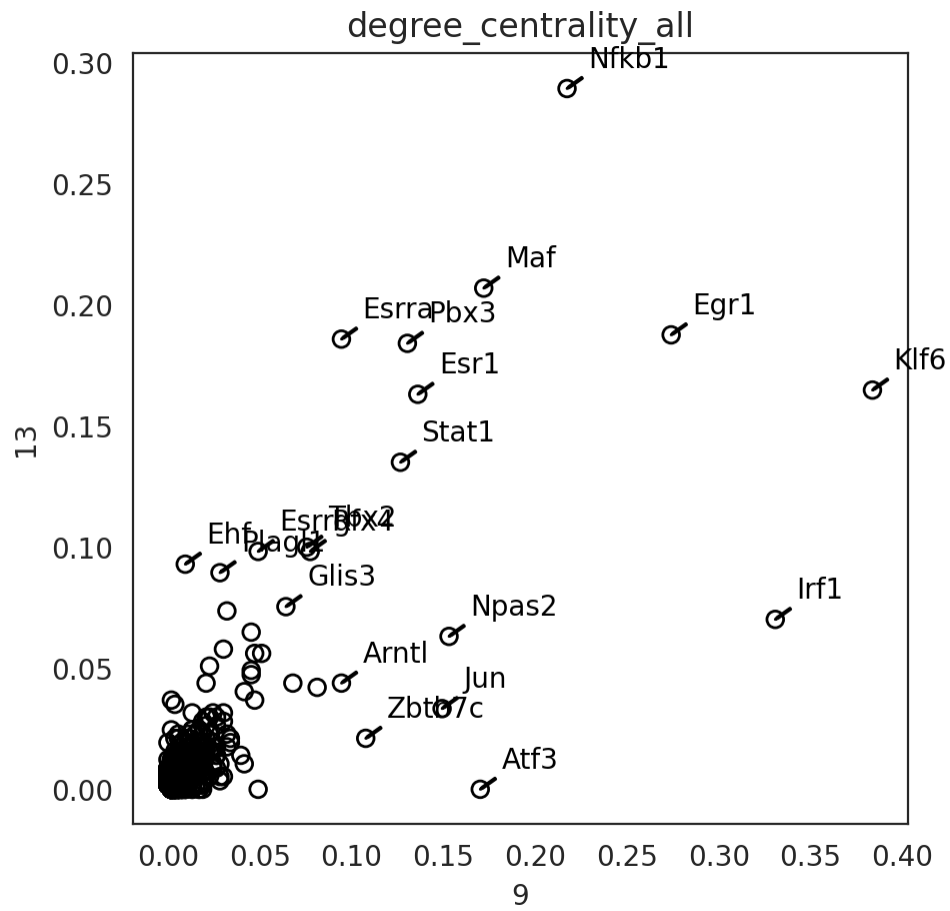
Processing clusters 9 and 11



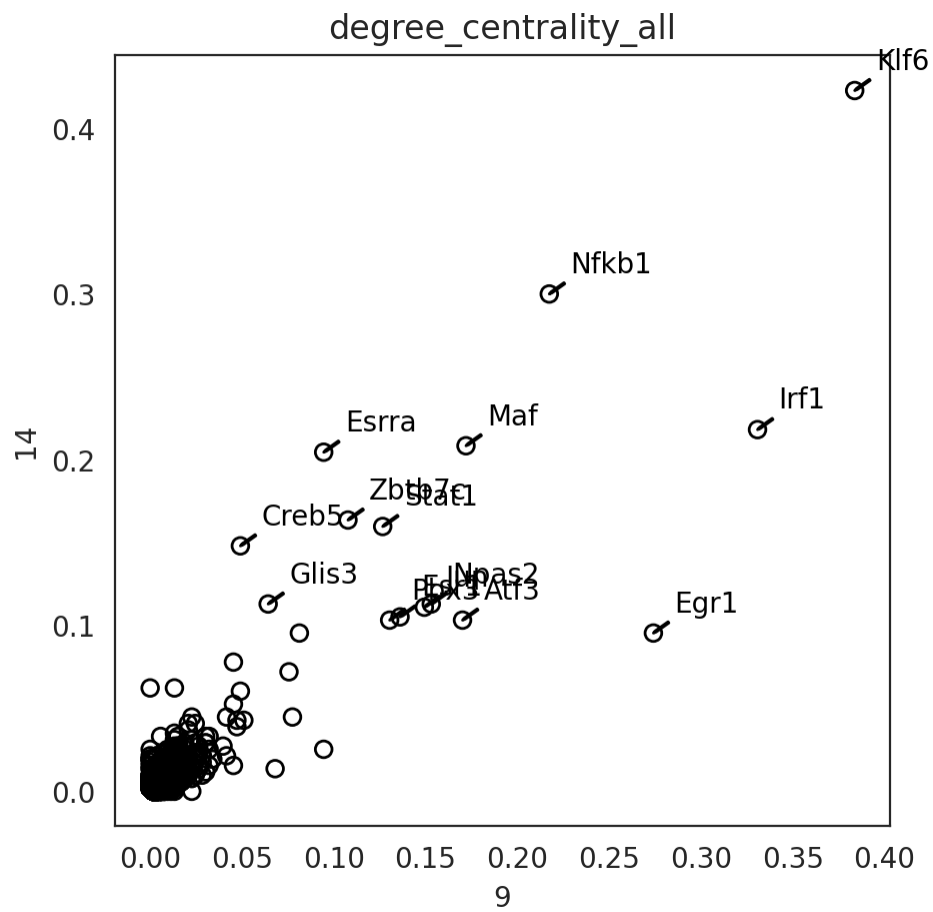
Processing clusters 9 and 12



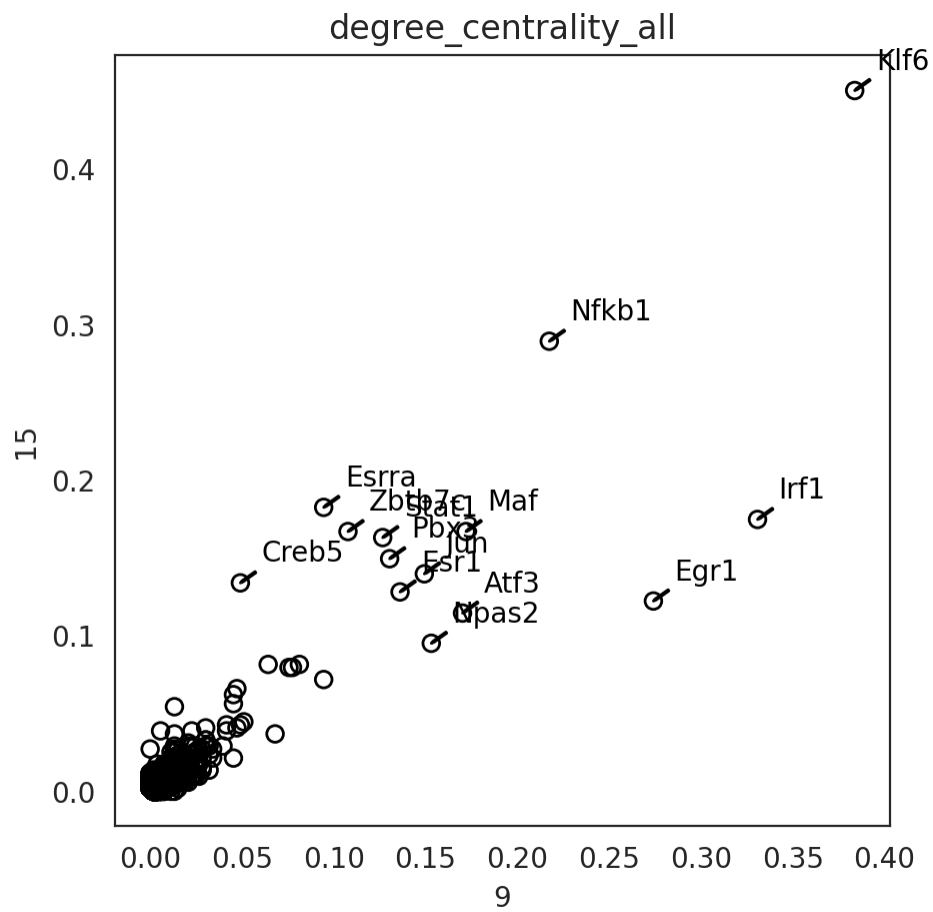
Processing clusters 9 and 13



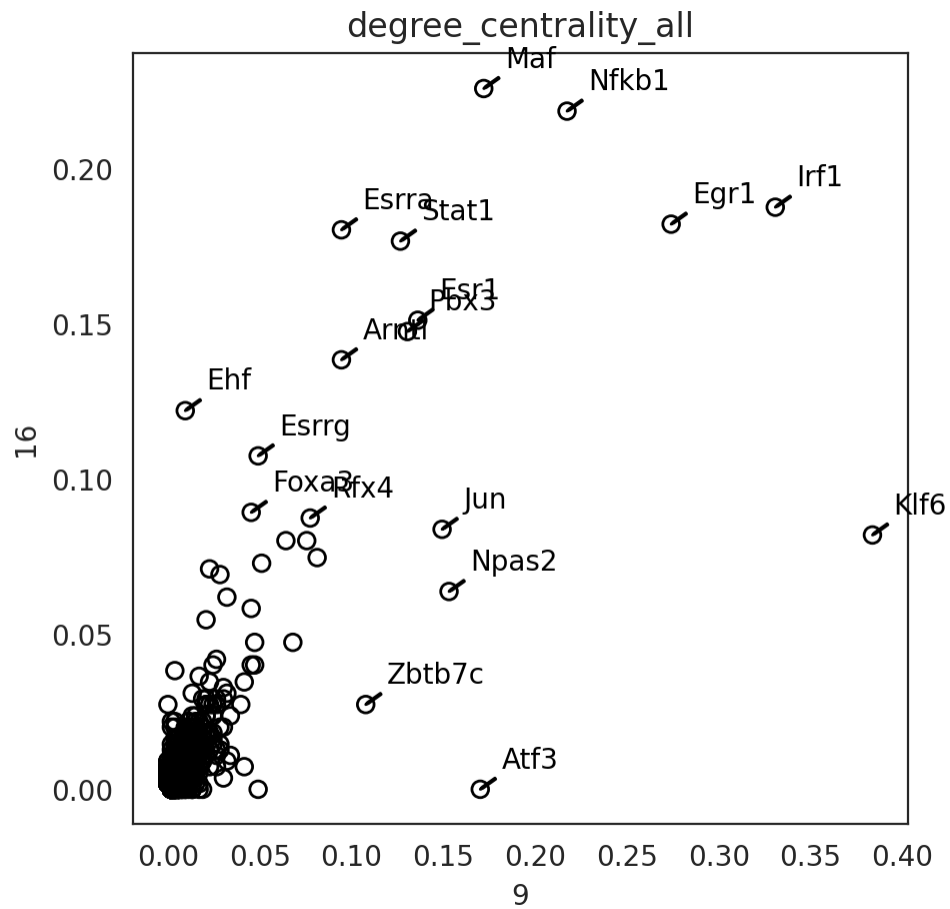
Processing clusters 9 and 14



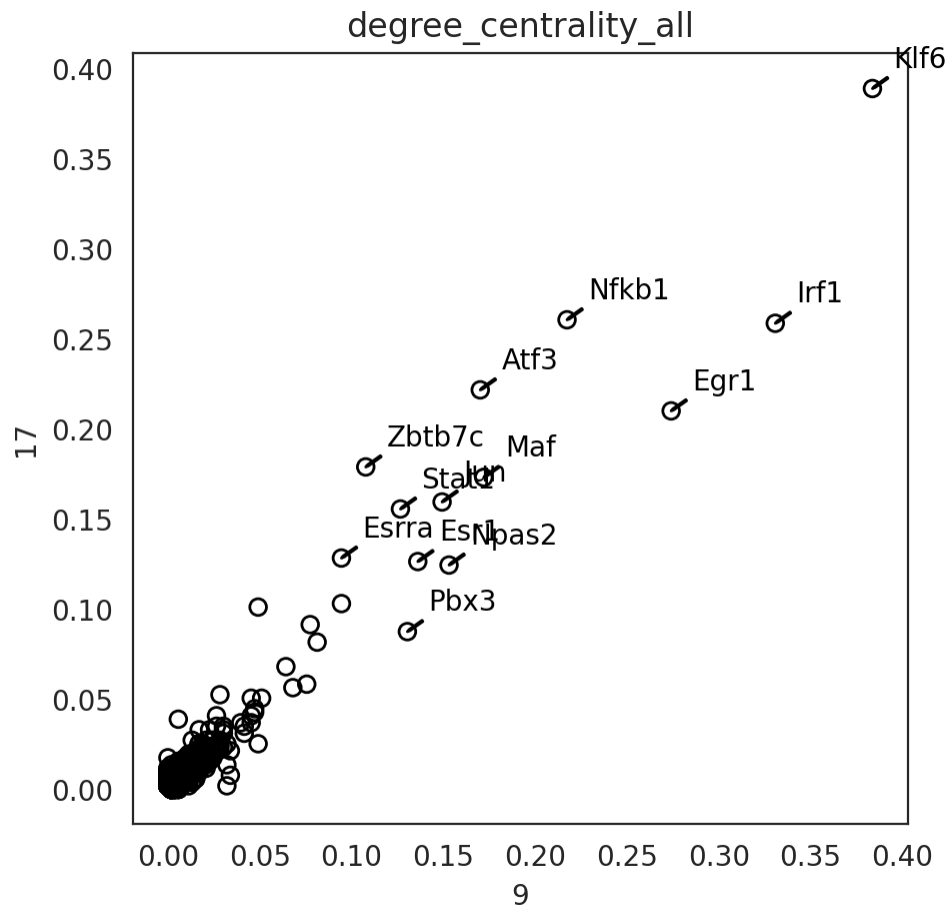
Processing clusters 9 and 15



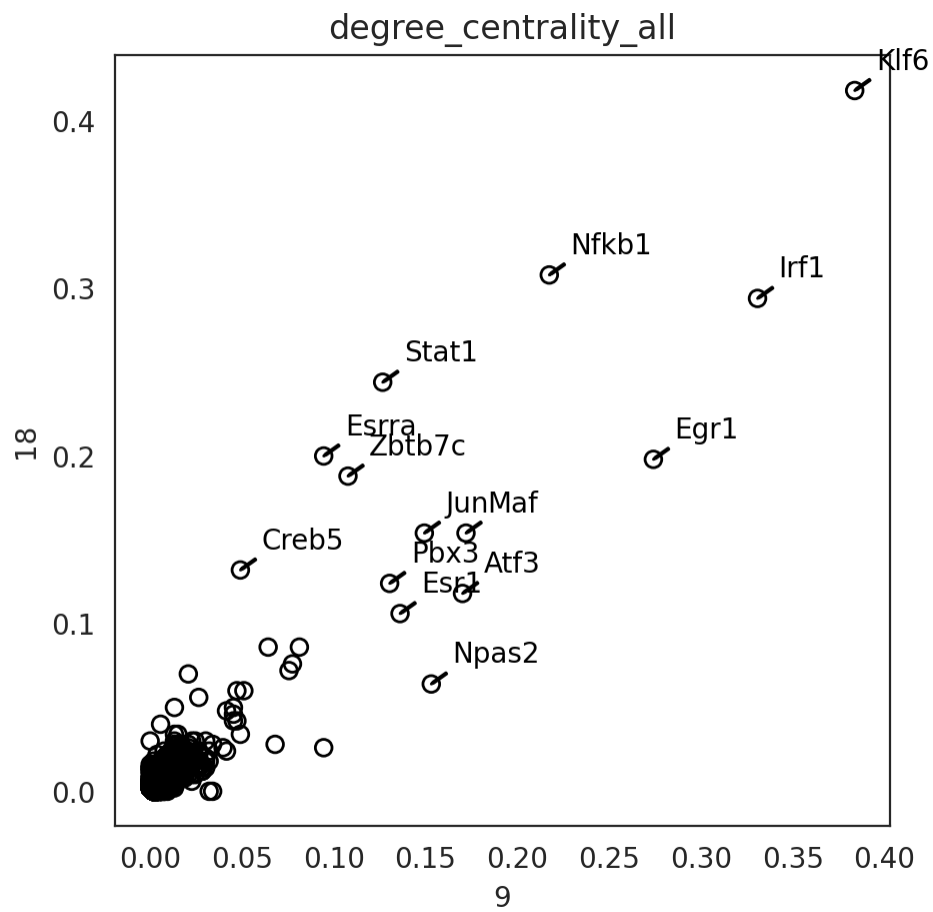
Processing clusters 9 and 16



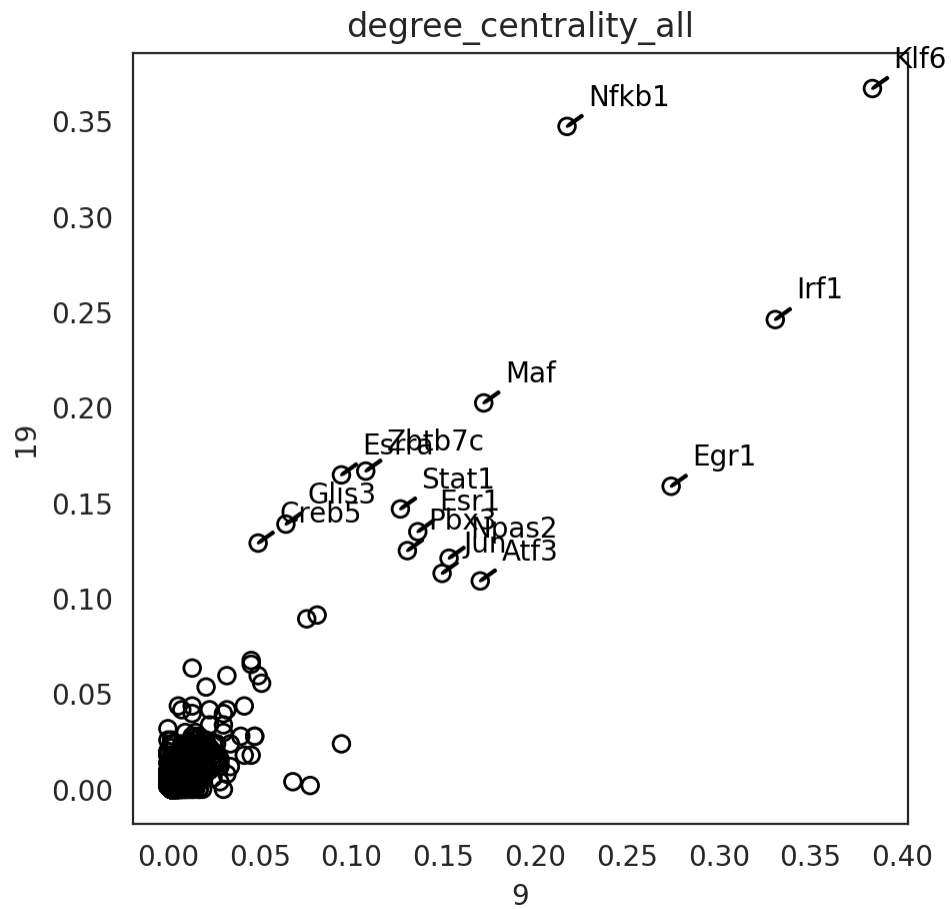
Processing clusters 9 and 17



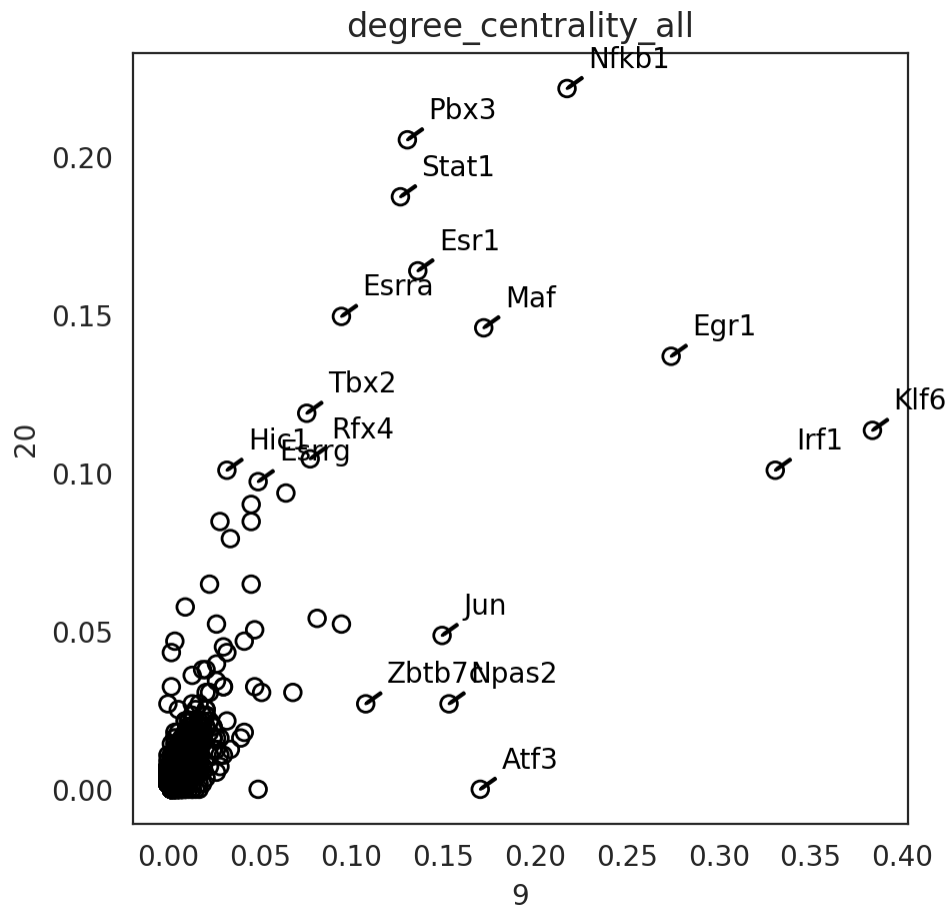
Processing clusters 9 and 18



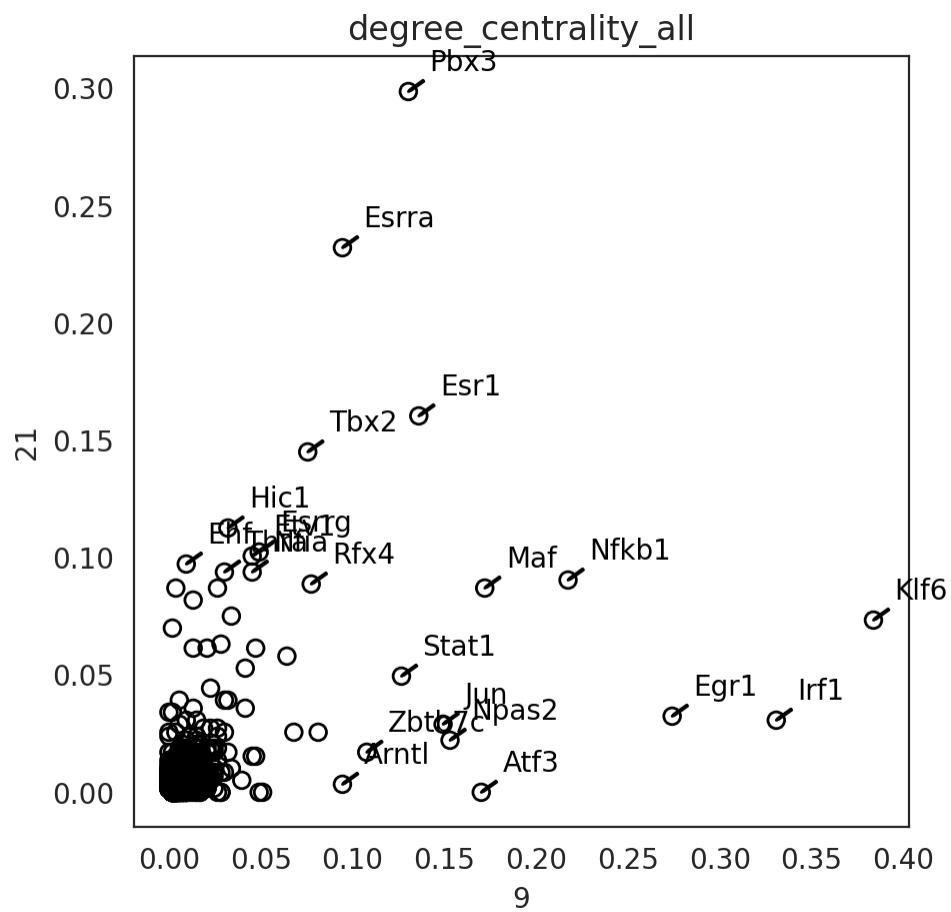
Processing clusters 9 and 19



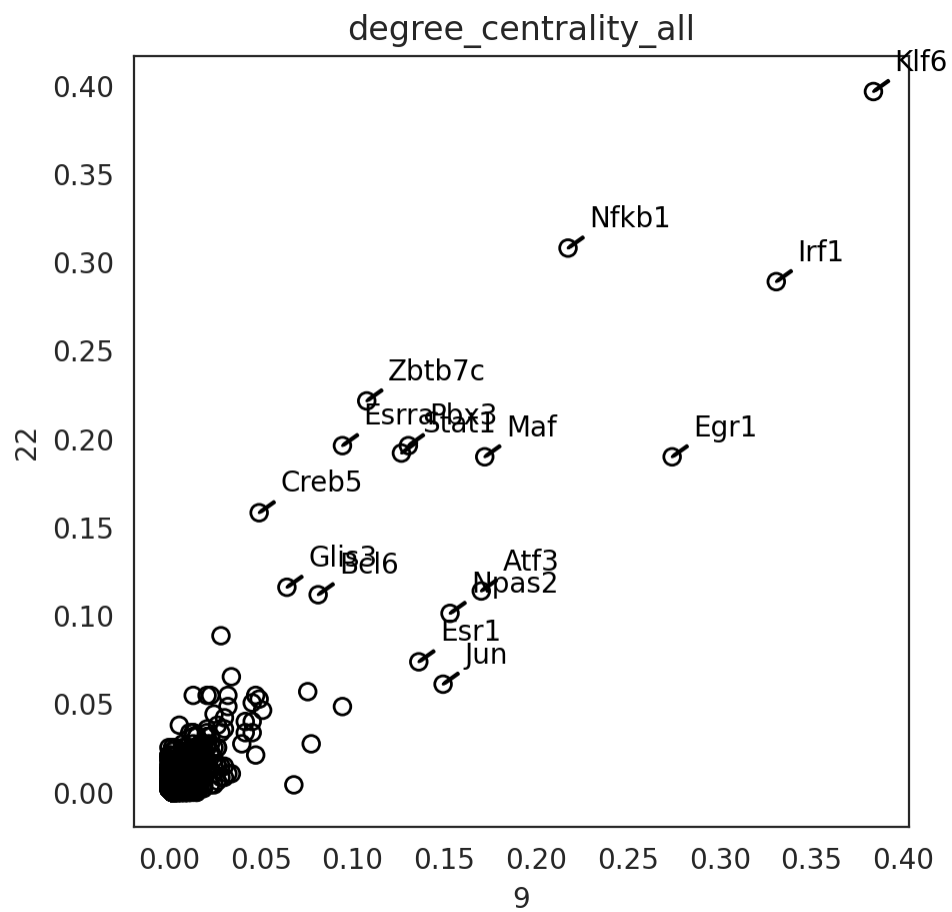
Processing clusters 9 and 20



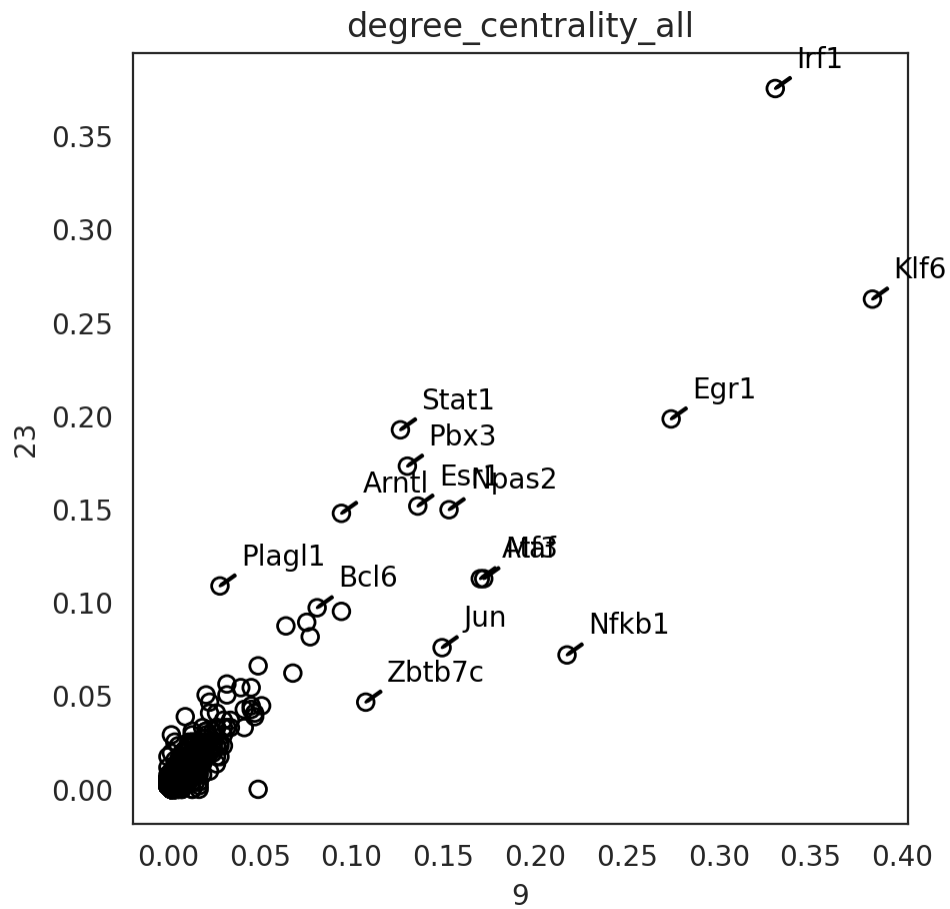
Processing clusters 9 and 21

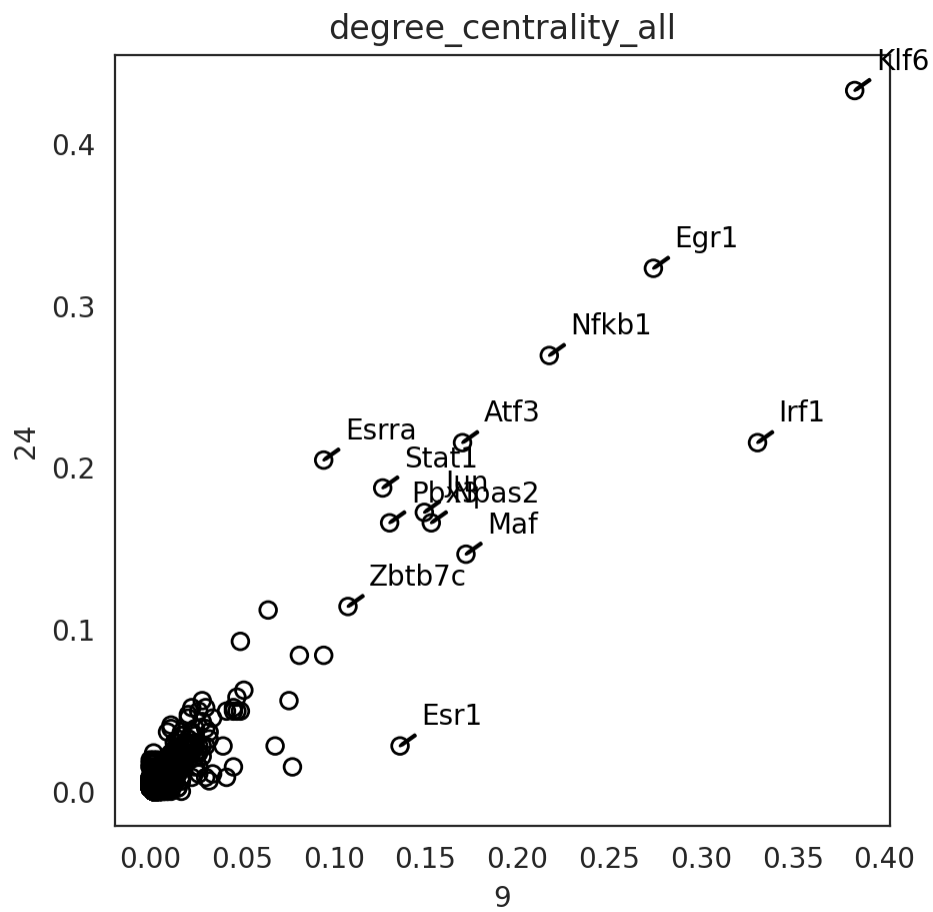


Processing clusters 9 and 22

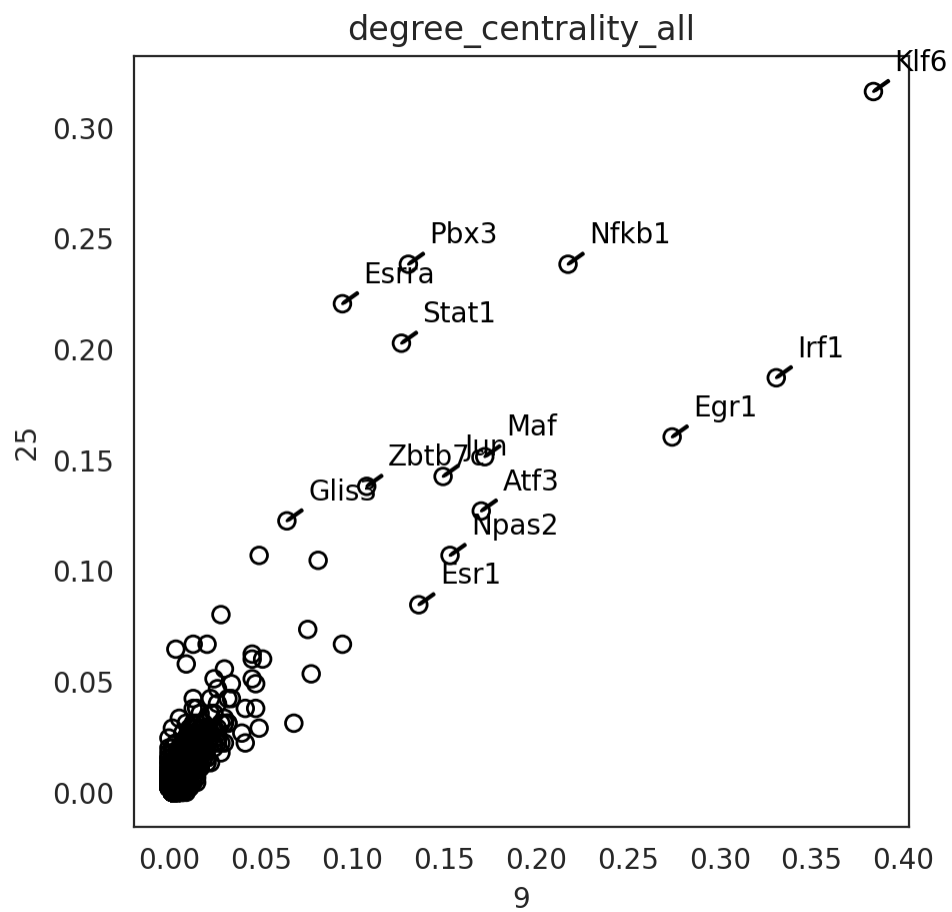


Processing clusters 9 and 23

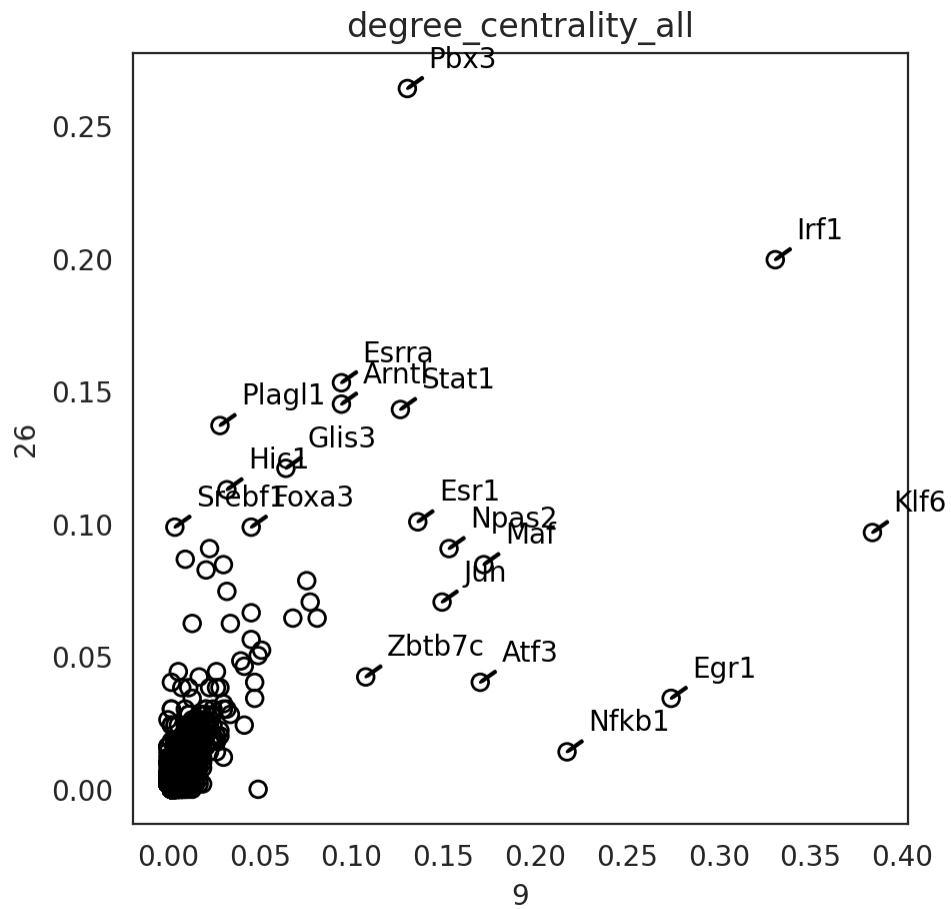




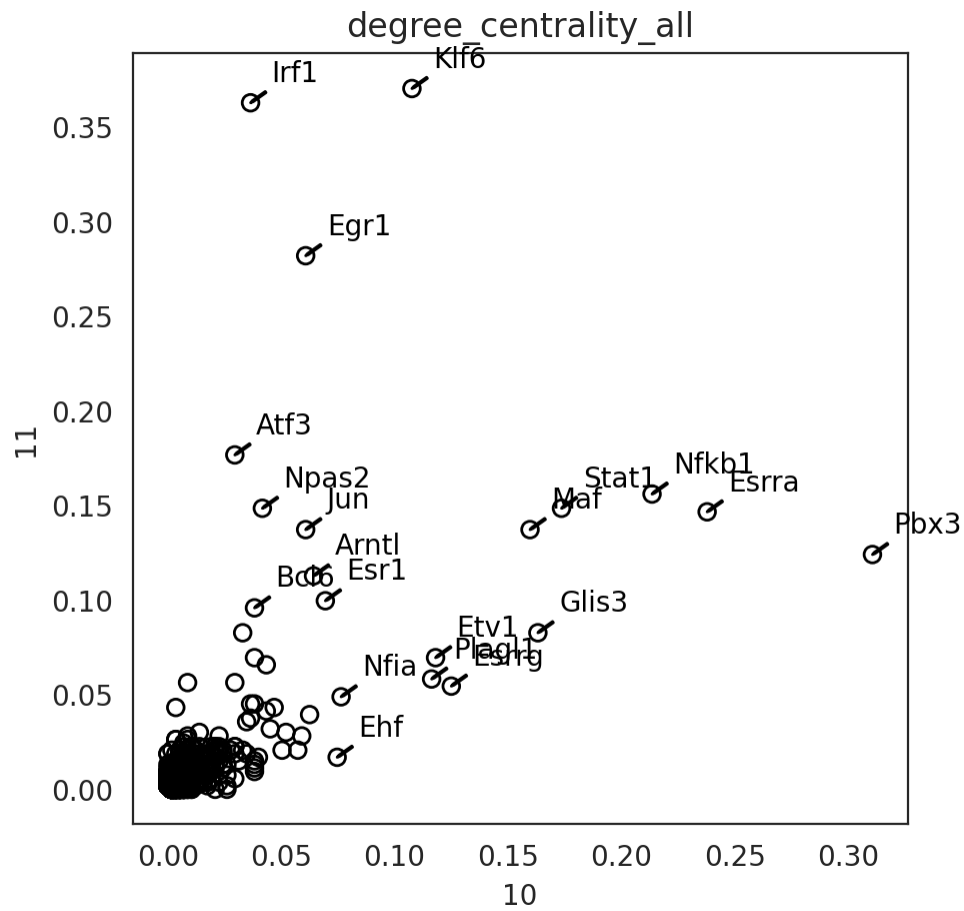
Processing clusters 9 and 25



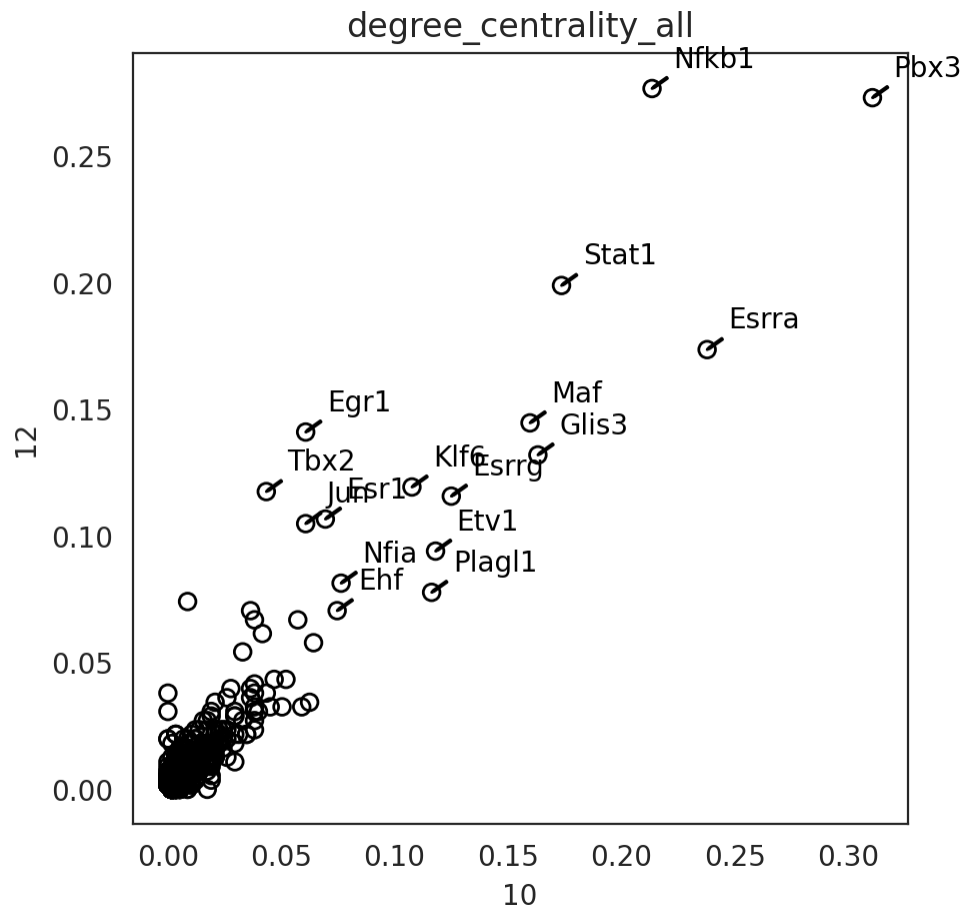
Processing clusters 9 and 26



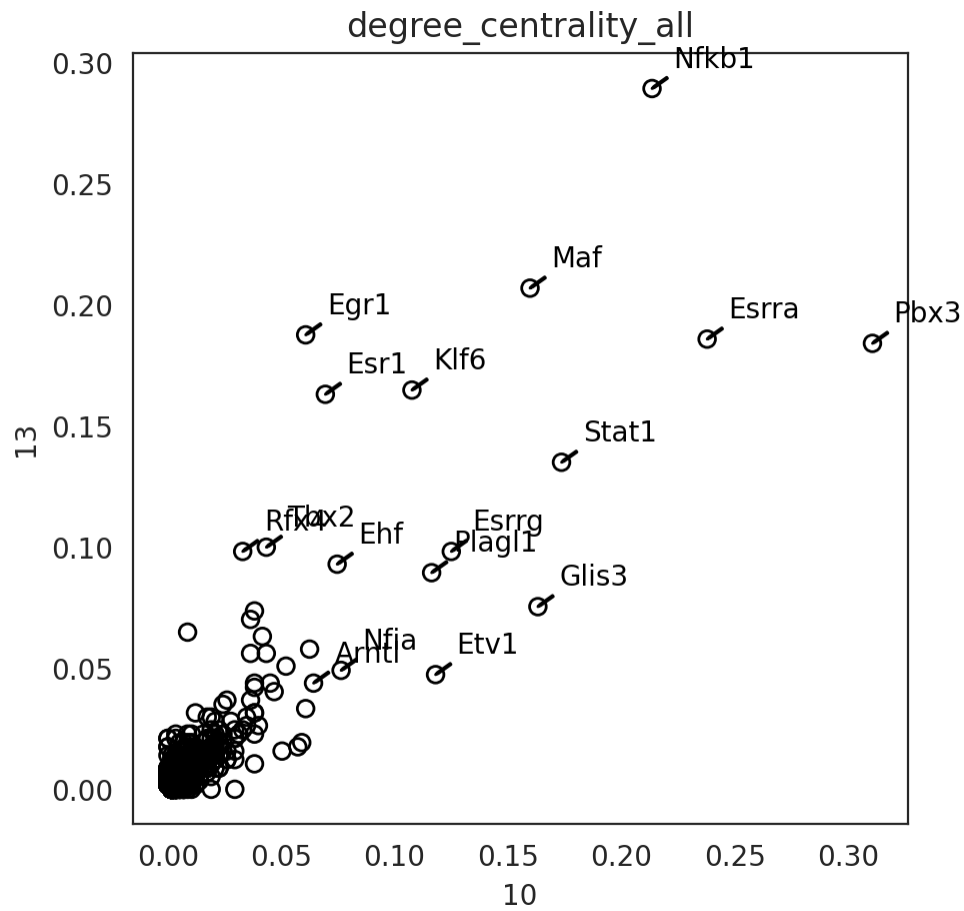
Processing clusters 10 and 11

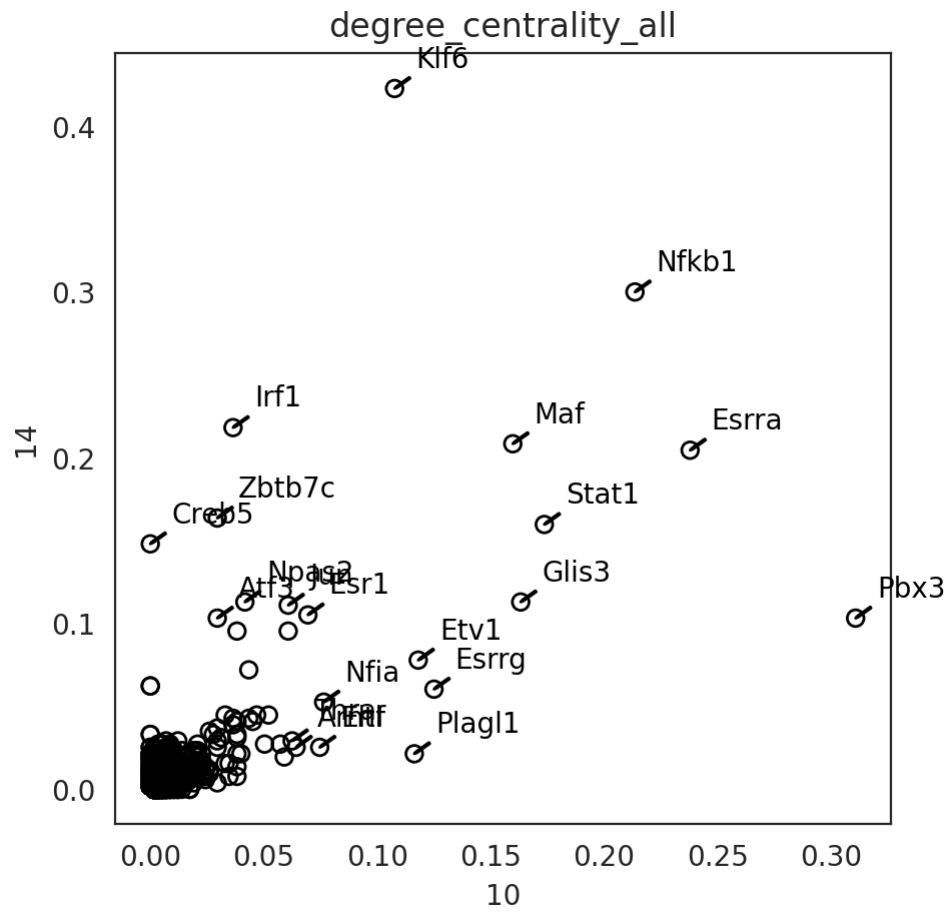


Processing clusters 10 and 12

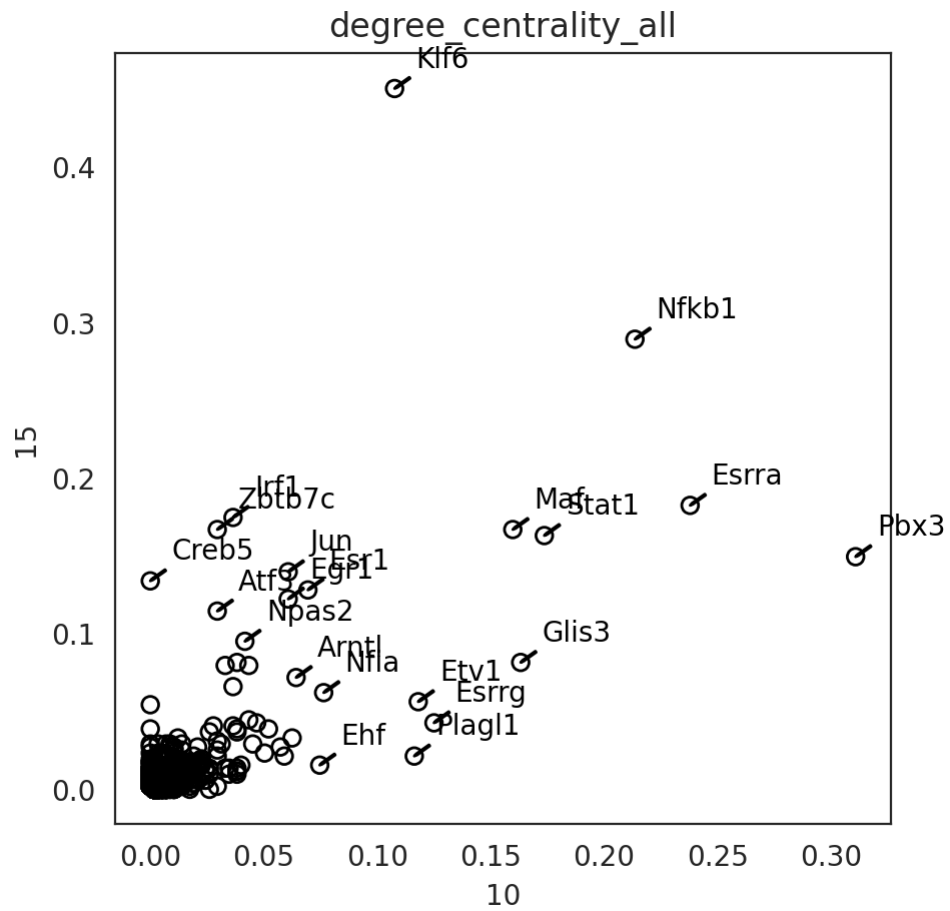


Processing clusters 10 and 13

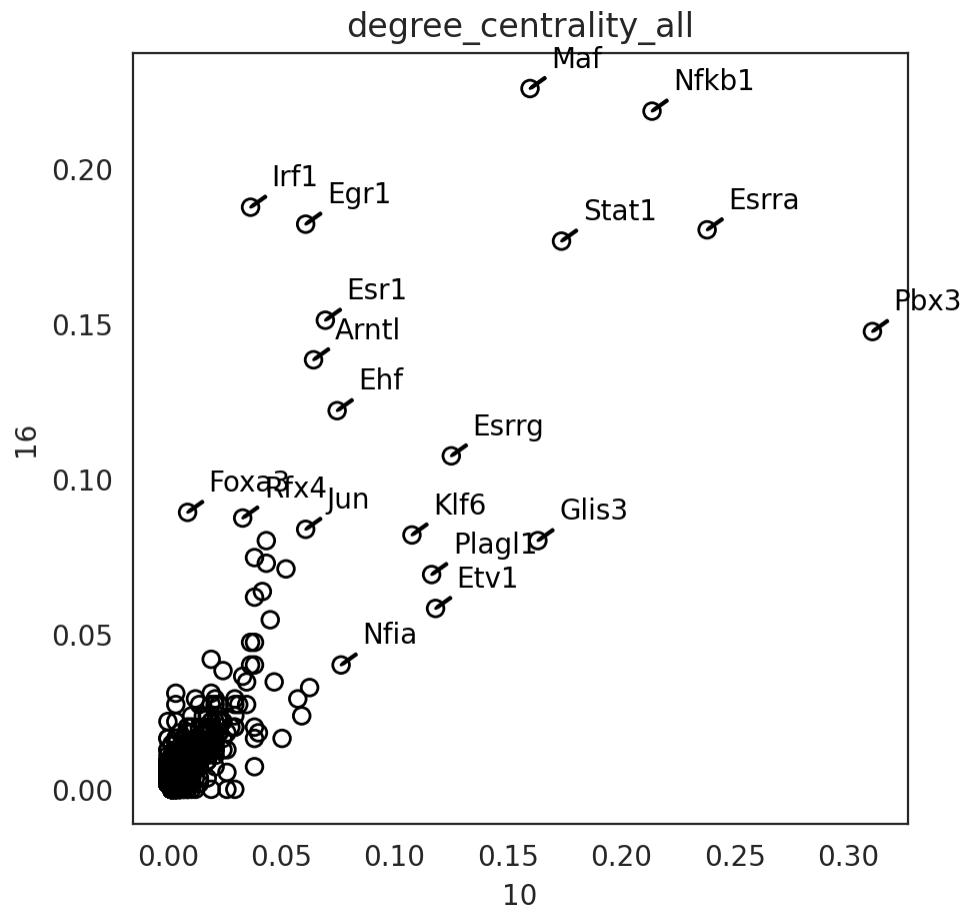




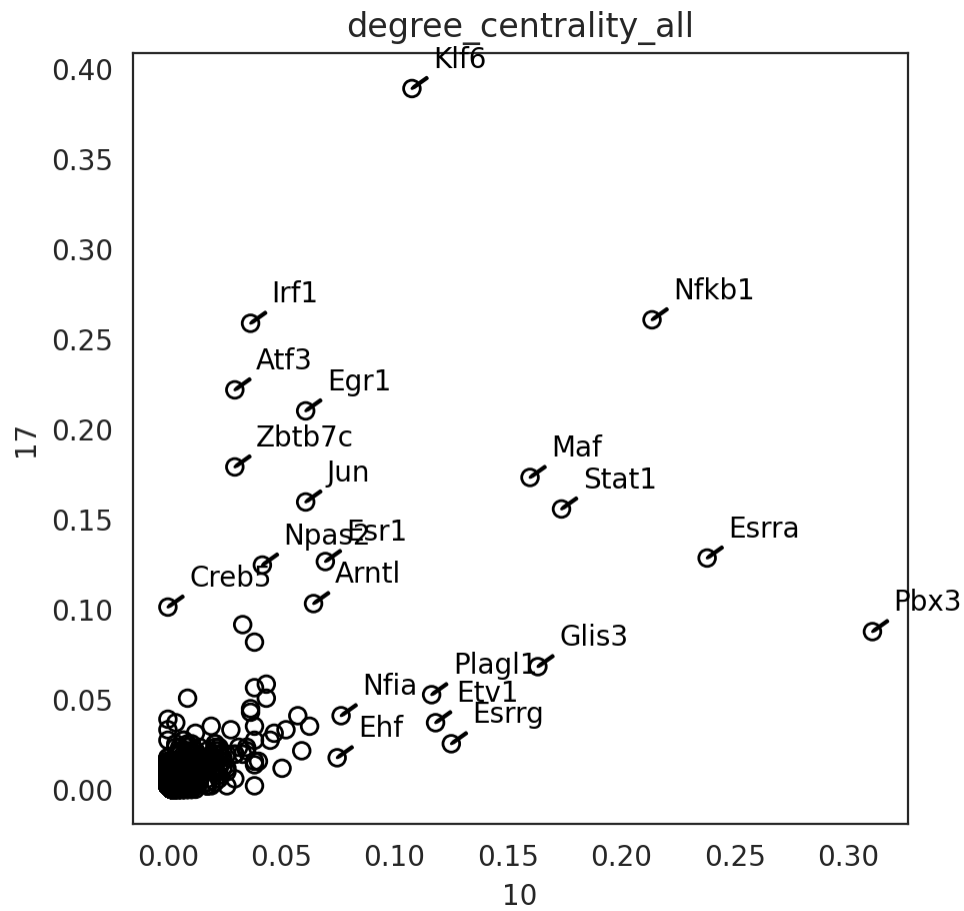
Processing clusters 10 and 15



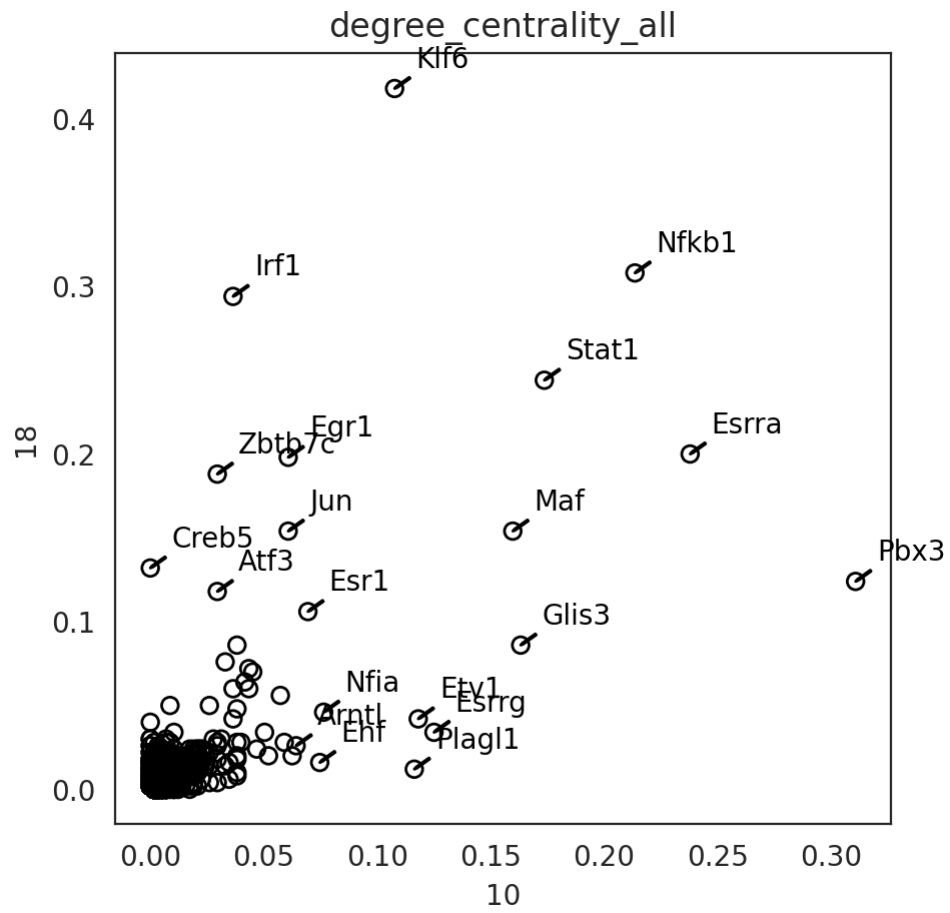
Processing clusters 10 and 16



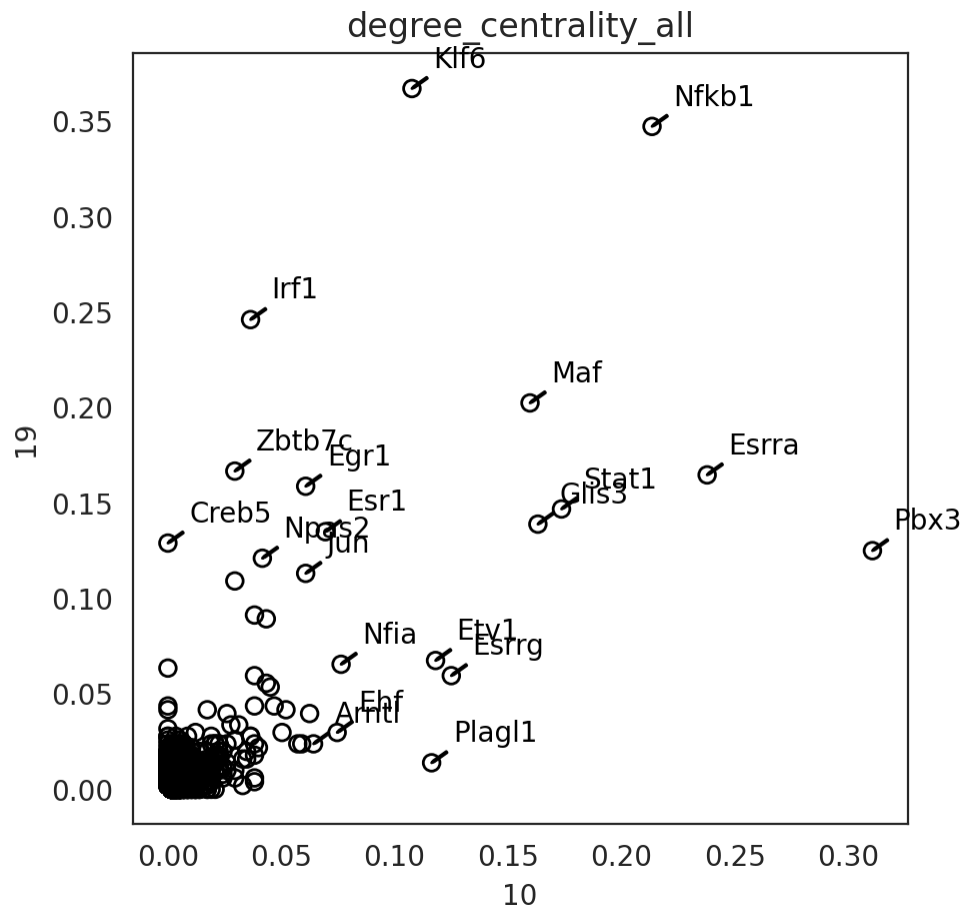
Processing clusters 10 and 17



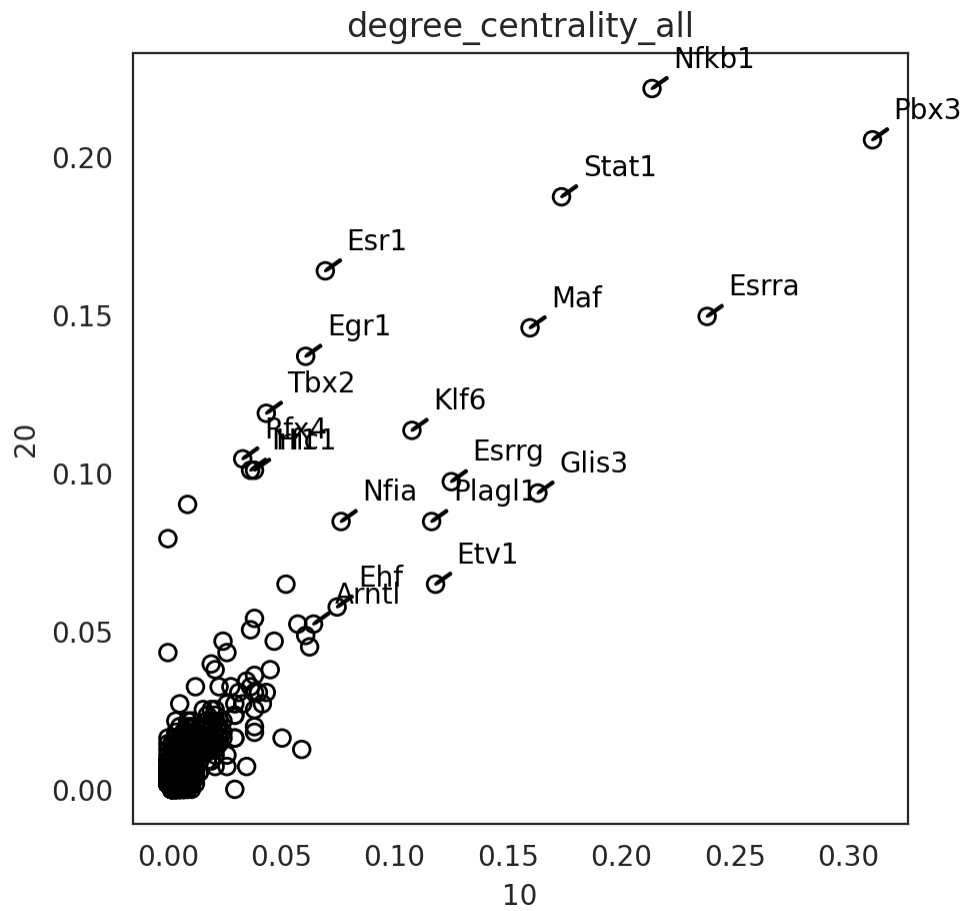
Processing clusters 10 and 18



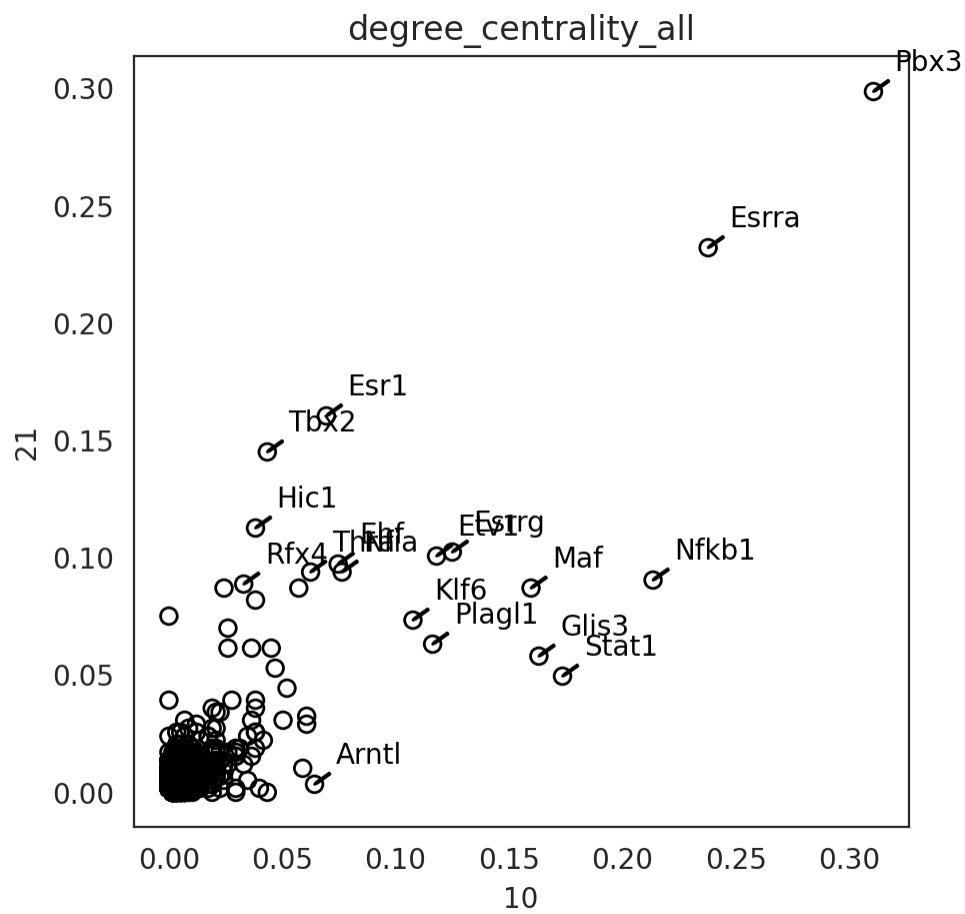
Processing clusters 10 and 19

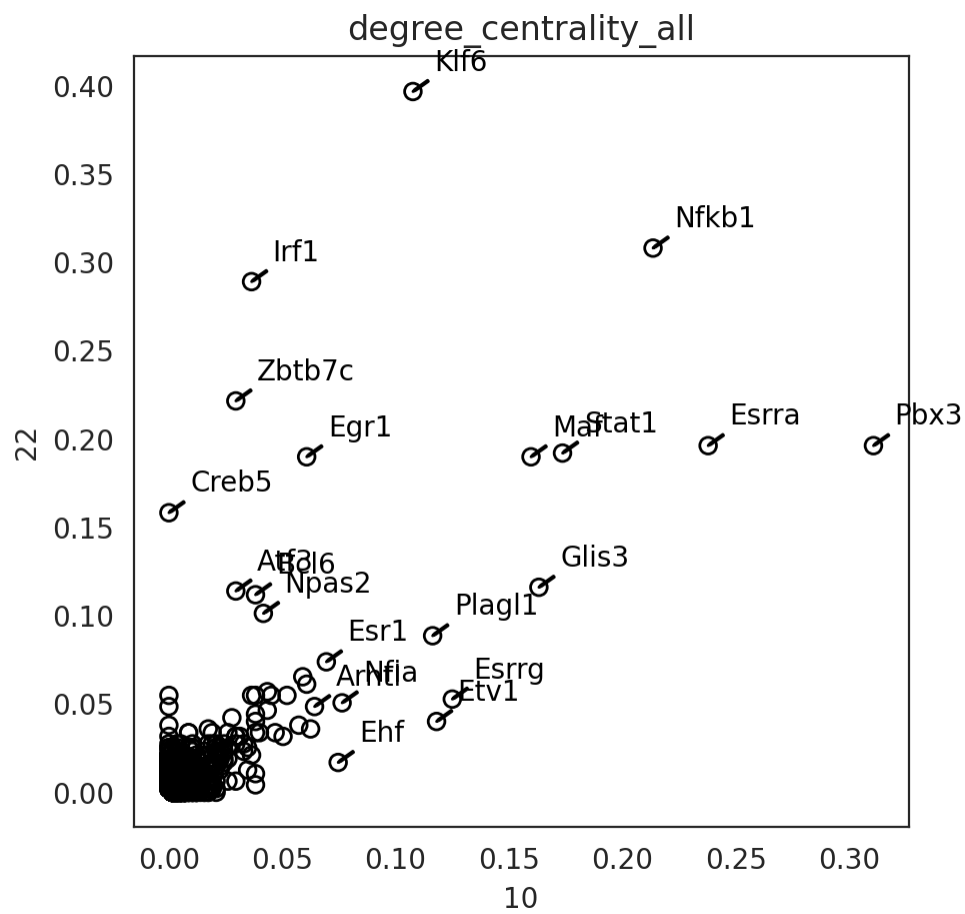


Processing clusters 10 and 20

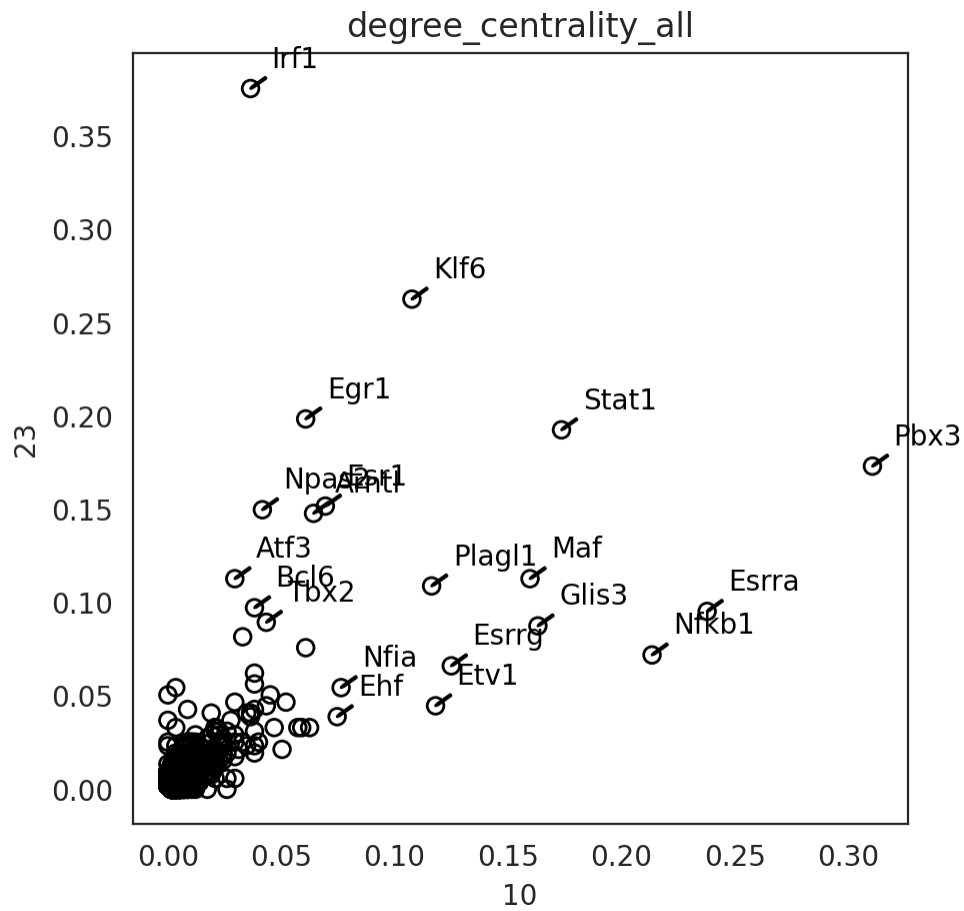


Processing clusters 10 and 21

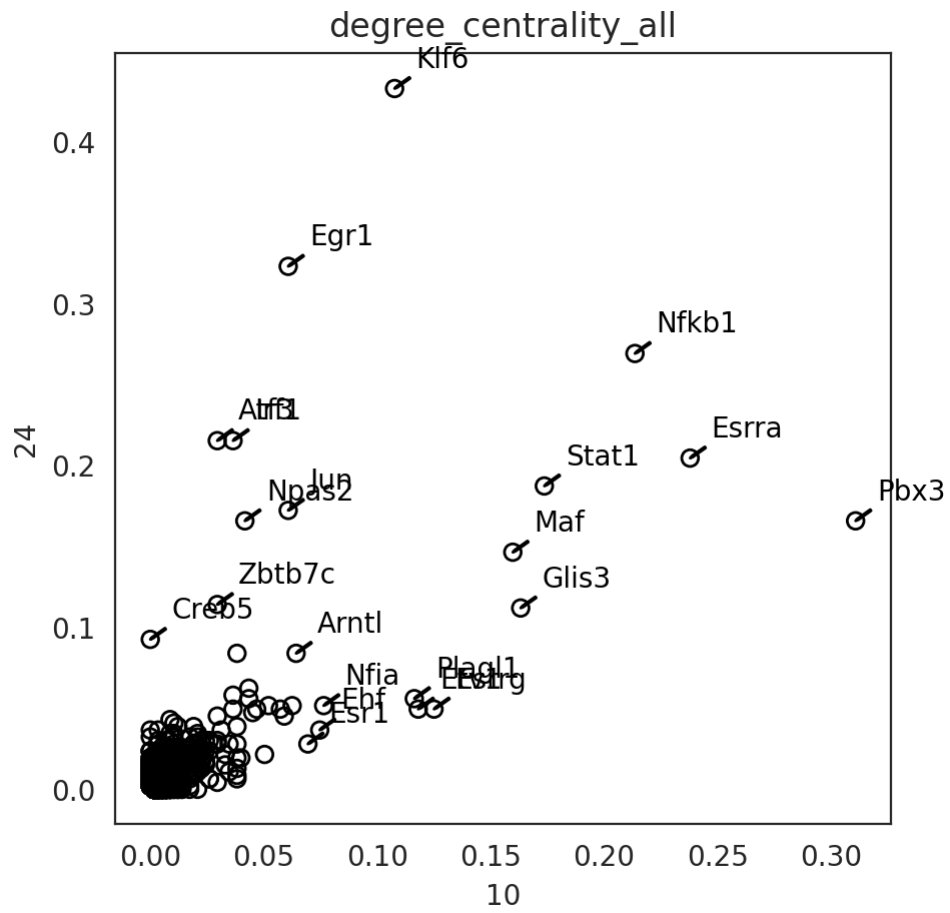




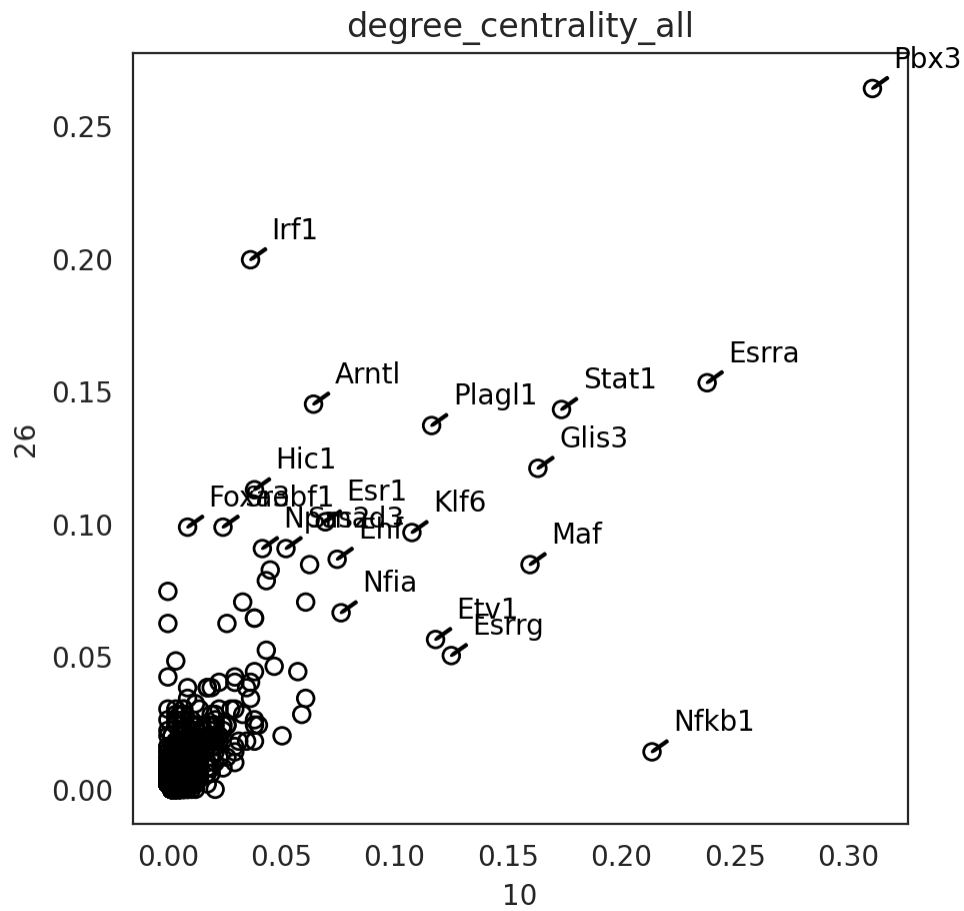
Processing clusters 10 and 23



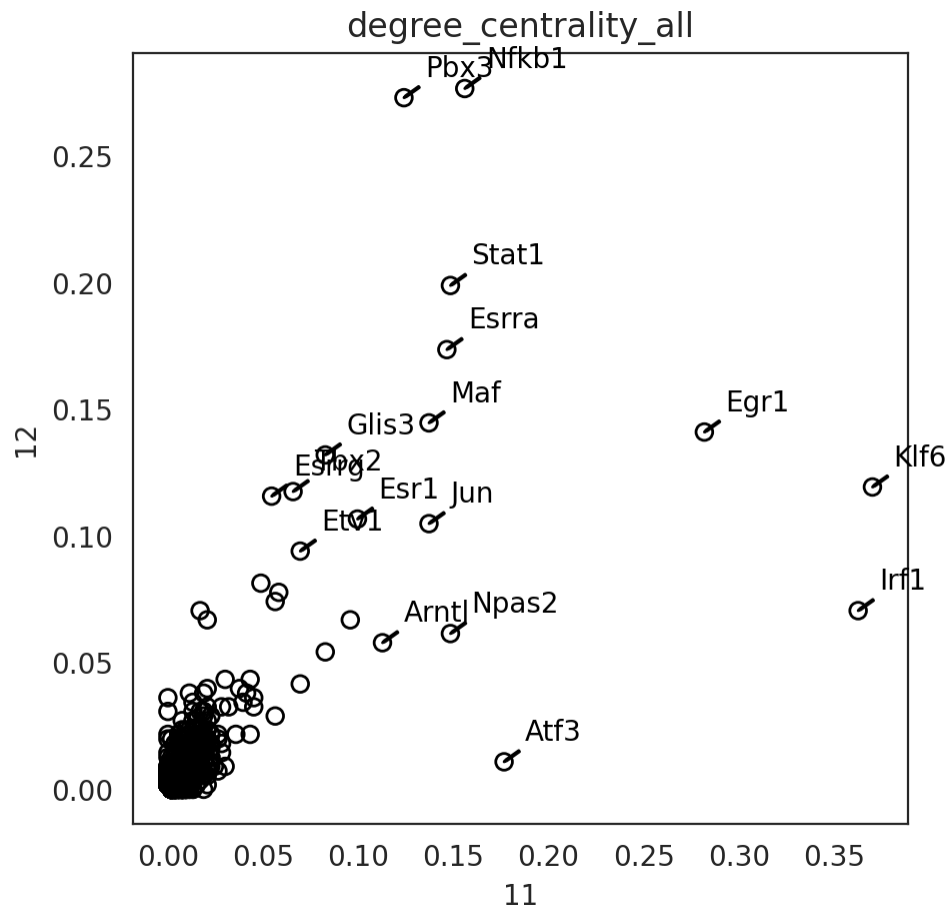
Processing clusters 10 and 24



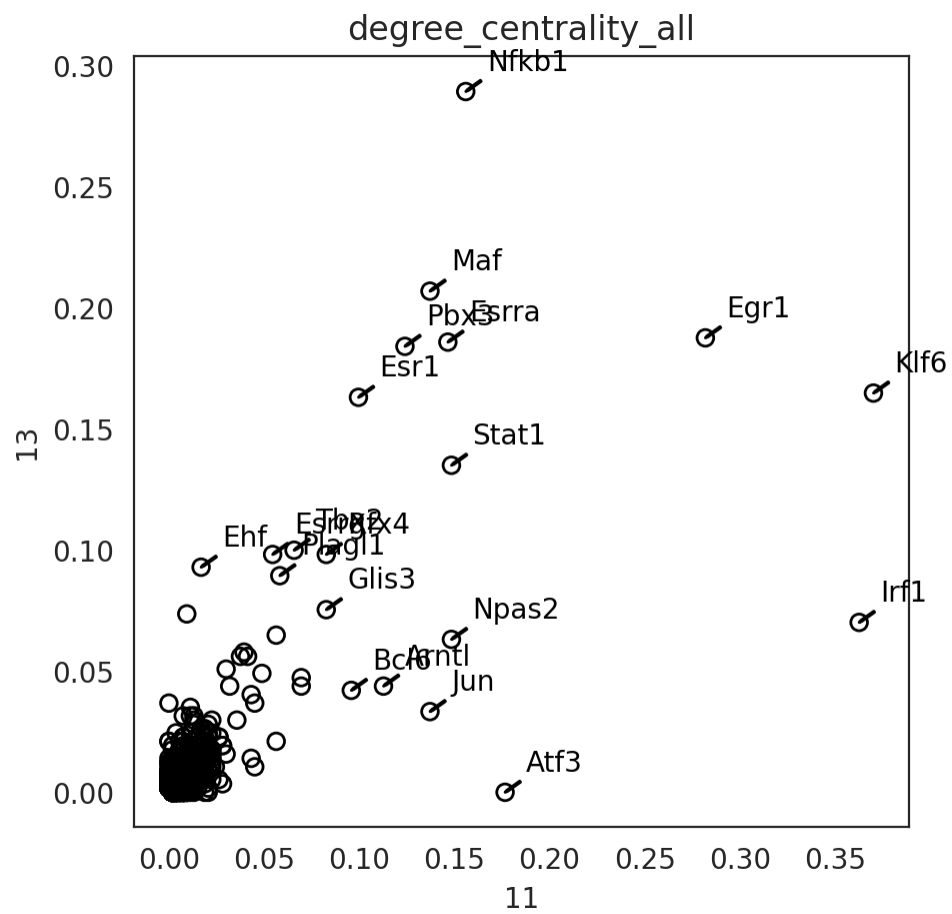
Processing clusters 10 and 25



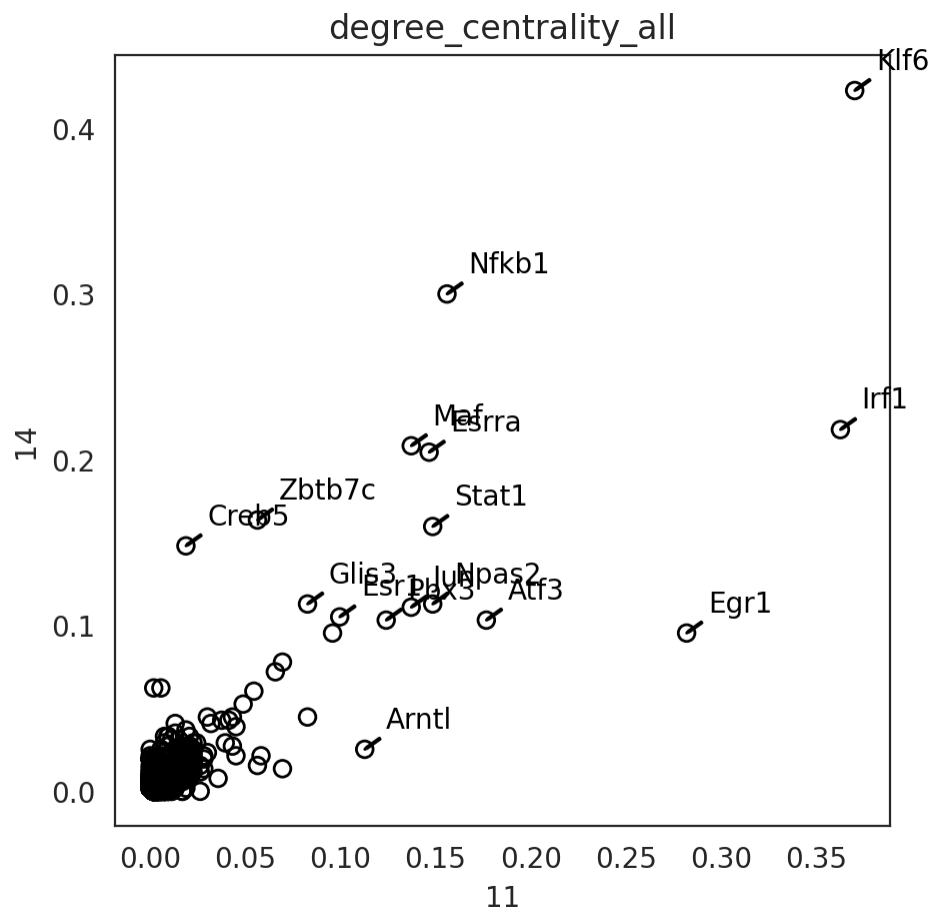
Processing clusters 11 and 12



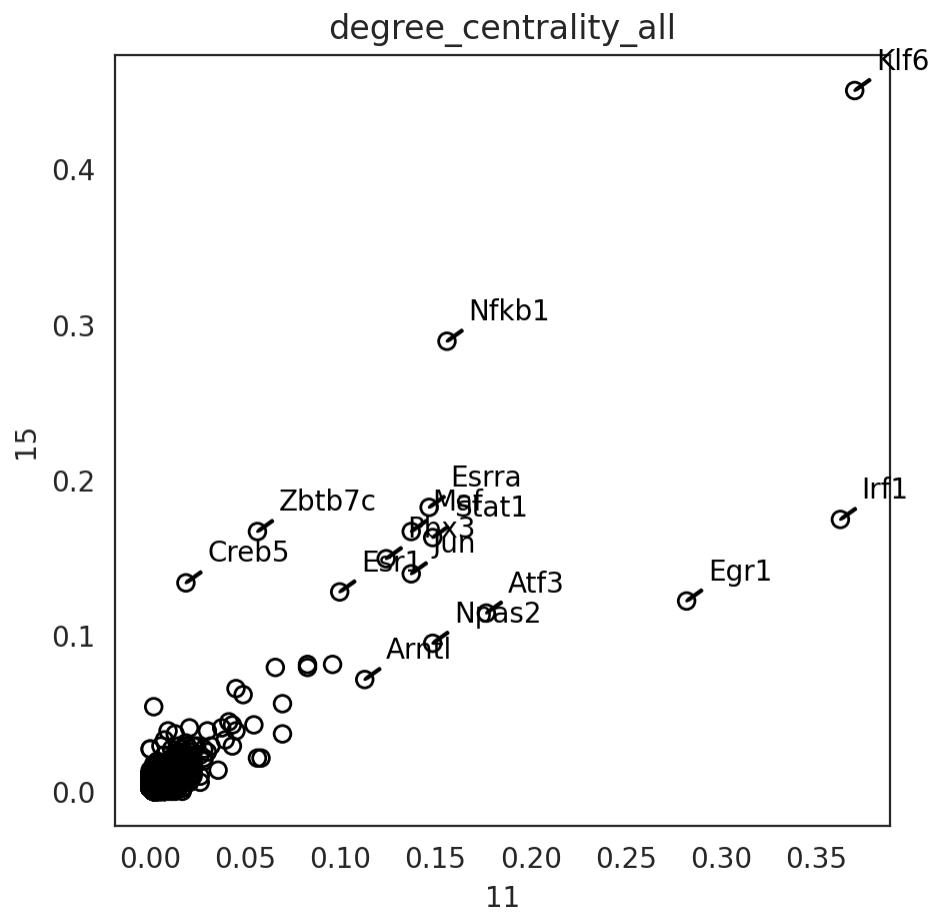
Processing clusters 11 and 13



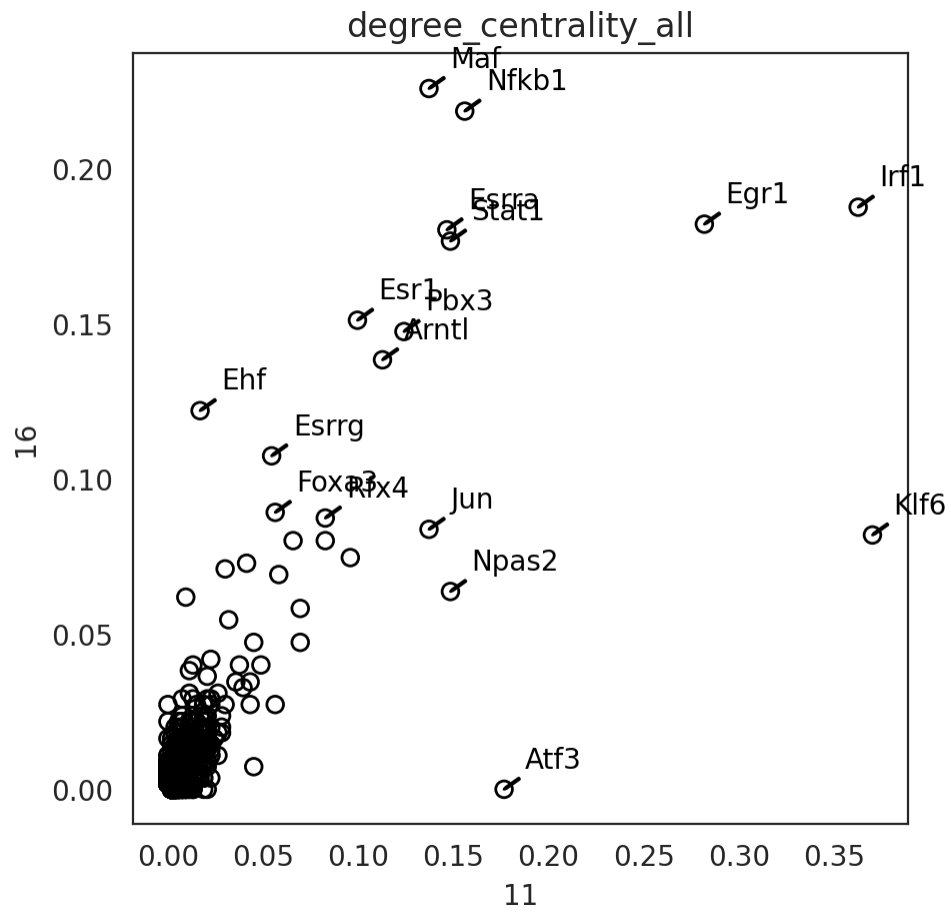
Processing clusters 11 and 14



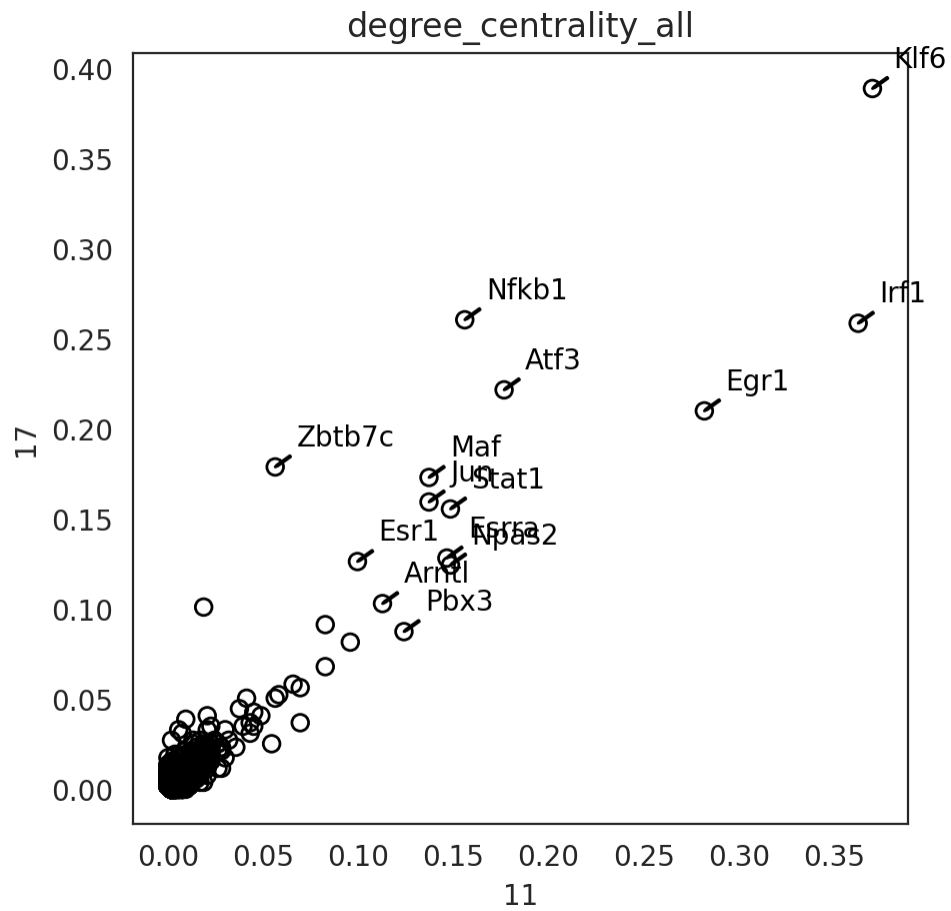
Processing clusters 11 and 15



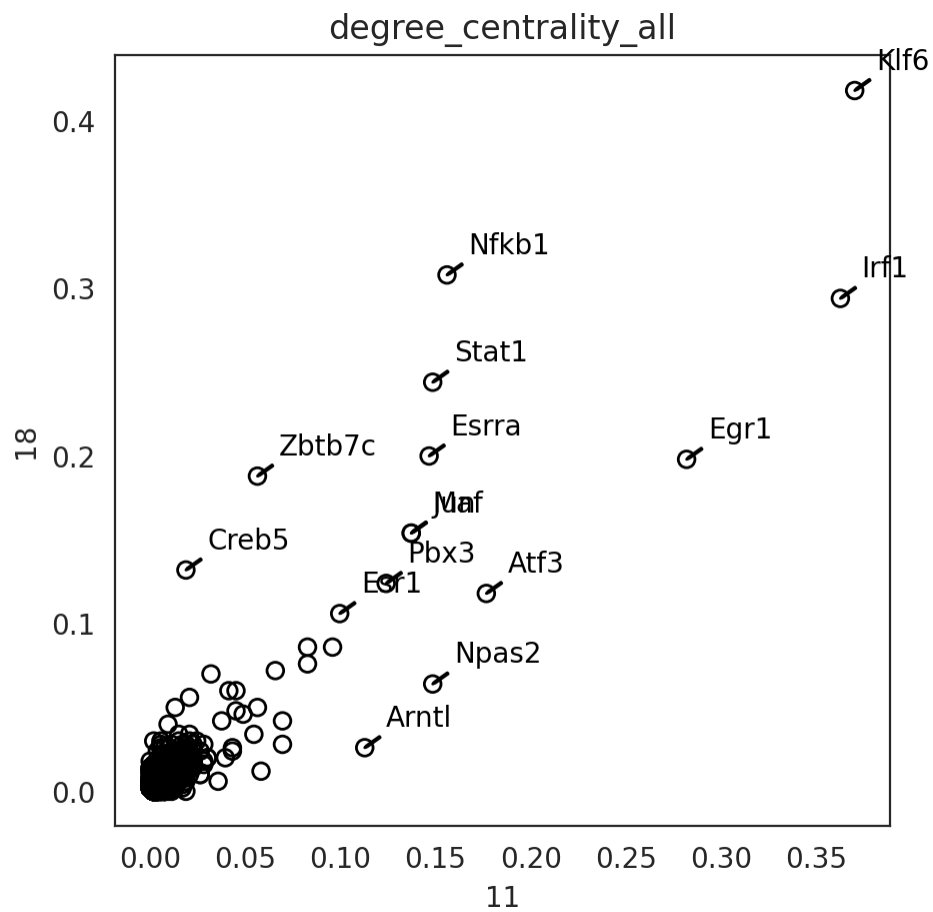
Processing clusters 11 and 16



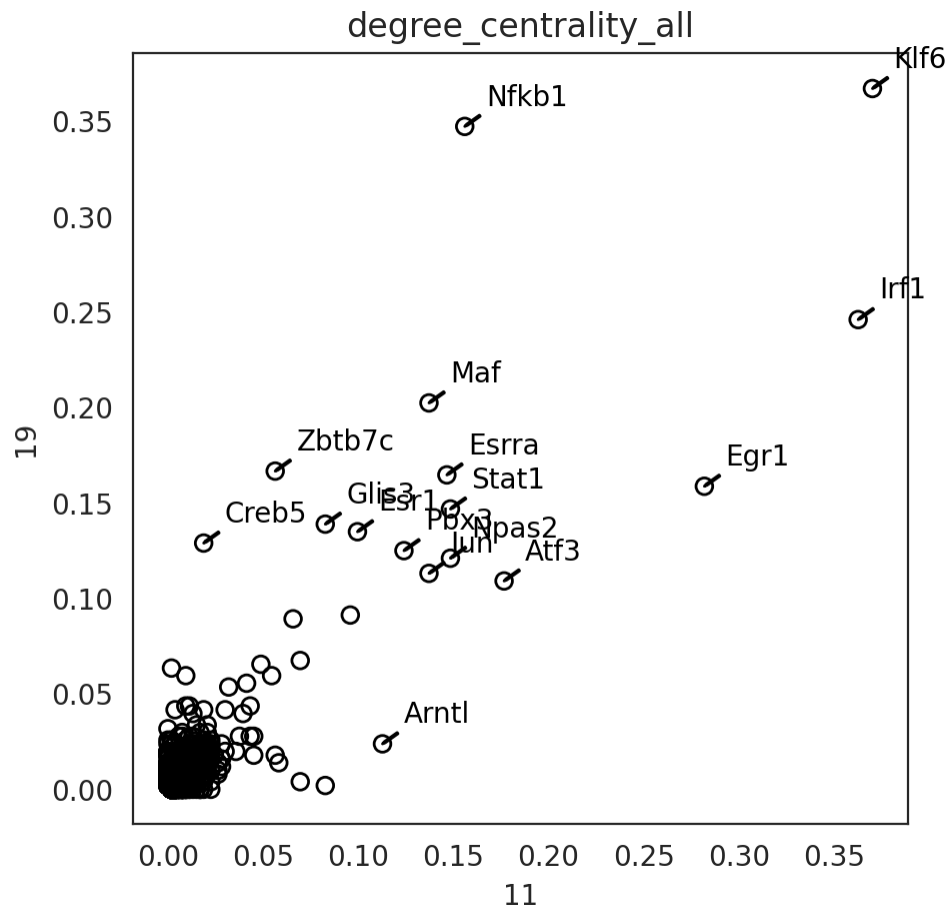
Processing clusters 11 and 17



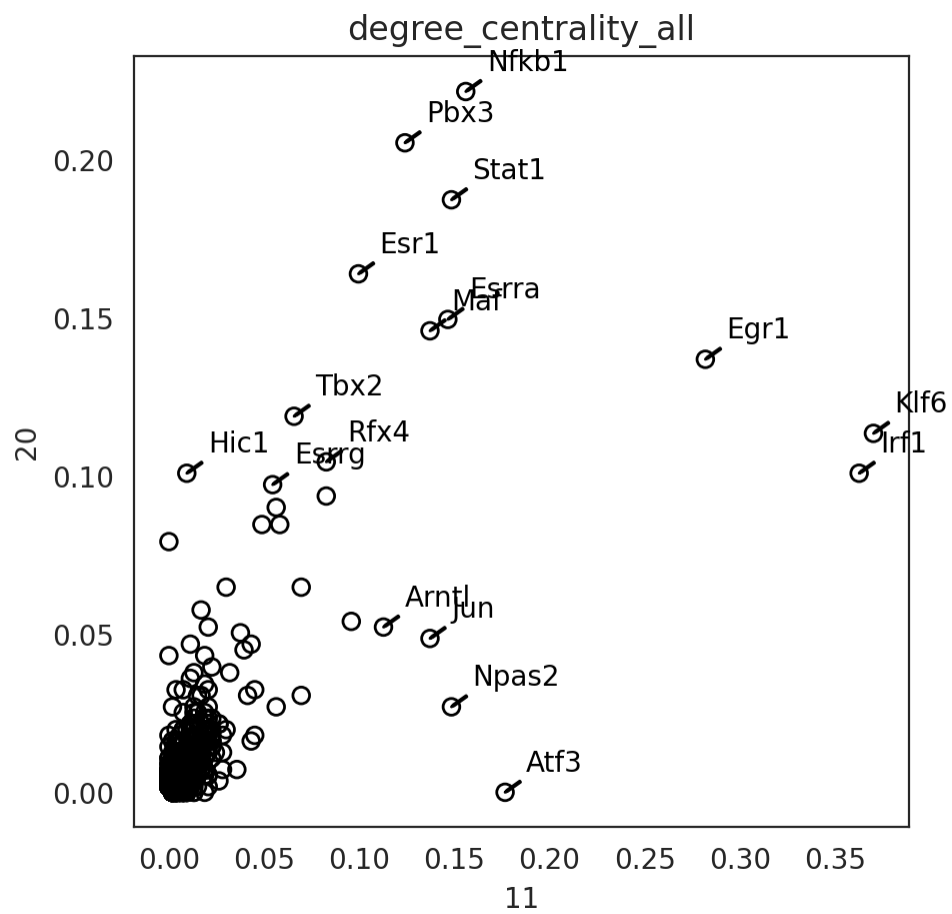
Processing clusters 11 and 18



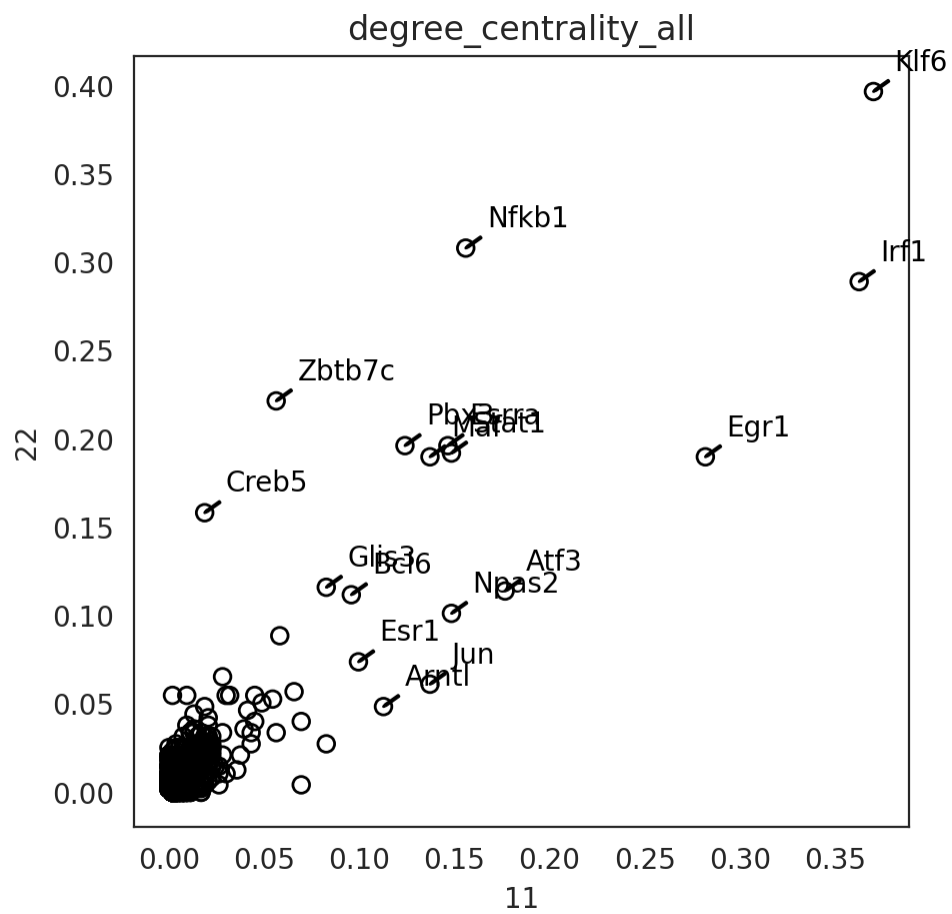
Processing clusters 11 and 19



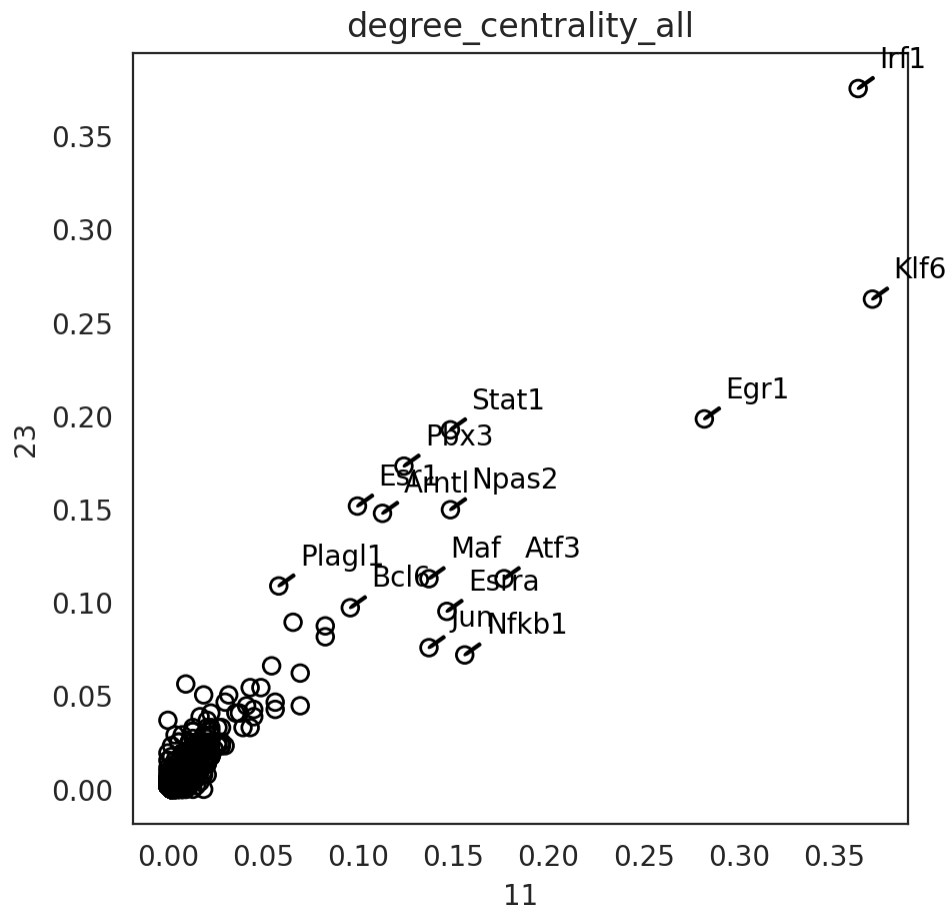
Processing clusters 11 and 20

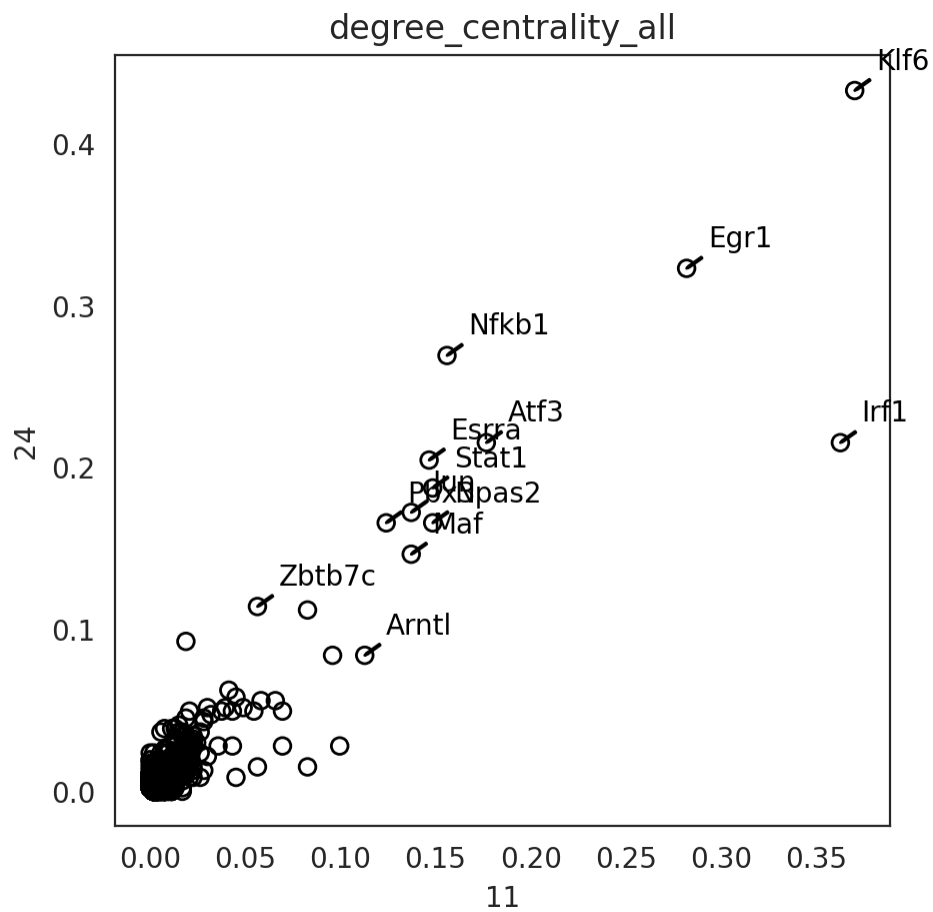


Processing clusters 11 and 21

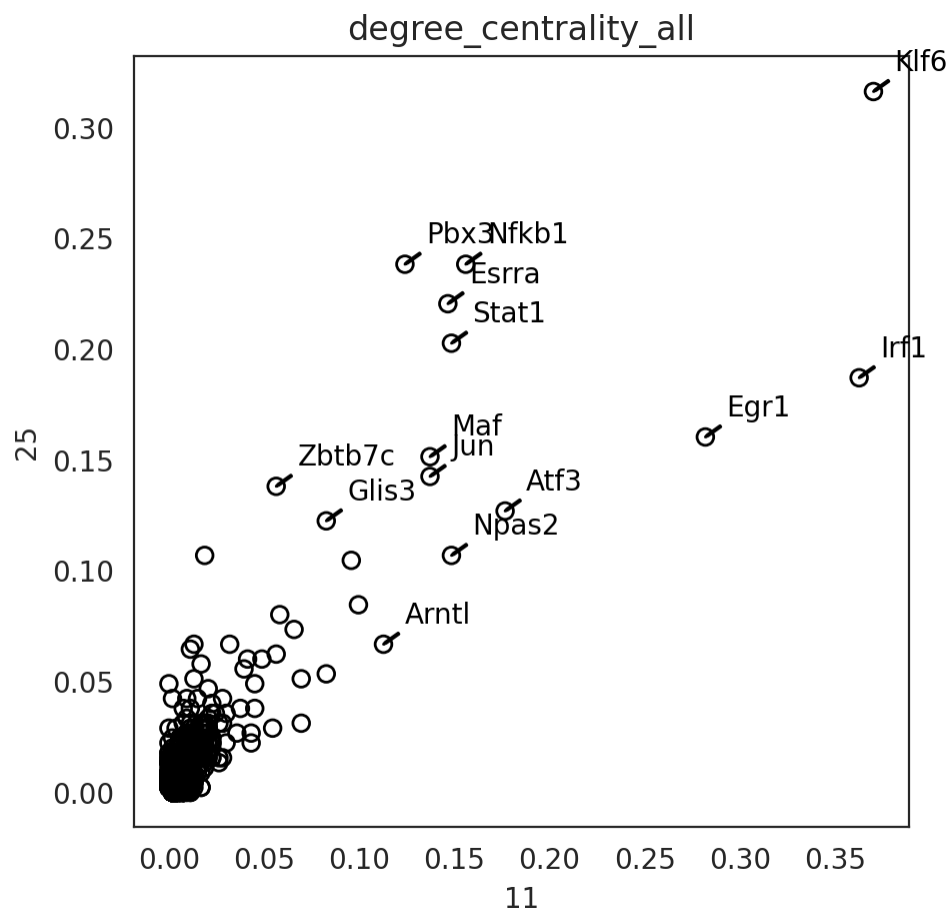


Processing clusters 11 and 23

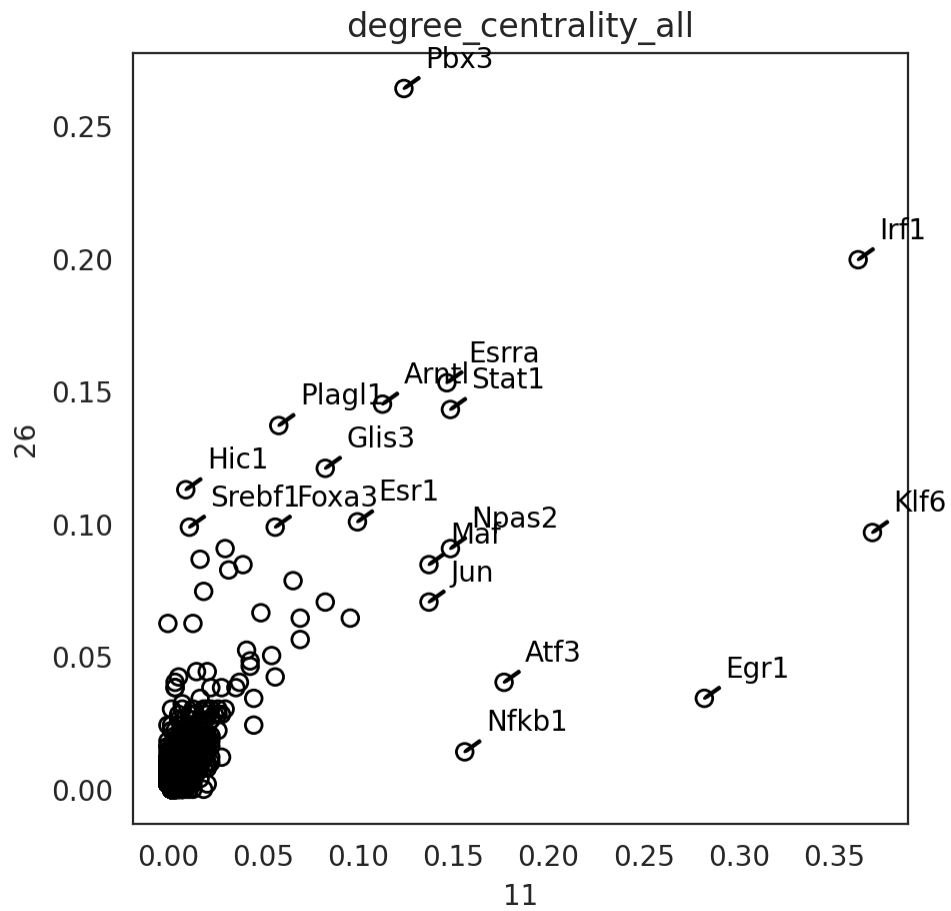




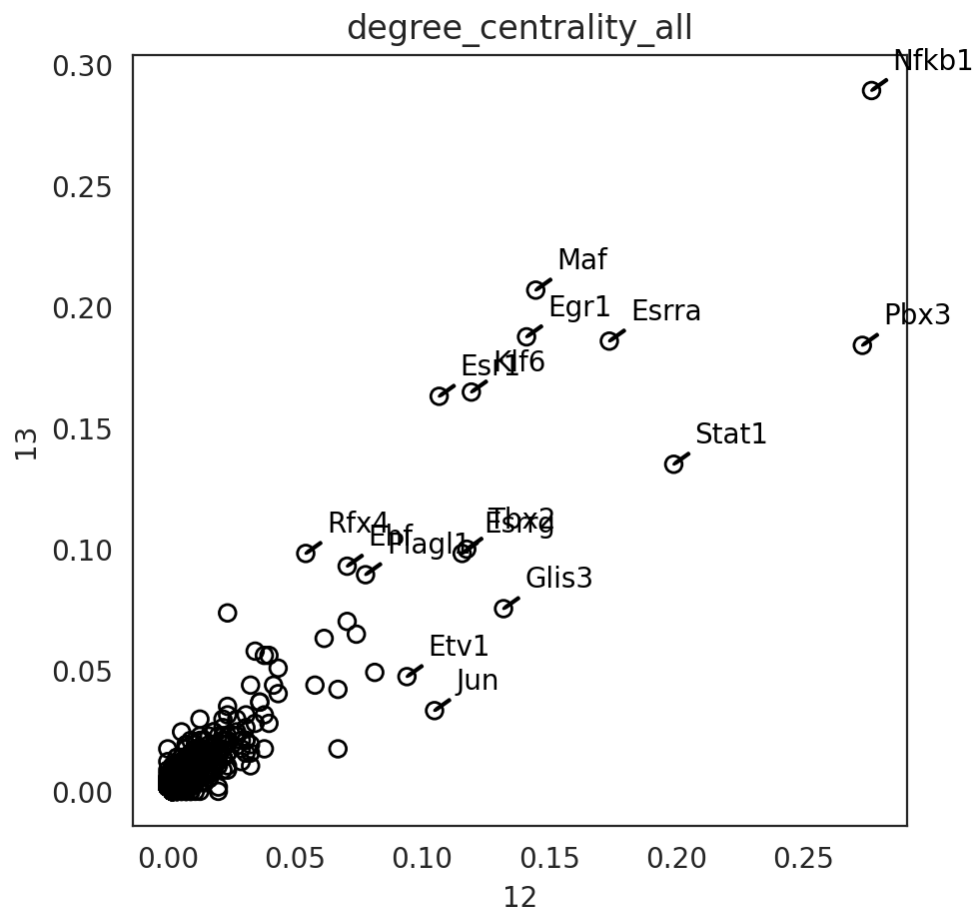
Processing clusters 11 and 25



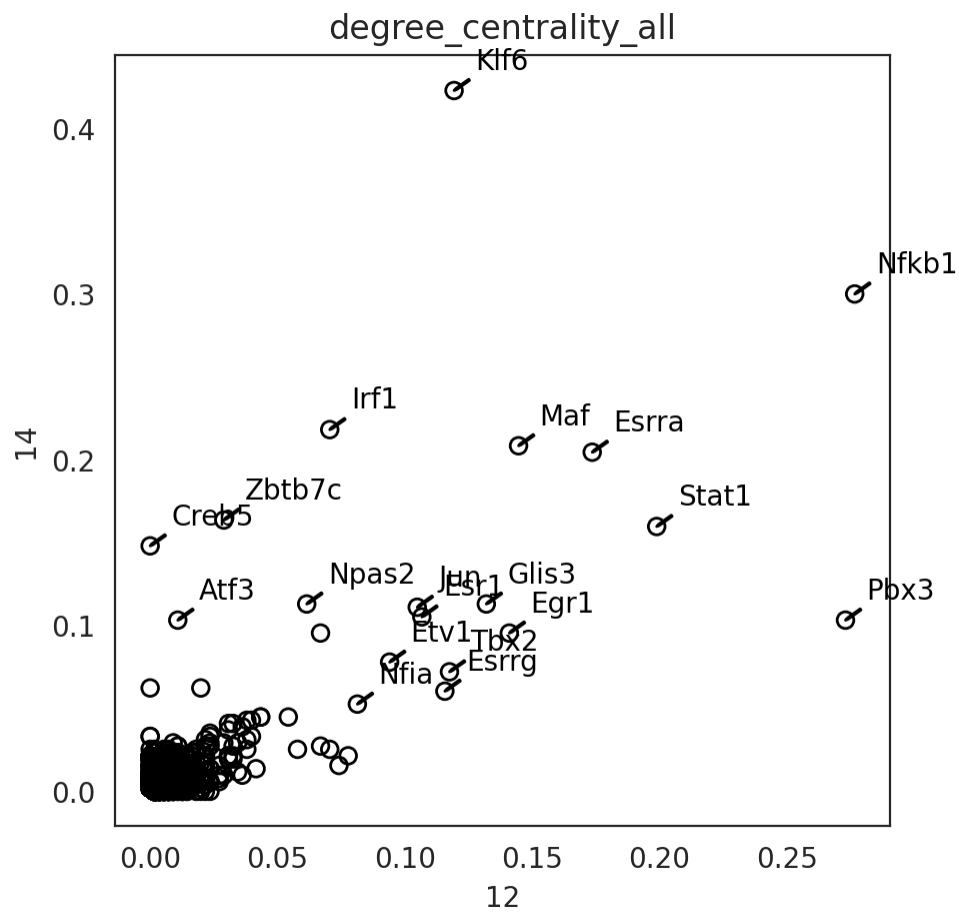
Processing clusters 11 and 26



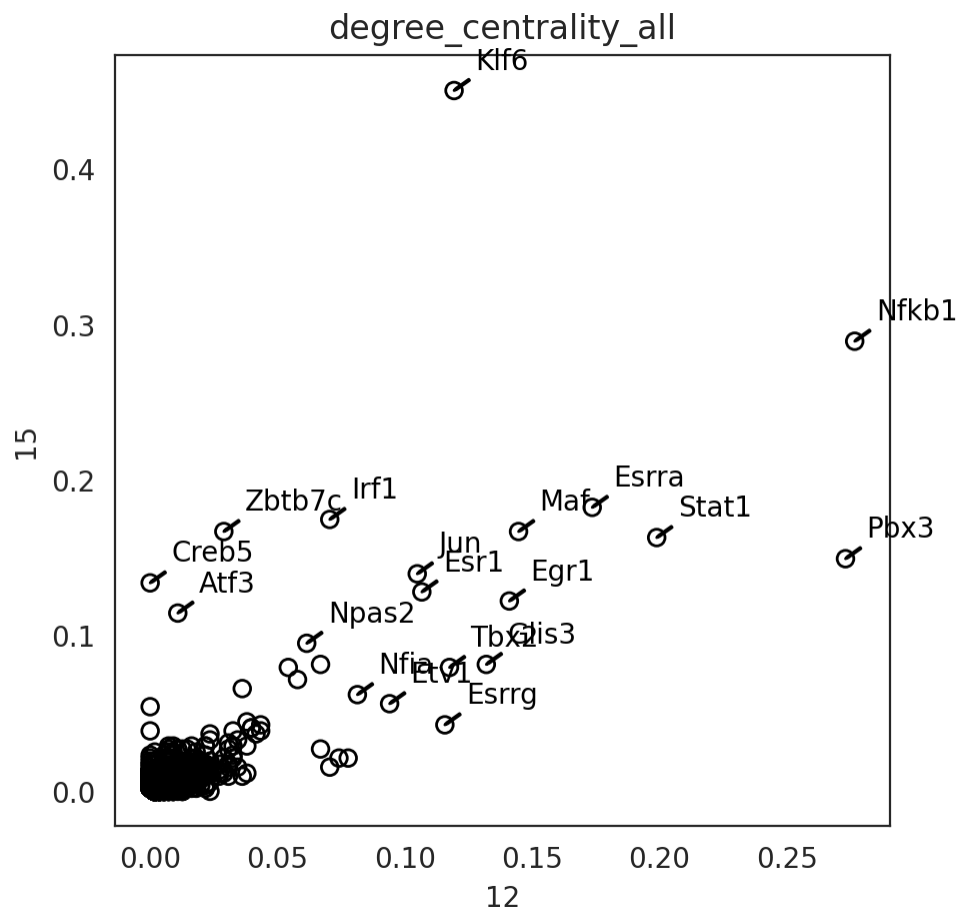
Processing clusters 12 and 13



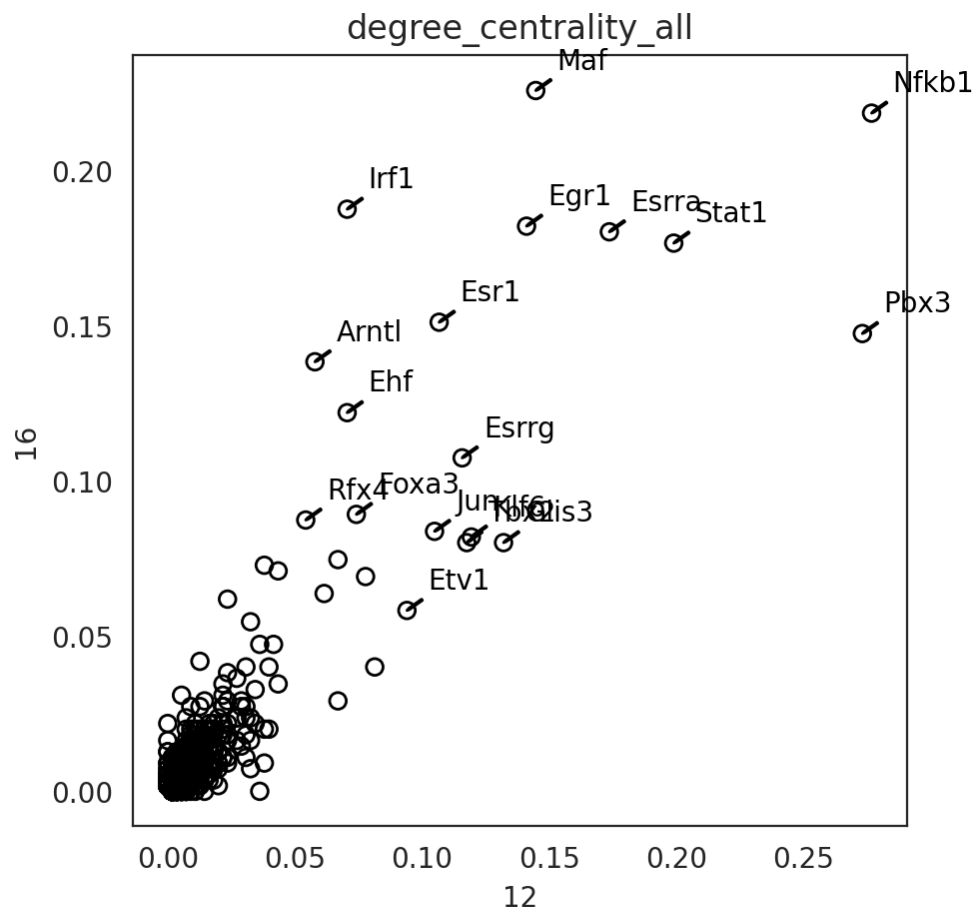
Processing clusters 12 and 14



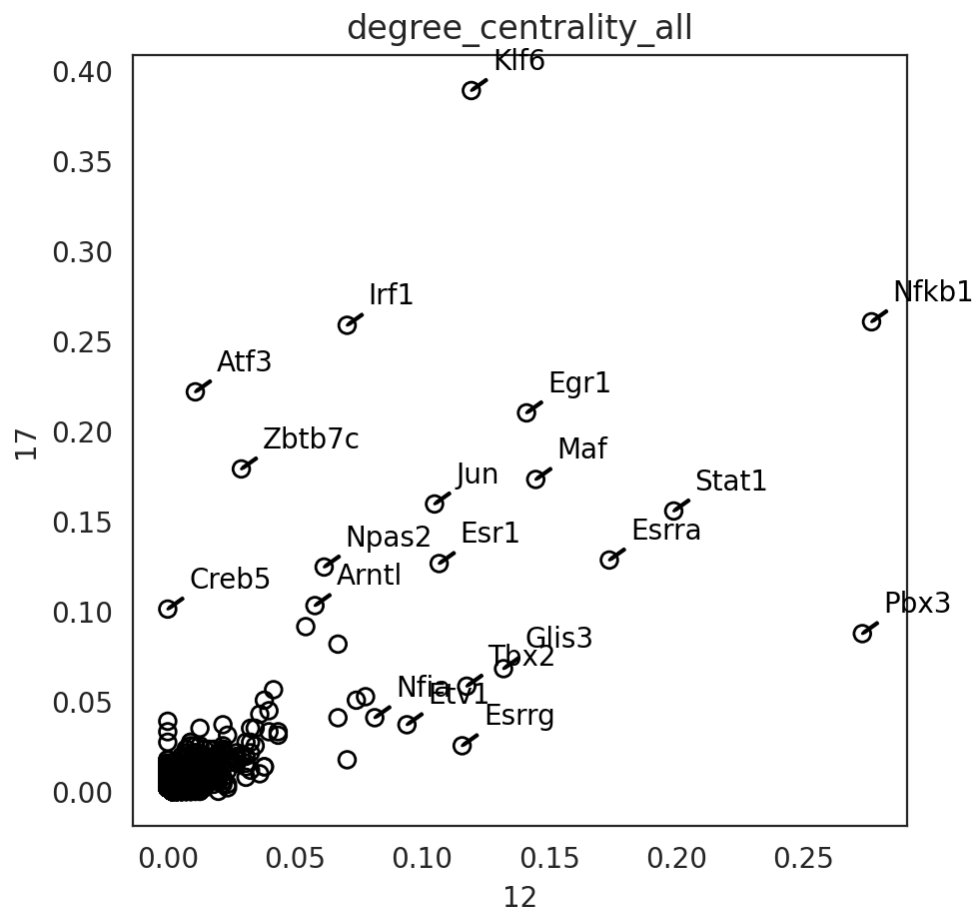
Processing clusters 12 and 15



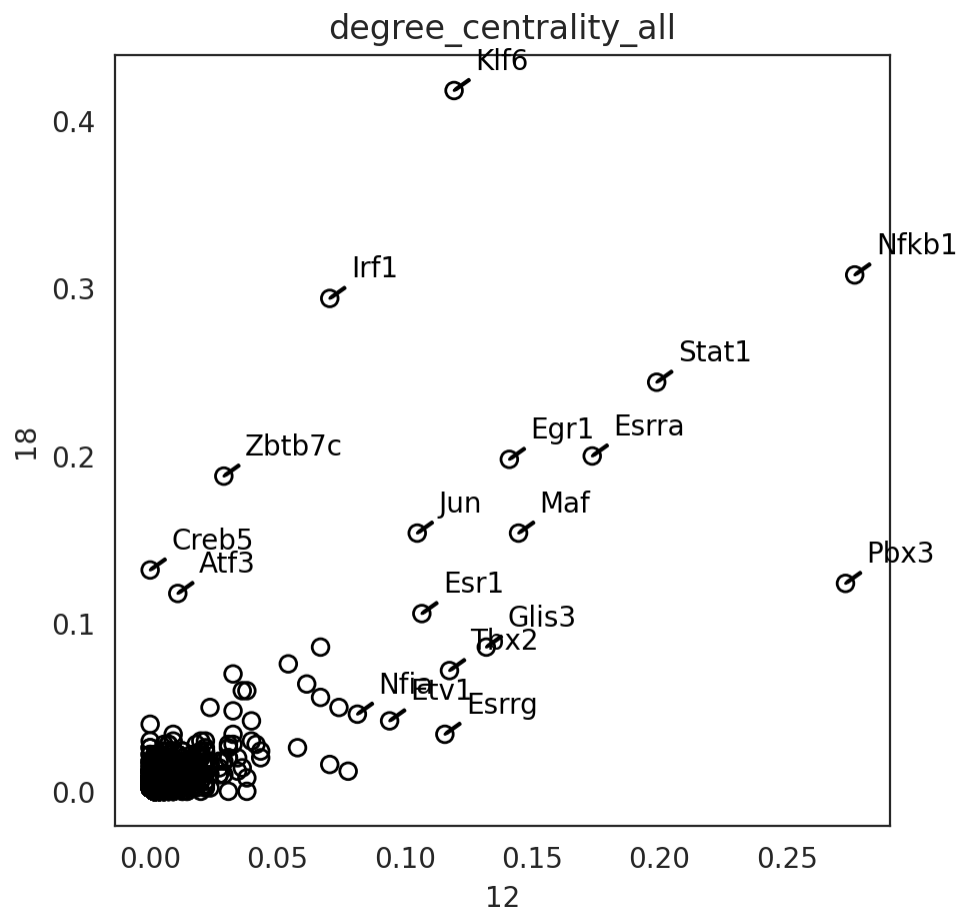
Processing clusters 12 and 16



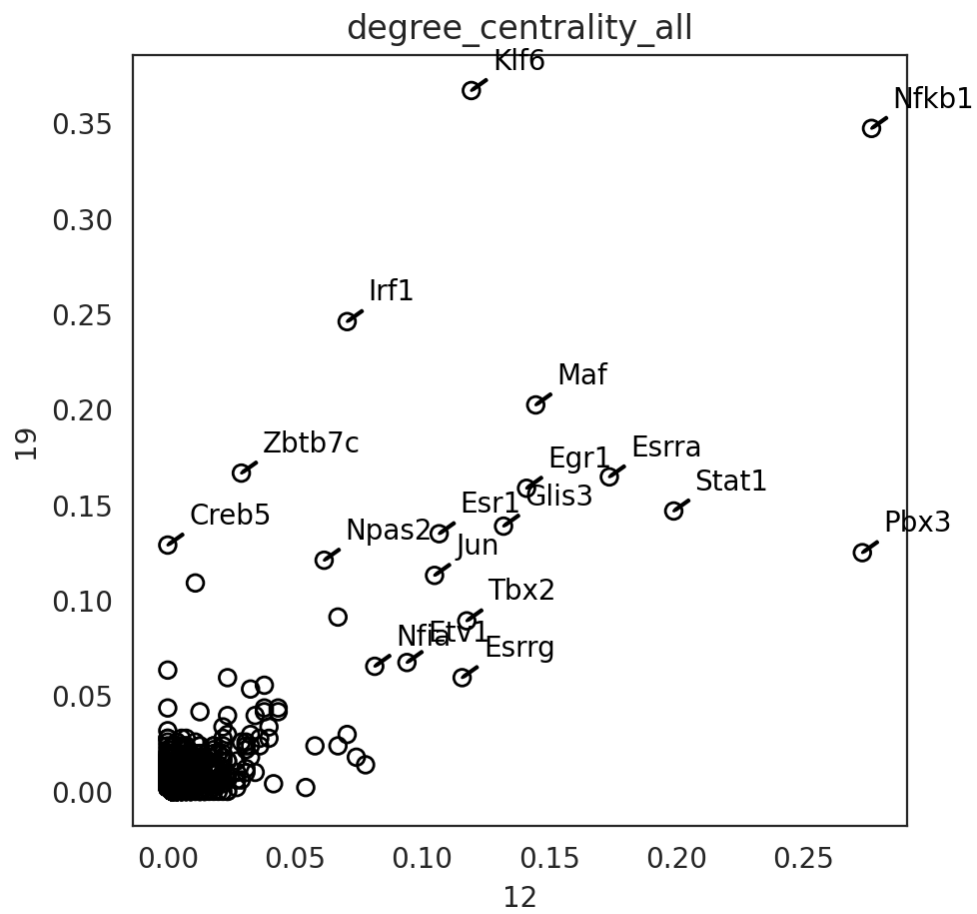
Processing clusters 12 and 17



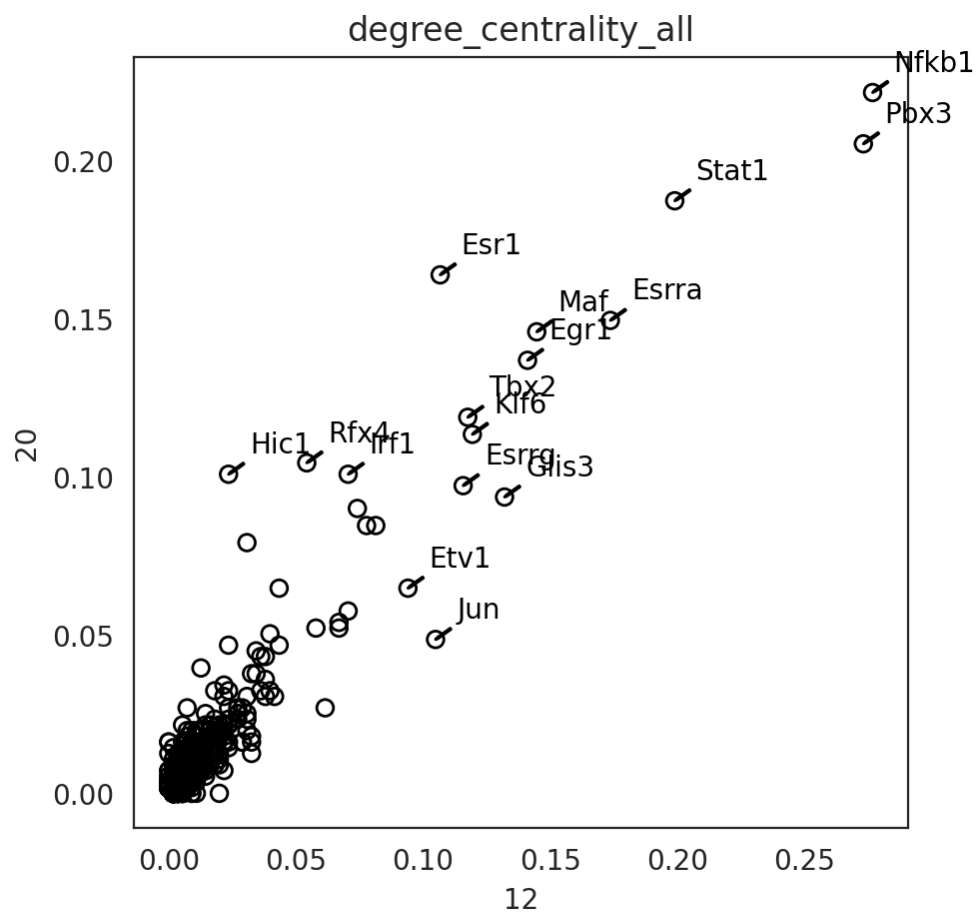
Processing clusters 12 and 18



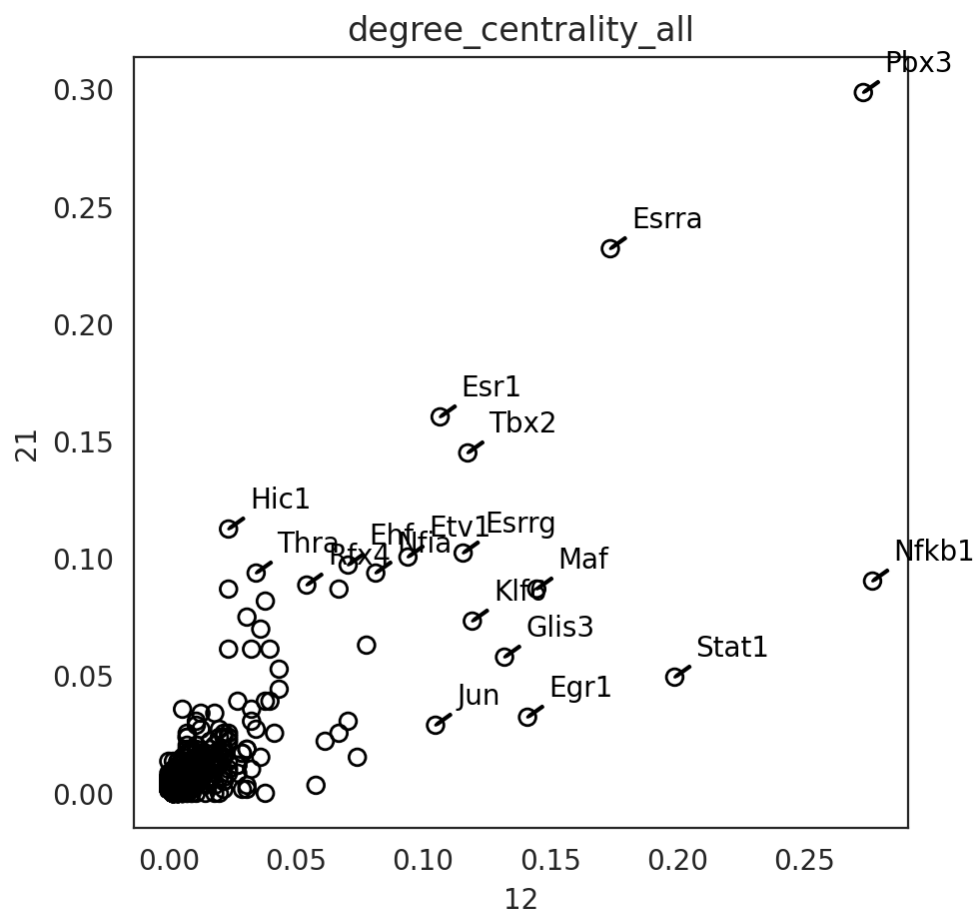
Processing clusters 12 and 19

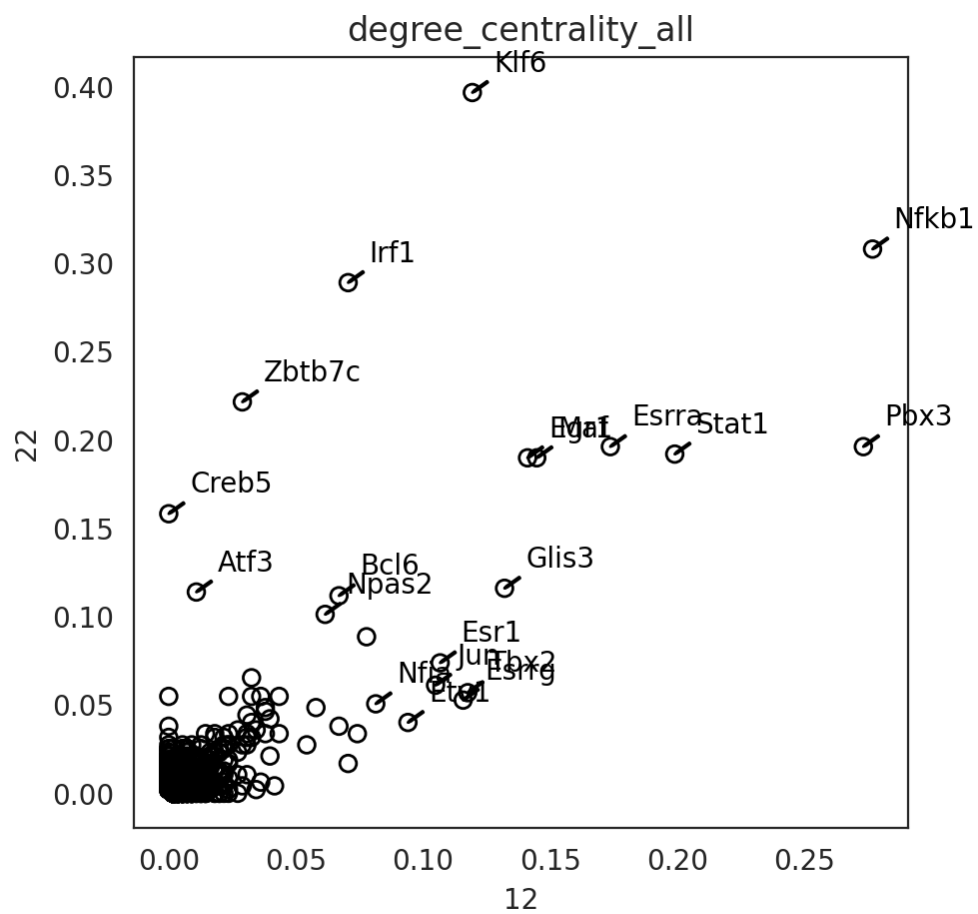


Processing clusters 12 and 20

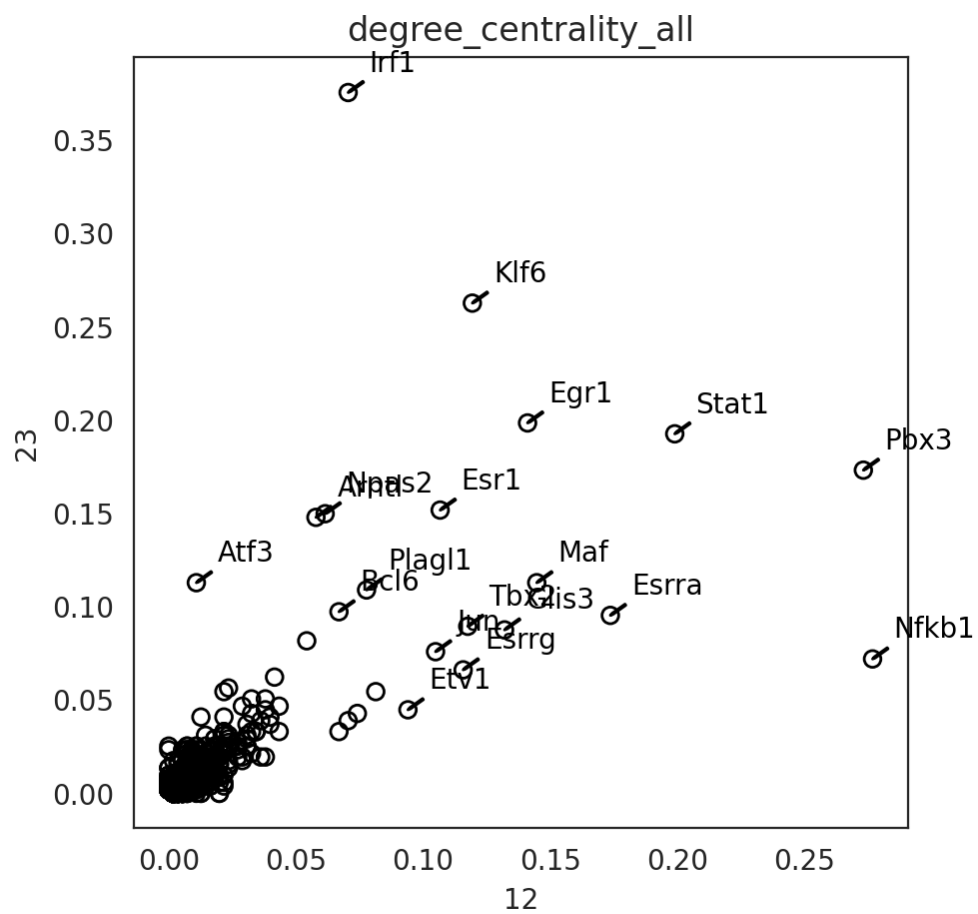


Processing clusters 12 and 21

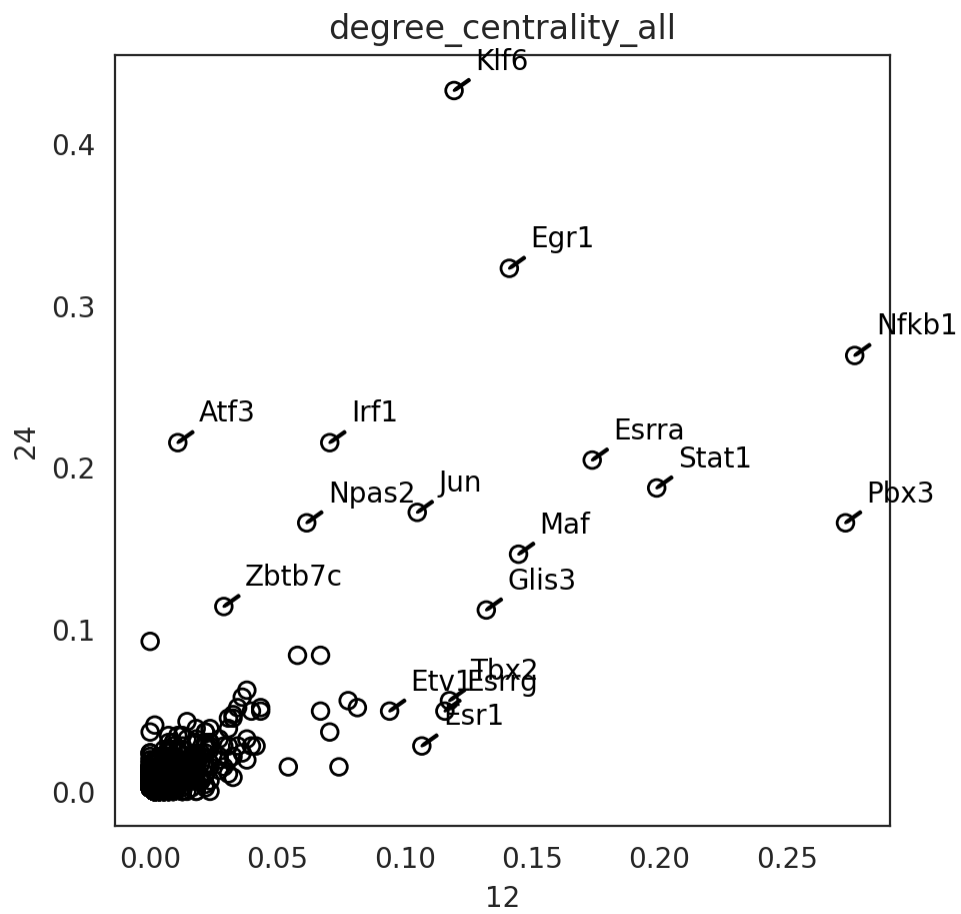




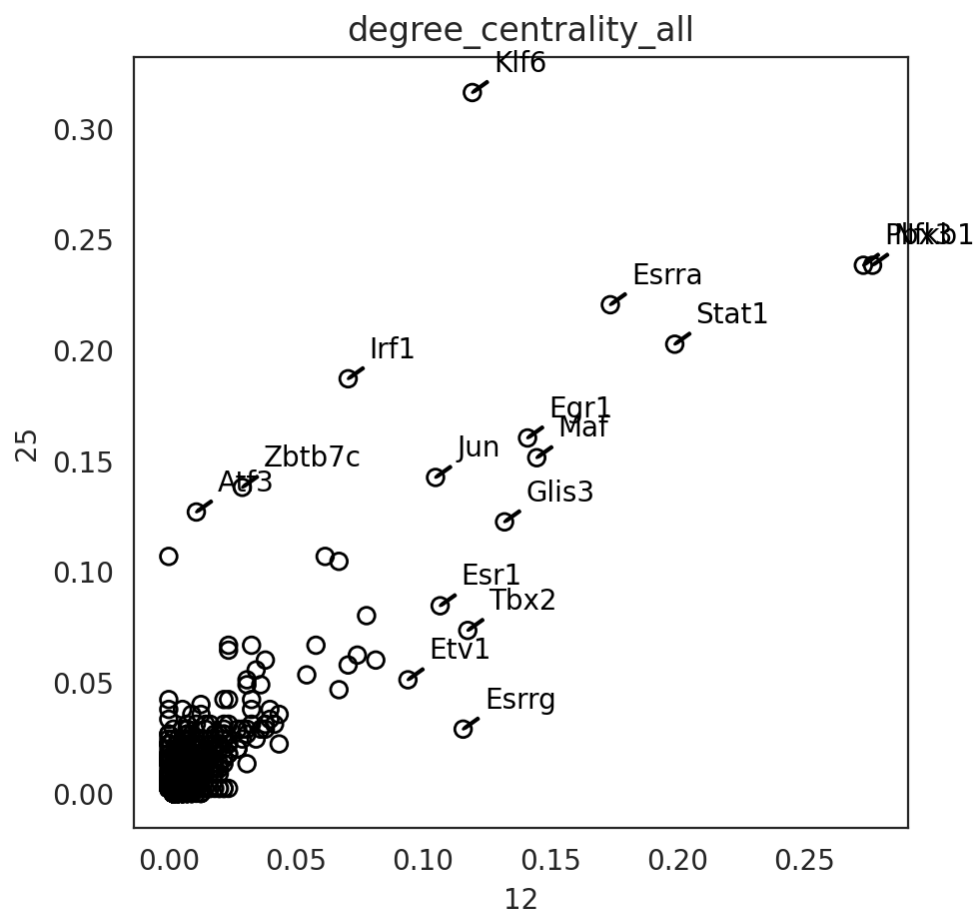
Processing clusters 12 and 23



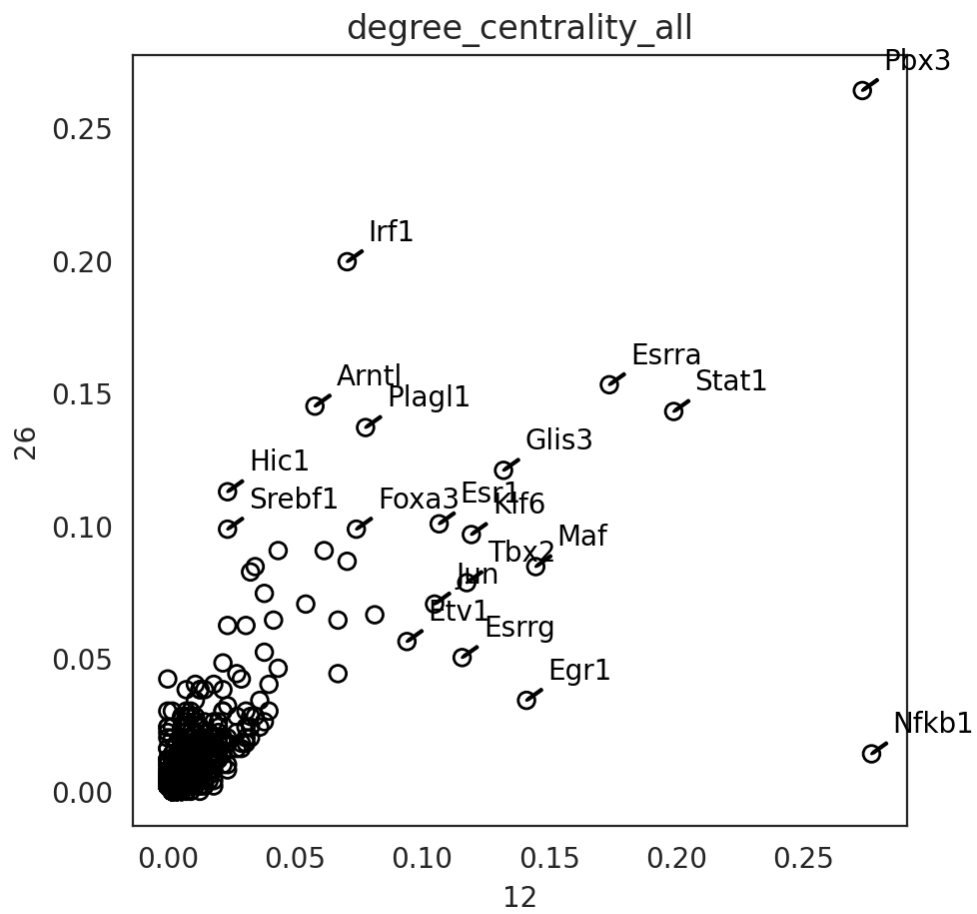
Processing clusters 12 and 24



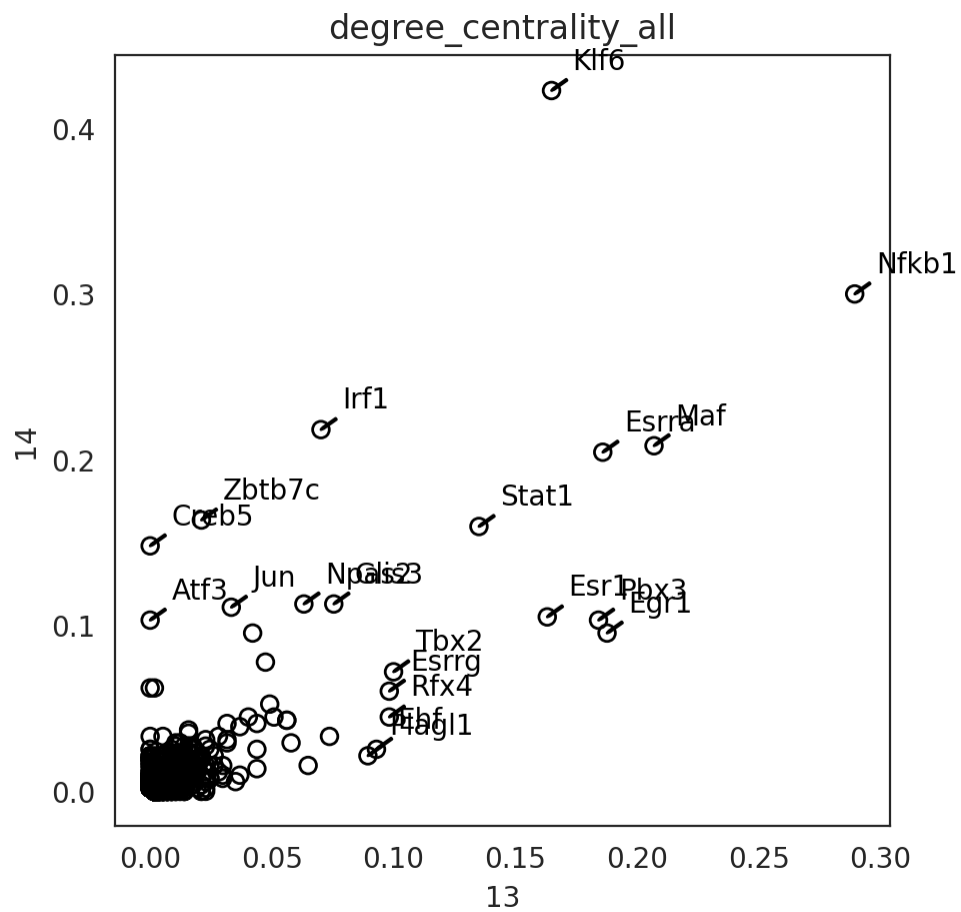
Processing clusters 12 and 25



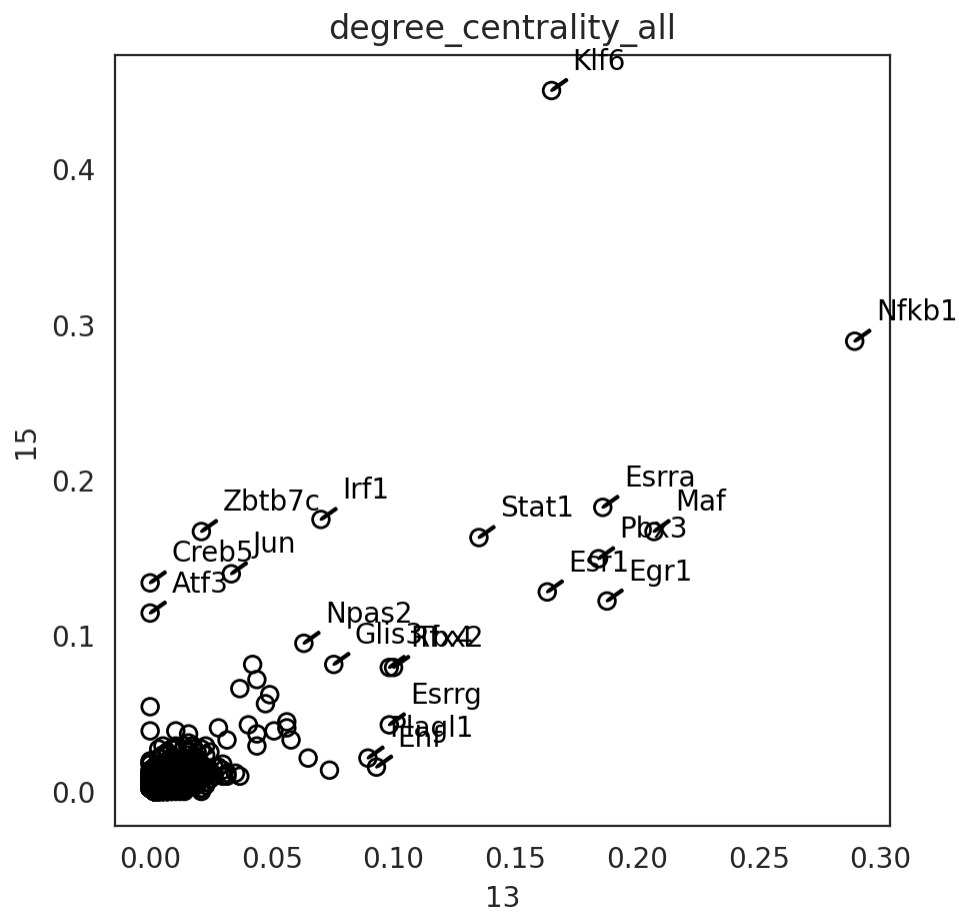
Processing clusters 12 and 26



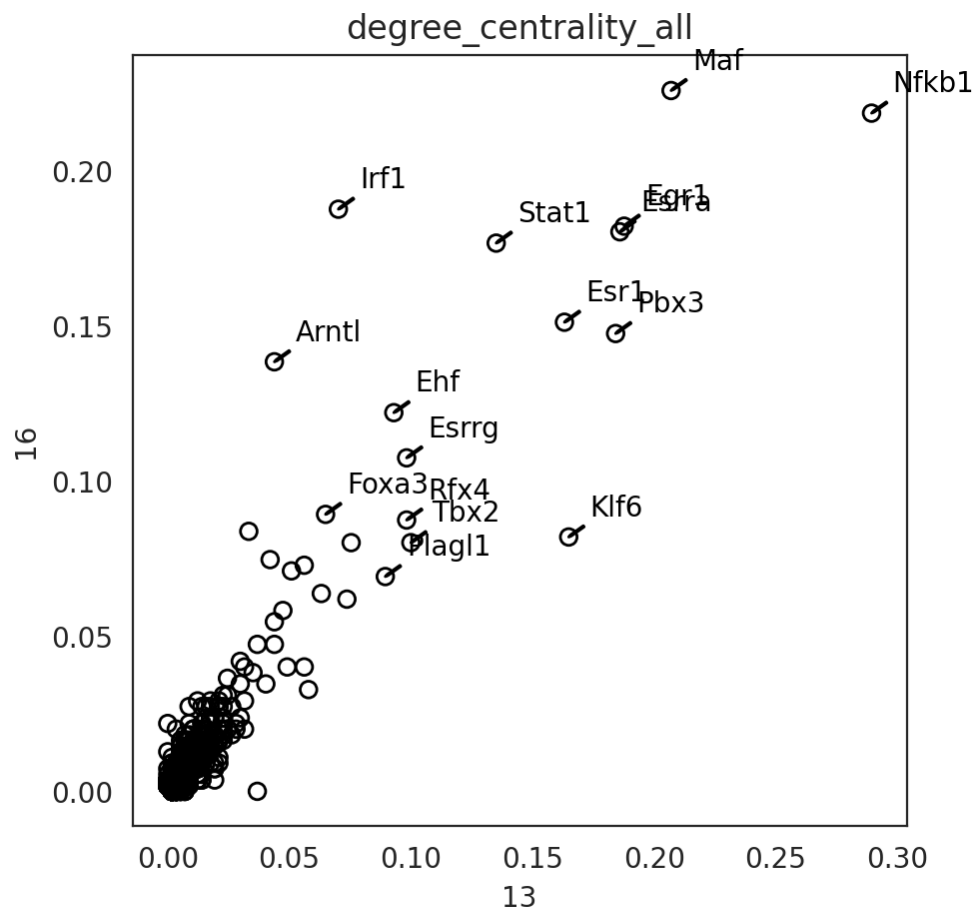
Processing clusters 13 and 14



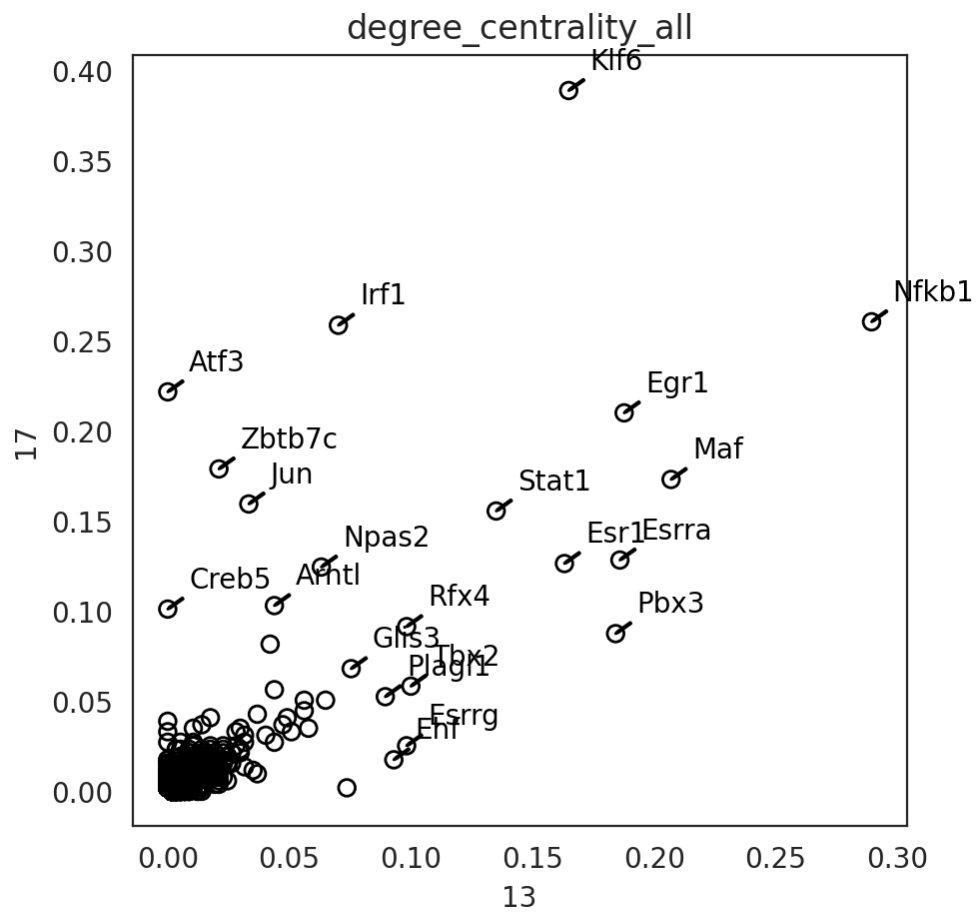
Processing clusters 13 and 15

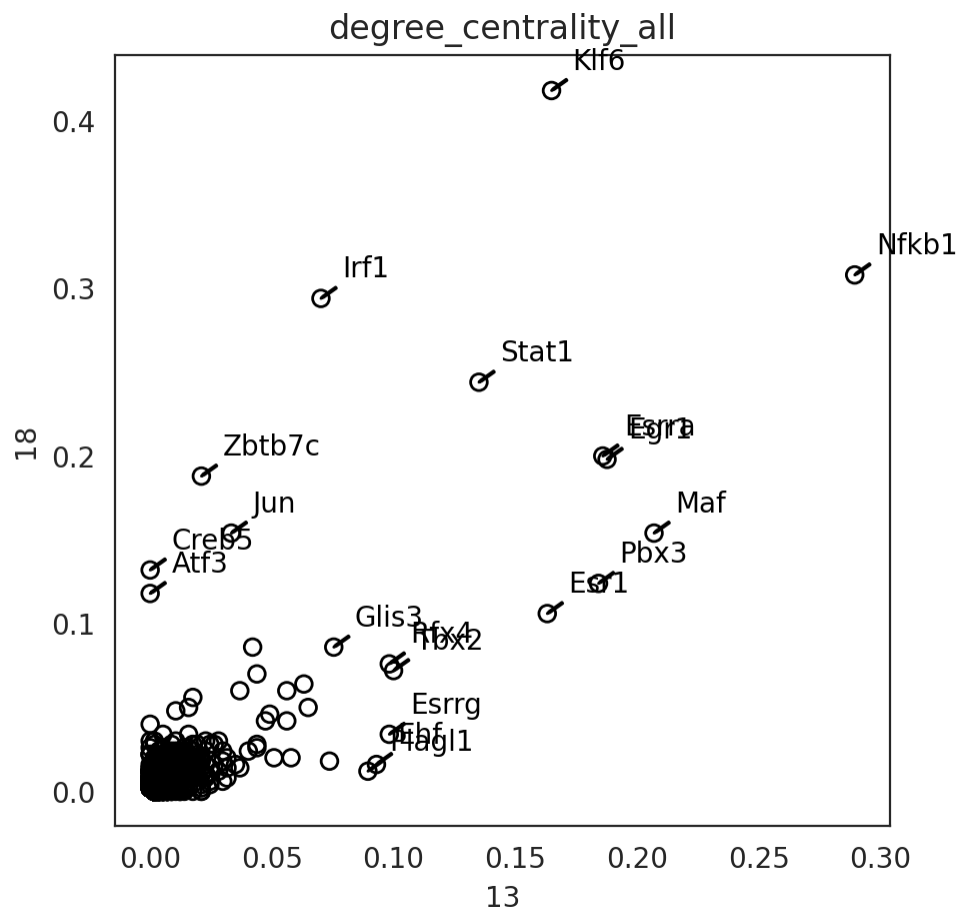


Processing clusters 13 and 16

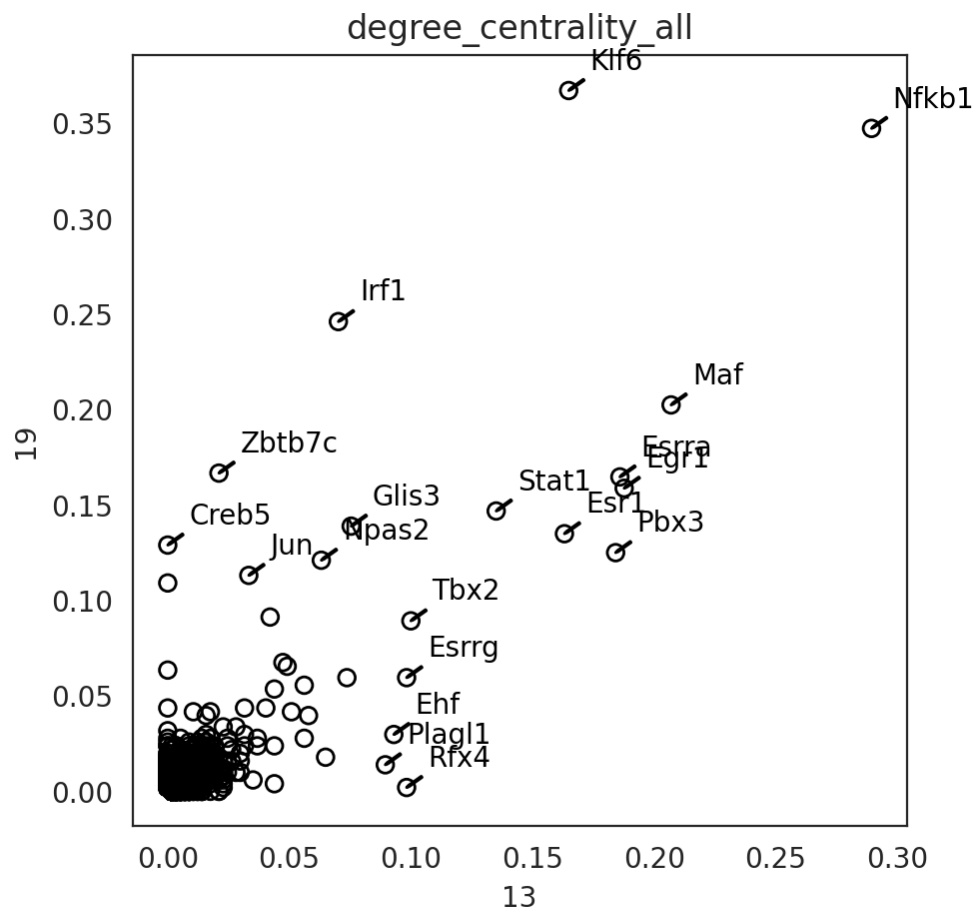


Processing clusters 13 and 17

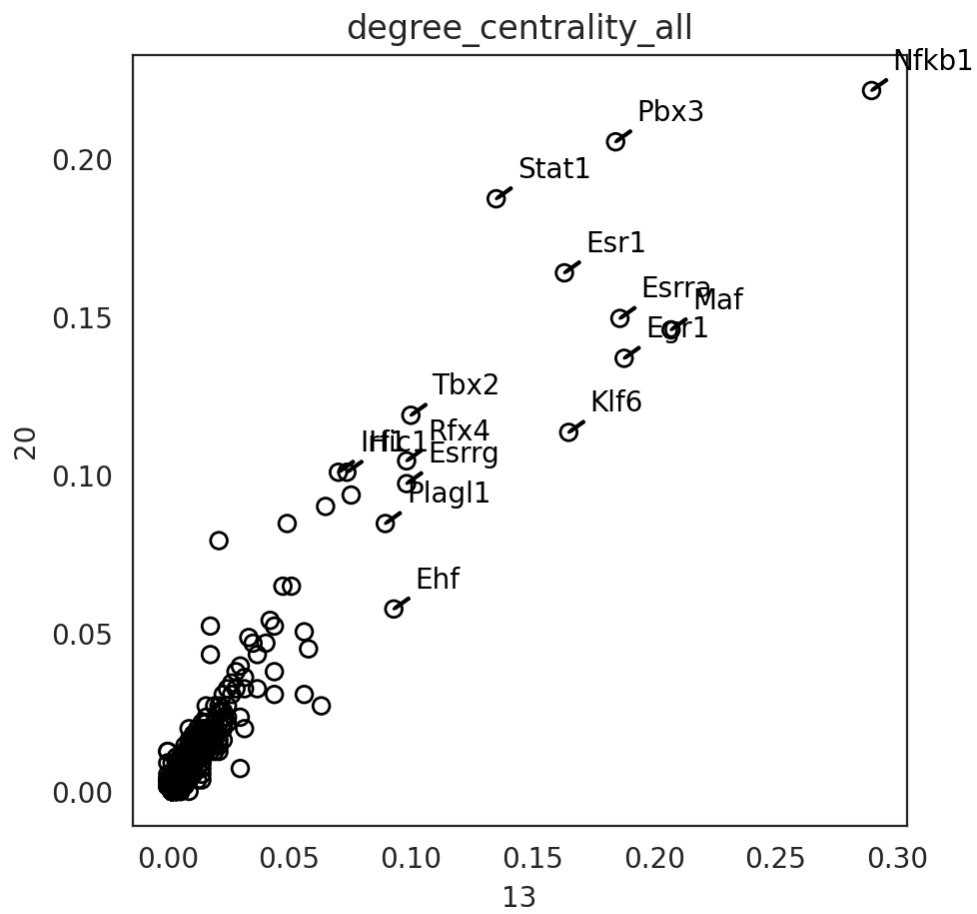




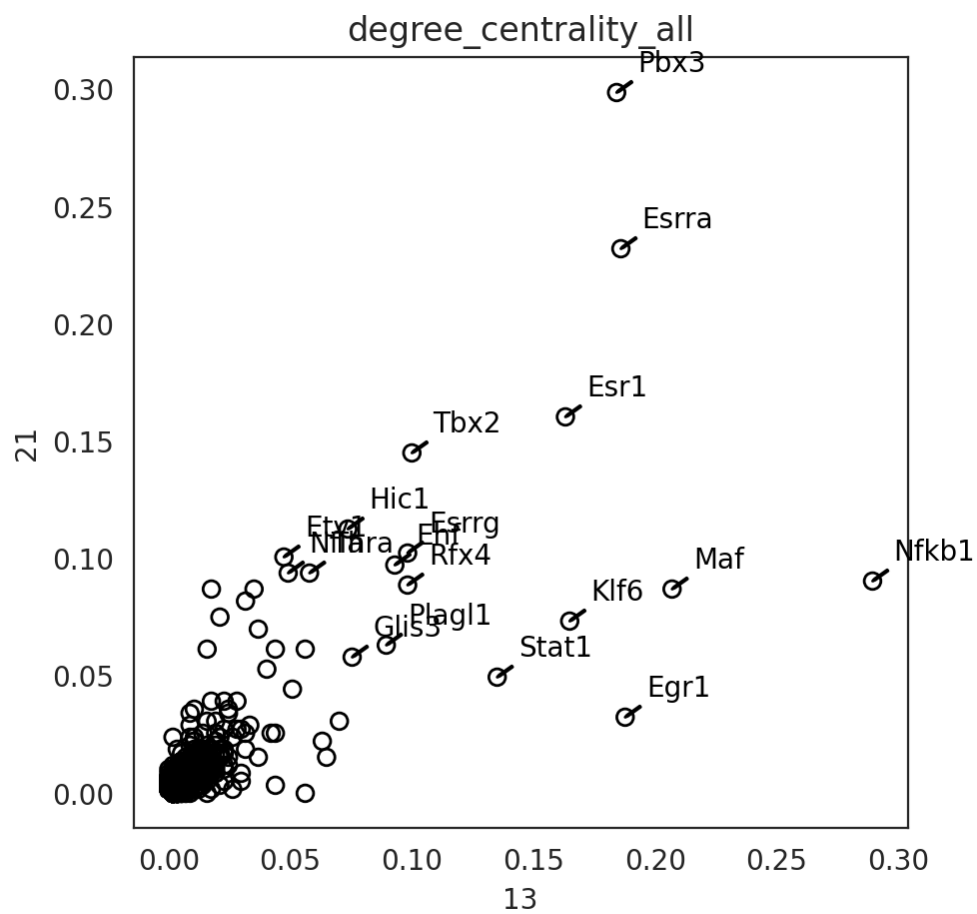
Processing clusters 13 and 19



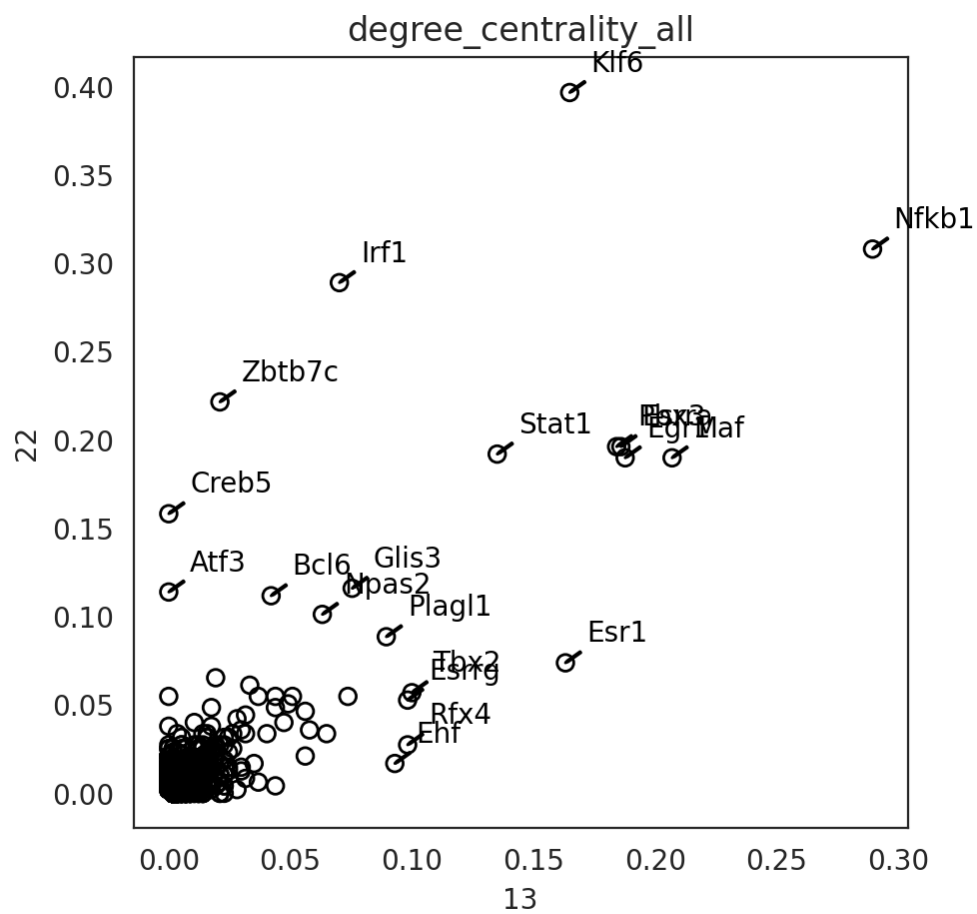
Processing clusters 13 and 20

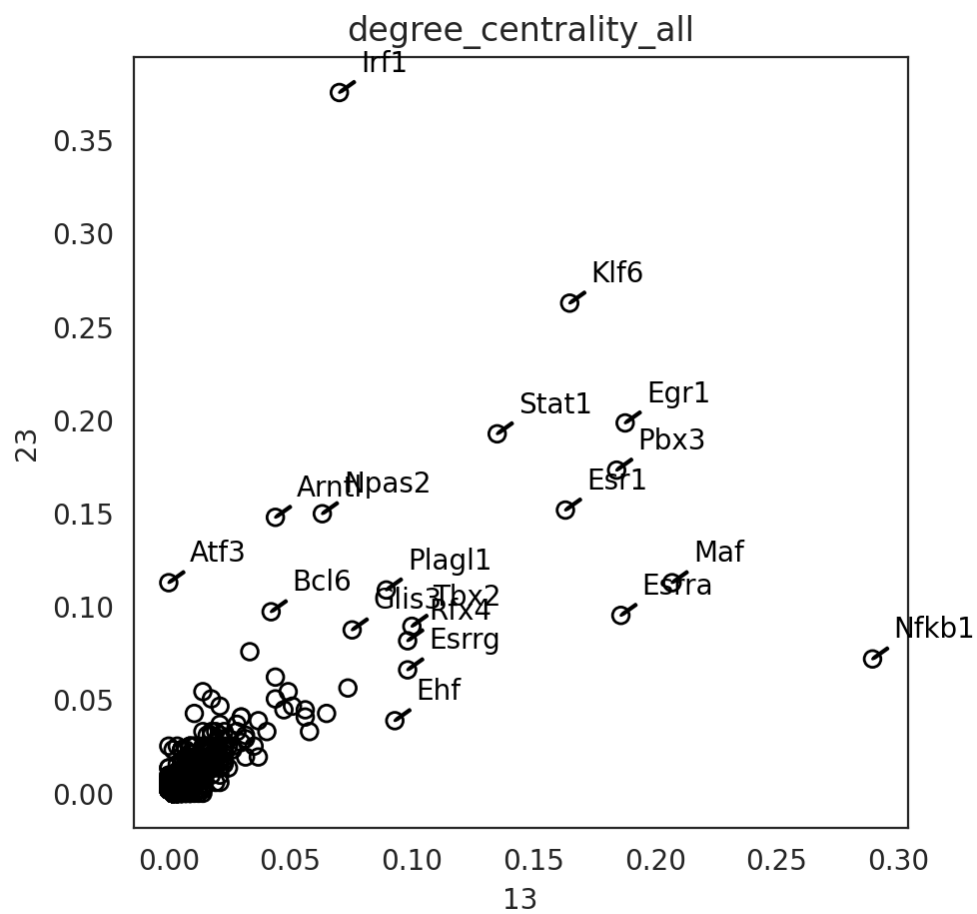


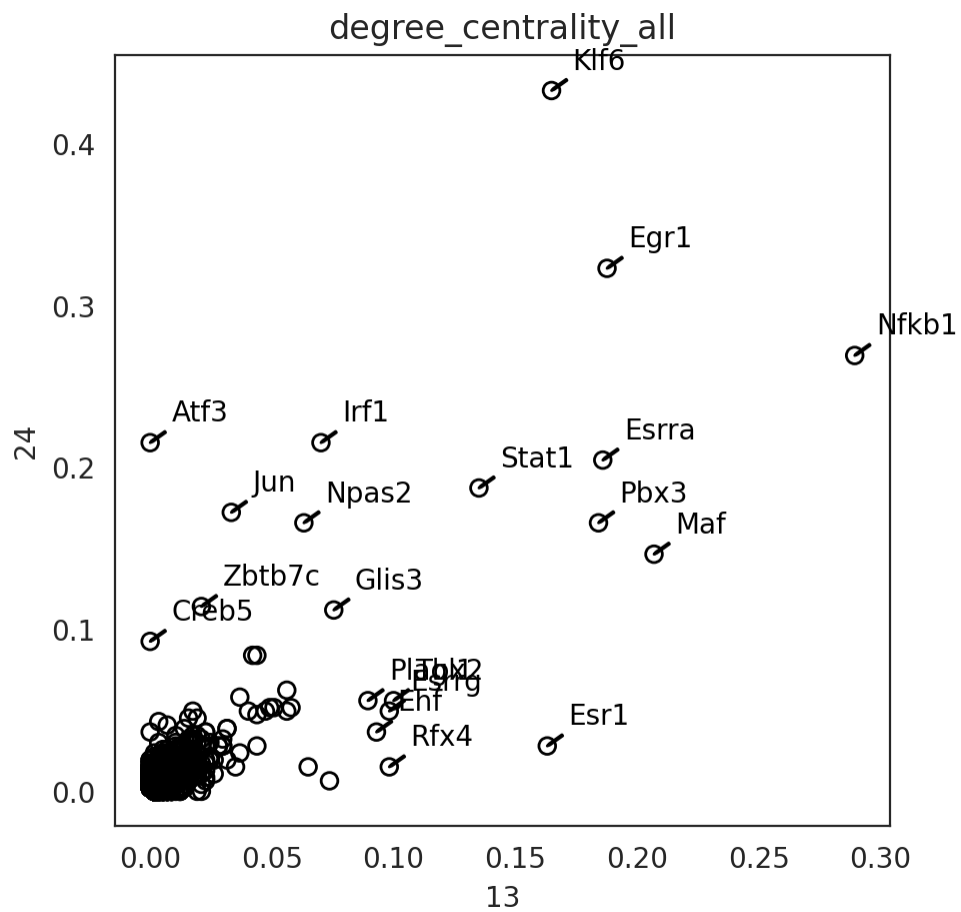
Processing clusters 13 and 21

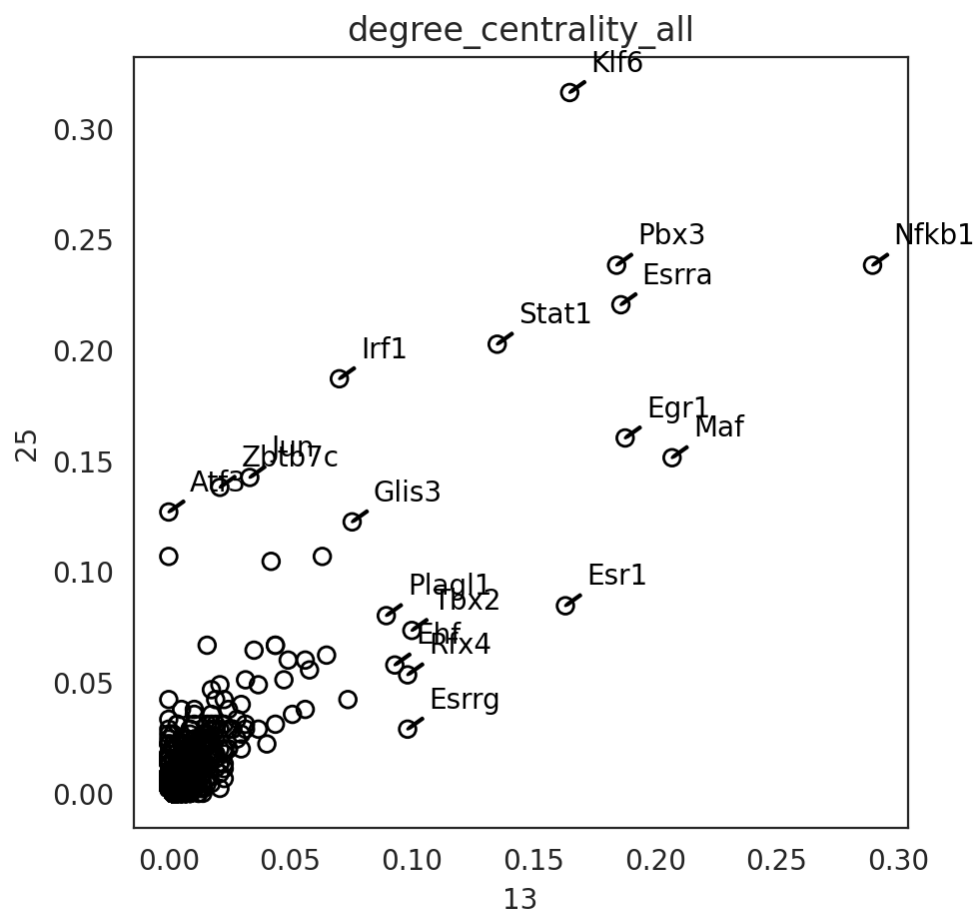


Processing clusters 13 and 22

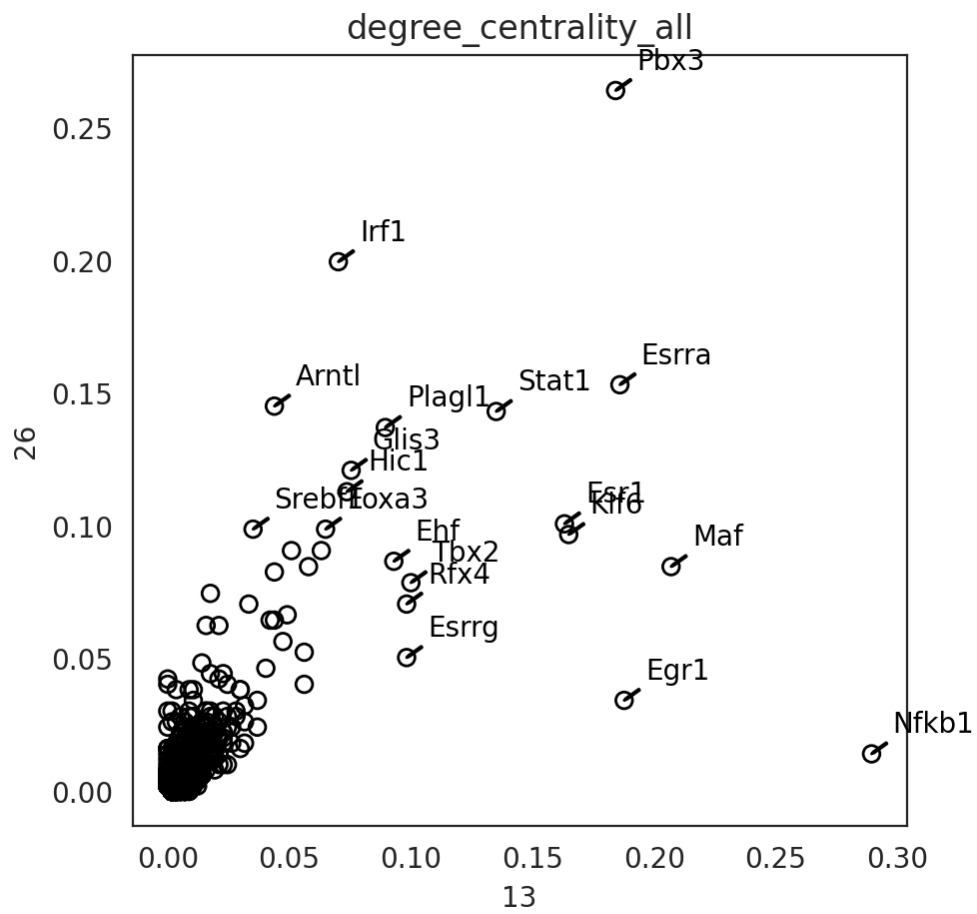


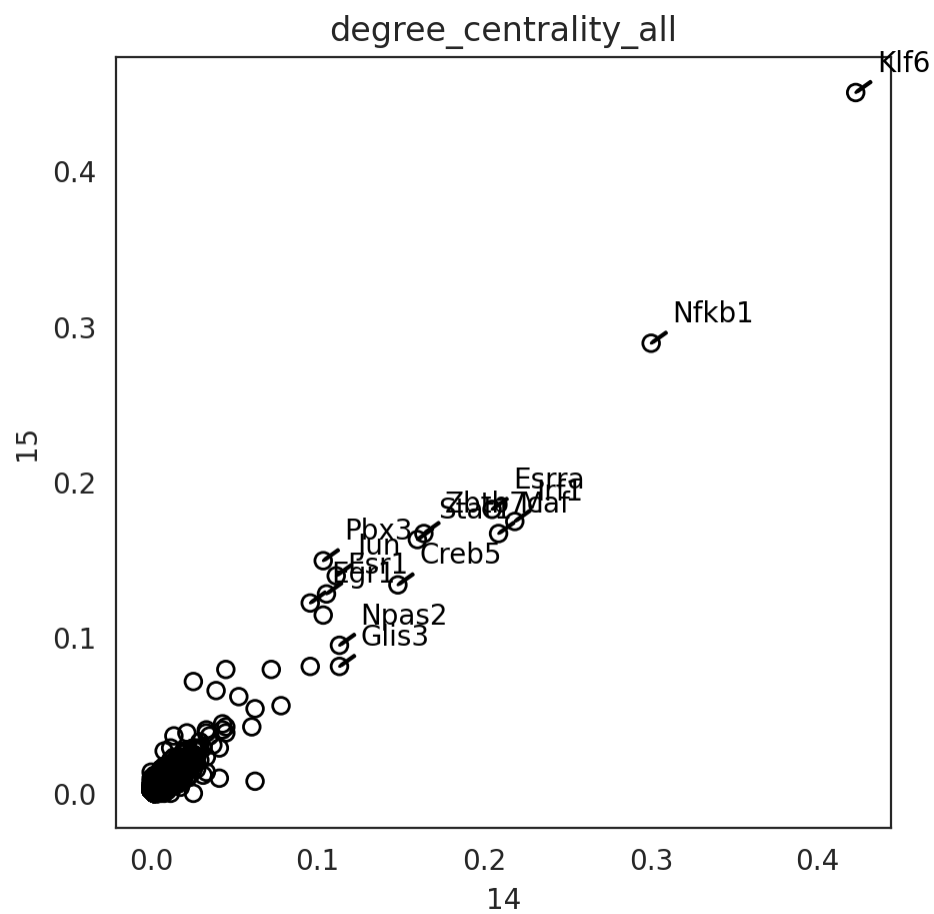




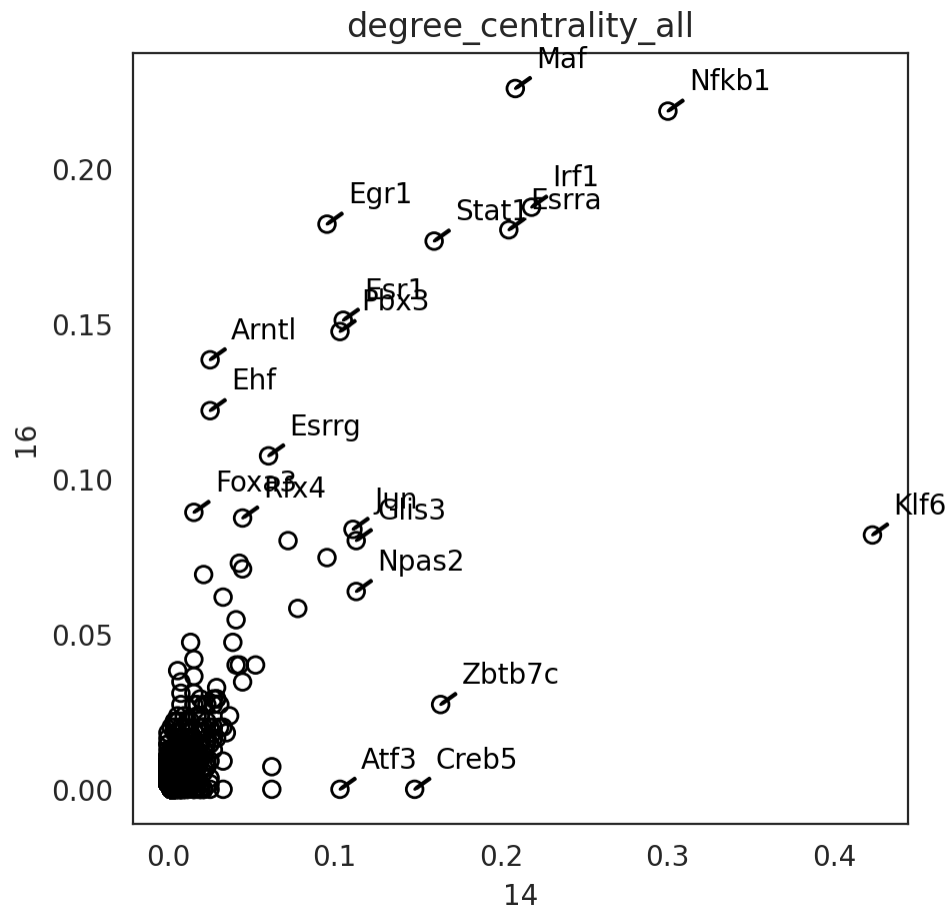


Processing clusters 13 and 26

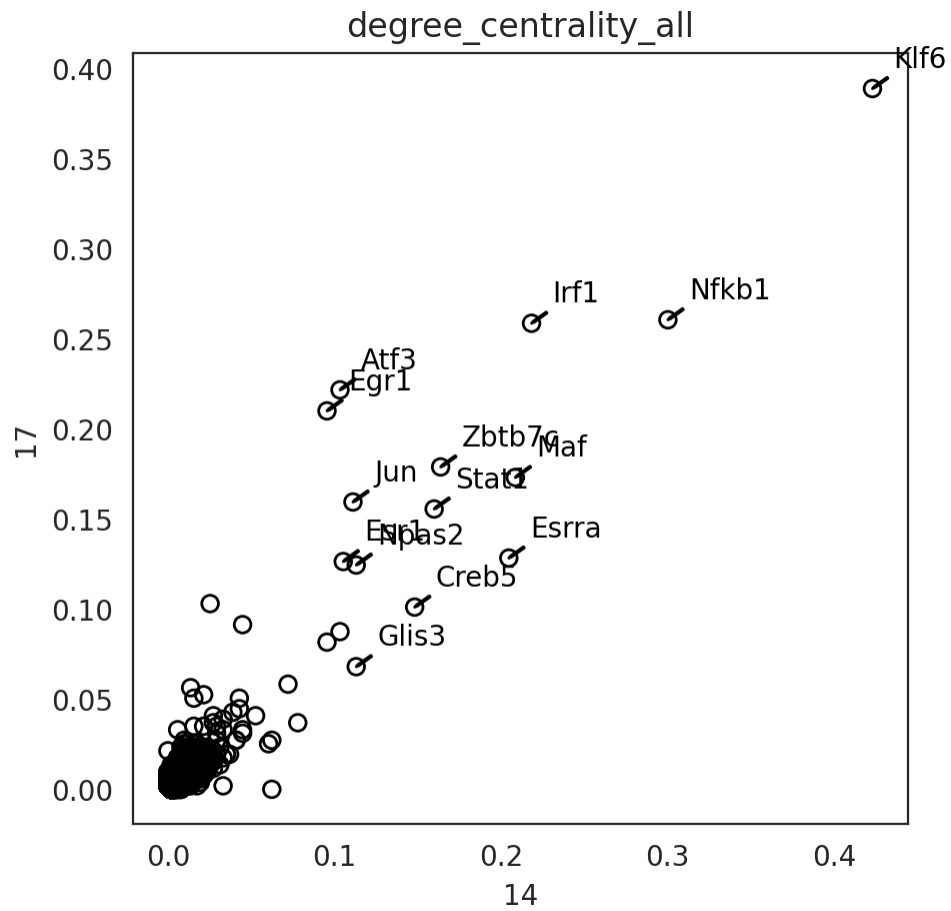




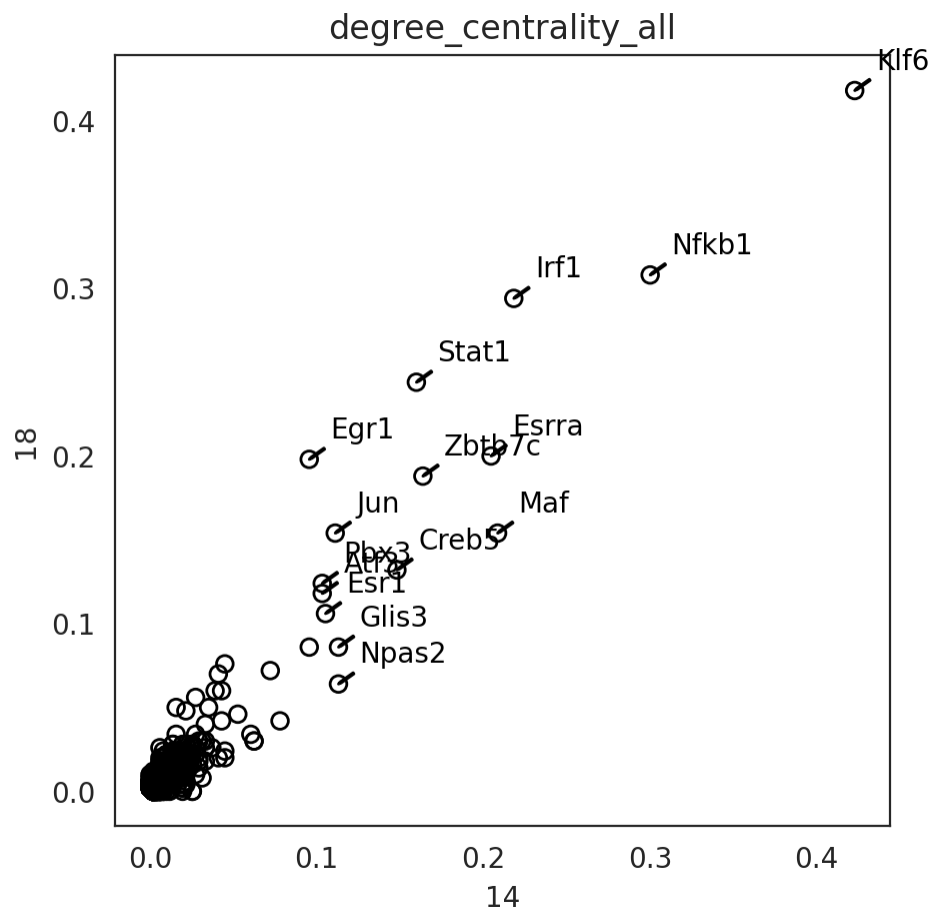
Processing clusters 14 and 16



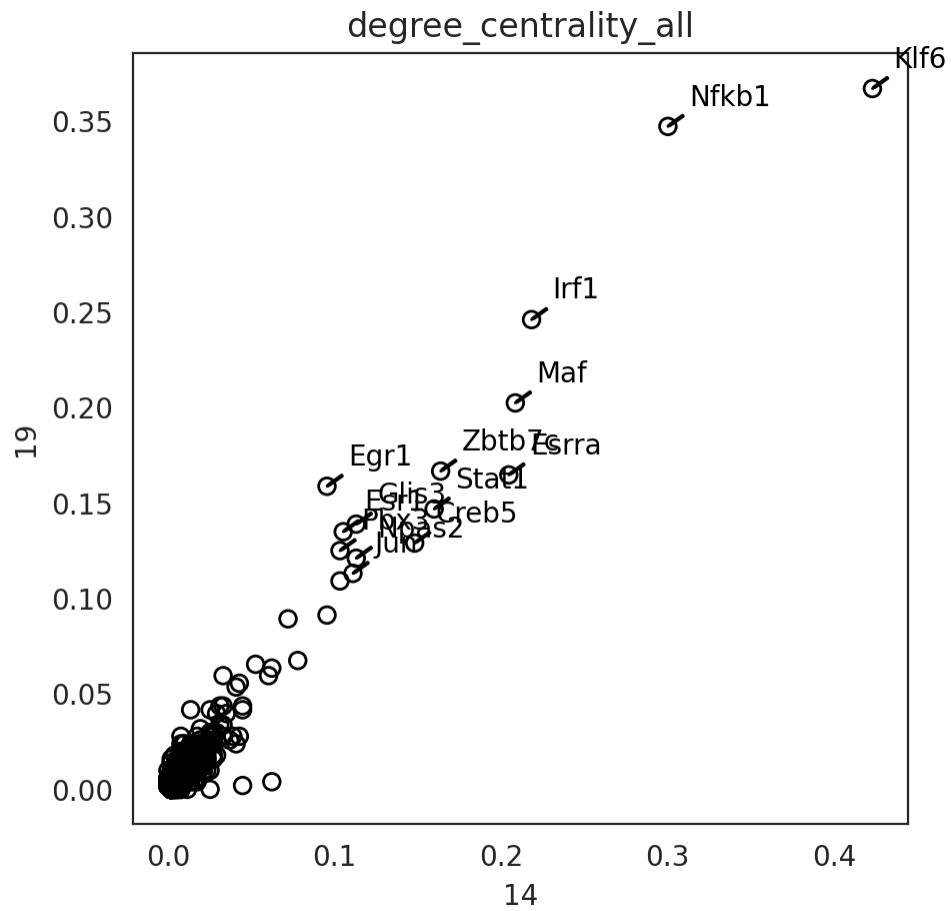
Processing clusters 14 and 17



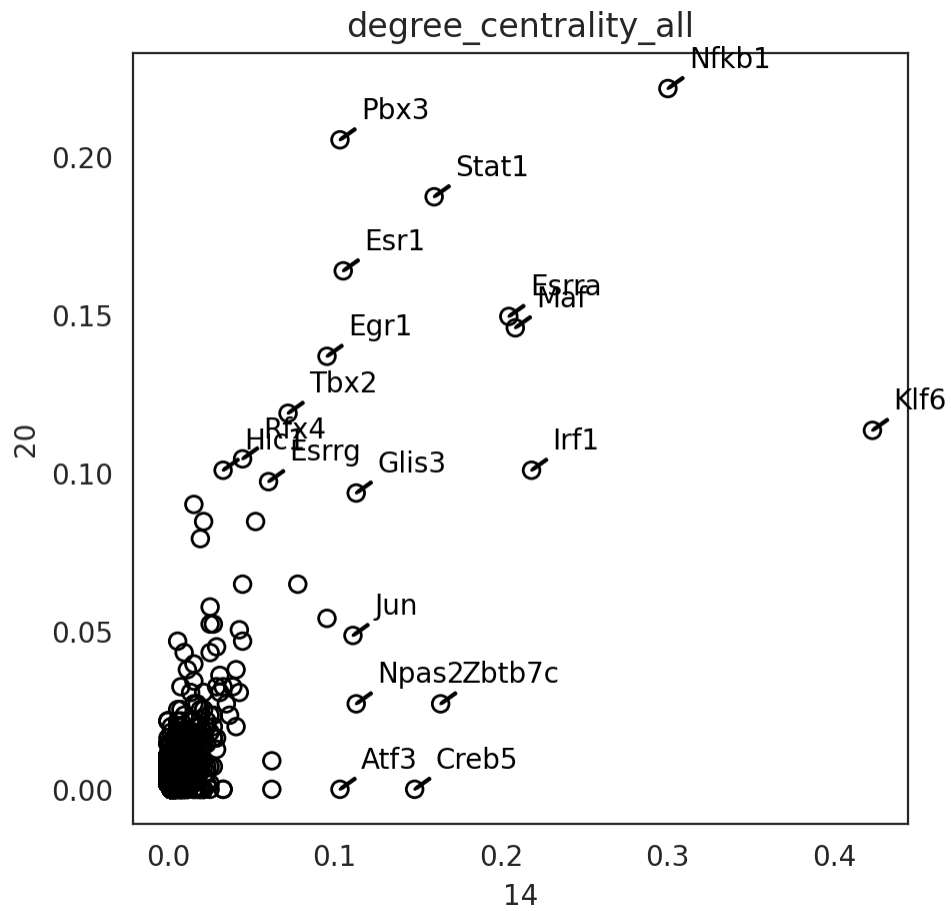
Processing clusters 14 and 18



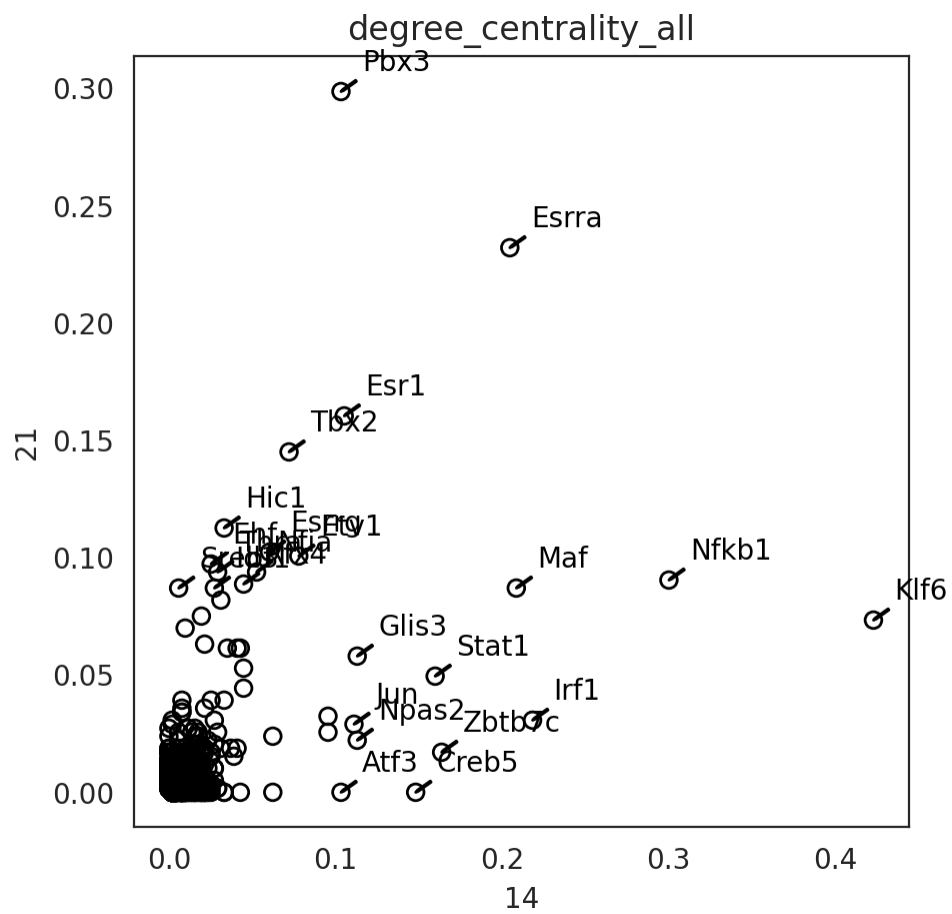
Processing clusters 14 and 19



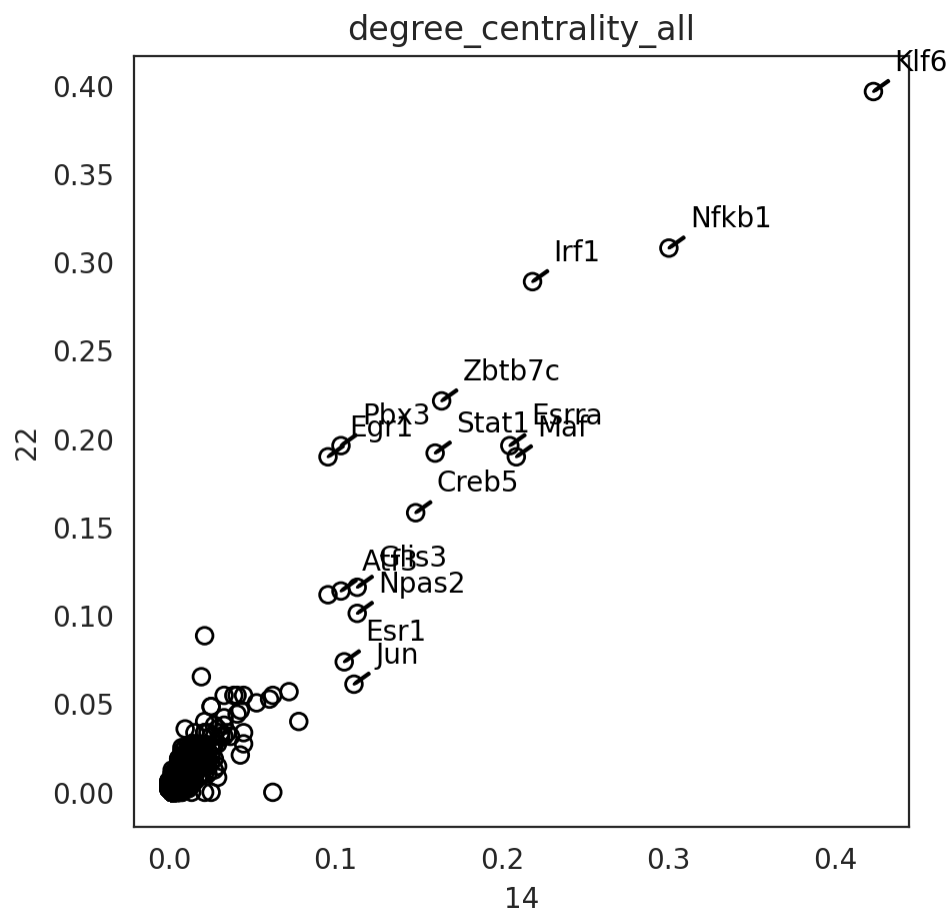
Processing clusters 14 and 20



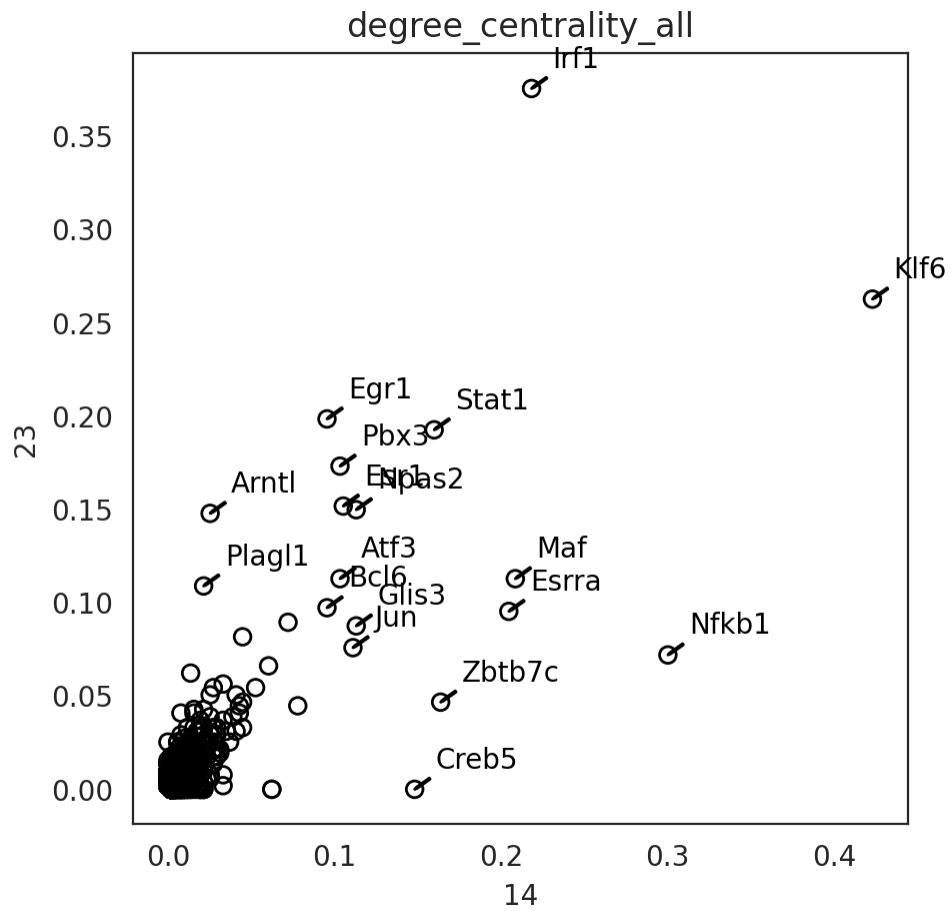
Processing clusters 14 and 21



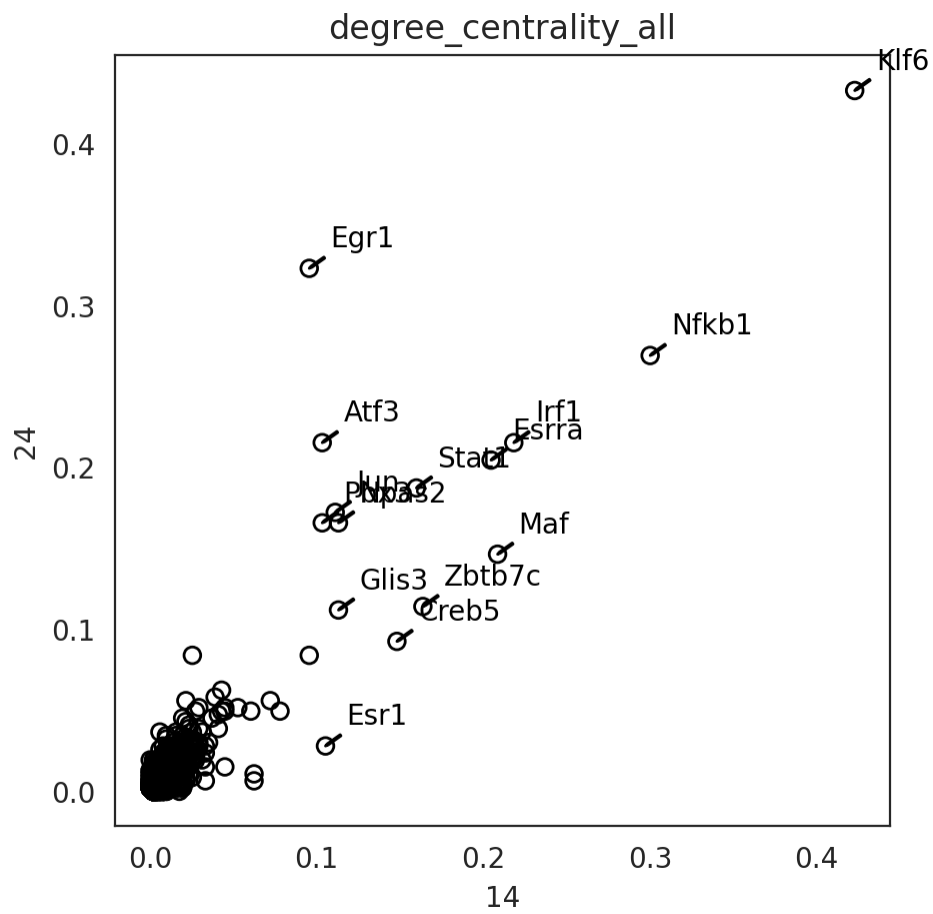
Processing clusters 14 and 22

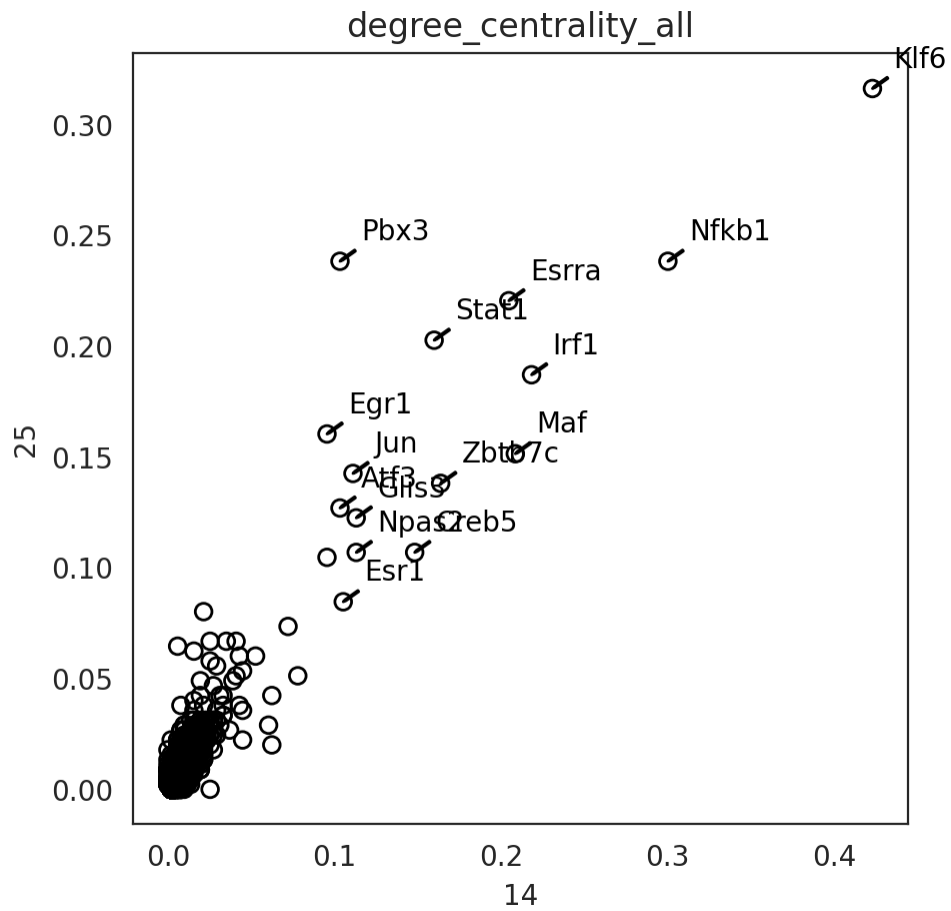


Processing clusters 14 and 23

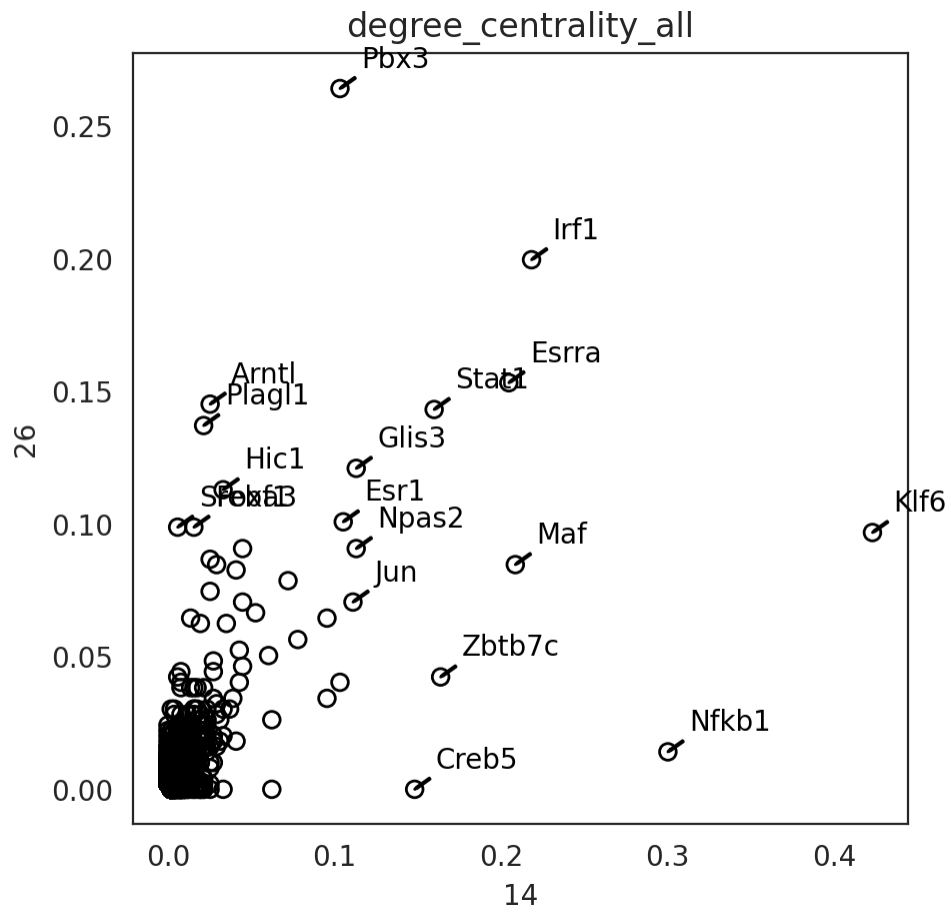


Processing clusters 14 and 24

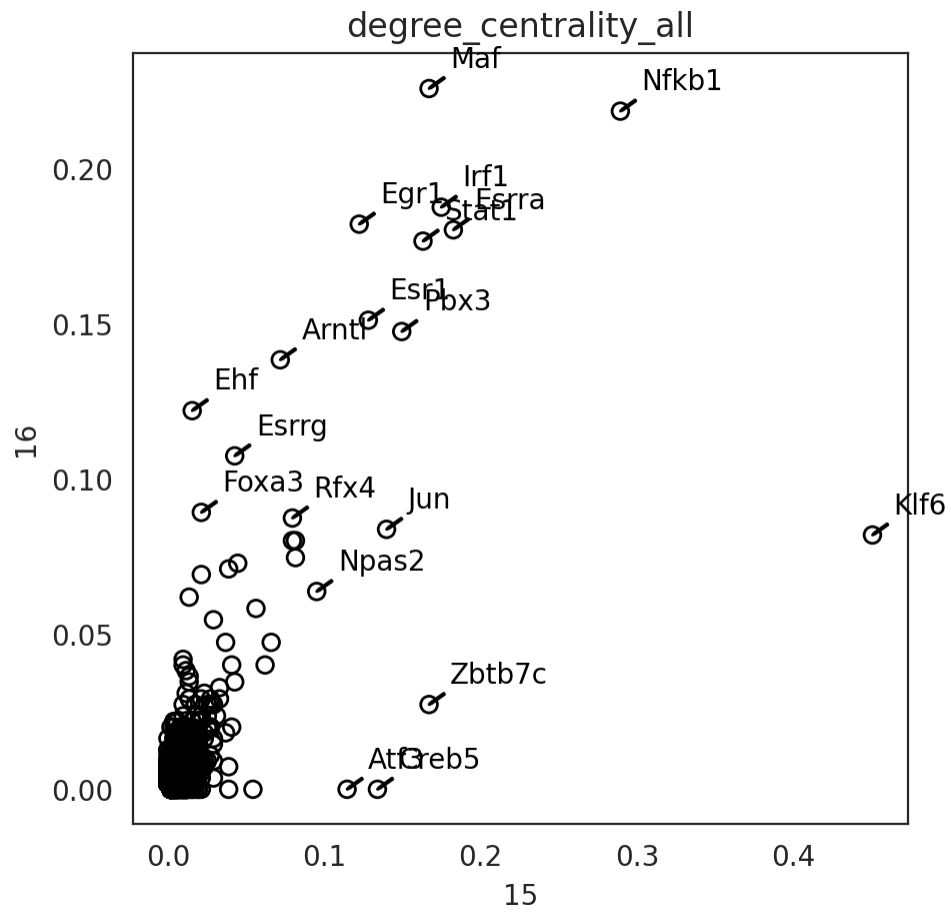




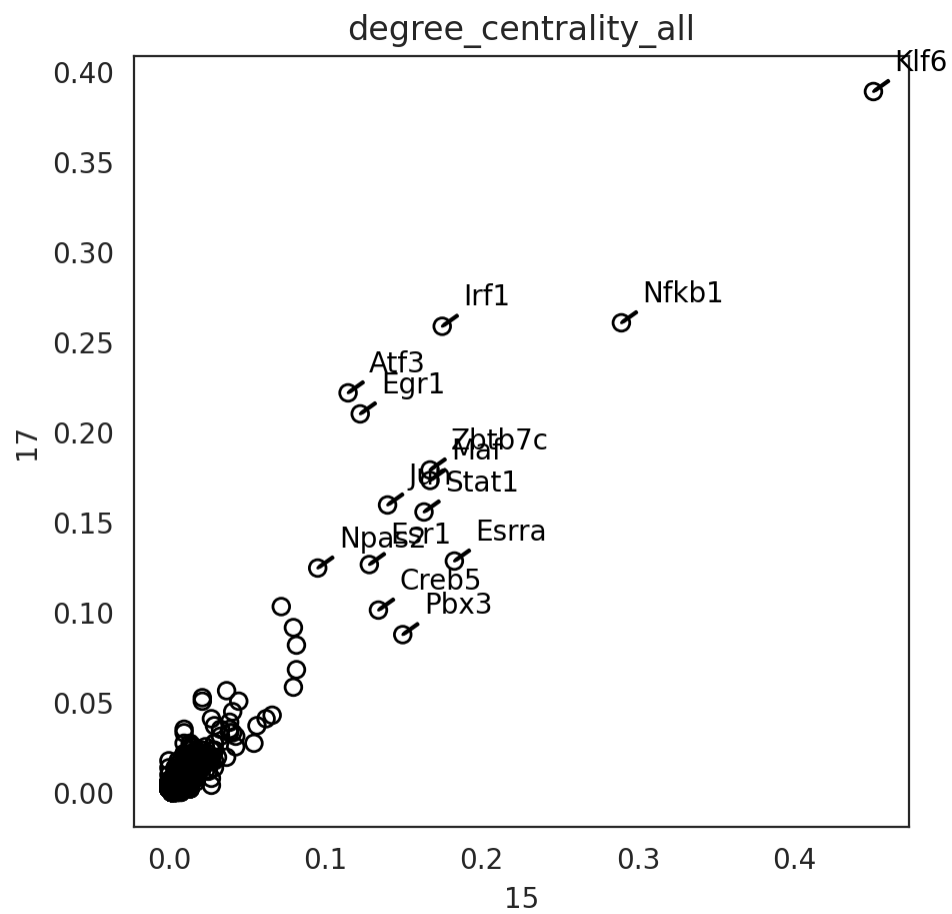
Processing clusters 14 and 26



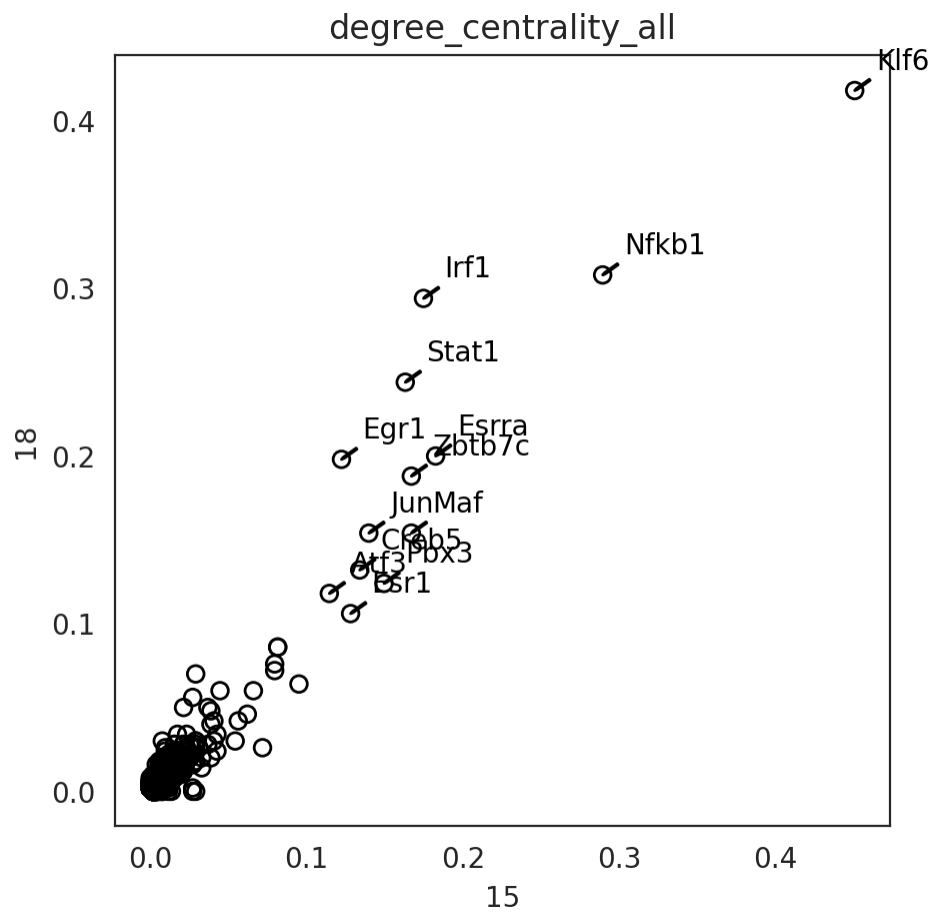
Processing clusters 15 and 16



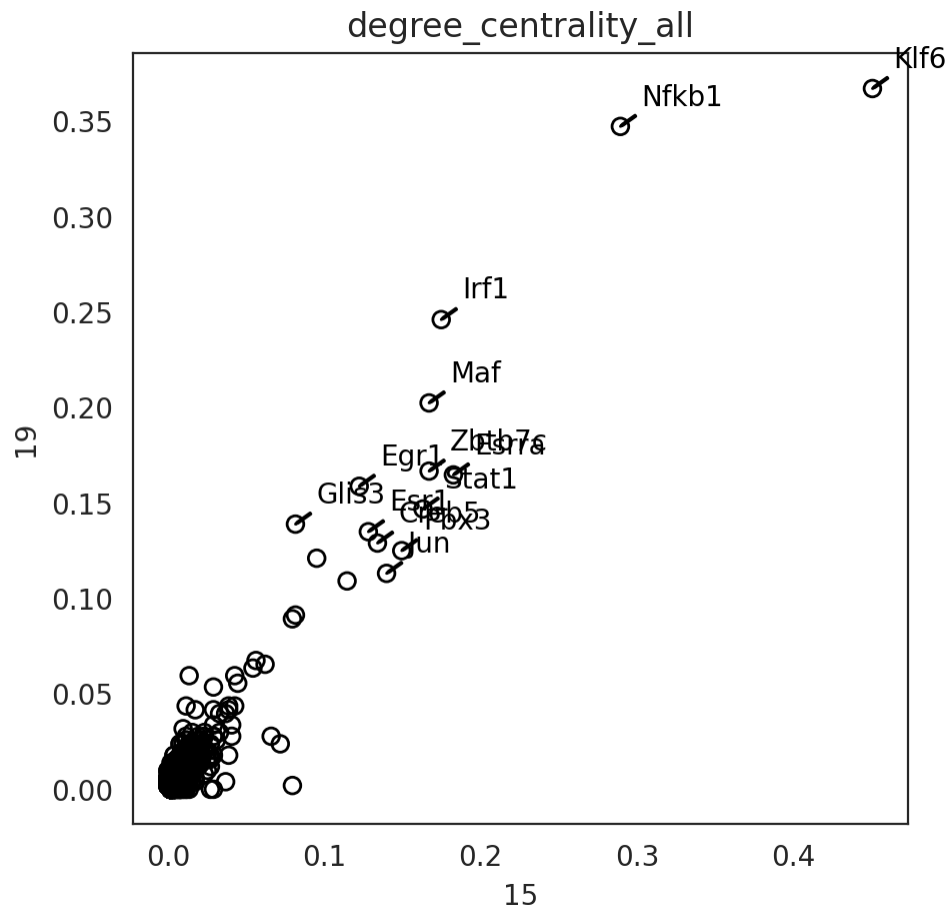
Processing clusters 15 and 17



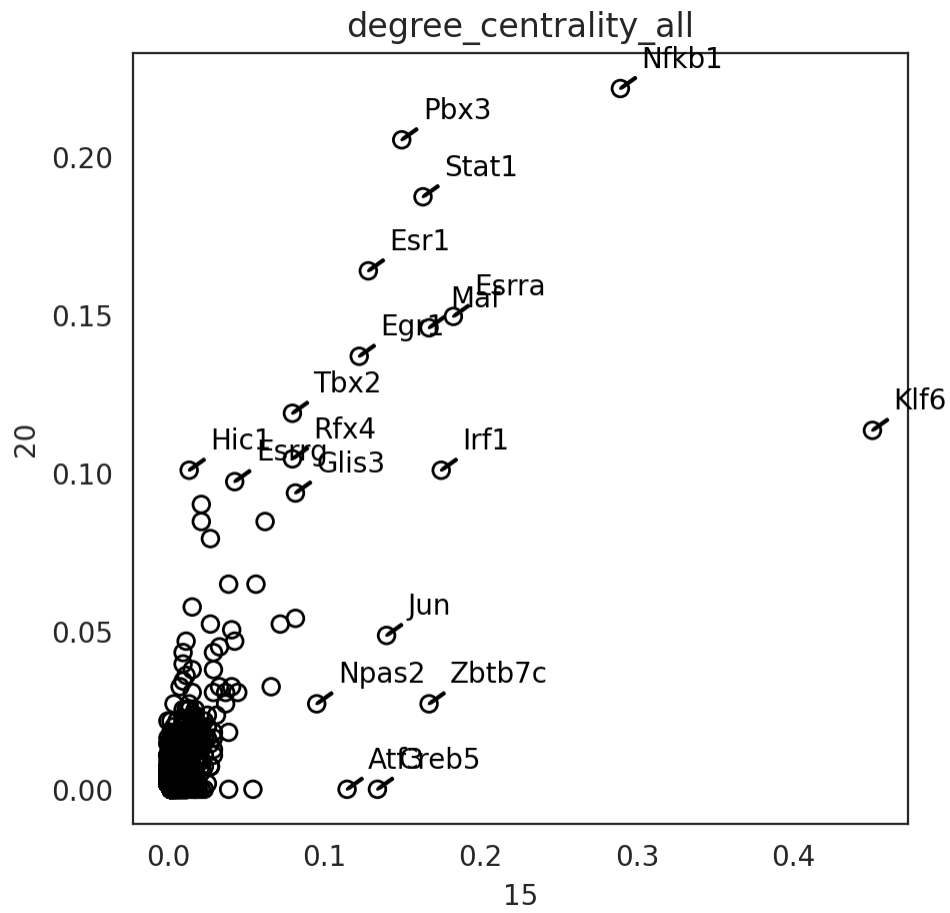
Processing clusters 15 and 18



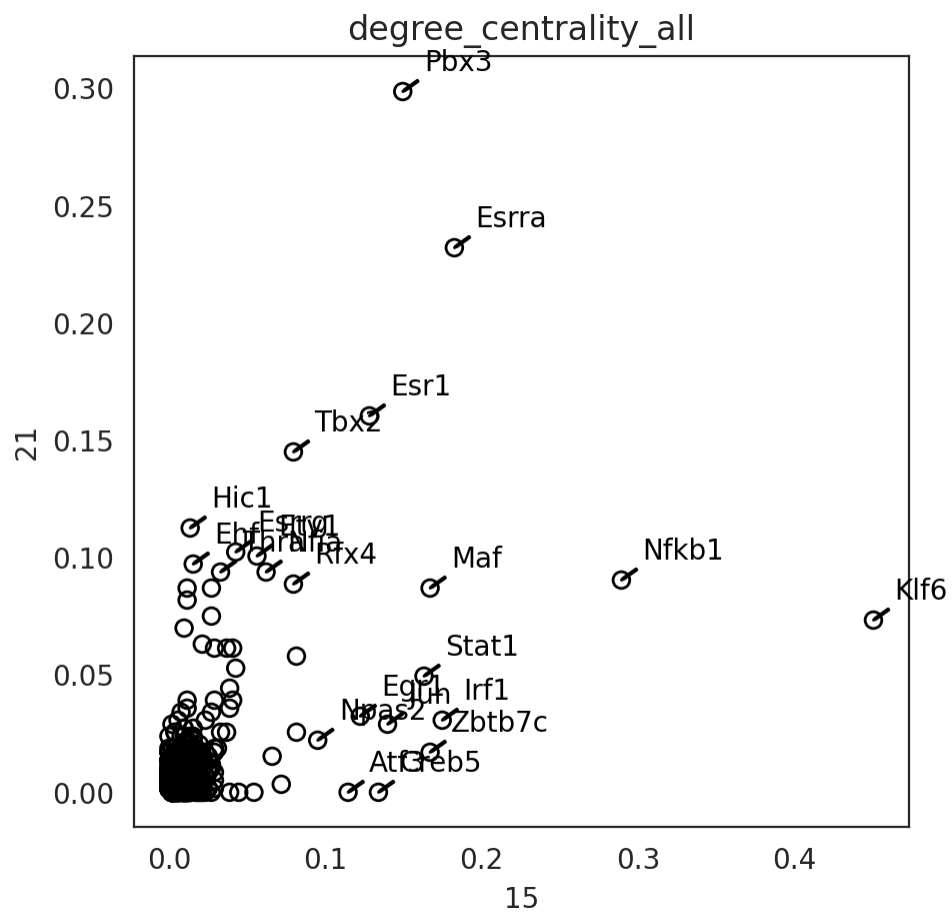
Processing clusters 15 and 19



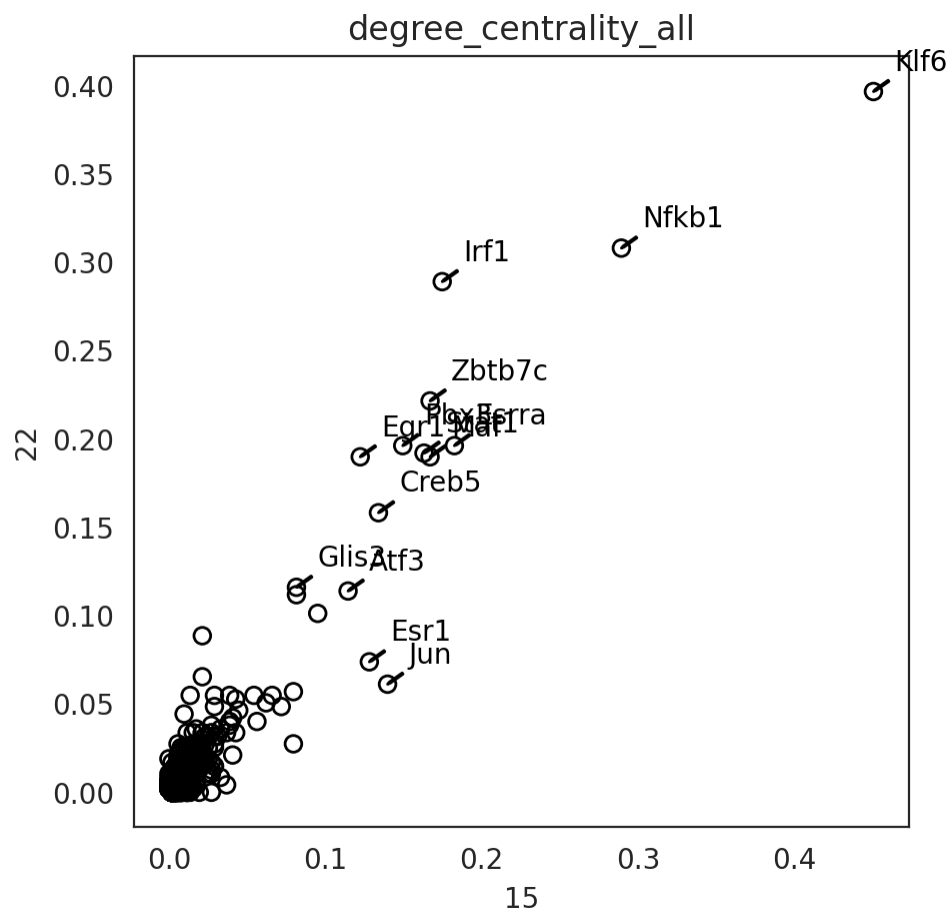
Processing clusters 15 and 20



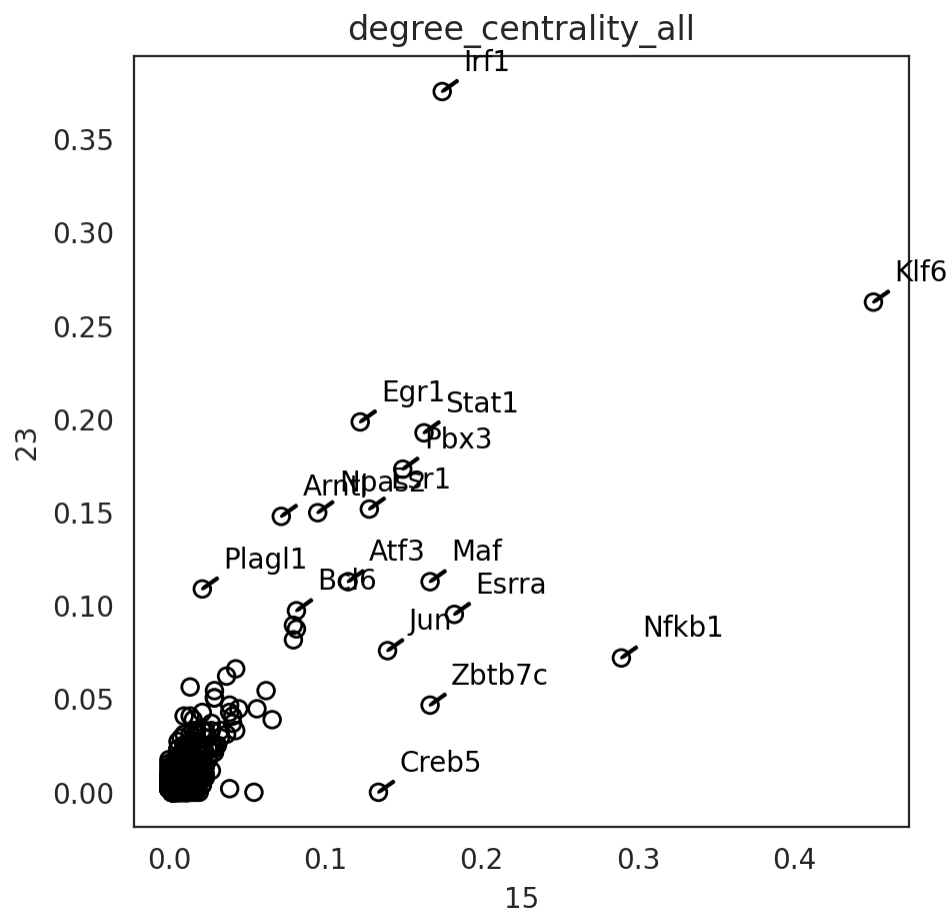
Processing clusters 15 and 21

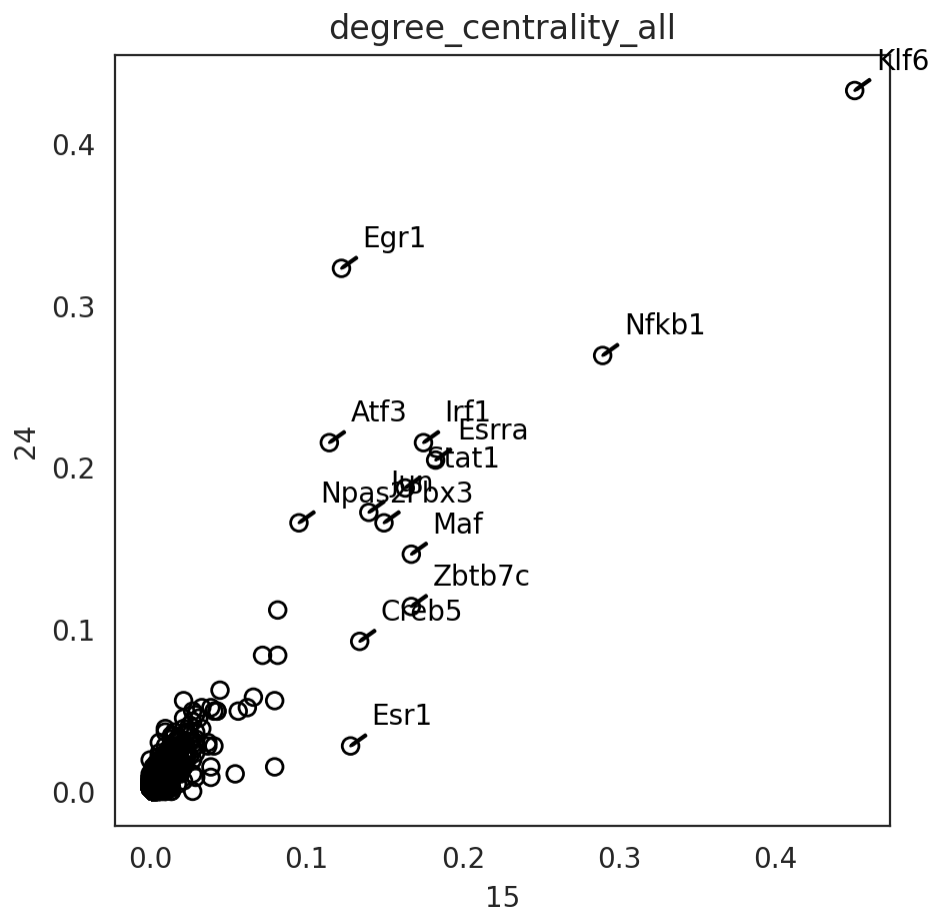


Processing clusters 15 and 22

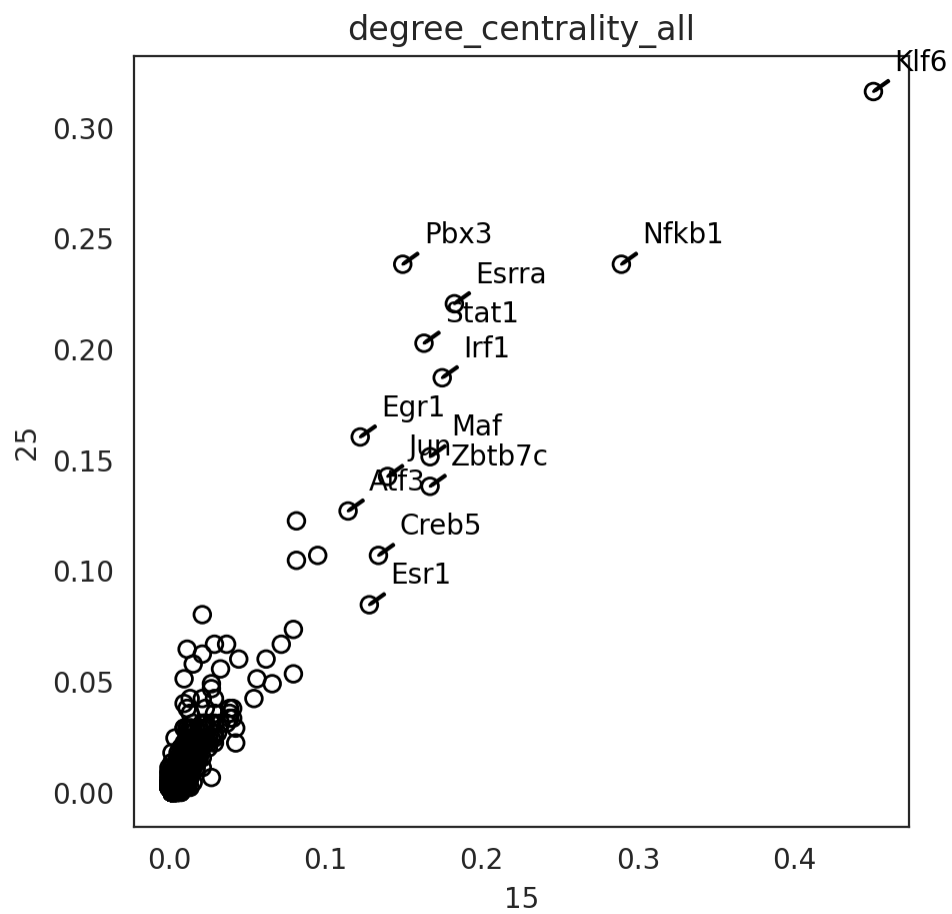


Processing clusters 15 and 23

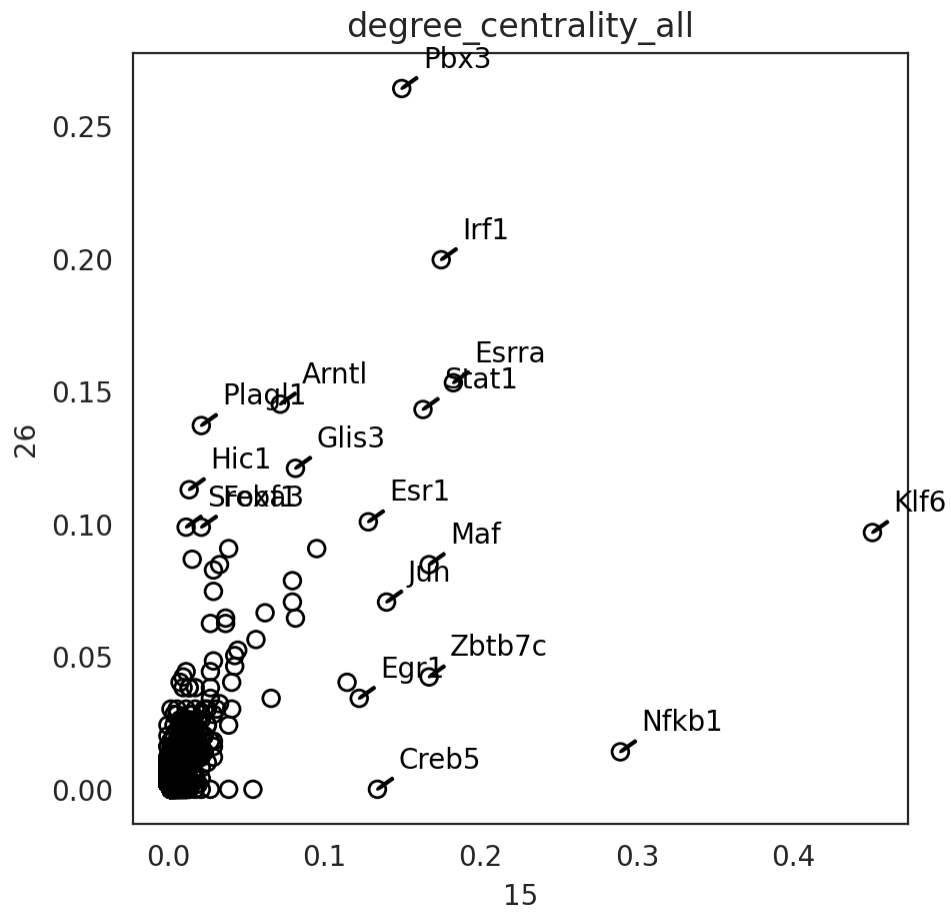




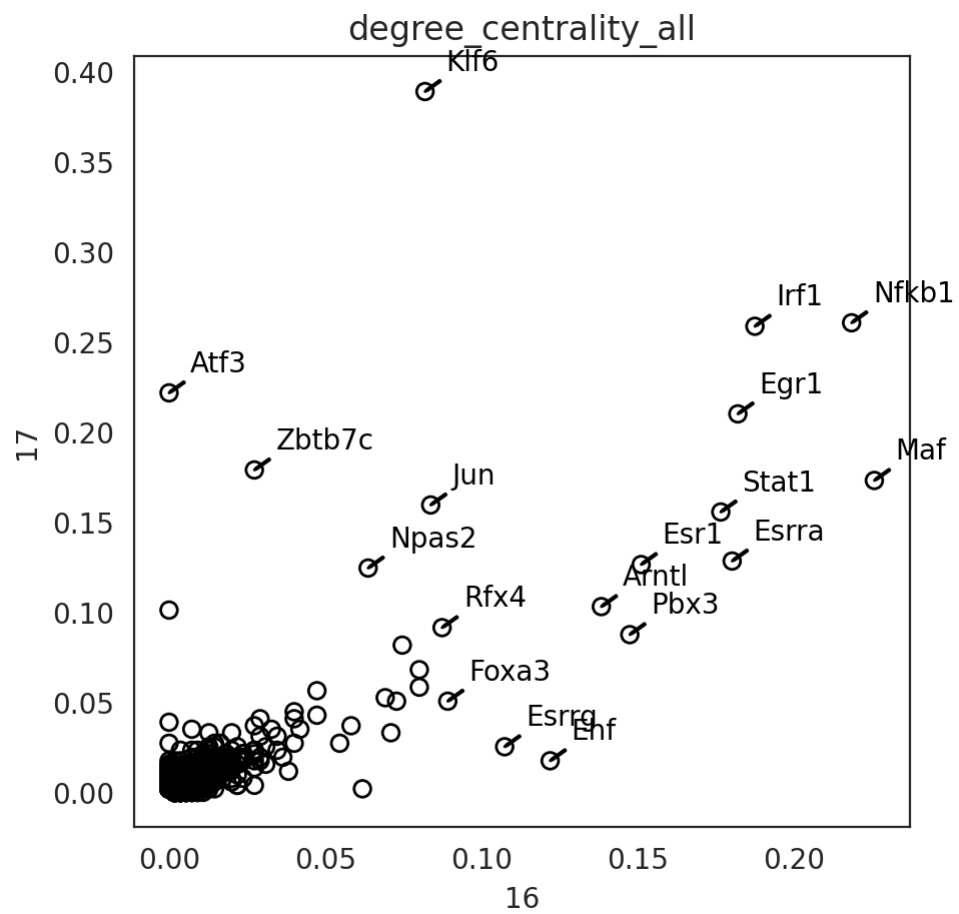
Processing clusters 15 and 25



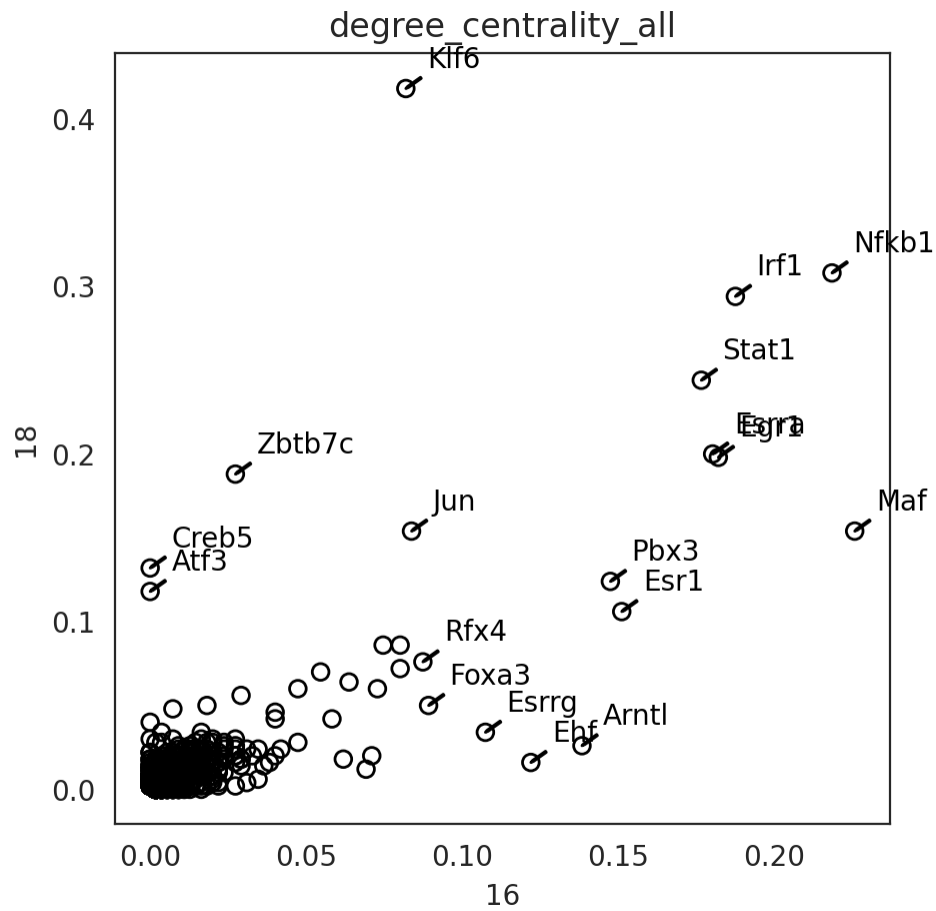
Processing clusters 15 and 26



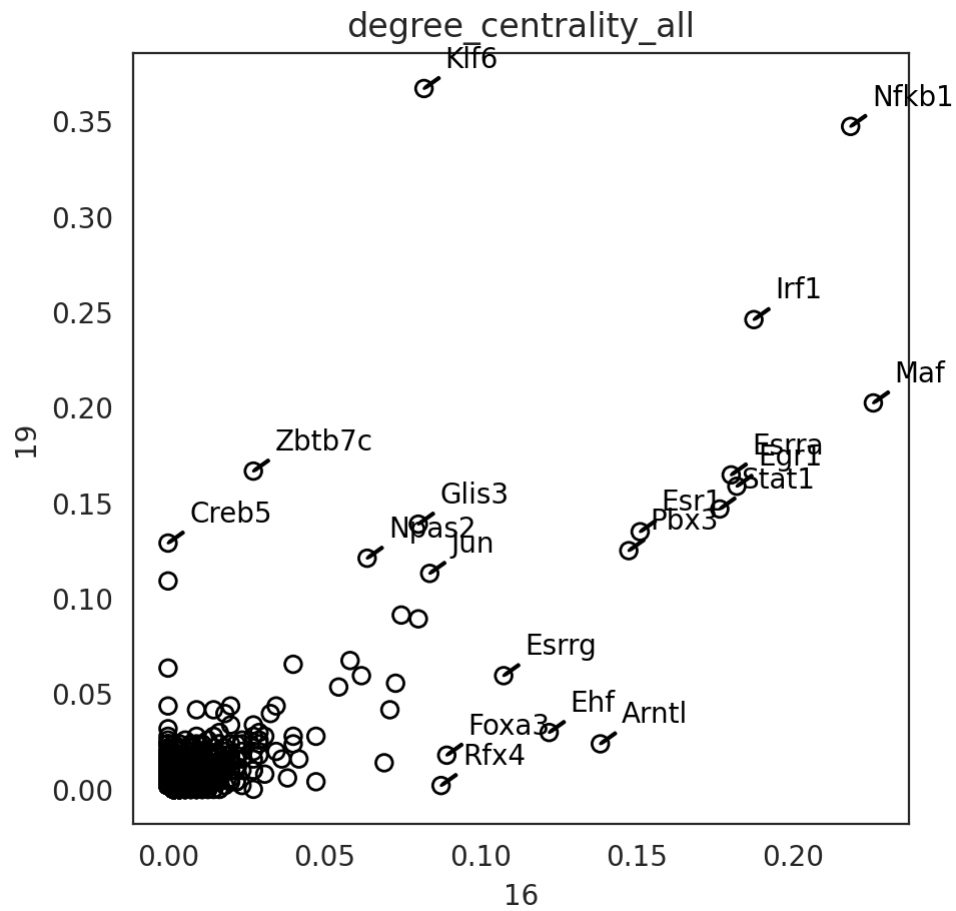
Processing clusters 16 and 17



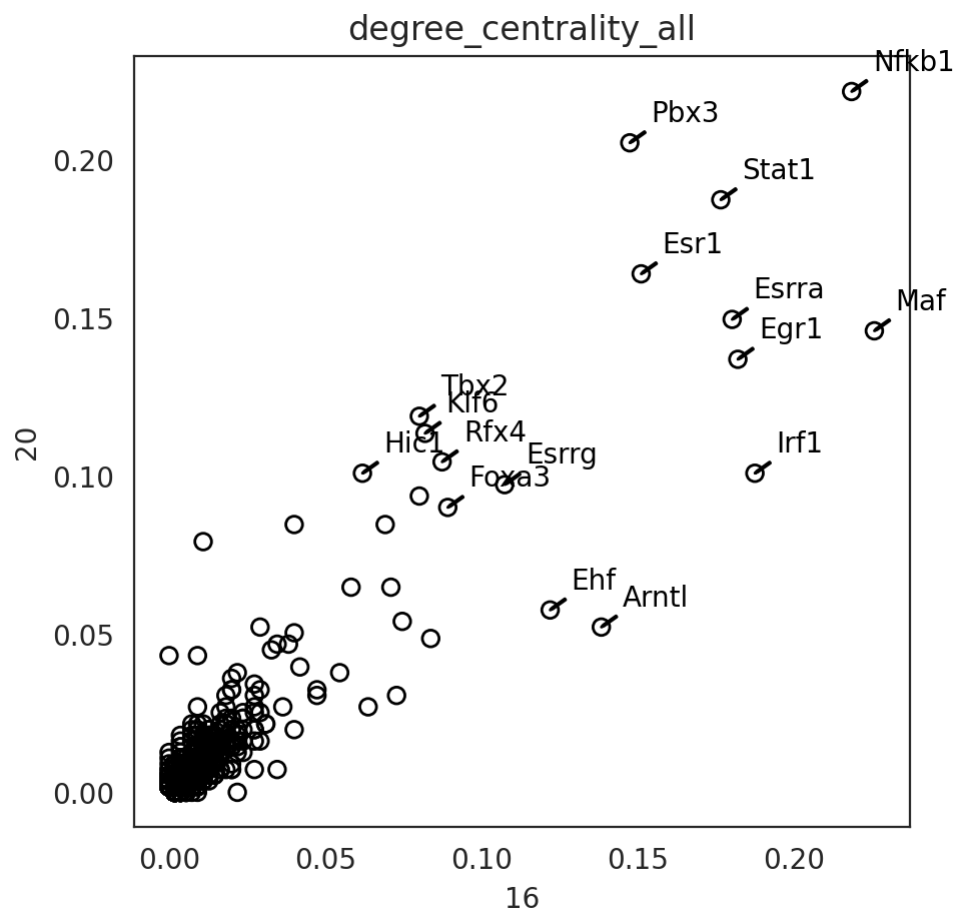
Processing clusters 16 and 18



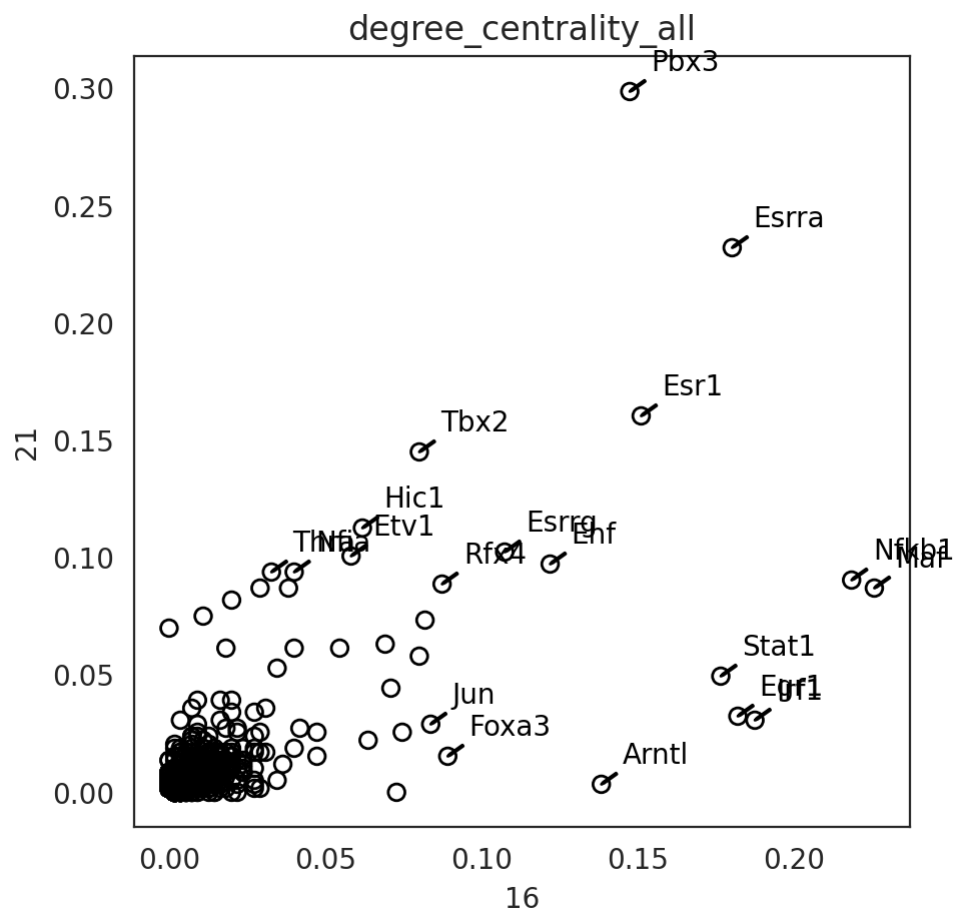
Processing clusters 16 and 19



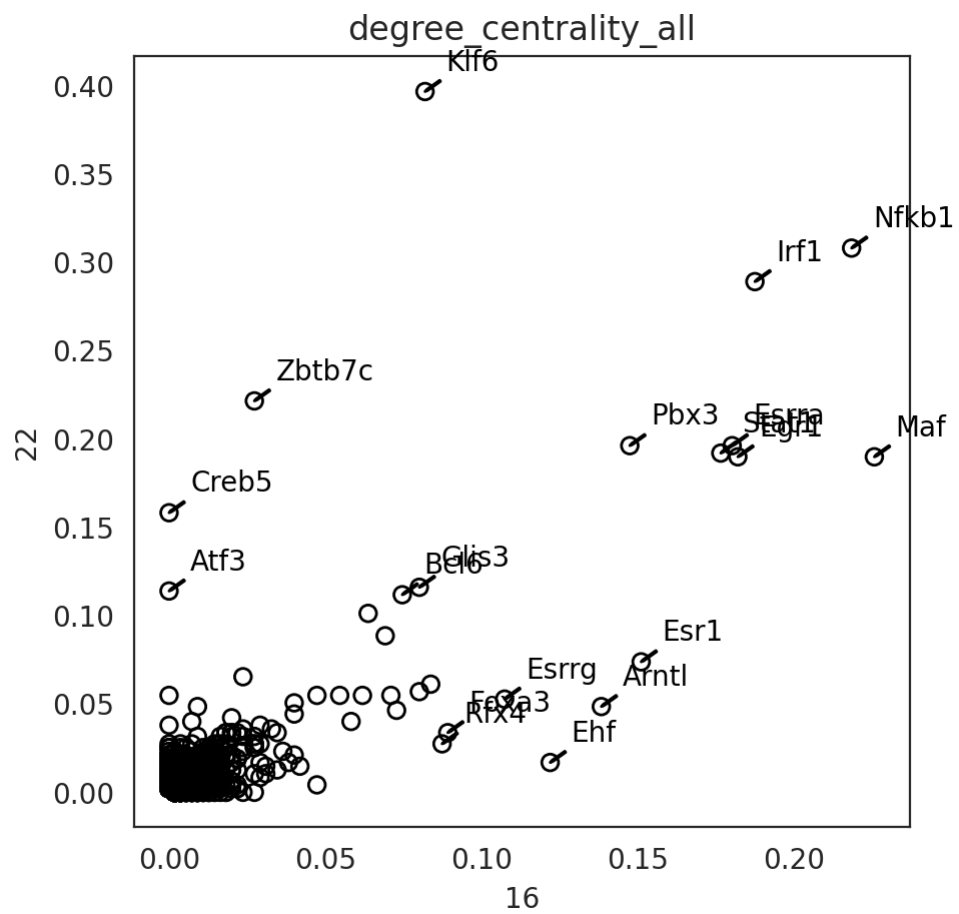
Processing clusters 16 and 20

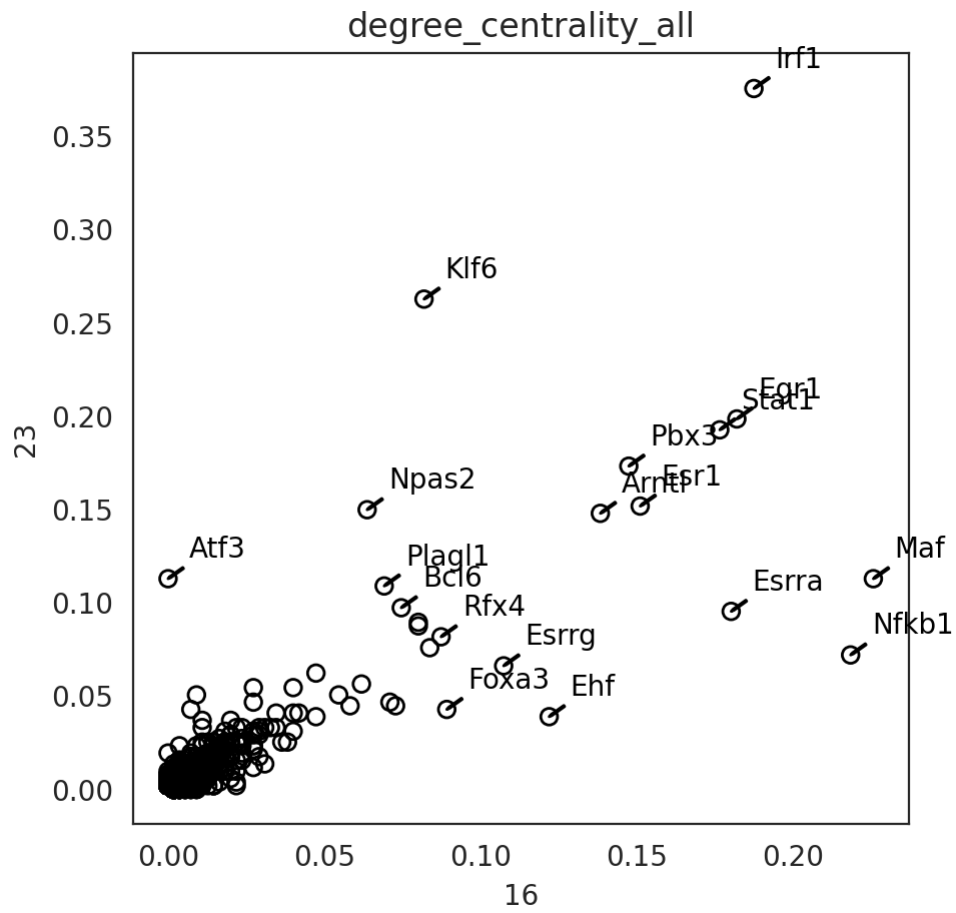


Processing clusters 16 and 21

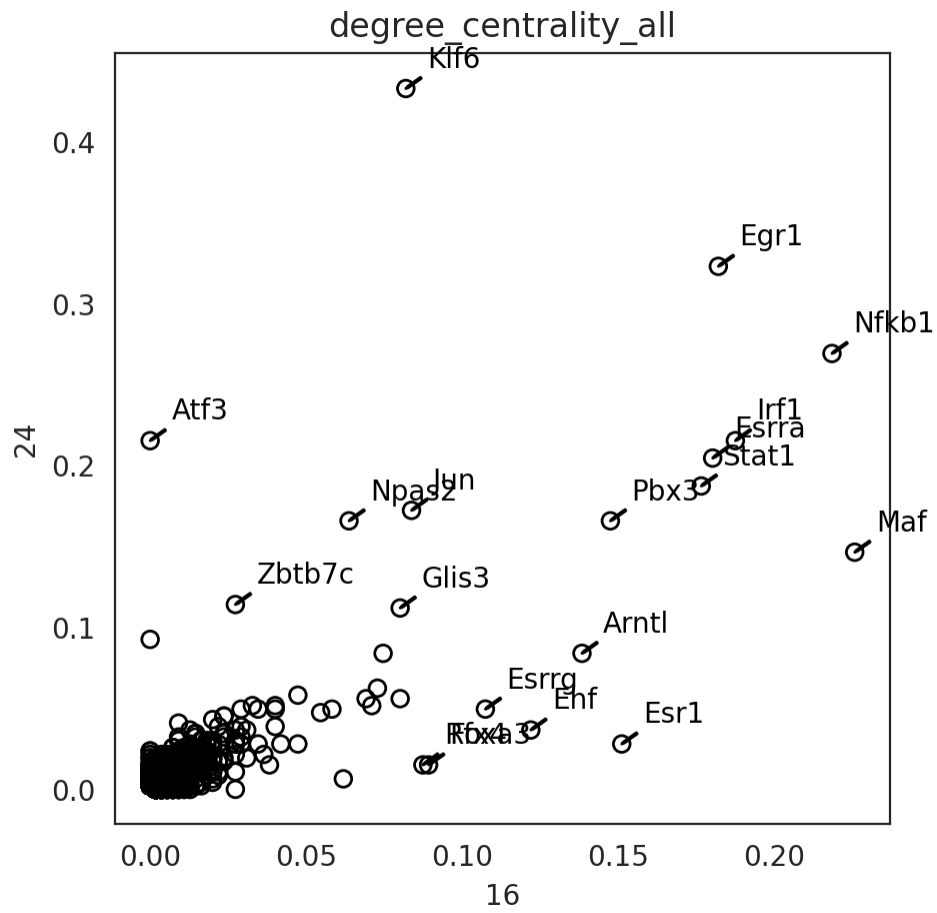


Processing clusters 16 and 22

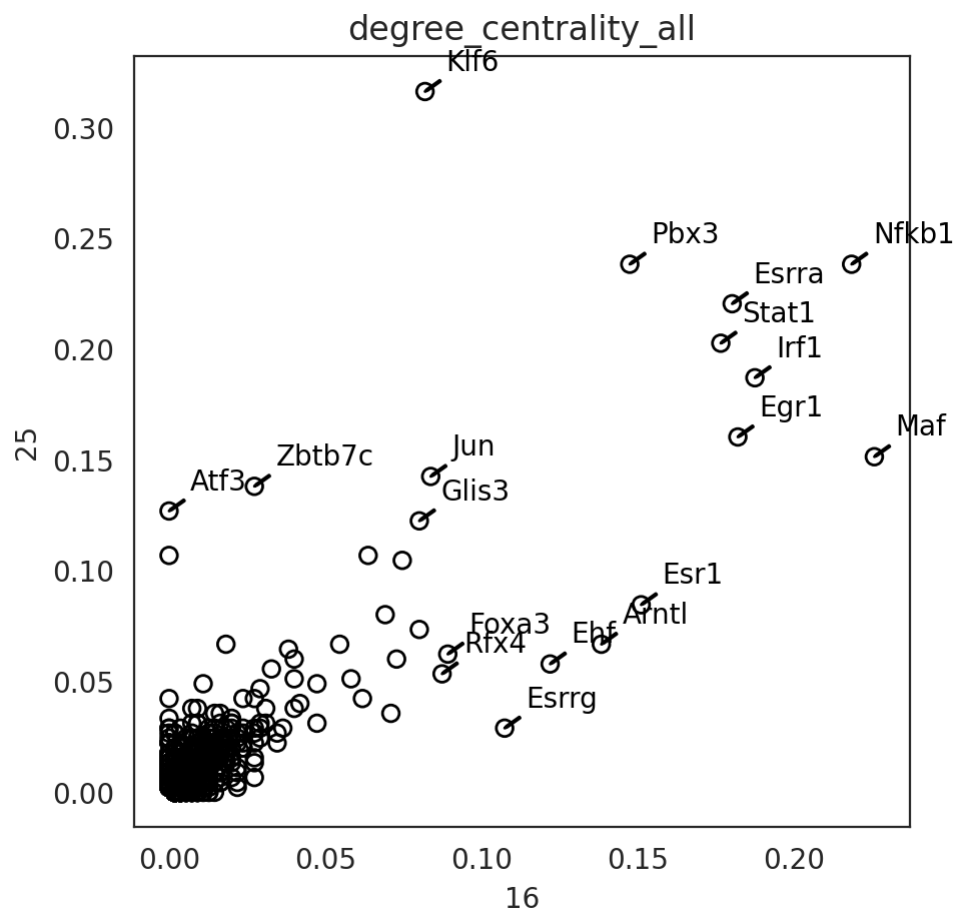




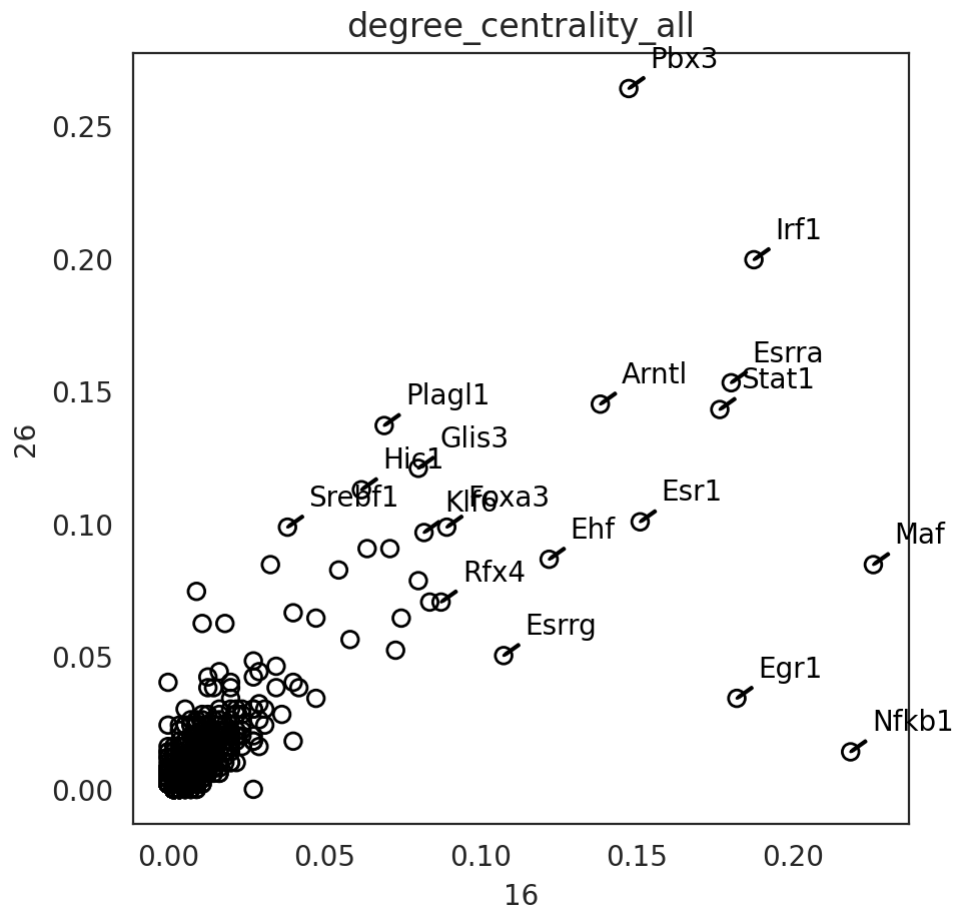
Processing clusters 16 and 24



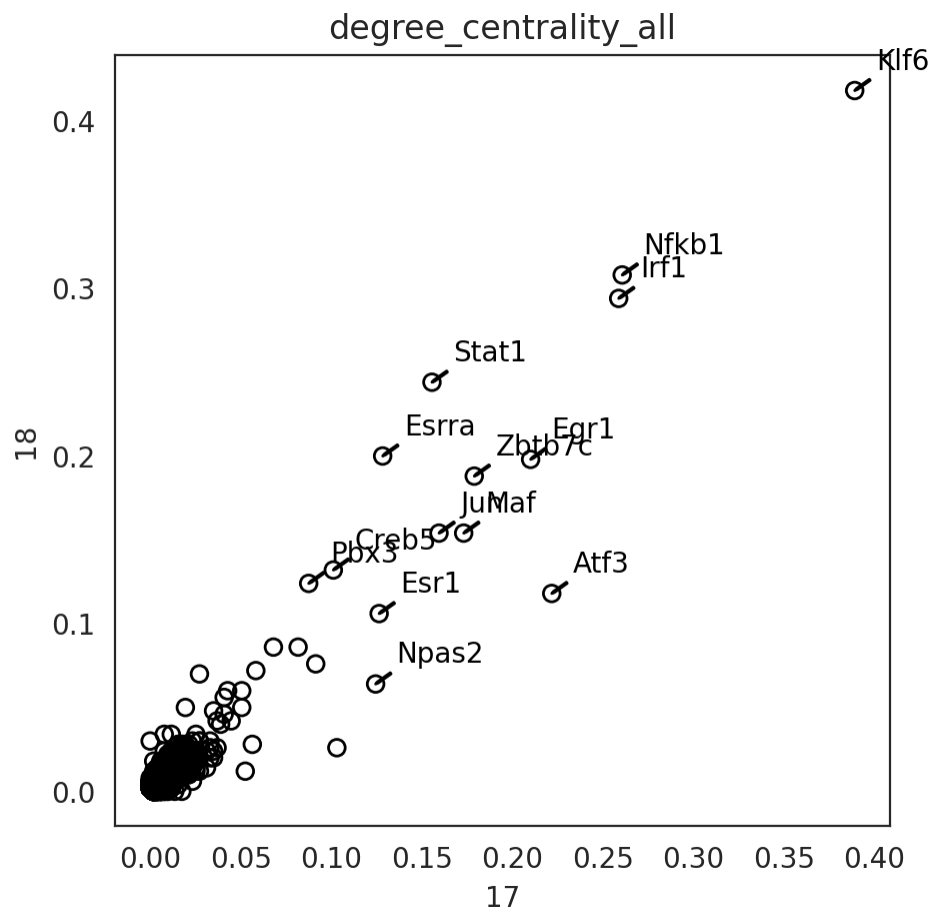
Processing clusters 16 and 25



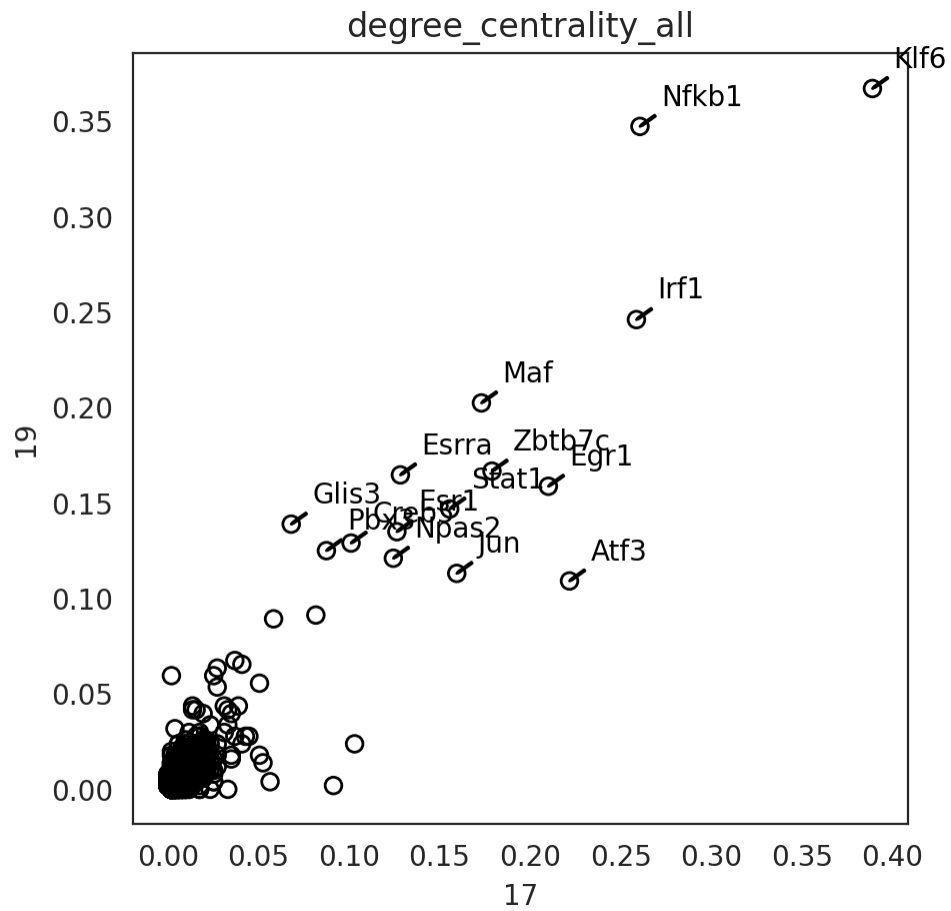
Processing clusters 16 and 26



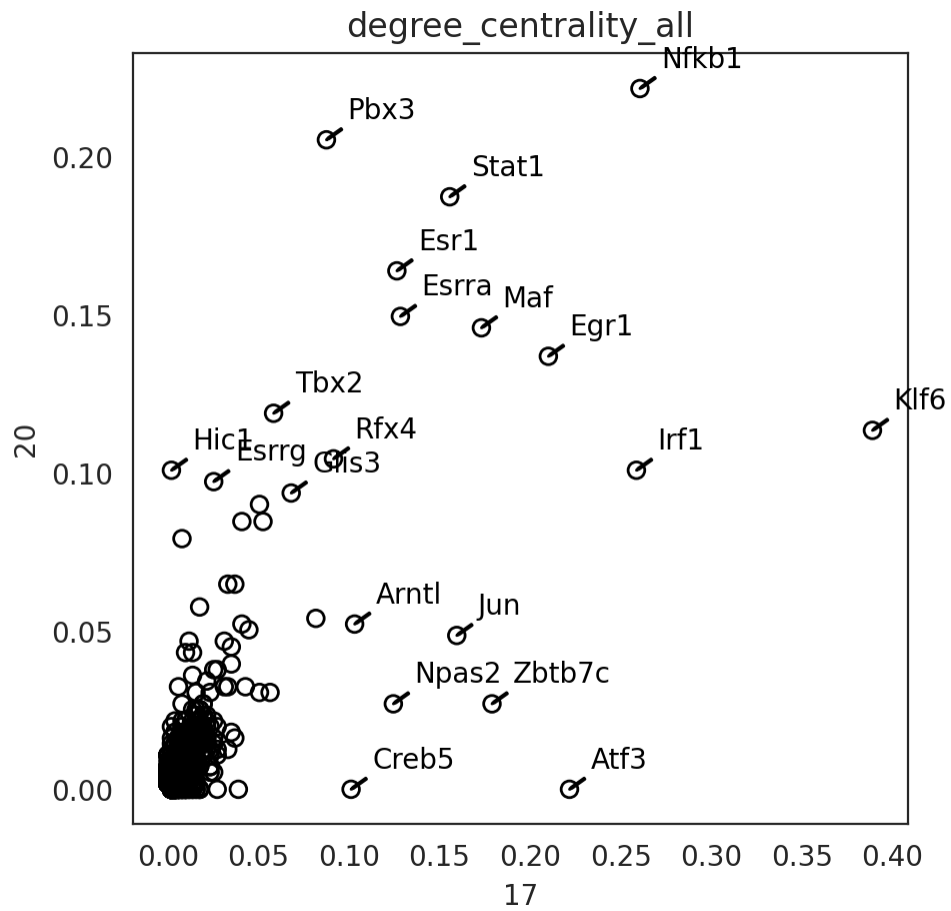
Processing clusters 17 and 18



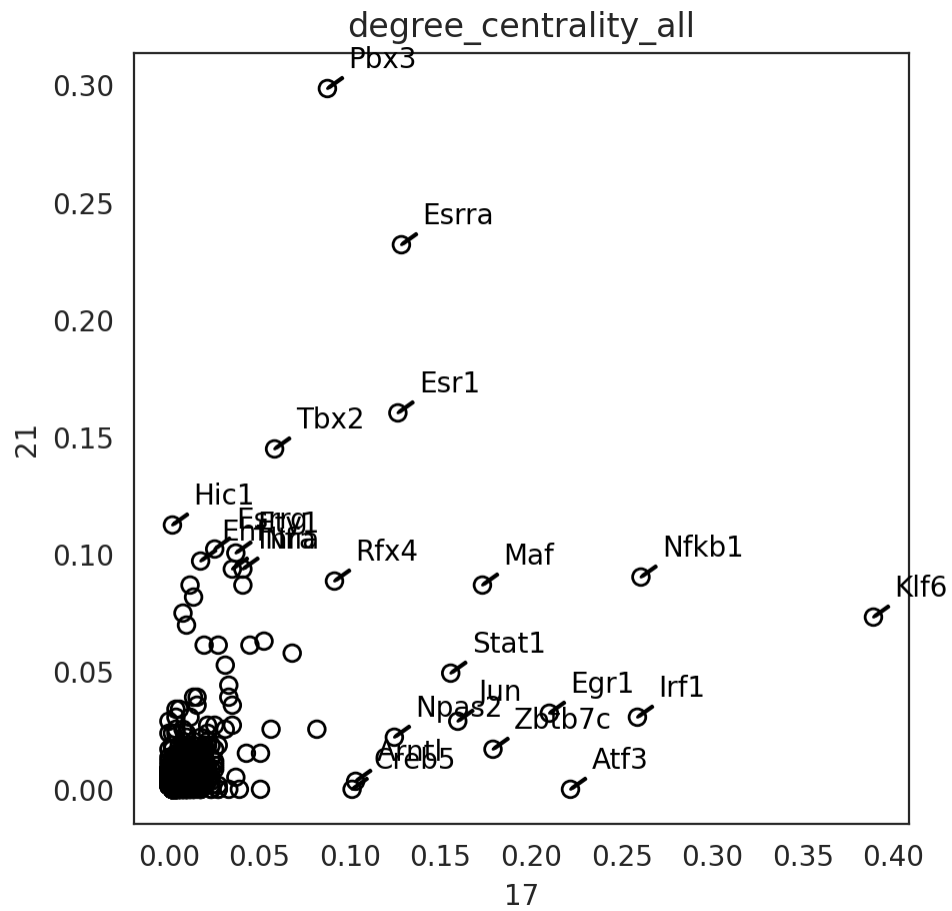
Processing clusters 17 and 19



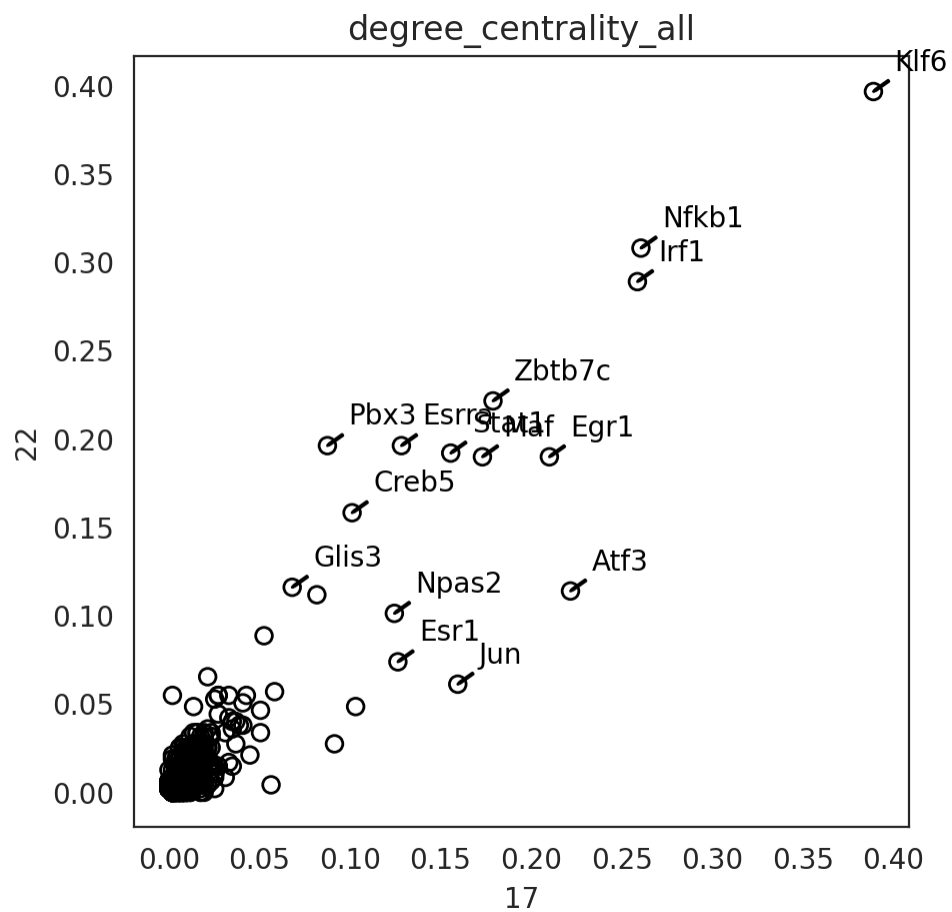
Processing clusters 17 and 20



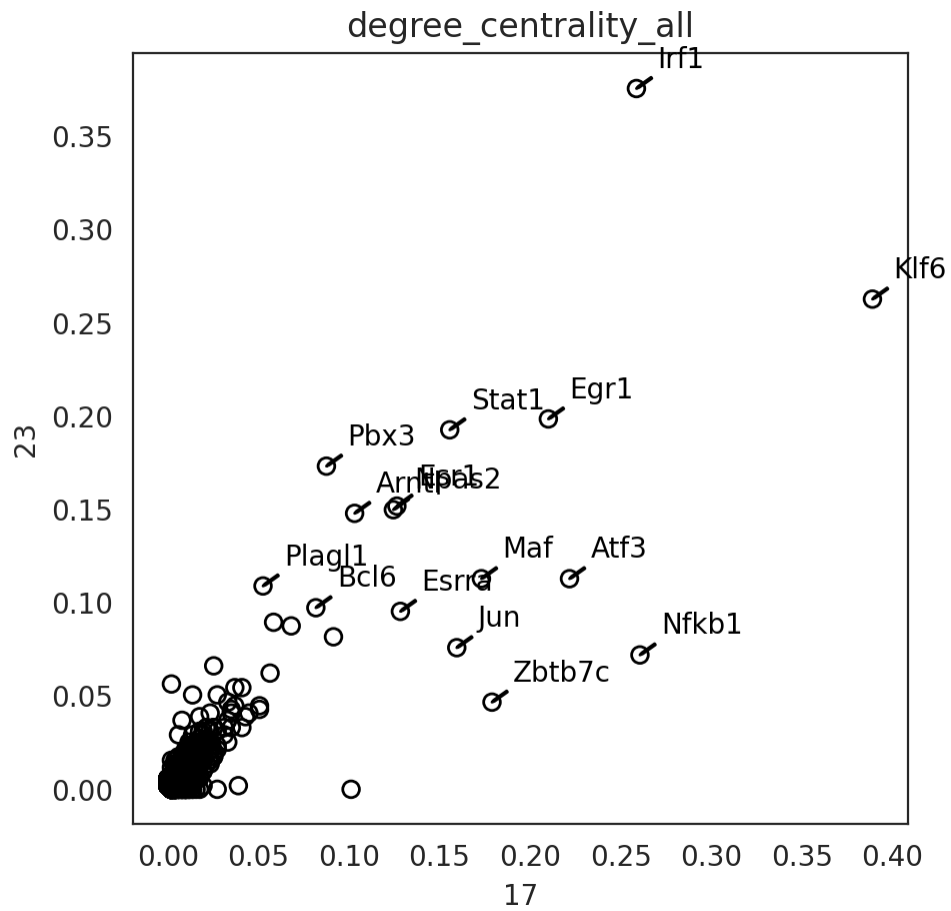
Processing clusters 17 and 21



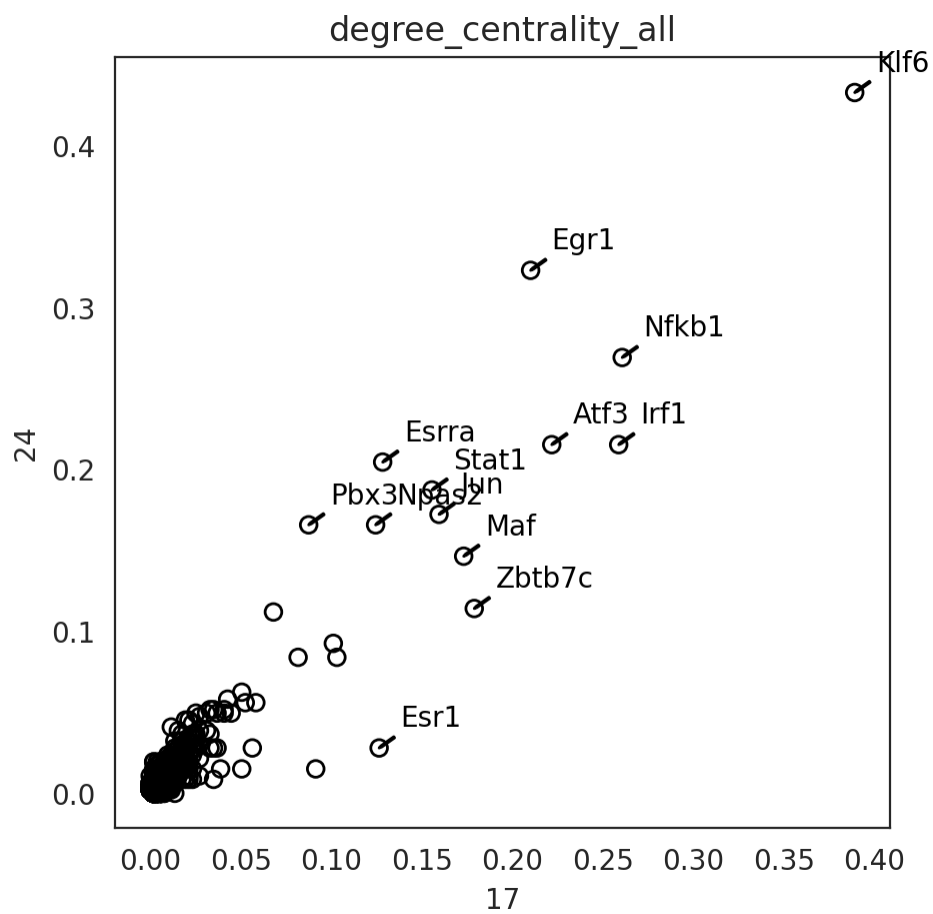
Processing clusters 17 and 22



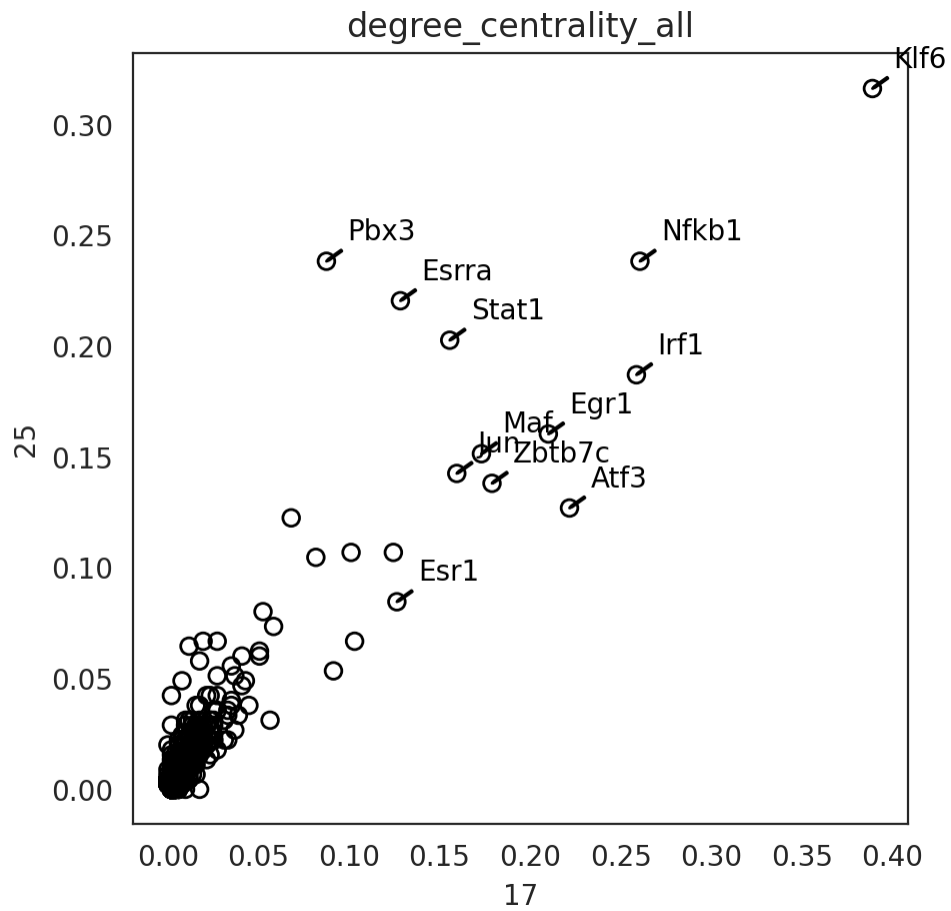
Processing clusters 17 and 23



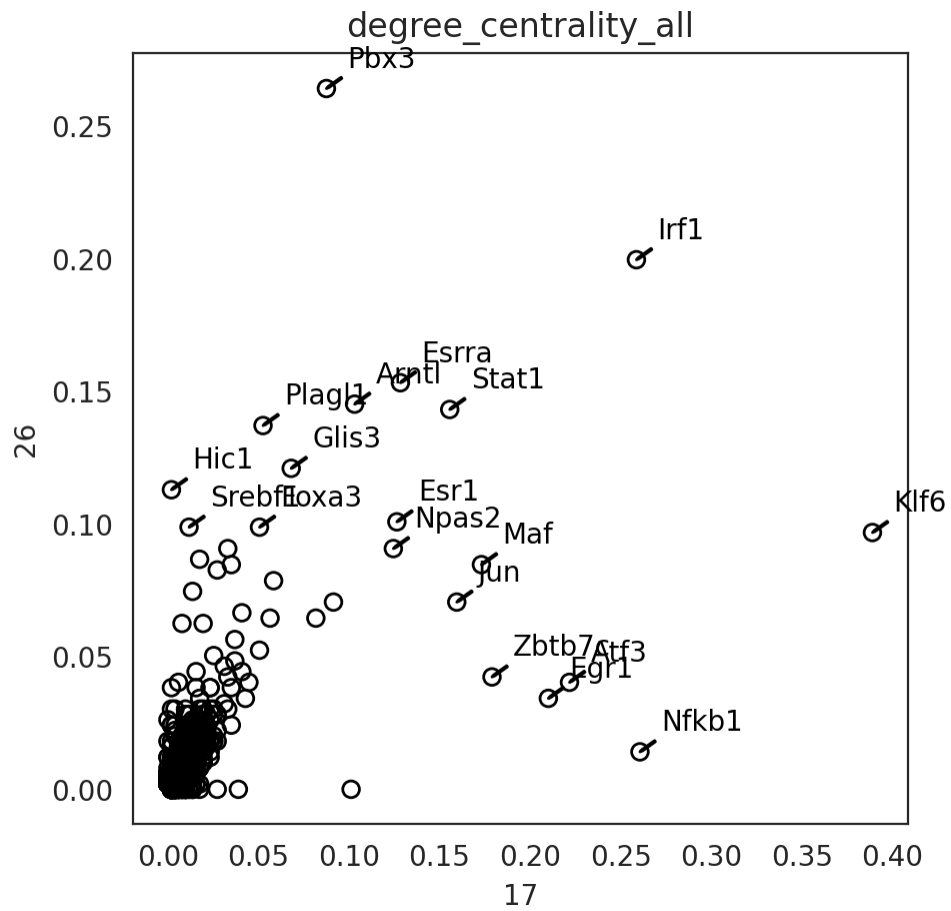
Processing clusters 17 and 24



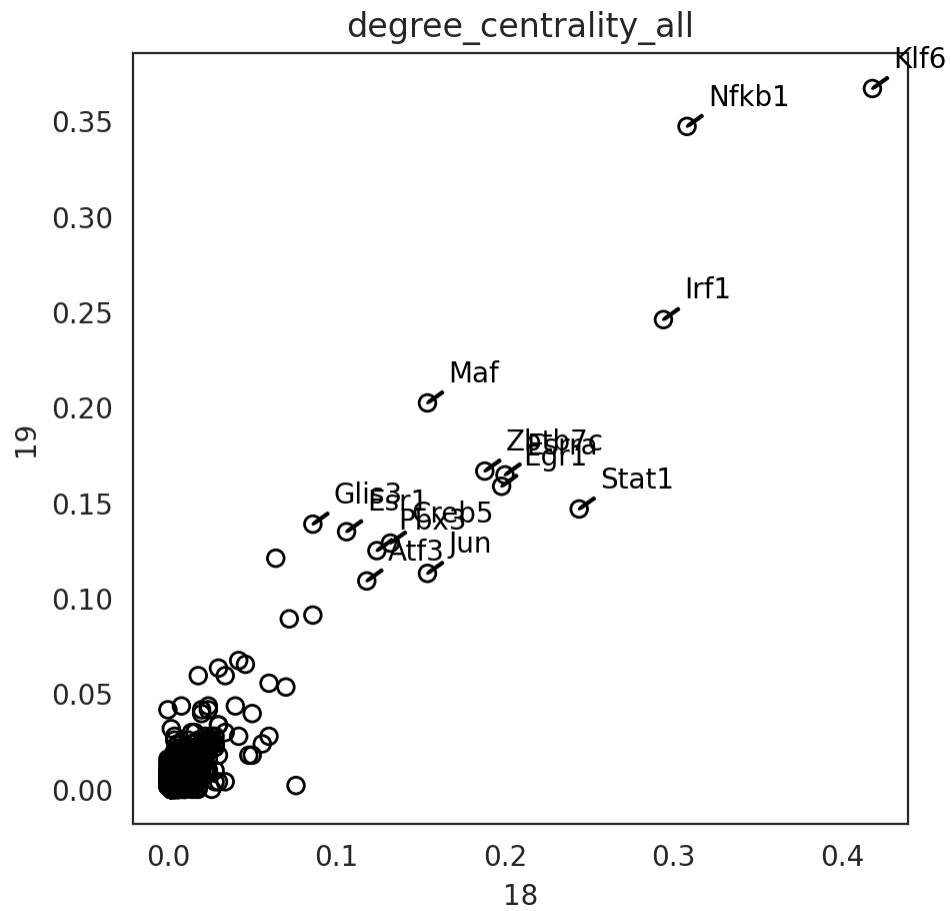
Processing clusters 17 and 25



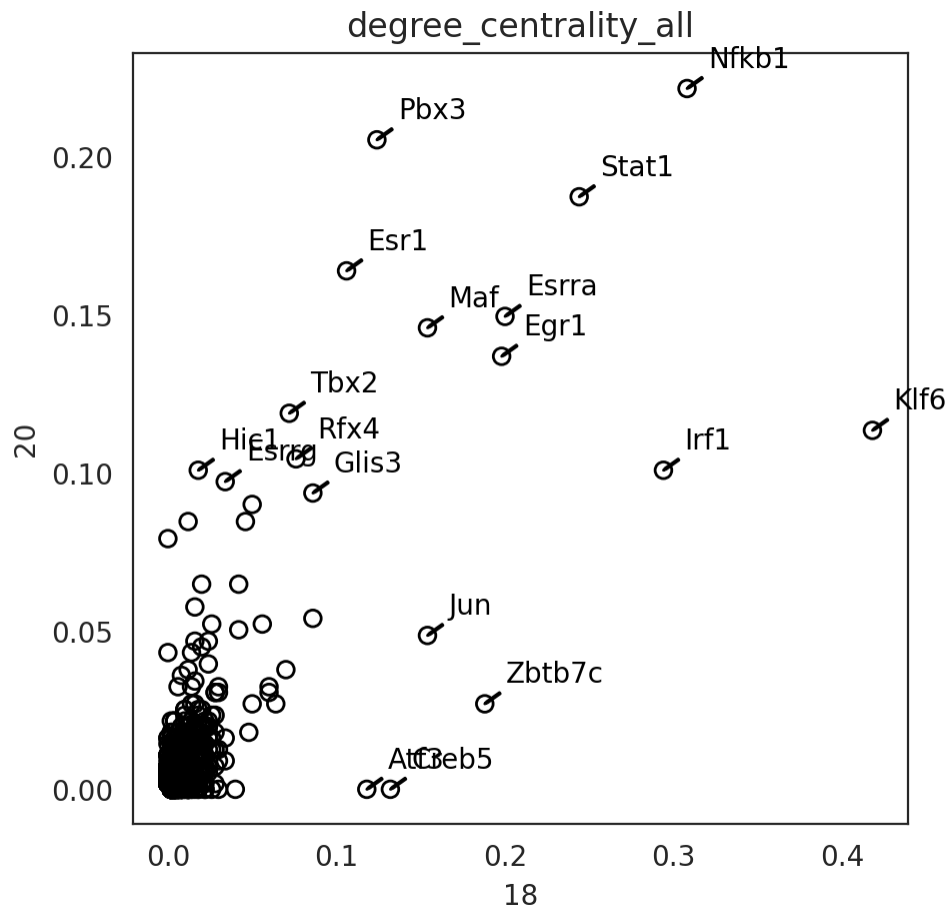
Processing clusters 17 and 26



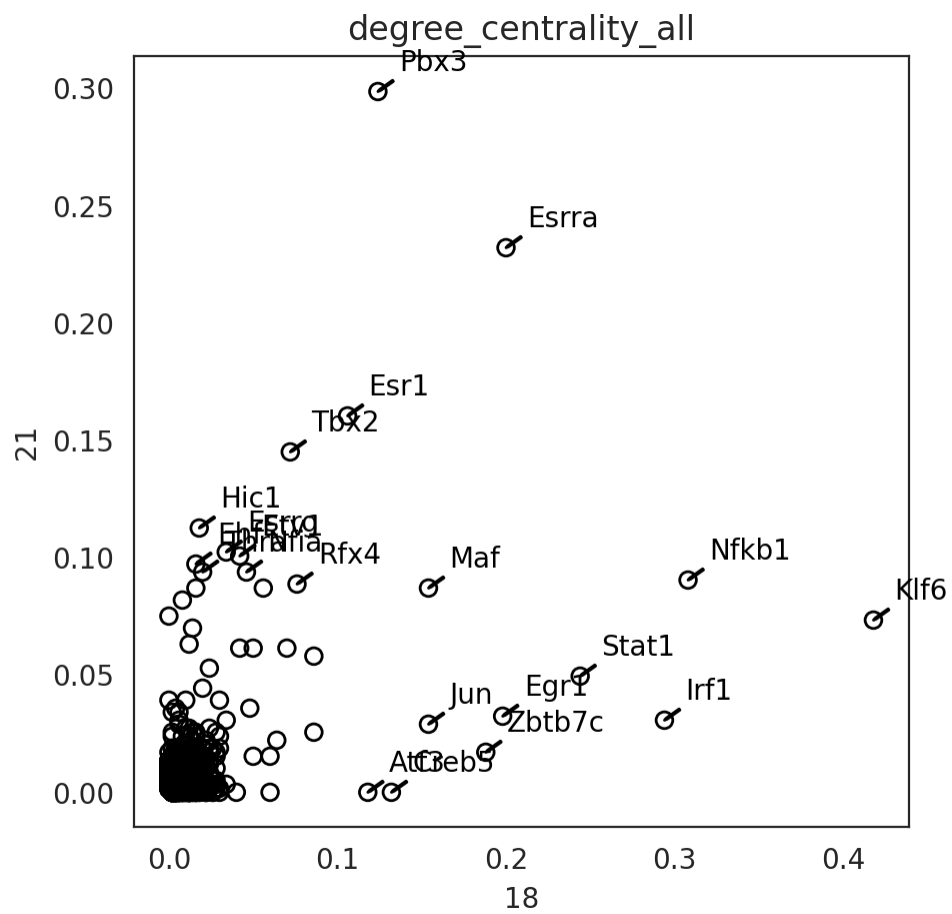
Processing clusters 18 and 19



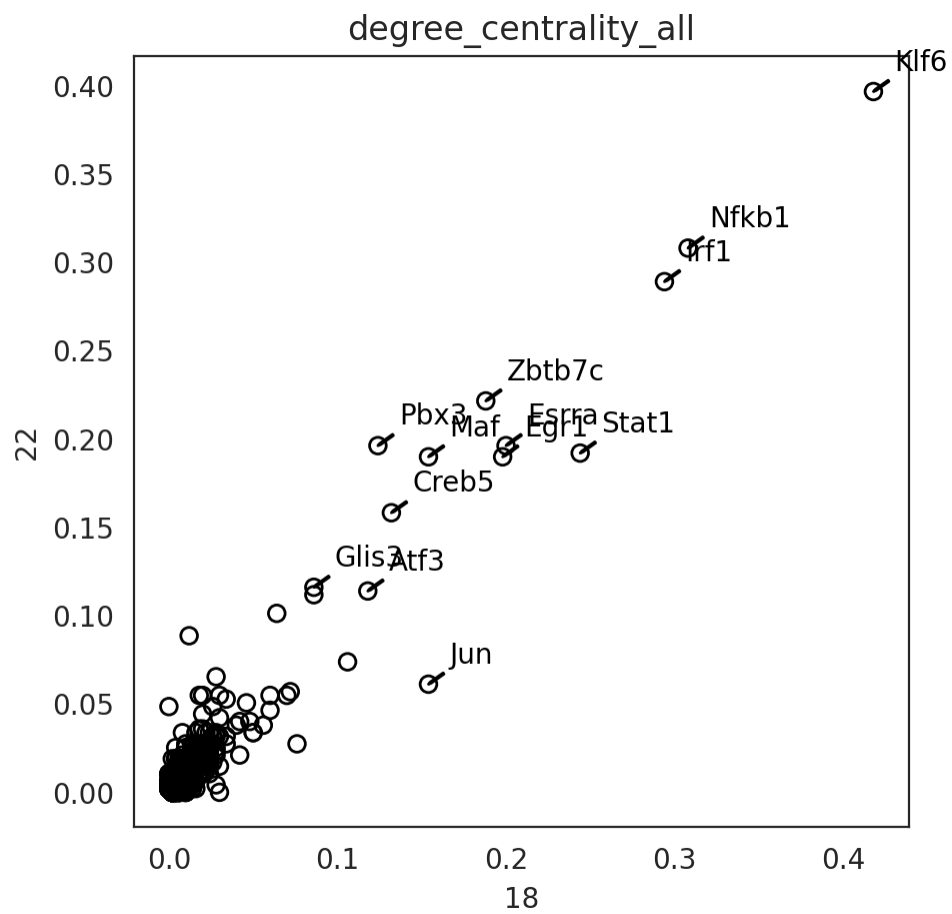
Processing clusters 18 and 20



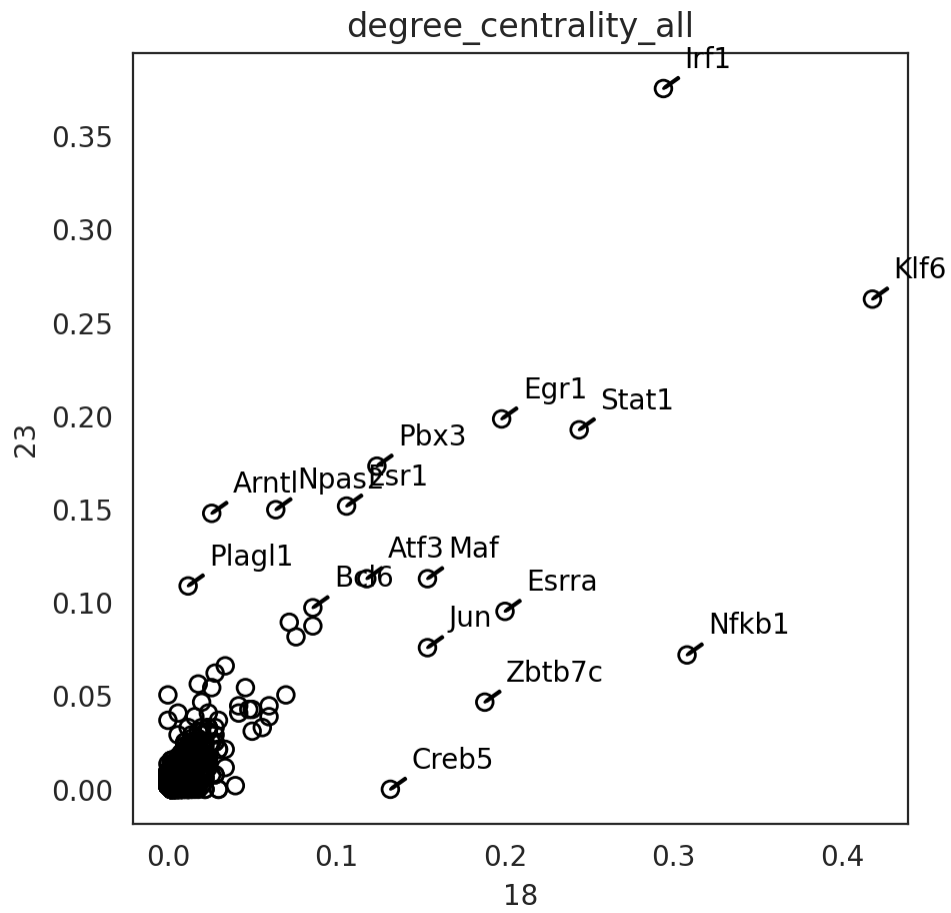
Processing clusters 18 and 21



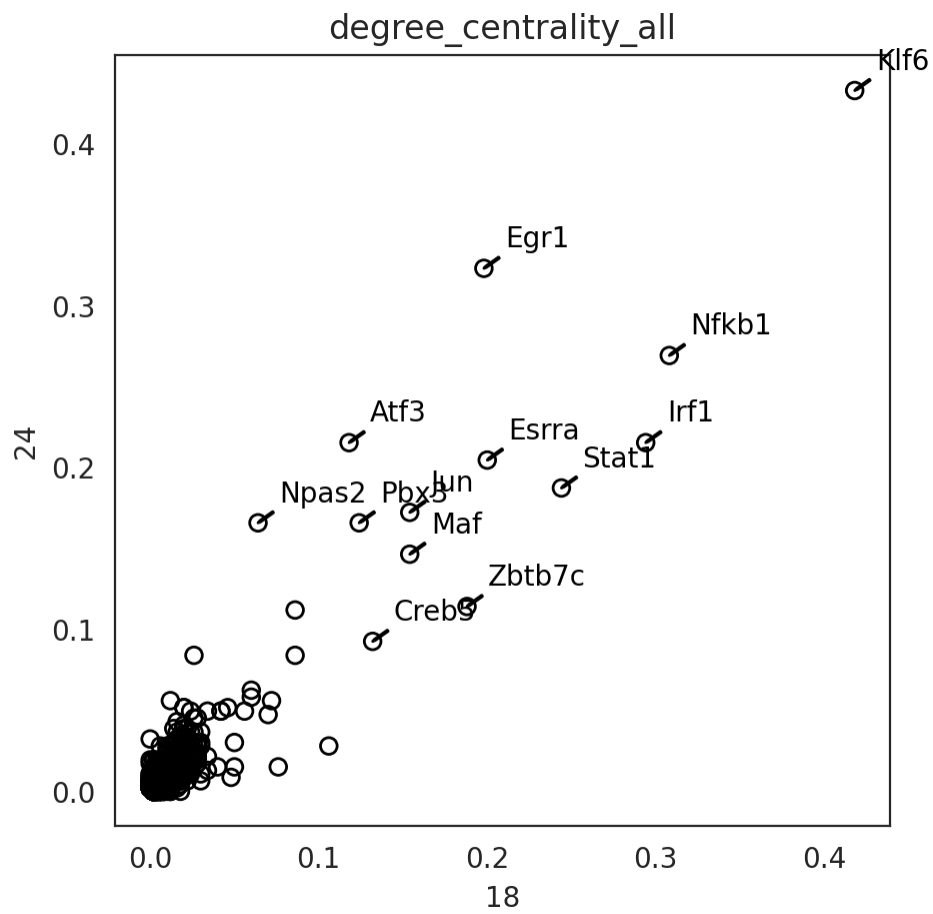
Processing clusters 18 and 22



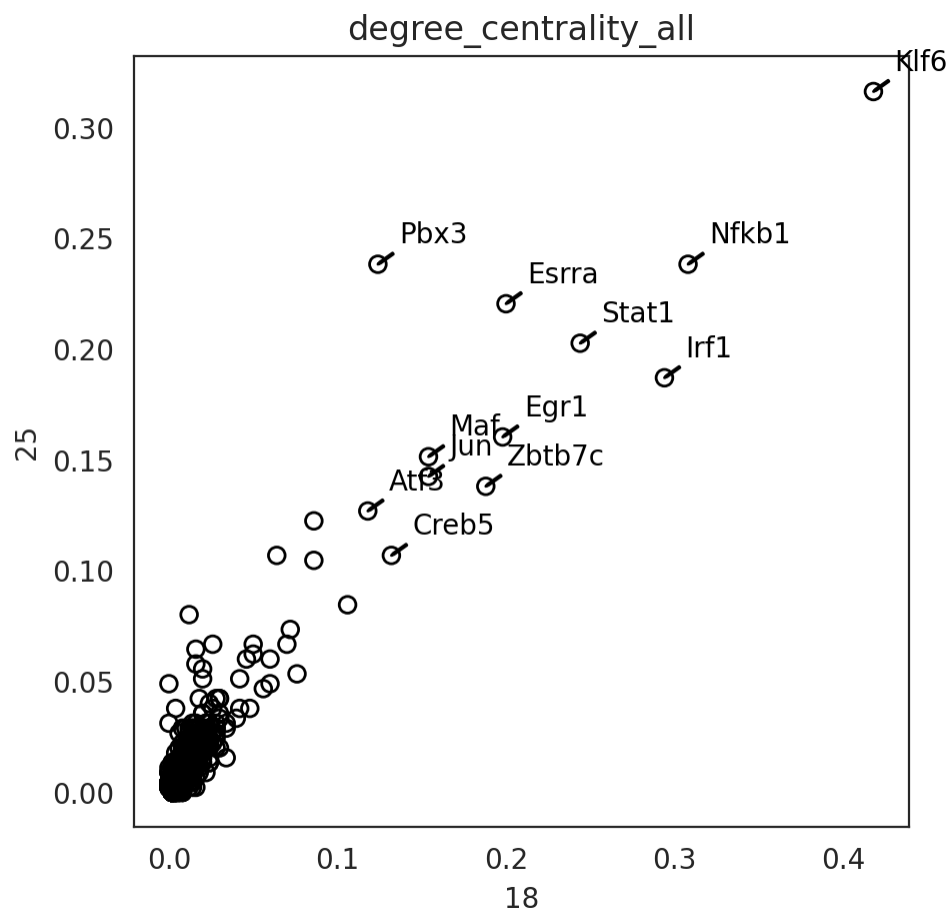
Processing clusters 18 and 23



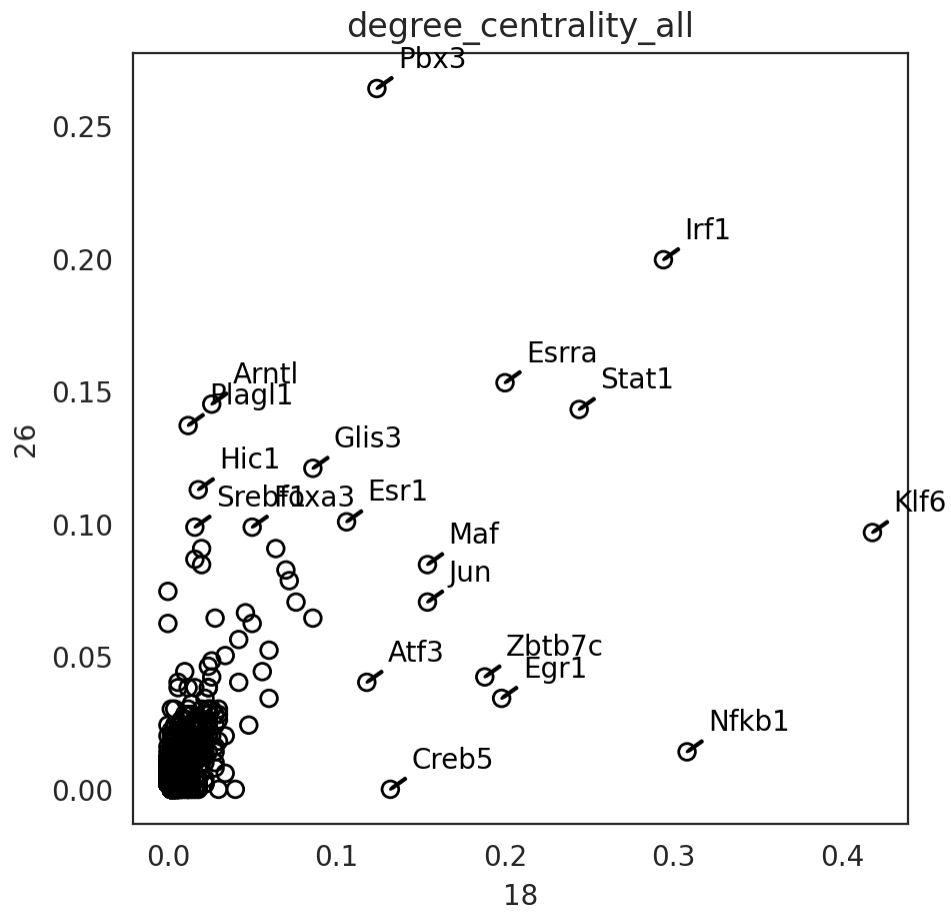
Processing clusters 18 and 24



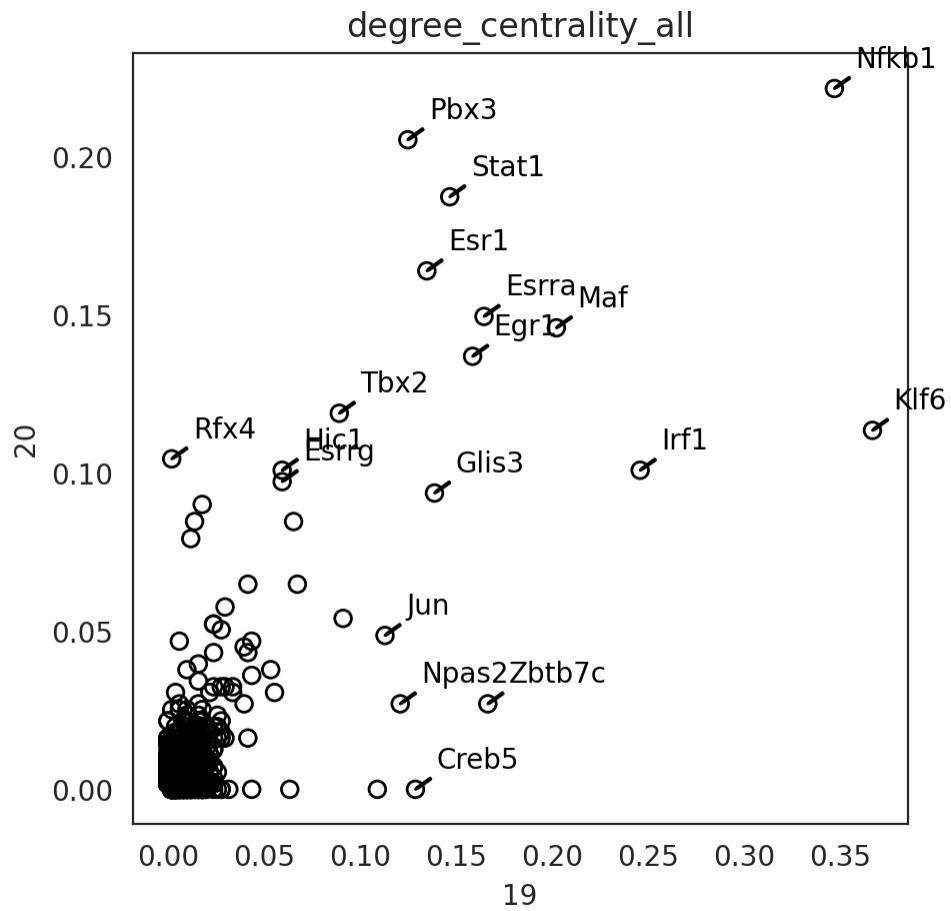
Processing clusters 18 and 25



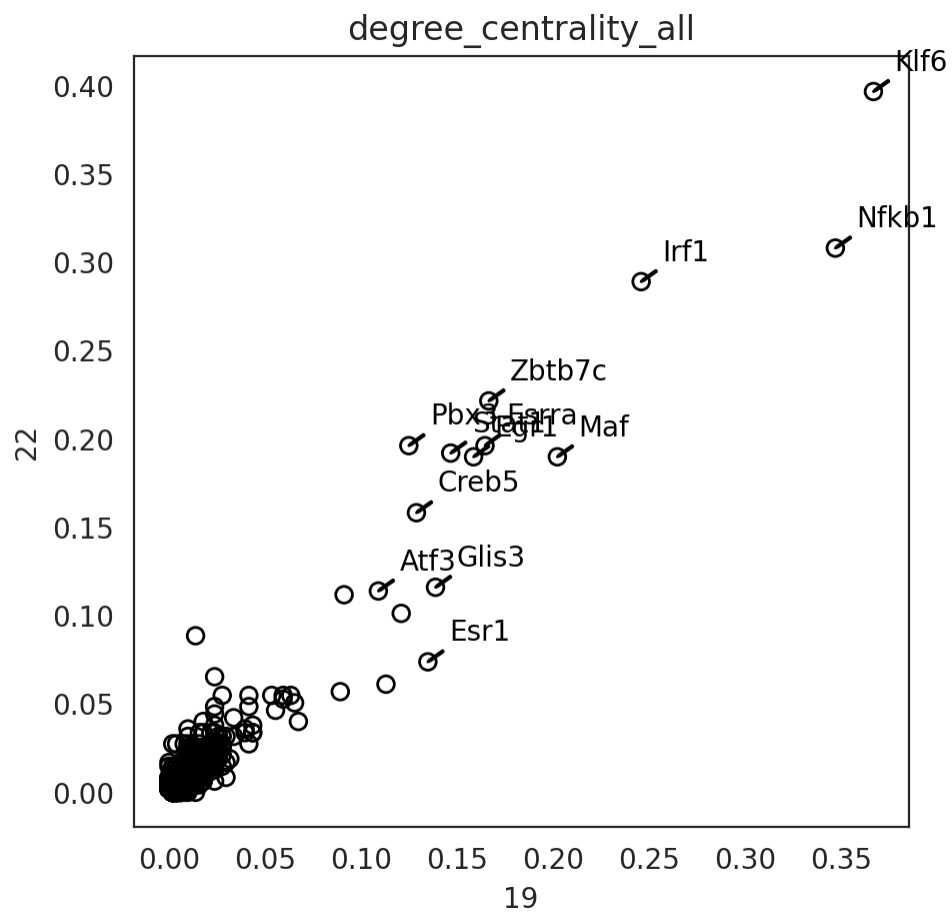
Processing clusters 18 and 26

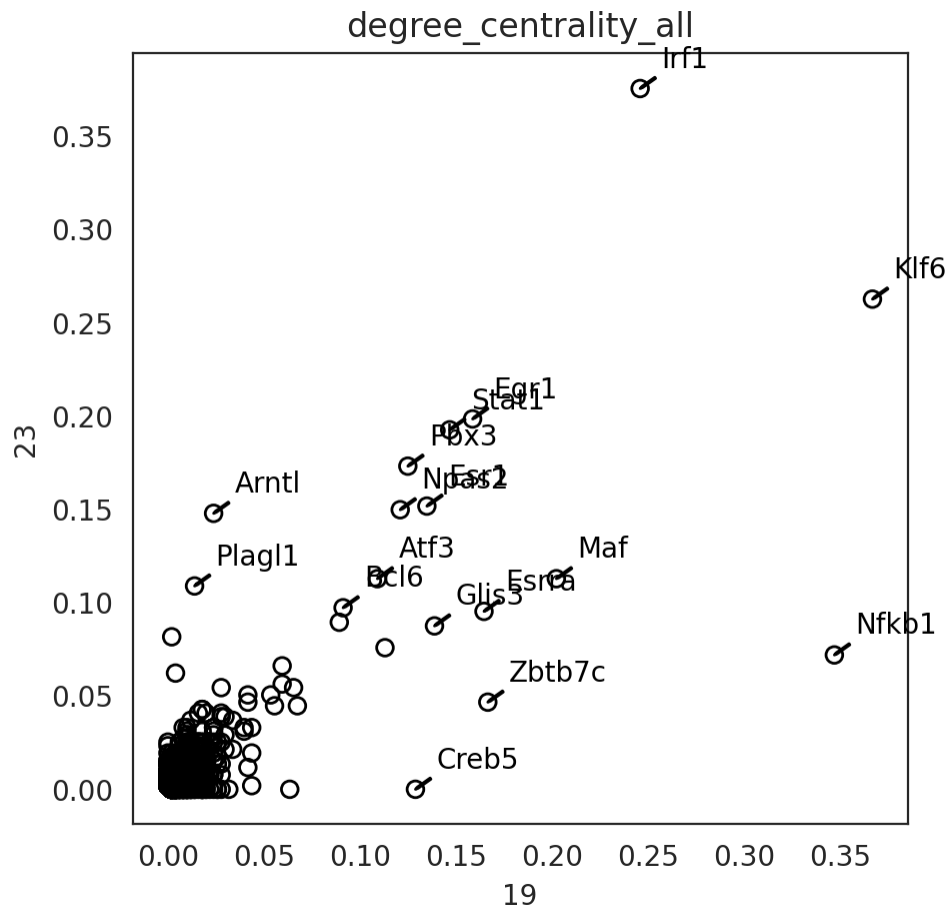


Processing clusters 19 and 20

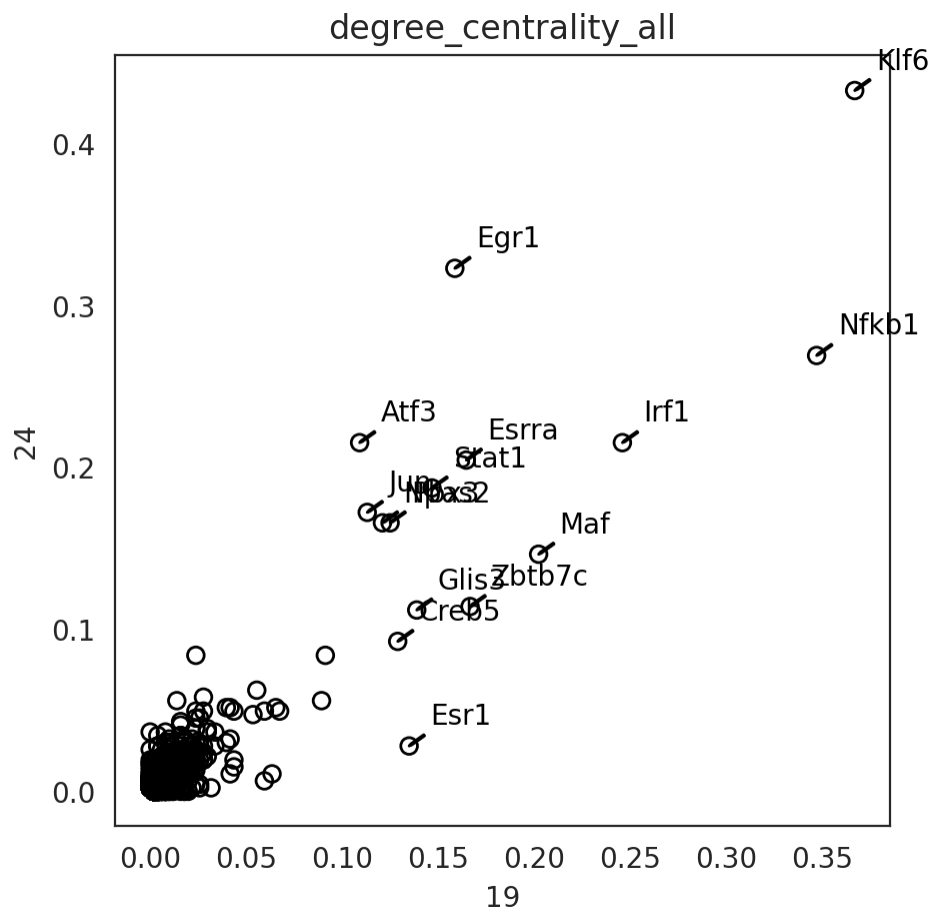


Processing clusters 19 and 21

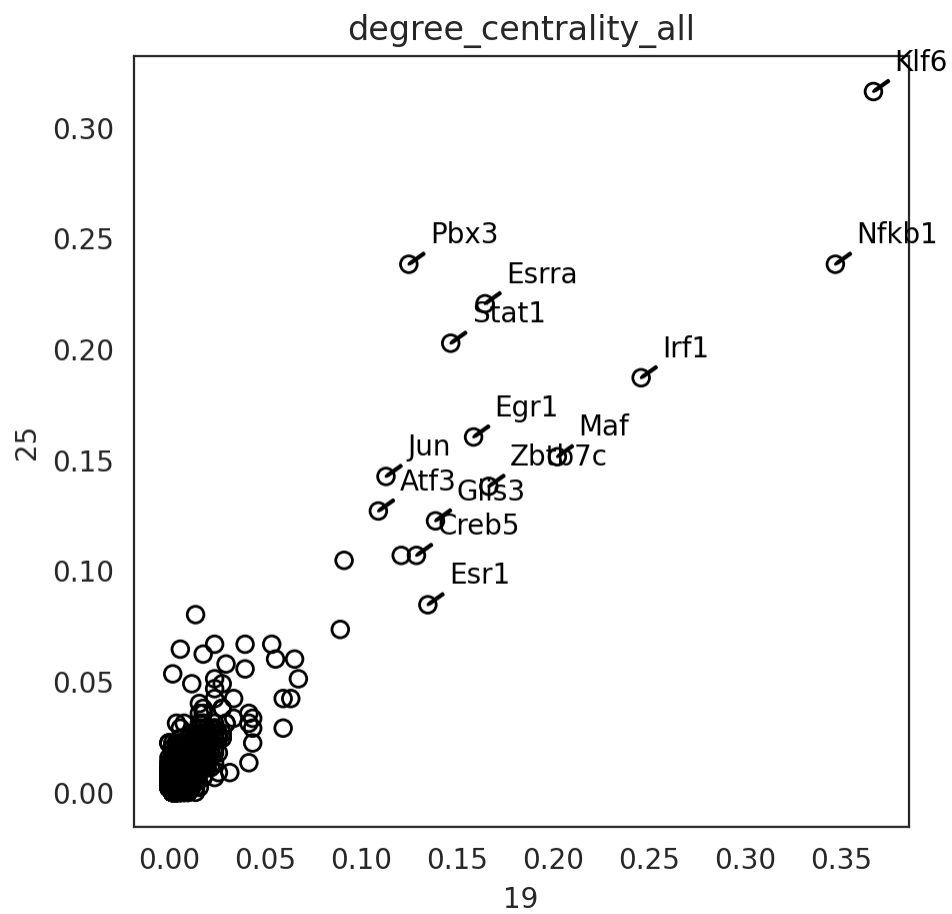




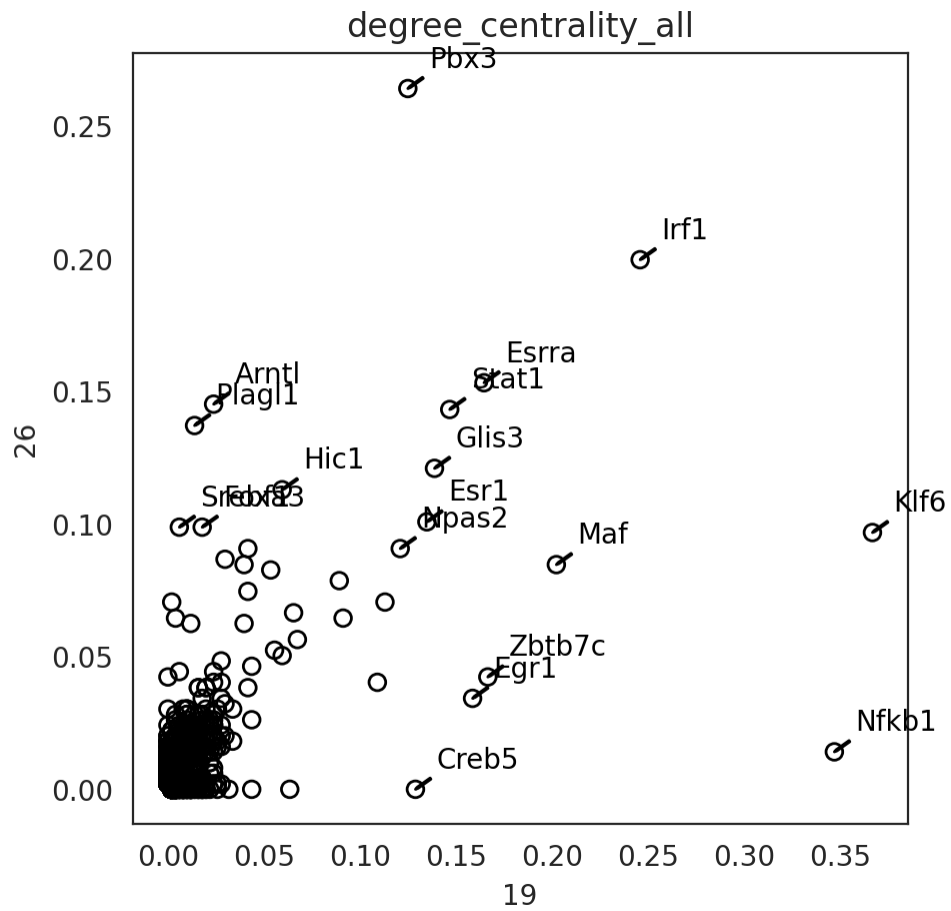
Processing clusters 19 and 24



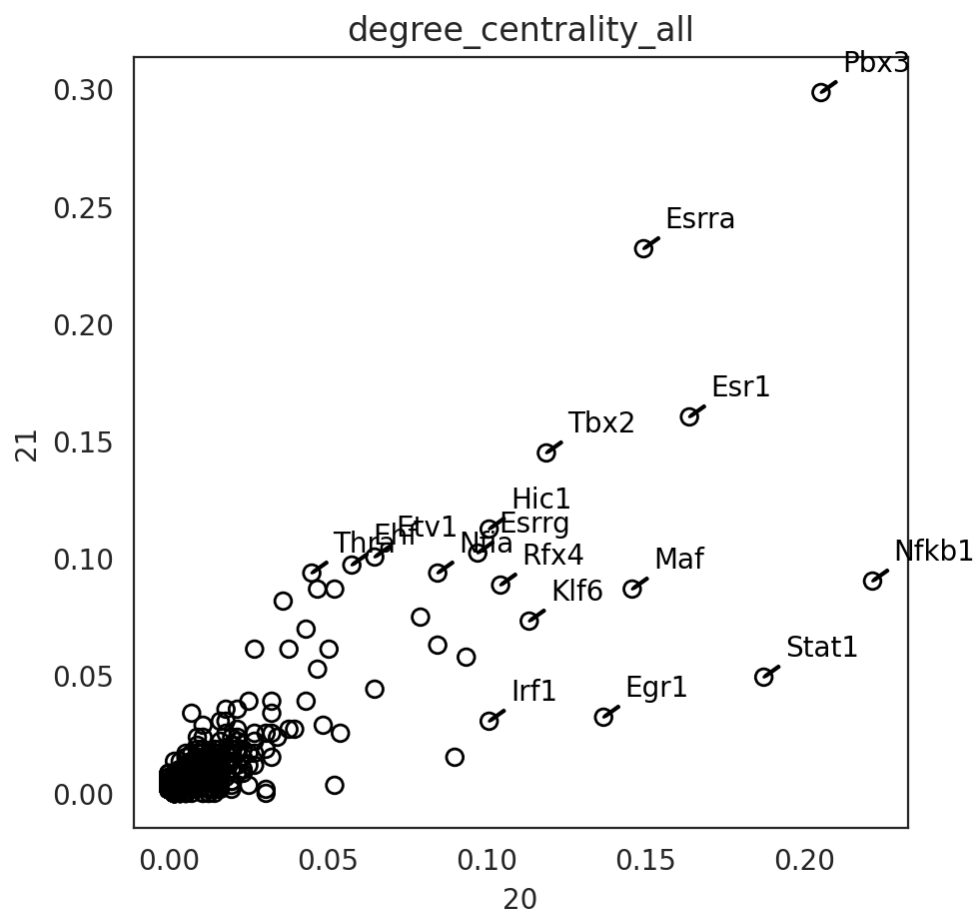
Processing clusters 19 and 25



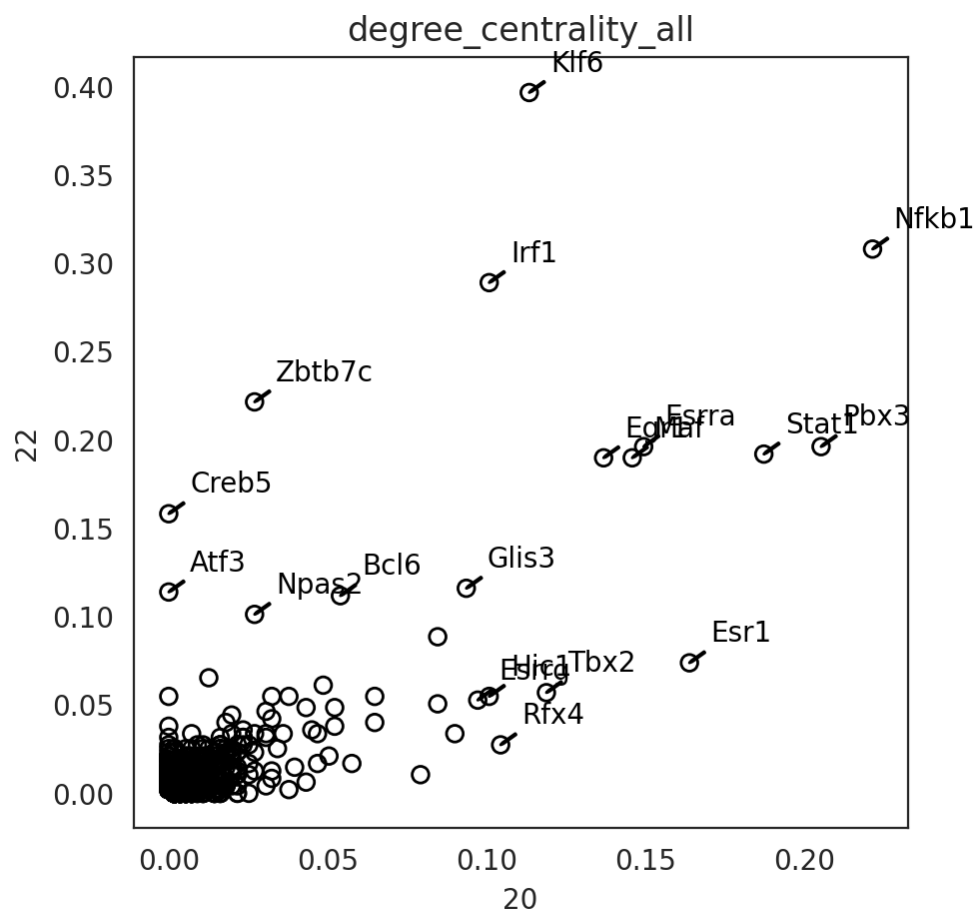
Processing clusters 19 and 26



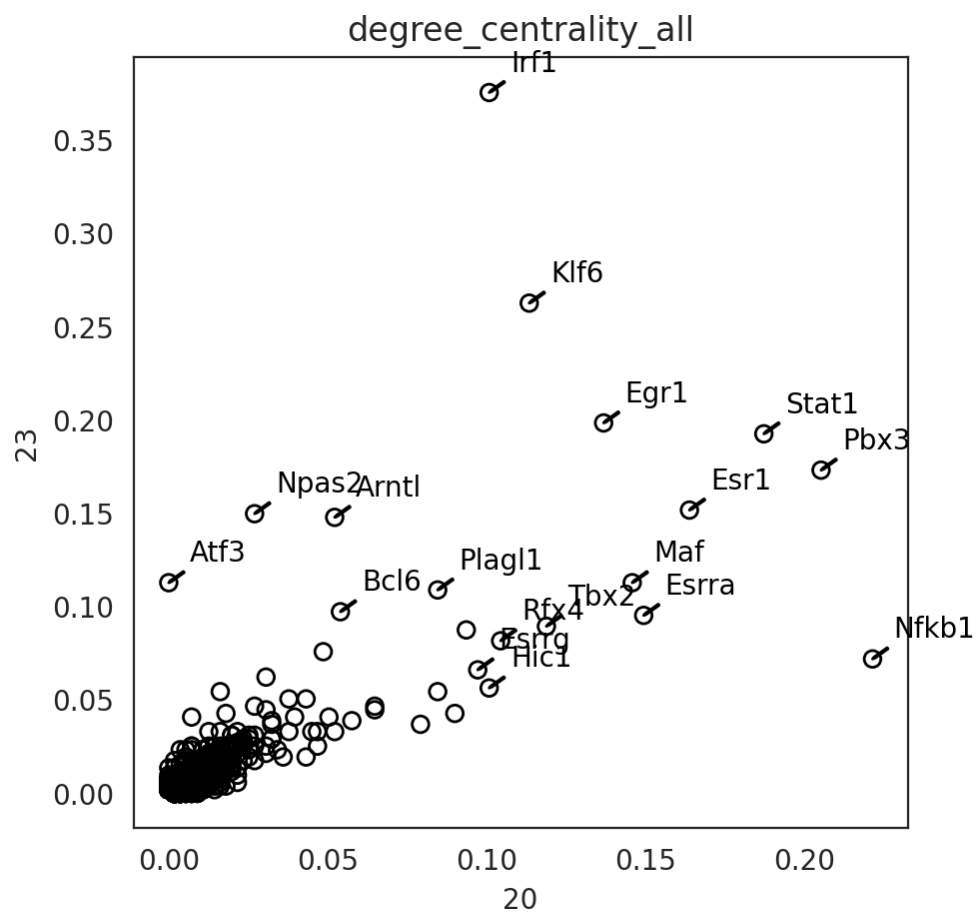
Processing clusters 20 and 21



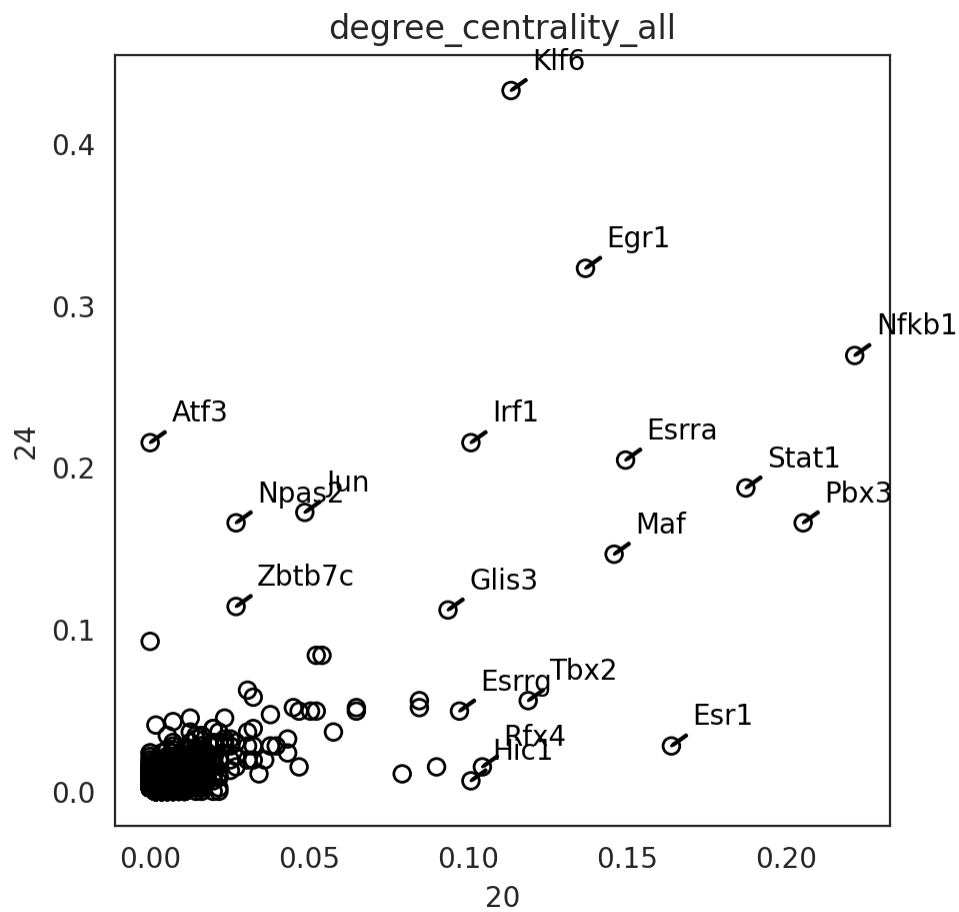
Processing clusters 20 and 22



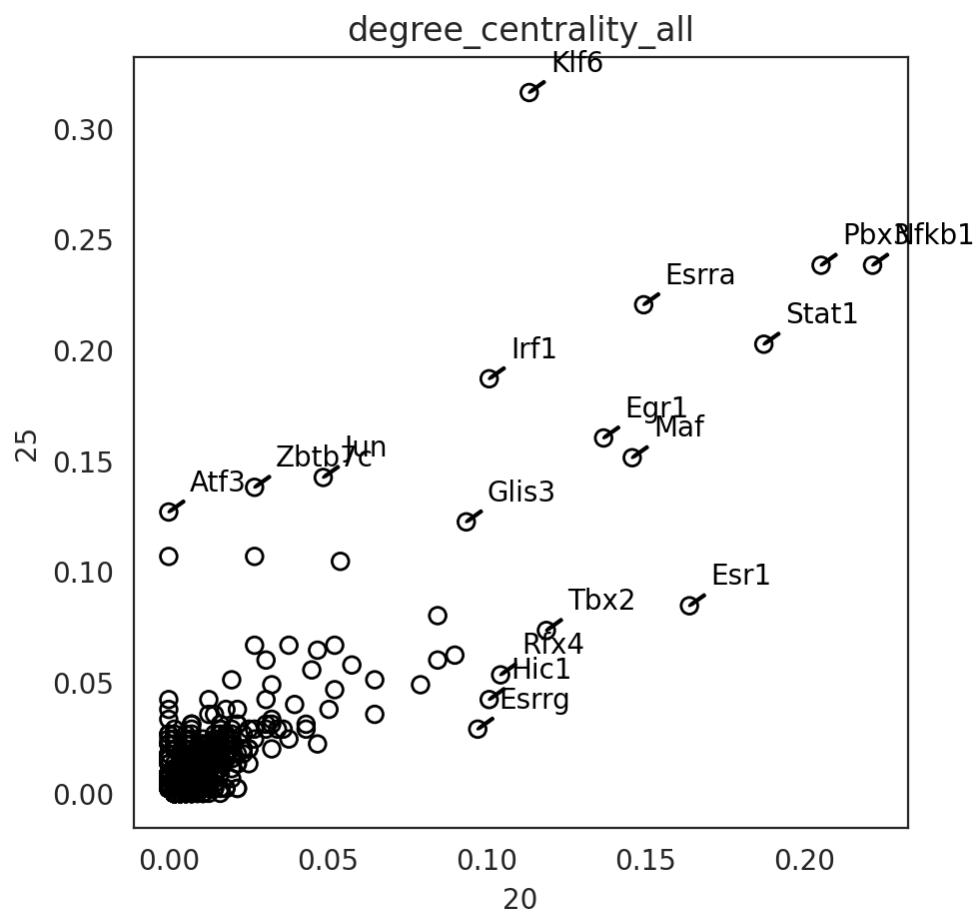
Processing clusters 20 and 23



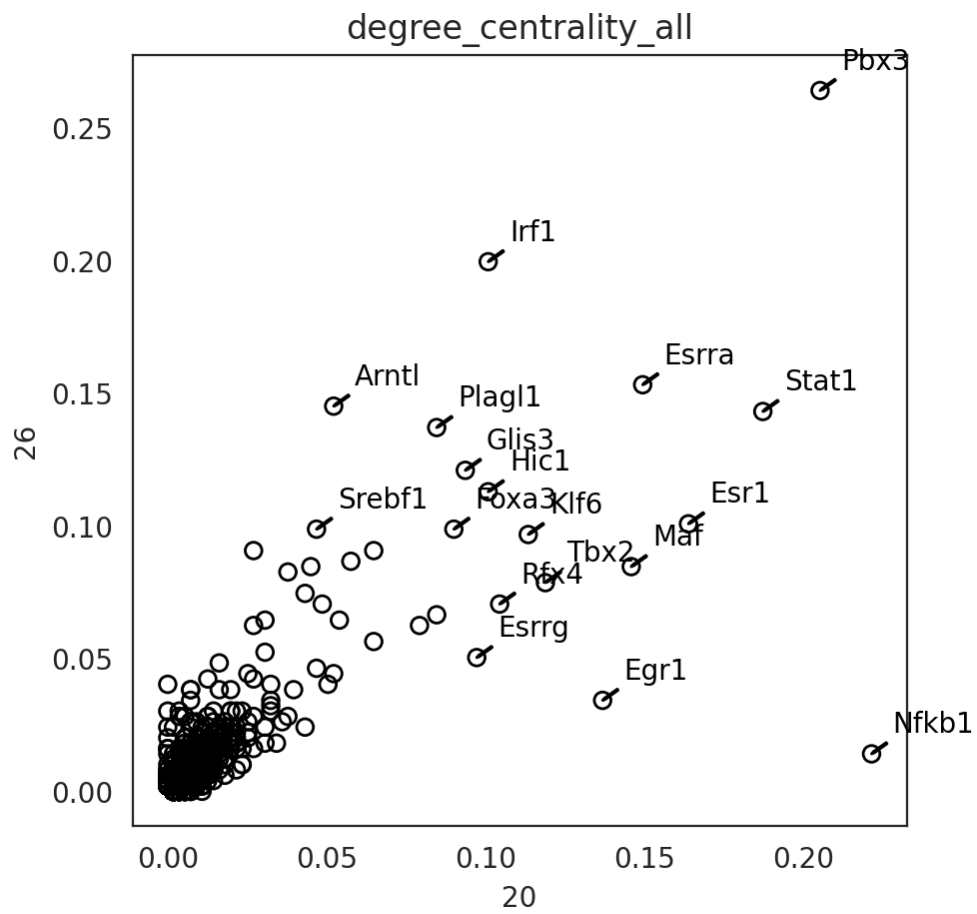
Processing clusters 20 and 24



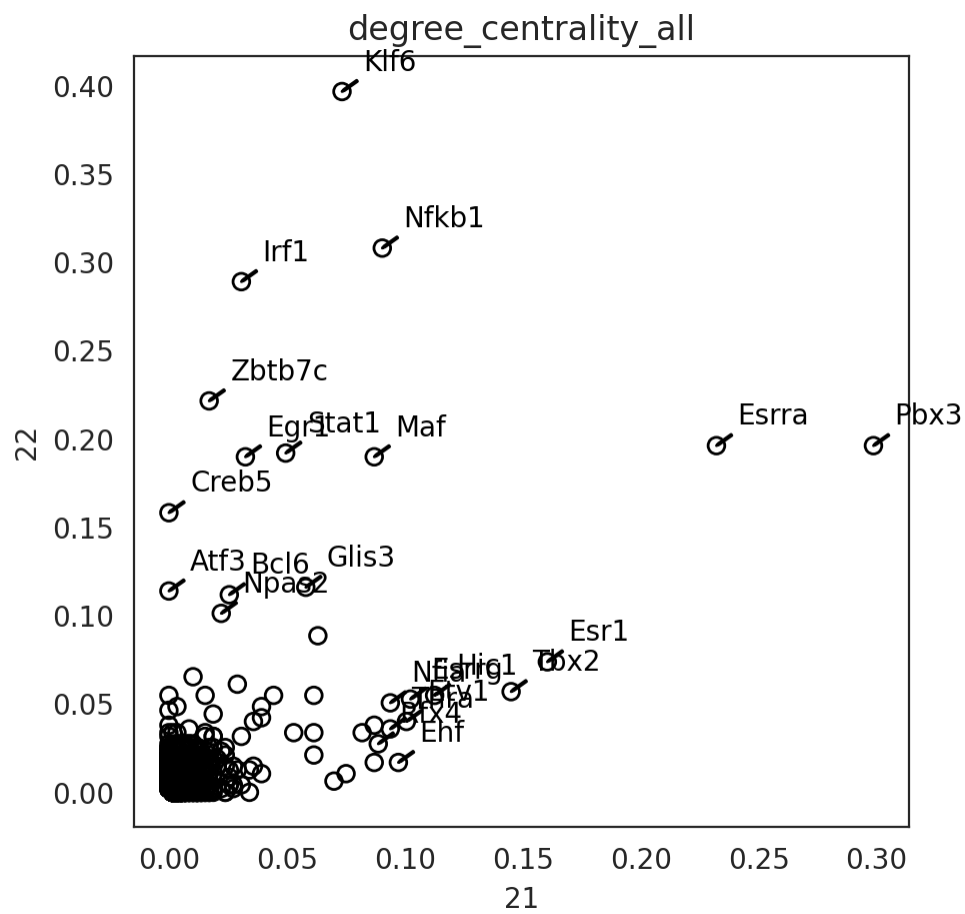
Processing clusters 20 and 25

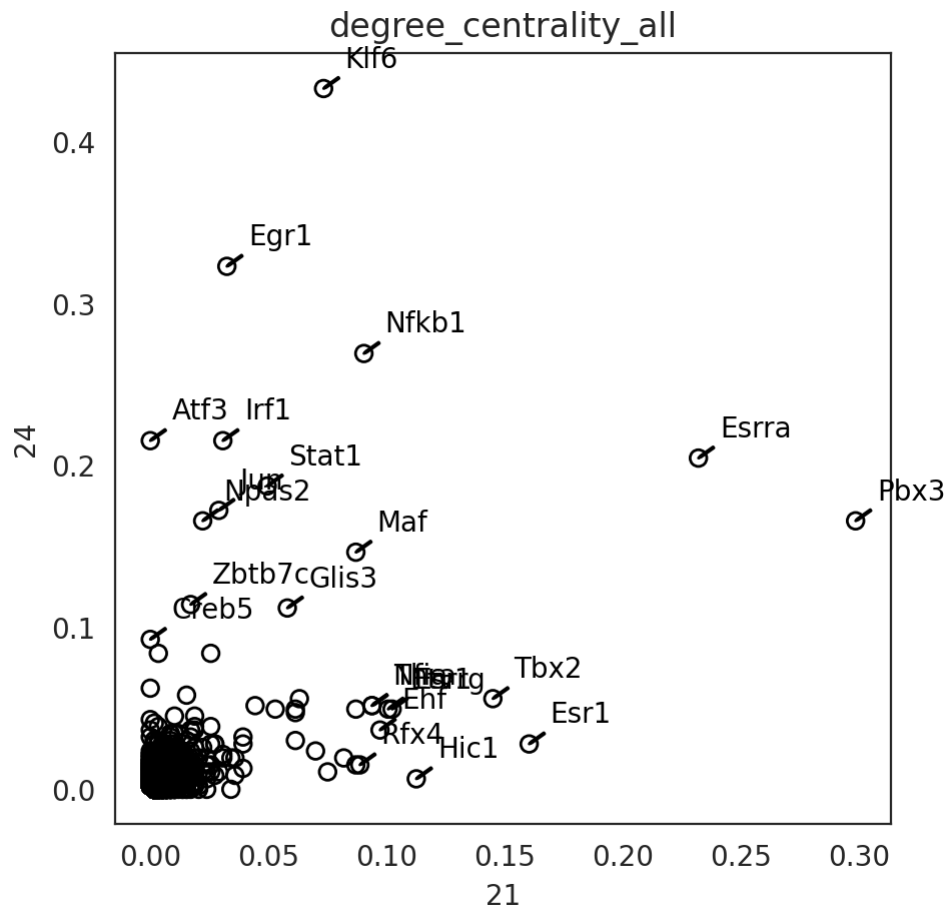


Processing clusters 20 and 26

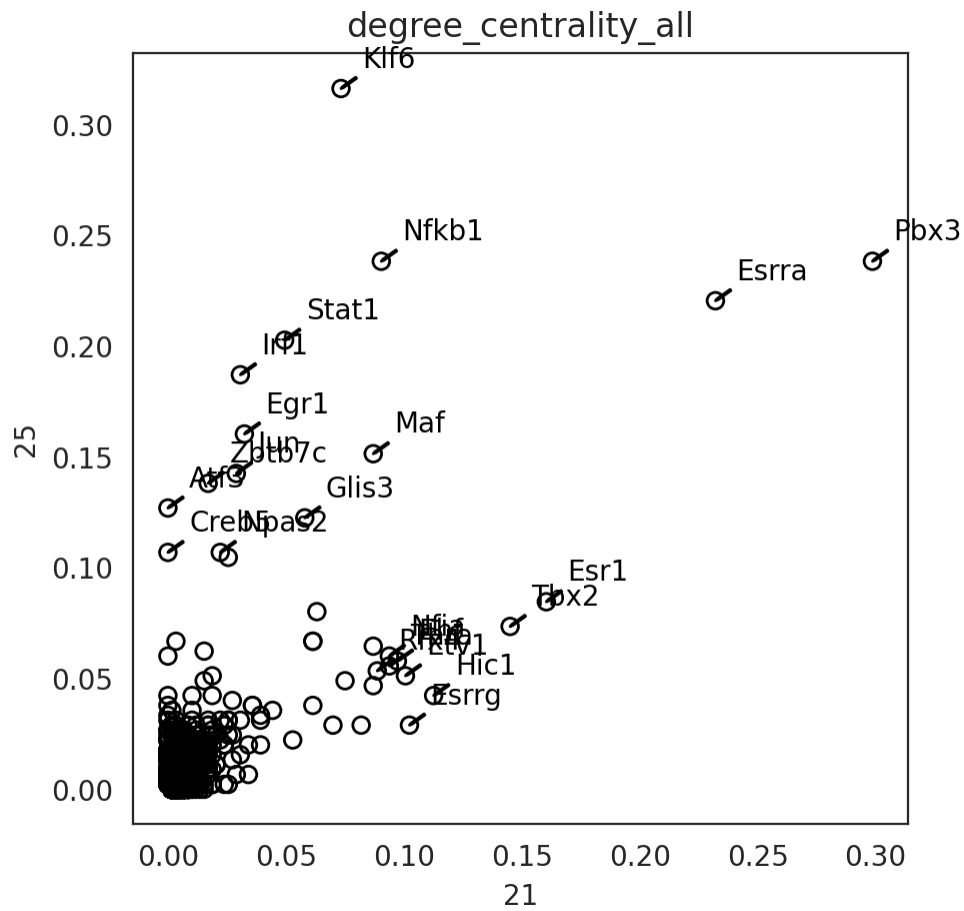


Processing clusters 21 and 22

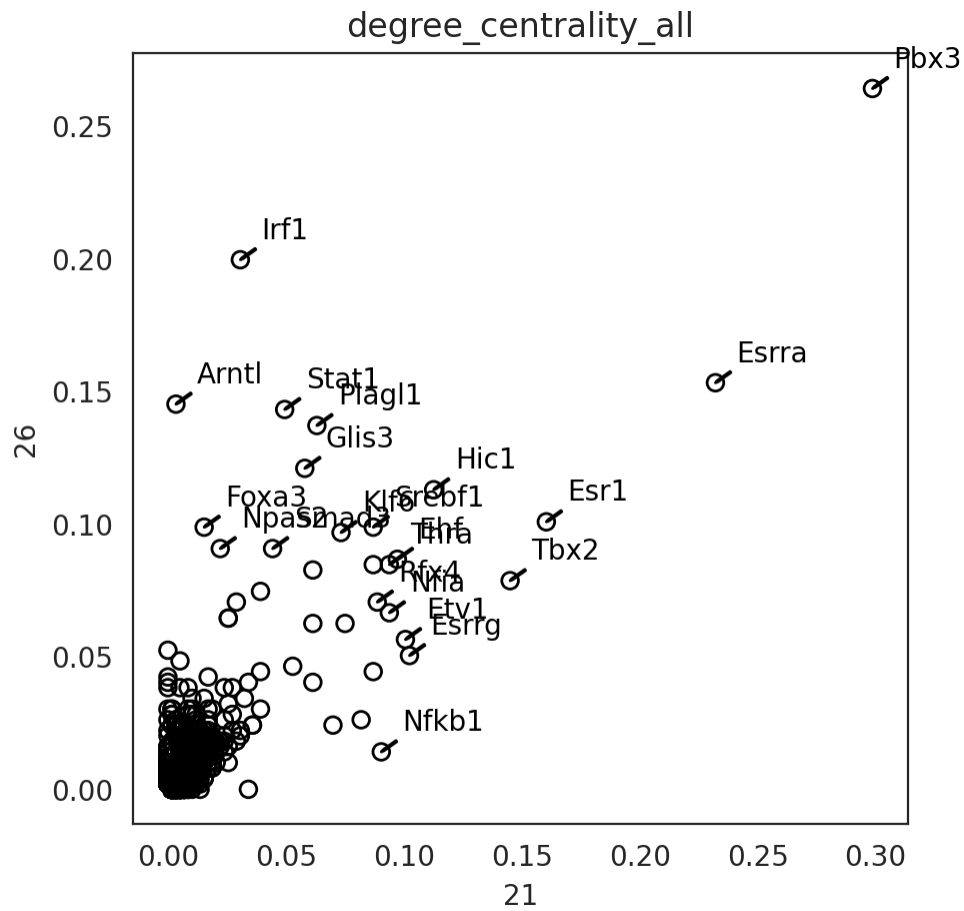




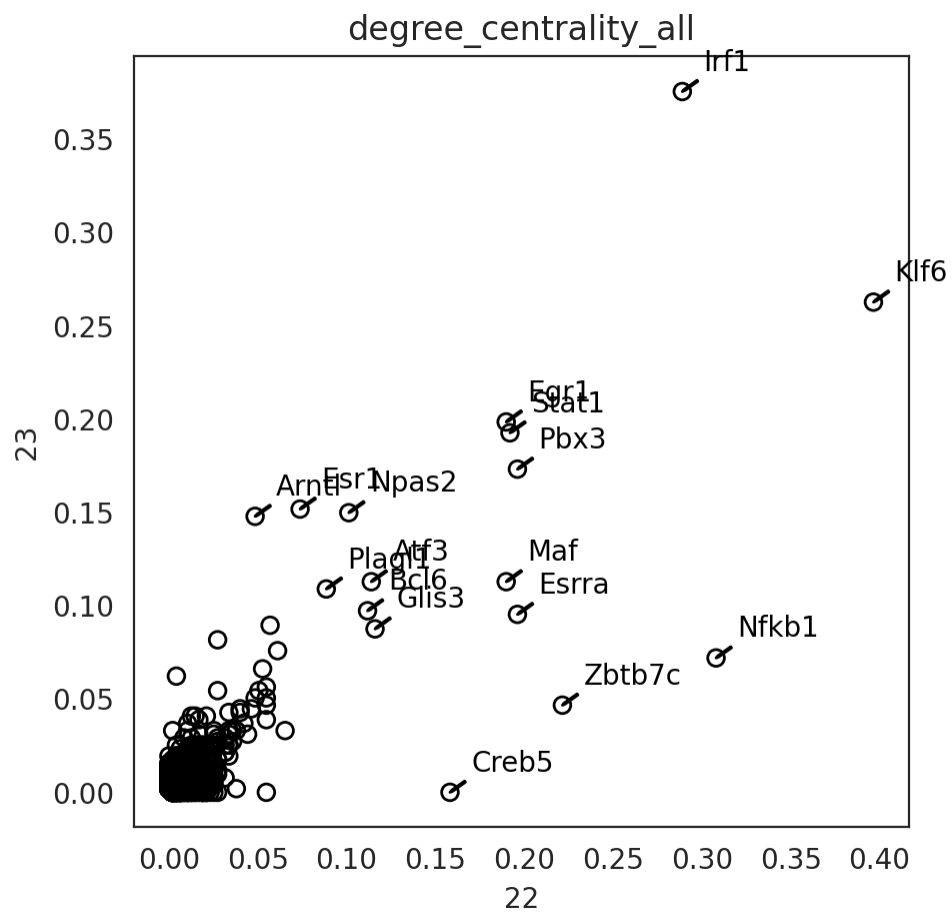
Processing clusters 21 and 25



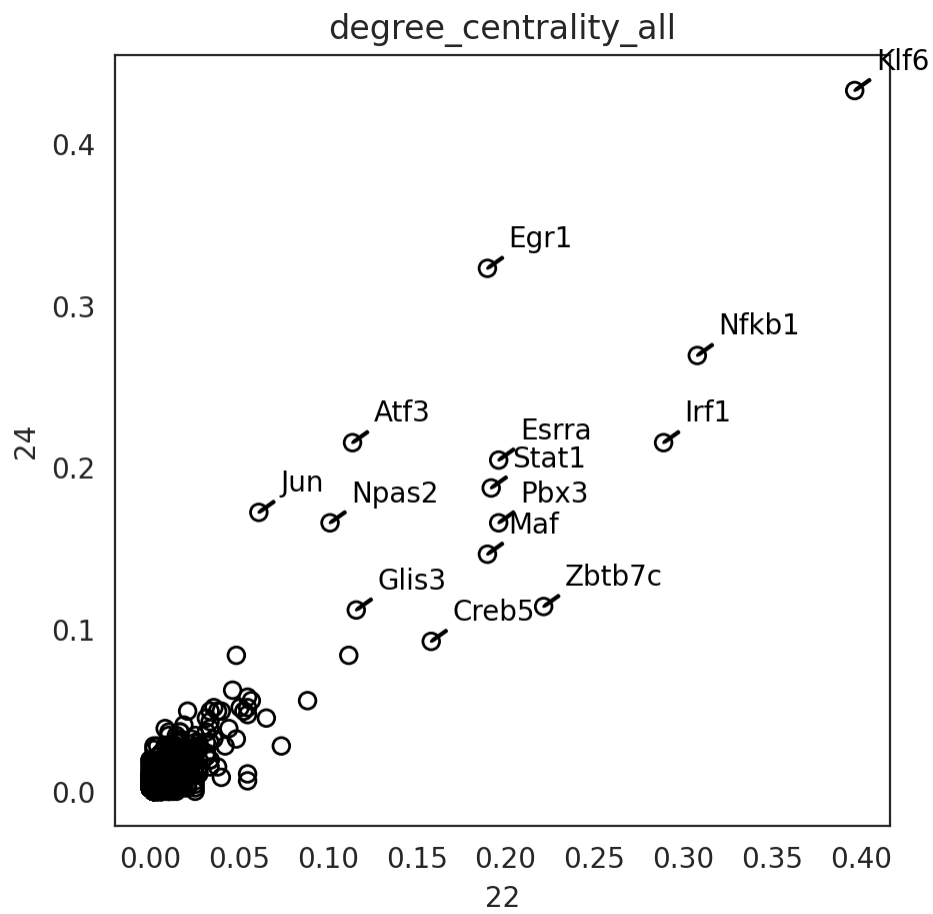
Processing clusters 21 and 26



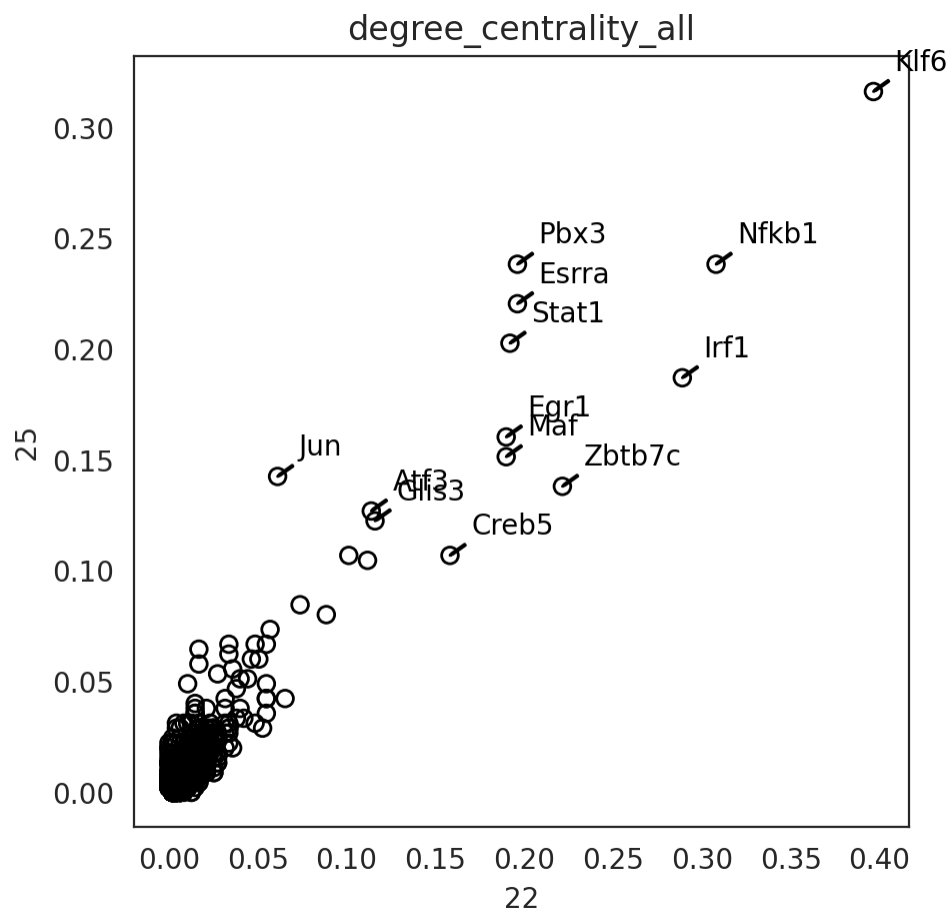
Processing clusters 22 and 23



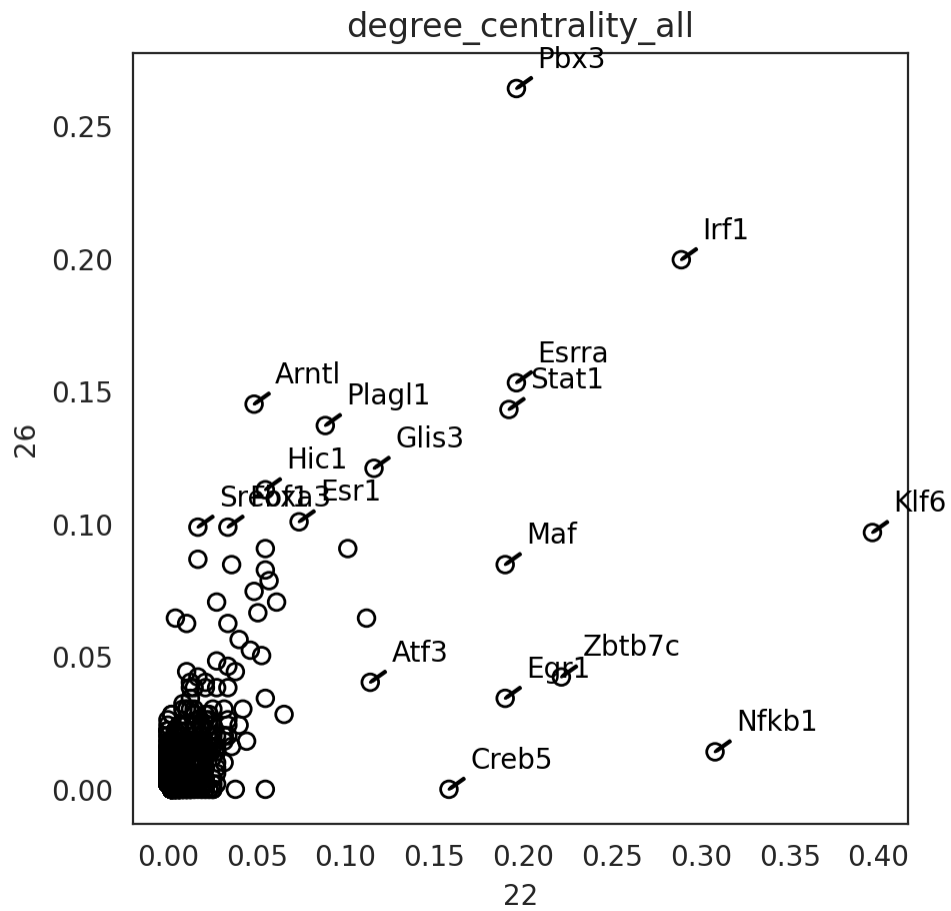
Processing clusters 22 and 24



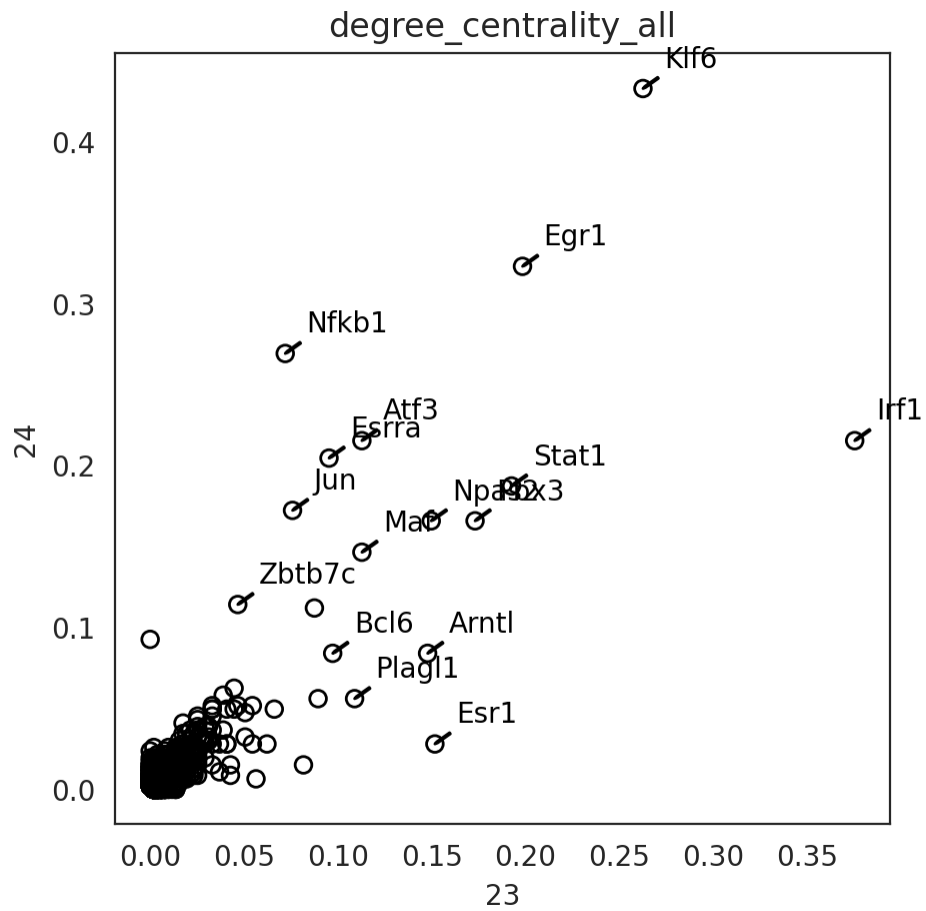
Processing clusters 22 and 25



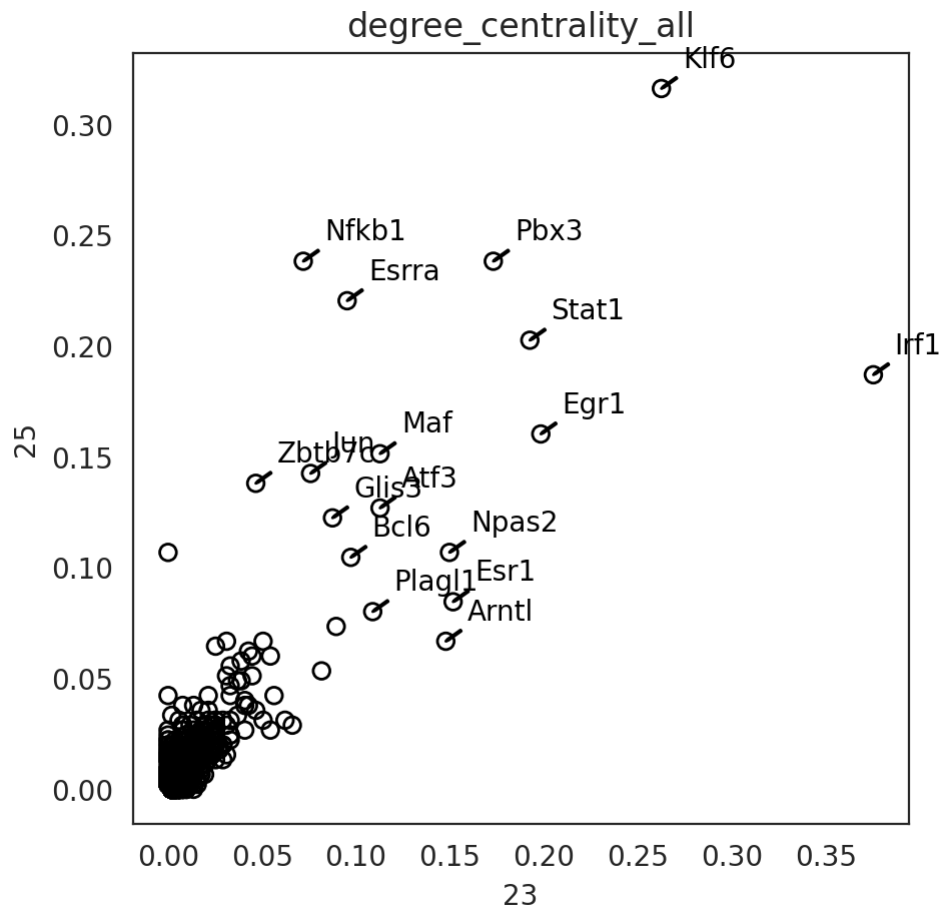
Processing clusters 22 and 26



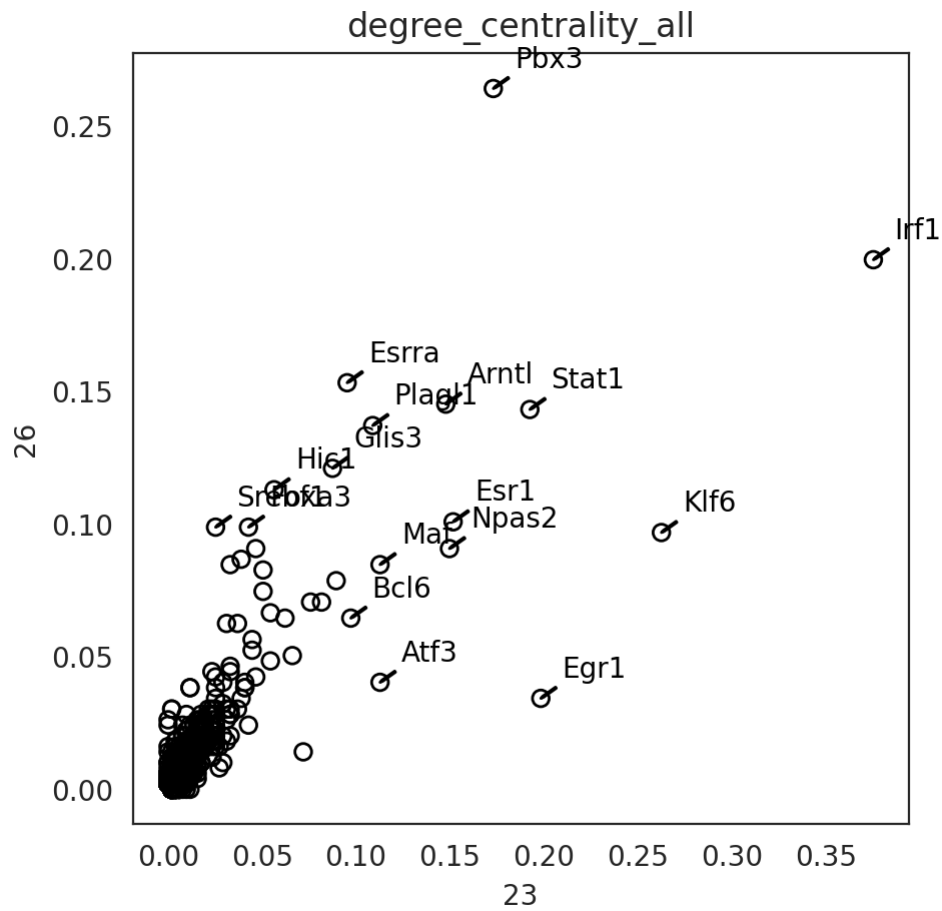
Processing clusters 23 and 24



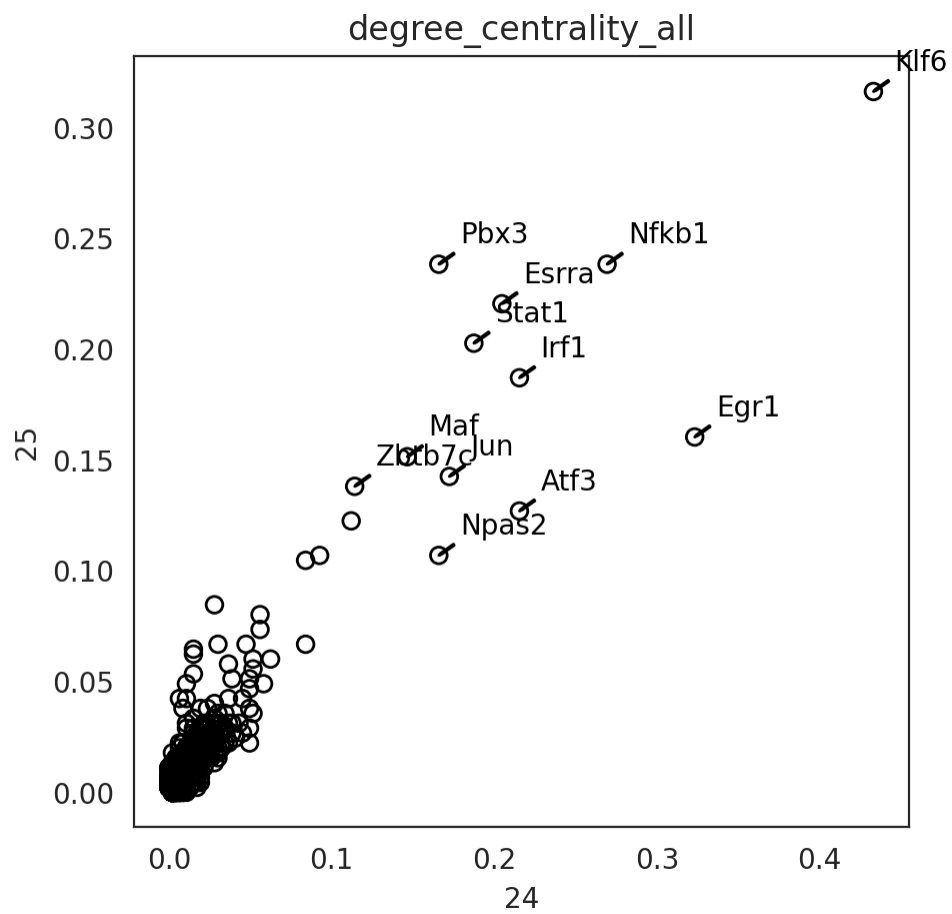
Processing clusters 23 and 25



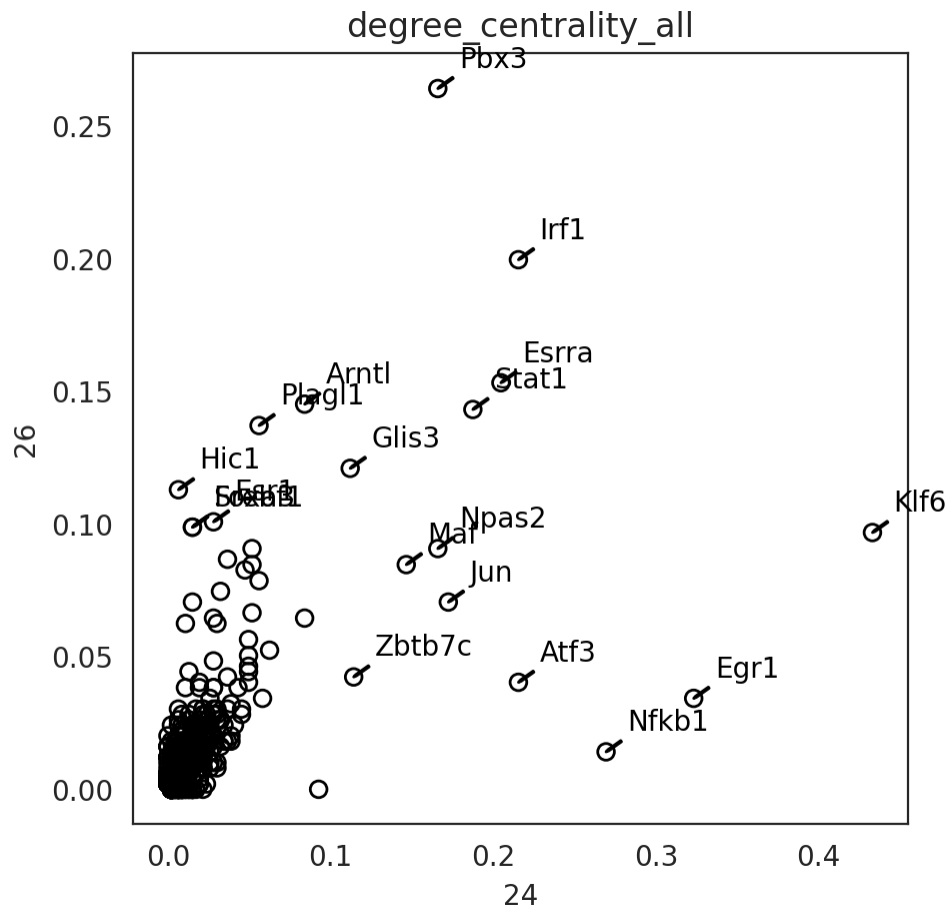
Processing clusters 23 and 26



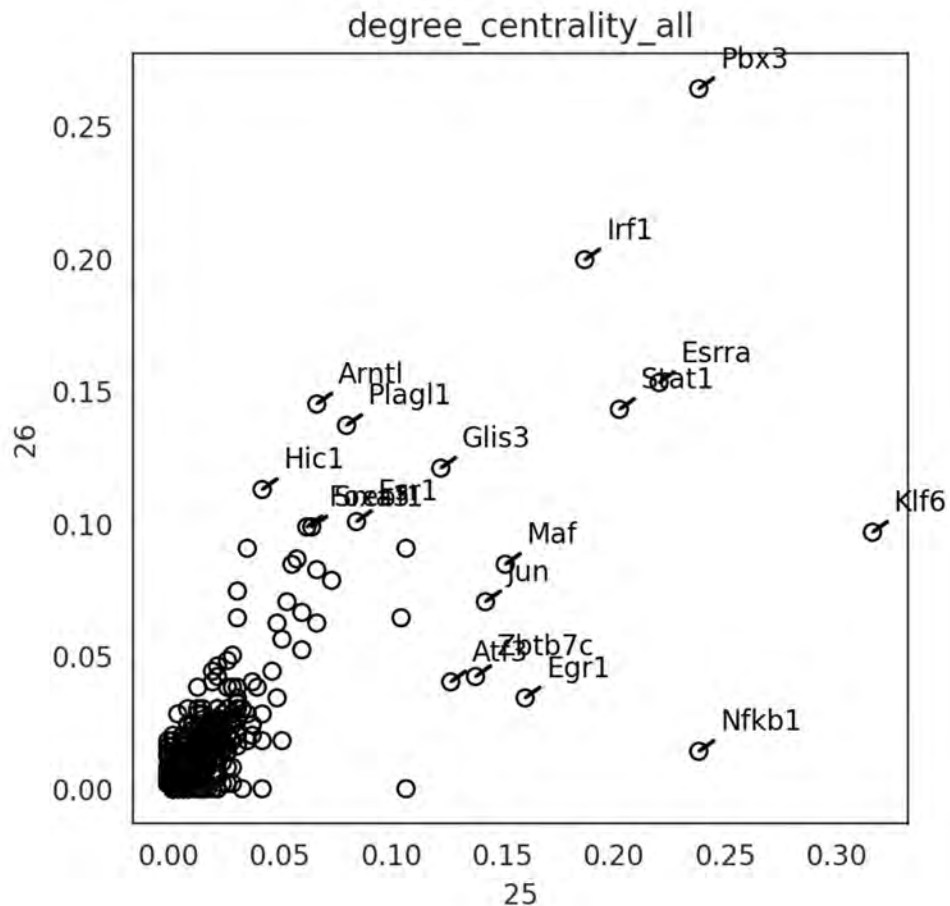
Processing clusters 24 and 25



Processing clusters 24 and 26



Processing clusters 25 and 26



```
In [ ]: # Define the specific groups of clusters
group1 = [0, 1, 2, 3]
group2 = [4, 5, 6, 7]
group3 = [8, 9, 10, 11]

# List of all groups
groups = [group1, group2, group3]

# Assume `links` is an instance of the object that has the `plot_score_comparison_2`
# and `save_folder` is the path to the folder where you want to save the plots
save_folder = 'path/to/save/folder' # Replace with your actual save folder path

# Iterate through all unique pairs of groups
for i in range(len(groups)):
    for j in range(i + 1, len(groups)):
        cluster1_str = "_".join(map(str, groups[i]))
        cluster2_str = "_".join(map(str, groups[j]))
        print(f"Processing clusters {groups[i]} and {groups[j]}")

        # Create the plot for the current pair of groups
        links.plot_score_comparison_2D(value="degree centrality_all",
                                       cluster1=cluster1_str, cluster2=cluster2_str,
                                       percentile=98,
                                       save=f"{save_folder}/PT_{cluster1_str}_vs_{c

plt.show()
```

```
In [54]: import celloracle as co
```

```

from celloracle.applications import Pseudotime_calculator
co.__version__

#plt.rcParams["font.family"] = "arial"
plt.rcParams["figure.figsize"] = [5,5]
%config InlineBackend.figure_format = 'retina'
plt.rcParams["savefig.dpi"] = 300

%matplotlib inline

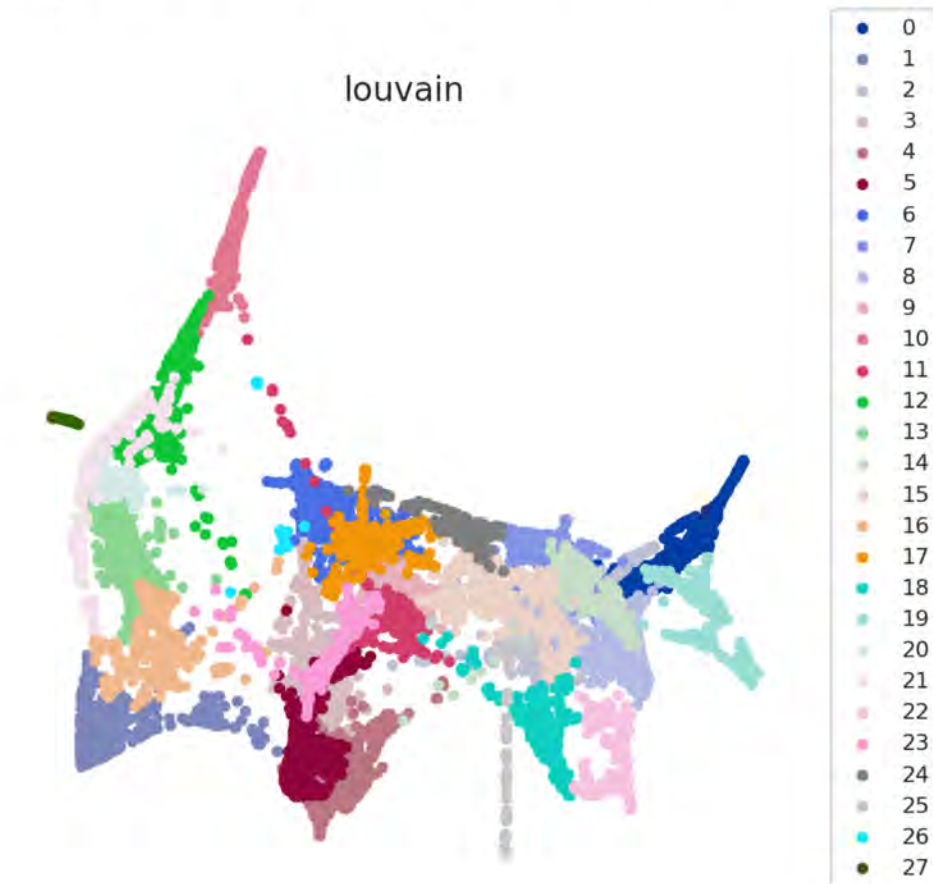
```

```
In [55]: pt = Pseudotime_calculator(oracle_object=oracle)
```

```
In [56]: print("Clustering name: ", pt.cluster_column_name)
print("Cluster list", pt.cluster_list)
pt.plot_cluster(fontsize=8)
```

Clustering name: louvain

Cluster list ['0', '1', '10', '11', '12', '13', '14', '15', '16', '17', '18', '19', '2', '20', '21', '22', '23', '24', '25', '26', '27', '3', '4', '5', '6', '7', '8', '9']



```
In [58]: # Replace 'cluster' with the actual name of the column that represents clusters in
cluster_column = 'louvain'

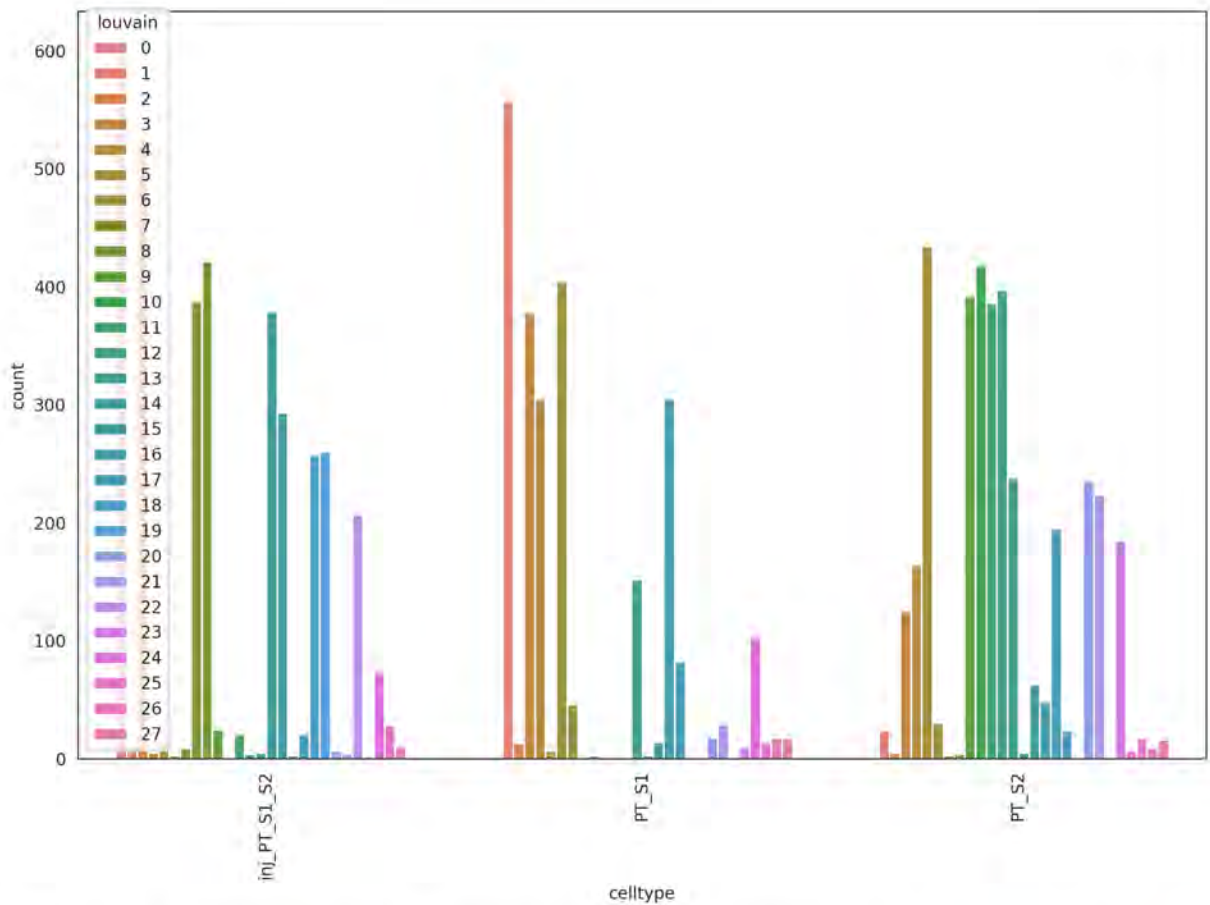
# Group by cell type and cluster
grouped = adata.obs.groupby(['celltype', cluster_column]).size().reset_index(name='

# Display the result
print(grouped)
```

	celltype	louvain	count
0	inj_PT_S1_S2	0	603
1	inj_PT_S1_S2	1	8
2	inj_PT_S1_S2	2	526
3	inj_PT_S1_S2	3	5
4	inj_PT_S1_S2	4	14
..	...	...	...
79	PT_S2	23	185
80	PT_S2	24	7
81	PT_S2	25	17
82	PT_S2	26	9
83	PT_S2	27	16

[84 rows x 3 columns]

```
In [59]: # Plot the distribution of clusters per cell type
plt.figure(figsize=(12, 8))
sns.countplot(data=adata.obs, x='celltype', hue=cluster_column)
plt.xticks(rotation=90)
plt.show()
```



```
In [60]: dominant_celltype = grouped.loc[grouped.groupby(cluster_column)['count'].idxmax()]
cluster_to_celltype = dict(zip(dominant_celltype[cluster_column], dominant_celltype
adata.obs['dominant_celltype'] = adata.obs[cluster_column].map(cluster_to_celltype)
print(adata.obs[['celltype', cluster_column, 'dominant_celltype']].head())
```

	celltype	louvain	dominant_celltype
AAACAGCCACCTATAG-2	inj_PT_S1_S2	8	inj_PT_S1_S2
AAACAGCCAGGAACAT-1	PT_S1	17	PT_S2
AAACAGCCAGTAAAGC-3	inj_PT_S1_S2	8	inj_PT_S1_S2
AAACAGCCATCGTTCT-2	inj_PT_S1_S2	14	inj_PT_S1_S2
AAACAGCCATGATTGT-4	PT_S2	20	PT_S2

```
In [61]: cluster_to_celltype
```



```
Out[61]: {'0': 'inj_PT_S1_S2',
          '1': 'PT_S1',
          '2': 'inj_PT_S1_S2',
          '3': 'PT_S1',
          '4': 'PT_S1',
          '5': 'PT_S2',
          '6': 'PT_S1',
          '7': 'inj_PT_S1_S2',
          '8': 'inj_PT_S1_S2',
          '9': 'PT_S2',
          '10': 'PT_S2',
          '11': 'PT_S2',
          '12': 'PT_S2',
          '13': 'PT_S2',
          '14': 'inj_PT_S1_S2',
          '15': 'inj_PT_S1_S2',
          '16': 'PT_S1',
          '17': 'PT_S2',
          '18': 'inj_PT_S1_S2',
          '19': 'inj_PT_S1_S2',
          '20': 'PT_S2',
          '21': 'PT_S2',
          '22': 'inj_PT_S1_S2',
          '23': 'PT_S2',
          '24': 'PT_S1',
          '25': 'inj_PT_S1_S2',
          '26': 'PT_S1',
          '27': 'PT_S1'}
```

```
In [122]: # Initialize empty lists for each lineage
clusters_in_PT1_lineage = []
clusters_in_PT2_lineage = []
clusters_in_inj_PT1_2_lineage = []
clusters_in_all_lineage = []

# Iterate over the cluster_to_celltype dictionary and populate the lists
for cluster, celltype in cluster_to_celltype.items():
    if celltype == 'PT_S1':
        clusters_in_PT1_lineage.append(cluster)
    elif celltype == 'PT_S2':
        clusters_in_PT2_lineage.append(cluster)
    elif celltype == 'inj_PT_S1_S2':
        clusters_in_inj_PT1_2_lineage.append(cluster)

# Make a dictionary
lineage_dictionary = {
    "Lineage_PT_S1": clusters_in_PT1_lineage,
    "Lineage_PT_S2": clusters_in_PT2_lineage,
    "Lineage_inj_PT_S1_S2": clusters_in_inj_PT1_2_lineage,
    "Lineage_all": ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12', '13', '14',
                    '15', '16', '17', '18', '19', '20', '21', '22', '23', '24', '25', '26', '27']
}

# Print the dictionary to verify
print(lineage_dictionary)
```

```
# Input lineage information into pseudotime object  
pt.set_lineage(lineage_dictionary=lineage_dictionary)
```

```
# Visualize Lineage information  
pt.plot_lineages()
```

```
{'Lineage_PT_S1': ['1', '3', '4', '6', '16', '24', '26', '27'], 'Lineage_PT_S2':  
['5', '9', '10', '11', '12', '13', '17', '20', '21', '23'], 'Lineage_inj_PT_S1_S2':  
['0', '2', '7', '8', '14', '15', '18', '19', '22', '25'], 'Lineage_all': ['0', '1',  
'2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12', '13', '14', '15', '16', '1  
7', '18', '19', '20', '21', '22', '23', '24', '25', '26', '27']}
```

Lineage_PT_S1



Lineage_PT_S2



Lineage_inj_PT_S1_S2



Lineage_all



```
In [77]: embedding_key3=pt.obsm_key
cluster_column_name3=pt.cluster_column_name

embedding = pt.adata.obsm[embedding_key3]
df = pd.DataFrame(embedding, columns=["x", "y"])
df["cluster"] = pt.adata.obs[cluster_column_name3].values
df["label"] = pt.adata.obs.index.values
df['cluster'] = df['cluster'].astype(int)
#fig = px.scatter(df, x="x", y="y", hover_name=df["label"], color="cluster")
#fig.show()
df
```


Out[77]:

	x	y	cluster	label
0	17281.153311	-3455.655491	8	AAACAGCCACCTATAG-2
1	-1286.794357	7008.297619	17	AAACAGCCAGGAACAT-1
2	16902.979995	-3436.457674	8	AAACAGCCAGTAAAGC-3
3	16136.026953	-732.726987	14	AAACAGCCATCGTTCT-2
4	-18644.192949	5668.710177	20	AAACAGCCATGATTGT-4
...	...	...	...	...
9701	-18834.743431	3838.319000	13	TTTGTGTTTCGCATTAA-4
9702	-17060.978550	10114.384949	21	TTTGTGTTCTAAGGAG-4
9703	-158.661333	1842.940343	9	TTTGTGTTCTCCTCAA-3
9704	-20739.176734	-9136.367931	1	TTTGTGTTCTTTGACT-4
9705	-52.462810	1774.080295	9	TTTGTGGTGCAACTA-1

9706 rows × 4 columns

```
In [92]: celltype_to_clusters = {
    'PT_S1': [1, 3, 4, 6, 16, 24, 26, 27],
    'PT_S2': [5, 9, 10, 11, 12, 13, 17, 20, 21, 23],
    'inj_PT_S1_S2': [0, 2, 7, 8, 14, 15, 18, 19, 22, 25]
}

def find_top_leftmost(df, clusters):
    subset = df[df['cluster'].isin(clusters)]
    if not subset.empty:
        top_leftmost = subset.sort_values(by=['y', 'x'], ascending=[False, True]).i
        return top_leftmost
    else:
        return None

# Function to find the bottom leftmost cell in given clusters
def find_bottom_leftmost(df, clusters):
    subset = df[df['cluster'].isin(clusters)]
    if not subset.empty:
        bottom_leftmost = subset.sort_values(by=['y', 'x']).iloc[0]
        return bottom_leftmost
    else:
        return None

# Function to find the bottom leftmost cell in a specific cluster
def find_bottom_leftmost_in_cluster(df, cluster):
    subset = df[df['cluster'] == cluster]
    if not subset.empty:
        bottom_leftmost = subset.sort_values(by=['y', 'x']).iloc[0]
        return bottom_leftmost
    else:
        return None
```

```

# Function to find the bottommost cell in given clusters
def find_bottommost(df, clusters):
    subset = df[df['cluster'].isin(clusters)]
    if not subset.empty:
        bottommost = subset.sort_values(by='y').iloc[0]
        return bottommost
    else:
        return None

# Initialize results dictionary
top_leftmost_cells = {}

# Find top-leftmost cell for each cell type
for celltype, clusters in celltype_to_clusters.items():
    top_leftmost_cells[celltype] = find_top_leftmost(df, clusters)

bottom_leftmost_5_or_7 = find_bottom_leftmost(df, [5, 7, 8])
bottommost_5_7_8 = find_bottommost(df, [5, 7, 8])

# Find the bottom leftmost cell in clusters 0, 1, 2, 3, 9 or 10
bottom_leftmost_0_1_2_3_9_10 = find_bottom_leftmost(df, [0, 1, 2, 3, 9, 10])

# Find the bottom leftmost cell in cluster 6
bottom_leftmost_cluster_6 = find_bottom_leftmost_in_cluster(df, 6)
df2 = df

def calculate_distance(df, center_x, center_y):
    df['distance'] = np.sqrt((df['x'] - center_x)**2 + (df['y'] - center_y)**2)
    return df

# Function to calculate projection along 315 degrees direction
def calculate_projection(df, center_x, center_y):
    # Direction vector for 315 degrees
    direction_x, direction_y = -1, -1
    # Normalize the direction vector
    norm = np.sqrt(direction_x**2 + direction_y**2)
    direction_x /= norm
    direction_y /= norm
    # Compute projection
    df['projection'] = (df['x'] - center_x) * direction_x + (df['y'] - center_y) * direction_y
    return df

# Function to find the cell facing 315 degrees
def find_furthest_315(df, clusters):
    subset = df[df['cluster'].isin(clusters)]
    if not subset.empty:
        center_x = subset['x'].mean()
        center_y = subset['y'].mean()
        subset = calculate_projection(subset, center_x, center_y)
        # We want to find the cell with the maximum projection
        furthest = subset.sort_values(by='projection', ascending=False).iloc[0]
        return furthest
    else:
        return None

```

```

top_leftmost_cells["inj_PT_S1_S2"] = find_furthest_315(df2, [0, 2, 7, 8, 14, 15, 18]
# Print the results
for celltype, cell in top_leftmost_cells.items():
    print(f"Top-leftmost cell for cell type {celltype}:", cell)

```

```

df[(df['x'] <= 6000) & (df['x'] >= 0) & (df['y'] <= 9000) & (df['y'] >= 0) & (df['clust

```

```

Top-leftmost cell for cell type PT_S1: x -8708.962998

```

```

y 11843.14357

```

```

cluster 26

```

```

label TTTCCACCAAGCTACC-1

```

```

Name: 9597, dtype: object

```

```

Top-leftmost cell for cell type PT_S2: x -8445.785567

```

```

y 24534.147196

```

```

cluster 10

```

```

label GACCTGATCATGTCAA-4

```

```

Name: 4864, dtype: object

```

```

Top-leftmost cell for cell type inj_PT_S1_S2: x 8278.717119

```

```

y -14163.892781

```

```

cluster 25

```

```

label TTCCTTGAGGCTAATC-1

```

```

projection 13293.285553

```

```

Name: 9280, dtype: object

```

```

In [117]: df[(df['x'] <= 0) & (df['x'] >= -7000) & (df['y'] <= 15000) & (df['y'] >= 0) & (df['clu

```

```

Out[117]:
```

	x	y	cluster	label
--	---	---	---------	-------

36	-5896.424787	6501.366730	6	AAACGCGCAAATAAG-3
----	--------------	-------------	---	-------------------

68	-3745.183786	5129.687339	6	AAAGCCGCACGGTTTA-3
----	--------------	-------------	---	--------------------

91	-4424.759603	5482.260922	6	AAAGGTTAGCGATAGA-3
----	--------------	-------------	---	--------------------

107	-5961.916374	702.845963	3	AAATGCCTCGTTTCTG-1
-----	--------------	------------	---	--------------------

126	-3351.989311	4469.022313	6	AACAGCAAGGTCCACA-3
-----	--------------	-------------	---	--------------------

...	...	...	...	...
-----	-----	-----	-----	-----

9531	-1143.076430	3510.288845	6	TTGTTTGGTCGACTCC-2
------	--------------	-------------	---	--------------------

9553	-6044.382184	6072.888509	6	TTTACGAAGATTGAGG-3
------	--------------	-------------	---	--------------------

9555	-4440.708993	1027.285625	6	TTTACGAAGCGGCTGT-3
------	--------------	-------------	---	--------------------

9625	-3122.588726	4182.432301	6	TTTGACCGTTAACGGC-2
------	--------------	-------------	---	--------------------

9650	-2338.545593	5452.794657	6	TTTGCGACACAATGTT-3
------	--------------	-------------	---	--------------------

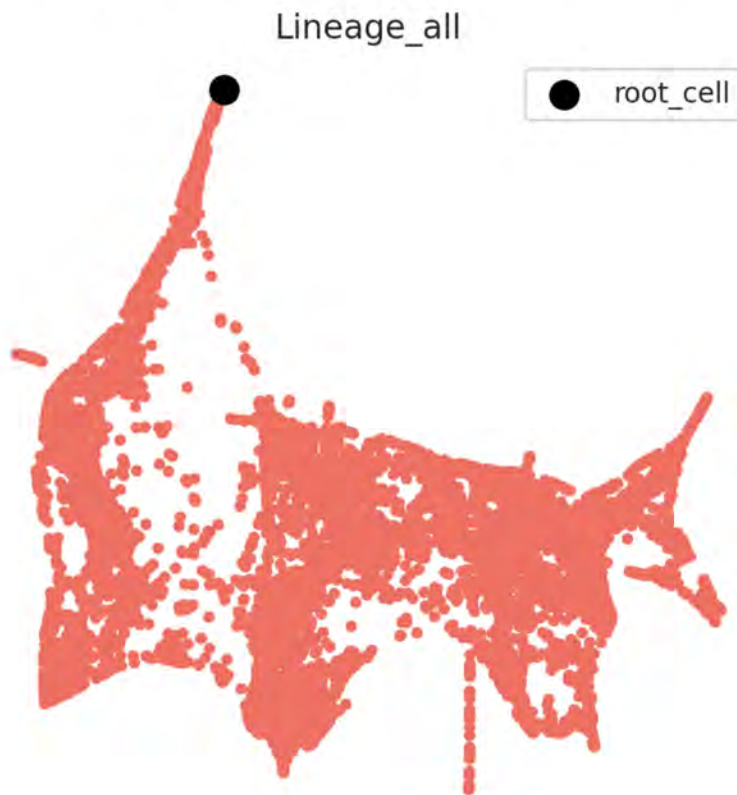
476 rows x 4 columns

```

In [123]: # Estimated root cell name for each Lineage
#root_cells = {"Lineage_PT_S1": "AAACGCGCAAATAAG-3", "Lineage_PT_S2": "GACCTGATCAT

```

```
root_cells = {"Lineage_all": "GACCTGATCATGTCAA-4"}  
pt.set_root_cells(root_cells=root_cells)  
pt.plot_root_cells()
```



```
In [112... # Check diffusion map data.  
"X_diffmap" in pt.adata.obsm
```

```
Out[112... True
```

```
In [124... # Calculate pseudotime  
pt.get_pseudotime_per_each_lineage()  
  
# Check results  
pt.plot_pseudotime(cmap="rainbow")
```


Pseudotime_Lineage_all



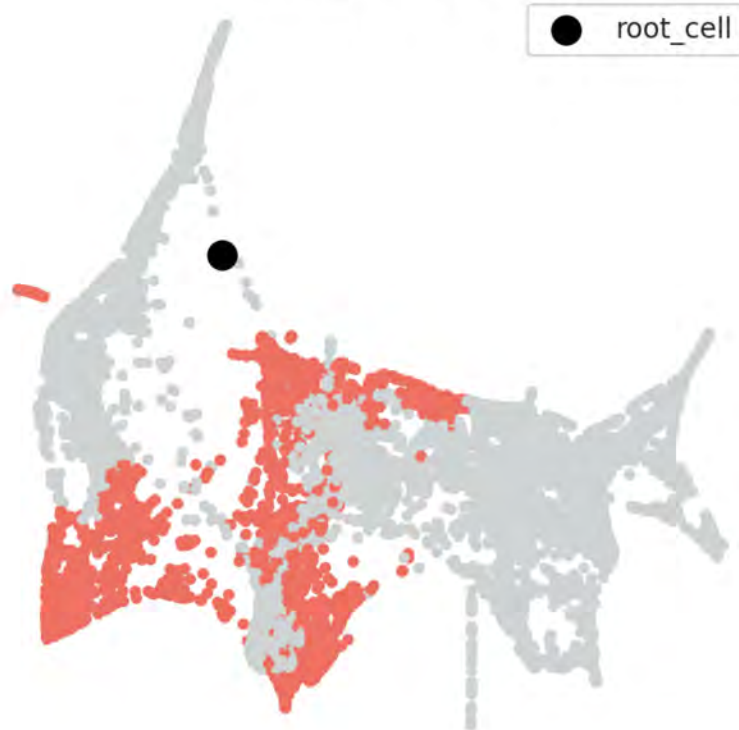
Pseudotime



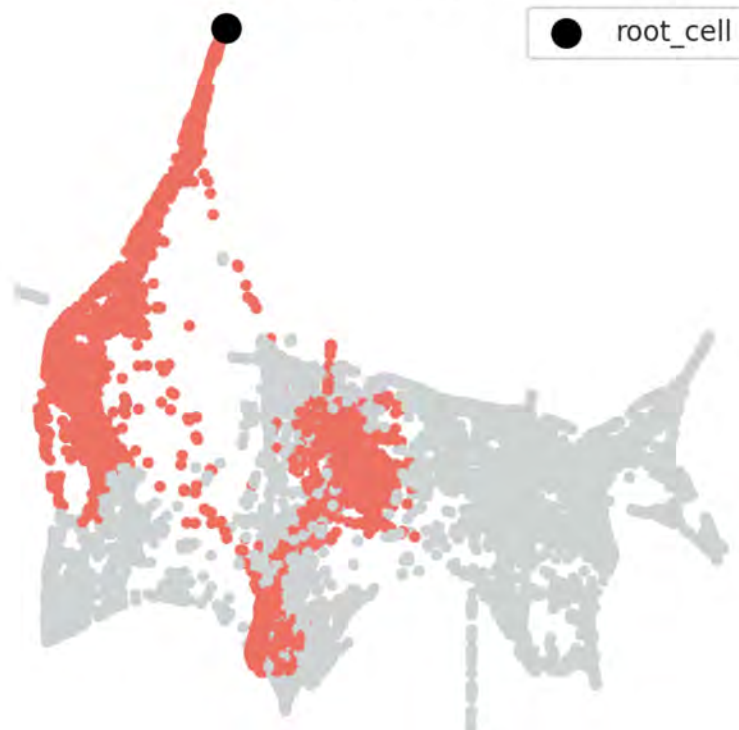
```
In [126... # Check result
pt.adata.obs[["Pseudotime"]].head()
pt.adata.obs[["PseudotimeAll"]] = pt.adata.obs[["Pseudotime"]]
```

```
In [127... # Estimated root cell name for each lineage
root_cells = {"Lineage_PT_S1": "TTTCCACCAAGCTACC-1", "Lineage_PT_S2": "GACCTGATCATG
#root_cells = {"Lineage_all": "GACCTGATCATGTCAA-4"}
pt.set_root_cells(root_cells=root_cells)
pt.plot_root_cells()
```

Lineage_PT_S1



Lineage_PT_S2



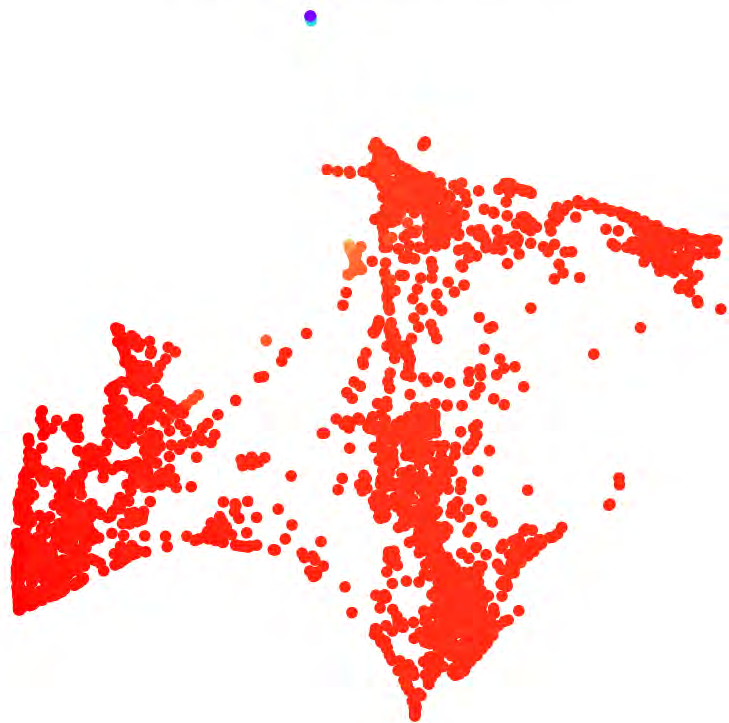
Lineage_inj_PT_S1_S2



```
In [128... # Calculate pseudotime
pt.get_pseudotime_per_each_lineage()

# Check results
pt.plot_pseudotime(cmap="rainbow")
```

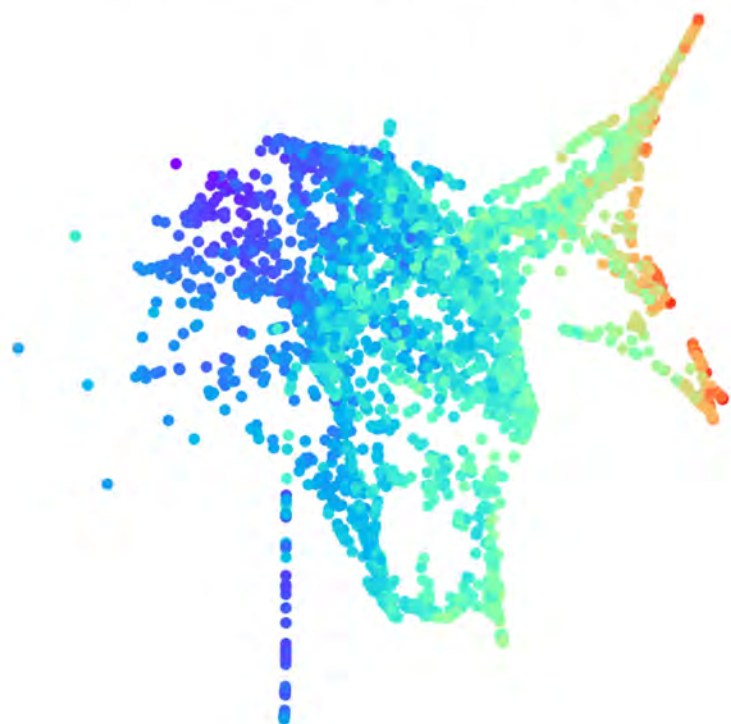
Pseudotime_Lineage_PT_S1



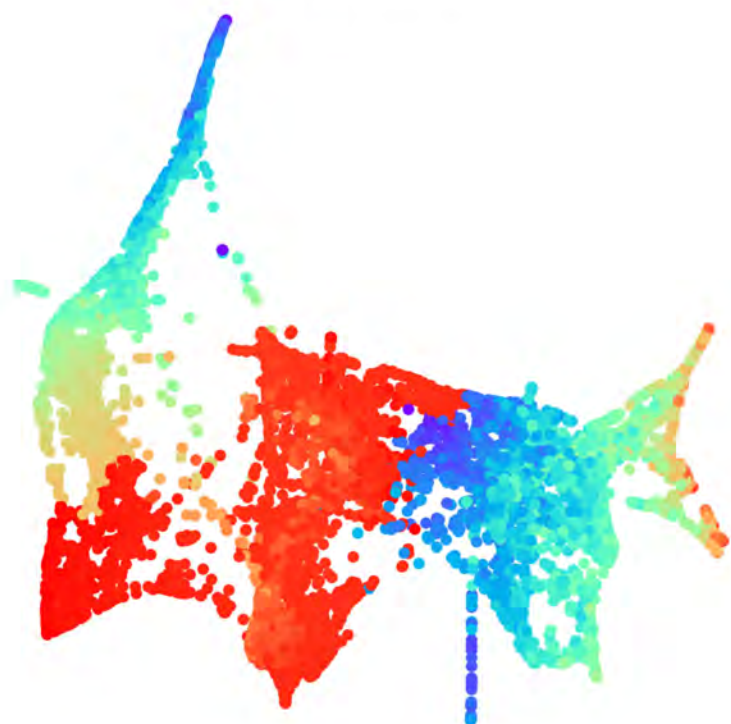
Pseudotime_Lineage_PT_S2



Pseudotime_Lineage_inj_PT_S1_S2



Pseudotime



```
In [129...] oracle.adata.obs = pt.adata.obs  
oracle.to_hdf5("/xxx/analysis/PT_subset/Sobj_PT1_2_inj_step4.celloracle.oracle")
```

```
In [146...] lineage_dictionary["Lineage_PT_S1"]
```

Out[146... ['1', '3', '4', '6', '16', '24', '26', '27']

In [154...

In []: